

■ TRIMMED DOCUMENT — READ BEFORE USE

This is a trimmed version of UM1725 Rev 8 — STM32F4 HAL and LL drivers reference manual.
Only chapters relevant to STM32F411xx are included.

About the Table of Contents (pages 2–16)

The table of contents included in this document is from the original UM1725 and lists ALL chapters of the full manual — including chapters that were removed from this trimmed version. Chapter numbers that are greyed out or show '????' in your PDF viewer's bookmark panel belong to peripherals not present on the STM32F411xx.

Chapters included in this document (HAL drivers only)

Ch. 1–5	General info, HAL/LL overview	Ch. 41	IWDG
Ch. 7–8	ADC, ADCEX	Ch. 48	PCD (USB Device)
Ch. 11	CORTEX (NVIC, SysTick)	Ch. 50	PWR
Ch. 12	CRC	Ch. 52–53	RCC, RCCEX
Ch. 21–22	DMA, DMAEX	Ch. 54	RNG
Ch. 24	EXTI	Ch. 55–56	RTC, RTCEX
Ch. 29–30	FLASH, FLASHEX	Ch. 63	SMARTCARD
Ch. 31	GPIO	Ch. 65	SPI
Ch. 33	HCD (USB Host)	Ch. 68–69	TIM, TIMEX
Ch. 36–37	I2C, I2CEX	Ch. 70	UART
Ch. 38–39	I2S, I2SEX	Ch. 71	USART
Ch. 40	IRDA	Ch. 73	WWDG

Source: UM1725 Rev 8 — STMicroelectronics, March 2023
F411 peripheral set based on Table 5 of UM1725, using STM32F401xE column as reference.
Generated by extract_f411_hal.py

Description of STM32F4 HAL and low-layer drivers

Introduction

STM32Cube is an STMicroelectronics original initiative to significantly improve developer productivity by reducing development effort, time and cost. STM32Cube covers the STM32 portfolio.

STM32Cube includes:

- **STM32CubeMX**, a graphical software configuration tool that allows the generation of C initialization code using graphical wizards.
- A comprehensive embedded software platform, delivered per Series (such as **STM32CubeF4** for STM32F4 Series)
 - The STM32Cube HAL, STM32 abstraction layer embedded software ensuring maximized portability across the STM32 portfolio. HAL APIs are available for all peripherals.
 - Low-layer APIs (LL) offering a fast light-weight expert-oriented layer which is closer to the hardware than the HAL. LL APIs are available only for a set of peripherals.
 - A consistent set of middleware components such as RTOS, USB, TCP/IP and Graphics.
 - All embedded software utilities, delivered with a full set of examples.

The HAL driver layer provides a simple, generic multi-instance set of APIs (application programming interfaces) to interact with the upper layer (application, libraries and stacks). The HAL driver APIs are split into two categories: generic APIs, which provide common and generic functions for all the STM32 series and extension APIs, which include specific and customized functions for a given line or part number. The HAL drivers include a complete set of ready-to-use APIs that simplify the user application implementation. For example, the communication peripherals contain APIs to initialize and configure the peripheral, manage data transfers in polling mode, handle interrupts or DMA, and manage communication errors. The HAL drivers are feature-oriented instead of IP-oriented. For example, the timer APIs are split into several categories following the IP functions, such as basic timer, capture and pulse width modulation (PWM). The HAL driver layer implements run-time failure detection by checking the input values of all functions. Such dynamic checking enhances the firmware robustness. Run-time detection is also suitable for user application development and debugging.

The LL drivers offer hardware services based on the available features of the STM32 peripherals. These services reflect exactly the hardware capabilities, and provide atomic operations that must be called by following the programming model described in the product line reference manual. As a result, the LL services are not based on standalone processes and do not require any additional memory resources to save their states, counter or data pointers. All operations are performed by changing the content of the associated peripheral registers. Unlike the HAL, LL APIs are not provided for peripherals for which optimized access is not a key feature, or for those requiring heavy software configuration and/or a complex upper-level stack (such as USB).

The HAL and LL are complementary and cover a wide range of application requirements:

- The HAL offers high-level and feature-oriented APIs with a high-portability level. These hide the MCU and peripheral complexity from the end-user.
- The LL offers low-level APIs at register level, with better optimization but less portability. These require deep knowledge of the MCU and peripheral specifications.

The HAL- and LL-driver source code is developed in Strict ANSI-C, which makes it independent of the development tools. It is checked with the CodeSonar® static analysis tool. It is fully documented.

They are compliant with MISRA C®:2004 standard.

This user manual is structured as follows:

- Overview of HAL drivers
- Overview of low-layer drivers
- Cohabiting of HAL and LL drivers
- Detailed description of each peripheral driver: configuration structures, functions, and how to use the given API to build your application



1 General information

The [STM32CubeF4](#) MCU Package runs on STM32F4 32-bit microcontrollers based on the Arm® Cortex®-M processor.

Note: Arm is a registered trademark of Arm Limited (or its subsidiaries) in the US and/or elsewhere.



2 Acronyms and definitions

Table 1. Acronyms and definitions

Acronym	Definition
ADC	Analog-to-digital converter
AES	Advanced encryption standard
ANSI	American national standards institute
API	Application programming interface
BSP	Board support package
CAN	Controller area network
CEC	Consumer electronic controller
CMSIS	Cortex microcontroller software interface standard
COMP	Comparator
CORDIC	Trigonometric calculation unit
CPU	Central processing unit
CRC	CRC calculation unit
CRYP	Cryptographic processor
DAC	Digital to analog converter
DCMI	Digital camera interface
DFSDM	Digital filter sigma delta modulator
DMA	Direct memory access
DMAMUX	Direct memory access request multiplexer
DSI	Display serial interface
DTS	Digital temperature sensor
ESE	Security enable Flash user option bit
ETH	Ethernet controller
EXTI	External interrupt/event controller
FDCAN	Flexible data-rate controller area network unit
FLASH	Flash memory
FMAC	Filtering mathematical calculation unit
FMC	Flexible memory controller
FW	Firewall
GFXMMU	Chrom-GRC
GPIO	General purpose I/Os
HAL	Hardware abstraction layer
HASH	Hash processor
HCD	USB host controller driver
HRTIM	High-resolution timer
I2C	Inter-integrated circuit
I2S	Inter-integrated sound
IRDA	Infrared data association
IWDG	Independent watchdog

Acronym	Definition
JPEG	Joint photographic experts group
LCD	Liquid crystal display controller
LTDC	LCD TFT display controller
LPTIM	Low-power timer
LPUART	Low-power universal asynchronous receiver/transmitter
MCO	Microcontroller clock output
MDIOS	Management data input/output (MDIO) slave
MDMA	Master direct memory access
MMC	MultiMediaCard
MPU	Memory protection unit
MSP	MCU specific package
NAND	NAND Flash memory
NOR	NOR Flash memory
NVIC	Nested vectored interrupt controller
OCTOSPI	Octo-SPI interface
OPAMP	Operational amplifier
OTFDEC	On-the-fly decryption engine
OTG-FS	USB on-the-go full-speed
PKA	Public key accelerator
PCD	USB peripheral controller driver
PPP	STM32 peripheral or block
PSSI	Parallel synchronous slave interface
PWR	Power controller
QSPI (QUADSPI)	Quad-SPI Flash memory
RAMECC	RAM ECC monitoring
RCC	Reset and clock controller
RNG	Random number generator
RTC	Real-time clock
SAI	Serial audio interface
SD	Secure digital
SDMMC	SD/SDIO/MultiMediaCard card host interface
SMARTCARD	Smartcard IC
SMBUS	System management bus
SPI	Serial peripheral interface
SPDIFRX	SPDIF-RX Receiver interface
SRAM	SRAM external memory
SWPMI	Serial wire protocol master interface
SysTick	System tick timer
TIM	Advanced-control, general-purpose or basic timer
TSC	Touch sensing controller
UART	Universal asynchronous receiver/transmitter

Acronym	Definition
UCPD	USB Type-C® and Power Delivery interface
USART	Universal synchronous receiver/transmitter
WWDG	Window watchdog
USB	Universal serial bus

3 Overview of HAL drivers

The HAL drivers are designed to offer a rich set of APIs and to interact easily with the application upper layers. Each driver consists of a set of functions covering the most common peripheral features. The development of each driver is driven by a common API which standardizes the driver structure, the functions and the parameter names.

The HAL drivers include a set of driver modules, each module being linked to a standalone peripheral. However, in some cases, the module is linked to a peripheral functional mode. As an example, several modules exist for the USART peripheral: UART driver module, USART driver module, SMARTCARD driver module and IRDA driver module.

The HAL main features are the following:

- Cross-family portable set of APIs covering the common peripheral features as well as extension APIs in case of specific peripheral features.
- Three API programming models: polling, interrupt and DMA.
- APIs are RTOS compliant:
 - Fully reentrant APIs
 - Systematic usage of timeouts in polling mode
- Support of peripheral multi-instance allowing concurrent API calls for multiple instances of a given peripheral (such as USART1 or USART2)
- All HAL APIs implement user-callback functions mechanism:
 - Peripheral Init/Delinit HAL APIs can call user-callback functions to perform peripheral system level Initialization/De-Initialization (clock, GPIOs, interrupt, DMA)
 - Peripherals interrupt events
 - Error events
- Object locking mechanism: safe hardware access to prevent multiple spurious accesses to shared resources.
- Timeout used for all blocking processes: the timeout can be a simple counter or a timebase.

3.1 HAL and user-application files

3.1.1 HAL driver files

HAL drivers are composed of the following set of files:

Table 2. HAL driver files

File	Description
<i>stm32f4xx_hal_ppp.c</i>	Main peripheral/module driver file It includes the APIs that are common to all STM32 devices. <i>Example: stm32f4xx_hal_adc.c, stm32f4xx_hal_irda.c.</i>
<i>stm32f4xx_hal_ppp.h</i>	Header file of the main driver C file It includes common data, handle and enumeration structures, define statements and macros, as well as the exported generic APIs. <i>Example: stm32f4xx_hal_adc.h, stm32f4xx_hal_irda.h.</i>
<i>stm32f4xx_hal_ppp_ex.c</i>	Extension file of a peripheral/module driver. It includes the specific APIs for a given part number or family, as well as the newly defined APIs that overwrite the default generic APIs if the internal process is implemented in different way. <i>Example: stm32f4xx_hal_adc_ex.c, stm32f4xx_hal_flash_ex.c.</i>
<i>stm32f4xx_hal_ppp_ex.h</i>	Header file of the extension C file It includes the specific data and enumeration structures, define statements and macros, as well as the exported device part number specific APIs <i>Example: stm32f4xx_hal_adc_ex.h, stm32f4xx_hal_flash_ex.h.</i>
<i>stm32f4xx_hal.c</i>	This file is used for HAL initialization and contains DBGMCU, Remap and Time Delay based on SysTick APIs.
<i>stm32f4xx_hal.h</i>	stm32f4xx_hal.c header file
<i>stm32f4xx_hal_msp_template.c</i>	Template file to be copied to the user application folder It contains the MSP initialization and de-initialization (main routine and callbacks) of the peripheral used in the user application.
<i>stm32f4xx_hal_conf_template.h</i>	Template file allowing to customize the drivers for a given application
<i>stm32f4xx_hal_def.h</i>	Common HAL resources such as common define statements, enumerations, structures and macros

3.1.2 User-application files

The minimum files required to build an application using the HAL are listed in the table below:

Table 3. User-application files

File	Description
<i>system_stm32f4xx.c</i>	This file contains SystemInit() that is called at startup just after reset and before branching to the main program. It does not configure the system clock at startup (contrary to the standard library). This is to be done using the HAL APIs in the user files. It allows relocating the vector table in internal SRAM and configuring the FSMC/FMC (when available) to use the external SRAM or SDRAM mounted on the evaluation board as data memory.
<i>startup_stm32f4xx.s</i>	Toolchain specific file that contains reset handler and exception vectors. For some toolchains, it allows adapting the stack/heap size to fit the application requirements.
<i>stm32f4xx_flash.icf</i> (optional)	Linker file for EWARM toolchain allowing mainly adapting the stack/heap size to fit the application requirements.
<i>stm32f4xx_hal_msp.c</i>	This file contains the MSP initialization and de-initialization (main routine and callbacks) of the peripheral used in the user application.
<i>stm32f4xx_hal_conf.h</i>	This file allows the user to customize the HAL drivers for a specific application.

File	Description
	It is not mandatory to modify this configuration. The application can use the default configuration without any modification.
<i>stm32f4xx_it.c/.h</i>	This file contains the exceptions handler and peripherals interrupt service routine, and calls HAL_IncTick() at regular time intervals to increment a local variable (declared in <i>stm32f4xx_hal.c</i>) used as HAL timebase. By default, this function is called each 1ms in SysTick ISR. The PPP_IRQHandler() routine must call HAL_PPP_IRQHandler() if an interrupt based process is used within the application.
<i>main.c/.h</i>	This file contains the main program routine, mainly: • Call to HAL_Init() • assert_failed() implementation • system clock configuration • peripheral HAL initialization and user application code.

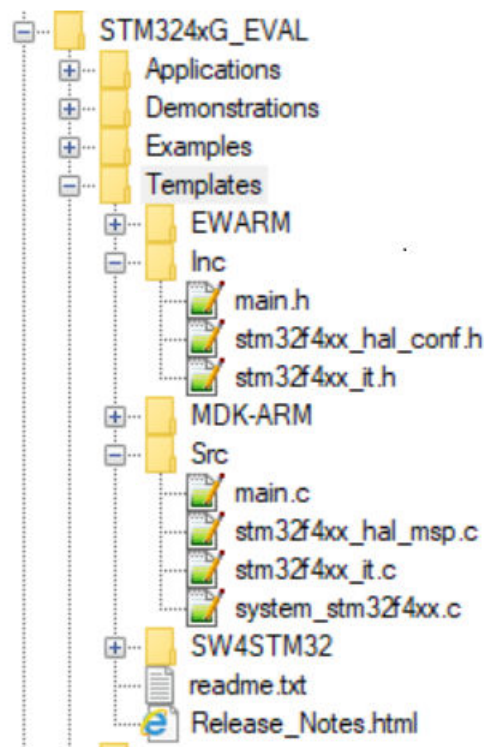
The STM32Cube package comes with ready-to-use project templates, one for each supported board. Each project contains the files listed above and a preconfigured project for the supported toolchains.

Each project template provides empty main loop function and can be used as a starting point to get familiar with project settings for STM32Cube. Its features are the following:

- It contains the sources of HAL, CMSIS and BSP drivers which are the minimal components to develop a code on a given board.
- It contains the include paths for all the firmware components.
- It defines the STM32 device supported, and allows configuring the CMSIS and HAL drivers accordingly.
- It provides ready to use user files preconfigured as defined below:
 - HAL is initialized
 - SysTick ISR implemented for HAL_Delay()
 - System clock configured with the maximum device frequency.

Note: *If an existing project is copied to another location, then include paths must be updated.*

Figure 1. Example of project template



3.2 HAL data structures

Each HAL driver can contain the following data structures:

- Peripheral handle structures
- Initialization and configuration structures
- Specific process structures.

3.2.1 Peripheral handle structures

The APIs have a modular generic multi-instance architecture that enables working with several IP instances simultaneously.

PPP_HandleTypeDef *handle is the main structure that is implemented in the HAL drivers. It handles the peripheral/module configuration and registers and embeds all the structures and variables needed to follow the peripheral device flow.

The peripheral handle is used for the following purposes:

- Multi-instance support: each peripheral/module instance has its own handle. As a result instance resources are independent.
- Peripheral process intercommunication: the handle is used to manage shared data resources between the process routines.
Example: global pointers, DMA handles, state machine.
- Storage : this handle is used also to manage global variables within a given HAL driver.

An example of peripheral structure is shown below:

```
typedef struct
{
    USART_TypeDef *Instance; /* USART registers base address */
    USART_InitTypeDef Init; /* Usart communication parameters */
    uint8_t *pTxBuffPtr; /* Pointer to Usart Tx transfer Buffer */
    uint16_t TxXferSize; /* Usart Tx Transfer size */
    __IO uint16_t TxXferCount; /* Usart Tx Transfer Counter */
    uint8_t *pRxBuffPtr; /* Pointer to Usart Rx transfer Buffer */
    uint16_t RxXferSize; /* Usart Rx Transfer size */
    __IO uint16_t RxXferCount; /* Usart Rx Transfer Counter */
    DMA_HandleTypeDef *hdmatx; /* Usart Tx DMA Handle parameters */
    DMA_HandleTypeDef *hdmarx; /* Usart Rx DMA Handle parameters */
    HAL_LockTypeDef Lock; /* Locking object */
    __IO HAL_USART_StateTypeDef State; /* Usart communication state */
    __IO HAL_USART_ErrorTypeDef ErrorCode; /* USART Error code */
}USART_HandleTypeDef;
```

Note:

1. *The multi-instance feature implies that all the APIs used in the application are reentrant and avoid using global variables because subroutines can fail to be reentrant if they rely on a global variable to remain unchanged but that variable is modified when the subroutine is recursively invoked. For this reason, the following rules are respected:*
 - *Reentrant code does not hold any static (or global) non-constant data: reentrant functions can work with global data. For example, a reentrant interrupt service routine can grab a piece of hardware status to work with (for example serial port read buffer) which is not only global, but volatile. Still, typical use of static variables and global data is not advised, in the sense that only atomic read-modify-write instructions should be used in these variables. It should not be possible for an interrupt or signal to occur during the execution of such an instruction.*
 - *Reentrant code does not modify its own code.*
2. *When a peripheral can manage several processes simultaneously using the DMA (full duplex case), the DMA interface handle for each process is added in the PPP_HandleTypeDef.*
3. *For the shared and system peripherals, no handle or instance object is used. The peripherals concerned by this exception are the following:*
 - **GPIO**
 - **SYSTICK**
 - **NVIC**
 - **PWR**
 - **RCC**
 - **FLASH**

3.2.2 Initialization and configuration structure

These structures are defined in the generic driver header file when it is common to all part numbers. When they can change from one part number to another, the structures are defined in the extension header file for each part number.

```
typedef struct
{
    uint32_t BaudRate; /*!< This member configures the UART communication baudrate.*/
    uint32_t WordLength; /*!< Specifies the number of data bits transmitted or received in a frame.*/
    uint32_t StopBits; /*!< Specifies the number of stop bits transmitted.*/
    uint32_t Parity; /*!< Specifies the parity mode. */
    uint32_t Mode; /*!< Specifies whether the Receive or Transmit mode is enabled or disabled.*/
    uint32_t HwFlowCtl; /*!< Specifies whether the hardware flow control mode is enabled or disabled.*/
    uint32_t OverSampling; /*!< Specifies whether the Over sampling 8 is enabled or disabled, to achieve higher speed (up to fPCLK/8).*/
}UART_InitTypeDef;
```

Note:

The config structure is used to initialize the sub-modules or sub-instances. See below example:

```
HAL_ADC_ConfigChannel (ADC_HandleTypeDef* hadc, ADC_ChannelConfTypeDef* sConfig)
```

3.2.3

Specific process structures

The specific process structures are used for specific process (common APIs). They are defined in the generic driver header file.

Example:

```
HAL_PPP_Process (PPP_HandleTypeDef* hadc, PPP_ProcessConfig* sConfig)
```

3.3 API classification

The HAL APIs are classified into the following categories:

- **Generic APIs:** common generic APIs applying to all STM32 devices. These APIs are consequently present in the generic HAL driver files of all STM32 microcontrollers.

```
HAL_StatusTypeDef HAL_ADC_Init(ADC_HandleTypeDef* hadc);
HAL_StatusTypeDef HAL_ADC_DeInit(ADC_HandleTypeDef *hadc);
HAL_StatusTypeDef HAL_ADC_Start(ADC_HandleTypeDef* hadc);
HAL_StatusTypeDef HAL_ADC_Stop(ADC_HandleTypeDef* hadc);
HAL_StatusTypeDef HAL_ADC_Start_IT(ADC_HandleTypeDef* hadc);
HAL_StatusTypeDef HAL_ADC_Stop_IT(ADC_HandleTypeDef* hadc);
void HAL_ADC_IRQHandler(ADC_HandleTypeDef* hadc);
```

- **Extension APIs:**

This set of API is divided into two sub-categories :

- **Family specific APIs:** APIs applying to a given family. They are located in the extension HAL driver file (see example below related to the ADC).

```
HAL_StatusTypeDef HAL_ADCEX_InjectedStop(ADC_HandleTypeDef* hadc);
HAL_StatusTypeDef HAL_ADCEX_InjectedStop_IT(ADC_HandleTypeDef* hadc); HAL_StatusTy
peDef HAL_ADCEX_InjectedStart(ADC_HandleTypeDef* hadc);
HAL_StatusTypeDef HAL_ADCEX_InjectedStart_IT(ADC_HandleTypeDef* hadc);
```

- **Device part number specific APIs:** These APIs are implemented in the extension file and delimited by specific define statements relative to a given part number.

```
#if defined(STM32F427xx) || defined(STM32F437xx) || defined(STM32F429xx)
|| defined(STM32F439xx) HAL_StatusTypeDef HAL_FLASHEx_OB_SelectPCROP(void); HAL_St
atusTypeDef HAL_FLASHEx_OB_DeSelectPCROP(void);
#endif /* STM32F427xx || STM32F437xx || STM32F429xx || STM32F439xx || */
```

Note: The data structure related to the specific APIs is delimited by the device part number define statement. It is located in the corresponding extension header C file.

The following table summarizes the location of the different categories of HAL APIs in the driver files.

Table 4. API classification

	Generic file	Extension file
Common APIs	X	X ⁽¹⁾
Family specific APIs	-	X
Device specific APIs	-	X

1. In some cases, the implementation for a specific device part number may change. In this case the generic API is declared as weak function in the extension file. The API is implemented again to overwrite the default function.

Note: Family specific APIs are only related to a given family. This means that if a specific API is implemented in another family, and the arguments of this latter family are different, additional structures and arguments might need to be added.

The IRQ handlers are used for common and family specific processes.

3.4 Devices supported by HAL/LL drivers

Table 5. List of devices supported by HAL/LL drivers

IP/Module	STM32F405xx	STM32F415xx	STM32F407xx	STM32F417xx	STM32F427xx	STM32F437xx	STM32F429xx	STM32F439xx	STM32F401xC	STM32F401xE	STM32F446xx	STM32F469xx	STM32F479xx	STM32F410xx	STM32F412xx	STM32F413xx	STM32F423xx
stm32f4xx_hal.c	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes
stm32f4xx_hal_adc.c	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes
stm32f4xx_hal_adc_ex.c	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes
stm32f4xx_hal_can.c	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	No	No	Yes	Yes	Yes	Yes	Yes	Yes	Yes
stm32f4xx_hal_cec.c	No	No	No	No	No	No	No	No	No	No	Yes	No	No	No	No	No	No
stm32f4xx_hal_cortex.c	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes
stm32f4xx_hal_crc.c	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes
stm32f4xx_hal_cryp.c	No	Yes	No	Yes	No	Yes	No	Yes	No	No	No	No	Yes	No	No	No	Yes
stm32f4xx_hal_cryp_ex.c	No	Yes	No	Yes	No	Yes	No	Yes	No	No	No	No	Yes	No	No	No	Yes
stm32f4xx_hal_dac.c	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	No	No	Yes	Yes	Yes	Yes	No	Yes	Yes
stm32f4xx_hal_dac_ex.c	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	No	No	Yes	Yes	Yes	Yes	No	Yes	Yes
stm32f4xx_hal_dcmi.c	No	No	Yes	Yes	Yes	Yes	Yes	Yes	No	No	Yes	Yes	Yes	No	No	No	No
stm32f4xx_hal_dcmi_ex.c	No	No	Yes	Yes	Yes	Yes	Yes	Yes	No	No	Yes	Yes	Yes	No	No	No	No
stm32f4xx_hal_dfldm.c	No	No	No	No	No	No	No	No	No	No	No	No	No	No	Yes	Yes	Yes
stm32f4xx_hal_dma.c	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes
stm32f4xx_hal_dma2d.c	No	No	No	No	Yes	Yes	Yes	Yes	No	No	No	Yes	Yes	No	No	No	No
stm32f4xx_hal_dma_ex.c	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes
stm32f4xx_hal_dsi.c	No	No	No	No	No	No	No	No	No	No	No	Yes	Yes	No	No	No	No
stm32f4xx_hal_eth.c	No	No	Yes	Yes	Yes	Yes	Yes	Yes	No	No	No	Yes	Yes	No	No	No	No
stm32f4xx_hal_flash.c	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes
stm32f4xx_hal_flash_ex.c	No	No	No	No	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes
stm32f4xx_hal_flash_ramfunc.c	No	No	No	No	No	No	No	No	No	No	Yes	No	No	Yes	Yes	No	No
stm32f4xx_hal_fmapi2c.c	No	No	No	No	No	No	No	No	No	No	Yes	No	No	Yes	Yes	Yes	Yes
stm32f4xx_hal_fmapi2c_ex.c	No	No	No	No	No	No	No	No	No	No	Yes	No	No	Yes	Yes	Yes	Yes
stm32f4xx_hal_gpio.c	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes
stm32f4xx_hal_hash.c	No	Yes	No	Yes	No	Yes	No	Yes	No	No	No	No	Yes	No	No	No	No
stm32f4xx_hal_hash_ex.c	No	No	No	No	No	Yes	No	Yes	No	No	No	No	Yes	No	No	No	No
stm32f4xx_hal_hcd.c	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	No	Yes	Yes	Yes
stm32f4xx_hal_i2c.c	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes
stm32f4xx_hal_i2c_ex.c	No	No	No	No	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes
stm32f4xx_hal_i2s.c	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes
stm32f4xx_hal_i2s_ex.c	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	No	Yes	Yes	Yes
stm32f4xx_hal_irda.c	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes
stm32f4xx_hal_iwdg.c	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes
stm32f4xx_hal_lptim.c	No	No	No	No	No	No	No	No	No	No	No	No	No	Yes	No	Yes	Yes

IP/Module	STM32F405xx	STM32F415xx	STM32F407xx	STM32F417xx	STM32F427xx	STM32F437xx	STM32F429xx	STM32F439xx	STM32F401xC	STM32F401xE	STM32F446xx	STM32F469xx	STM32F479xx	STM32F410xx	STM32F412xx	STM32F413xx	STM32F423xx
stm32f4xx_hal_itdc.c	No	No	No	No	No	No	Yes	Yes	No	No	No	Yes	Yes	No	No	No	No
stm32f4xx_hal_itdc_ex.c	No	No	No	No	No	No	No	No	No	No	No	Yes	Yes	No	No	No	No
stm32f4xx_hal_msp_template.c	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA
stm32f4xx_hal_nand.c	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	No	No	Yes	Yes	Yes	No	No	No	No
stm32f4xx_hal_nor.c	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	No	No	Yes	Yes	Yes	No	Yes	Yes	Yes
stm32f4xx_hal_pccard.c	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	No	No	No	No	No	No	No	No	No
stm32f4xx_hal_pcd.c	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	No	Yes	Yes	Yes
stm32f4xx_hal_pcd_ex.c	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	No	Yes	Yes	Yes
stm32f4xx_hal_pwr.c	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes
stm32f4xx_hal_pwr_ex.c	No	No	No	No	Yes	Yes	Yes	Yes	No	No	Yes	Yes	Yes	Yes	Yes	Yes	Yes
stm32f4xx_hal_qspi.c	No	No	No	No	No	No	No	No	No	No	Yes	Yes	Yes	No	Yes	Yes	Yes
stm32f4xx_hal_rcc.c	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes
stm32f4xx_hal_rcc_ex.c	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes
stm32f4xx_hal_rng.c	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	No	Yes	Yes	Yes	Yes	Yes	Yes
stm32f4xx_hal_rtc.c	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes
stm32f4xx_hal_rtc_ex.c	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes
stm32f4xx_hal_sai.c	No	No	No	No	Yes	Yes	Yes	Yes	No	No	Yes	Yes	Yes	No	No	Yes	Yes
stm32f4xx_hal_sai_ex.c	No	No	No	No	Yes	Yes	Yes	Yes	No	No	Yes	Yes	Yes	No	No	Yes	Yes
stm32f4xx_hal_sd.c	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	No	Yes	Yes	Yes
stm32f4xx_hal_sdram.c	No	No	No	No	Yes	Yes	Yes	Yes	No	No	Yes	Yes	Yes	No	No	No	No
stm32f4xx_hal_smartcard.c	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes
stm32f4xx_hal_spdifrx.c	No	No	No	No	No	No	No	No	No	No	Yes	No	No	No	No	No	No
stm32f4xx_hal_spi.c	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes
stm32f4xx_hal_sram.c	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	No	No	Yes	Yes	Yes	No	Yes	Yes	Yes
stm32f4xx_hal_tim.c	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes
stm32f4xx_hal_tim_ex.c	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes
stm32f4xx_hal_uart.c	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes
stm32f4xx_hal_usart.c	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes
stm32f4xx_hal_wwdg.c	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes
stm32f4xx_ll_fmc.c	No	No	No	No	Yes	Yes	Yes	Yes	No	No	Yes	Yes	Yes	No	No	No	No
stm32f4xx_ll_fsmc.c	Yes	Yes	Yes	Yes	No	No	No	No	No	No	No	No	No	No	Yes	No	No
stm32f4xx_ll_sdmmc.c	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	No	Yes	No	No
stm32f4xx_ll_usb.c	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	No	Yes	No	No

3.5 HAL driver rules

3.5.1 HAL API naming rules

The following naming rules are used in HAL drivers:

Table 6. HAL API naming rules

	Generic	Family specific APIs	Device specific APIs
File names	<i>stm32f4xx_hal_ppp (c/h)</i>	<i>stm32f4xx_hal_ppp_ex (c/h)</i>	<i>stm32f4xx_hal_ppp_ex (c/h)</i>
Module name	<i>HAL_PPP_MODULE</i>		
Function name	<i>HAL_PPP_Function</i> <i>HAL_PPP_FeatureFunction_MODE</i>	<i>HAL_PPPEX_Function</i> <i>HAL_PPPEX_FeatureFunction_MODE</i>	<i>HAL_PPPEX_Function</i> <i>HAL_PPPEX_FeatureFunction_MODE</i>
Handle name	<i>PPP_HandleTypeDef</i>	NA	NA
Init structure name	<i>PPP_InitTypeDef</i>	NA	<i>PPP_InitTypeDef</i>
Enum name	<i>HAL_PPP_StructnameTypeDef</i>	NA	NA

- The **PPP** prefix refers to the peripheral functional mode and not to the peripheral itself. For example, if the USART, PPP can be USART, IRDA, UART or SMARTCARD depending on the peripheral mode.
- The constants used in one file are defined within this file. A constant used in several files is defined in a header file. All constants are written in uppercase, except for peripheral driver function parameters.
- typedef variable names should be suffixed with *_TypeDef*.
- Registers are considered as constants. In most cases, their name is in uppercase and uses the same acronyms as in the STM32F4 reference manuals.
- Peripheral registers are declared in the *PPP_TypeDef* structure (for example *ADC_TypeDef*) in the *stm32f4xxx.h* header file:
stm32f4xxx.h corresponds to *stm32f401xc.h*, *stm32f401xe.h*, *stm32f405xx.h*, *stm32f415xx.h*, *stm32f407xx.h*, *stm32f417xx.h*, *stm32f427xx.h*, *stm32f437xx.h*, *stm32f429xx.h*, *stm32f439xx.h*, *stm32f446xx.h*, *stm32f469xx.h*, *stm32f479xx.h*, *stm32f410cx.h*, *stm32f410tx.h*, *stm32f410rx.h*, *stm32f412cx.h*, *stm32f412rx.h*, *stm32f412vx.h*, *stm32f412zx.h*, *stm32f413xx.h*, or *stm32f423xx.h*.
- Peripheral function names are prefixed by *HAL_*, then the corresponding peripheral acronym in uppercase followed by an underscore. The first letter of each word is in uppercase (for example *HAL_UART_Transmit()*). Only one underscore is allowed in a function name to separate the peripheral acronym from the rest of the function name.
- The structure containing the PPP peripheral initialization parameters are named *PPP_InitTypeDef* (for example *ADC_InitTypeDef*).
- The structure containing the Specific configuration parameters for the PPP peripheral are named *PPP_xxxxConfTypeDef* (for example *ADC_ChannelConfTypeDef*).
- Peripheral handle structures are named *PPP_HandleTypeDef* (e.g *DMA_HandleTypeDef*)
- The functions used to initialize the PPP peripheral according to parameters specified in *PPP_InitTypeDef* are named *HAL_PPP_Init* (for example *HAL_TIM_Init()*).
- The functions used to reset the PPP peripheral registers to their default values are named *HAL_PPP_DeInit* (for example *HAL_TIM_DeInit()*).
- The **MODE** suffix refers to the process mode, which can be polling, interrupt or DMA. As an example, when the DMA is used in addition to the native resources, the function should be called: *HAL_PPP_Function_DMA()*.
- The **Feature** prefix should refer to the new feature.
Example: *HAL_ADCEX_InjectedStart()* refers to the injection mode.

3.5.2 HAL general naming rules

- For the shared and system peripherals, no handle or instance object is used. This rule applies to the following peripherals:
 - GPIO
 - SYSTICK
 - NVIC
 - RCC
 - FLASH

Example: The `HAL_GPIO_Init()` requires only the GPIO address and its configuration parameters.

```
HAL_StatusTypeDef HAL_GPIO_Init (GPIO_TypeDef* GPIOx, GPIO_InitTypeDef *Init)
{
  /*GPIO Initialization body */
}
```

- The macros that handle interrupts and specific clock configurations are defined in each peripheral/module driver. These macros are exported in the peripheral driver header files so that they can be used by the extension file. The list of these macros is defined below:

Note: This list is not exhaustive and other macros related to peripheral features can be added, so that they can be used in the user application.

Table 7. Macros handling interrupts and specific clock configurations

Macros	Description
<code>__HAL_PPP_ENABLE_IT(__HANDLE__, __INTERRUPT__)</code>	Enables a specific peripheral interrupt
<code>__HAL_PPP_DISABLE_IT(__HANDLE__, __INTERRUPT__)</code>	Disables a specific peripheral interrupt
<code>__HAL_PPP_GET_IT (__HANDLE__, __ INTERRUPT __)</code>	Gets a specific peripheral interrupt status
<code>__HAL_PPP_CLEAR_IT (__HANDLE__, __ INTERRUPT __)</code>	Clears a specific peripheral interrupt status
<code>__HAL_PPP_GET_FLAG (__HANDLE__, __FLAG__)</code>	Gets a specific peripheral flag status
<code>__HAL_PPP_CLEAR_FLAG (__HANDLE__, __FLAG__)</code>	Clears a specific peripheral flag status
<code>__HAL_PPP_ENABLE(__HANDLE__)</code>	Enables a peripheral
<code>__HAL_PPP_DISABLE(__HANDLE__)</code>	Disables a peripheral
<code>__HAL_PPP_XXXX (__HANDLE__, __PARAM__)</code>	Specific PPP HAL driver macro
<code>__HAL_PPP_GET_IT_SOURCE (__HANDLE__, __ INTERRUPT __)</code>	Checks the source of specified interrupt

- NVIC and SYSTICK are two Arm® Cortex® core features. The APIs related to these features are located in the `stm32f4xx_hal_cortex.c` file.
- The PPP handles are valid before using the `HAL_PPP_Init()` API. The init function performs a check before modifying the handle fields.

```
HAL_PPP_Init(PPP_HandleTypeDef)
if(hppp == NULL)
{
  return HAL_ERROR;
}
```

- The macros defined below are used:

- Conditional macro:

```
#define ABS(x) (((x) > 0) ? (x) : -(x))
```

- Pseudo-code macro (multiple instructions macro):

```
#define __HAL_LINKDMA(__HANDLE__, __PPP_DMA_FIELD__, __DMA_HANDLE__) \
do{ \
(__HANDLE__)->__PPP_DMA_FIELD__ = &(__DMA_HANDLE__); \
(__DMA_HANDLE__).Parent = (__HANDLE__); \
}while(0)
```

3.5.3 HAL interrupt handler and callback functions

Besides the APIs, HAL peripheral drivers include:

- HAL_PPP_IRQHandler() peripheral interrupt handler that should be called from stm32f4xx_it.c
- User callback functions.

The user callback functions are defined as empty functions with “weak” attribute. They have to be defined in the user code.

There are three types of user callbacks functions:

- Peripheral system level initialization/ de-Initialization callbacks: HAL_PPP_MspInit() and HAL_PPP_MspDeInit
- Process complete callbacks : HAL_PPP_ProcessCpltCallback
- Error callback: HAL_PPP_ErrorCallback.

Table 8. Callback functions

Callback functions	Example
HAL_PPP_MspInit() / _DeInit()	Example: HAL_USART_MspInit() Called from HAL_PPP_Init() API function to perform peripheral system level initialization (GPIOs, clock, DMA, interrupt)
HAL_PPP_ProcessCpltCallback	Example: HAL_USART_TxCpltCallback Called by peripheral or DMA interrupt handler when the process completes
HAL_PPP_ErrorCallback	Example: HAL_USART_ErrorCallback Called by peripheral or DMA interrupt handler when an error occurs

3.6 HAL generic APIs

The generic APIs provide common generic functions applying to all STM32 devices. They are composed of four APIs groups:

- Initialization and de-initialization functions:** HAL_PPP_Init(), HAL_PPP_DeInit()
- IO operation functions:** HAL_PPP_Read(), HAL_PPP_Write(), HAL_PPP_Transmit(), HAL_PPP_Receive()
- Control functions:** HAL_PPP_Set (), HAL_PPP_Get ().
- State and Errors functions:** HAL_PPP_GetState (), HAL_PPP_GetError ().

For some peripheral/module drivers, these groups are modified depending on the peripheral/module implementation.

Example: in the timer driver, the API grouping is based on timer features (such as PWM, OC and IC).

The initialization and de-initialization functions allow initializing a peripheral and configuring the low-level resources, mainly clocks, GPIO, alternate functions (AF) and possibly DMA and interrupts. The *HAL_DeInit()* function restores the peripheral default state, frees the low-level resources and removes any direct dependency with the hardware.

The IO operation functions perform a row access to the peripheral payload data in write and read modes.

The control functions are used to change dynamically the peripheral configuration and set another operating mode.

The peripheral state and errors functions allow retrieving in run time the peripheral and data flow states, and identifying the type of errors that occurred. The example below is based on the ADC peripheral. The list of generic APIs is not exhaustive. It is only given as an example.

Table 9. HAL generic APIs

Function group	Common API name	Description
<i>Initialization group</i>	<i>HAL_ADC_Init()</i>	This function initializes the peripheral and configures the low -level resources (clocks, GPIO, AF..)
	<i>HAL_ADC_DeInit()</i>	This function restores the peripheral default state, frees the low-level resources and removes any direct dependency with the hardware.
<i>IO operation group</i>	<i>HAL_ADC_Start ()</i>	This function starts ADC conversions when the polling method is used
	<i>HAL_ADC_Stop ()</i>	This function stops ADC conversions when the polling method is used
	<i>HAL_ADC_PollForConversion()</i>	This function allows waiting for the end of conversions when the polling method is used. In this case, a timeout value is specified by the user according to the application.
	<i>HAL_ADC_Start_IT()</i>	This function starts ADC conversions when the interrupt method is used
	<i>HAL_ADC_Stop_IT()</i>	This function stops ADC conversions when the interrupt method is used
	<i>HAL_ADC_IRQHandler()</i>	This function handles ADC interrupt requests
	<i>HAL_ADC_ConvCpltCallback()</i>	Callback function called in the IT subroutine to indicate the end of the current process or when a DMA transfer has completed
	<i>HAL_ADC_ErrorCallback()</i>	Callback function called in the IT subroutine if a peripheral error or a DMA transfer error occurred
<i>Control group</i>	<i>HAL_ADC_ConfigChannel()</i>	This function configures the selected ADC regular channel, the corresponding rank in the sequencer and the sample time
	<i>HAL_ADC_AnalogWDGConfig</i>	This function configures the analog watchdog for the selected ADC
<i>State and Errors group</i>	<i>HAL_ADC_GetState()</i>	This function allows getting in run time the peripheral and the data flow states.
	<i>HAL_ADC_GetError()</i>	This fuction allows getting in run time the error that occurred during IT routine

3.7 HAL extension APIs

3.7.1 HAL extension model overview

The extension APIs provide specific functions or overwrite modified APIs for a specific family (series) or specific part number within the same family.

The extension model consists of an additional file, *stm32f4xx_hal_ppp_ex.c*, that includes all the specific functions and define statements (*stm32f4xx_hal_ppp_ex.h*) for a given part number.

Below an example based on the ADC peripheral:

Table 10. HAL extension APIs

Function group	Common API name
<i>HAL_ADCEx_InjectedStart()</i>	This function starts injected channel ADC conversions when the polling method is used
<i>HAL_ADCEx_InjectedStop()</i>	This function stops injected channel ADC conversions when the polling method is used

Function group	Common API name
<i>HAL_ADCEx_InjectedStart_IT()</i>	This function starts injected channel ADC conversions when the interrupt method is used
<i>HAL_ADCEx_InjectedStop_IT()</i>	This function stops injected channel ADC conversions when the interrupt method is used
<i>HAL_ADCEx_InjectedConfigChannel()</i>	This function configures the selected ADC Injected channel (corresponding rank in the sequencer and sample time)

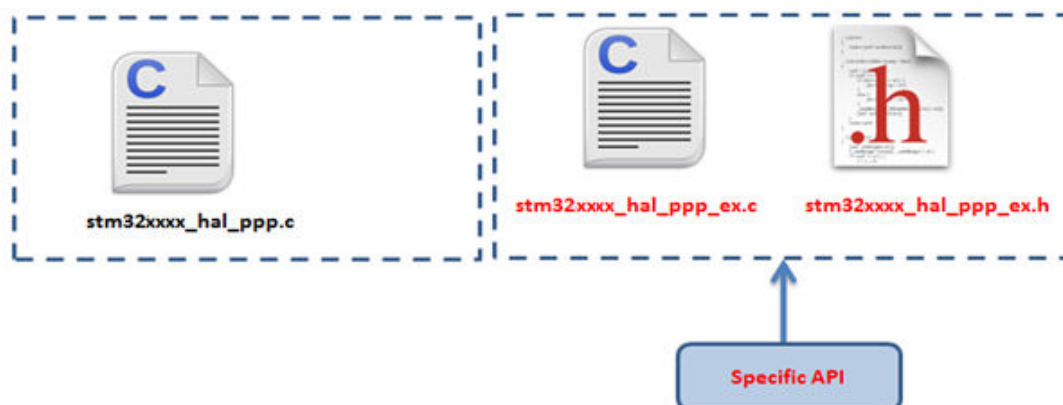
3.7.2 HAL extension model cases

The specific peripheral features can be handled by the HAL drivers in five different ways. They are described below.

Adding a part number-specific function

When a new feature specific to a given device is required, the new APIs are added in the stm32f4xx_hal_ppp_ex.c extension file. They are named HAL_PPPEX_Function().

Figure 2. Adding device-specific functions



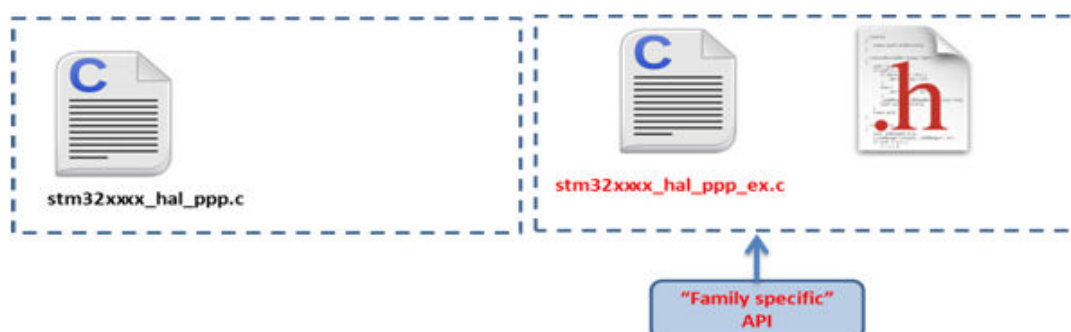
Example: `stm32f4xx_hal_flash_ex.c/h`

```
#if defined(STM32F427xx) || defined(STM32F437xx) || defined(STM32F429xx) || defined(STM32F439xx)
HAL_StatusTypeDef HAL_FLASHEx_OB_SelectPCROP(void);
HAL_StatusTypeDef HAL_FLASHEx_OB_DeSelectPCROP(void);
#endif /* STM32F427xx || STM32F437xx || STM32F429xx || STM32F439xx */
```

Adding a family-specific function

In this case, the API is added in the extension driver C file and named HAL_PPPEX_Function().

Figure 3. Adding family-specific functions



Example: stm32f4xx_hal_adc_ex.c/h

```
HAL_StatusTypeDef HAL_ADCEx_InjectedStop(ADC_HandleTypeDef* hadc);
HAL_StatusTypeDef HAL_ADCEx_InjectedStop_IT(ADC_HandleTypeDef* hadc);
HAL_StatusTypeDef HAL_ADCEx_InjectedStart(ADC_HandleTypeDef* hadc);
HAL_StatusTypeDef HAL_ADCEx_InjectedStart_IT(ADC_HandleTypeDef* hadc);
```

Adding a new peripheral (specific to a device belonging to a given family)

When a peripheral which is available only in a specific device is required, the APIs corresponding to this new peripheral/module (newPPP) are added in a new stm32f4xx_hal_newppp.c. However the inclusion of this file is selected in the stm32f4xx_hal_conf.h using the macro:

```
#define HAL_NEWPPP_MODULE_ENABLED
```

Figure 4. Adding new peripherals

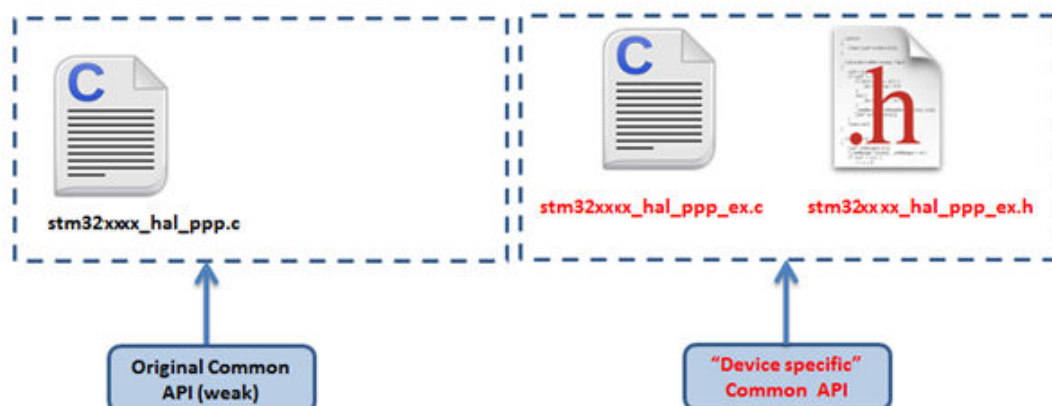


Example: stm32f4xx_hal_adc.c/h

Updating existing common APIs

In this case, the routines are defined with the same names in the stm32f4xx_hal_ppp_ex.c extension file, while the generic API is defined as *weak*, so that the compiler overwrites the original routine by the new defined function.

Figure 5. Updating existing APIs



Updating existing data structures

The data structure for a specific device part number (for example PPP_InitTypeDef) can have different fields. In this case, the data structure is defined in the extension header file and delimited by the specific part number define statement.

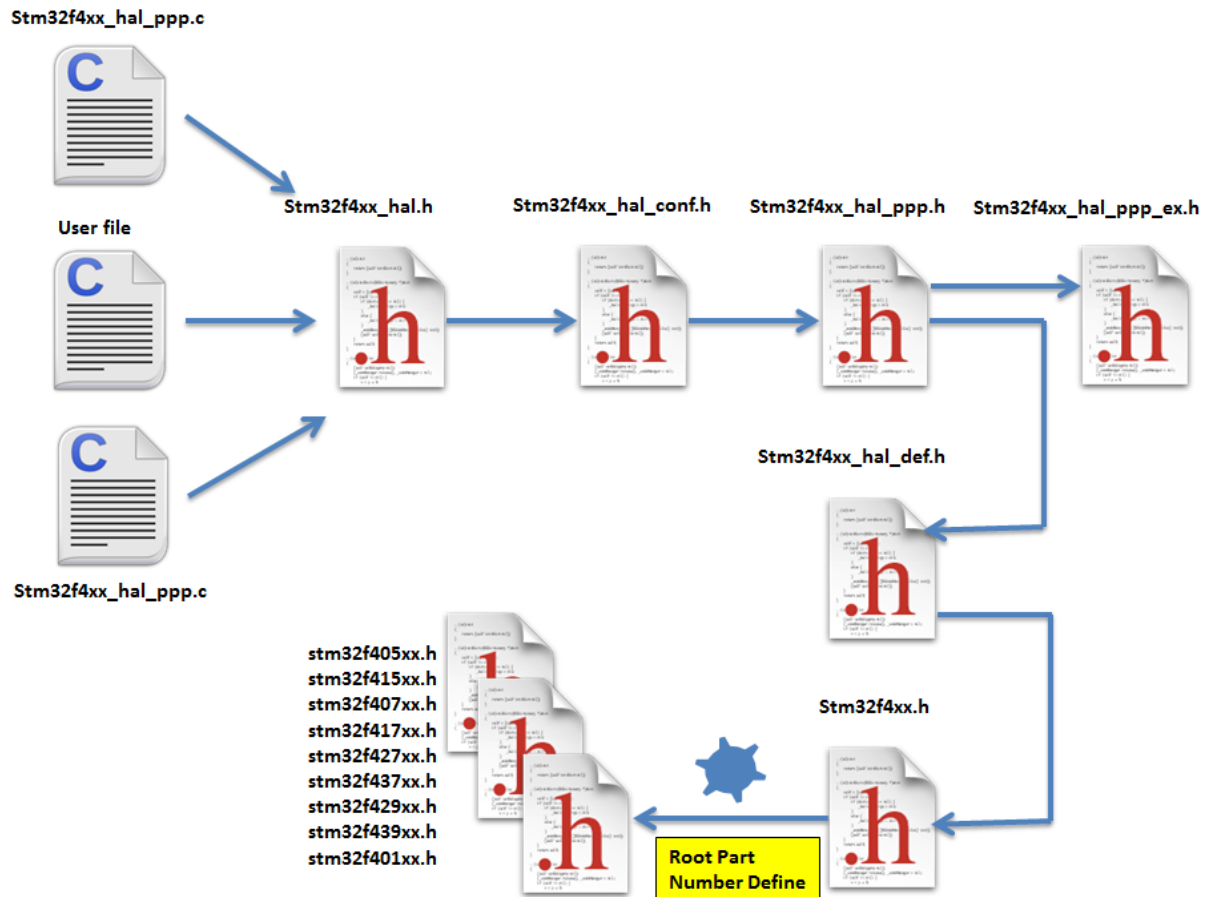
Example:

```
#if defined (STM32F401xx)
typedef struct
{
    (...)
}PPP_InitTypeDef;
#endif /* STM32F401xx */
```

3.8 File inclusion model

The header of the common HAL driver file (stm32f4xx_hal.h) includes the common configurations for the whole HAL library. It is the only header file that is included in the user sources and the HAL C sources files to be able to use the HAL resources.

Figure 6. File inclusion model



A PPP driver is a standalone module which is used in a project. The user must enable the corresponding `USE_HAL_PPP_MODULE` define statement in the configuration file.

```

/*****
 * @file stm32f4xx_hal_conf.h
 * @author MCD Application Team
 * @version VX.Y.Z * @date dd-mm-yyyy
 * @brief This file contains the modules to be used
 *****/
(...)
#define USE_HAL_USART_MODULE
#define USE_HAL_IRDA_MODULE
#define USE_HAL_DMA_MODULE
#define USE_HAL_RCC_MODULE
(...)

```


3.9 HAL common resources

The common HAL resources, such as common define enumerations, structures and macros, are defined in *stm32f4xx_hal_def.h*. The main common define enumeration is *HAL_StatusTypeDef*.

- **HAL Status**

The HAL status is used by almost all HAL APIs, except for boolean functions and IRQ handler. It returns the status of the current API operations. It has four possible values as described below:

```
typedef enum
{
    HAL_OK = 0x00,
    HAL_ERROR = 0x01,
    HAL_BUSY = 0x02,
    HAL_TIMEOUT = 0x03
} HAL_StatusTypeDef;
```

- **HAL Locked**

The HAL lock is used by all HAL APIs to prevent accessing by accident shared resources.

```
typedef enum
{
    HAL_UNLOCKED = 0x00, /*!<Resources unlocked */
    HAL_LOCKED = 0x01 /*!< Resources locked */
} HAL_LockTypeDef;
```

In addition to common resources, the *stm32f4xx_hal_def.h* file calls the *stm32f4xx.h* file in CMSIS library to get the data structures and the address mapping for all peripherals:

- Declarations of peripheral registers and bits definition.
- Macros to access peripheral registers hardware (such as Write register or Read register).

- **Common macros**

- Macro defining *HAL_MAX_DELAY*

```
#define HAL_MAX_DELAY 0xFFFFFFFF
```

- Macro linking a PPP peripheral to a DMA structure pointer:

```
#define __HAL_LINKDMA(__HANDLE__, __PPP_DMA_FIELD__, __DMA_HANDLE__) \
do{ \
    (__HANDLE__)->__PPP_DMA_FIELD__ = &(__DMA_HANDLE__); \
    (__DMA_HANDLE__).Parent = (__HANDLE__); \
}while(0)
```

3.10 HAL configuration

The configuration file, *stm32f4xx_hal_conf.h*, allows customizing the drivers for the user application. Modifying this configuration is not mandatory: the application can use the default configuration without any modification.

To configure these parameters, the user should enable, disable or modify some options by uncommenting, commenting or modifying the values of the related define statements as described in the table below:

Table 11. Define statements used for HAL configuration

Configuration item	Description	Default Value
HSE_VALUE	Defines the value of the external oscillator (HSE) expressed in Hz. The user must adjust this define statement when using a different crystal value.	25 000 000 Hz
HSE_STARTUP_TIMEOUT	Timeout for HSE start-up, expressed in ms	5000 ms
HSI_VALUE	Defines the value of the internal oscillator (HSI) expressed in Hz.	16 000 000 Hz
EXTERNAL_CLOCK_VALUE	This value is used by the I2S/SAI HAL module to compute the I2S/SAI clock source frequency. This source is inserted directly through I2S_CKIN pad.	12288000 Hz
VDD_VALUE	V _{DD} value	3300 mV
USE_RTOS	Enables the use of RTOS	FALSE (for future use)

Configuration item	Description	Default Value
PREFETCH_ENABLE	Enables prefetch feature	TRUE
INSTRUCTION_CACHE_ENABLE	Enables instruction cache	TRUE
DATA_CACHE_ENABLE	Enables data cache	TRUE
USE_HAL_PPP_MODULE	Enables module to be used in the HAL driver	
MAC_ADDRx	Ethernet peripheral configuration : MAC address	
ETH_RX_BUF_SIZE	Ethernet buffer size for data reception	ETH_MAX_PACKET_SIZE
ETH_TX_BUF_SIZE	Ethernet buffer size for transmit	ETH_MAX_PACKET_SIZE
ETH_RXBUFNB	The number of Rx buffers of size ETH_RX_BUF_SIZE	4
ETH_TXBUFNB	The number of Tx buffers of size ETH_RX_BUF_SIZE	4
DP83848_PHY_ADDRESS	DB83848 Ethernet PHY Address	0x01
PHY_RESET_DELAY	PHY Reset delay these values are based on a 1 ms SysTick interrupt	0x000000FF
PHY_CONFIG_DELAY	PHY Configuration delay	0x00000FFF
PHY_BCR PHY_BSR	Common PHY Registers	
PHY_SR PHY_MICR PHY_MISR	Extended PHY registers	

Note: The `stm32f4xx_hal_conf_template.h` file is located in the HAL drivers Inc folder. It should be copied to the user folder, renamed and modified as described above.

By default, the values defined in the `stm32f4xx_hal_conf_template.h` file are the same as the ones used for the examples and demonstrations. All HAL include files are enabled so that they can be used in the user code without modifications.

3.11 HAL system peripheral handling

This section gives an overview of how the system peripherals are handled by the HAL drivers. The full API list is provided within each peripheral driver description section.

3.11.1 Clocks

Two main functions can be used to configure the system clock:

- `HAL_RCC_OscConfig` (`RCC_OscInitTypeDef *RCC_OscInitStruct`). This function configures/enables multiple clock sources (HSE, HSI, LSE, LSI, PLL).
- `HAL_RCC_ClockConfig` (`RCC_ClkInitTypeDef *RCC_ClkInitStruct, uint32_t FLatency`). This function
 - selects the system clock source
 - configures AHB, APB1, and APB2 clock dividers
 - configures the number of Flash memory wait states
 - updates the SysTick configuration when HCLK clock changes.

Some peripheral clocks are not derived from the system clock (such as RTC, SDIO, I2S, Audio, PLL).

In this case, the clock configuration is performed by an extended API defined in `stm32f4xx_hal_rcc_ex.c`: `HAL_RCCEx_PeriphCLKConfig` (`RCC_PeriphCLKInitTypeDef *PeriphClkInit`).

Additional RCC HAL driver functions are available:

- `HAL_RCC_DeInit()` Clock de-initialization function that returns clock configuration to reset state
- Get clock functions that allow retrieving various clock configurations (such as system clock, HCLK, PCLK1 or PCLK2)
- MCO and CSS configuration functions

A set of macros are defined in `stm32f4xx_hal_rcc.h` and `stm32f4xx_hal_rcc_ex.h`. They allow executing elementary operations on RCC block registers, such as peripherals clock gating/reset control:

- `__HAL_PPP_CLK_ENABLE/ __HAL_PPP_CLK_DISABLE` to enable/disable the peripheral clock.
- `__HAL_PPP_FORCE_RESET/ __HAL_PPP_RELEASE_RESET` to force/release peripheral reset

- `__HAL_PPP_CLK_SLEEP_ENABLE/ __HAL_PPP_CLK_SLEEP_DISABLE` to enable/disable the peripheral clock during Sleep mode.

3.11.2

GPIOs

GPIO HAL APIs are the following:

- `HAL_GPIO_Init()` / `HAL_GPIO_DeInit()`
- `HAL_GPIO_ReadPin()` / `HAL_GPIO_WritePin()`
- `HAL_GPIO_TogglePin()`.

In addition to standard GPIO modes (input, output, analog), the pin mode can be configured as EXTI with interrupt or event generation.

When selecting EXTI mode with interrupt generation, the user must call `HAL_GPIO_EXTI_IRQHandler()` from `stm32f4xx_it.c` and implement `HAL_GPIO_EXTI_Callback()`

The table below describes the `GPIO_InitTypeDef` structure field.

Table 12. Description of `GPIO_InitTypeDef` structure

Structure field	Description
Pin	Specifies the GPIO pins to be configured. Possible values: <code>GPIO_PIN_x</code> or <code>GPIO_PIN_All</code> , where <code>x[0..15]</code>
Mode	Specifies the operating mode for the selected pins: GPIO mode or EXTI mode. Possible values are: • <u>GPIO mode</u> – <code>GPIO_MODE_INPUT</code> : Input floating – <code>GPIO_MODE_OUTPUT_PP</code> : Output push-pull – <code>GPIO_MODE_OUTPUT_OD</code> : Output open drain – <code>GPIO_MODE_AF_PP</code> : Alternate function push-pull – <code>GPIO_MODE_AF_OD</code> : Alternate function open drain – <code>GPIO_MODE_ANALOG</code> : Analog mode • <u>External Interrupt mode</u> – <code>GPIO_MODE_IT_RISING</code> : Rising edge trigger detection – <code>GPIO_MODE_IT_FALLING</code> : Falling edge trigger detection – <code>GPIO_MODE_IT_RISING_FALLING</code> : Rising/Falling edge trigger detection • <u>External Event mode</u> – <code>GPIO_MODE_EVT_RISING</code> : Rising edge trigger detection – <code>GPIO_MODE_EVT_FALLING</code> : Falling edge trigger detection – <code>GPIO_MODE_EVT_RISING_FALLING</code> : Rising/Falling edge trigger detection
Pull	Specifies the Pull-up or Pull-down activation for the selected pins. Possible values are: <code>GPIO_NOPULL</code> <code>GPIO_PULLUP</code> <code>GPIO_PULLDOWN</code>
Speed	Specifies the speed for the selected pins Possible values are: <code>GPIO_SPEED_LOW</code> <code>GPIO_SPEED_MEDIUM</code> <code>GPIO_SPEED_FAST</code> <code>GPIO_SPEED_HIGH</code>
Alternate	Peripheral to be connected to the selected pins. Possible values: <code>GPIO_AFx_PPP</code> , where <code>AFx</code> : is the alternate function index and <code>PPP</code> : is the peripheral instance Example: use <code>GPIO_AF1_TIM2</code> to connect TIM2 I/Os on AF1.

Structure field	Description
	These values are defined in the GPIO extended driver, since the AF mapping may change between product lines. <i>Note:</i> Refer to the “Alternate function mapping” table in the datasheets for the detailed description of the system and peripheral I/O alternate functions.

Please find below typical GPIO configuration examples:

- Configuring GPIOs as output push-pull to drive external LEDs:

```
GPIO_InitStruct.Pin = GPIO_PIN_12 | GPIO_PIN_13 | GPIO_PIN_14 | GPIO_PIN_15;
GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
GPIO_InitStruct.Pull = GPIO_PULLUP;
GPIO_InitStruct.Speed = GPIO_SPEED_FAST;
HAL_GPIO_Init(GPIOD, &GPIO_InitStruct);
```

- Configuring PA0 as external interrupt with falling edge sensitivity:

```
GPIO_InitStructure.Mode = GPIO_MODE_IT_FALLING;
GPIO_InitStructure.Pull = GPIO_NOPULL;
GPIO_InitStructure.Pin = GPIO_PIN_0;
HAL_GPIO_Init(GPIOA, &GPIO_InitStructure);
```

- Configuring USART3 Tx (PC10, mapped on AF7) as alternate function:

```
GPIO_InitStruct.Pin = GPIO_PIN_10;
GPIO_InitStruct.Mode = GPIO_MODE_AF_PP;
GPIO_InitStruct.Pull = GPIO_PULLUP;
GPIO_InitStruct.Speed = GPIO_SPEED_FAST;
GPIO_InitStruct.Alternate = GPIO_AF7_USART3;
HAL_GPIO_Init(GPIOC, &GPIO_InitStruct);
```

3.11.3 Cortex® NVIC and SysTick timer

The Cortex® HAL driver, stm32f4xx_hal_cortex.c, provides APIs to handle NVIC and SysTick. The supported APIs include:

- HAL_NVIC_SetPriorityGrouping()
 - HAL_NVIC_SetPriority()
 - HAL_NVIC_EnableIRQ()/HAL_NVIC_DisableIRQ()
 - HAL_NVIC_SystemReset()
 - HAL_NVIC_GetPendingIRQ() / HAL_NVIC_SetPendingIRQ () / HAL_NVIC_ClearPendingIRQ()
 - HAL_SYSTICK_Config()
 - HAL_SYSTICK_CLKSourceConfig()

3.11.4 PWR

The PWR HAL driver handles power management. The features shared between all STM32 Series are listed below:

- PVD configuration, enabling/disabling and interrupt handling
 - HAL_PWR_ConfigPVD()
 - HAL_PWR_EnablePVD() / HAL_PWR_DisablePVD()
 - HAL_PWR_PVD_IRQHandler()
 - HAL_PWR_PVDCallback()
- Wakeup pin configuration
 - HAL_PWR_EnableWakeUpPin() / HAL_PWR_DisableWakeUpPin()
- Low-power mode entry
 - HAL_PWR_EnterSLEEPMode()
 - HAL_PWR_EnterSTOPMode()
 - HAL_PWR_EnterSTANDBYMode()

Depending on the STM32 Series, extension functions are available in stm32f4xx_hal_pwr_ex. Here are a few examples (the list is not exhaustive):

- Backup domain registers enable/disable
 - HAL_PWREx_EnableBkUpReg() / HAL_PWREx_DisableBkUpReg()
- Flash overdrive control and Flash power-down (available only on STM32F429/F439xx)
 - HAL_PWREx_ActivateOverDrive()
 - HAL_PWREx_EnableFlashPowerDown().

3.11.5

EXTI

The EXTI is not considered as a standalone peripheral but rather as a service used by other peripheral, that are handled through EXTI HAL APIs. In addition, each peripheral HAL driver implements the associated EXTI configuration and function as macros in its header file.

The first 16 EXTI lines connected to the GPIOs, are managed within the GPIO driver. The GPIO_InitTypeDef structure allows configuring an I/O as external interrupt or external event.

The EXTI lines connected internally to the PVD, RTC, COMP, and USB, are configured within the HAL drivers of these peripheral through the macros given in the table below.

The EXTI internal connections depend on the targeted STM32 microcontroller (refer to the product datasheet for more details):

Table 13. Description of EXTI configuration macros

Macros	Description
PPP_EXTI_LINE_FUNCTION	Defines the EXTI line connected to the internal peripheral. Example: #define PWR_EXTI_LINE_PVD ((uint32_t)0x00010000) /*! <External interrupt line 16 Connected to the PVD EXTI Line */
__HAL_PPP_EXTI_ENABLE_IT(__EXTI_LINE__)	Enables a given EXTI line Example: __HAL_PVD_EXTI_ENABLE_IT(PWR_EXTI_LINE_PVD)
__HAL_PPP_EXTI_DISABLE_IT(__EXTI_LINE__)	Disables a given EXTI line. Example: __HAL_PVD_EXTI_DISABLE_IT(PWR_EXTI_LINE_PVD)
__HAL_PPP_EXTI_GET_FLAG(__EXTI_LINE__)	Gets a given EXTI line interrupt flag pending bit status. Example: __HAL_PVD_EXTI_GET_FLAG(PWR_EXTI_LINE_PVD)
__HAL_PPP_EXTI_CLEAR_FLAG(__EXTI_LINE__)	Clears a given EXTI line interrupt flag pending bit. Example; __HAL_PVD_EXTI_CLEAR_FLAG(PWR_EXTI_LINE_PVD)
__HAL_PPP_EXTI_GENERATE_SWIT (__EXTI_LINE__)	Generates a software interrupt for a given EXTI line. Example: __HAL_PVD_EXTI_GENERATE_SWIT (PWR_EXTI_LINE_PVD)

If the EXTI interrupt mode is selected, the user application must call HAL_PPP_FUNCTION_IRQHandler() (for example HAL_PWR_PVD_IRQHandler()), from stm32f4xx_it.c file, and implement HAL_PPP_FUNCTIONCallback() callback function (for example HAL_PWR_PVDCallback()).

3.11.6

DMA

The DMA HAL driver allows enabling and configuring the peripheral to be connected to the DMA Channels (except for internal SRAM/FLASH memory which do not require any initialization). Refer to the product reference manual for details on the DMA request corresponding to each peripheral.

For a given channel, HAL_DMA_Init() API allows programming the required configuration through the following parameters:

- Transfer direction
- Source and destination data formats
- Circular, Normal or peripheral flow control mode
- Channel priority level
- Source and destination Increment mode
- FIFO mode and its threshold (if needed)
- Burst mode for source and/or destination (if needed).

Two operating modes are available:

- Polling mode I/O operation
 1. Use HAL_DMA_Start() to start DMA transfer when the source and destination addresses and the Length of data to be transferred have been configured.
 2. Use HAL_DMA_PollForTransfer() to poll for the end of current transfer. In this case a fixed timeout can be configured depending on the user application.
- Interrupt mode I/O operation
 1. Configure the DMA interrupt priority using HAL_NVIC_SetPriority().
 2. Enable the DMA IRQ handler using HAL_NVIC_EnableIRQ().
 3. Use HAL_DMA_Start_IT() to start DMA transfer when the source and destination addresses and the length of data to be transferred have been configured. In this case the DMA interrupt is configured.
 4. Use HAL_DMA_IRQHandler() called under DMA_IRQHandler() Interrupt subroutine.
 5. When data transfer is complete, HAL_DMA_IRQHandler() function is executed and a user function can be called by customizing XferCpltCallback and XferErrorCallback function pointer (that is a member of DMA handle structure).

Additional functions and macros are available to ensure efficient DMA management:

- Use HAL_DMA_GetState() function to return the DMA state and HAL_DMA_GetError() in case of error detection.
- Use HAL_DMA_Abort() function to abort the current transfer.

The most used DMA HAL driver macros are the following:

- __HAL_DMA_ENABLE: enables the specified DMA channel
- __HAL_DMA_DISABLE: disables the specified DMA channel
- __HAL_DMA_GET_FS: returns the current DMA Stream FIFO filled level.
- __HAL_DMA_GET_FLAG: gets the DMA channel pending flags
- __HAL_DMA_CLEAR_FLAG: clears the DMA channel pending flags
- __HAL_DMA_ENABLE_IT: enables the specified DMA channel interrupts
- __HAL_DMA_DISABLE_IT: disables the specified DMA channel interrupts
- __HAL_DMA_GET_IT_SOURCE: checks whether the specified DMA channel interrupt has been enabled or not

Note: When a peripheral is used in DMA mode, the DMA initialization must be done in the HAL_PPP_MspInit() callback. In addition, the user application must associate the DMA handle to the PPP handle (refer to section "HAL IO operation functions").

Note: DMA double-buffering feature is handled as an extension API.

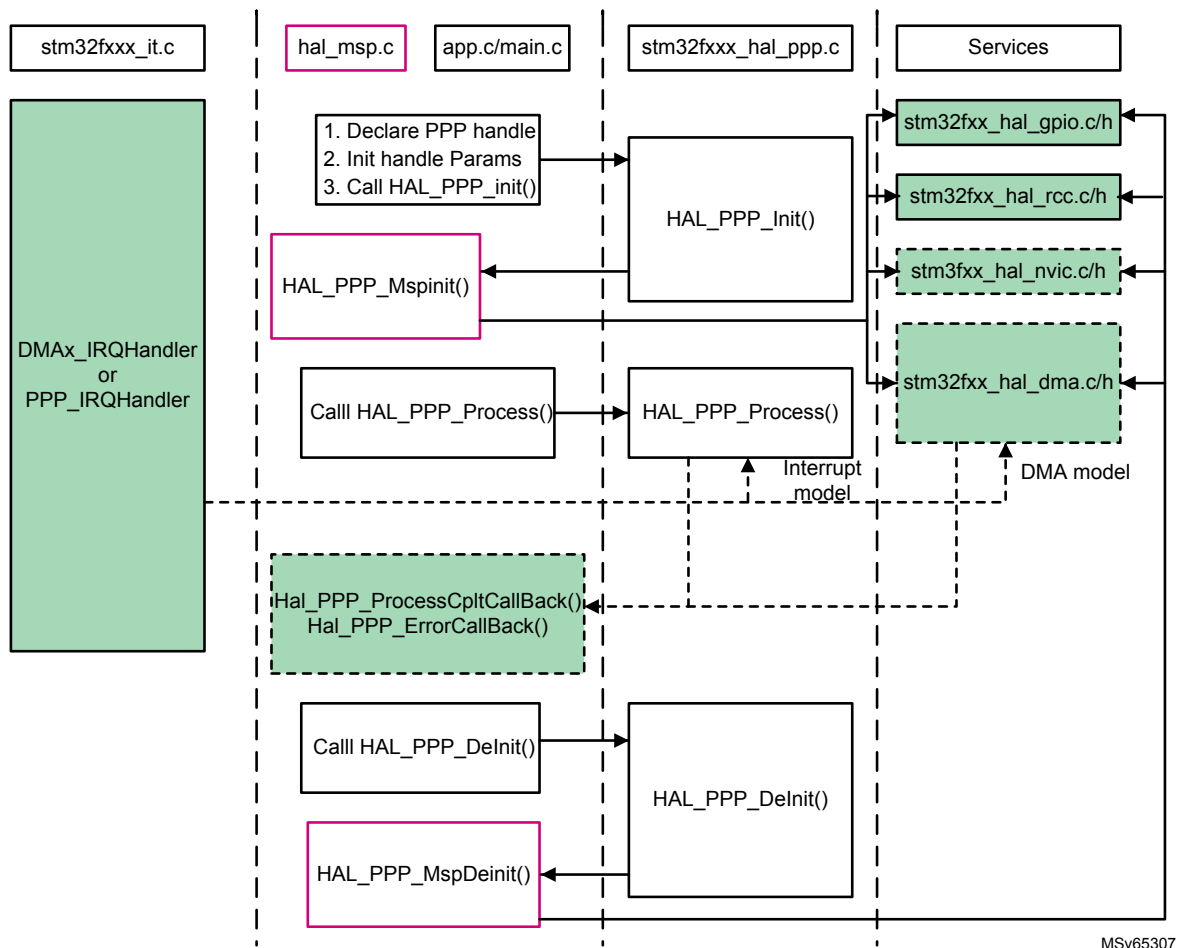
Note: DMA channel callbacks need to be initialized by the user application only in case of memory-to-memory transfer. However when peripheral-to-memory transfers are used, these callbacks are automatically initialized by calling a process API function that uses the DMA.

3.12 How to use HAL drivers

3.12.1 HAL usage models

The following figure shows the typical use of the HAL driver and the interaction between the application user, the HAL driver and the interrupts.

Figure 7. HAL driver model



MSv65307

Note: The functions implemented in the HAL driver are shown in green, the functions called from interrupt handlers in dotted lines, and the msp functions implemented in the user application in red. Non-dotted lines represent the interactions between the user application functions.

Basically, the HAL driver APIs are called from user files and optionally from interrupt handlers file when the APIs based on the DMA or the PPP peripheral dedicated interrupts are used.

When DMA or PPP peripheral interrupts are used, the PPP process complete callbacks are called to inform the user about the process completion in real-time event mode (interrupts). Note that the same process completion callbacks are used for DMA in interrupt mode.

3.12.2 HAL initialization

3.12.2.1 HAL global initialization

In addition to the peripheral initialization and de-initialization functions, a set of APIs are provided to initialize the HAL core implemented in file `stm32f4xx_hal.c`.

- **HAL_Init():** this function must be called at application startup to
 - initialize data/instruction cache and pre-fetch queue
 - set SysTick timer to generate an interrupt each 1ms (based on HSI clock) with the lowest priority
 - set priority grouping to 4 preemption bits
 - call HAL_MspInit() user callback function to perform system level initializations (Clock, GPIOs, DMA, interrupts). HAL_MspInit() is defined as “weak” empty function in the HAL drivers.
- **HAL_DeInit()**
 - resets all peripherals
 - calls function HAL_MspDeInit() which is a user callback function to do system level De-Initializations.
- **HAL_GetTick():** this function gets current SysTick counter value (incremented in SysTick interrupt) used by peripherals drivers to handle timeouts.
- **HAL_Delay().** this function implements a delay (expressed in milliseconds) using the SysTick timer. Care must be taken when using HAL_Delay() since this function provides an accurate delay (expressed in milliseconds) based on a variable incremented in SysTick ISR. This means that if HAL_Delay() is called from a peripheral ISR, then the SysTick interrupt must have highest priority (numerically lower) than the peripheral interrupt, otherwise the caller ISR is blocked.

Note: *In STM32Cube V1.0 implemented in STM32CubeF2 and STM32CubeF4 first versions, the SysTick timer is used as default timebase. This has been modified to allow implementing user-defined timebases (such as a general-purpose timer), keeping in mind that the timebase duration must be kept at 1 ms since all PPP_TIMEOUT_VALUES are defined and handled in milliseconds. This enhancement is implemented in STM32Cube V1.1 that is deployed starting from STM32CubeL0/F0/F3 and later. This modification is backward compatible with STM32Cube V1.0 implementation. Functions affecting timebase configurations are declared as `__Weak` to allow different implementations in the user file.*

3.12.2.2 System clock initialization

The clock configuration is done at the beginning of the user code. However the user can change the configuration of the clock in his own code.

Below the typical clock configuration sequence.

```
static void SystemClock_Config(void)
{
  RCC_ClkInitTypeDef RCC_ClkInitStruct;
  RCC_OscInitTypeDef RCC_OscInitStruct;
  /* Enable HSE Oscillator and activate PLL with HSE as source */
  RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSE;
  RCC_OscInitStruct.HSEState = RCC_HSE_ON;
  RCC_OscInitStruct.PLL.PLLState = RCC_PLL_ON;
  RCC_OscInitStruct.PLL.PLLSource = RCC_PLLSOURCE_HSE;
  RCC_OscInitStruct.PLL.PLLM = 25;
  RCC_OscInitStruct.PLL.PLLN = 336;
  RCC_OscInitStruct.PLL.PLLP = RCC_PLLP_DIV2;
  RCC_OscInitStruct.PLL.PLLQ = 7;
  HAL_RCC_OscConfig(&RCC_OscInitStruct);
  /* Select PLL as system clock source and configure the HCLK, PCLK1 and PCLK2 clock dividers */
  /*
  RCC_ClkInitStruct.ClockType = (RCC_CLOCKTYPE_SYSCLK | RCC_CLOCKTYPE_HCLK |
  RCC_CLOCKTYPE_PCLK1 | RCC_CLOCKTYPE_PCLK2);
  RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_PLLCLK;
  RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1;
  RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV4;
  RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV2;
  HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_5); */
}
```

3.12.2.3 HAL MSP initialization process

The peripheral initialization is done through `HAL_PPP_Init()` while the hardware resources initialization used by a peripheral (PPP) is performed during this initialization by calling MSP callback function `HAL_PPP_MspInit()`.

The MspInit callback performs the low level initialization related to the different additional hardware resources: RCC, GPIO, NVIC and DMA.

All the HAL drivers with handles include two MSP callbacks for initialization and de-initialization:


```

/**
 * @brief Initializes the PPP MSP.
 * @param hppp: PPP handle
 * @retval None */
void __weak HAL_PPP_MspInit(PPP_HandleTypeDef *hppp) {
/* NOTE : This function Should not be modified, when the callback is needed,
the HAL_PPP_MspInit could be implemented in the user file */
}
/**
 * @brief DeInitializes PPP MSP.
 * @param hppp: PPP handle
 * @retval None */
void __weak HAL_PPP_MspDeInit(PPP_HandleTypeDef *hppp) {
/* NOTE : This function Should not be modified, when the callback is needed,
the HAL_PPP_MspDeInit could be implemented in the user file */
}

```

The MSP callbacks are declared empty as weak functions in each peripheral driver. The user can use them to set the low level initialization code or omit them and use his own initialization routine.

The HAL MSP callback is implemented inside the *stm32f4xx_hal_msp.c* file in the user folders. An *stm32f4xx_hal_msp.c* file template is located in the HAL folder and should be copied to the user folder. It can be generated automatically by STM32CubeMX tool and further modified. Note that all the routines are declared as weak functions and could be overwritten or removed to use user low level initialization code.

stm32f4xx_hal_msp.c file contains the following functions:

Table 14. MSP functions

Routine	Description
void HAL_MspInit()	Global MSP initialization routine
void HAL_MspDeInit()	Global MSP de-initialization routine
void HAL_PPP_MspInit()	PPP MSP initialization routine
void HAL_PPP_MspDeInit()	PPP MSP de-initialization routine

By default, if no peripheral needs to be de-initialized during the program execution, the whole MSP initialization is done in *Hal_MspInit()* and MSP De-Initialization in the *Hal_MspDeInit()*. In this case the *HAL_PPP_MspInit()* and *HAL_PPP_MspDeInit()* are not implemented.

When one or more peripherals needs to be de-initialized in run time and the low level resources of a given peripheral need to be released and used by another peripheral, *HAL_PPP_MspDeInit()* and *HAL_PPP_MspInit()* are implemented for the concerned peripheral and other peripherals initialization and de-Initialization are kept in the global *HAL_MspInit()* and the *HAL_MspDeInit()*.

If there is nothing to be initialized by the global *HAL_MspInit()* and *HAL_MspDeInit()*, the two routines can simply be omitted.

3.12.3 HAL I/O operation process

The HAL functions with internal data processing like transmit, receive, write and read are generally provided with three data processing modes as follows:

- Polling mode
- Interrupt mode
- DMA mode

3.12.3.1 Polling mode

In Polling mode, the HAL functions return the process status when the data processing in blocking mode is complete. The operation is considered complete when the function returns the *HAL_OK* status, otherwise an error status is returned. The user can get more information through the *HAL_PPP_GetState()* function. The data processing is handled internally in a loop. A timeout (expressed in ms) is used to prevent process hanging.

The example below shows the typical Polling mode processing sequence :

```

HAL_StatusTypeDef HAL_PPP_Transmit ( PPP_HandleTypeDef * phandle, uint8_t pData,
int16_t Size, uint32_t Timeout)
{
  if((pData == NULL) || (Size == 0))
  {
    return HAL_ERROR;
  }
  (...) while (data processing is running)
  {
    if( timeout reached )
    {
      return HAL_TIMEOUT;
    }
  }
  (...)
  return HAL_OK; }

```

3.12.3.2 Interrupt mode

In Interrupt mode, the HAL function returns the process status after starting the data processing and enabling the appropriate interruption. The end of the operation is indicated by a callback declared as a weak function. It can be customized by the user to be informed in real-time about the process completion. The user can also get the process status through the *HAL_PPP_GetState()* function.

In Interrupt mode, four functions are declared in the driver:

- *HAL_PPP_Process_IT()*: launches the process
- *HAL_PPP_IRQHandler()*: global PPP peripheral interruption
- *__weak HAL_PPP_ProcessCpltCallback ()*: callback relative to the process completion.
- *__weak HAL_PPP_ProcessErrorCallback()*: callback relative to the process Error.

To use a process in Interrupt mode, *HAL_PPP_Process_IT()* is called in the user file and *HAL_PPP_IRQHandler* in *stm32f4xx_it.c*.

The *HAL_PPP_ProcessCpltCallback()* function is declared as weak function in the driver. This means that the user can declare it again in the application. The function in the driver is not modified.

An example of use is illustrated below:

main.c file:

```

UART_HandleTypeDef UartHandle;
int main(void)
{
  /* Set User Parameters */
  UartHandle.Init.BaudRate = 9600;
  UartHandle.Init.WordLength = UART_DATABITS_8;
  UartHandle.Init.StopBits = UART_STOPBITS_1;
  UartHandle.Init.Parity = UART_PARITY_NONE;
  UartHandle.Init.HwFlowCtl = UART_HWCONTROL_NONE;
  UartHandle.Init.Mode = UART_MODE_TX_RX;
  UartHandle.Init.Instance = USART3;
  HAL_UART_Init(&UartHandle);
  HAL_UART_SendIT(&UartHandle, TxBuffer, sizeof(TxBuffer));
  while (1);
}

void HAL_UART_TxCpltCallback(UART_HandleTypeDef *huart)
{
}

void HAL_UART_ErrorCallback(UART_HandleTypeDef *huart)
{
}

```

stm32f4xx_it.c file:

```
extern UART_HandleTypeDef UartHandle;
void USART3_IRQHandler(void)
{
  HAL_UART_IRQHandler(&UartHandle);
}
```

3.12.3.3 DMA mode

In DMA mode, the HAL function returns the process status after starting the data processing through the DMA and after enabling the appropriate DMA interruption. The end of the operation is indicated by a callback declared as a weak function and can be customized by the user to be informed in real-time about the process completion. The user can also get the process status through the *HAL_PPP_GetState()* function. For the DMA mode, three functions are declared in the driver:

- *HAL_PPP_Process_DMA()*: launch the process
- *HAL_PPP_DMA_IRQHandler()*: the DMA interruption used by the PPP peripheral
- *__weak HAL_PPP_ProcessCpltCallback()*: the callback relative to the process completion.
- *__weak HAL_PPP_ErrorCpltCallback()*: the callback relative to the process Error.

To use a process in DMA mode, *HAL_PPP_Process_DMA()* is called in the user file and the *HAL_PPP_DMA_IRQHandler()* is placed in the *stm32f4xx_it.c*. When DMA mode is used, the DMA initialization is done in the *HAL_PPP_MspInit()* callback. The user should also associate the DMA handle to the PPP handle. For this purpose, the handles of all the peripheral drivers that use the DMA must be declared as follows:

```
typedef struct
{
  PPP_TypeDef *Instance; /* Register base address */
  PPP_InitTypeDef Init; /* PPP communication parameters */
  HAL_StateTypeDef State; /* PPP communication state */
  (...)
  DMA_HandleTypeDef *hdma; /* associated DMA handle */
} PPP_HandleTypeDef;
```

The initialization is done as follows (UART example):

```
int main(void)
{
  /* Set User Parameters */
  UartHandle.Init.BaudRate = 9600;
  UartHandle.Init.WordLength = UART_DATABITS_8;
  UartHandle.Init.StopBits = UART_STOPBITS_1;
  UartHandle.Init.Parity = UART_PARITY_NONE;
  UartHandle.Init.HwFlowCtl = UART_HWCONTROL_NONE;
  UartHandle.Init.Mode = UART_MODE_TX_RX;
  UartHandle.Init.Instance = UART3;
  HAL_UART_Init(&UartHandle);
  (...)
}

void HAL_USART_MspInit (UART_HandleTypeDef * huart)
{
  static DMA_HandleTypeDef hdma_tx;
  static DMA_HandleTypeDef hdma_rx;
  (...)
  __HAL_LINKDMA(UartHandle, DMA_Handle_tx, hdma_tx);
  __HAL_LINKDMA(UartHandle, DMA_Handle_rx, hdma_rx);
  (...)
}
```

The *HAL_PPP_ProcessCpltCallback()* function is declared as weak function in the driver that means, the user can declare it again in the application code. The function in the driver should not be modified.

An example of use is illustrated below:

main.c file:

```
UART_HandleTypeDef UartHandle;
int main(void)
{
  /* Set User Paramaters */
  UartHandle.Init.BaudRate = 9600;
  UartHandle.Init.WordLength = UART_DATABITS_8;
  UartHandle.Init.StopBits = UART_STOPBITS_1;
  UartHandle.Init.Parity = UART_PARITY_NONE;
  UartHandle.Init.HwFlowCtl = UART_HWCONTROL_NONE;
  UartHandle.Init.Mode = UART_MODE_TX_RX; UartHandle.Instance = USART3;
  HAL_UART_Init(&UartHandle);
  HAL_UART_Send_DMA(&UartHandle, TxBuffer, sizeof(TxBuffer));
  while (1);
}

void HAL_UART_TxCpltCallback(UART_HandleTypeDef *phuart)
{
}

void HAL_UART_TxErrorCallback(UART_HandleTypeDef *phuart)
{
}
```

stm32f4xx_it.c file:

```
extern UART_HandleTypeDef UartHandle;
void DMAx_IRQHandler(void)
{
  HAL_DMA_IRQHandler(&UartHandle.DMA_Handle_tx);
}
```

HAL_USART_TxCpltCallback() and *HAL_USART_ErrorCallback()* should be linked in the *HAL_PPP_Process_DMA()* function to the DMA transfer complete callback and the DMA transfer Error callback by using the following statement:

```
HAL_PPP_Process_DMA (PPP_HandleTypeDef *hppp, Params....)
{
  (...)
  hppp->DMA_Handle->XferCpltCallback = HAL_UART_TxCpltCallback ;
  hppp->DMA_Handle->XferErrorCallback = HAL_UART_ErrorCallback ;
  (...)
}
```

3.12.4 Timeout and error management

3.12.4.1 Timeout management

The timeout is often used for the APIs that operate in Polling mode. It defines the delay during which a blocking process should wait till an error is returned. An example is provided below:

```
HAL_StatusTypeDef HAL_DMA_PollForTransfer(DMA_HandleTypeDef *hdma, uint32_t CompleteLevel, uint32_t Timeout)
```

The timeout possible values are the following:

Table 15. Timeout values

Timeout value	Description
0	No poll : Immediate process check and exit
1 ... (HAL_MAX_DELAY -1) ⁽¹⁾	Timeout in ms
HAL_MAX_DELAY	Infinite poll till process is successful

1. *HAL_MAX_DELAY* is defined in the *stm32f4xx_hal_def.h* as 0xFFFFFFFF

However, in some cases, a fixed timeout is used for system peripherals or internal HAL driver processes. In these cases, the timeout has the same meaning and is used in the same way, except when it is defined locally in the drivers and cannot be modified or introduced as an argument in the user application.

Example of fixed timeout:

```
#define LOCAL_PROCESS_TIMEOUT 100
HAL_StatusTypeDef HAL_PPP_Process(PPP_HandleTypeDef *hppp)
{
    (...)
    tickstart = HAL_GetTick();
    (...)
    while (ProcessOngoing)
    {
        (...)
        if ((HAL_GetTick() - tickstart) > LOCAL_PROCESS_TIMEOUT)
        {
            hppp->ErrorCode |= HAL_PPP_ERROR_TIMEOUT;
            hppp->State= HAL_PPP_STATE_READY;
            return HAL_ERROR;
        }
    }
    (...)
}
```

The following example shows how to use the timeout inside the polling functions:

```
HAL_StatusTypeDef HAL_PPP_Poll(PPP_HandleTypeDef *hppp, uint32_t Timeout)
{
    (...)
    tickstart = HAL_GetTick();
    (...)
    while (ProcessOngoing)
    {
        (...)
        if (((HAL_GetTick() - tickstart) > Timeout) || (Timeout == 0))
        {
            hppp->ErrorCode |= HAL_PPP_ERROR_TIMEOUT;
            hppp->State= HAL_PPP_STATE_READY;
            return HAL_ERROR;
        }
    }
    (...)
}
```

3.12.4.2 Error management

The HAL drivers implement a check on the following items:

- Valid parameters: for some process the used parameters should be valid and already defined, otherwise the system may crash or go into an undefined state. These critical parameters are checked before being used (see example below).

```
HAL_StatusTypeDef HAL_PPP_Process(PPP_HandleTypeDef* hppp, uint32_t *pdata, uint32 Size)
{
    if ((pdata == NULL) || (Size == 0))
    {
        return HAL_ERROR;
    }
}
```

- Valid handle: the PPP peripheral handle is the most important argument since it keeps the PPP driver vital parameters. It is always checked in the beginning of the *HAL_PPP_Init()* function.

```
HAL_StatusTypeDef HAL_PPP_Init(PPP_HandleTypeDef* hppp)
{
    if (hppp == NULL) //the handle should be already allocated
    {
        return HAL_ERROR;
    }
}
```

- Timeout error: the following statement is used when a timeout error occurs:

```
while (Process ongoing)
{
  timeout = HAL_GetTick() + Timeout; while (data processing is running)
  {
    if(timeout) { return HAL_TIMEOUT;
  }
}
```

When an error occurs during a peripheral process, *HAL_PPP_Process ()* returns with a *HAL_ERROR* status. The HAL PPP driver implements the *HAL_PPP_GetError ()* to allow retrieving the origin of the error.

```
HAL_PPP_ErrorTypeDef HAL_PPP_GetError (PPP_HandleTypeDef *hPPP);
```

In all peripheral handles, a *HAL_PPP_ErrorTypeDef* is defined and used to store the last error code.

```
typedef struct
{
  PPP_TypeDef * Instance; /* PPP registers base address */
  PPP_InitTypeDef Init; /* PPP initialization parameters */
  HAL_LockTypeDef Lock; /* PPP locking object */
  __IO HAL_PPP_StateTypeDef State; /* PPP state */
  __IO HAL_PPP_ErrorTypeDef ErrorCode; /* PPP Error code */
  (...)
  /* PPP specific parameters */
}
PPP_HandleTypeDef;
```

The error state and the peripheral global state are always updated before returning an error:

```
PPP->State = HAL_PPP_READY; /* Set the peripheral ready */
PPP->ErrorCode = HAL_ERRORCODE ; /* Set the error code */
__HAL_UNLOCK(PPP) ; /* Unlock the PPP resources */
return HAL_ERROR; /*return with HAL error */
```

HAL_PPP_GetError () must be used in interrupt mode in the error callback:

```
void HAL_PPP_ProcessCpltCallback(PPP_HandleTypeDef *hspi)
{
  ErrorCode = HAL_PPP_GetError (hPPP); /* retrieve error code */
}
```

3.12.4.3 Run-time checking

The HAL implements run-time failure detection by checking the input values of all HAL driver functions. The run-time checking is achieved by using an *assert_param* macro. This macro is used in all the HAL driver functions which have an input parameter. It allows verifying that the input value lies within the parameter allowed values.

To enable the run-time checking, use the *assert_param* macro, and leave the define ***USE_FULL_ASSERT*** uncommented in *stm32f4xx_hal_conf.h* file.

```
void HAL_UART_Init(UART_HandleTypeDef *huart)
{
  (...) /* Check the parameters */
  assert_param(IS_UART_INSTANCE(huart->Instance));
  assert_param(IS_UART_BAUDRATE(huart->Init.BaudRate));
  assert_param(IS_UART_WORD_LENGTH(huart->Init.WordLength));
  assert_param(IS_UART_STOPBITS(huart->Init.StopBits));
  assert_param(IS_UART_PARITY(huart->Init.Parity));
  assert_param(IS_UART_MODE(huart->Init.Mode));
  assert_param(IS_UART_HARDWARE_FLOW_CONTROL(huart->Init.HwFlowCtl));
  (...)
}
```

```
/** @defgroup UART_Word_Length *
@{
*/
#define UART_WORDLENGTH_8B ((uint32_t)0x00000000)
#define UART_WORDLENGTH_9B ((uint32_t)USART_CR1_M)
#define IS_UART_WORD_LENGTH(LENGTH) (((LENGTH) == UART_WORDLENGTH_8B) || \
  \ ((LENGTH) == UART_WORDLENGTH_9B))
```

If the expression passed to the `assert_param` macro is false, the `assert_failed` function is called and returns the name of the source file and the source line number of the call that failed. If the expression is true, no value is returned.

The `assert_param` macro is implemented in `stm32f4xx_hal_conf.h`:

```
/* Exported macro -----*/
#ifdef USE_FULL_ASSERT
/**
 * @brief The assert_param macro is used for function's parameters check.
 * @param expr: If expr is false, it calls assert_failed function
 * which reports the name of the source file and the source
 * line number of the call that failed.
 * If expr is true, it returns no value.
 * @retval None */
#define assert_param(expr) ((expr)?(void)0:assert_failed((__FILE__, __LINE__))
/* Exported functions -----*/
void assert_failed(uint8_t* file, uint32_t line);
#else
#define assert_param(expr) ((void)0)
#endif /* USE_FULL_ASSERT */
```

The `assert_failed` function is implemented in the `main.c` file or in any other user C file:

```
#ifdef USE_FULL_ASSERT /**
 * @brief Reports the name of the source file and the source line number
 * where the assert_param error has occurred.
 * @param file: pointer to the source file name
 * @param line: assert_param error line source number
 * @retval None */
void assert_failed(uint8_t* file, uint32_t line)
{
/* User can add his own implementation to report the file name and line number,
ex: printf("Wrong parameters value: file %s on line %d\r\n", file, line) */
/* Infinite loop */
while (1)
{
}
}
```

Attention: *Because of the overhead run-time checking introduces, it is recommended to use it during application code development and debugging, and to remove it from the final application to improve code size and speed.*

4 Overview of low-layer drivers

The low-layer (LL) drivers are designed to offer a fast light-weight expert-oriented layer which is closer to the hardware than the HAL. Contrary to the HAL, LL APIs are not provided for peripherals where optimized access is not a key feature, or those requiring heavy software configuration and/or complex upper-level stack (such as USB).

The LL drivers feature:

- A set of functions to initialize peripheral main features according to the parameters specified in data structures
- A set of functions used to fill initialization data structures with the reset values of each field
- Functions to perform peripheral de-initialization (peripheral registers restored to their default values)
- A set of inline functions for direct and atomic register access
- Full independence from HAL since LL drivers can be used either in standalone mode (without HAL drivers) or in mixed mode (with HAL drivers)
- Full coverage of the supported peripheral features

The low-layer drivers provide hardware services based on the available features of the STM32 peripherals. These services reflect exactly the hardware capabilities and provide one-shot operations that must be called following the programming model described in the microcontroller line reference manual. As a result, the LL services do not implement any processing and do not require any additional memory resources to save their states, counter or data pointers: all the operations are performed by changing the associated peripheral registers content.

4.1 Low-layer files

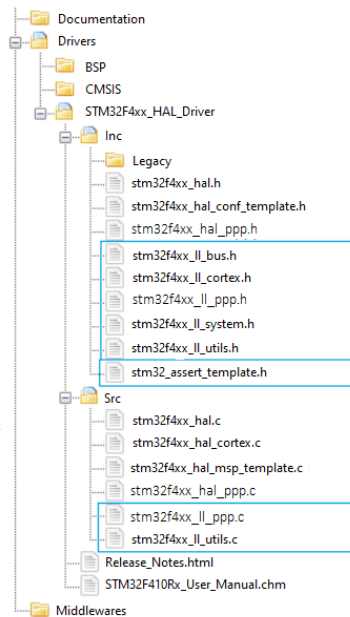
The low-layer drivers are built around header/C files (one per each supported peripheral) plus five header files for some System and Cortex related features.

Table 16. LL driver files

File	Description
<i>stm32f4xx_ll_bus.h</i>	This is the h-source file for core bus control and peripheral clock activation and deactivation <i>Example: LL_AHB2_GRP1_EnableClock</i>
<i>stm32f4xx_ll_pmp.h/.c</i>	stm32f4xx_ll_pmp.c provides peripheral initialization functions such as LL_PMP_Init(), LL_PMP_StructInit(), LL_PMP_DeInit(). All the other APIs are defined within stm32f4xx_ll_pmp.h file. The low-layer PMP driver is a standalone module. To use it, the application must include it in the stm32f4xx_ll_pmp.h file.
<i>stm32f4xx_ll_cortex.h</i>	Cortex-M related register operation APIs including the SysTick, Low power (such as LL_SYSTICK_XXXX and LL_LPM_XXXX "Low Power Mode")
<i>stm32f4xx_ll_utils.h/.c</i>	This file covers the generic APIs: • Read of device unique ID and electronic signature • Timebase and delay management • System clock configuration.
<i>stm32f4xx_ll_system.h</i>	System related operations. <i>Example: LL_SYSCFG_XXX, LL_DBGMCU_XXX, LL_FLASH_XXX, and LL_VREBUF_XXX</i>
<i>stm32_assert_template.h</i>	Template file allowing to define the assert_param macro, that is used when run-time checking is enabled. This file is required only when the LL drivers are used in standalone mode (without calling the HAL APIs). It should be copied to the application folder and renamed to stm32_assert.h.

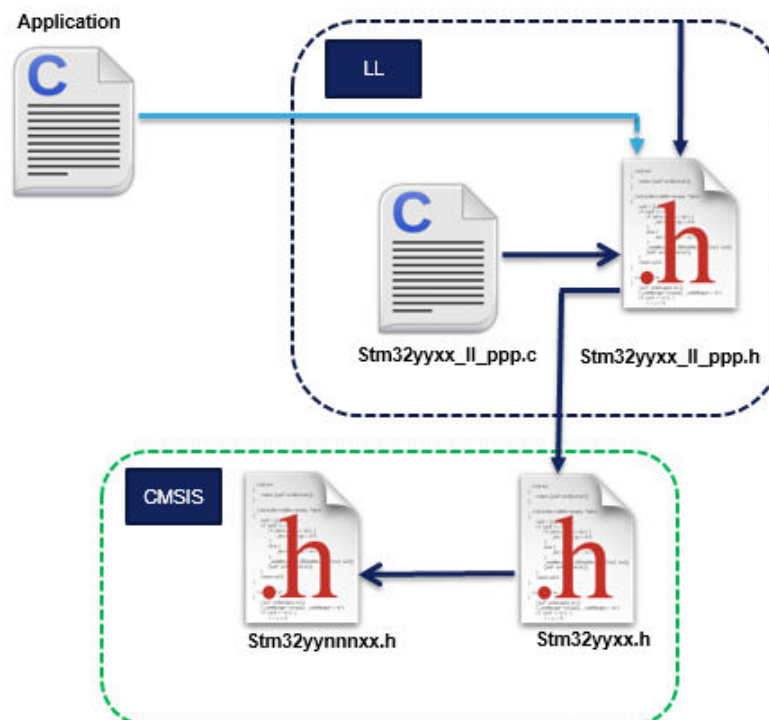
Note: *There is no configuration file for the LL drivers.*

The low-layer files are located in the same HAL driver folder.

Figure 8. Low-layer driver folders


In general, low-layer drivers include only the STM32 CMSIS device file.

```
#include "stm32yyxx.h"
```

Figure 9. Low-layer driver CMSIS files


Application files have to include only the used low-layer driver header files.

4.2 Overview of low-layer APIs and naming rules

4.2.1 Peripheral initialization functions

The LL drivers offer three sets of initialization functions. They are defined in `stm32f4xx_ll_ppp.c` file:

- Functions to initialize peripheral main features according to the parameters specified in data structures
- A set of functions used to fill initialization data structures with the reset values of each field
- Function for peripheral de-initialization (peripheral registers restored to their default values)

The definition of these LL initialization functions and associated resources (structure, literals and prototypes) is conditioned by a compilation switch: `USE_FULL_LL_DRIVER`. To use these functions, this switch must be added in the toolchain compiler preprocessor or to any generic header file which is processed before the LL drivers.

The below table shows the list of the common functions provided for all the supported peripherals:

Table 17. Common peripheral initialization functions

Functions	Return Type	Parameters	Description
LL_PPP_Init	ErrorStatus	• <i>PPP_TypeDef* PPPx</i> • <i>LL_PPP_InitTypeDef* PPP_InitStruct</i>	Initializes the peripheral main features according to the parameters specified in <code>PPP_InitStruct</code>. Example: <code>LL_USART_Init(USART_TypeDef *USARTx, LL_USART_InitTypeDef *USART_InitStruct)</code>
LL_PPP_StructInit	void	• <i>LL_PPP_InitTypeDef* PPP_InitStruct</i>	Fills each <code>PPP_InitStruct</code> member with its default value. Example: <code>LL_USART_StructInit(LL_USART_InitTypeDef *USART_InitStruct)</code>
LL_PPP_DeInit	ErrorStatus	• <i>PPP_TypeDef* PPPx</i>	De-initializes the peripheral registers, that is restore them to their default reset values. Example: <code>LL_USART_DeInit(USART_TypeDef *USARTx)</code>

Additional functions are available for some peripherals (refer to [Table 18. Optional peripheral initialization functions](#)).

Table 18. Optional peripheral initialization functions

Functions	Return Type	Parameters	Examples
LL_PPP{CATEGORY}_Init	ErrorStatus	• <i>PPP_TypeDef* PPPx</i> • <i>LL_PPP{CATEGORY}_InitTypeDef* PPP{CATEGORY}_InitStruct</i>	Initializes peripheral features according to the parameters specified in <code>PPP_InitStruct</code>. Example: <code>LL_ADC_INJ_Init(ADC_TypeDef *ADCx, LL_ADC_INJ_InitTypeDef *ADC_INJ_InitStruct)</code> <code>LL_RTC_TIME_Init(RTC_TypeDef *RTCx, uint32_t RTC_Format, LL_RTC_TimeTypeDef *RTC_TimeStruct)</code> <code>LL_RTC_DATE_Init(RTC_TypeDef *RTCx, uint32_t RTC_Format, LL_RTC_DateTypeDef *RTC_DateStruct)</code> <code>LL_TIM_IC_Init(TIM_TypeDef* TIMx, uint32_t Channel, LL_TIM_IC_InitTypeDef* TIM_IC_InitStruct)</code> <code>LL_TIM_ENCODER_Init(TIM_TypeDef* TIMx, LL_TIM_ENCODER_InitTypeDef* TIM_EncoderInitStruct)</code>
LL_PPP{CATEGORY}_StructInit	void	<i>LL_PPP{CATEGORY}_InitTypeDef* PPP{CATEGORY}_InitStruct</i>	Fills each <code>PPP{CATEGORY}_InitStruct</code> member with its default value. Example: <code>LL_ADC_INJ_StructInit(LL_ADC_INJ_InitTypeDef *ADC_INJ_InitStruct)</code>
LL_PPP_CommonInit	ErrorStatus	• <i>PPP_TypeDef* PPPx</i>	Initializes the common features shared between different instances of the same peripheral.

Functions	Return Type	Parameters	Examples
		<i>LL_PPP_CommonInitTypeDef*</i> <i>PPP_CommonInitStruct</i>	Example: <i>LL_ADC_CommonInit(ADC_Common_TypeDef *ADCxy_COMMON, LL_ADC_CommonInitTypeDef *ADC_CommonInitStruct)</i>
<i>LL_PPP_CommonStructInit</i>	<i>void</i>	<i>LL_PPP_CommonInitTypeDef*</i> <i>PPP_CommonInitStruct</i>	Fills each <i>PPP_CommonInitStruct</i> member with its default value Example: <i>LL_ADC_CommonStructInit(LL_ADC_CommonInitTypeDef *ADC_CommonInitStruct)</i>
<i>LL_PPP_ClockInit</i>	<i>ErrorStatus</i>	<i>PPP_TypeDef* PPPx</i> <i>LL_PPP_ClockInitTypeDef*</i> <i>PPP_ClockInitStruct</i>	Initializes the peripheral clock configuration in synchronous mode. Example: <i>LL_USART_ClockInit(USART_TypeDef *USARTx, LL_USART_ClockInitTypeDef *USART_ClockInitStruct)</i>
<i>LL_PPP_ClockStructInit</i>	<i>void</i>	<i>LL_PPP_ClockInitTypeDef*</i> <i>PPP_ClockInitStruct</i>	Fills each <i>PPP_ClockInitStruct</i> member with its default value Example: <i>LL_USART_ClockStructInit(LL_USART_ClockInitTypeDef *USART_ClockInitStruct)</i>

4.2.1.1 Run-time checking

Like HAL drivers, LL initialization functions implement run-time failure detection by checking the input values of all LL driver functions. For more details, refer to [Section 3.12.4.3 Run-time checking](#).

When using the LL drivers in standalone mode (without calling HAL functions), the following actions are required to use run-time checking:

1. Copy *stm32_assert_template.h* to the application folder and rename it to *stm32_assert.h*. This file defines the *assert_param* macro which is used when run-time checking is enabled.
2. Include *stm32_assert.h* file within the application main header file.
3. Add the *USE_FULL_ASSERT* compilation switch in the toolchain compiler preprocessor or in any generic header file which is processed before the *stm32_assert.h* driver.

Note: *Run-time checking is not available for LL inline functions.*

4.2.2 Peripheral register-level configuration functions

On top of the peripheral initialization functions, the LL drivers offer a set of inline functions for direct atomic register access. Their format is as follows:

```
__STATIC_INLINE return_type LL_PPP_Function (PPP_TypeDef *PPPx, args)
```

The “Function” naming is defined depending to the action category:

- **Specific Interrupt, DMA request and status flags management:** Set/Get/Clear/Enable/Disable flags on interrupt and status registers

Table 19. Specific Interrupt, DMA request and status flags management

Name	Examples
<i>LL_PPP_{CATEGORY}_ActionItem_BITNAME</i> <i>LL_PPP{CATEGORY}_IsItem_BITNAME_Action</i>	<i>LL_RCC_IsActiveFlag_LSIRDY</i> <i>LL_RCC_IsActiveFlag_FWRST()</i> <i>LL_ADC_ClearFlag_EOC(ADC1)</i> <i>LL_DMA_ClearFlag_TCx(DMA_TypeDef* DMAx)</i>

Table 20. Available function formats

Item	Action	Format
Flag	Get	<i>LL_PPP_IsActiveFlag_BITNAME</i>
	Clear	<i>LL_PPP_ClearFlag_BITNAME</i>
Interrupts	Enable	<i>LL_PPP_EnableIT_BITNAME</i>

Item	Action	Format
Interrupts	Disable	<code>LL_PPP_DisableIT_BITNAME</code>
	Get	<code>LL_PPP_IsEnabledIT_BITNAME</code>
DMA	Enable	<code>LL_PPP_EnableDMAReq_BITNAME</code>
	Disable	<code>LL_PPP_DisableDMAReq_BITNAME</code>
	Get	<code>LL_PPP_IsEnabledDMAReq_BITNAME</code>

Note: *BITNAME refers to the peripheral register bit name as described in the product line reference manual.*

- **Peripheral clock activation/deactivation management:** Enable/Disable/Reset a peripheral clock

Table 21. Peripheral clock activation/deactivation management

Name	Examples
<code>LL_BUS_GRPx_ActionClock{Mode}</code>	• <code>LL_AHB2_GRP1_EnableClock (LL_AHB2_GRP1_PERIPH_GPIOA LL_AHB2_GRP1_PERIPH_GPIOB)</code> • <code>LL_APB1_GRP1_EnableClockSleep (LL_APB1_GRP1_PERIPH_DAC1)</code>

Note: *'x' corresponds to the group index and refers to the index of the modified register on a given bus. 'bus' corresponds to the bus name.*

- **Peripheral activation/deactivation management :** Enable/disable a peripheral or activate/deactivate specific peripheral features

Table 22. Peripheral activation/deactivation management

Name	Examples
<code>LL_PPP[_CATEGORY]_Action{Item}</code> <code>LL_PPP[_CATEGORY]_IsItemAction</code>	• <code>LL_ADC_Enable ()</code> • <code>LL_ADC_StartCalibration();</code> • <code>LL_ADC_IsCalibrationOnGoing;</code> • <code>LL_RCC_HSI_Enable ()</code> • <code>LL_RCC_HSI_IsReady()</code>

- **Peripheral configuration management :** Set/get a peripheral configuration settings

Table 23. Peripheral configuration management

Name	Examples
<code>LL_PPP[_CATEGORY]_Set{ or Get}ConfigItem</code>	<code>LL_USART_SetBaudRate (USART2, Clock, LL_USART_BAUDRATE_9600)</code>

- **Peripheral register management :** Write/read the content of a register/retrun DMA relative register address

Table 24. Peripheral register management

Name
<code>LL_PPP_WriteReg(__INSTANCE__, __REG__, __VALUE__)</code>
<code>LL_PPP_ReadReg(__INSTANCE__, __REG__)</code>
<code>LL_PPP_DMA_GetRegAddr (PPP_TypeDef *PPPx,{Sub Instance if any ex: Channel} , {uint32_t Propriety})</code>

Note: *The Propriety is a variable used to identify the DMA transfer direction or the data register type.*

5 Cohabiting of HAL and LL

The low-layer APIs are designed to be used in standalone mode or combined with the HAL. They cannot be automatically used with the HAL for the same peripheral instance. If you use the LL APIs for a specific instance, you can still use the HAL APIs for other instances. Be careful that the low-layer APIs might overwrite some registers which content is mirrored in the HAL handles.

5.1 Low-layer driver used in Standalone mode

The low-layer APIs can be used without calling the HAL driver services. This is done by simply including `stm32f4xx_ll_ppp.h` in the application files. The LL APIs for a given peripheral are called by executing the same sequence as the one recommended by the programming model in the corresponding product line reference manual. In this case the HAL drivers associated to the used peripheral can be removed from the workspace. However the [STM32CubeF4](#) framework should be used in the same way as in the HAL drivers case which means that System file, startup file and CMSIS should always be used.

Note: When the BSP drivers are included, the used HAL drivers associated with the BSP functions drivers should be included in the workspace, even if they are not used by the application layer.

5.2 Mixed use of low-layer APIs and HAL drivers

In this case the low-layer APIs are used in conjunction with the HAL drivers to achieve direct and register level based operations.

Mixed use is allowed, however some consideration should be taken into account:

- It is recommended to avoid using simultaneously the HAL APIs and the combination of low-layer APIs for a given peripheral instance. If this is the case, one or more private fields in the HAL PPP handle structure should be updated accordingly.
- For operations and processes that do not alter the handle fields including the initialization structure, the HAL driver APIs and the low-layer services can be used together for the same peripheral instance.
- The low-layer drivers can be used without any restriction with all the HAL drivers that are not based on handle objects (RCC, common HAL, Flash and GPIO).

Several examples showing how to use HAL and LL in the same application are provided within `stm32f4` firmware package (refer to `Examples_MIX` projects).

- Note:*
1. When the HAL `Init/DelInit` APIs are not used and are replaced by the low-layer macros, the `InitMsp()` functions are not called and the MSP initialization should be done in the user application.
 2. When process APIs are not used and the corresponding function is performed through the low-layer APIs, the callbacks are not called and post processing or error management should be done by the user application.
 3. When the LL APIs is used for process operations, the IRQ handler HAL APIs cannot be called and the IRQ should be implemented by the user application. Each LL driver implements the macros needed to read and clear the associated interrupt flags.

6 HAL System Driver

6.1 HAL Firmware driver API description

The following section lists the various functions of the HAL library.

6.1.1 How to use this driver

The common HAL driver contains a set of generic and common APIs that can be used by the PPP peripheral drivers and the user to start using the HAL.

The HAL contains two APIs' categories:

- Common HAL APIs
- Services HAL APIs

6.1.2 Initialization and Configuration functions

This section provides functions allowing to:

- Initializes the Flash interface the NVIC allocation and initial clock configuration. It initializes the systick also when timeout is needed and the backup domain when enabled.
- De-Initializes common part of the HAL.
- Configure the time base source to have 1ms time base with a dedicated Tick interrupt priority.
 - SysTick timer is used by default as source of time base, but user can eventually implement his proper time base source (a general purpose timer for example or other time source), keeping in mind that Time base duration should be kept 1ms since PPP_TIMEOUT_VALUES are defined and handled in milliseconds basis.
 - Time base configuration function (HAL_InitTick ()) is called automatically at the beginning of the program after reset by HAL_Init() or at any time when clock is configured, by HAL_RCC_ClockConfig().
 - Source of time base is configured to generate interrupts at regular time intervals. Care must be taken if HAL_Delay() is called from a peripheral ISR process, the Tick interrupt line must have higher priority (numerically lower) than the peripheral interrupt. Otherwise the caller ISR process will be blocked.
 - functions affecting time base configurations are declared as __weak to make override possible in case of other implementations in user file.

This section contains the following APIs:

- **HAL_Init()**
- **HAL_DeInit()**
- **HAL_MspInit()**
- **HAL_MspDeInit()**
- **HAL_InitTick()**

6.1.3 HAL Control functions

This section provides functions allowing to:

- Provide a tick value in millisecond
- Provide a blocking delay in millisecond
- Suspend the time base source interrupt
- Resume the time base source interrupt
- Get the HAL API driver version
- Get the device identifier
- Get the device revision identifier
- Enable/Disable Debug module during SLEEP mode
- Enable/Disable Debug module during STOP mode
- Enable/Disable Debug module during STANDBY mode

This section contains the following APIs:

- [*HAL_IncTick\(\)*](#)
- [*HAL_GetTick\(\)*](#)
- [*HAL_GetTickPrio\(\)*](#)
- [*HAL_SetTickFreq\(\)*](#)
- [*HAL_GetTickFreq\(\)*](#)
- [*HAL_Delay\(\)*](#)
- [*HAL_SuspendTick\(\)*](#)
- [*HAL_ResumeTick\(\)*](#)
- [*HAL_GetHalVersion\(\)*](#)
- [*HAL_GetREVID\(\)*](#)
- [*HAL_GetDEVID\(\)*](#)
- [*HAL_DBGMCU_EnableDBGSleepMode\(\)*](#)
- [*HAL_DBGMCU_DisableDBGSleepMode\(\)*](#)
- [*HAL_DBGMCU_EnableDBGStopMode\(\)*](#)
- [*HAL_DBGMCU_DisableDBGStopMode\(\)*](#)
- [*HAL_DBGMCU_EnableDBGStandbyMode\(\)*](#)
- [*HAL_DBGMCU_DisableDBGStandbyMode\(\)*](#)
- [*HAL_EnableCompensationCell\(\)*](#)
- [*HAL_DisableCompensationCell\(\)*](#)
- [*HAL_GetUIDw0\(\)*](#)
- [*HAL_GetUIDw1\(\)*](#)
- [*HAL_GetUIDw2\(\)*](#)
- [*HAL_EnableMemorySwappingBank\(\)*](#)
- [*HAL_DisableMemorySwappingBank\(\)*](#)

6.1.4 Detailed description of functions

HAL_Init

Function name

HAL_StatusTypeDef HAL_Init (void)

Function description

This function is used to initialize the HAL Library; it must be the first instruction to be executed in the main program (before to call any other HAL function), it performs the following: Configure the Flash prefetch, instruction and Data caches.

Return values

- **HAL:** status

Notes

- SysTick is used as time base for the HAL_Delay() function, the application need to ensure that the SysTick time base is always set to 1 millisecond to have correct HAL operation.

HAL_DeInit

Function name

HAL_StatusTypeDef HAL_DeInit (void)

Function description

This function de-Initializes common part of the HAL and stops the systick.

Return values

- **HAL:** status

HAL_MspInit

Function name

void HAL_MspInit (void)

Function description

Initialize the MSP.

Return values

- **None:**

HAL_MspDeInit

Function name

void HAL_MspDeInit (void)

Function description

DeInitializes the MSP.

Return values

- **None:**

HAL_InitTick

Function name

HAL_StatusTypeDef HAL_InitTick (uint32_t TickPriority)

Function description

This function configures the source of the time base.

Parameters

- **TickPriority:** Tick interrupt priority.

Return values

- **HAL:** status

Notes

- This function is called automatically at the beginning of program after reset by HAL_Init() or at any time when clock is reconfigured by HAL_RCC_ClockConfig().
- In the default implementation, SysTick timer is the source of time base. It is used to generate interrupts at regular time intervals. Care must be taken if HAL_Delay() is called from a peripheral ISR process, The SysTick interrupt must have higher priority (numerically lower) than the peripheral interrupt. Otherwise the caller ISR process will be blocked. The function is declared as __weak to be overwritten in case of other implementation in user file.

HAL_IncTick

Function name

void HAL_IncTick (void)

Function description

This function is called to increment a global variable "uwTick" used as application time base.

Return values

- **None:**

Notes

- In the default implementation, this variable is incremented each 1ms in SysTick ISR.
- This function is declared as `__weak` to be overwritten in case of other implementations in user file.

HAL_Delay

Function name

void HAL_Delay (uint32_t Delay)

Function description

This function provides minimum delay (in milliseconds) based on variable incremented.

Parameters

- **Delay:** specifies the delay time length, in milliseconds.

Return values

- **None:**

Notes

- In the default implementation , SysTick timer is the source of time base. It is used to generate interrupts at regular time intervals where uwTick is incremented.
- This function is declared as `__weak` to be overwritten in case of other implementations in user file.

HAL_GetTick

Function name

uint32_t HAL_GetTick (void)

Function description

Provides a tick value in millisecond.

Return values

- **tick:** value

Notes

- This function is declared as `__weak` to be overwritten in case of other implementations in user file.

HAL_GetTickPrio

Function name

uint32_t HAL_GetTickPrio (void)

Function description

This function returns a tick priority.

Return values

- **tick:** priority

HAL_SetTickFreq

Function name

HAL_StatusTypeDef HAL_SetTickFreq (HAL_TickFreqTypeDef Freq)

Function description

Set new tick Freq.

Return values

- **Status:**

HAL_GetTickFreq

Function name

HAL_TickFreqTypeDef HAL_GetTickFreq (void)

Function description

Return tick frequency.

Return values

- **tick:** period in Hz

HAL_SuspendTick

Function name

void HAL_SuspendTick (void)

Function description

Suspend Tick increment.

Return values

- **None:**

Notes

- In the default implementation , SysTick timer is the source of time base. It is used to generate interrupts at regular time intervals. Once HAL_SuspendTick() is called, the SysTick interrupt will be disabled and so Tick increment is suspended.
- This function is declared as __weak to be overwritten in case of other implementations in user file.

HAL_ResumeTick

Function name

void HAL_ResumeTick (void)

Function description

Resume Tick increment.

Return values

- **None:**

Notes

- In the default implementation , SysTick timer is the source of time base. It is used to generate interrupts at regular time intervals. Once HAL_ResumeTick() is called, the SysTick interrupt will be enabled and so Tick increment is resumed.
- This function is declared as __weak to be overwritten in case of other implementations in user file.

HAL_GetHalVersion

Function name

uint32_t HAL_GetHalVersion (void)

Function description

Returns the HAL revision.

Return values

- **version:** : 0xXYZR (8bits for each decimal, R for RC)

HAL_GetREVID

Function name

uint32_t HAL_GetREVID (void)

Function description

Returns the device revision identifier.

Return values

- **Device:** revision identifier

HAL_GetDEVID

Function name

uint32_t HAL_GetDEVID (void)

Function description

Returns the device identifier.

Return values

- **Device:** identifier

HAL_DBGMCU_EnableDBGSleepMode

Function name

void HAL_DBGMCU_EnableDBGSleepMode (void)

Function description

Enable the Debug Module during SLEEP mode.

Return values

- **None:**

HAL_DBGMCU_DisableDBGSleepMode

Function name

void HAL_DBGMCU_DisableDBGSleepMode (void)

Function description

Disable the Debug Module during SLEEP mode.

Return values

- **None:**

HAL_DBGMCU_EnableDBGStopMode

Function name

void HAL_DBGMCU_EnableDBGStopMode (void)

Function description

Enable the Debug Module during STOP mode.

Return values

- **None:**

HAL_DBGMCU_DisableDBGStopMode

Function name

void HAL_DBGMCU_DisableDBGStopMode (void)

Function description

Disable the Debug Module during STOP mode.

Return values

- **None:**

HAL_DBGMCU_EnableDBGStandbyMode

Function name

void HAL_DBGMCU_EnableDBGStandbyMode (void)

Function description

Enable the Debug Module during STANDBY mode.

Return values

- **None:**

HAL_DBGMCU_DisableDBGStandbyMode

Function name

void HAL_DBGMCU_DisableDBGStandbyMode (void)

Function description

Disable the Debug Module during STANDBY mode.

Return values

- **None:**

HAL_EnableCompensationCell

Function name

void HAL_EnableCompensationCell (void)

Function description

Enables the I/O Compensation Cell.

Return values

- **None:**

Notes

- The I/O compensation cell can be used only when the device supply voltage ranges from 2.4 to 3.6 V.

HAL_DisableCompensationCell

Function name

void HAL_DisableCompensationCell (void)

Function description

Power-down the I/O Compensation Cell.

Return values

- **None:**

Notes

- The I/O compensation cell can be used only when the device supply voltage ranges from 2.4 to 3.6 V.

HAL_GetUIDw0

Function name

uint32_t HAL_GetUIDw0 (void)

Function description

Returns first word of the unique device identifier (UID based on 96 bits)

Return values

- Device:** identifier

HAL_GetUIDw1

Function name

uint32_t HAL_GetUIDw1 (void)

Function description

Returns second word of the unique device identifier (UID based on 96 bits)

Return values

- Device:** identifier

HAL_GetUIDw2

Function name

uint32_t HAL_GetUIDw2 (void)

Function description

Returns third word of the unique device identifier (UID based on 96 bits)

Return values

- Device:** identifier

HAL_EnableMemorySwappingBank

Function name

void HAL_EnableMemorySwappingBank (void)

Function description

Enables the Internal FLASH Bank Swapping.

Return values

- None:**

Notes

- This function can be used only for STM32F42xxx/43xxx/469xx/479xx devices.
- Flash Bank2 mapped at 0x08000000 (and aliased @0x00000000) and Flash Bank1 mapped at 0x08100000 (and aliased at 0x00100000)

HAL_DisableMemorySwappingBank

Function name

void HAL_DisableMemorySwappingBank (void)

Function description

Disables the Internal FLASH Bank Swapping.

Return values

- **None:**

Notes

- This function can be used only for STM32F42xxx/43xxx/469xx/479xx devices.
- The default state : Flash Bank1 mapped at 0x08000000 (and aliased @0x00000000) and Flash Bank2 mapped at 0x08100000 (and aliased at 0x00100000)

6.2 HAL Firmware driver defines

The following section lists the various define and macros of the module.

6.2.1 HAL

HAL

ETH States

HAL_ETH_STATE_RESET

Peripheral not yet Initialized or disabled

HAL_ETH_STATE_READY

Peripheral Communication started

HAL_ETH_STATE_BUSY

an internal process is ongoing

HAL_ETH_STATE_STARTED

an internal process is started

HAL_ETH_STATE_ERROR

Error State

HAL Exported Macros

__HAL_DBGMCU_FREEZE_TIM2

__HAL_DBGMCU_FREEZE_TIM3

__HAL_DBGMCU_FREEZE_TIM4

__HAL_DBGMCU_FREEZE_TIM5

__HAL_DBGMCU_FREEZE_TIM6

__HAL_DBGMCU_FREEZE_TIM7

__HAL_DBGMCU_FREEZE_TIM12

__HAL_DBGMCU_FREEZE_TIM13

__HAL_DBGMCU_FREEZE_TIM14

__HAL_DBGMCU_FREEZE_RTC

__HAL_DBGMCU_FREEZE_WWDG

__HAL_DBGMCU_FREEZE_IWDG
__HAL_DBGMCU_FREEZE_I2C1_TIMEOUT
__HAL_DBGMCU_FREEZE_I2C2_TIMEOUT
__HAL_DBGMCU_FREEZE_I2C3_TIMEOUT
__HAL_DBGMCU_FREEZE_CAN1
__HAL_DBGMCU_FREEZE_CAN2
__HAL_DBGMCU_FREEZE_TIM1
__HAL_DBGMCU_FREEZE_TIM8
__HAL_DBGMCU_FREEZE_TIM9
__HAL_DBGMCU_FREEZE_TIM10
__HAL_DBGMCU_FREEZE_TIM11
__HAL_DBGMCU_UNFREEZE_TIM2
__HAL_DBGMCU_UNFREEZE_TIM3
__HAL_DBGMCU_UNFREEZE_TIM4
__HAL_DBGMCU_UNFREEZE_TIM5
__HAL_DBGMCU_UNFREEZE_TIM6
__HAL_DBGMCU_UNFREEZE_TIM7
__HAL_DBGMCU_UNFREEZE_TIM12
__HAL_DBGMCU_UNFREEZE_TIM13
__HAL_DBGMCU_UNFREEZE_TIM14
__HAL_DBGMCU_UNFREEZE_RTC
__HAL_DBGMCU_UNFREEZE_WWDG
__HAL_DBGMCU_UNFREEZE_IWDG
__HAL_DBGMCU_UNFREEZE_I2C1_TIMEOUT
__HAL_DBGMCU_UNFREEZE_I2C2_TIMEOUT
__HAL_DBGMCU_UNFREEZE_I2C3_TIMEOUT
__HAL_DBGMCU_UNFREEZE_CAN1
__HAL_DBGMCU_UNFREEZE_CAN2

__HAL_DBGMCU_UNFREEZE_TIM1

__HAL_DBGMCU_UNFREEZE_TIM8

__HAL_DBGMCU_UNFREEZE_TIM9

__HAL_DBGMCU_UNFREEZE_TIM10

__HAL_DBGMCU_UNFREEZE_TIM11

__HAL_SYSCFG_REMAPMEMORY_FLASH

__HAL_SYSCFG_REMAPMEMORY_SYSTEMFLASH

__HAL_SYSCFG_REMAPMEMORY_SRAM

__HAL_SYSCFG_REMAPMEMORY_FMC

__HAL_SYSCFG_REMAPMEMORY_FMC_SDRAM

7 HAL ADC Generic Driver

7.1 ADC Firmware driver registers structures

7.1.1 ADC_InitTypeDef

ADC_InitTypeDef is defined in the `stm32f4xx_hal_adc.h`

Data Fields

- ***uint32_t ClockPrescaler***
- ***uint32_t Resolution***
- ***uint32_t DataAlign***
- ***uint32_t ScanConvMode***
- ***uint32_t EOCSelection***
- ***FunctionalState ContinuousConvMode***
- ***uint32_t NbrOfConversion***
- ***FunctionalState DiscontinuousConvMode***
- ***uint32_t NbrOfDiscConversion***
- ***uint32_t ExternalTrigConv***
- ***uint32_t ExternalTrigConvEdge***
- ***FunctionalState DMAContinuousRequests***

Field Documentation

- ***uint32_t ADC_InitTypeDef::ClockPrescaler***
Select ADC clock prescaler. The clock is common for all the ADCs. This parameter can be a value of [ADC_ClockPrescaler](#)
- ***uint32_t ADC_InitTypeDef::Resolution***
Configures the ADC resolution. This parameter can be a value of [ADC_Resolution](#)
- ***uint32_t ADC_InitTypeDef::DataAlign***
Specifies ADC data alignment to right (MSB on register bit 11 and LSB on register bit 0) (default setting) or to left (if regular group: MSB on register bit 15 and LSB on register bit 4, if injected group (MSB kept as signed value due to potential negative value after offset application): MSB on register bit 14 and LSB on register bit 3). This parameter can be a value of [ADC_Data_align](#)
- ***uint32_t ADC_InitTypeDef::ScanConvMode***
Configures the sequencer of regular and injected groups. This parameter can be associated to parameter 'DiscontinuousConvMode' to have main sequence subdivided in successive parts. If disabled: Conversion is performed in single mode (one channel converted, the one defined in rank 1). Parameters 'NbrOfConversion' and 'InjectedNbrOfConversion' are discarded (equivalent to set to 1). If enabled: Conversions are performed in sequence mode (multiple ranks defined by 'NbrOfConversion'/'InjectedNbrOfConversion' and each channel rank). Scan direction is upward: from rank1 to rank 'n'. This parameter can be set to ENABLE or DISABLE
- ***uint32_t ADC_InitTypeDef::EOCSelection***
Specifies what EOC (End Of Conversion) flag is used for conversion by polling and interruption: end of conversion of each rank or complete sequence. This parameter can be a value of [ADC_EOCSelection](#).
Note: For injected group, end of conversion (flag&IT) is raised only at the end of the sequence.
Therefore, if end of conversion is set to end of each conversion, injected group should not be used with interruption (HAL_ADCEx_InjectedStart_IT) or polling (HAL_ADCEx_InjectedStart and HAL_ADCEx_InjectedPollForConversion). By the way, polling is still possible since driver will use an estimated timing for end of injected conversion. Note: If overrun feature is intended to be used, use ADC in mode 'interruption' (function **HAL_ADC_Start_IT()**) with parameter EOCSelection set to end of each conversion or in mode 'transfer by DMA' (function **HAL_ADC_Start_DMA()**). If overrun feature is intended to be bypassed, use ADC in mode 'polling' or 'interruption' with parameter EOCSelection must be set to end of sequence
- ***FunctionalState ADC_InitTypeDef::ContinuousConvMode***
Specifies whether the conversion is performed in single mode (one conversion) or continuous mode for regular group, after the selected trigger occurred (software start or external trigger). This parameter can be set to ENABLE or DISABLE.

- **`uint32_t ADC_InitTypeDef::NbrOfConversion`**
Specifies the number of ranks that will be converted within the regular group sequencer. To use regular group sequencer and convert several ranks, parameter 'ScanConvMode' must be enabled. This parameter must be a number between Min_Data = 1 and Max_Data = 16.
- **`FunctionalState ADC_InitTypeDef::DiscontinuousConvMode`**
Specifies whether the conversions sequence of regular group is performed in Complete-sequence/ Discontinuous-sequence (main sequence subdivided in successive parts). Discontinuous mode is used only if sequencer is enabled (parameter 'ScanConvMode'). If sequencer is disabled, this parameter is discarded. Discontinuous mode can be enabled only if continuous mode is disabled. If continuous mode is enabled, this parameter setting is discarded. This parameter can be set to ENABLE or DISABLE.
- **`uint32_t ADC_InitTypeDef::NbrOfDiscConversion`**
Specifies the number of discontinuous conversions in which the main sequence of regular group (parameter NbrOfConversion) will be subdivided. If parameter 'DiscontinuousConvMode' is disabled, this parameter is discarded. This parameter must be a number between Min_Data = 1 and Max_Data = 8.
- **`uint32_t ADC_InitTypeDef::ExternalTrigConv`**
Selects the external event used to trigger the conversion start of regular group. If set to ADC_SOFTWARE_START, external triggers are disabled. If set to external trigger source, triggering is on event rising edge by default. This parameter can be a value of [ADC_External_trigger_Source_Regular](#)
- **`uint32_t ADC_InitTypeDef::ExternalTrigConvEdge`**
Selects the external trigger edge of regular group. If trigger is set to ADC_SOFTWARE_START, this parameter is discarded. This parameter can be a value of [ADC_External_trigger_edge_Regular](#)
- **`FunctionalState ADC_InitTypeDef::DMAContinuousRequests`**
Specifies whether the DMA requests are performed in one shot mode (DMA transfer stop when number of conversions is reached) or in Continuous mode (DMA transfer unlimited, whatever number of conversions). Note: In continuous mode, DMA must be configured in circular mode. Otherwise an overrun will be triggered when DMA buffer maximum pointer is reached. Note: This parameter must be modified when no conversion is on going on both regular and injected groups (ADC disabled, or ADC enabled without continuous mode or external trigger that could launch a conversion). This parameter can be set to ENABLE or DISABLE.

7.1.2

ADC_ChannelConfTypeDef

`ADC_ChannelConfTypeDef` is defined in the `stm32f4xx_hal_adc.h`

Data Fields

- **`uint32_t Channel`**
- **`uint32_t Rank`**
- **`uint32_t SamplingTime`**
- **`uint32_t Offset`**

Field Documentation

- **`uint32_t ADC_ChannelConfTypeDef::Channel`**
Specifies the channel to configure into ADC regular group. This parameter can be a value of [ADC_channels](#)
- **`uint32_t ADC_ChannelConfTypeDef::Rank`**
Specifies the rank in the regular group sequencer. This parameter must be a number between Min_Data = 1 and Max_Data = 16
- **`uint32_t ADC_ChannelConfTypeDef::SamplingTime`**
Sampling time value to be set for the selected channel. Unit: ADC clock cycles Conversion time is the addition of sampling time and processing time (12 ADC clock cycles at ADC resolution 12 bits, 11 cycles at 10 bits, 9 cycles at 8 bits, 7 cycles at 6 bits). This parameter can be a value of [ADC_sampling_times](#)
Caution: This parameter updates the parameter property of the channel, that can be used into regular and/or injected groups. If this same channel has been previously configured in the other group (regular/ injected), it will be updated to last setting. Note: In case of usage of internal measurement channels (VrefInt/Vbat/TempSensor), sampling time constraints must be respected (sampling time can be adjusted in function of ADC clock frequency and sampling time setting) Refer to device datasheet for timings values, parameters TS_vrefint, TS_temp (values rough order: 4us min).
- **`uint32_t ADC_ChannelConfTypeDef::Offset`**
Reserved for future use, can be set to 0

7.1.3

ADC_AnalogWDGConfTypeDef

ADC_AnalogWDGConfTypeDef is defined in the stm32f4xx_hal_adc.h

Data Fields

- **uint32_t WatchdogMode**
- **uint32_t HighThreshold**
- **uint32_t LowThreshold**
- **uint32_t Channel**
- **FunctionalState ITMode**
- **uint32_t WatchdogNumber**

Field Documentation

- **uint32_t ADC_AnalogWDGConfTypeDef::WatchdogMode**
Configures the ADC analog watchdog mode. This parameter can be a value of [ADC_analog_watchdog_selection](#)
- **uint32_t ADC_AnalogWDGConfTypeDef::HighThreshold**
Configures the ADC analog watchdog High threshold value. This parameter must be a 12-bit value.
- **uint32_t ADC_AnalogWDGConfTypeDef::LowThreshold**
Configures the ADC analog watchdog High threshold value. This parameter must be a 12-bit value.
- **uint32_t ADC_AnalogWDGConfTypeDef::Channel**
Configures ADC channel for the analog watchdog. This parameter has an effect only if watchdog mode is configured on single channel This parameter can be a value of [ADC_channels](#)
- **FunctionalState ADC_AnalogWDGConfTypeDef::ITMode**
Specifies whether the analog watchdog is configured is interrupt mode or in polling mode. This parameter can be set to ENABLE or DISABLE
- **uint32_t ADC_AnalogWDGConfTypeDef::WatchdogNumber**
Reserved for future use, can be set to 0

7.1.4

ADC_HandleTypeDef

ADC_HandleTypeDef is defined in the stm32f4xx_hal_adc.h

Data Fields

- **ADC_TypeDef* Instance**
- **ADC_InitTypeDef Init**
- **__IO uint32_t NbrOfCurrentConversionRank**
- **DMA_HandleTypeDef* DMA_Handle**
- **HAL_LockTypeDef Lock**
- **__IO uint32_t State**
- **__IO uint32_t ErrorCode**

Field Documentation

- **ADC_TypeDef* ADC_HandleTypeDef::Instance**
Register base address
- **ADC_InitTypeDef ADC_HandleTypeDef::Init**
ADC required parameters
- **__IO uint32_t ADC_HandleTypeDef::NbrOfCurrentConversionRank**
ADC number of current conversion rank
- **DMA_HandleTypeDef* ADC_HandleTypeDef::DMA_Handle**
Pointer DMA Handler
- **HAL_LockTypeDef ADC_HandleTypeDef::Lock**
ADC locking object
- **__IO uint32_t ADC_HandleTypeDef::State**
ADC communication state
- **__IO uint32_t ADC_HandleTypeDef::ErrorCode**
ADC Error code

7.2 ADC Firmware driver API description

The following section lists the various functions of the ADC library.

7.2.1 ADC Peripheral features

1. 12-bit, 10-bit, 8-bit or 6-bit configurable resolution.
2. Interrupt generation at the end of conversion, end of injected conversion, and in case of analog watchdog or overrun events
3. Single and continuous conversion modes.
4. Scan mode for automatic conversion of channel 0 to channel x.
5. Data alignment with in-built data coherency.
6. Channel-wise programmable sampling time.
7. External trigger option with configurable polarity for both regular and injected conversion.
8. Dual/Triple mode (on devices with 2 ADCs or more).
9. Configurable DMA data storage in Dual/Triple ADC mode.
10. Configurable delay between conversions in Dual/Triple interleaved mode.
11. ADC conversion type (refer to the datasheets).
12. ADC supply requirements: 2.4 V to 3.6 V at full speed and down to 1.8 V at slower speed.
13. ADC input range: $V_{REF}(\text{minus}) = V_{IN} = V_{REF}(\text{plus})$.
14. DMA request generation during regular channel conversion.

7.2.2 How to use this driver

1. Initialize the ADC low level resources by implementing the `HAL_ADC_MspInit()`:
 - a. Enable the ADC interface clock using `__HAL_RCC_ADC_CLK_ENABLE()`
 - b. ADC pins configuration
 - Enable the clock for the ADC GPIOs using the following function: `__HAL_RCC_GPIOx_CLK_ENABLE()`
 - Configure these ADC pins in analog mode using `HAL_GPIO_Init()`
 - c. In case of using interrupts (e.g. `HAL_ADC_Start_IT()`)
 - Configure the ADC interrupt priority using `HAL_NVIC_SetPriority()`
 - Enable the ADC IRQ handler using `HAL_NVIC_EnableIRQ()`
 - In ADC IRQ handler, call `HAL_ADC_IRQHandler()`
 - d. In case of using DMA to control data transfer (e.g. `HAL_ADC_Start_DMA()`)
 - Enable the DMAx interface clock using `__HAL_RCC_DMAx_CLK_ENABLE()`
 - Configure and enable two DMA streams stream for managing data transfer from peripheral to memory (output stream)
 - Associate the initialized DMA handle to the CRYIP DMA handle using `__HAL_LINKDMA()`
 - Configure the priority and enable the NVIC for the transfer complete interrupt on the two DMA Streams. The output stream should have higher priority than the input stream.

Configuration of ADC, groups regular/injected, channels parameters

1. Configure the ADC parameters (resolution, data alignment, ...) and regular group parameters (conversion trigger, sequencer, ...) using function `HAL_ADC_Init()`.
2. Configure the channels for regular group parameters (channel number, channel rank into sequencer, ..., into regular group) using function `HAL_ADC_ConfigChannel()`.
3. Optionally, configure the injected group parameters (conversion trigger, sequencer, ..., of injected group) and the channels for injected group parameters (channel number, channel rank into sequencer, ..., into injected group) using function `HAL_ADCEX_InjectedConfigChannel()`.
4. Optionally, configure the analog watchdog parameters (channels monitored, thresholds, ...) using function `HAL_ADC_AnalogWDGConfig()`.
5. Optionally, for devices with several ADC instances: configure the multimode parameters using function `HAL_ADCEX_MultiModeConfigChannel()`.

Execution of ADC conversions

1. ADC driver can be used among three modes: polling, interruption, transfer by DMA.

Polling mode IO operation

- Start the ADC peripheral using HAL_ADC_Start()
- Wait for end of conversion using HAL_ADC_PollForConversion(), at this stage user can specify the value of timeout according to his end application
- To read the ADC converted values, use the HAL_ADC_GetValue() function.
- Stop the ADC peripheral using HAL_ADC_Stop()

Interrupt mode IO operation

- Start the ADC peripheral using HAL_ADC_Start_IT()
- Use HAL_ADC_IRQHandler() called under ADC_IRQHandler() Interrupt subroutine
- At ADC end of conversion HAL_ADC_ConvCpltCallback() function is executed and user can add his own code by customization of function pointer HAL_ADC_ConvCpltCallback
- In case of ADC Error, HAL_ADC_ErrorCallback() function is executed and user can add his own code by customization of function pointer HAL_ADC_ErrorCallback
- Stop the ADC peripheral using HAL_ADC_Stop_IT()

DMA mode IO operation

- Start the ADC peripheral using HAL_ADC_Start_DMA(), at this stage the user specify the length of data to be transferred at each end of conversion
- At The end of data transfer by HAL_ADC_ConvCpltCallback() function is executed and user can add his own code by customization of function pointer HAL_ADC_ConvCpltCallback
- In case of transfer Error, HAL_ADC_ErrorCallback() function is executed and user can add his own code by customization of function pointer HAL_ADC_ErrorCallback
- Stop the ADC peripheral using HAL_ADC_Stop_DMA()

ADC HAL driver macros list

Below the list of most used macros in ADC HAL driver.

- __HAL_ADC_ENABLE : Enable the ADC peripheral
- __HAL_ADC_DISABLE : Disable the ADC peripheral
- __HAL_ADC_ENABLE_IT: Enable the ADC end of conversion interrupt
- __HAL_ADC_DISABLE_IT: Disable the ADC end of conversion interrupt
- __HAL_ADC_GET_IT_SOURCE: Check if the specified ADC interrupt source is enabled or disabled
- __HAL_ADC_CLEAR_FLAG: Clear the ADC's pending flags
- __HAL_ADC_GET_FLAG: Get the selected ADC's flag status
- ADC_GET_RESOLUTION: Return resolution bits in CR1 register

Note: You can refer to the ADC HAL driver header file for more useful macros

Deinitialization of ADC

1. Disable the ADC interface
 - ADC clock can be hard reset and disabled at RCC top level.
 - Hard reset of ADC peripherals using macro `__HAL_RCC_ADC_FORCE_RESET()`, `__HAL_RCC_ADC_RELEASE_RESET()`.
 - ADC clock disable using the equivalent macro/functions as configuration step.
 - Example: Into `HAL_ADC_MspDeInit()` (recommended code location) or with other device clock parameters configuration:
 - `HAL_RCC_GetOscConfig(&RCC_OscInitStructure);`
 - `RCC_OscInitStructure.OscillatorType = RCC_OSCILLATORTYPE_HSI;`
 - `RCC_OscInitStructure.HSIState = RCC_HSI_OFF;` (if not used for system clock)
 - `HAL_RCC_OscConfig(&RCC_OscInitStructure);`
2. ADC pins configuration
 - Disable the clock for the ADC GPIOs using macro `__HAL_RCC_GPIOx_CLK_DISABLE()`
3. Optionally, in case of usage of ADC with interruptions:
 - Disable the NVIC for ADC using function `HAL_NVIC_DisableIRQ(ADCx_IRQn)`
4. Optionally, in case of usage of DMA:
 - Deinitialize the DMA using function `HAL_DMA_DeInit()`.
 - Disable the NVIC for DMA using function `HAL_NVIC_DisableIRQ(DMAx_Channelx_IRQn)`

Callback registration

The compilation flag `USE_HAL_ADC_REGISTER_CALLBACKS`, when set to 1, allows the user to configure dynamically the driver callbacks. Use Functions `HAL_ADC_RegisterCallback()` to register an interrupt callback.

Function `HAL_ADC_RegisterCallback()` allows to register following callbacks:

- `ConvCpltCallback` : ADC conversion complete callback
- `ConvHalfCpltCallback` : ADC conversion DMA half-transfer callback
- `LevelOutOfWindowCallback` : ADC analog watchdog 1 callback
- `ErrorCallback` : ADC error callback
- `InjectedConvCpltCallback` : ADC group injected conversion complete callback
- `InjectedQueueOverflowCallback` : ADC group injected context queue overflow callback
- `LevelOutOfWindow2Callback` : ADC analog watchdog 2 callback
- `LevelOutOfWindow3Callback` : ADC analog watchdog 3 callback
- `EndOfSamplingCallback` : ADC end of sampling callback
- `MspInitCallback` : ADC Msp Init callback
- `MspDeInitCallback` : ADC Msp DeInit callback This function takes as parameters the HAL peripheral handle, the Callback ID and a pointer to the user callback function.

Use function `HAL_ADC_UnRegisterCallback` to reset a callback to the default weak function.

`HAL_ADC_UnRegisterCallback` takes as parameters the HAL peripheral handle, and the Callback ID. This function allows to reset following callbacks:

- `ConvCpltCallback` : ADC conversion complete callback
- `ConvHalfCpltCallback` : ADC conversion DMA half-transfer callback
- `LevelOutOfWindowCallback` : ADC analog watchdog 1 callback
- `ErrorCallback` : ADC error callback
- `InjectedConvCpltCallback` : ADC group injected conversion complete callback
- `InjectedQueueOverflowCallback` : ADC group injected context queue overflow callback
- `LevelOutOfWindow2Callback` : ADC analog watchdog 2 callback
- `LevelOutOfWindow3Callback` : ADC analog watchdog 3 callback
- `EndOfSamplingCallback` : ADC end of sampling callback
- `MspInitCallback` : ADC Msp Init callback

- MspDeInitCallback : ADC Msp DeInit callback

By default, after the HAL_ADC_Init() and when the state is HAL_ADC_STATE_RESET all callbacks are set to the corresponding weak functions: examples HAL_ADC_ConvCpltCallback(), HAL_ADC_ErrorCallback(). Exception done for MspInit and MspDeInit functions that are reset to the legacy weak functions in the HAL_ADC_Init()/HAL_ADC_DeInit() only when these callbacks are null (not registered beforehand).

If MspInit or MspDeInit are not null, the HAL_ADC_Init()/HAL_ADC_DeInit() keep and use the user MspInit/MspDeInit callbacks (registered beforehand) whatever the state.

Callbacks can be registered/unregistered in HAL_ADC_STATE_READY state only. Exception done

MspInit/MspDeInit functions that can be registered/unregistered in HAL_ADC_STATE_READY or HAL_ADC_STATE_RESET state, thus registered (user) MspInit/DeInit callbacks can be used during the Init/DeInit.

Then, the user first registers the MspInit/MspDeInit user callbacks using HAL_ADC_RegisterCallback() before calling HAL_ADC_DeInit() or HAL_ADC_Init() function.

When the compilation flag USE_HAL_ADC_REGISTER_CALLBACKS is set to 0 or not defined, the callback registration feature is not available and all callbacks are set to the corresponding weak functions.

7.2.3 Initialization and de-initialization functions

This section provides functions allowing to:

- Initialize and configure the ADC.
- De-initialize the ADC.

This section contains the following APIs:

- **HAL_ADC_Init()**
- **HAL_ADC_DeInit()**
- **HAL_ADC_RegisterCallback()**
- **HAL_ADC_UnRegisterCallback()**
- **HAL_ADC_MspInit()**
- **HAL_ADC_MspDeInit()**

7.2.4 IO operation functions

This section provides functions allowing to:

- Start conversion of regular channel.
- Stop conversion of regular channel.
- Start conversion of regular channel and enable interrupt.
- Stop conversion of regular channel and disable interrupt.
- Start conversion of regular channel and enable DMA transfer.
- Stop conversion of regular channel and disable DMA transfer.
- Handle ADC interrupt request.

This section contains the following APIs:

- **HAL_ADC_Start()**
- **HAL_ADC_Stop()**
- **HAL_ADC_PollForConversion()**
- **HAL_ADC_PollForEvent()**
- **HAL_ADC_Start_IT()**
- **HAL_ADC_Stop_IT()**
- **HAL_ADC_IRQHandler()**
- **HAL_ADC_Start_DMA()**
- **HAL_ADC_Stop_DMA()**
- **HAL_ADC_GetValue()**
- **HAL_ADC_ConvCpltCallback()**
- **HAL_ADC_ConvHalfCpltCallback()**
- **HAL_ADC_LevelOutOfWindowCallback()**

- [*HAL_ADC_ErrorCallback\(\)*](#)

7.2.5 Peripheral Control functions

This section provides functions allowing to:

- Configure regular channels.
- Configure injected channels.
- Configure multimode.
- Configure the analog watch dog.

This section contains the following APIs:

- [*HAL_ADC_ConfigChannel\(\)*](#)
- [*HAL_ADC_AnalogWDGConfig\(\)*](#)

7.2.6 Peripheral State and errors functions

This subsection provides functions allowing to

- Check the ADC state
- Check the ADC Error

This section contains the following APIs:

- [*HAL_ADC_GetState\(\)*](#)
- [*HAL_ADC_GetError\(\)*](#)

7.2.7 Detailed description of functions

HAL_ADC_Init

Function name

HAL_StatusTypeDef HAL_ADC_Init (ADC_HandleTypeDef * hadc)

Function description

Initializes the ADCx peripheral according to the specified parameters in the ADC_InitStruct and initializes the ADC MSP.

Parameters

- **hadc:** pointer to a ADC_HandleTypeDef structure that contains the configuration information for the specified ADC.

Return values

- **HAL:** status

Notes

- This function is used to configure the global features of the ADC (ClockPrescaler, Resolution, Data Alignment and number of conversion), however, the rest of the configuration parameters are specific to the regular channels group (scan mode activation, continuous mode activation, External trigger source and edge, DMA continuous request after the last transfer and End of conversion selection).

HAL_ADC_DeInit

Function name

HAL_StatusTypeDef HAL_ADC_DeInit (ADC_HandleTypeDef * hadc)

Function description

Deinitializes the ADCx peripheral registers to their default reset values.

Parameters

- **hadc:** pointer to a ADC_HandleTypeDef structure that contains the configuration information for the specified ADC.

Return values

- **HAL:** status

HAL_ADC_MspInit

Function name

void HAL_ADC_MspInit (ADC_HandleTypeDef * hadc)

Function description

Initializes the ADC MSP.

Parameters

- **hadc:** pointer to a ADC_HandleTypeDef structure that contains the configuration information for the specified ADC.

Return values

- **None:**

HAL_ADC_MspDeInit

Function name

void HAL_ADC_MspDeInit (ADC_HandleTypeDef * hadc)

Function description

DeInitializes the ADC MSP.

Parameters

- **hadc:** pointer to a ADC_HandleTypeDef structure that contains the configuration information for the specified ADC.

Return values

- **None:**

HAL_ADC_RegisterCallback

Function name

HAL_StatusTypeDef HAL_ADC_RegisterCallback (ADC_HandleTypeDef * hadc, HAL_ADC_CallbackIDTypeDef CallbackID, pADC_CallbackTypeDef pCallback)

Function description

Register a User ADC Callback To be used instead of the weak predefined callback.

Parameters

- **hadc:** Pointer to a ADC_HandleTypeDef structure that contains the configuration information for the specified ADC.
- **CallbackID:** ID of the callback to be registered This parameter can be one of the following values:
 - HAL_ADC_CONVERSION_COMPLETE_CB_ID ADC conversion complete callback ID
 - HAL_ADC_CONVERSION_HALF_CB_ID ADC conversion DMA half-transfer callback ID
 - HAL_ADC_LEVEL_OUT_OF_WINDOW_1_CB_ID ADC analog watchdog 1 callback ID
 - HAL_ADC_ERROR_CB_ID ADC error callback ID
 - HAL_ADC_INJ_CONVERSION_COMPLETE_CB_ID ADC group injected conversion complete callback ID
 - HAL_ADC_MSPINIT_CB_ID ADC Msp Init callback ID
 - HAL_ADC_MSPDEINIT_CB_ID ADC Msp DeInit callback ID
- **pCallback:** pointer to the Callback function

Return values

- **HAL:** status

HAL_ADC_UnRegisterCallback

Function name

HAL_StatusTypeDef HAL_ADC_UnRegisterCallback (ADC_HandleTypeDef * hadc, HAL_ADC_CallbackIDTypeDef CallbackID)

Function description

Unregister a ADC Callback ADC callback is redirected to the weak predefined callback.

Parameters

- **hadc:** Pointer to a ADC_HandleTypeDef structure that contains the configuration information for the specified ADC.
- **CallbackID:** ID of the callback to be unregistered This parameter can be one of the following values:
 - HAL_ADC_CONVERSION_COMPLETE_CB_ID ADC conversion complete callback ID
 - HAL_ADC_CONVERSION_HALF_CB_ID ADC conversion DMA half-transfer callback ID
 - HAL_ADC_LEVEL_OUT_OF_WINDOW_1_CB_ID ADC analog watchdog 1 callback ID
 - HAL_ADC_ERROR_CB_ID ADC error callback ID
 - HAL_ADC_INJ_CONVERSION_COMPLETE_CB_ID ADC group injected conversion complete callback ID
 - HAL_ADC_MSPINIT_CB_ID ADC Msp Init callback ID
 - HAL_ADC_MSPDEINIT_CB_ID ADC Msp DeInit callback ID

Return values

- **HAL:** status

HAL_ADC_Start

Function name

HAL_StatusTypeDef HAL_ADC_Start (ADC_HandleTypeDef * hadc)

Function description

Enables ADC and starts conversion of the regular channels.

Parameters

- **hadc:** pointer to a ADC_HandleTypeDef structure that contains the configuration information for the specified ADC.

Return values

- **HAL:** status

HAL_ADC_Stop

Function name

HAL_StatusTypeDef HAL_ADC_Stop (ADC_HandleTypeDef * hadc)

Function description

Disables ADC and stop conversion of regular channels.

Parameters

- **hadc:** pointer to a ADC_HandleTypeDef structure that contains the configuration information for the specified ADC.

Return values

- **HAL:** status.

Notes

- Caution: This function will stop also injected channels.

HAL_ADC_PollForConversion

Function name

HAL_StatusTypeDef HAL_ADC_PollForConversion (ADC_HandleTypeDef * hadc, uint32_t Timeout)

Function description

Poll for regular conversion complete.

Parameters

- **hadc**: pointer to a ADC_HandleTypeDef structure that contains the configuration information for the specified ADC.
- **Timeout**: Timeout value in millisecond.

Return values

- **HAL**: status

Notes

- ADC conversion flags EOS (end of sequence) and EOC (end of conversion) are cleared by this function.
- This function cannot be used in a particular setup: ADC configured in DMA mode and polling for end of each conversion (ADC init parameter "EOCSelection" set to ADC_EOC_SINGLE_CONV). In this case, DMA resets the flag EOC and polling cannot be performed on each conversion. Nevertheless, polling can still be performed on the complete sequence.

HAL_ADC_PollForEvent

Function name

HAL_StatusTypeDef HAL_ADC_PollForEvent (ADC_HandleTypeDef * hadc, uint32_t EventType, uint32_t Timeout)

Function description

Poll for conversion event.

Parameters

- **hadc**: pointer to a ADC_HandleTypeDef structure that contains the configuration information for the specified ADC.
- **EventType**: the ADC event type. This parameter can be one of the following values:
 - ADC_AWD_EVENT: ADC Analog watch Dog event.
 - ADC_OVR_EVENT: ADC Overrun event.
- **Timeout**: Timeout value in millisecond.

Return values

- **HAL**: status

HAL_ADC_Start_IT

Function name

HAL_StatusTypeDef HAL_ADC_Start_IT (ADC_HandleTypeDef * hadc)

Function description

Enables the interrupt and starts ADC conversion of regular channels.

Parameters

- **hadc**: pointer to a ADC_HandleTypeDef structure that contains the configuration information for the specified ADC.

Return values

- **HAL:** status.

HAL_ADC_Stop_IT

Function name

HAL_StatusTypeDef HAL_ADC_Stop_IT (ADC_HandleTypeDef * hadc)

Function description

Disables the interrupt and stop ADC conversion of regular channels.

Parameters

- **hadc:** pointer to a ADC_HandleTypeDef structure that contains the configuration information for the specified ADC.

Return values

- **HAL:** status.

Notes

- Caution: This function will stop also injected channels.

HAL_ADC_IRQHandler

Function name

void HAL_ADC_IRQHandler (ADC_HandleTypeDef * hadc)

Function description

Handles ADC interrupt request.

Parameters

- **hadc:** pointer to a ADC_HandleTypeDef structure that contains the configuration information for the specified ADC.

Return values

- **None:**

HAL_ADC_Start_DMA

Function name

HAL_StatusTypeDef HAL_ADC_Start_DMA (ADC_HandleTypeDef * hadc, uint32_t * pData, uint32_t Length)

Function description

Enables ADC DMA request after last transfer (Single-ADC mode) and enables ADC peripheral.

Parameters

- **hadc:** pointer to a ADC_HandleTypeDef structure that contains the configuration information for the specified ADC.
- **pData:** The destination Buffer address.
- **Length:** The length of data to be transferred from ADC peripheral to memory.

Return values

- **HAL:** status

HAL_ADC_Stop_DMA

Function name

HAL_StatusTypeDef HAL_ADC_Stop_DMA (ADC_HandleTypeDef * hadc)

Function description

Disables ADC DMA (Single-ADC mode) and disables ADC peripheral.

Parameters

- **hadc:** pointer to a ADC_HandleTypeDef structure that contains the configuration information for the specified ADC.

Return values

- **HAL:** status

HAL_ADC_GetValue

Function name

uint32_t HAL_ADC_GetValue (ADC_HandleTypeDef * hadc)

Function description

Gets the converted value from data register of regular channel.

Parameters

- **hadc:** pointer to a ADC_HandleTypeDef structure that contains the configuration information for the specified ADC.

Return values

- **Converted:** value

HAL_ADC_ConvCpltCallback

Function name

void HAL_ADC_ConvCpltCallback (ADC_HandleTypeDef * hadc)

Function description

Regular conversion complete callback in non blocking mode.

Parameters

- **hadc:** pointer to a ADC_HandleTypeDef structure that contains the configuration information for the specified ADC.

Return values

- **None:**

HAL_ADC_ConvHalfCpltCallback

Function name

void HAL_ADC_ConvHalfCpltCallback (ADC_HandleTypeDef * hadc)

Function description

Regular conversion half DMA transfer callback in non blocking mode.

Parameters

- **hadc:** pointer to a ADC_HandleTypeDef structure that contains the configuration information for the specified ADC.

Return values

- **None:**

HAL_ADC_LevelOutOfWindowCallback

Function name

```
void HAL_ADC_LevelOutOfWindowCallback (ADC_HandleTypeDef * hadc)
```

Function description

Analog watchdog callback in non blocking mode.

Parameters

- **hadc:** pointer to a ADC_HandleTypeDef structure that contains the configuration information for the specified ADC.

Return values

- **None:**

HAL_ADC_ErrorCallback

Function name

```
void HAL_ADC_ErrorCallback (ADC_HandleTypeDef * hadc)
```

Function description

Error ADC callback.

Parameters

- **hadc:** pointer to a ADC_HandleTypeDef structure that contains the configuration information for the specified ADC.

Return values

- **None:**

Notes

- In case of error due to overrun when using ADC with DMA transfer (HAL ADC handle parameter "ErrorCode" to state "HAL_ADC_ERROR_OVR"): Reinitialize the DMA using function "HAL_ADC_Stop_DMA()". If needed, restart a new ADC conversion using function "HAL_ADC_Start_DMA()" (this function is also clearing overrun flag)

HAL_ADC_ConfigChannel

Function name

```
HAL_StatusTypeDef HAL_ADC_ConfigChannel (ADC_HandleTypeDef * hadc, ADC_ChannelConfTypeDef * sConfig)
```

Function description

Configures for the selected ADC regular channel its corresponding rank in the sequencer and its sample time.

Parameters

- **hadc:** pointer to a ADC_HandleTypeDef structure that contains the configuration information for the specified ADC.
- **sConfig:** ADC configuration structure.

Return values

- **HAL:** status

HAL_ADC_AnalogWDGConfig

Function name

```
HAL_StatusTypeDef HAL_ADC_AnalogWDGConfig (ADC_HandleTypeDef * hadc, ADC_AnalogWDGConfTypeDef * AnalogWDGConfig)
```

Function description

Configures the analog watchdog.

Parameters

- **hadc**: pointer to a ADC_HandleTypeDef structure that contains the configuration information for the specified ADC.
- **AnalogWDGConfig**: pointer to an ADC_AnalogWDGConfTypeDef structure that contains the configuration information of ADC analog watchdog.

Return values

- **HAL**: status

Notes

- Analog watchdog thresholds can be modified while ADC conversion is on going. In this case, some constraints must be taken into account: The programmed threshold values are effective from the next ADC EOC (end of unitary conversion). Considering that registers write delay may happen due to bus activity, this might cause an uncertainty on the effective timing of the new programmed threshold values.

HAL_ADC_GetState

Function name

uint32_t HAL_ADC_GetState (ADC_HandleTypeDef * hadc)

Function description

return the ADC state

Parameters

- **hadc**: pointer to a ADC_HandleTypeDef structure that contains the configuration information for the specified ADC.

Return values

- **HAL**: state

HAL_ADC_GetError

Function name

uint32_t HAL_ADC_GetError (ADC_HandleTypeDef * hadc)

Function description

Return the ADC error code.

Parameters

- **hadc**: pointer to a ADC_HandleTypeDef structure that contains the configuration information for the specified ADC.

Return values

- **ADC**: Error Code

7.3 ADC Firmware driver defines

The following section lists the various define and macros of the module.

7.3.1 ADC

ADC

ADC Analog Watchdog Selection

ADC_ANALOGWATCHDOG_SINGLE_REG

ADC_ANALOGWATCHDOG_SINGLE_INJEC

ADC_ANALOGWATCHDOG_SINGLE_REGINJEC

ADC_ANALOGWATCHDOG_ALL_REG

ADC_ANALOGWATCHDOG_ALL_INJEC

ADC_ANALOGWATCHDOG_ALL_REGINJEC

ADC_ANALOGWATCHDOG_NONE

ADC Common Channels

ADC_CHANNEL_0

ADC_CHANNEL_1

ADC_CHANNEL_2

ADC_CHANNEL_3

ADC_CHANNEL_4

ADC_CHANNEL_5

ADC_CHANNEL_6

ADC_CHANNEL_7

ADC_CHANNEL_8

ADC_CHANNEL_9

ADC_CHANNEL_10

ADC_CHANNEL_11

ADC_CHANNEL_12

ADC_CHANNEL_13

ADC_CHANNEL_14

ADC_CHANNEL_15

ADC_CHANNEL_16

ADC_CHANNEL_17

ADC_CHANNEL_18

ADC_CHANNEL_VREFINT

ADC_CHANNEL_VBAT

ADC Channels Type**ADC_ALL_CHANNELS****ADC_REGULAR_CHANNELS**

reserved for future use

ADC_INJECTED_CHANNELS

reserved for future use

ADC Clock Prescaler**ADC_CLOCK_SYNC_PCLK_DIV2****ADC_CLOCK_SYNC_PCLK_DIV4****ADC_CLOCK_SYNC_PCLK_DIV6****ADC_CLOCK_SYNC_PCLK_DIV8****ADC Data Align****ADC_DATAALIGN_RIGHT****ADC_DATAALIGN_LEFT****ADC Delay Between 2 Sampling Phases****ADC_TWOSAMPLINGDELAY_5CYCLES****ADC_TWOSAMPLINGDELAY_6CYCLES****ADC_TWOSAMPLINGDELAY_7CYCLES****ADC_TWOSAMPLINGDELAY_8CYCLES****ADC_TWOSAMPLINGDELAY_9CYCLES****ADC_TWOSAMPLINGDELAY_10CYCLES****ADC_TWOSAMPLINGDELAY_11CYCLES****ADC_TWOSAMPLINGDELAY_12CYCLES****ADC_TWOSAMPLINGDELAY_13CYCLES****ADC_TWOSAMPLINGDELAY_14CYCLES****ADC_TWOSAMPLINGDELAY_15CYCLES****ADC_TWOSAMPLINGDELAY_16CYCLES****ADC_TWOSAMPLINGDELAY_17CYCLES****ADC_TWOSAMPLINGDELAY_18CYCLES****ADC_TWOSAMPLINGDELAY_19CYCLES**

ADC_TWOSAMPLINGDELAY_20CYCLES

ADC EOC Selection

ADC_EOC_SEQ_CONV

ADC_EOC_SINGLE_CONV

ADC_EOC_SINGLE_SEQ_CONV

reserved for future use

ADC Error Code

HAL_ADC_ERROR_NONE

No error

HAL_ADC_ERROR_INTERNAL

ADC IP internal error: if problem of clocking, enable/disable, erroneous state

HAL_ADC_ERROR_OVR

Overrun error

HAL_ADC_ERROR_DMA

DMA transfer error

HAL_ADC_ERROR_INVALID_CALLBACK

Invalid Callback error

ADC Event Type

ADC_AWD_EVENT

ADC_OVR_EVENT

ADC Exported Macros

__HAL_ADC_RESET_HANDLE_STATE

Description:

- Reset ADC handle state.

Parameters:

- __HANDLE__: ADC handle

Return value:

- None

__HAL_ADC_ENABLE

Description:

- Enable the ADC peripheral.

Parameters:

- __HANDLE__: ADC handle

Return value:

- None

__HAL_ADC_DISABLE

Description:

- Disable the ADC peripheral.

Parameters:

- __HANDLE__: ADC handle

Return value:

- None

__HAL_ADC_ENABLE_IT

Description:

- Enable the ADC end of conversion interrupt.

Parameters:

- __HANDLE__: specifies the ADC Handle.
- __INTERRUPT__: ADC Interrupt.

Return value:

- None

__HAL_ADC_DISABLE_IT

Description:

- Disable the ADC end of conversion interrupt.

Parameters:

- __HANDLE__: specifies the ADC Handle.
- __INTERRUPT__: ADC interrupt.

Return value:

- None

__HAL_ADC_GET_IT_SOURCE

Description:

- Check if the specified ADC interrupt source is enabled or disabled.

Parameters:

- __HANDLE__: specifies the ADC Handle.
- __INTERRUPT__: specifies the ADC interrupt source to check.

Return value:

- The: new state of __IT__ (TRUE or FALSE).

__HAL_ADC_CLEAR_FLAG

Description:

- Clear the ADC's pending flags.

Parameters:

- __HANDLE__: specifies the ADC Handle.
- __FLAG__: ADC flag.

Return value:

- None

__HAL_ADC_GET_FLAG

Description:

- Get the selected ADC's flag status.

Parameters:

- __HANDLE__: specifies the ADC Handle.
- __FLAG__: ADC flag.

Return value:

- None

ADC Exported Types

HAL_ADC_STATE_RESET

ADC not yet initialized or disabled

HAL_ADC_STATE_READY

ADC peripheral ready for use

HAL_ADC_STATE_BUSY_INTERNAL

ADC is busy to internal process (initialization, calibration)

HAL_ADC_STATE_TIMEOUT

TimeOut occurrence

HAL_ADC_STATE_ERROR_INTERNAL

Internal error occurrence

HAL_ADC_STATE_ERROR_CONFIG

Configuration error occurrence

HAL_ADC_STATE_ERROR_DMA

DMA error occurrence

HAL_ADC_STATE_REG_BUSY

A conversion on group regular is ongoing or can occur (either by continuous mode, external trigger, low power auto power-on (if feature available), multimode ADC master control (if feature available))

HAL_ADC_STATE_REG_EOC

Conversion data available on group regular

HAL_ADC_STATE_REG_OVR

Overrun occurrence

HAL_ADC_STATE_INJ_BUSY

A conversion on group injected is ongoing or can occur (either by auto-injection mode, external trigger, low power auto power-on (if feature available), multimode ADC master control (if feature available))

HAL_ADC_STATE_INJ_EOC

Conversion data available on group injected

HAL_ADC_STATE_AWD1

Out-of-window occurrence of analog watchdog 1

HAL_ADC_STATE_AWD2

Not available on STM32F4 device: Out-of-window occurrence of analog watchdog 2

HAL_ADC_STATE_AWD3

Not available on STM32F4 device: Out-of-window occurrence of analog watchdog 3

HAL_ADC_STATE_MULTIMODE_SLAVE

Not available on STM32F4 device: ADC in multimode slave state, controlled by another ADC master (

ADC External Trigger Edge Regular**ADC_EXTERNALTRIGCONVEDGE_NONE****ADC_EXTERNALTRIGCONVEDGE_RISING****ADC_EXTERNALTRIGCONVEDGE_FALLING****ADC_EXTERNALTRIGCONVEDGE_RISINGFALLING*****ADC External Trigger Source Regular*****ADC_EXTERNALTRIGCONV_T1_CC1****ADC_EXTERNALTRIGCONV_T1_CC2****ADC_EXTERNALTRIGCONV_T1_CC3****ADC_EXTERNALTRIGCONV_T2_CC2****ADC_EXTERNALTRIGCONV_T2_CC3****ADC_EXTERNALTRIGCONV_T2_CC4****ADC_EXTERNALTRIGCONV_T2_TRGO****ADC_EXTERNALTRIGCONV_T3_CC1****ADC_EXTERNALTRIGCONV_T3_TRGO****ADC_EXTERNALTRIGCONV_T4_CC4****ADC_EXTERNALTRIGCONV_T5_CC1****ADC_EXTERNALTRIGCONV_T5_CC2****ADC_EXTERNALTRIGCONV_T5_CC3****ADC_EXTERNALTRIGCONV_T8_CC1****ADC_EXTERNALTRIGCONV_T8_TRGO****ADC_EXTERNALTRIGCONV_Ext_IT11****ADC_SOFTWARE_START*****ADC Flags Definition*****ADC_FLAG_AWD****ADC_FLAG_EOC****ADC_FLAG_JEOC**

ADC_FLAG_JSTRT

ADC_FLAG_STRT

ADC_FLAG_OVR

ADC Interrupts Definition

ADC_IT_EOC

ADC_IT_AWD

ADC_IT_JEOC

ADC_IT_OVR

ADC Resolution

ADC_RESOLUTION_12B

ADC_RESOLUTION_10B

ADC_RESOLUTION_8B

ADC_RESOLUTION_6B

ADC Sampling Times

ADC_SAMPLETIME_3CYCLES

ADC_SAMPLETIME_15CYCLES

ADC_SAMPLETIME_28CYCLES

ADC_SAMPLETIME_56CYCLES

ADC_SAMPLETIME_84CYCLES

ADC_SAMPLETIME_112CYCLES

ADC_SAMPLETIME_144CYCLES

ADC_SAMPLETIME_480CYCLES

8 HAL ADC Extension Driver

8.1 ADCEx Firmware driver registers structures

8.1.1 ADC_InjectionConfTypeDef

ADC_InjectionConfTypeDef is defined in the stm32f4xx_hal_adc_ex.h

Data Fields

- *uint32_t InjectedChannel*
- *uint32_t InjectedRank*
- *uint32_t InjectedSamplingTime*
- *uint32_t InjectedOffset*
- *uint32_t InjectedNbrOfConversion*
- *FunctionalState InjectedDiscontinuousConvMode*
- *FunctionalState AutoInjectedConv*
- *uint32_t ExternalTrigInjecConv*
- *uint32_t ExternalTrigInjecConvEdge*

Field Documentation

- *uint32_t ADC_InjectionConfTypeDef::InjectedChannel*
Selection of ADC channel to configure This parameter can be a value of **ADC_channels** Note: Depending on devices, some channels may not be available on package pins. Refer to device datasheet for channels availability.
- *uint32_t ADC_InjectionConfTypeDef::InjectedRank*
Rank in the injected group sequencer This parameter must be a value of **ADCEx_injected_rank** Note: In case of need to disable a channel or change order of conversion sequencer, rank containing a previous channel setting can be overwritten by the new channel setting (or parameter number of conversions can be adjusted)
- *uint32_t ADC_InjectionConfTypeDef::InjectedSamplingTime*
Sampling time value to be set for the selected channel. Unit: ADC clock cycles Conversion time is the addition of sampling time and processing time (12 ADC clock cycles at ADC resolution 12 bits, 11 cycles at 10 bits, 9 cycles at 8 bits, 7 cycles at 6 bits). This parameter can be a value of **ADC_sampling_times** Caution: This parameter updates the parameter property of the channel, that can be used into regular and/or injected groups. If this same channel has been previously configured in the other group (regular/ injected), it will be updated to last setting. Note: In case of usage of internal measurement channels (VrefInt/Vbat/TempSensor), sampling time constraints must be respected (sampling time can be adjusted in function of ADC clock frequency and sampling time setting) Refer to device datasheet for timings values, parameters TS_vrefint, TS_temp (values rough order: 4us min).
- *uint32_t ADC_InjectionConfTypeDef::InjectedOffset*
Defines the offset to be subtracted from the raw converted data (for channels set on injected group only). Offset value must be a positive number. Depending of ADC resolution selected (12, 10, 8 or 6 bits), this parameter must be a number between Min_Data = 0x000 and Max_Data = 0xFFF, 0x3FF, 0xFF or 0x3F respectively.
- *uint32_t ADC_InjectionConfTypeDef::InjectedNbrOfConversion*
Specifies the number of ranks that will be converted within the injected group sequencer. To use the injected group sequencer and convert several ranks, parameter 'ScanConvMode' must be enabled. This parameter must be a number between Min_Data = 1 and Max_Data = 4. Caution: this setting impacts the entire injected group. Therefore, call of **HAL_ADCEx_InjectedConfigChannel()** to configure a channel on injected group can impact the configuration of other channels previously set.

- FunctionalState ADC_InjectionConfTypeDef::InjectedDiscontinuousConvMode**
Specifies whether the conversions sequence of injected group is performed in Complete-sequence/ Discontinuous-sequence (main sequence subdivided in successive parts). Discontinuous mode is used only if sequencer is enabled (parameter 'ScanConvMode'). If sequencer is disabled, this parameter is discarded. Discontinuous mode can be enabled only if continuous mode is disabled. If continuous mode is enabled, this parameter setting is discarded. This parameter can be set to ENABLE or DISABLE. Note: For injected group, number of discontinuous ranks increment is fixed to one-by-one. Caution: this setting impacts the entire injected group. Therefore, call of **HAL_ADCEx_InjectedConfigChannel()** to configure a channel on injected group can impact the configuration of other channels previously set.
- FunctionalState ADC_InjectionConfTypeDef::AutoInjectedConv**
Enables or disables the selected ADC automatic injected group conversion after regular one This parameter can be set to ENABLE or DISABLE. Note: To use Automatic injected conversion, discontinuous mode must be disabled ('DiscontinuousConvMode' and 'InjectedDiscontinuousConvMode' set to DISABLE) Note: To use Automatic injected conversion, injected group external triggers must be disabled ('ExternalTrigInjecConv' set to ADC_SOFTWARE_START) Note: In case of DMA used with regular group: if DMA configured in normal mode (single shot) JAUTO will be stopped upon DMA transfer complete. To maintain JAUTO always enabled, DMA must be configured in circular mode. Caution: this setting impacts the entire injected group. Therefore, call of **HAL_ADCEx_InjectedConfigChannel()** to configure a channel on injected group can impact the configuration of other channels previously set.
- uint32_t ADC_InjectionConfTypeDef::ExternalTrigInjecConv**
Selects the external event used to trigger the conversion start of injected group. If set to ADC_INJECTED_SOFTWARE_START, external triggers are disabled. If set to external trigger source, triggering is on event rising edge. This parameter can be a value of **ADCEx_External_trigger_Source_Injected** Note: This parameter must be modified when ADC is disabled (before ADC start conversion or after ADC stop conversion). If ADC is enabled, this parameter setting is bypassed without error reporting (as it can be the expected behaviour in case of another parameter update on the fly) Caution: this setting impacts the entire injected group. Therefore, call of **HAL_ADCEx_InjectedConfigChannel()** to configure a channel on injected group can impact the configuration of other channels previously set.
- uint32_t ADC_InjectionConfTypeDef::ExternalTrigInjecConvEdge**
Selects the external trigger edge of injected group. This parameter can be a value of **ADCEx_External_trigger_edge_Injected**. If trigger is set to ADC_INJECTED_SOFTWARE_START, this parameter is discarded. Caution: this setting impacts the entire injected group. Therefore, call of **HAL_ADCEx_InjectedConfigChannel()** to configure a channel on injected group can impact the configuration of other channels previously set.

8.1.2

ADC_MultiModeTypeDef

ADC_MultiModeTypeDef is defined in the stm32f4xx_hal_adc_ex.h

Data Fields

- uint32_t Mode**
- uint32_t DMAAccessMode**
- uint32_t TwoSamplingDelay**

Field Documentation

- uint32_t ADC_MultiModeTypeDef::Mode**
Configures the ADC to operate in independent or multi mode. This parameter can be a value of **ADCEx_Common_mode**
- uint32_t ADC_MultiModeTypeDef::DMAAccessMode**
Configures the Direct memory access mode for multi ADC mode. This parameter can be a value of **ADCEx_Direct_memory_access_mode_for_multi_mode**
- uint32_t ADC_MultiModeTypeDef::TwoSamplingDelay**
Configures the Delay between 2 sampling phases. This parameter can be a value of **ADC_delay_between_2_sampling_phases**

8.2

ADCEx Firmware driver API description

The following section lists the various functions of the ADCEx library.

8.2.1

How to use this driver

1. Initialize the ADC low level resources by implementing the HAL_ADC_MspInit():
 - a. Enable the ADC interface clock using __HAL_RCC_ADC_CLK_ENABLE()
 - b. ADC pins configuration
 - Enable the clock for the ADC GPIOs using the following function: __HAL_RCC_GPIOx_CLK_ENABLE()
 - Configure these ADC pins in analog mode using HAL_GPIO_Init()
 - c. In case of using interrupts (e.g. HAL_ADC_Start_IT())
 - Configure the ADC interrupt priority using HAL_NVIC_SetPriority()
 - Enable the ADC IRQ handler using HAL_NVIC_EnableIRQ()
 - In ADC IRQ handler, call HAL_ADC_IRQHandler()
 - d. In case of using DMA to control data transfer (e.g. HAL_ADC_Start_DMA())
 - Enable the DMAx interface clock using __HAL_RCC_DMAx_CLK_ENABLE()
 - Configure and enable two DMA streams stream for managing data transfer from peripheral to memory (output stream)
 - Associate the initialized DMA handle to the ADC DMA handle using __HAL_LINKDMA()
 - Configure the priority and enable the NVIC for the transfer complete interrupt on the two DMA Streams. The output stream should have higher priority than the input stream.
2. Configure the ADC Prescaler, conversion resolution and data alignment using the HAL_ADC_Init() function.
3. Configure the ADC Injected channels group features, use HAL_ADC_Init() and HAL_ADC_ConfigChannel() functions.
4. Three operation modes are available within this driver:

Polling mode IO operation

- Start the ADC peripheral using HAL_ADCEx_InjectedStart()
- Wait for end of conversion using HAL_ADC_PollForConversion(), at this stage user can specify the value of timeout according to his end application
- To read the ADC converted values, use the HAL_ADCEx_InjectedGetValue() function.
- Stop the ADC peripheral using HAL_ADCEx_InjectedStop()

Interrupt mode IO operation

- Start the ADC peripheral using HAL_ADCEx_InjectedStart_IT()
- Use HAL_ADC_IRQHandler() called under ADC_IRQHandler() Interrupt subroutine
- At ADC end of conversion HAL_ADCEx_InjectedConvCpltCallback() function is executed and user can add his own code by customization of function pointer HAL_ADCEx_InjectedConvCpltCallback
- In case of ADC Error, HAL_ADCEx_InjectedErrorCallback() function is executed and user can add his own code by customization of function pointer HAL_ADCEx_InjectedErrorCallback
- Stop the ADC peripheral using HAL_ADCEx_InjectedStop_IT()

Multi mode ADCs Regular channels configuration

- Select the Multi mode ADC regular channels features (dual or triple mode) and configure the DMA mode using HAL_ADCEx_MultiModeConfigChannel() functions.
- Start the ADC peripheral using HAL_ADCEx_MultiModeStart_DMA(), at this stage the user specify the length of data to be transferred at each end of conversion
- Read the ADCs converted values using the HAL_ADCEx_MultiModeGetValue() function.

8.2.2

Extended features functions

This section provides functions allowing to:

- Start conversion of injected channel.
- Stop conversion of injected channel.
- Start multimode and enable DMA transfer.
- Stop multimode and disable DMA transfer.

- Get result of injected channel conversion.
- Get result of multimode conversion.
- Configure injected channels.
- Configure multimode.

This section contains the following APIs:

- [*HAL_ADCEX_InjectedStart\(\)*](#)
- [*HAL_ADCEX_InjectedStart_IT\(\)*](#)
- [*HAL_ADCEX_InjectedStop\(\)*](#)
- [*HAL_ADCEX_InjectedPollForConversion\(\)*](#)
- [*HAL_ADCEX_InjectedStop_IT\(\)*](#)
- [*HAL_ADCEX_InjectedGetValue\(\)*](#)
- [*HAL_ADCEX_MultiModeStart_DMA\(\)*](#)
- [*HAL_ADCEX_MultiModeStop_DMA\(\)*](#)
- [*HAL_ADCEX_MultiModeGetValue\(\)*](#)
- [*HAL_ADCEX_InjectedConvCpltCallback\(\)*](#)
- [*HAL_ADCEX_InjectedConfigChannel\(\)*](#)
- [*HAL_ADCEX_MultiModeConfigChannel\(\)*](#)

8.2.3 Detailed description of functions

HAL_ADCEX_InjectedStart

Function name

HAL_StatusTypeDef HAL_ADCEX_InjectedStart (ADC_HandleTypeDef * hadc)

Function description

Enables the selected ADC software start conversion of the injected channels.

Parameters

- **hadc:** pointer to a ADC_HandleTypeDef structure that contains the configuration information for the specified ADC.

Return values

- **HAL:** status

HAL_ADCEX_InjectedStop

Function name

HAL_StatusTypeDef HAL_ADCEX_InjectedStop (ADC_HandleTypeDef * hadc)

Function description

Stop conversion of injected channels.

Parameters

- **hadc:** ADC handle

Return values

- **None:**

Notes

- If ADC must be disabled and if conversion is on going on regular group, function HAL_ADC_Stop must be used to stop both injected and regular groups, and disable the ADC.
- If injected group mode auto-injection is enabled, function HAL_ADC_Stop must be used.
- In case of auto-injection mode, HAL_ADC_Stop must be used.

HAL_ADCEX_InjectedPollForConversion

Function name

HAL_StatusTypeDef HAL_ADCEX_InjectedPollForConversion (ADC_HandleTypeDef * hadc, uint32_t Timeout)

Function description

Poll for injected conversion complete.

Parameters

- **hadc:** pointer to a ADC_HandleTypeDef structure that contains the configuration information for the specified ADC.
- **Timeout:** Timeout value in millisecond.

Return values

- **HAL:** status

HAL_ADCEX_InjectedStart_IT

Function name

HAL_StatusTypeDef HAL_ADCEX_InjectedStart_IT (ADC_HandleTypeDef * hadc)

Function description

Enables the interrupt and starts ADC conversion of injected channels.

Parameters

- **hadc:** pointer to a ADC_HandleTypeDef structure that contains the configuration information for the specified ADC.

Return values

- **HAL:** status.

HAL_ADCEX_InjectedStop_IT

Function name

HAL_StatusTypeDef HAL_ADCEX_InjectedStop_IT (ADC_HandleTypeDef * hadc)

Function description

Stop conversion of injected channels, disable interruption of end-of-conversion.

Parameters

- **hadc:** ADC handle

Return values

- **None:**

Notes

- If ADC must be disabled and if conversion is on going on regular group, function HAL_ADC_Stop must be used to stop both injected and regular groups, and disable the ADC.
- If injected group mode auto-injection is enabled, function HAL_ADC_Stop must be used.

HAL_ADCEX_InjectedGetValue

Function name

uint32_t HAL_ADCEX_InjectedGetValue (ADC_HandleTypeDef * hadc, uint32_t InjectedRank)

Function description

Gets the converted value from data register of injected channel.

Parameters

- **hadc**: pointer to a ADC_HandleTypeDef structure that contains the configuration information for the specified ADC.
- **InjectedRank**: the ADC injected rank. This parameter can be one of the following values:
 - ADC_INJECTED_RANK_1: Injected Channel1 selected
 - ADC_INJECTED_RANK_2: Injected Channel2 selected
 - ADC_INJECTED_RANK_3: Injected Channel3 selected
 - ADC_INJECTED_RANK_4: Injected Channel4 selected

Return values

- **None**:

HAL_ADCEx_MultiModeStart_DMA

Function name

HAL_StatusTypeDef HAL_ADCEx_MultiModeStart_DMA (ADC_HandleTypeDef * hadc, uint32_t * pData, uint32_t Length)

Function description

Enables ADC DMA request after last transfer (Multi-ADC mode) and enables ADC peripheral.

Parameters

- **hadc**: pointer to a ADC_HandleTypeDef structure that contains the configuration information for the specified ADC.
- **pData**: Pointer to buffer in which transferred from ADC peripheral to memory will be stored.
- **Length**: The length of data to be transferred from ADC peripheral to memory.

Return values

- **HAL**: status

Notes

- Caution: This function must be used only with the ADC master.

HAL_ADCEx_MultiModeStop_DMA

Function name

HAL_StatusTypeDef HAL_ADCEx_MultiModeStop_DMA (ADC_HandleTypeDef * hadc)

Function description

Disables ADC DMA (multi-ADC mode) and disables ADC peripheral.

Parameters

- **hadc**: pointer to a ADC_HandleTypeDef structure that contains the configuration information for the specified ADC.

Return values

- **HAL**: status

HAL_ADCEx_MultiModeGetValue

Function name

uint32_t HAL_ADCEx_MultiModeGetValue (ADC_HandleTypeDef * hadc)

Function description

Returns the last ADC1, ADC2 and ADC3 regular conversions results data in the selected multi mode.

Parameters

- **hadc:** pointer to a ADC_HandleTypeDef structure that contains the configuration information for the specified ADC.

Return values

- **The:** converted data value.

HAL_ADCEx_InjectedConvCpltCallback

Function name

void HAL_ADCEx_InjectedConvCpltCallback (ADC_HandleTypeDef * hadc)

Function description

Injected conversion complete callback in non blocking mode.

Parameters

- **hadc:** pointer to a ADC_HandleTypeDef structure that contains the configuration information for the specified ADC.

Return values

- **None:**

HAL_ADCEx_InjectedConfigChannel

Function name

HAL_StatusTypeDef HAL_ADCEx_InjectedConfigChannel (ADC_HandleTypeDef * hadc, ADC_InjectionConfTypeDef * sConfigInjected)

Function description

Configures for the selected ADC injected channel its corresponding rank in the sequencer and its sample time.

Parameters

- **hadc:** pointer to a ADC_HandleTypeDef structure that contains the configuration information for the specified ADC.
- **sConfigInjected:** ADC configuration structure for injected channel.

Return values

- **None:**

HAL_ADCEx_MultiModeConfigChannel

Function name

HAL_StatusTypeDef HAL_ADCEx_MultiModeConfigChannel (ADC_HandleTypeDef * hadc, ADC_MultiModeTypeDef * multimode)

Function description

Configures the ADC multi-mode.

Parameters

- **hadc:** pointer to a ADC_HandleTypeDef structure that contains the configuration information for the specified ADC.
- **multimode:** pointer to an ADC_MultiModeTypeDef structure that contains the configuration information for multimode.

Return values

- **HAL:** status

8.3 ADCEX Firmware driver defines

The following section lists the various define and macros of the module.

8.3.1 ADCEX

ADCEX

ADC Specific Channels

ADC_CHANNEL_DIFFERENCIATION_TEMPSENSOR_VBAT

ADC_CHANNEL_TEMPSENSOR

ADC Common Mode

ADC_MODE_INDEPENDENT

ADC_DUALMODE_REGSIMULT_INJECSIMULT

ADC_DUALMODE_REGSIMULT_ALTERTRIG

ADC_DUALMODE_INJECSIMULT

ADC_DUALMODE_REGSIMULT

ADC_DUALMODE_INTERL

ADC_DUALMODE_ALTERTRIG

ADC_TRIPLEMODE_REGSIMULT_INJECSIMULT

ADC_TRIPLEMODE_REGSIMULT_AlterTrig

ADC_TRIPLEMODE_INJECSIMULT

ADC_TRIPLEMODE_REGSIMULT

ADC_TRIPLEMODE_INTERL

ADC_TRIPLEMODE_ALTERTRIG

ADC Direct Memory Access Mode For Multi Mode

ADC_DMAACCESSMODE_DISABLED

DMA mode disabled

ADC_DMAACCESSMODE_1

DMA mode 1 enabled (2 / 3 half-words one by one - 1 then 2 then 3)

ADC_DMAACCESSMODE_2

DMA mode 2 enabled (2 / 3 half-words by pairs - 2&1 then 1&3 then 3&2)

ADC_DMAACCESSMODE_3

DMA mode 3 enabled (2 / 3 bytes by pairs - 2&1 then 1&3 then 3&2)

ADC External Trigger Edge Injected

ADC_EXTERNALTRIGINJECCONVEDGE_NONE
ADC_EXTERNALTRIGINJECCONVEDGE_RISING
ADC_EXTERNALTRIGINJECCONVEDGE_FALLING
ADC_EXTERNALTRIGINJECCONVEDGE_RISINGFALLING

ADC External Trigger Source Injected

ADC_EXTERNALTRIGINJECCONV_T1_CC4
ADC_EXTERNALTRIGINJECCONV_T1_TRGO
ADC_EXTERNALTRIGINJECCONV_T2_CC1
ADC_EXTERNALTRIGINJECCONV_T2_TRGO
ADC_EXTERNALTRIGINJECCONV_T3_CC2
ADC_EXTERNALTRIGINJECCONV_T3_CC4
ADC_EXTERNALTRIGINJECCONV_T4_CC1
ADC_EXTERNALTRIGINJECCONV_T4_CC2
ADC_EXTERNALTRIGINJECCONV_T4_CC3
ADC_EXTERNALTRIGINJECCONV_T4_TRGO
ADC_EXTERNALTRIGINJECCONV_T5_CC4
ADC_EXTERNALTRIGINJECCONV_T5_TRGO
ADC_EXTERNALTRIGINJECCONV_T8_CC2
ADC_EXTERNALTRIGINJECCONV_T8_CC3
ADC_EXTERNALTRIGINJECCONV_T8_CC4
ADC_EXTERNALTRIGINJECCONV_EXT_IT15
ADC_INJECTED_SOFTWARE_START

ADC Injected Rank

ADC_INJECTED_RANK_1
ADC_INJECTED_RANK_2
ADC_INJECTED_RANK_3
ADC_INJECTED_RANK_4

11 HAL CORTEX Generic Driver

11.1 CORTEX Firmware driver registers structures

11.1.1 MPU_Region_InitTypeDef

MPU_Region_InitTypeDef is defined in the stm32f4xx_hal_cortex.h

Data Fields

- **uint8_t Enable**
- **uint8_t Number**
- **uint32_t BaseAddress**
- **uint8_t Size**
- **uint8_t SubRegionDisable**
- **uint8_t TypeExtField**
- **uint8_t AccessPermission**
- **uint8_t DisableExec**
- **uint8_t IsShareable**
- **uint8_t IsCacheable**
- **uint8_t IsBufferable**

Field Documentation

- **uint8_t MPU_Region_InitTypeDef::Enable**
Specifies the status of the region. This parameter can be a value of [CORTEX_MPU_Region_Enable](#)
- **uint8_t MPU_Region_InitTypeDef::Number**
Specifies the number of the region to protect. This parameter can be a value of [CORTEX_MPU_Region_Number](#)
- **uint32_t MPU_Region_InitTypeDef::BaseAddress**
Specifies the base address of the region to protect.
- **uint8_t MPU_Region_InitTypeDef::Size**
Specifies the size of the region to protect. This parameter can be a value of [CORTEX_MPU_Region_Size](#)
- **uint8_t MPU_Region_InitTypeDef::SubRegionDisable**
Specifies the number of the subregion protection to disable. This parameter must be a number between Min_Data = 0x00 and Max_Data = 0xFF
- **uint8_t MPU_Region_InitTypeDef::TypeExtField**
Specifies the TEX field level. This parameter can be a value of [CORTEX_MPU_TEX_Levels](#)
- **uint8_t MPU_Region_InitTypeDef::AccessPermission**
Specifies the region access permission type. This parameter can be a value of [CORTEX_MPU_Region_Permission_Attributes](#)
- **uint8_t MPU_Region_InitTypeDef::DisableExec**
Specifies the instruction access status. This parameter can be a value of [CORTEX_MPU_Instruction_Access](#)
- **uint8_t MPU_Region_InitTypeDef::IsShareable**
Specifies the shareability status of the protected region. This parameter can be a value of [CORTEX_MPU_Access_Shareable](#)
- **uint8_t MPU_Region_InitTypeDef::IsCacheable**
Specifies the cacheable status of the region protected. This parameter can be a value of [CORTEX_MPU_Access_Cacheable](#)
- **uint8_t MPU_Region_InitTypeDef::IsBufferable**
Specifies the bufferable status of the protected region. This parameter can be a value of [CORTEX_MPU_Access_Bufferable](#)

11.2 CORTEX Firmware driver API description

The following section lists the various functions of the CORTEX library.

11.2.1 How to use this driver

How to configure Interrupts using CORTEX HAL driver

This section provides functions allowing to configure the NVIC interrupts (IRQ). The Cortex-M4 exceptions are managed by CMSIS functions.

1. Configure the NVIC Priority Grouping using HAL_NVIC_SetPriorityGrouping() function according to the following table.
2. Configure the priority of the selected IRQ Channels using HAL_NVIC_SetPriority().
3. Enable the selected IRQ Channels using HAL_NVIC_EnableIRQ().
4. please refer to programming manual for details in how to configure priority.

Note: When the NVIC_PRIORITYGROUP_0 is selected, IRQ preemption is no more possible. The pending IRQ priority will be managed only by the sub priority.

Note: IRQ priority order (sorted by highest to lowest priority):

- Lowest preemption priority
- Lowest sub priority
- Lowest hardware priority (IRQ number)

How to configure SysTick using CORTEX HAL driver

Setup SysTick Timer for time base.

- The HAL_SYSTICK_Config() function calls the SysTick_Config() function which is a CMSIS function that:
 - Configures the SysTick Reload register with value passed as function parameter.
 - Configures the SysTick IRQ priority to the lowest value 0x0F.
 - Resets the SysTick Counter register.
 - Configures the SysTick Counter clock source to be Core Clock Source (HCLK).
 - Enables the SysTick Interrupt.
 - Starts the SysTick Counter.
- You can change the SysTick Clock source to be HCLK_Div8 by calling the macro __HAL_CORTEX_SYSTICKCLK_CONFIG(SYSTICK_CLKSOURCE_HCLK_DIV8) just after the HAL_SYSTICK_Config() function call. The __HAL_CORTEX_SYSTICKCLK_CONFIG() macro is defined inside the stm32f4xx_hal_cortex.h file.
- You can change the SysTick IRQ priority by calling the HAL_NVIC_SetPriority(SysTick_IRQn,...) function just after the HAL_SYSTICK_Config() function call. The HAL_NVIC_SetPriority() call the NVIC_SetPriority() function which is a CMSIS function.
- To adjust the SysTick time base, use the following formula: Reload Value = SysTick Counter Clock (Hz) x Desired Time base (s)
 - Reload Value is the parameter to be passed for HAL_SYSTICK_Config() function
 - Reload Value should not exceed 0xFFFFFF

11.2.2 Initialization and de-initialization functions

This section provides the CORTEX HAL driver functions allowing to configure Interrupts SysTick functionalities
This section contains the following APIs:

- **HAL_NVIC_SetPriorityGrouping()**
- **HAL_NVIC_SetPriority()**
- **HAL_NVIC_EnableIRQ()**
- **HAL_NVIC_DisableIRQ()**
- **HAL_NVIC_SystemReset()**
- **HAL_SYSTICK_Config()**

11.2.3 Peripheral Control functions

This subsection provides a set of functions allowing to control the CORTEX (NVIC, SYSTICK, MPU) functionalities.

This section contains the following APIs:

- [*HAL_MPU_Disable\(\)*](#)
- [*HAL_MPU_Enable\(\)*](#)
- [*HAL_MPU_ConfigRegion\(\)*](#)
- [*HAL_NVIC_GetPriorityGrouping\(\)*](#)
- [*HAL_NVIC_GetPriority\(\)*](#)
- [*HAL_NVIC_SetPendingIRQ\(\)*](#)
- [*HAL_NVIC_GetPendingIRQ\(\)*](#)
- [*HAL_NVIC_ClearPendingIRQ\(\)*](#)
- [*HAL_NVIC_GetActive\(\)*](#)
- [*HAL_SYSTICK_CLKSourceConfig\(\)*](#)
- [*HAL_SYSTICK_IRQHandler\(\)*](#)
- [*HAL_SYSTICK_Callback\(\)*](#)

11.2.4 Detailed description of functions

HAL_NVIC_SetPriorityGrouping

Function name

void HAL_NVIC_SetPriorityGrouping (uint32_t PriorityGroup)

Function description

Sets the priority grouping field (preemption priority and subpriority) using the required unlock sequence.

Parameters

- **PriorityGroup:** The priority grouping bits length. This parameter can be one of the following values:
 - NVIC_PRIORITYGROUP_0: 0 bits for preemption priority 4 bits for subpriority
 - NVIC_PRIORITYGROUP_1: 1 bits for preemption priority 3 bits for subpriority
 - NVIC_PRIORITYGROUP_2: 2 bits for preemption priority 2 bits for subpriority
 - NVIC_PRIORITYGROUP_3: 3 bits for preemption priority 1 bits for subpriority
 - NVIC_PRIORITYGROUP_4: 4 bits for preemption priority 0 bits for subpriority

Return values

- **None:**

Notes

- When the NVIC_PriorityGroup_0 is selected, IRQ preemption is no more possible. The pending IRQ priority will be managed only by the subpriority.

HAL_NVIC_SetPriority

Function name

void HAL_NVIC_SetPriority (IRQn_Type IRQn, uint32_t PreemptPriority, uint32_t SubPriority)

Function description

Sets the priority of an interrupt.

Parameters

- **IRQn:** External interrupt number. This parameter can be an enumerator of IRQn_Type enumeration (For the complete STM32 Devices IRQ Channels list, please refer to the appropriate CMSIS device file (stm32f4xxx.h))
- **PreemptPriority:** The preemption priority for the IRQn channel. This parameter can be a value between 0 and 15 A lower priority value indicates a higher priority
- **SubPriority:** the subpriority level for the IRQ channel. This parameter can be a value between 0 and 15 A lower priority value indicates a higher priority.

Return values

- **None:**

HAL_NVIC_EnableIRQ

Function name

void HAL_NVIC_EnableIRQ (IRQn_Type IRQn)

Function description

Enables a device specific interrupt in the NVIC interrupt controller.

Parameters

- **IRQn:** External interrupt number. This parameter can be an enumerator of IRQn_Type enumeration (For the complete STM32 Devices IRQ Channels list, please refer to the appropriate CMSIS device file (stm32f4xxx.h))

Return values

- **None:**

Notes

- To configure interrupts priority correctly, the NVIC_PriorityGroupConfig() function should be called before.

HAL_NVIC_DisableIRQ

Function name

void HAL_NVIC_DisableIRQ (IRQn_Type IRQn)

Function description

Disables a device specific interrupt in the NVIC interrupt controller.

Parameters

- **IRQn:** External interrupt number. This parameter can be an enumerator of IRQn_Type enumeration (For the complete STM32 Devices IRQ Channels list, please refer to the appropriate CMSIS device file (stm32f4xxx.h))

Return values

- **None:**

HAL_NVIC_SystemReset

Function name

void HAL_NVIC_SystemReset (void)

Function description

Initiates a system reset request to reset the MCU.

Return values

- **None:**

HAL_SYSTICK_Config

Function name

uint32_t HAL_SYSTICK_Config (uint32_t TicksNumb)

Function description

Initializes the System Timer and its interrupt, and starts the System Tick Timer.

Parameters

- **TicksNumb:** Specifies the ticks Number of ticks between two interrupts.

Return values

- **status:** - 0 Function succeeded.
– 1 Function failed.

HAL_NVIC_GetPriorityGrouping

Function name

uint32_t HAL_NVIC_GetPriorityGrouping (void)

Function description

Gets the priority grouping field from the NVIC Interrupt Controller.

Return values

- **Priority:** grouping field (SCB->AIRCR [10:8] PRIGROUP field)

HAL_NVIC_GetPriority

Function name

void HAL_NVIC_GetPriority (IRQn_Type IRQn, uint32_t PriorityGroup, uint32_t * pPreemptPriority, uint32_t * pSubPriority)

Function description

Gets the priority of an interrupt.

Parameters

- **IRQn:** External interrupt number. This parameter can be an enumerator of IRQn_Type enumeration (For the complete STM32 Devices IRQ Channels list, please refer to the appropriate CMSIS device file (stm32f4xxx.h))
- **PriorityGroup:** the priority grouping bits length. This parameter can be one of the following values:
 - NVIC_PRIORITYGROUP_0: 0 bits for preemption priority 4 bits for subpriority
 - NVIC_PRIORITYGROUP_1: 1 bits for preemption priority 3 bits for subpriority
 - NVIC_PRIORITYGROUP_2: 2 bits for preemption priority 2 bits for subpriority
 - NVIC_PRIORITYGROUP_3: 3 bits for preemption priority 1 bits for subpriority
 - NVIC_PRIORITYGROUP_4: 4 bits for preemption priority 0 bits for subpriority
- **pPreemptPriority:** Pointer on the Preemptive priority value (starting from 0).
- **pSubPriority:** Pointer on the Subpriority value (starting from 0).

Return values

- **None:**

HAL_NVIC_GetPendingIRQ

Function name

uint32_t HAL_NVIC_GetPendingIRQ (IRQn_Type IRQn)

Function description

Gets Pending Interrupt (reads the pending register in the NVIC and returns the pending bit for the specified interrupt).

Parameters

- **IRQn:** External interrupt number. This parameter can be an enumerator of IRQn_Type enumeration (For the complete STM32 Devices IRQ Channels list, please refer to the appropriate CMSIS device file (stm32f4xxx.h))

Return values

- **status:** - 0 Interrupt status is not pending.
– 1 Interrupt status is pending.

HAL_NVIC_SetPendingIRQ

Function name

void HAL_NVIC_SetPendingIRQ (IRQn_Type IRQn)

Function description

Sets Pending bit of an external interrupt.

Parameters

- **IRQn:** External interrupt number This parameter can be an enumerator of IRQn_Type enumeration (For the complete STM32 Devices IRQ Channels list, please refer to the appropriate CMSIS device file (stm32f4xxx.h))

Return values

- **None:**

HAL_NVIC_ClearPendingIRQ

Function name

void HAL_NVIC_ClearPendingIRQ (IRQn_Type IRQn)

Function description

Clears the pending bit of an external interrupt.

Parameters

- **IRQn:** External interrupt number. This parameter can be an enumerator of IRQn_Type enumeration (For the complete STM32 Devices IRQ Channels list, please refer to the appropriate CMSIS device file (stm32f4xxx.h))

Return values

- **None:**

HAL_NVIC_GetActive

Function name

uint32_t HAL_NVIC_GetActive (IRQn_Type IRQn)

Function description

Gets active interrupt (reads the active register in NVIC and returns the active bit).

Parameters

- **IRQn:** External interrupt number This parameter can be an enumerator of IRQn_Type enumeration (For the complete STM32 Devices IRQ Channels list, please refer to the appropriate CMSIS device file (stm32f4xxx.h))

Return values

- **status:** - 0 Interrupt status is not pending.
 - 1 Interrupt status is pending.

HAL_SYSTICK_CLKSourceConfig

Function name

void HAL_SYSTICK_CLKSourceConfig (uint32_t CLKSource)

Function description

Configures the SysTick clock source.

Parameters

- **CLKSource:** specifies the SysTick clock source. This parameter can be one of the following values:
 - SYSTICK_CLKSOURCE_HCLK_DIV8: AHB clock divided by 8 selected as SysTick clock source.
 - SYSTICK_CLKSOURCE_HCLK: AHB clock selected as SysTick clock source.

Return values

- **None:**

HAL_SYSTICK_IRQHandler

Function name

void HAL_SYSTICK_IRQHandler (void)

Function description

This function handles SYSTICK interrupt request.

Return values

- **None:**

HAL_SYSTICK_Callback

Function name

void HAL_SYSTICK_Callback (void)

Function description

SYSTICK callback.

Return values

- **None:**

HAL_MPU_Enable

Function name

void HAL_MPU_Enable (uint32_t MPU_Control)

Function description

Enable the MPU.

Parameters

- **MPU_Control:** Specifies the control mode of the MPU during hard fault, NMI, FAULTMASK and privileged access to the default memory This parameter can be one of the following values:
 - MPU_HFNMI_PRIVDEF_NONE
 - MPU_HARDFAULT_NMI
 - MPU_PRIVILEGED_DEFAULT
 - MPU_HFNMI_PRIVDEF

Return values

- **None:**

HAL_MPU_Disable

Function name

void HAL_MPU_Disable (void)

Function description

Disables the MPU.

Return values

- **None:**

HAL_MPU_ConfigRegion

Function name

void HAL_MPU_ConfigRegion (MPU_Region_InitTypeDef * MPU_Init)

Function description

Initializes and configures the Region and the memory to be protected.

Parameters

- **MPU_Init:** Pointer to a MPU_Region_InitTypeDef structure that contains the initialization and configuration information.

Return values

- **None:**

11.3 CORTEx Firmware driver defines

The following section lists the various define and macros of the module.

11.3.1 CORTEx

CORTEx

CORTEx MPU Instruction Access Bufferable

MPU_ACCESS_BUFFERABLE

MPU_ACCESS_NOT_BUFFERABLE

CORTEx MPU Instruction Access Cacheable

MPU_ACCESS_CACHEABLE

MPU_ACCESS_NOT_CACHEABLE

CORTEx MPU Instruction Access Shareable

MPU_ACCESS_SHAREABLE

MPU_ACCESS_NOT_SHAREABLE

MPU HFNMI and PRIVILEGED Access control

MPU_HFNMI_PRIVDEF_NONE

MPU_HARDFAULT_NMI

MPU_PRIVILEGED_DEFAULT

MPU_HFNMI_PRIVDEF

CORTEx MPU Instruction Access

MPU_INSTRUCTION_ACCESS_ENABLE

MPU_INSTRUCTION_ACCESS_DISABLE

CORTEx MPU Region Enable

MPU_REGION_ENABLE

MPU_REGION_DISABLE

CORTEx MPU Region Number

MPU_REGION_NUMBER0

MPU_REGION_NUMBER1

MPU_REGION_NUMBER2

MPU_REGION_NUMBER3

MPU_REGION_NUMBER4

MPU_REGION_NUMBER5

MPU_REGION_NUMBER6

MPU_REGION_NUMBER7

CORTEx MPU Region Permission Attributes

MPU_REGION_NO_ACCESS

MPU_REGION_PRIV_RW

MPU_REGION_PRIV_RW_URO

MPU_REGION_FULL_ACCESS

MPU_REGION_PRIV_RO

MPU_REGION_PRIV_RO_URO

CORTEx MPU Region Size

MPU_REGION_SIZE_32B

MPU_REGION_SIZE_64B

MPU_REGION_SIZE_128B

MPU_REGION_SIZE_256B

MPU_REGION_SIZE_512B

MPU_REGION_SIZE_1KB

MPU_REGION_SIZE_2KB

MPU_REGION_SIZE_4KB

MPU_REGION_SIZE_8KB

MPU_REGION_SIZE_16KB

MPU_REGION_SIZE_32KB

MPU_REGION_SIZE_64KB

MPU_REGION_SIZE_128KB

MPU_REGION_SIZE_256KB

MPU_REGION_SIZE_512KB

MPU_REGION_SIZE_1MB

MPU_REGION_SIZE_2MB

MPU_REGION_SIZE_4MB

MPU_REGION_SIZE_8MB

MPU_REGION_SIZE_16MB

MPU_REGION_SIZE_32MB

MPU_REGION_SIZE_64MB

MPU_REGION_SIZE_128MB

MPU_REGION_SIZE_256MB

MPU_REGION_SIZE_512MB

MPU_REGION_SIZE_1GB

MPU_REGION_SIZE_2GB

MPU_REGION_SIZE_4GB

MPU TEX Levels

MPU_TEX_LEVEL0

MPU_TEX_LEVEL1

MPU_TEX_LEVEL2

CORTEx Preemption Priority Group

NVIC_PRIORITYGROUP_0

0 bits for pre-emption priority 4 bits for subpriority

NVIC_PRIORITYGROUP_1

1 bits for pre-emption priority 3 bits for subpriority

NVIC_PRIORITYGROUP_2

2 bits for pre-emption priority 2 bits for subpriority

NVIC_PRIORITYGROUP_3

3 bits for pre-emption priority 1 bits for subpriority

NVIC_PRIORITYGROUP_4

4 bits for pre-emption priority 0 bits for subpriority

CORTEx _SysTick clock source

SYSTICK_CLKSOURCE_HCLK_DIV8

SYSTICK_CLKSOURCE_HCLK

12 HAL CRC Generic Driver

12.1 CRC Firmware driver registers structures

12.1.1 CRC_HandleTypeDef

CRC_HandleTypeDef is defined in the stm32f4xx_hal_crc.h

Data Fields

- **CRC_TypeDef* Instance**
- **HAL_LockTypeDef Lock**
- **__IO HAL_CRC_StateTypeDef State**

Field Documentation

- **CRC_TypeDef* CRC_HandleTypeDef::Instance**
Register base address
- **HAL_LockTypeDef CRC_HandleTypeDef::Lock**
CRC Locking object
- **__IO HAL_CRC_StateTypeDef CRC_HandleTypeDef::State**
CRC communication state

12.2 CRC Firmware driver API description

The following section lists the various functions of the CRC library.

12.2.1 How to use this driver

- Enable CRC AHB clock using `__HAL_RCC_CRC_CLK_ENABLE();`
- Initialize CRC calculator
 - specify generating polynomial (peripheral default or non-default one)
 - specify initialization value (peripheral default or non-default one)
 - specify input data format
 - specify input or output data inversion mode if any
- Use `HAL_CRC_Accumulate()` function to compute the CRC value of the input data buffer starting with the previously computed CRC as initialization value
- Use `HAL_CRC_Calculate()` function to compute the CRC value of the input data buffer starting with the defined initialization value (default or non-default) to initiate CRC calculation

12.2.2 Initialization and de-initialization functions

This section provides functions allowing to:

- Initialize the CRC according to the specified parameters in the `CRC_InitTypeDef` and create the associated handle
- DeInitialize the CRC peripheral
- Initialize the CRC MSP (MCU Specific Package)
- DeInitialize the CRC MSP

This section contains the following APIs:

- **HAL_CRC_Init()**
- **HAL_CRC_DeInit()**
- **HAL_CRC_MspInit()**
- **HAL_CRC_MspDeInit()**

12.2.3 Peripheral Control functions

This section provides functions allowing to:

- compute the 32-bit CRC value of a 32-bit data buffer using combination of the previous CRC value and the new one.

or

- compute the 32-bit CRC value of a 32-bit data buffer independently of the previous CRC value.

This section contains the following APIs:

- [HAL_CRC_Accumulate\(\)](#)
- [HAL_CRC_Calculate\(\)](#)

12.2.4 Peripheral State functions

This subsection permits to get in run-time the status of the peripheral.

This section contains the following APIs:

- [HAL_CRC_GetState\(\)](#)

12.2.5 Detailed description of functions

HAL_CRC_Init

Function name

HAL_StatusTypeDef HAL_CRC_Init (CRC_HandleTypeDef * hcrc)

Function description

Initialize the CRC according to the specified parameters in the CRC_InitTypeDef and create the associated handle.

Parameters

- **hcrc**: CRC handle

Return values

- **HAL**: status

HAL_CRC_DeInit

Function name

HAL_StatusTypeDef HAL_CRC_DeInit (CRC_HandleTypeDef * hcrc)

Function description

Deinitialize the CRC peripheral.

Parameters

- **hcrc**: CRC handle

Return values

- **HAL**: status

HAL_CRC_MspInit

Function name

void HAL_CRC_MspInit (CRC_HandleTypeDef * hcrc)

Function description

Initializes the CRC MSP.

Parameters

- **hcrc**: CRC handle

Return values

- **None:**

HAL_CRC_MspDeInit

Function name

void HAL_CRC_MspDeInit (CRC_HandleTypeDef * hcrc)

Function description

DeInitialize the CRC MSP.

Parameters

- **hcrc:** CRC handle

Return values

- **None:**

HAL_CRC_Accumulate

Function name

uint32_t HAL_CRC_Accumulate (CRC_HandleTypeDef * hcrc, uint32_t pBuffer, uint32_t BufferLength)

Function description

Compute the 32-bit CRC value of a 32-bit data buffer starting with the previously computed CRC as initialization value.

Parameters

- **hcrc:** CRC handle
- **pBuffer:** pointer to the input data buffer.
- **BufferLength:** input data buffer length (number of uint32_t words).

Return values

- **uint32_t:** CRC (returned value LSBs for CRC shorter than 32 bits)

HAL_CRC_Calculate

Function name

uint32_t HAL_CRC_Calculate (CRC_HandleTypeDef * hcrc, uint32_t pBuffer, uint32_t BufferLength)

Function description

Compute the 32-bit CRC value of a 32-bit data buffer starting with hcrc->Instance->INIT as initialization value.

Parameters

- **hcrc:** CRC handle
- **pBuffer:** pointer to the input data buffer.
- **BufferLength:** input data buffer length (number of uint32_t words).

Return values

- **uint32_t:** CRC (returned value LSBs for CRC shorter than 32 bits)

HAL_CRC_GetState

Function name

HAL_CRC_StateTypeDef HAL_CRC_GetState (CRC_HandleTypeDef * hcrc)

Function description

Return the CRC handle state.

Parameters

- **hcrc:** CRC handle

Return values

- **HAL:** state

12.3 CRC Firmware driver defines

The following section lists the various define and macros of the module.

12.3.1 CRC

CRC

CRC Exported Macros

__HAL_CRC_RESET_HANDLE_STATE

Description:

- Reset CRC handle state.

Parameters:

- **__HANDLE__:** CRC handle.

Return value:

- None

__HAL_CRC_DR_RESET

Description:

- Reset CRC Data Register.

Parameters:

- **__HANDLE__:** CRC handle

Return value:

- None

__HAL_CRC_SET_IDR

Description:

- Store data in the Independent Data (ID) register.

Parameters:

- **__HANDLE__:** CRC handle
- **__VALUE__:** Value to be stored in the ID register

Return value:

- None

Notes:

- Refer to the Reference Manual to get the authorized **__VALUE__** length in bits

__HAL_CRC_GET_IDR

Description:

- Return the data stored in the Independent Data (ID) register.

Parameters:

- **__HANDLE__:** CRC handle

Return value:

- Value: of the ID register

Notes:

- Refer to the Reference Manual to get the authorized **__VALUE__** length in bits

21 HAL DMA Generic Driver

21.1 DMA Firmware driver registers structures

21.1.1 DMA_InitTypeDef

DMA_InitTypeDef is defined in the `stm32f4xx_hal_dma.h`

Data Fields

- **`uint32_t Channel`**
- **`uint32_t Direction`**
- **`uint32_t PeriphInc`**
- **`uint32_t MemInc`**
- **`uint32_t PeriphDataAlignment`**
- **`uint32_t MemDataAlignment`**
- **`uint32_t Mode`**
- **`uint32_t Priority`**
- **`uint32_t FIFOmode`**
- **`uint32_t FIFOThreshold`**
- **`uint32_t MemBurst`**
- **`uint32_t PeriphBurst`**

Field Documentation

- **`uint32_t DMA_InitTypeDef::Channel`**
Specifies the channel used for the specified stream. This parameter can be a value of [DMA_Channel_selection](#)
- **`uint32_t DMA_InitTypeDef::Direction`**
Specifies if the data will be transferred from memory to peripheral, from memory to memory or from peripheral to memory. This parameter can be a value of [DMA_Data_transfer_direction](#)
- **`uint32_t DMA_InitTypeDef::PeriphInc`**
Specifies whether the Peripheral address register should be incremented or not. This parameter can be a value of [DMA_Peripheral_incremented_mode](#)
- **`uint32_t DMA_InitTypeDef::MemInc`**
Specifies whether the memory address register should be incremented or not. This parameter can be a value of [DMA_Memory_incremented_mode](#)
- **`uint32_t DMA_InitTypeDef::PeriphDataAlignment`**
Specifies the Peripheral data width. This parameter can be a value of [DMA_Peripheral_data_size](#)
- **`uint32_t DMA_InitTypeDef::MemDataAlignment`**
Specifies the Memory data width. This parameter can be a value of [DMA_Memory_data_size](#)
- **`uint32_t DMA_InitTypeDef::Mode`**
Specifies the operation mode of the DMAy Streamx. This parameter can be a value of [DMA_mode](#)
Note:
 - The circular buffer mode cannot be used if the memory-to-memory data transfer is configured on the selected Stream
- **`uint32_t DMA_InitTypeDef::Priority`**
Specifies the software priority for the DMAy Streamx. This parameter can be a value of [DMA_Priority_level](#)
- **`uint32_t DMA_InitTypeDef::FIFOmode`**
Specifies if the FIFO mode or Direct mode will be used for the specified stream. This parameter can be a value of [DMA_FIFO_direct_mode](#)
Note:
 - The Direct mode (FIFO mode disabled) cannot be used if the memory-to-memory data transfer is configured on the selected stream
- **`uint32_t DMA_InitTypeDef::FIFOThreshold`**
Specifies the FIFO threshold level. This parameter can be a value of [DMA_FIFO_threshold_level](#)

- **`uint32_t DMA_InitTypeDef::MemBurst`**
Specifies the Burst transfer configuration for the memory transfers. It specifies the amount of data to be transferred in a single non interruptible transaction. This parameter can be a value of [DMA_Memory_burst](#)
Note:
 - The burst mode is possible only if the address Increment mode is enabled.
- **`uint32_t DMA_InitTypeDef::PeriphBurst`**
Specifies the Burst transfer configuration for the peripheral transfers. It specifies the amount of data to be transferred in a single non interruptible transaction. This parameter can be a value of [DMA_Peripheral_burst](#)
Note:
 - The burst mode is possible only if the address Increment mode is enabled.

21.1.2 __DMA_HandleTypeDef

`__DMA_HandleTypeDef` is defined in the `stm32f4xx_hal_dma.h`

Data Fields

- **`DMA_Stream_TypeDef * Instance`**
- **`DMA_InitTypeDef Init`**
- **`HAL_LockTypeDef Lock`**
- **`__IO HAL_DMA_StateTypeDef State`**
- **`void * Parent`**
- **`void(* XferCpltCallback`**
- **`void(* XferHalfCpltCallback`**
- **`void(* XferM1CpltCallback`**
- **`void(* XferM1HalfCpltCallback`**
- **`void(* XferErrorCallback`**
- **`void(* XferAbortCallback`**
- **`__IO uint32_t ErrorCode`**
- **`uint32_t StreamBaseAddress`**
- **`uint32_t StreamIndex`**

Field Documentation

- **`DMA_Stream_TypeDef* __DMA_HandleTypeDef::Instance`**
Register base address
- **`DMA_InitTypeDef __DMA_HandleTypeDef::Init`**
DMA communication parameters
- **`HAL_LockTypeDef __DMA_HandleTypeDef::Lock`**
DMA locking object
- **`__IO HAL_DMA_StateTypeDef __DMA_HandleTypeDef::State`**
DMA transfer state
- **`void* __DMA_HandleTypeDef::Parent`**
Parent object state
- **`void(* __DMA_HandleTypeDef::XferCpltCallback)(struct __DMA_HandleTypeDef *hdma)`**
DMA transfer complete callback
- **`void(* __DMA_HandleTypeDef::XferHalfCpltCallback)(struct __DMA_HandleTypeDef *hdma)`**
DMA Half transfer complete callback
- **`void(* __DMA_HandleTypeDef::XferM1CpltCallback)(struct __DMA_HandleTypeDef *hdma)`**
DMA transfer complete Memory1 callback
- **`void(* __DMA_HandleTypeDef::XferM1HalfCpltCallback)(struct __DMA_HandleTypeDef *hdma)`**
DMA transfer Half complete Memory1 callback
- **`void(* __DMA_HandleTypeDef::XferErrorCallback)(struct __DMA_HandleTypeDef *hdma)`**
DMA transfer error callback
- **`void(* __DMA_HandleTypeDef::XferAbortCallback)(struct __DMA_HandleTypeDef *hdma)`**
DMA transfer Abort callback
- **`__IO uint32_t __DMA_HandleTypeDef::ErrorCode`**
DMA Error code

- `uint32_t __DMA_HandleTypeDef::StreamBaseAddress`
DMA Stream Base Address
- `uint32_t __DMA_HandleTypeDef::StreamIndex`
DMA Stream Index

21.2 DMA Firmware driver API description

The following section lists the various functions of the DMA library.

21.2.1 How to use this driver

1. Enable and configure the peripheral to be connected to the DMA Stream (except for internal SRAM/ FLASH memories: no initialization is necessary) please refer to Reference manual for connection between peripherals and DMA requests.
2. For a given Stream, program the required configuration through the following parameters: Transfer Direction, Source and Destination data formats, Circular, Normal or peripheral flow control mode, Stream Priority level, Source and Destination Increment mode, FIFO mode and its Threshold (if needed), Burst mode for Source and/or Destination (if needed) using `HAL_DMA_Init()` function.

Note: *Prior to `HAL_DMA_Init()` the clock must be enabled for DMA through the following macros: `__HAL_RCC_DMA1_CLK_ENABLE()` or `__HAL_RCC_DMA2_CLK_ENABLE()`.*

Polling mode IO operation

- Use `HAL_DMA_Start()` to start DMA transfer after the configuration of Source address and destination address and the Length of data to be transferred.
- Use `HAL_DMA_PollForTransfer()` to poll for the end of current transfer, in this case a fixed Timeout can be configured by User depending from his application.
- Use `HAL_DMA_Abort()` function to abort the current transfer.

Interrupt mode IO operation

- Configure the DMA interrupt priority using `HAL_NVIC_SetPriority()`
 - Enable the DMA IRQ handler using `HAL_NVIC_EnableIRQ()`
 - Use `HAL_DMA_Start_IT()` to start DMA transfer after the configuration of Source address and destination address and the Length of data to be transferred. In this case the DMA interrupt is configured
 - Use `HAL_DMA_IRQHandler()` called under `DMA_IRQHandler()` Interrupt subroutine
 - At the end of data transfer `HAL_DMA_IRQHandler()` function is executed and user can add his own function by customization of function pointer `XferCpltCallback` and `XferErrorCallback` (i.e a member of DMA handle structure).
1. Use `HAL_DMA_GetState()` function to return the DMA state and `HAL_DMA_GetError()` in case of error detection.
 2. Use `HAL_DMA_Abort_IT()` function to abort the current transfer

Note: *In Memory-to-Memory transfer mode, Circular mode is not allowed.*

Note: *The FIFO is used mainly to reduce bus usage and to allow data packing/unpacking: it is possible to set different Data Sizes for the Peripheral and the Memory (ie. you can set Half-Word data size for the peripheral to access its data register and set Word data size for the Memory to gain in access time. Each two half words will be packed and written in a single access to a Word in the Memory).*

Note: *When FIFO is disabled, it is not allowed to configure different Data Sizes for Source and Destination. In this case the Peripheral Data Size will be applied to both Source and Destination.*

DMA HAL driver macros list

Below the list of most used macros in DMA HAL driver.

- `__HAL_DMA_ENABLE`: Enable the specified DMA Stream.
- `__HAL_DMA_DISABLE`: Disable the specified DMA Stream.
- `__HAL_DMA_GET_IT_SOURCE`: Check whether the specified DMA Stream interrupt has occurred or not.

Note: *You can refer to the DMA HAL driver header file for more useful macros*

21.2.2 Initialization and de-initialization functions

This section provides functions allowing to initialize the DMA Stream source and destination addresses, incrementation and data sizes, transfer direction, circular/normal mode selection, memory-to-memory mode selection and Stream priority value.

The HAL_DMA_Init() function follows the DMA configuration procedures as described in reference manual.

This section contains the following APIs:

- [HAL_DMA_Init\(\)](#)
- [HAL_DMA_DeInit\(\)](#)

21.2.3 IO operation functions

This section provides functions allowing to:

- Configure the source, destination address and data length and Start DMA transfer
- Configure the source, destination address and data length and Start DMA transfer with interrupt
- Abort DMA transfer
- Poll for transfer complete
- Handle DMA interrupt request

This section contains the following APIs:

- [HAL_DMA_Start\(\)](#)
- [HAL_DMA_Start_IT\(\)](#)
- [HAL_DMA_Abort\(\)](#)
- [HAL_DMA_Abort_IT\(\)](#)
- [HAL_DMA_PollForTransfer\(\)](#)
- [HAL_DMA_IRQHandler\(\)](#)
- [HAL_DMA_RegisterCallback\(\)](#)
- [HAL_DMA_UnRegisterCallback\(\)](#)
- [HAL_DMA_CleanCallbacks\(\)](#)

21.2.4 State and Errors functions

This subsection provides functions allowing to

- Check the DMA state
- Get error code

This section contains the following APIs:

- [HAL_DMA_GetState\(\)](#)
- [HAL_DMA_GetError\(\)](#)

21.2.5 Detailed description of functions

HAL_DMA_Init

Function name

HAL_StatusTypeDef HAL_DMA_Init (DMA_HandleTypeDef * hdma)

Function description

Initialize the DMA according to the specified parameters in the DMA_InitTypeDef and create the associated handle.

Parameters

- **hdma:** Pointer to a DMA_HandleTypeDef structure that contains the configuration information for the specified DMA Stream.

Return values

- **HAL:** status

HAL_DMA_DeInit

Function name

HAL_StatusTypeDef HAL_DMA_DeInit (DMA_HandleTypeDef * hdma)

Function description

DeInitializes the DMA peripheral.

Parameters

- **hdma:** pointer to a DMA_HandleTypeDef structure that contains the configuration information for the specified DMA Stream.

Return values

- **HAL:** status

HAL_DMA_Start

Function name

HAL_StatusTypeDef HAL_DMA_Start (DMA_HandleTypeDef * hdma, uint32_t SrcAddress, uint32_t DstAddress, uint32_t DataLength)

Function description

Starts the DMA Transfer.

Parameters

- **hdma:** pointer to a DMA_HandleTypeDef structure that contains the configuration information for the specified DMA Stream.
- **SrcAddress:** The source memory Buffer address
- **DstAddress:** The destination memory Buffer address
- **DataLength:** The length of data to be transferred from source to destination

Return values

- **HAL:** status

HAL_DMA_Start_IT

Function name

HAL_StatusTypeDef HAL_DMA_Start_IT (DMA_HandleTypeDef * hdma, uint32_t SrcAddress, uint32_t DstAddress, uint32_t DataLength)

Function description

Start the DMA Transfer with interrupt enabled.

Parameters

- **hdma:** pointer to a DMA_HandleTypeDef structure that contains the configuration information for the specified DMA Stream.
- **SrcAddress:** The source memory Buffer address
- **DstAddress:** The destination memory Buffer address
- **DataLength:** The length of data to be transferred from source to destination

Return values

- **HAL:** status

HAL_DMA_Abort

Function name

HAL_StatusTypeDef HAL_DMA_Abort (DMA_HandleTypeDef * hdma)

Function description

Aborts the DMA Transfer.

Parameters

- **hdma:** pointer to a DMA_HandleTypeDef structure that contains the configuration information for the specified DMA Stream.

Return values

- **HAL:** status

Notes

- After disabling a DMA Stream, a check for wait until the DMA Stream is effectively disabled is added. If a Stream is disabled while a data transfer is ongoing, the current data will be transferred and the Stream will be effectively disabled only after the transfer of this single data is finished.

HAL_DMA_Abort_IT

Function name

HAL_StatusTypeDef HAL_DMA_Abort_IT (DMA_HandleTypeDef * hdma)

Function description

Aborts the DMA Transfer in Interrupt mode.

Parameters

- **hdma:** pointer to a DMA_HandleTypeDef structure that contains the configuration information for the specified DMA Stream.

Return values

- **HAL:** status

HAL_DMA_PollForTransfer

Function name

HAL_StatusTypeDef HAL_DMA_PollForTransfer (DMA_HandleTypeDef * hdma, HAL_DMA_LevelCompleteTypeDef CompleteLevel, uint32_t Timeout)

Function description

Polling for transfer complete.

Parameters

- **hdma:** pointer to a DMA_HandleTypeDef structure that contains the configuration information for the specified DMA Stream.
- **CompleteLevel:** Specifies the DMA level complete.
- **Timeout:** Timeout duration.

Return values

- **HAL:** status

Notes

- The polling mode is kept in this version for legacy. it is recommended to use the IT model instead. This model could be used for debug purpose.
- The HAL_DMA_PollForTransfer API cannot be used in circular and double buffering mode (automatic circular mode).

HAL_DMA_IRQHandler

Function name

void HAL_DMA_IRQHandler (DMA_HandleTypeDef * hdma)

Function description

Handles DMA interrupt request.

Parameters

- **hdma**: pointer to a DMA_HandleTypeDef structure that contains the configuration information for the specified DMA Stream.

Return values

- **None**:

HAL_DMA_CleanCallbacks

Function name

HAL_StatusTypeDef HAL_DMA_CleanCallbacks (DMA_HandleTypeDef * hdma)

Function description

HAL_DMA_RegisterCallback

Function name

HAL_StatusTypeDef HAL_DMA_RegisterCallback (DMA_HandleTypeDef * hdma, HAL_DMA_CallbackIDTypeDef CallbackID, void(*) (DMA_HandleTypeDef * _hdma) pCallback)

Function description

Register callbacks.

Parameters

- **hdma**: pointer to a DMA_HandleTypeDef structure that contains the configuration information for the specified DMA Stream.
- **CallbackID**: User Callback identifier a DMA_HandleTypeDef structure as parameter.
- **pCallback**: pointer to private callback function which has pointer to a DMA_HandleTypeDef structure as parameter.

Return values

- **HAL**: status

HAL_DMA_UnRegisterCallback

Function name

HAL_StatusTypeDef HAL_DMA_UnRegisterCallback (DMA_HandleTypeDef * hdma, HAL_DMA_CallbackIDTypeDef CallbackID)

Function description

UnRegister callbacks.

Parameters

- **hdma**: pointer to a DMA_HandleTypeDef structure that contains the configuration information for the specified DMA Stream.
- **CallbackID**: User Callback identifier a HAL_DMA_CallbackIDTypeDef ENUM as parameter.

Return values

- **HAL**: status

HAL_DMA_GetState

Function name

HAL_DMA_StateTypeDef HAL_DMA_GetState (DMA_HandleTypeDef * hdma)

Function description

Returns the DMA state.

Parameters

- **hdma**: pointer to a DMA_HandleTypeDef structure that contains the configuration information for the specified DMA Stream.

Return values

- **HAL**: state

HAL_DMA_GetError

Function name

uint32_t HAL_DMA_GetError (DMA_HandleTypeDef * hdma)

Function description

Return the DMA error code.

Parameters

- **hdma**: pointer to a DMA_HandleTypeDef structure that contains the configuration information for the specified DMA Stream.

Return values

- **DMA**: Error Code

21.3 DMA Firmware driver defines

The following section lists the various define and macros of the module.

21.3.1 DMA

DMA

DMA Channel selection

DMA_CHANNEL_0

DMA Channel 0

DMA_CHANNEL_1

DMA Channel 1

DMA_CHANNEL_2

DMA Channel 2

DMA_CHANNEL_3

DMA Channel 3

DMA_CHANNEL_4

DMA Channel 4

DMA_CHANNEL_5

DMA Channel 5

DMA_CHANNEL_6

DMA Channel 6

DMA_CHANNEL_7

DMA Channel 7

DMA Data transfer direction

DMA_PERIPH_TO_MEMORY

Peripheral to memory direction

DMA_MEMORY_TO_PERIPH

Memory to peripheral direction

DMA_MEMORY_TO_MEMORY

Memory to memory direction

DMA Error Code**HAL_DMA_ERROR_NONE**

No error

HAL_DMA_ERROR_TE

Transfer error

HAL_DMA_ERROR_FE

FIFO error

HAL_DMA_ERROR_DME

Direct Mode error

HAL_DMA_ERROR_TIMEOUT

Timeout error

HAL_DMA_ERROR_PARAM

Parameter error

HAL_DMA_ERROR_NO_XFER

Abort requested with no Xfer ongoing

HAL_DMA_ERROR_NOT_SUPPORTED

Not supported mode

DMA FIFO direct mode**DMA_FIFOMODE_DISABLE**

FIFO mode disable

DMA_FIFOMODE_ENABLE

FIFO mode enable

DMA FIFO threshold level**DMA_FIFO_THRESHOLD_1QUARTERFULL**

FIFO threshold 1 quart full configuration

DMA_FIFO_THRESHOLD_HALFFULL

FIFO threshold half full configuration

DMA_FIFO_THRESHOLD_3QUARTERSFULL

FIFO threshold 3 quarts full configuration

DMA_FIFO_THRESHOLD_FULL

FIFO threshold full configuration

DMA flag definitions

DMA_FLAG_FEIF0_4

DMA_FLAG_DMEIF0_4

DMA_FLAG_TEIF0_4

DMA_FLAG_HTIF0_4

DMA_FLAG_TCIF0_4

DMA_FLAG_FEIF1_5

DMA_FLAG_DMEIF1_5

DMA_FLAG_TEIF1_5

DMA_FLAG_HTIF1_5

DMA_FLAG_TCIF1_5

DMA_FLAG_FEIF2_6

DMA_FLAG_DMEIF2_6

DMA_FLAG_TEIF2_6

DMA_FLAG_HTIF2_6

DMA_FLAG_TCIF2_6

DMA_FLAG_FEIF3_7

DMA_FLAG_DMEIF3_7

DMA_FLAG_TEIF3_7

DMA_FLAG_HTIF3_7

DMA_FLAG_TCIF3_7

TIM DMA Handle Index**TIM_DMA_ID_UPDATE**

Index of the DMA handle used for Update DMA requests

TIM_DMA_ID_CC1

Index of the DMA handle used for Capture/Compare 1 DMA requests

TIM_DMA_ID_CC2

Index of the DMA handle used for Capture/Compare 2 DMA requests

TIM_DMA_ID_CC3

Index of the DMA handle used for Capture/Compare 3 DMA requests

TIM_DMA_ID_CC4

Index of the DMA handle used for Capture/Compare 4 DMA requests

TIM_DMA_ID_COMMUTATION

Index of the DMA handle used for Commutation DMA requests

TIM_DMA_ID_TRIGGER

Index of the DMA handle used for Trigger DMA requests

DMA interrupt enable definitions**DMA_IT_TC****DMA_IT_HT****DMA_IT_TE****DMA_IT_DME****DMA_IT_FE*****DMA Memory burst*****DMA_MBURST_SINGLE****DMA_MBURST_INC4****DMA_MBURST_INC8****DMA_MBURST_INC16*****DMA Memory data size*****DMA_MDATAALIGN_BYTE**

Memory data alignment: Byte

DMA_MDATAALIGN_HALFWORD

Memory data alignment: HalfWord

DMA_MDATAALIGN_WORD

Memory data alignment: Word

DMA Memory incremented mode**DMA_MINC_ENABLE**

Memory increment mode enable

DMA_MINC_DISABLE

Memory increment mode disable

DMA mode**DMA_NORMAL**

Normal mode

DMA_CIRCULAR

Circular mode

DMA_PFCTRL

Peripheral flow control mode

DMA Peripheral burst**DMA_PBURST_SINGLE****DMA_PBURST_INC4****DMA_PBURST_INC8****DMA_PBURST_INC16*****DMA Peripheral data size*****DMA_PDATAALIGN_BYTE**

Peripheral data alignment: Byte

DMA_PDATAALIGN_HALFWORD

Peripheral data alignment: HalfWord

DMA_PDATAALIGN_WORD

Peripheral data alignment: Word

DMA Peripheral incremented mode**DMA_PINC_ENABLE**

Peripheral increment mode enable

DMA_PINC_DISABLE

Peripheral increment mode disable

DMA Priority level**DMA_PRIORITY_LOW**

Priority level: Low

DMA_PRIORITY_MEDIUM

Priority level: Medium

DMA_PRIORITY_HIGH

Priority level: High

DMA_PRIORITY_VERY_HIGH

Priority level: Very High

22 HAL DMA Extension Driver

22.1 DMAEx Firmware driver API description

The following section lists the various functions of the DMAEx library.

22.1.1 How to use this driver

The DMA Extension HAL driver can be used as follows:

1. Start a multi buffer transfer using the `HAL_DMA_MultiBufferStart()` function for polling mode or `HAL_DMA_MultiBufferStart_IT()` for interrupt mode.

Note: In Memory-to-Memory transfer mode, Multi (Double) Buffer mode is not allowed.

Note: When Multi (Double) Buffer mode is enabled the, transfer is circular by default.

Note: In Multi (Double) buffer mode, it is possible to update the base address for the AHB memory port on the fly (`DMA_SxM0AR` or `DMA_SxM1AR`) when the stream is enabled.

22.1.2 Extended features functions

This section provides functions allowing to:

- Configure the source, destination address and data length and Start MultiBuffer DMA transfer
- Configure the source, destination address and data length and Start MultiBuffer DMA transfer with interrupt
- Change on the fly the memory0 or memory1 address.

This section contains the following APIs:

- [`HAL\_DMAEx\_MultiBufferStart\(\)`](#)
- [`HAL\_DMAEx\_MultiBufferStart\_IT\(\)`](#)
- [`HAL\_DMAEx\_ChangeMemory\(\)`](#)

22.1.3 Detailed description of functions

HAL_DMAEx_MultiBufferStart

Function name

`HAL_StatusTypeDef HAL_DMAEx_MultiBufferStart (DMA_HandleTypeDef * hdma, uint32_t SrcAddress, uint32_t DstAddress, uint32_t SecondMemAddress, uint32_t DataLength)`

Function description

Starts the multi_buffer DMA Transfer.

Parameters

- **hdma:** pointer to a `DMA_HandleTypeDef` structure that contains the configuration information for the specified DMA Stream.
- **SrcAddress:** The source memory Buffer address
- **DstAddress:** The destination memory Buffer address
- **SecondMemAddress:** The second memory Buffer address in case of multi buffer Transfer
- **DataLength:** The length of data to be transferred from source to destination

Return values

- **HAL:** status

HAL_DMAEx_MultiBufferStart_IT

Function name

`HAL_StatusTypeDef HAL_DMAEx_MultiBufferStart_IT (DMA_HandleTypeDef * hdma, uint32_t SrcAddress, uint32_t DstAddress, uint32_t SecondMemAddress, uint32_t DataLength)`

Function description

Starts the multi_buffer DMA Transfer with interrupt enabled.

Parameters

- **hdma:** pointer to a DMA_HandleTypeDef structure that contains the configuration information for the specified DMA Stream.
- **SrcAddress:** The source memory Buffer address
- **DstAddress:** The destination memory Buffer address
- **SecondMemAddress:** The second memory Buffer address in case of multi buffer Transfer
- **DataLength:** The length of data to be transferred from source to destination

Return values

- **HAL:** status

HAL_DMAEx_ChangeMemory

Function name

HAL_StatusTypeDef HAL_DMAEx_ChangeMemory (DMA_HandleTypeDef * hdma, uint32_t Address, HAL_DMA_MemoryTypeDef memory)

Function description

Change the memory0 or memory1 address on the fly.

Parameters

- **hdma:** pointer to a DMA_HandleTypeDef structure that contains the configuration information for the specified DMA Stream.
- **Address:** The new address
- **memory:** the memory to be changed, This parameter can be one of the following values: MEMORY0 / MEMORY1

Return values

- **HAL:** status

Notes

- The MEMORY0 address can be changed only when the current transfer use MEMORY1 and the MEMORY1 address can be changed only when the current transfer use MEMORY0.

24 HAL EXTI Generic Driver

24.1 EXTI Firmware driver registers structures

24.1.1 EXTI_HandleTypeDef

EXTI_HandleTypeDef is defined in the stm32f4xx_hal_exti.h

Data Fields

- **uint32_t Line**
- **void(* PendingCallback**

Field Documentation

- **uint32_t EXTI_HandleTypeDef::Line**
Exti line number
- **void(* EXTI_HandleTypeDef::PendingCallback)(void)**
Exti pending callback

24.1.2 EXTI_ConfigTypeDef

EXTI_ConfigTypeDef is defined in the stm32f4xx_hal_exti.h

Data Fields

- **uint32_t Line**
- **uint32_t Mode**
- **uint32_t Trigger**
- **uint32_t GPIOSel**

Field Documentation

- **uint32_t EXTI_ConfigTypeDef::Line**
The Exti line to be configured. This parameter can be a value of [EXTI_Line](#)
- **uint32_t EXTI_ConfigTypeDef::Mode**
The Exit Mode to be configured for a core. This parameter can be a combination of [EXTI_Mode](#)
- **uint32_t EXTI_ConfigTypeDef::Trigger**
The Exti Trigger to be configured. This parameter can be a value of [EXTI_Trigger](#)
- **uint32_t EXTI_ConfigTypeDef::GPIOSel**
The Exti GPIO multiplexer selection to be configured. This parameter is only possible for line 0 to 15. It can be a value of [EXTI_GPIOSel](#)

24.2 EXTI Firmware driver API description

The following section lists the various functions of the EXTI library.

24.2.1 EXTI Peripheral features

- Each Exti line can be configured within this driver.
- Exti line can be configured in 3 different modes
 - Interrupt
 - Event
 - Both of them
- Configurable Exti lines can be configured with 3 different triggers
 - Rising
 - Falling
 - Both of them

- When set in interrupt mode, configurable Exti lines have two different interrupts pending registers which allow to distinguish which transition occurs:
 - Rising edge pending interrupt
 - Falling
- Exti lines 0 to 15 are linked to gpio pin number 0 to 15. Gpio port can be selected through multiplexer.

24.2.2 How to use this driver

1. Configure the EXTI line using HAL_EXTI_SetConfigLine().
 - Choose the interrupt line number by setting "Line" member from EXTI_ConfigTypeDef structure.
 - Configure the interrupt and/or event mode using "Mode" member from EXTI_ConfigTypeDef structure.
 - For configurable lines, configure rising and/or falling trigger "Trigger" member from EXTI_ConfigTypeDef structure.
 - For Exti lines linked to gpio, choose gpio port using "GPIOSeI" member from GPIO_InitTypeDef structure.
2. Get current Exti configuration of a dedicated line using HAL_EXTI_GetConfigLine().
 - Provide exiting handle as parameter.
 - Provide pointer on EXTI_ConfigTypeDef structure as second parameter.
3. Clear Exti configuration of a dedicated line using HAL_EXTI_GetConfigLine().
 - Provide exiting handle as parameter.
4. Register callback to treat Exti interrupts using HAL_EXTI_RegisterCallback().
 - Provide exiting handle as first parameter.
 - Provide which callback will be registered using one value from EXTI_CallbackIDTypeDef.
 - Provide callback function pointer.
5. Get interrupt pending bit using HAL_EXTI_GetPending().
6. Clear interrupt pending bit using HAL_EXTI_GetPending().
7. Generate software interrupt using HAL_EXTI_GenerateSWI().

24.2.3 Configuration functions

This section contains the following APIs:

- [*HAL_EXTI_SetConfigLine\(\)*](#)
- [*HAL_EXTI_GetConfigLine\(\)*](#)
- [*HAL_EXTI_ClearConfigLine\(\)*](#)
- [*HAL_EXTI_RegisterCallback\(\)*](#)
- [*HAL_EXTI_GetHandle\(\)*](#)

24.2.4 Detailed description of functions

HAL_EXTI_SetConfigLine

Function name

HAL_StatusTypeDef HAL_EXTI_SetConfigLine (EXTI_HandleTypeDef * hexti, EXTI_ConfigTypeDef * pExtiConfig)

Function description

Set configuration of a dedicated Exti line.

Parameters

- **hexti**: Exti handle.
- **pExtiConfig**: Pointer on EXTI configuration to be set.

Return values

- **HAL**: Status.

HAL_EXTI_GetConfigLine

Function name

HAL_StatusTypeDef HAL_EXTI_GetConfigLine (EXTI_HandleTypeDef * hexti, EXTI_ConfigTypeDef * pExtiConfig)

Function description

Get configuration of a dedicated Exti line.

Parameters

- **hexti**: Exti handle.
- **pExtiConfig**: Pointer on structure to store Exti configuration.

Return values

- **HAL**: Status.

HAL_EXTI_ClearConfigLine

Function name

HAL_StatusTypeDef HAL_EXTI_ClearConfigLine (EXTI_HandleTypeDef * hexti)

Function description

Clear whole configuration of a dedicated Exti line.

Parameters

- **hexti**: Exti handle.

Return values

- **HAL**: Status.

HAL_EXTI_RegisterCallback

Function name

HAL_StatusTypeDef HAL_EXTI_RegisterCallback (EXTI_HandleTypeDef * hexti, EXTI_CallbackIDTypeDef CallbackID, void(*)(void) pPendingCbf)

Function description

Register callback for a dedicated Exti line.

Parameters

- **hexti**: Exti handle.
- **CallbackID**: User callback identifier. This parameter can be one of
– EXTI_CallbackIDTypeDef values.
- **pPendingCbf**: function pointer to be stored as callback.

Return values

- **HAL**: Status.

HAL_EXTI_GetHandle

Function name

HAL_StatusTypeDef HAL_EXTI_GetHandle (EXTI_HandleTypeDef * hexti, uint32_t ExtiLine)

Function description

Store line number as handle private field.

Parameters

- **hexti**: Exti handle.
- **ExtiLine**: Exti line number. This parameter can be from 0 to EXTI_LINE_NB.

Return values

- **HAL**: Status.

HAL_EXTI_IRQHandler

Function name

```
void HAL_EXTI_IRQHandler (EXTI_HandleTypeDef * hexti)
```

Function description

Handle EXTI interrupt request.

Parameters

- **hexti**: Exti handle.

Return values

- **none.**:

HAL_EXTI_GetPending

Function name

```
uint32_t HAL_EXTI_GetPending (EXTI_HandleTypeDef * hexti, uint32_t Edge)
```

Function description

Get interrupt pending bit of a dedicated line.

Parameters

- **hexti**: Exti handle.
- **Edge**: Specify which pending edge as to be checked. This parameter can be one of the following values:
 - EXTI_TRIGGER_RISING_FALLING This parameter is kept for compatibility with other series.

Return values

- **1**: if interrupt is pending else 0.

HAL_EXTI_ClearPending

Function name

```
void HAL_EXTI_ClearPending (EXTI_HandleTypeDef * hexti, uint32_t Edge)
```

Function description

Clear interrupt pending bit of a dedicated line.

Parameters

- **hexti**: Exti handle.
- **Edge**: Specify which pending edge as to be clear. This parameter can be one of the following values:
 - EXTI_TRIGGER_RISING_FALLING This parameter is kept for compatibility with other series.

Return values

- **None.**:

HAL_EXTI_GenerateSWI

Function name

```
void HAL_EXTI_GenerateSWI (EXTI_HandleTypeDef * hexti)
```


Function description

Generate a software interrupt for a dedicated line.

Parameters

- **hexti:** Exti handle.

Return values

- **None.:**

24.3 EXTI Firmware driver defines

The following section lists the various define and macros of the module.

24.3.1 EXTI

EXTI
EXTI GPIOSeI

EXTI_GPIOA

EXTI_GPIOB

EXTI_GPIOC

EXTI_GPIOD

EXTI_GPIOE

EXTI_GPIOF

EXTI_GPIOG

EXTI_GPIOH

EXTI_GPIOI

EXTI_GPIOJ

EXTI_GPIOK

EXTI Line

EXTI_LINE_0

External interrupt line 0

EXTI_LINE_1

External interrupt line 1

EXTI_LINE_2

External interrupt line 2

EXTI_LINE_3

External interrupt line 3

EXTI_LINE_4

External interrupt line 4

EXTI_LINE_5

External interrupt line 5

EXTI_LINE_6

External interrupt line 6

EXTI_LINE_7

External interrupt line 7

EXTI_LINE_8

External interrupt line 8

EXTI_LINE_9

External interrupt line 9

EXTI_LINE_10

External interrupt line 10

EXTI_LINE_11

External interrupt line 11

EXTI_LINE_12

External interrupt line 12

EXTI_LINE_13

External interrupt line 13

EXTI_LINE_14

External interrupt line 14

EXTI_LINE_15

External interrupt line 15

EXTI_LINE_16

External interrupt line 16 Connected to the PVD Output

EXTI_LINE_17

External interrupt line 17 Connected to the RTC Alarm event

EXTI_LINE_18

External interrupt line 18 Connected to the USB OTG FS Wakeup from suspend event

EXTI_LINE_19

External interrupt line 19 Connected to the Ethernet Wakeup event

EXTI_LINE_20

External interrupt line 20 Connected to the USB OTG HS (configured in FS) Wakeup event

EXTI_LINE_21

External interrupt line 21 Connected to the RTC Tamper and Time Stamp events

EXTI_LINE_22

External interrupt line 22 Connected to the RTC Wakeup event

EXTI Mode**EXTI_MODE_NONE**

EXTI_MODE_INTERRUPT

EXTI_MODE_EVENT

EXTI Trigger

EXTI_TRIGGER_NONE

EXTI_TRIGGER_RISING

EXTI_TRIGGER_FALLING

EXTI_TRIGGER_RISING_FALLING

25 HAL ETH Generic Driver

25.1 ETH Firmware driver registers structures

25.1.1 ETH_DMADescTypeDef

ETH_DMADescTypeDef is defined in the stm32f4xx_hal_eth.h

Data Fields

- `__IO uint32_t DESC0`
- `__IO uint32_t DESC1`
- `__IO uint32_t DESC2`
- `__IO uint32_t DESC3`
- `__IO uint32_t DESC4`
- `__IO uint32_t DESC5`
- `__IO uint32_t DESC6`
- `__IO uint32_t DESC7`
- `uint32_t BackupAddr0`
- `uint32_t BackupAddr1`

Field Documentation

- `__IO uint32_t ETH_DMADescTypeDef::DESC0`
- `__IO uint32_t ETH_DMADescTypeDef::DESC1`
- `__IO uint32_t ETH_DMADescTypeDef::DESC2`
- `__IO uint32_t ETH_DMADescTypeDef::DESC3`
- `__IO uint32_t ETH_DMADescTypeDef::DESC4`
- `__IO uint32_t ETH_DMADescTypeDef::DESC5`
- `__IO uint32_t ETH_DMADescTypeDef::DESC6`
- `__IO uint32_t ETH_DMADescTypeDef::DESC7`
- `uint32_t ETH_DMADescTypeDef::BackupAddr0`
- `uint32_t ETH_DMADescTypeDef::BackupAddr1`

25.1.2 __ETH_BufferTypeDef

__ETH_BufferTypeDef is defined in the stm32f4xx_hal_eth.h

Data Fields

- `uint8_t * buffer`
- `uint32_t len`
- `struct __ETH_BufferTypeDef * next`

Field Documentation

- `uint8_t* __ETH_BufferTypeDef::buffer`
- `uint32_t __ETH_BufferTypeDef::len`
- `struct __ETH_BufferTypeDef* __ETH_BufferTypeDef::next`

25.1.3 ETH_TxDescListTypeDef

ETH_TxDescListTypeDef is defined in the stm32f4xx_hal_eth.h

Data Fields

- `uint32_t TxDesc`
- `uint32_t CurTxDesc`
- `uint32_t * PacketAddress`
- `uint32_t * CurrentPacketAddress`
- `uint32_t BuffersInUse`

- `uint32_t releaseIndex`

Field Documentation

- `uint32_t ETH_TxDescListTypeDef::TxDesc[ETH_TX_DESC_CNT]`
- `uint32_t ETH_TxDescListTypeDef::CurTxDesc`
- `uint32_t* ETH_TxDescListTypeDef::PacketAddress[ETH_TX_DESC_CNT]`
- `uint32_t* ETH_TxDescListTypeDef::CurrentPacketAddress`
- `uint32_t ETH_TxDescListTypeDef::BuffersInUse`
- `uint32_t ETH_TxDescListTypeDef::releaseIndex`

25.1.4

ETH_TxPacketConfig

`ETH_TxPacketConfig` is defined in the `stm32f4xx_hal_eth.h`

Data Fields

- `uint32_t Attributes`
- `uint32_t Length`
- `ETH_BufferTypeDef * TxBuffer`
- `uint32_t SrcAddrCtrl`
- `uint32_t CRCPadCtrl`
- `uint32_t ChecksumCtrl`
- `uint32_t MaxSegmentSize`
- `uint32_t PayloadLen`
- `uint32_t TCPHeaderLen`
- `uint32_t VlanTag`
- `uint32_t VlanCtrl`
- `uint32_t InnerVlanTag`
- `uint32_t InnerVlanCtrl`
- `void * pData`

Field Documentation

- `uint32_t ETH_TxPacketConfig::Attributes`
Tx packet HW features capabilities. This parameter can be a combination of `ETH_Tx_Packet_Attributes`
- `uint32_t ETH_TxPacketConfig::Length`
Total packet length
- `ETH_BufferTypeDef* ETH_TxPacketConfig::TxBuffer`
Tx buffers pointers
- `uint32_t ETH_TxPacketConfig::SrcAddrCtrl`
Specifies the source address insertion control. This parameter can be a value of `ETH_Tx_Packet_Source_Addr_Control`
- `uint32_t ETH_TxPacketConfig::CRCPadCtrl`
Specifies the CRC and Pad insertion and replacement control. This parameter can be a value of `ETH_Tx_Packet_CRC_Pad_Control`
- `uint32_t ETH_TxPacketConfig::ChecksumCtrl`
Specifies the checksum insertion control. This parameter can be a value of `ETH_Tx_Packet_Checksum_Control`
- `uint32_t ETH_TxPacketConfig::MaxSegmentSize`
Sets TCP maximum segment size only when TCP segmentation is enabled. This parameter can be a value from 0x0 to 0x3FFF
- `uint32_t ETH_TxPacketConfig::PayloadLen`
Sets Total payload length only when TCP segmentation is enabled. This parameter can be a value from 0x0 to 0x3FFFF
- `uint32_t ETH_TxPacketConfig::TCPHeaderLen`
Sets TCP header length only when TCP segmentation is enabled. This parameter can be a value from 0x5 to 0xF
- `uint32_t ETH_TxPacketConfig::VlanTag`
Sets VLAN Tag only when VLAN is enabled. This parameter can be a value from 0x0 to 0xFFFF

- ***uint32_t ETH_TxPacketConfig::VlanCtrl***
Specifies VLAN Tag insertion control only when VLAN is enabled. This parameter can be a value of ***ETH_Tx_Packet_VLAN_Control***
- ***uint32_t ETH_TxPacketConfig::InnerVlanTag***
Sets Inner VLAN Tag only when Inner VLAN is enabled. This parameter can be a value from 0x0 to 0x3FFFF
- ***uint32_t ETH_TxPacketConfig::InnerVlanCtrl***
Specifies Inner VLAN Tag insertion control only when Inner VLAN is enabled. This parameter can be a value of ***ETH_Tx_Packet_Inner_VLAN_Control***
- ***void* ETH_TxPacketConfig::pData***
Specifies Application packet pointer to save

25.1.5 ETH_TimeStampTypeDef

ETH_TimeStampTypeDef is defined in the stm32f4xx_hal_eth.h

Data Fields

- ***uint32_t TimeStampLow***
- ***uint32_t TimeStampHigh***

Field Documentation

- ***uint32_t ETH_TimeStampTypeDef::TimeStampLow***
- ***uint32_t ETH_TimeStampTypeDef::TimeStampHigh***

25.1.6 ETH_TimeTypeDef

ETH_TimeTypeDef is defined in the stm32f4xx_hal_eth.h

Data Fields

- ***uint32_t Seconds***
- ***uint32_t NanoSeconds***

Field Documentation

- ***uint32_t ETH_TimeTypeDef::Seconds***
- ***uint32_t ETH_TimeTypeDef::NanoSeconds***

25.1.7 ETH_RxDescListTypeDef

ETH_RxDescListTypeDef is defined in the stm32f4xx_hal_eth.h

Data Fields

- ***uint32_t RxDesc***
- ***uint32_t ItMode***
- ***uint32_t RxDescIdx***
- ***uint32_t RxDescCnt***
- ***uint32_t RxDataLength***
- ***uint32_t RxBuildDescIdx***
- ***uint32_t RxBuildDescCnt***
- ***uint32_t pRxLastRxDesc***
- ***ETH_TimeStampTypeDef TimeStamp***
- ***void *pRxStart***
- ***void *pRxEnd***

Field Documentation

- ***uint32_t ETH_RxDescListTypeDef::RxDesc[ETH_RX_DESC_CNT]***
- ***uint32_t ETH_RxDescListTypeDef::ItMode***
- ***uint32_t ETH_RxDescListTypeDef::RxDescIdx***
- ***uint32_t ETH_RxDescListTypeDef::RxDescCnt***
- ***uint32_t ETH_RxDescListTypeDef::RxDataLength***
- ***uint32_t ETH_RxDescListTypeDef::RxBuildDescIdx***

- `uint32_t ETH_RxDescListTypeDef::RxBuildDescCnt`
- `uint32_t ETH_RxDescListTypeDef::pRxLastRxDesc`
- `ETH_TimeStampTypeDef ETH_RxDescListTypeDef::TimeStamp`
- `void* ETH_RxDescListTypeDef::pRxStart`
- `void* ETH_RxDescListTypeDef::pRxEnd`

25.1.8

ETH_MACConfigTypeDef

`ETH_MACConfigTypeDef` is defined in the `stm32f4xx_hal_eth.h`

Data Fields

- `uint32_t SourceAddrControl`
- `FunctionalState ChecksumOffload`
- `uint32_t InterPacketGapVal`
- `FunctionalState GiantPacketSizeLimitControl`
- `FunctionalState Support2KPacket`
- `FunctionalState CRCStripTypePacket`
- `FunctionalState AutomaticPadCRCStrip`
- `FunctionalState Watchdog`
- `FunctionalState Jabber`
- `FunctionalState JumboPacket`
- `uint32_t Speed`
- `uint32_t DuplexMode`
- `FunctionalState LoopbackMode`
- `FunctionalState CarrierSenseBeforeTransmit`
- `FunctionalState ReceiveOwn`
- `FunctionalState CarrierSenseDuringTransmit`
- `FunctionalState RetryTransmission`
- `uint32_t BackOffLimit`
- `FunctionalState DeferralCheck`
- `uint32_t PreambleLength`
- `FunctionalState SlowProtocolDetect`
- `FunctionalState CRCCheckingRxPackets`
- `uint32_t GiantPacketSizeLimit`
- `FunctionalState ExtendedInterPacketGap`
- `uint32_t ExtendedInterPacketGapVal`
- `FunctionalState ProgrammableWatchdog`
- `uint32_t WatchdogTimeout`
- `uint32_t PauseTime`
- `FunctionalState ZeroQuantaPause`
- `uint32_t PauseLowThreshold`
- `FunctionalState TransmitFlowControl`
- `FunctionalState UnicastPausePacketDetect`
- `FunctionalState ReceiveFlowControl`
- `uint32_t TransmitQueueMode`
- `uint32_t ReceiveQueueMode`
- `FunctionalState DropTCPIPChecksumErrorPacket`
- `FunctionalState ForwardRxErrorPacket`
- `FunctionalState ForwardRxUndersizedGoodPacket`

Field Documentation

- ***uint32_t ETH_MACConfigTypeDef::SourceAddrControl***
Selects the Source Address Insertion or Replacement Control. This parameter can be a value of [*ETH_Source_Addr_Control*](#)
- ***FunctionalState ETH_MACConfigTypeDef::ChecksumOffload***
Enables or Disable the checksum checking for received packet payloads TCP, UDP or ICMP headers
- ***uint32_t ETH_MACConfigTypeDef::InterPacketGapVal***
Sets the minimum IPG between Packet during transmission. This parameter can be a value of [*ETH_Inter_Packet_Gap*](#)
- ***FunctionalState ETH_MACConfigTypeDef::GiantPacketSizeLimitControl***
Enables or disables the Giant Packet Size Limit Control.
- ***FunctionalState ETH_MACConfigTypeDef::Support2KPacket***
Enables or disables the IEEE 802.3as Support for 2K length Packets
- ***FunctionalState ETH_MACConfigTypeDef::CRCStripTypePacket***
Enables or disables the CRC stripping for Type packets.
- ***FunctionalState ETH_MACConfigTypeDef::AutomaticPadCRCStrip***
Enables or disables the Automatic MAC Pad/CRC Stripping.
- ***FunctionalState ETH_MACConfigTypeDef::Watchdog***
Enables or disables the Watchdog timer on Rx path.
- ***FunctionalState ETH_MACConfigTypeDef::Jabber***
Enables or disables Jabber timer on Tx path.
- ***FunctionalState ETH_MACConfigTypeDef::JumboPacket***
Enables or disables receiving Jumbo Packet When enabled, the MAC allows jumbo packets of 9,018 bytes without reporting a giant packet error
- ***uint32_t ETH_MACConfigTypeDef::Speed***
Sets the Ethernet speed: 10/100 Mbps. This parameter can be a value of [*ETH_Speed*](#)
- ***uint32_t ETH_MACConfigTypeDef::DuplexMode***
Selects the MAC duplex mode: Half-Duplex or Full-Duplex mode This parameter can be a value of [*ETH_Duplex_Mode*](#)
- ***FunctionalState ETH_MACConfigTypeDef::LoopbackMode***
Enables or disables the loopback mode
- ***FunctionalState ETH_MACConfigTypeDef::CarrierSenseBeforeTransmit***
Enables or disables the Carrier Sense Before Transmission in Full Duplex Mode.
- ***FunctionalState ETH_MACConfigTypeDef::ReceiveOwn***
Enables or disables the Receive Own in Half Duplex mode.
- ***FunctionalState ETH_MACConfigTypeDef::CarrierSenseDuringTransmit***
Enables or disables the Carrier Sense During Transmission in the Half Duplex mode
- ***FunctionalState ETH_MACConfigTypeDef::RetryTransmission***
Enables or disables the MAC retry transmission, when a collision occurs in Half Duplex mode.
- ***uint32_t ETH_MACConfigTypeDef::BackOffLimit***
Selects the BackOff limit value. This parameter can be a value of [*ETH_Back_Off_Limit*](#)
- ***FunctionalState ETH_MACConfigTypeDef::DeferralCheck***
Enables or disables the deferral check function in Half Duplex mode.
- ***uint32_t ETH_MACConfigTypeDef::PreambleLength***
Selects or not the Preamble Length for Transmit packets (Full Duplex mode). This parameter can be a value of [*ETH_Preamble_Length*](#)
- ***FunctionalState ETH_MACConfigTypeDef::SlowProtocolDetect***
Enable or disables the Slow Protocol Detection.
- ***FunctionalState ETH_MACConfigTypeDef::CRCCheckingRxPackets***
Enable or disables the CRC Checking for Received Packets.
- ***uint32_t ETH_MACConfigTypeDef::GiantPacketSizeLimit***
Specifies the packet size that the MAC will declare it as Giant, If it's size is greater than the value programmed in this field in units of bytes This parameter must be a number between Min_Data = 0x618 (1518 byte) and Max_Data = 0x3FFF (32 Kbyte).
- ***FunctionalState ETH_MACConfigTypeDef::ExtendedInterPacketGap***
Enable or disables the extended inter packet gap.

ETH_TXDMABURSTLENGTH_4XPBL_16BEAT

maximum number of beats to be transferred in one TxDMA (or both) transaction is 16

ETH_TXDMABURSTLENGTH_4XPBL_32BEAT

maximum number of beats to be transferred in one TxDMA (or both) transaction is 32

ETH_TXDMABURSTLENGTH_4XPBL_64BEAT

maximum number of beats to be transferred in one TxDMA (or both) transaction is 64

ETH_TXDMABURSTLENGTH_4XPBL_128BEAT

maximum number of beats to be transferred in one TxDMA (or both) transaction is 128

ETH Tx Packet Attributes**ETH_TX_PACKETS_FEATURES_CSUM****ETH_TX_PACKETS_FEATURES_SAIC****ETH_TX_PACKETS_FEATURES_VLANTAG****ETH_TX_PACKETS_FEATURES_INNERVLANTAG****ETH_TX_PACKETS_FEATURES_TSO****ETH_TX_PACKETS_FEATURES_CRCPAD*****ETH Tx Packet Checksum Control*****ETH_CHECKSUM_DISABLE****ETH_CHECKSUM_IPHDR_INSERT****ETH_CHECKSUM_IPHDR_PAYLOAD_INSERT****ETH_CHECKSUM_IPHDR_PAYLOAD_INSERT_PHDR_CALC*****ETH Tx Packet CRC Pad Control*****ETH_CRC_PAD_DISABLE****ETH_CRC_PAD_INSERT****ETH_CRC_INSERT*****ETH Tx Packet Inner VLAN Control*****ETH_INNER_VLAN_DISABLE****ETH_INNER_VLAN_REMOVE****ETH_INNER_VLAN_INSERT****ETH_INNER_VLAN_REPLACE*****ETH Tx Packet Source Addr Control*****ETH_SRC_ADDR_CONTROL_DISABLE**

ETH_SRC_ADDR_INSERT

ETH_SRC_ADDR_REPLACE

ETH Tx Packet VLAN Control

ETH_VLAN_DISABLE

ETH_VLAN_REMOVE

ETH_VLAN_INSERT

ETH_VLAN_REPLACE

ETH Unicast Frames Filter

ETH_UNICASTFRAMESFILTER_PERFECTHASHTABLE

ETH_UNICASTFRAMESFILTER_HASHTABLE

ETH_UNICASTFRAMESFILTER_PERFECT

ETH Unicast Pause Frame Detect

ETH_UNICASTPAUSEFRAMEDETECT_ENABLE

ETH_UNICASTPAUSEFRAMEDETECT_DISABLE

ETH VLAN Tag Comparison

ETH_VLANTAGCOMPARISON_12BIT

ETH_VLANTAGCOMPARISON_16BIT

ETH Watchdog

ETH_WATCHDOG_ENABLE

ETH_WATCHDOG_DISABLE

ETH Watchdog Timeout

ETH_WATCHDOGTIMEOUT_2KB

ETH_WATCHDOGTIMEOUT_3KB

ETH_WATCHDOGTIMEOUT_4KB

ETH_WATCHDOGTIMEOUT_5KB

ETH_WATCHDOGTIMEOUT_6KB

ETH_WATCHDOGTIMEOUT_7KB

ETH_WATCHDOGTIMEOUT_8KB

ETH_WATCHDOGTIMEOUT_9KB

ETH_WATCHDOGTIMEOUT_10KB

ETH_WATCHDOGTIMEOUT_11KB

ETH_WATCHDOGTIMEOUT_12KB

ETH_WATCHDOGTIMEOUT_13KB

ETH_WATCHDOGTIMEOUT_14KB

ETH_WATCHDOGTIMEOUT_15KB

ETH_WATCHDOGTIMEOUT_16KB

ETH Zero Quanta Pause

ETH_ZEROQUANTAPAUSE_ENABLE

ETH_ZEROQUANTAPAUSE_DISABLE

26 HAL FLASH Generic Driver

26.1 FLASH Firmware driver registers structures

26.1.1 FLASH_ProcessTypeDef

FLASH_ProcessTypeDef is defined in the stm32f4xx_hal_flash.h

Data Fields

- `__IO FLASH_ProcedureTypeDef ProcedureOnGoing`
- `__IO uint32_t NbSectorsToErase`
- `__IO uint8_t VoltageForErase`
- `__IO uint32_t Sector`
- `__IO uint32_t Bank`
- `__IO uint32_t Address`
- `HAL_LockTypeDef Lock`
- `__IO uint32_t ErrorCode`

Field Documentation

- `__IO FLASH_ProcedureTypeDef FLASH_ProcessTypeDef::ProcedureOnGoing`
- `__IO uint32_t FLASH_ProcessTypeDef::NbSectorsToErase`
- `__IO uint8_t FLASH_ProcessTypeDef::VoltageForErase`
- `__IO uint32_t FLASH_ProcessTypeDef::Sector`
- `__IO uint32_t FLASH_ProcessTypeDef::Bank`
- `__IO uint32_t FLASH_ProcessTypeDef::Address`
- `HAL_LockTypeDef FLASH_ProcessTypeDef::Lock`
- `__IO uint32_t FLASH_ProcessTypeDef::ErrorCode`

26.2 FLASH Firmware driver API description

The following section lists the various functions of the FLASH library.

26.2.1 FLASH peripheral features

The Flash memory interface manages CPU AHB I-Code and D-Code accesses to the Flash memory. It implements the erase and program Flash memory operations and the read and write protection mechanisms.

The Flash memory interface accelerates code execution with a system of instruction prefetch and cache lines.

The FLASH main features are:

- Flash memory read operations
- Flash memory program/erase operations
- Read / write protections
- Prefetch on I-Code
- 64 cache lines of 128 bits on I-Code
- 8 cache lines of 128 bits on D-Code

26.2.2 How to use this driver

This driver provides functions and macros to configure and program the FLASH memory of all STM32F4xx devices.

1. FLASH Memory IO Programming functions:
 - Lock and Unlock the FLASH interface using HAL_FLASH_Unlock() and HAL_FLASH_Lock() functions
 - Program functions: byte, half word, word and double word
 - There Two modes of programming :
 - Polling mode using HAL_FLASH_Program() function
 - Interrupt mode using HAL_FLASH_Program_IT() function
2. Interrupts and flags management functions :
 - Handle FLASH interrupts by calling HAL_FLASH_IRQHandler()
 - Wait for last FLASH operation according to its status
 - Get error flag status by calling HAL_SetErrorCode()

In addition to these functions, this driver includes a set of macros allowing to handle the following operations:

- Set the latency
- Enable/Disable the prefetch buffer
- Enable/Disable the Instruction cache and the Data cache
- Reset the Instruction cache and the Data cache
- Enable/Disable the FLASH interrupts
- Monitor the FLASH flags status

26.2.3 Programming operation functions

This subsection provides a set of functions allowing to manage the FLASH program operations.

This section contains the following APIs:

- [*HAL_FLASH_Program\(\)*](#)
- [*HAL_FLASH_Program_IT\(\)*](#)
- [*HAL_FLASH_IRQHandler\(\)*](#)
- [*HAL_FLASH_EndOfOperationCallback\(\)*](#)
- [*HAL_FLASH_OperationErrorCallback\(\)*](#)

26.2.4 Peripheral Control functions

This subsection provides a set of functions allowing to control the FLASH memory operations.

This section contains the following APIs:

- [*HAL_FLASH_Unlock\(\)*](#)
- [*HAL_FLASH_Lock\(\)*](#)
- [*HAL_FLASH_OB_Unlock\(\)*](#)
- [*HAL_FLASH_OB_Lock\(\)*](#)
- [*HAL_FLASH_OB_Launch\(\)*](#)

26.2.5 Peripheral Errors functions

This subsection permits to get in run-time Errors of the FLASH peripheral.

This section contains the following APIs:

- [*HAL_FLASH_GetError\(\)*](#)
- [*FLASH_WaitForLastOperation\(\)*](#)

26.2.6 Detailed description of functions

HAL_FLASH_Program

Function name

HAL_StatusTypeDef HAL_FLASH_Program (uint32_t TypeProgram, uint32_t Address, uint64_t Data)

Function description

Program byte, halfword, word or double word at a specified address.

Parameters

- **TypeProgram:** Indicate the way to program at a specified address. This parameter can be a value of FLASH Type Program
- **Address:** specifies the address to be programmed.
- **Data:** specifies the data to be programmed

Return values

- **HAL_StatusTypeDef:** HAL Status

HAL_FLASH_Program_IT

Function name

HAL_StatusTypeDef HAL_FLASH_Program_IT (uint32_t TypeProgram, uint32_t Address, uint64_t Data)

Function description

Program byte, halfword, word or double word at a specified address with interrupt enabled.

Parameters

- **TypeProgram:** Indicate the way to program at a specified address. This parameter can be a value of FLASH Type Program
- **Address:** specifies the address to be programmed.
- **Data:** specifies the data to be programmed

Return values

- **HAL:** Status

HAL_FLASH_IRQHandler

Function name

void HAL_FLASH_IRQHandler (void)

Function description

This function handles FLASH interrupt request.

Return values

- **None:**

HAL_FLASH_EndOfOperationCallback

Function name

void HAL_FLASH_EndOfOperationCallback (uint32_t ReturnValue)

Function description

FLASH end of operation interrupt callback.

Parameters

- **ReturnValue:** The value saved in this parameter depends on the ongoing procedure Mass Erase: Bank number which has been requested to erase Sectors Erase: Sector which has been erased (if 0xFFFFFFFFU, it means that all the selected sectors have been erased) Program: Address which was selected for data program

Return values

- **None:**

HAL_FLASH_OperationErrorCallback

Function name

void HAL_FLASH_OperationErrorCallback (uint32_t ReturnValue)

Function description

FLASH operation error interrupt callback.

Parameters

- **ReturnValue:** The value saved in this parameter depends on the ongoing procedure Mass Erase: Bank number which has been requested to erase Sectors Erase: Sector number which returned an error Program: Address which was selected for data program

Return values

- **None:**

HAL_FLASH_Unlock

Function name

HAL_StatusTypeDef HAL_FLASH_Unlock (void)

Function description

Unlock the FLASH control register access.

Return values

- **HAL:** Status

HAL_FLASH_Lock

Function name

HAL_StatusTypeDef HAL_FLASH_Lock (void)

Function description

Locks the FLASH control register access.

Return values

- **HAL:** Status

HAL_FLASH_OB_Unlock

Function name

HAL_StatusTypeDef HAL_FLASH_OB_Unlock (void)

Function description

Unlock the FLASH Option Control Registers access.

Return values

- **HAL:** Status

HAL_FLASH_OB_Lock

Function name

HAL_StatusTypeDef HAL_FLASH_OB_Lock (void)

Function description

Lock the FLASH Option Control Registers access.

Return values

- **HAL:** Status

HAL_FLASH_OB_Launch

Function name

HAL_StatusTypeDef HAL_FLASH_OB_Launch (void)

Function description

Launch the option byte loading.

Return values

- **HAL:** Status

HAL_FLASH_GetError

Function name

uint32_t HAL_FLASH_GetError (void)

Function description

Get the specific FLASH error flag.

Return values

- **FLASH_ErrorCode:** The returned value can be a combination of:
 - HAL_FLASH_ERROR_RD: FLASH Read Protection error flag (PCROP)
 - HAL_FLASH_ERROR_PGS: FLASH Programming Sequence error flag
 - HAL_FLASH_ERROR_PGP: FLASH Programming Parallelism error flag
 - HAL_FLASH_ERROR_PGA: FLASH Programming Alignment error flag
 - HAL_FLASH_ERROR_WRP: FLASH Write protected error flag
 - HAL_FLASH_ERROR_OPERATION: FLASH operation Error flag

FLASH_WaitForLastOperation

Function name

HAL_StatusTypeDef FLASH_WaitForLastOperation (uint32_t Timeout)

Function description

Wait for a FLASH operation to complete.

Parameters

- **Timeout:** maximum flash operation timeout

Return values

- **HAL:** Status

26.3 FLASH Firmware driver defines

The following section lists the various define and macros of the module.

26.3.1 FLASH

FLASH

FLASH Error Code

HAL_FLASH_ERROR_NONE

No error

HAL_FLASH_ERROR_RD

Read Protection error

HAL_FLASH_ERROR_PGS

Programming Sequence error

HAL_FLASH_ERROR_PGP

Programming Parallelism error

HAL_FLASH_ERROR_PGA

Programming Alignment error

HAL_FLASH_ERROR_WRP

Write protection error

HAL_FLASH_ERROR_OPERATION

Operation Error

FLASH Exported Macros

__HAL_FLASH_SET_LATENCY

Description:

- Set the FLASH Latency.

Parameters:

- `__LATENCY__`: FLASH Latency The value of this parameter depend on device used within the same series

Return value:

- none

__HAL_FLASH_GET_LATENCY

Description:

- Get the FLASH Latency.

Return value:

- FLASH: Latency The value of this parameter depend on device used within the same series

__HAL_FLASH_PREFETCH_BUFFER_ENABLE

Description:

- Enable the FLASH prefetch buffer.

Return value:

- none

__HAL_FLASH_PREFETCH_BUFFER_DISABLE

Description:

- Disable the FLASH prefetch buffer.

Return value:

- none

__HAL_FLASH_INSTRUCTION_CACHE_ENABLE

Description:

- Enable the FLASH instruction cache.

Return value:

- none

__HAL_FLASH_INSTRUCTION_CACHE_DISABLE

Description:

- Disable the FLASH instruction cache.

Return value:

- none

__HAL_FLASH_DATA_CACHE_ENABLE

Description:

- Enable the FLASH data cache.

Return value:

- none

__HAL_FLASH_DATA_CACHE_DISABLE

Description:

- Disable the FLASH data cache.

Return value:

- none

__HAL_FLASH_INSTRUCTION_CACHE_RESET

Description:

- Resets the FLASH instruction Cache.

Return value:

- None

Notes:

- This function must be used only when the Instruction Cache is disabled.

__HAL_FLASH_DATA_CACHE_RESET

Description:

- Resets the FLASH data Cache.

Return value:

- None

Notes:

- This function must be used only when the data Cache is disabled.

__HAL_FLASH_ENABLE_IT

Description:

- Enable the specified FLASH interrupt.

Parameters:

- **__INTERRUPT__**: FLASH interrupt This parameter can be any combination of the following values:
 - FLASH_IT_EOP: End of FLASH Operation Interrupt
 - FLASH_IT_ERR: Error Interrupt

Return value:

- none

__HAL_FLASH_DISABLE_IT

Description:

- Disable the specified FLASH interrupt.

Parameters:

- `__INTERRUPT__`: FLASH interrupt This parameter can be any combination of the following values:
 - `FLASH_IT_EOP`: End of FLASH Operation Interrupt
 - `FLASH_IT_ERR`: Error Interrupt

Return value:

- none

__HAL_FLASH_GET_FLAG

Description:

- Get the specified FLASH flag status.

Parameters:

- `__FLAG__`: specifies the FLASH flags to check. This parameter can be any combination of the following values:
 - `FLASH_FLAG_EOP` : FLASH End of Operation flag
 - `FLASH_FLAG_OPERR` : FLASH operation Error flag
 - `FLASH_FLAG_WRPERR`: FLASH Write protected error flag
 - `FLASH_FLAG_PGAERR`: FLASH Programming Alignment error flag
 - `FLASH_FLAG_PGPERR`: FLASH Programming Parallelism error flag
 - `FLASH_FLAG_PGSERR`: FLASH Programming Sequence error flag
 - `FLASH_FLAG_RDERR` : FLASH Read Protection error flag (PCROP) (*)
 - `FLASH_FLAG_BSY` : FLASH Busy flag (*) `FLASH_FLAG_RDERR` is not available for STM32F405xx/407xx/415xx/417xx devices

Return value:

- The: new state of `__FLAG__` (SET or RESET).

__HAL_FLASH_CLEAR_FLAG

Description:

- Clear the specified FLASH flags.

Parameters:

- `__FLAG__`: specifies the FLASH flags to clear. This parameter can be any combination of the following values:
 - `FLASH_FLAG_EOP` : FLASH End of Operation flag
 - `FLASH_FLAG_OPERR` : FLASH operation Error flag
 - `FLASH_FLAG_WRPERR`: FLASH Write protected error flag
 - `FLASH_FLAG_PGAERR`: FLASH Programming Alignment error flag
 - `FLASH_FLAG_PGPERR`: FLASH Programming Parallelism error flag
 - `FLASH_FLAG_PGSERR`: FLASH Programming Sequence error flag
 - `FLASH_FLAG_RDERR` : FLASH Read Protection error flag (PCROP) (*) (*) `FLASH_FLAG_RDERR` is not available for STM32F405xx/407xx/415xx/417xx devices

Return value:

- none

FLASH Flag definition

FLASH_FLAG_EOP

FLASH End of Operation flag

FLASH_FLAG_OPERR

FLASH operation Error flag

FLASH_FLAG_WRPERR

FLASH Write protected error flag

FLASH_FLAG_PGAERR

FLASH Programming Alignment error flag

FLASH_FLAG_PGPERR

FLASH Programming Parallelism error flag

FLASH_FLAG_PGSERR

FLASH Programming Sequence error flag

FLASH_FLAG_RDERR

Read Protection error flag (PCROP)

FLASH_FLAG_BSY

FLASH Busy flag

FLASH Interrupt definition**FLASH_IT_EOP**

End of FLASH Operation Interrupt source

FLASH_IT_ERR

Error Interrupt source

FLASH Private macros to check input parameters**IS_FLASH_TYPEPROGRAM*****FLASH Keys*****RDP_KEY****FLASH_KEY1****FLASH_KEY2****FLASH_OPT_KEY1****FLASH_OPT_KEY2*****FLASH Latency*****FLASH_LATENCY_0**

FLASH Zero Latency cycle

FLASH_LATENCY_1

FLASH One Latency cycle

FLASH_LATENCY_2

FLASH Two Latency cycles

FLASH_LATENCY_3

FLASH Three Latency cycles

FLASH_LATENCY_4

FLASH Four Latency cycles

FLASH_LATENCY_5

FLASH Five Latency cycles

FLASH_LATENCY_6

FLASH Six Latency cycles

FLASH_LATENCY_7

FLASH Seven Latency cycles

FLASH_LATENCY_8

FLASH Eight Latency cycles

FLASH_LATENCY_9

FLASH Nine Latency cycles

FLASH_LATENCY_10

FLASH Ten Latency cycles

FLASH_LATENCY_11

FLASH Eleven Latency cycles

FLASH_LATENCY_12

FLASH Twelve Latency cycles

FLASH_LATENCY_13

FLASH Thirteen Latency cycles

FLASH_LATENCY_14

FLASH Fourteen Latency cycles

FLASH_LATENCY_15

FLASH Fifteen Latency cycles

FLASH Program Parallelism**FLASH_PSIZE_BYTE****FLASH_PSIZE_HALF_WORD****FLASH_PSIZE_WORD****FLASH_PSIZE_DOUBLE_WORD****CR_PSIZE_MASK*****FLASH Type Program*****FLASH_TYPEPROGRAM_BYTE**

Program byte (8-bit) at a specified address

FLASH_TYPEPROGRAM_HALFWORD

Program a half-word (16-bit) at a specified address

FLASH_TYPEPROGRAM_WORD

Program a word (32-bit) at a specified address

FLASH_TYPEPROGRAM_DOUBLEWORD

Program a double word (64-bit) at a specified address

27 HAL FLASH Extension Driver

27.1 FLASHEx Firmware driver registers structures

27.1.1 FLASH_EraseInitTypeDef

FLASH_EraseInitTypeDef is defined in the stm32f4xx_hal_flash_ex.h

Data Fields

- **uint32_t TypeErase**
- **uint32_t Banks**
- **uint32_t Sector**
- **uint32_t NbSectors**
- **uint32_t VoltageRange**

Field Documentation

- **uint32_t FLASH_EraseInitTypeDef::TypeErase**
Mass erase or sector Erase. This parameter can be a value of [FLASHEx_Type_Erase](#)
- **uint32_t FLASH_EraseInitTypeDef::Banks**
Select banks to erase when Mass erase is enabled. This parameter must be a value of [FLASHEx_Banks](#)
- **uint32_t FLASH_EraseInitTypeDef::Sector**
Initial FLASH sector to erase when Mass erase is disabled This parameter must be a value of [FLASHEx_Sectors](#)
- **uint32_t FLASH_EraseInitTypeDef::NbSectors**
Number of sectors to be erased. This parameter must be a value between 1 and (max number of sectors - value of Initial sector)
- **uint32_t FLASH_EraseInitTypeDef::VoltageRange**
The device voltage range which defines the erase parallelism This parameter must be a value of [FLASHEx_Voltage_Range](#)

27.1.2 FLASH_OBProgramInitTypeDef

FLASH_OBProgramInitTypeDef is defined in the stm32f4xx_hal_flash_ex.h

Data Fields

- **uint32_t OptionType**
- **uint32_t WRPState**
- **uint32_t WRPSector**
- **uint32_t Banks**
- **uint32_t RDPLLevel**
- **uint32_t BORLevel**
- **uint8_t USERConfig**

Field Documentation

- **uint32_t FLASH_OBProgramInitTypeDef::OptionType**
Option byte to be configured. This parameter can be a value of [FLASHEx_Option_Type](#)
- **uint32_t FLASH_OBProgramInitTypeDef::WRPState**
Write protection activation or deactivation. This parameter can be a value of [FLASHEx_WRP_State](#)
- **uint32_t FLASH_OBProgramInitTypeDef::WRPSector**
Specifies the sector(s) to be write protected. The value of this parameter depend on device used within the same series
- **uint32_t FLASH_OBProgramInitTypeDef::Banks**
Select banks for WRP activation/deactivation of all sectors. This parameter must be a value of [FLASHEx_Banks](#)
- **uint32_t FLASH_OBProgramInitTypeDef::RDPLLevel**
Set the read protection level. This parameter can be a value of [FLASHEx_Option_Bytes_Read_Protection](#)

- **`uint32_t FLASH_OBProgramInitTypeDef::BORLevel`**
Set the BOR Level. This parameter can be a value of [FLASHEx_BOR_Reset_Level](#)
- **`uint8_t FLASH_OBProgramInitTypeDef::USERConfig`**
Program the FLASH User Option Byte: IWDG_SW / RST_STOP / RST_STDBY.

27.1.3 FLASH_AdvOBProgramInitTypeDef

`FLASH_AdvOBProgramInitTypeDef` is defined in the `stm32f4xx_hal_flash_ex.h`

Data Fields

- **`uint32_t OptionType`**
- **`uint32_t PCROPState`**
- **`uint32_t Banks`**
- **`uint16_t SectorsBank1`**
- **`uint16_t SectorsBank2`**
- **`uint8_t BootConfig`**

Field Documentation

- **`uint32_t FLASH_AdvOBProgramInitTypeDef::OptionType`**
Option byte to be configured for extension. This parameter can be a value of [FLASHEx_Advanced_Option_Type](#)
- **`uint32_t FLASH_AdvOBProgramInitTypeDef::PCROPState`**
PCROP activation or deactivation. This parameter can be a value of [FLASHEx_PCROP_State](#)
- **`uint32_t FLASH_AdvOBProgramInitTypeDef::Banks`**
Select banks for PCROP activation/deactivation of all sectors. This parameter must be a value of [FLASHEx_Banks](#)
- **`uint16_t FLASH_AdvOBProgramInitTypeDef::SectorsBank1`**
Specifies the sector(s) set for PCROP for Bank1. This parameter can be a value of [FLASHEx_Option_Bytes_PC_ReadWrite_Protection](#)
- **`uint16_t FLASH_AdvOBProgramInitTypeDef::SectorsBank2`**
Specifies the sector(s) set for PCROP for Bank2. This parameter can be a value of [FLASHEx_Option_Bytes_PC_ReadWrite_Protection](#)
- **`uint8_t FLASH_AdvOBProgramInitTypeDef::BootConfig`**
Specifies Option bytes for boot config. This parameter can be a value of [FLASHEx_Dual_Boot](#)

27.2 FLASHEx Firmware driver API description

The following section lists the various functions of the FLASHEx library.

27.2.1 Flash Extension features

Comparing to other previous devices, the FLASH interface for STM32F427xx/437xx and STM32F429xx/439xx devices contains the following additional features

- Capacity up to 2 Mbyte with dual bank architecture supporting read-while-write capability (RWW)
- Dual bank memory organization
- PCROP protection for all banks

27.2.2 How to use this driver

This driver provides functions to configure and program the FLASH memory of all STM32F427xx/437xx, STM32F429xx/439xx, STM32F469xx/479xx and STM32F446xx devices. It includes

1. FLASH Memory Erase functions:
 - Lock and Unlock the FLASH interface using `HAL_FLASH_Unlock()` and `HAL_FLASH_Lock()` functions
 - Erase function: Erase sector, erase all sectors
 - There are two modes of erase :
 - Polling Mode using `HAL_FLASHEx_Erase()`
 - Interrupt Mode using `HAL_FLASHEx_Erase_IT()`

2. Option Bytes Programming functions: Use HAL_FLASHEx_OBProgram() to :
 - Set/Reset the write protection
 - Set the Read protection Level
 - Set the BOR level
 - Program the user Option Bytes
3. Advanced Option Bytes Programming functions: Use HAL_FLASHEx_AdvOBProgram() to :
 - Extended space (bank 2) erase function
 - Full FLASH space (2 Mo) erase (bank 1 and bank 2)
 - Dual Boot activation
 - Write protection configuration for bank 2
 - PCROP protection configuration and control for both banks

27.2.3 Extended programming operation functions

This subsection provides a set of functions allowing to manage the Extension FLASH programming operations.

This section contains the following APIs:

- [HAL_FLASHEx_Erase\(\)](#)
- [HAL_FLASHEx_Erase_IT\(\)](#)
- [HAL_FLASHEx_OBProgram\(\)](#)
- [HAL_FLASHEx_OBGetConfig\(\)](#)
- [HAL_FLASHEx_AdvOBProgram\(\)](#)
- [HAL_FLASHEx_AdvOBGetConfig\(\)](#)
- [HAL_FLASHEx_OB_SelectPCROP\(\)](#)
- [HAL_FLASHEx_OB_DeSelectPCROP\(\)](#)
- [HAL_FLASHEx_OB_GetBank2WRP\(\)](#)

27.2.4 Detailed description of functions

HAL_FLASHEx_Erase

Function name

HAL_StatusTypeDef HAL_FLASHEx_Erase (FLASH_EraseInitTypeDef * pEraseInit, uint32_t * SectorError)

Function description

Perform a mass erase or erase the specified FLASH memory sectors.

Parameters

- **pEraseInit**: pointer to an FLASH_EraseInitTypeDef structure that contains the configuration information for the erasing.
- **SectorError**: pointer to variable that contains the configuration information on faulty sector in case of error (0xFFFFFFFFU means that all the sectors have been correctly erased)

Return values

- **HAL**: Status

HAL_FLASHEx_Erase_IT

Function name

HAL_StatusTypeDef HAL_FLASHEx_Erase_IT (FLASH_EraseInitTypeDef * pEraseInit)

Function description

Perform a mass erase or erase the specified FLASH memory sectors with interrupt enabled.

Parameters

- **pEraseInit:** pointer to an FLASH_EraseInitTypeDef structure that contains the configuration information for the erasing.

Return values

- **HAL:** Status

HAL_FLASHEx_OBProgram

Function name

HAL_StatusTypeDef HAL_FLASHEx_OBProgram (FLASH_OBProgramInitTypeDef * pOBInit)

Function description

Program option bytes.

Parameters

- **pOBInit:** pointer to an FLASH_OBInitStruct structure that contains the configuration information for the programming.

Return values

- **HAL:** Status

HAL_FLASHEx_OBGetConfig

Function name

void HAL_FLASHEx_OBGetConfig (FLASH_OBProgramInitTypeDef * pOBInit)

Function description

Get the Option byte configuration.

Parameters

- **pOBInit:** pointer to an FLASH_OBInitStruct structure that contains the configuration information for the programming.

Return values

- **None:**

HAL_FLASHEx_AdvOBProgram

Function name

HAL_StatusTypeDef HAL_FLASHEx_AdvOBProgram (FLASH_AdvOBProgramInitTypeDef * pAdvOBInit)

Function description

Program option bytes.

Parameters

- **pAdvOBInit:** pointer to an FLASH_AdvOBProgramInitTypeDef structure that contains the configuration information for the programming.

Return values

- **HAL:** Status

HAL_FLASHEx_AdvOBGetConfig

Function name

void HAL_FLASHEx_AdvOBGetConfig (FLASH_AdvOBProgramInitTypeDef * pAdvOBInit)

Function description

Get the OBEX byte configuration.

Parameters

- **pAdvOBInit:** pointer to an FLASH_AdvOBProgramInitTypeDef structure that contains the configuration information for the programming.

Return values

- **None:**

HAL_FLASHEx_OB_SelectPCROP

Function name

HAL_StatusTypeDef HAL_FLASHEx_OB_SelectPCROP (void)

Function description

Select the Protection Mode.

Return values

- **HAL:** Status

Notes

- After PCROP activated Option Byte modification NOT POSSIBLE! excepted Global Read Out Protection modification (from level1 to level0)
- Once SPRMOD bit is active unprotection of a protected sector is not possible
- Read a protected sector will set RDERR Flag and write a protected sector will set WRPERR Flag
- This function can be used only for STM32F42xxx/STM32F43xxx/STM32F401xx/STM32F411xx/STM32F446xx/ STM32F469xx/STM32F479xx/STM32F412xx/STM32F413xx devices.

HAL_FLASHEx_OB_DeSelectPCROP

Function name

HAL_StatusTypeDef HAL_FLASHEx_OB_DeSelectPCROP (void)

Function description

Deselect the Protection Mode.

Return values

- **HAL:** Status

Notes

- After PCROP activated Option Byte modification NOT POSSIBLE! excepted Global Read Out Protection modification (from level1 to level0)
- Once SPRMOD bit is active unprotection of a protected sector is not possible
- Read a protected sector will set RDERR Flag and write a protected sector will set WRPERR Flag
- This function can be used only for STM32F42xxx/STM32F43xxx/STM32F401xx/STM32F411xx/STM32F446xx/ STM32F469xx/STM32F479xx/STM32F412xx/STM32F413xx devices.

HAL_FLASHEx_OB_GetBank2WRP

Function name

uint16_t HAL_FLASHEx_OB_GetBank2WRP (void)

Function description

Returns the FLASH Write Protection Option Bytes value for Bank 2.

Return values

- **The:** FLASH Write Protection Option Bytes value

Notes

- This function can be used only for STM32F42xxx/STM32F43xxx/STM32F469xx/STM32F479xx devices.

FLASH_Erase_Sector

Function name

void FLASH_Erase_Sector (uint32_t Sector, uint8_t VoltageRange)

Function description

Erase the specified FLASH memory sector.

Parameters

- **Sector:** FLASH sector to erase The value of this parameter depend on device used within the same series
- **VoltageRange:** The device voltage range which defines the erase parallelism. This parameter can be one of the following values:
 - FLASH_VOLTAGE_RANGE_1: when the device voltage range is 1.8V to 2.1V, the operation will be done by byte (8-bit)
 - FLASH_VOLTAGE_RANGE_2: when the device voltage range is 2.1V to 2.7V, the operation will be done by half word (16-bit)
 - FLASH_VOLTAGE_RANGE_3: when the device voltage range is 2.7V to 3.6V, the operation will be done by word (32-bit)
 - FLASH_VOLTAGE_RANGE_4: when the device voltage range is 2.7V to 3.6V + External Vpp, the operation will be done by double word (64-bit)

Return values

- **None:**

FLASH_FlushCaches

Function name

void FLASH_FlushCaches (void)

Function description

Flush the instruction and data caches.

Return values

- **None:**

27.3 FLASHEx Firmware driver defines

The following section lists the various define and macros of the module.

27.3.1 FLASHEx

FLASHEx

FLASH Advanced Option Type

OPTIONBYTE_PCROP

PCROP option byte configuration

OPTIONBYTE_BOOTCONFIG

BOOTConfig option byte configuration

FLASH Banks

FLASH_BANK_1

Bank 1

FLASH_BANK_2

Bank 2

FLASH_BANK_BOTH

Bank1 and Bank2

FLASH BOR Reset Level**OB_BOR_LEVEL3**

Supply voltage ranges from 2.70 to 3.60 V

OB_BOR_LEVEL2

Supply voltage ranges from 2.40 to 2.70 V

OB_BOR_LEVEL1

Supply voltage ranges from 2.10 to 2.40 V

OB_BOR_OFF

Supply voltage ranges from 1.62 to 2.10 V

FLASH Dual Boot**OB_DUAL_BOOT_ENABLE**

Dual Bank Boot Enable

OB_DUAL_BOOT_DISABLE

Dual Bank Boot Disable, always boot on User Flash

FLASH Private macros to check input parameters**IS_FLASH_TYPEERASE****IS_VOLTAGERANGE****IS_WRPSTATE****IS_OPTIONBYTE****IS_OB_RDP_LEVEL****IS_OB_IWDG_SOURCE****IS_OB_STOP_SOURCE****IS_OB_STDBY_SOURCE****IS_OB_BOR_LEVEL****IS_PCROPSTATE****IS_OBEX****IS_FLASH_LATENCY**

IS_FLASH_BANK

IS_FLASH_SECTOR

IS_FLASH_ADDRESS

IS_FLASH_NBSECTORS

IS_OB_WRP_SECTOR

IS_OB_PCROP

IS_OB_BOOT

IS_OB_PCROP_SELECT

FLASH Mass Erase bit

FLASH_MER_BIT

2 MER bits here to clear

FLASH Option Bytes IWatchdog

OB_IWDG_SW

Software IWDG selected

OB_IWDG_HW

Hardware IWDG selected

FLASH Option Bytes nRST_STDBY

OB_STDBY_NO_RST

No reset generated when entering in STANDBY

OB_STDBY_RST

Reset generated when entering in STANDBY

FLASH Option Bytes nRST_STOP

OB_STOP_NO_RST

No reset generated when entering in STOP

OB_STOP_RST

Reset generated when entering in STOP

FLASH Option Bytes PC ReadWrite Protection

OB_PCROP_SECTOR_0

PC Read/Write protection of Sector0

OB_PCROP_SECTOR_1

PC Read/Write protection of Sector1

OB_PCROP_SECTOR_2

PC Read/Write protection of Sector2

OB_PCROP_SECTOR_3

PC Read/Write protection of Sector3

OB_PCROP_SECTOR_4

PC Read/Write protection of Sector4

OB_PCROP_SECTOR_5

PC Read/Write protection of Sector5

OB_PCROP_SECTOR_6

PC Read/Write protection of Sector6

OB_PCROP_SECTOR_7

PC Read/Write protection of Sector7

OB_PCROP_SECTOR_8

PC Read/Write protection of Sector8

OB_PCROP_SECTOR_9

PC Read/Write protection of Sector9

OB_PCROP_SECTOR_10

PC Read/Write protection of Sector10

OB_PCROP_SECTOR_11

PC Read/Write protection of Sector11

OB_PCROP_SECTOR_12

PC Read/Write protection of Sector12

OB_PCROP_SECTOR_13

PC Read/Write protection of Sector13

OB_PCROP_SECTOR_14

PC Read/Write protection of Sector14

OB_PCROP_SECTOR_15

PC Read/Write protection of Sector15

OB_PCROP_SECTOR_16

PC Read/Write protection of Sector16

OB_PCROP_SECTOR_17

PC Read/Write protection of Sector17

OB_PCROP_SECTOR_18

PC Read/Write protection of Sector18

OB_PCROP_SECTOR_19

PC Read/Write protection of Sector19

OB_PCROP_SECTOR_20

PC Read/Write protection of Sector20

OB_PCROP_SECTOR_21

PC Read/Write protection of Sector21

OB_PCROP_SECTOR_22

PC Read/Write protection of Sector22

OB_PCROP_SECTOR_23

PC Read/Write protection of Sector23

OB_PCROP_SECTOR_All

PC Read/Write protection of all Sectors

FLASH Option Bytes Read Protection**OB_RDP_LEVEL_0****OB_RDP_LEVEL_1****OB_RDP_LEVEL_2**

Warning: When enabling read protection level 2 it s no more possible to go back to level 1 or 0

FLASH Option Bytes Write Protection**OB_WRP_SECTOR_0**

Write protection of Sector0

OB_WRP_SECTOR_1

Write protection of Sector1

OB_WRP_SECTOR_2

Write protection of Sector2

OB_WRP_SECTOR_3

Write protection of Sector3

OB_WRP_SECTOR_4

Write protection of Sector4

OB_WRP_SECTOR_5

Write protection of Sector5

OB_WRP_SECTOR_6

Write protection of Sector6

OB_WRP_SECTOR_7

Write protection of Sector7

OB_WRP_SECTOR_8

Write protection of Sector8

OB_WRP_SECTOR_9

Write protection of Sector9

OB_WRP_SECTOR_10

Write protection of Sector10

OB_WRP_SECTOR_11

Write protection of Sector11

OB_WRP_SECTOR_12

Write protection of Sector12

OB_WRP_SECTOR_13

Write protection of Sector13

OB_WRP_SECTOR_14

Write protection of Sector14

OB_WRP_SECTOR_15

Write protection of Sector15

OB_WRP_SECTOR_16

Write protection of Sector16

OB_WRP_SECTOR_17

Write protection of Sector17

OB_WRP_SECTOR_18

Write protection of Sector18

OB_WRP_SECTOR_19

Write protection of Sector19

OB_WRP_SECTOR_20

Write protection of Sector20

OB_WRP_SECTOR_21

Write protection of Sector21

OB_WRP_SECTOR_22

Write protection of Sector22

OB_WRP_SECTOR_23

Write protection of Sector23

OB_WRP_SECTOR_All

Write protection of all Sectors

FLASH Option Type**OPTIONBYTE_WRP**

WRP option byte configuration

OPTIONBYTE_RDP

RDP option byte configuration

OPTIONBYTE_USER

USER option byte configuration

OPTIONBYTE_BOR

BOR option byte configuration

FLASH PCROP State**OB_PCROP_STATE_DISABLE**

Disable PCROP

OB_PCROP_STATE_ENABLE

Enable PCROP

FLASH Sectors**FLASH_SECTOR_0**

Sector Number 0

FLASH_SECTOR_1

Sector Number 1

FLASH_SECTOR_2

Sector Number 2

FLASH_SECTOR_3

Sector Number 3

FLASH_SECTOR_4

Sector Number 4

FLASH_SECTOR_5

Sector Number 5

FLASH_SECTOR_6

Sector Number 6

FLASH_SECTOR_7

Sector Number 7

FLASH_SECTOR_8

Sector Number 8

FLASH_SECTOR_9

Sector Number 9

FLASH_SECTOR_10

Sector Number 10

FLASH_SECTOR_11

Sector Number 11

FLASH_SECTOR_12

Sector Number 12

FLASH_SECTOR_13

Sector Number 13

FLASH_SECTOR_14

Sector Number 14

FLASH_SECTOR_15

Sector Number 15

FLASH_SECTOR_16

Sector Number 16

FLASH_SECTOR_17

Sector Number 17

FLASH_SECTOR_18

Sector Number 18

FLASH_SECTOR_19

Sector Number 19

FLASH_SECTOR_20

Sector Number 20

FLASH_SECTOR_21

Sector Number 21

FLASH_SECTOR_22

Sector Number 22

FLASH_SECTOR_23

Sector Number 23

FLASH Selection Protection Mode**OB_PCROP_DESELECTED**

Disabled PcROP, nWPRI bits used for Write Protection on sector i

OB_PCROP_SELECTED

Enable PcROP, nWPRI bits used for PCRoP Protection on sector i

FLASH Type Erase**FLASH_TYPEERASE_SECTORS**

Sectors erase only

FLASH_TYPEERASE_MASSERASE

Flash Mass erase activation

FLASH Voltage Range**FLASH_VOLTAGE_RANGE_1**

Device operating range: 1.8V to 2.1V

FLASH_VOLTAGE_RANGE_2

Device operating range: 2.1V to 2.7V

FLASH_VOLTAGE_RANGE_3

Device operating range: 2.7V to 3.6V

FLASH_VOLTAGE_RANGE_4

Device operating range: 2.7V to 3.6V + External Vpp

FLASH WRP State**OB_WRPSTATE_DISABLE**

Disable the write protection of the desired bank 1 sectors

OB_WRPSTATE_ENABLE

Enable the write protection of the desired bank 1 sectors

28 HAL FLASH__RAMFUNC Generic Driver

28.1 FLASH__RAMFUNC Firmware driver API description

The following section lists the various functions of the FLASH__RAMFUNC library.

28.1.1 APIs executed from Internal RAM

ARM Compiler

RAM functions are defined using the toolchain options. Functions that are to be executed in RAM should reside in a separate source module. Using the 'Options for File' dialog you can simply change the 'Code / Const' area of a module to a memory space in physical RAM. Available memory areas are declared in the 'Target' tab of the 'Options for Target' dialog.

ICCARM Compiler

RAM functions are defined using a specific toolchain keyword "`__ramfunc`".

GNU Compiler

RAM functions are defined using a specific toolchain attribute "`__attribute__((section(".RamFunc")))`".

28.1.2 ramfunc functions

This subsection provides a set of functions that should be executed from RAM transfers.

This section contains the following APIs:

- [`HAL\_FLASHEx\_StopFlashInterfaceClk\(\)`](#)
- [`HAL\_FLASHEx\_StartFlashInterfaceClk\(\)`](#)
- [`HAL\_FLASHEx\_EnableFlashSleepMode\(\)`](#)
- [`HAL\_FLASHEx\_DisableFlashSleepMode\(\)`](#)

28.1.3 Detailed description of functions

HAL_FLASHEx_StopFlashInterfaceClk

Function name

`__RAM_FUNC HAL_StatusTypeDef HAL_FLASHEx_StopFlashInterfaceClk (void )`

Function description

Stop the flash interface while System Run.

Return values

- **HAL:** status

Notes

- This mode is only available for STM32F41xxx/STM32F446xx devices.
- This mode couldn't be set while executing with the flash itself. It should be done with specific routine executed from RAM.

HAL_FLASHEx_StartFlashInterfaceClk

Function name

`__RAM_FUNC HAL_StatusTypeDef HAL_FLASHEx_StartFlashInterfaceClk (void )`

Function description

Start the flash interface while System Run.

Return values

- **HAL:** status

Notes

- This mode is only available for STM32F411xx/STM32F446xx devices.
- This mode couldn't be set while executing with the flash itself. It should be done with specific routine executed from RAM.

HAL_FLASHEx_EnableFlashSleepMode

Function name

__RAM_FUNC HAL_StatusTypeDef HAL_FLASHEx_EnableFlashSleepMode (void)

Function description

Enable the flash sleep while System Run.

Return values

- **HAL:** status

Notes

- This mode is only available for STM32F41xxx/STM32F446xx devices.
- This mode could n't be set while executing with the flash itself. It should be done with specific routine executed from RAM.

HAL_FLASHEx_DisableFlashSleepMode

Function name

__RAM_FUNC HAL_StatusTypeDef HAL_FLASHEx_DisableFlashSleepMode (void)

Function description

Disable the flash sleep while System Run.

Return values

- **HAL:** status

Notes

- This mode is only available for STM32F41xxx/STM32F446xx devices.
- This mode couldn't be set while executing with the flash itself. It should be done with specific routine executed from RAM.

29 HAL FMPI2C Generic Driver

29.1 FMPI2C Firmware driver registers structures

29.1.1 FMPI2C_InitTypeDef

FMPI2C_InitTypeDef is defined in the `stm32f4xx_hal_fmapi2c.h`

Data Fields

- *uint32_t Timing*
- *uint32_t OwnAddress1*
- *uint32_t AddressingMode*
- *uint32_t DualAddressMode*
- *uint32_t OwnAddress2*
- *uint32_t OwnAddress2Masks*
- *uint32_t GeneralCallMode*
- *uint32_t NoStretchMode*

Field Documentation

- *uint32_t FMPI2C_InitTypeDef::Timing*
Specifies the FMPI2C_TIMINGR_register value. This parameter calculated by referring to FMPI2C initialization section in Reference manual
- *uint32_t FMPI2C_InitTypeDef::OwnAddress1*
Specifies the first device own address. This parameter can be a 7-bit or 10-bit address.
- *uint32_t FMPI2C_InitTypeDef::AddressingMode*
Specifies if 7-bit or 10-bit addressing mode is selected. This parameter can be a value of [FMPI2C_ADDRESSING_MODE](#)
- *uint32_t FMPI2C_InitTypeDef::DualAddressMode*
Specifies if dual addressing mode is selected. This parameter can be a value of [FMPI2C_DUAL_ADDRESSING_MODE](#)
- *uint32_t FMPI2C_InitTypeDef::OwnAddress2*
Specifies the second device own address if dual addressing mode is selected This parameter can be a 7-bit address.
- *uint32_t FMPI2C_InitTypeDef::OwnAddress2Masks*
Specifies the acknowledge mask address second device own address if dual addressing mode is selected. This parameter can be a value of [FMPI2C_OWN_ADDRESS2_MASKS](#)
- *uint32_t FMPI2C_InitTypeDef::GeneralCallMode*
Specifies if general call mode is selected. This parameter can be a value of [FMPI2C_GENERAL_CALL_ADDRESSING_MODE](#)
- *uint32_t FMPI2C_InitTypeDef::NoStretchMode*
Specifies if nostretch mode is selected. This parameter can be a value of [FMPI2C_NOSTRETCH_MODE](#)

29.1.2 __FMPI2C_HandleTypeDef

__FMPI2C_HandleTypeDef is defined in the `stm32f4xx_hal_fmapi2c.h`

Data Fields

- *FMPI2C_TypeDef * Instance*
- *FMPI2C_InitTypeDef Init*
- *uint8_t * pBuffPtr*
- *uint16_t XferSize*
- *__IO uint16_t XferCount*
- *__IO uint32_t XferOptions*
- *__IO uint32_t PreviousState*
- *HAL_StatusTypeDef(* XferISR*
- *DMA_HandleTypeDef * hdmatx*

- **DMA_HandleTypeDef * hdmrx**
- **HAL_LockTypeDef Lock**
- **__IO HAL_FMPI2C_StateTypeDef State**
- **__IO HAL_FMPI2C_ModeTypeDef Mode**
- **__IO uint32_t ErrorCode**
- **__IO uint32_t AddrEventCount**
- **__IO uint32_t Devaddress**
- **__IO uint32_t Memaddress**
- **void(* MasterTxCpltCallback**
- **void(* MasterRxCpltCallback**
- **void(* SlaveTxCpltCallback**
- **void(* SlaveRxCpltCallback**
- **void(* ListenCpltCallback**
- **void(* MemTxCpltCallback**
- **void(* MemRxCpltCallback**
- **void(* ErrorCallback**
- **void(* AbortCpltCallback**
- **void(* AddrCallback**
- **void(* MspInitCallback**
- **void(* MspDeInitCallback**

Field Documentation

- **FMPI2C_TypeDef* __FMPI2C_HandleTypeDef::Instance**
FMPI2C registers base address
- **FMPI2C_InitTypeDef __FMPI2C_HandleTypeDef::Init**
FMPI2C communication parameters
- **uint8_t* __FMPI2C_HandleTypeDef::pBuffPtr**
Pointer to FMPI2C transfer buffer
- **uint16_t __FMPI2C_HandleTypeDef::XferSize**
FMPI2C transfer size
- **__IO uint16_t __FMPI2C_HandleTypeDef::XferCount**
FMPI2C transfer counter
- **__IO uint32_t __FMPI2C_HandleTypeDef::XferOptions**
FMPI2C sequential transfer options, this parameter can be a value of **FMPI2C_XFEROPTIONS**
- **__IO uint32_t __FMPI2C_HandleTypeDef::PreviousState**
FMPI2C communication Previous state
- **HAL_StatusTypeDef(* __FMPI2C_HandleTypeDef::XferISR)(struct __FMPI2C_HandleTypeDef *hfmapi2c, uint32_t ITFlags, uint32_t ITSources)**
FMPI2C transfer IRQ handler function pointer
- **DMA_HandleTypeDef* __FMPI2C_HandleTypeDef::hdmatx**
FMPI2C Tx DMA handle parameters
- **DMA_HandleTypeDef* __FMPI2C_HandleTypeDef::hdmrx**
FMPI2C Rx DMA handle parameters
- **HAL_LockTypeDef __FMPI2C_HandleTypeDef::Lock**
FMPI2C locking object
- **__IO HAL_FMPI2C_StateTypeDef __FMPI2C_HandleTypeDef::State**
FMPI2C communication state
- **__IO HAL_FMPI2C_ModeTypeDef __FMPI2C_HandleTypeDef::Mode**
FMPI2C communication mode
- **__IO uint32_t __FMPI2C_HandleTypeDef::ErrorCode**
FMPI2C Error code
- **__IO uint32_t __FMPI2C_HandleTypeDef::AddrEventCount**
FMPI2C Address Event counter

- **`__IO uint32_t __FMPI2C_HandleTypeDef::Devaddress`**
FMPI2C Target device address
- **`__IO uint32_t __FMPI2C_HandleTypeDef::Memaddress`**
FMPI2C Target memory address
- **`void(* __FMPI2C_HandleTypeDef::MasterTxCpltCallback)(struct __FMPI2C_HandleTypeDef *hfmapi2c)`**
FMPI2C Master Tx Transfer completed callback
- **`void(* __FMPI2C_HandleTypeDef::MasterRxCpltCallback)(struct __FMPI2C_HandleTypeDef *hfmapi2c)`**
FMPI2C Master Rx Transfer completed callback
- **`void(* __FMPI2C_HandleTypeDef::SlaveTxCpltCallback)(struct __FMPI2C_HandleTypeDef *hfmapi2c)`**
FMPI2C Slave Tx Transfer completed callback
- **`void(* __FMPI2C_HandleTypeDef::SlaveRxCpltCallback)(struct __FMPI2C_HandleTypeDef *hfmapi2c)`**
FMPI2C Slave Rx Transfer completed callback
- **`void(* __FMPI2C_HandleTypeDef::ListenCpltCallback)(struct __FMPI2C_HandleTypeDef *hfmapi2c)`**
FMPI2C Listen Complete callback
- **`void(* __FMPI2C_HandleTypeDef::MemTxCpltCallback)(struct __FMPI2C_HandleTypeDef *hfmapi2c)`**
FMPI2C Memory Tx Transfer completed callback
- **`void(* __FMPI2C_HandleTypeDef::MemRxCpltCallback)(struct __FMPI2C_HandleTypeDef *hfmapi2c)`**
FMPI2C Memory Rx Transfer completed callback
- **`void(* __FMPI2C_HandleTypeDef::ErrorCallback)(struct __FMPI2C_HandleTypeDef *hfmapi2c)`**
FMPI2C Error callback
- **`void(* __FMPI2C_HandleTypeDef::AbortCpltCallback)(struct __FMPI2C_HandleTypeDef *hfmapi2c)`**
FMPI2C Abort callback
- **`void(* __FMPI2C_HandleTypeDef::AddrCallback)(struct __FMPI2C_HandleTypeDef *hfmapi2c, uint8_t TransferDirection, uint16_t AddrMatchCode)`**
FMPI2C Slave Address Match callback
- **`void(* __FMPI2C_HandleTypeDef::MspInitCallback)(struct __FMPI2C_HandleTypeDef *hfmapi2c)`**
FMPI2C Msp Init callback
- **`void(* __FMPI2C_HandleTypeDef::MspDeInitCallback)(struct __FMPI2C_HandleTypeDef *hfmapi2c)`**
FMPI2C Msp DeInit callback

29.2 FMPI2C Firmware driver API description

The following section lists the various functions of the FMPI2C library.

29.2.1 How to use this driver

The FMPI2C HAL driver can be used as follows:

1. Declare a FMPI2C_HandleTypeDef handle structure, for example: FMPI2C_HandleTypeDef hfmapi2c;

2. Initialize the FMPI2C low level resources by implementing the HAL_FMPI2C_Msplnit() API:
 - a. Enable the FMPI2Cx interface clock
 - b. FMPI2C pins configuration
 - Enable the clock for the FMPI2C GPIOs
 - Configure FMPI2C pins as alternate function open-drain
 - c. NVIC configuration if you need to use interrupt process
 - Configure the FMPI2Cx interrupt priority
 - Enable the NVIC FMPI2C IRQ Channel
 - d. DMA Configuration if you need to use DMA process
 - Declare a DMA_HandleTypeDef handle structure for the transmit or receive stream
 - Enable the DMAx interface clock using
 - Configure the DMA handle parameters
 - Configure the DMA Tx or Rx stream
 - Associate the initialized DMA handle to the hfmapi2c DMA Tx or Rx handle
 - Configure the priority and enable the NVIC for the transfer complete interrupt on the DMA Tx or Rx stream
3. Configure the Communication Clock Timing, Own Address1, Master Addressing mode, Dual Addressing mode, Own Address2, Own Address2 Mask, General call and Nostretch mode in the hfmapi2c Init structure.
4. Initialize the FMPI2C registers by calling the HAL_FMPI2C_Init(), configures also the low level Hardware (GPIO, CLOCK, NVIC...etc) by calling the customized HAL_FMPI2C_Msplnit(&hfmapi2c) API.
5. To check if target device is ready for communication, use the function HAL_FMPI2C_IsDeviceReady()
6. For FMPI2C IO and IO MEM operations, three operation modes are available within this driver :

Polling mode IO operation

- Transmit in master mode an amount of data in blocking mode using HAL_FMPI2C_Master_Transmit()
- Receive in master mode an amount of data in blocking mode using HAL_FMPI2C_Master_Receive()
- Transmit in slave mode an amount of data in blocking mode using HAL_FMPI2C_Slave_Transmit()
- Receive in slave mode an amount of data in blocking mode using HAL_FMPI2C_Slave_Receive()

Polling mode IO MEM operation

- Write an amount of data in blocking mode to a specific memory address using HAL_FMPI2C_Mem_Write()
- Read an amount of data in blocking mode from a specific memory address using HAL_FMPI2C_Mem_Read()

Interrupt mode IO operation

- Transmit in master mode an amount of data in non-blocking mode using HAL_FMPI2C_Master_Transmit_IT()
- At transmission end of transfer, HAL_FMPI2C_MasterTxCpltCallback() is executed and users can add their own code by customization of function pointer HAL_FMPI2C_MasterTxCpltCallback()
- Receive in master mode an amount of data in non-blocking mode using HAL_FMPI2C_Master_Receive_IT()
- At reception end of transfer, HAL_FMPI2C_MasterRxCpltCallback() is executed and users can add their own code by customization of function pointer HAL_FMPI2C_MasterRxCpltCallback()
- Transmit in slave mode an amount of data in non-blocking mode using HAL_FMPI2C_Slave_Transmit_IT()
- At transmission end of transfer, HAL_FMPI2C_SlaveTxCpltCallback() is executed and users can add their own code by customization of function pointer HAL_FMPI2C_SlaveTxCpltCallback()
- Receive in slave mode an amount of data in non-blocking mode using HAL_FMPI2C_Slave_Receive_IT()
- At reception end of transfer, HAL_FMPI2C_SlaveRxCpltCallback() is executed and users can add their own code by customization of function pointer HAL_FMPI2C_SlaveRxCpltCallback()
- In case of transfer Error, HAL_FMPI2C_ErrorCallback() function is executed and users can add their own code by customization of function pointer HAL_FMPI2C_ErrorCallback()

- Abort a master FMPI2C process communication with Interrupt using `HAL_FMPI2C_Master_Abort_IT()`
- End of abort process, `HAL_FMPI2C_AbortCpltCallback()` is executed and users can add their own code by customization of function pointer `HAL_FMPI2C_AbortCpltCallback()`
- Discard a slave FMPI2C process communication using `__HAL_FMPI2C_GENERATE_NACK()` macro. This action will inform Master to generate a Stop condition to discard the communication.

Interrupt mode or DMA mode IO sequential operation

Note:

These interfaces allow to manage a sequential transfer with a repeated start condition when a direction change during transfer

- A specific option field manage the different steps of a sequential transfer
- Option field values are defined through `FMPI2C_XFEROPTIONS` and are listed below:
 - `FMPI2C_FIRST_AND_LAST_FRAME`: No sequential usage, functional is same as associated interfaces in no sequential mode
 - `FMPI2C_FIRST_FRAME`: Sequential usage, this option allow to manage a sequence with start condition, address and data to transfer without a final stop condition
 - `FMPI2C_FIRST_AND_NEXT_FRAME`: Sequential usage (Master only), this option allow to manage a sequence with start condition, address and data to transfer without a final stop condition, an then permit a call the same master sequential interface several times (like `HAL_FMPI2C_Master_Seq_Transmit_IT()` then `HAL_FMPI2C_Master_Seq_Transmit_IT()` or `HAL_FMPI2C_Master_Seq_Transmit_DMA()` then `HAL_FMPI2C_Master_Seq_Transmit_DMA()`)
 - `FMPI2C_NEXT_FRAME`: Sequential usage, this option allow to manage a sequence with a restart condition, address and with new data to transfer if the direction change or manage only the new data to transfer if no direction change and without a final stop condition in both cases
 - `FMPI2C_LAST_FRAME`: Sequential usage, this option allow to manage a sequence with a restart condition, address and with new data to transfer if the direction change or manage only the new data to transfer if no direction change and with a final stop condition in both cases
 - `FMPI2C_LAST_FRAME_NO_STOP`: Sequential usage (Master only), this option allow to manage a restart condition after several call of the same master sequential interface several times (link with option `FMPI2C_FIRST_AND_NEXT_FRAME`). Usage can, transfer several bytes one by one using `HAL_FMPI2C_Master_Seq_Transmit_IT` or `HAL_FMPI2C_Master_Seq_Receive_IT` or `HAL_FMPI2C_Master_Seq_Transmit_DMA` or `HAL_FMPI2C_Master_Seq_Receive_DMA` with option `FMPI2C_FIRST_AND_NEXT_FRAME` then `FMPI2C_NEXT_FRAME`. Then usage of this option `FMPI2C_LAST_FRAME_NO_STOP` at the last Transmit or Receive sequence permit to call the opposite interface Receive or Transmit without stopping the communication and so generate a restart condition.
 - `FMPI2C_OTHER_FRAME`: Sequential usage (Master only), this option allow to manage a restart condition after each call of the same master sequential interface. Usage can, transfer several bytes one by one with a restart with slave address between each bytes using `HAL_FMPI2C_Master_Seq_Transmit_IT` or `HAL_FMPI2C_Master_Seq_Receive_IT` or `HAL_FMPI2C_Master_Seq_Transmit_DMA` or `HAL_FMPI2C_Master_Seq_Receive_DMA` with option `FMPI2C_FIRST_FRAME` then `FMPI2C_OTHER_FRAME`. Then usage of this option `FMPI2C_OTHER_AND_LAST_FRAME` at the last frame to help automatic generation of STOP condition.

- Different sequential FMPI2C interfaces are listed below:
 - Sequential transmit in master FMPI2C mode an amount of data in non-blocking mode using HAL_FMPI2C_Master_Seq_Transmit_IT() or using HAL_FMPI2C_Master_Seq_Transmit_DMA()
 - At transmission end of current frame transfer, HAL_FMPI2C_MasterTxCpltCallback() is executed and users can add their own code by customization of function pointer HAL_FMPI2C_MasterTxCpltCallback()
 - Sequential receive in master FMPI2C mode an amount of data in non-blocking mode using HAL_FMPI2C_Master_Seq_Receive_IT() or using HAL_FMPI2C_Master_Seq_Receive_DMA()
 - At reception end of current frame transfer, HAL_FMPI2C_MasterRxCpltCallback() is executed and users can add their own code by customization of function pointer HAL_FMPI2C_MasterRxCpltCallback()
 - Abort a master IT or DMA FMPI2C process communication with Interrupt using HAL_FMPI2C_Master_Abort_IT()
 - End of abort process, HAL_FMPI2C_AbortCpltCallback() is executed and users can add their own code by customization of function pointer HAL_FMPI2C_AbortCpltCallback()
 - Enable/disable the Address listen mode in slave FMPI2C mode using HAL_FMPI2C_EnableListen_IT() HAL_FMPI2C_DisableListen_IT()
 - When address slave FMPI2C match, HAL_FMPI2C_AddrCallback() is executed and users can add their own code to check the Address Match Code and the transmission direction request by master (Write/Read).
 - At Listen mode end HAL_FMPI2C_ListenCpltCallback() is executed and users can add their own code by customization of function pointer HAL_FMPI2C_ListenCpltCallback()
 - Sequential transmit in slave FMPI2C mode an amount of data in non-blocking mode using HAL_FMPI2C_Slave_Seq_Transmit_IT() or using HAL_FMPI2C_Slave_Seq_Transmit_DMA()
 - At transmission end of current frame transfer, HAL_FMPI2C_SlaveTxCpltCallback() is executed and users can add their own code by customization of function pointer HAL_FMPI2C_SlaveTxCpltCallback()
 - Sequential receive in slave FMPI2C mode an amount of data in non-blocking mode using HAL_FMPI2C_Slave_Seq_Receive_IT() or using HAL_FMPI2C_Slave_Seq_Receive_DMA()
 - At reception end of current frame transfer, HAL_FMPI2C_SlaveRxCpltCallback() is executed and users can add their own code by customization of function pointer HAL_FMPI2C_SlaveRxCpltCallback()
 - In case of transfer Error, HAL_FMPI2C_ErrorCallback() function is executed and users can add their own code by customization of function pointer HAL_FMPI2C_ErrorCallback()
 - Discard a slave FMPI2C process communication using __HAL_FMPI2C_GENERATE_NACK() macro. This action will inform Master to generate a Stop condition to discard the communication.

Interrupt mode IO MEM operation

- Write an amount of data in non-blocking mode with Interrupt to a specific memory address using HAL_FMPI2C_Mem_Write_IT()
- At Memory end of write transfer, HAL_FMPI2C_MemTxCpltCallback() is executed and users can add their own code by customization of function pointer HAL_FMPI2C_MemTxCpltCallback()
- Read an amount of data in non-blocking mode with Interrupt from a specific memory address using HAL_FMPI2C_Mem_Read_IT()
- At Memory end of read transfer, HAL_FMPI2C_MemRxCpltCallback() is executed and users can add their own code by customization of function pointer HAL_FMPI2C_MemRxCpltCallback()
- In case of transfer Error, HAL_FMPI2C_ErrorCallback() function is executed and users can add their own code by customization of function pointer HAL_FMPI2C_ErrorCallback()

DMA mode IO operation

- Transmit in master mode an amount of data in non-blocking mode (DMA) using HAL_FMPI2C_Master_Transmit_DMA()
- At transmission end of transfer, HAL_FMPI2C_MasterTxCpltCallback() is executed and users can add their own code by customization of function pointer HAL_FMPI2C_MasterTxCpltCallback()

- Receive in master mode an amount of data in non-blocking mode (DMA) using `HAL_FMPI2C_Master_Receive_DMA()`
- At reception end of transfer, `HAL_FMPI2C_MasterRxCpltCallback()` is executed and users can add their own code by customization of function pointer `HAL_FMPI2C_MasterRxCpltCallback()`
- Transmit in slave mode an amount of data in non-blocking mode (DMA) using `HAL_FMPI2C_Slave_Transmit_DMA()`
- At transmission end of transfer, `HAL_FMPI2C_SlaveTxCpltCallback()` is executed and users can add their own code by customization of function pointer `HAL_FMPI2C_SlaveTxCpltCallback()`
- Receive in slave mode an amount of data in non-blocking mode (DMA) using `HAL_FMPI2C_Slave_Receive_DMA()`
- At reception end of transfer, `HAL_FMPI2C_SlaveRxCpltCallback()` is executed and users can add their own code by customization of function pointer `HAL_FMPI2C_SlaveRxCpltCallback()`
- In case of transfer Error, `HAL_FMPI2C_ErrorCallback()` function is executed and users can add their own code by customization of function pointer `HAL_FMPI2C_ErrorCallback()`
- Abort a master FMPI2C process communication with Interrupt using `HAL_FMPI2C_Master_Abort_IT()`
- End of abort process, `HAL_FMPI2C_AbortCpltCallback()` is executed and users can add their own code by customization of function pointer `HAL_FMPI2C_AbortCpltCallback()`
- Discard a slave FMPI2C process communication using `__HAL_FMPI2C_GENERATE_NACK()` macro. This action will inform Master to generate a Stop condition to discard the communication.

DMA mode IO MEM operation

- Write an amount of data in non-blocking mode with DMA to a specific memory address using `HAL_FMPI2C_Mem_Write_DMA()`
- At Memory end of write transfer, `HAL_FMPI2C_MemTxCpltCallback()` is executed and users can add their own code by customization of function pointer `HAL_FMPI2C_MemTxCpltCallback()`
- Read an amount of data in non-blocking mode with DMA from a specific memory address using `HAL_FMPI2C_Mem_Read_DMA()`
- At Memory end of read transfer, `HAL_FMPI2C_MemRxCpltCallback()` is executed and users can add their own code by customization of function pointer `HAL_FMPI2C_MemRxCpltCallback()`
- In case of transfer Error, `HAL_FMPI2C_ErrorCallback()` function is executed and users can add their own code by customization of function pointer `HAL_FMPI2C_ErrorCallback()`

FMPI2C HAL driver macros list

Below the list of most used macros in FMPI2C HAL driver.

- `__HAL_FMPI2C_ENABLE`: Enable the FMPI2C peripheral
- `__HAL_FMPI2C_DISABLE`: Disable the FMPI2C peripheral
- `__HAL_FMPI2C_GENERATE_NACK`: Generate a Non-Acknowledge FMPI2C peripheral in Slave mode
- `__HAL_FMPI2C_GET_FLAG`: Check whether the specified FMPI2C flag is set or not
- `__HAL_FMPI2C_CLEAR_FLAG`: Clear the specified FMPI2C pending flag
- `__HAL_FMPI2C_ENABLE_IT`: Enable the specified FMPI2C interrupt
- `__HAL_FMPI2C_DISABLE_IT`: Disable the specified FMPI2C interrupt

Callback registration

The compilation flag `USE_HAL_FMPI2C_REGISTER_CALLBACKS` when set to 1 allows the user to configure dynamically the driver callbacks. Use Functions `HAL_FMPI2C_RegisterCallback()` or `HAL_FMPI2C_RegisterAddrCallback()` to register an interrupt callback.

Function `HAL_FMPI2C_RegisterCallback()` allows to register following callbacks:

- `MasterTxCpltCallback` : callback for Master transmission end of transfer.
- `MasterRxCpltCallback` : callback for Master reception end of transfer.
- `SlaveTxCpltCallback` : callback for Slave transmission end of transfer.
- `SlaveRxCpltCallback` : callback for Slave reception end of transfer.
- `ListenCpltCallback` : callback for end of listen mode.
- `MemTxCpltCallback` : callback for Memory transmission end of transfer.

- `MemRxCpltCallback` : callback for Memory reception end of transfer.
- `ErrorCallback` : callback for error detection.
- `AbortCpltCallback` : callback for abort completion process.
- `MspInitCallback` : callback for Msp Init.
- `MspDeInitCallback` : callback for Msp DeInit. This function takes as parameters the HAL peripheral handle, the Callback ID and a pointer to the user callback function.

For specific callback `AddrCallback` use dedicated register callbacks : `HAL_FMPI2C_RegisterAddrCallback()`.

Use function `HAL_FMPI2C_UnRegisterCallback` to reset a callback to the default weak function.

`HAL_FMPI2C_UnRegisterCallback` takes as parameters the HAL peripheral handle, and the Callback ID. This function allows to reset following callbacks:

- `MasterTxCpltCallback` : callback for Master transmission end of transfer.
- `MasterRxCpltCallback` : callback for Master reception end of transfer.
- `SlaveTxCpltCallback` : callback for Slave transmission end of transfer.
- `SlaveRxCpltCallback` : callback for Slave reception end of transfer.
- `ListenCpltCallback` : callback for end of listen mode.
- `MemTxCpltCallback` : callback for Memory transmission end of transfer.
- `MemRxCpltCallback` : callback for Memory reception end of transfer.
- `ErrorCallback` : callback for error detection.
- `AbortCpltCallback` : callback for abort completion process.
- `MspInitCallback` : callback for Msp Init.
- `MspDeInitCallback` : callback for Msp DeInit.

For callback `AddrCallback` use dedicated register callbacks : `HAL_FMPI2C_UnRegisterAddrCallback()`.

By default, after the `HAL_FMPI2C_Init()` and when the state is `HAL_FMPI2C_STATE_RESET` all callbacks are set to the corresponding weak functions: examples `HAL_FMPI2C_MasterTxCpltCallback()`, `HAL_FMPI2C_MasterRxCpltCallback()`. Exception done for `MspInit` and `MspDeInit` functions that are reset to the legacy weak functions in the `HAL_FMPI2C_Init()/ HAL_FMPI2C_DeInit()` only when these callbacks are null (not registered beforehand). If `MspInit` or `MspDeInit` are not null, the `HAL_FMPI2C_Init()/ HAL_FMPI2C_DeInit()` keep and use the user `MspInit/MspDeInit` callbacks (registered beforehand) whatever the state.

Callbacks can be registered/unregistered in `HAL_FMPI2C_STATE_READY` state only. Exception done `MspInit/MspDeInit` functions that can be registered/unregistered in `HAL_FMPI2C_STATE_READY` or `HAL_FMPI2C_STATE_RESET` state, thus registered (user) `MspInit/DeInit` callbacks can be used during the `Init/DeInit`. Then, the user first registers the `MspInit/MspDeInit` user callbacks using `HAL_FMPI2C_RegisterCallback()` before calling `HAL_FMPI2C_DeInit()` or `HAL_FMPI2C_Init()` function.

When the compilation flag `USE_HAL_FMPI2C_REGISTER_CALLBACKS` is set to 0 or not defined, the callback registration feature is not available and all callbacks are set to the corresponding weak functions.

Note: You can refer to the *FMPI2C HAL driver header file* for more useful macros

29.2.2 Initialization and de-initialization functions

This subsection provides a set of functions allowing to initialize and deinitialize the FMPI2Cx peripheral:

- User must Implement `HAL_FMPI2C_MspInit()` function in which he configures all related peripherals resources (CLOCK, GPIO, DMA, IT and NVIC).
- Call the function `HAL_FMPI2C_Init()` to configure the selected device with the selected configuration:
 - Clock Timing
 - Own Address 1
 - Addressing mode (Master, Slave)
 - Dual Addressing mode
 - Own Address 2
 - Own Address 2 Mask
 - General call mode
 - Nostretch mode
- Call the function `HAL_FMPI2C_DeInit()` to restore the default configuration of the selected FMPI2Cx peripheral.

This section contains the following APIs:

- *HAL_FMPI2C_Init()*
- *HAL_FMPI2C_DeInit()*
- *HAL_FMPI2C_MspInit()*
- *HAL_FMPI2C_MspDeInit()*
- *HAL_FMPI2C_RegisterCallback()*
- *HAL_FMPI2C_UnRegisterCallback()*
- *HAL_FMPI2C_RegisterAddrCallback()*
- *HAL_FMPI2C_UnRegisterAddrCallback()*

29.2.3

IO operation functions

This subsection provides a set of functions allowing to manage the FMPI2C data transfers.

1. There are two modes of transfer:
 - Blocking mode : The communication is performed in the polling mode. The status of all data processing is returned by the same function after finishing transfer.
 - No-Blocking mode : The communication is performed using Interrupts or DMA. These functions return the status of the transfer startup. The end of the data processing will be indicated through the dedicated FMPI2C IRQ when using Interrupt mode or the DMA IRQ when using DMA mode.
2. Blocking mode functions are :
 - *HAL_FMPI2C_Master_Transmit()*
 - *HAL_FMPI2C_Master_Receive()*
 - *HAL_FMPI2C_Slave_Transmit()*
 - *HAL_FMPI2C_Slave_Receive()*
 - *HAL_FMPI2C_Mem_Write()*
 - *HAL_FMPI2C_Mem_Read()*
 - *HAL_FMPI2C_IsDeviceReady()*
3. No-Blocking mode functions with Interrupt are :
 - *HAL_FMPI2C_Master_Transmit_IT()*
 - *HAL_FMPI2C_Master_Receive_IT()*
 - *HAL_FMPI2C_Slave_Transmit_IT()*
 - *HAL_FMPI2C_Slave_Receive_IT()*
 - *HAL_FMPI2C_Mem_Write_IT()*
 - *HAL_FMPI2C_Mem_Read_IT()*
 - *HAL_FMPI2C_Master_Seq_Transmit_IT()*
 - *HAL_FMPI2C_Master_Seq_Receive_IT()*
 - *HAL_FMPI2C_Slave_Seq_Transmit_IT()*
 - *HAL_FMPI2C_Slave_Seq_Receive_IT()*
 - *HAL_FMPI2C_EnableListen_IT()*
 - *HAL_FMPI2C_DisableListen_IT()*
 - *HAL_FMPI2C_Master_Abort_IT()*

4. No-Blocking mode functions with DMA are :
 - HAL_FMPI2C_Master_Transmit_DMA()
 - HAL_FMPI2C_Master_Receive_DMA()
 - HAL_FMPI2C_Slave_Transmit_DMA()
 - HAL_FMPI2C_Slave_Receive_DMA()
 - HAL_FMPI2C_Mem_Write_DMA()
 - HAL_FMPI2C_Mem_Read_DMA()
 - HAL_FMPI2C_Master_Seq_Transmit_DMA()
 - HAL_FMPI2C_Master_Seq_Receive_DMA()
 - HAL_FMPI2C_Slave_Seq_Transmit_DMA()
 - HAL_FMPI2C_Slave_Seq_Receive_DMA()
5. A set of Transfer Complete Callbacks are provided in non Blocking mode:
 - HAL_FMPI2C_MasterTxCpltCallback()
 - HAL_FMPI2C_MasterRxCpltCallback()
 - HAL_FMPI2C_SlaveTxCpltCallback()
 - HAL_FMPI2C_SlaveRxCpltCallback()
 - HAL_FMPI2C_MemTxCpltCallback()
 - HAL_FMPI2C_MemRxCpltCallback()
 - HAL_FMPI2C_AddrCallback()
 - HAL_FMPI2C_ListenCpltCallback()
 - HAL_FMPI2C_ErrorCallback()
 - HAL_FMPI2C_AbortCpltCallback()

This section contains the following APIs:

- *HAL_FMPI2C_Master_Transmit()*
- *HAL_FMPI2C_Master_Receive()*
- *HAL_FMPI2C_Slave_Transmit()*
- *HAL_FMPI2C_Slave_Receive()*
- *HAL_FMPI2C_Master_Transmit_IT()*
- *HAL_FMPI2C_Master_Receive_IT()*
- *HAL_FMPI2C_Slave_Transmit_IT()*
- *HAL_FMPI2C_Slave_Receive_IT()*
- *HAL_FMPI2C_Master_Transmit_DMA()*
- *HAL_FMPI2C_Master_Receive_DMA()*
- *HAL_FMPI2C_Slave_Transmit_DMA()*
- *HAL_FMPI2C_Slave_Receive_DMA()*
- *HAL_FMPI2C_Mem_Write()*
- *HAL_FMPI2C_Mem_Read()*
- *HAL_FMPI2C_Mem_Write_IT()*
- *HAL_FMPI2C_Mem_Read_IT()*
- *HAL_FMPI2C_Mem_Write_DMA()*
- *HAL_FMPI2C_Mem_Read_DMA()*
- *HAL_FMPI2C_IsDeviceReady()*
- *HAL_FMPI2C_Master_Seq_Transmit_IT()*
- *HAL_FMPI2C_Master_Seq_Transmit_DMA()*
- *HAL_FMPI2C_Master_Seq_Receive_IT()*
- *HAL_FMPI2C_Master_Seq_Receive_DMA()*
- *HAL_FMPI2C_Slave_Seq_Transmit_IT()*
- *HAL_FMPI2C_Slave_Seq_Transmit_DMA()*
- *HAL_FMPI2C_Slave_Seq_Receive_IT()*
- *HAL_FMPI2C_Slave_Seq_Receive_DMA()*

- [*HAL_FMPI2C_EnableListen_IT\(\)*](#)
- [*HAL_FMPI2C_DisableListen_IT\(\)*](#)
- [*HAL_FMPI2C_Master_Abort_IT\(\)*](#)

29.2.4 Peripheral State, Mode and Error functions

This subsection permit to get in run-time the status of the peripheral and the data flow.

This section contains the following APIs:

- [*HAL_FMPI2C_GetState\(\)*](#)
- [*HAL_FMPI2C_GetMode\(\)*](#)
- [*HAL_FMPI2C_GetError\(\)*](#)

29.2.5 Detailed description of functions

HAL_FMPI2C_Init

Function name

HAL_StatusTypeDef HAL_FMPI2C_Init (FMPI2C_HandleTypeDef * hfmapi2c)

Function description

Initializes the FMPI2C according to the specified parameters in the FMPI2C_InitTypeDef and initialize the associated handle.

Parameters

- **hfmapi2c:** Pointer to a FMPI2C_HandleTypeDef structure that contains the configuration information for the specified FMPI2C.

Return values

- **HAL:** status

HAL_FMPI2C_DeInit

Function name

HAL_StatusTypeDef HAL_FMPI2C_DeInit (FMPI2C_HandleTypeDef * hfmapi2c)

Function description

DeInitialize the FMPI2C peripheral.

Parameters

- **hfmapi2c:** Pointer to a FMPI2C_HandleTypeDef structure that contains the configuration information for the specified FMPI2C.

Return values

- **HAL:** status

HAL_FMPI2C_MspsInit

Function name

void HAL_FMPI2C_MspsInit (FMPI2C_HandleTypeDef * hfmapi2c)

Function description

Initialize the FMPI2C MSP.

Parameters

- **hfmapi2c:** Pointer to a FMPI2C_HandleTypeDef structure that contains the configuration information for the specified FMPI2C.

Return values

- **None:**

HAL_FMPI2C_MspDeInit

Function name

void HAL_FMPI2C_MspDeInit (FMPI2C_HandleTypeDef * hfmapi2c)

Function description

DeInitialize the FMPI2C MSP.

Parameters

- **hfmapi2c:** Pointer to a FMPI2C_HandleTypeDef structure that contains the configuration information for the specified FMPI2C.

Return values

- **None:**

HAL_FMPI2C_RegisterCallback

Function name

HAL_StatusTypeDef HAL_FMPI2C_RegisterCallback (FMPI2C_HandleTypeDef * hfmapi2c, HAL_FMPI2C_CallbackIDTypeDef CallbackID, pFMPI2C_CallbackTypeDef pCallback)

Function description

Register a User FMPI2C Callback To be used instead of the weak predefined callback.

Parameters

- **hfmapi2c:** Pointer to a FMPI2C_HandleTypeDef structure that contains the configuration information for the specified FMPI2C.
- **CallbackID:** ID of the callback to be registered This parameter can be one of the following values:
 - HAL_FMPI2C_MASTER_TX_COMPLETE_CB_ID Master Tx Transfer completed callback ID
 - HAL_FMPI2C_MASTER_RX_COMPLETE_CB_ID Master Rx Transfer completed callback ID
 - HAL_FMPI2C_SLAVE_TX_COMPLETE_CB_ID Slave Tx Transfer completed callback ID
 - HAL_FMPI2C_SLAVE_RX_COMPLETE_CB_ID Slave Rx Transfer completed callback ID
 - HAL_FMPI2C_LISTEN_COMPLETE_CB_ID Listen Complete callback ID
 - HAL_FMPI2C_MEM_TX_COMPLETE_CB_ID Memory Tx Transfer callback ID
 - HAL_FMPI2C_MEM_RX_COMPLETE_CB_ID Memory Rx Transfer completed callback ID
 - HAL_FMPI2C_ERROR_CB_ID Error callback ID
 - HAL_FMPI2C_ABORT_CB_ID Abort callback ID
 - HAL_FMPI2C_MSPINIT_CB_ID Mspinit callback ID
 - HAL_FMPI2C_MSPDEINIT_CB_ID MspDeinit callback ID
- **pCallback:** pointer to the Callback function

Return values

- **HAL:** status

Notes

- The HAL_FMPI2C_RegisterCallback() may be called before HAL_FMPI2C_Init() in HAL_FMPI2C_STATE_RESET to register callbacks for HAL_FMPI2C_MSPINIT_CB_ID and HAL_FMPI2C_MSPDEINIT_CB_ID.

HAL_FMPI2C_UnRegisterCallback

Function name

HAL_StatusTypeDef HAL_FMPI2C_UnRegisterCallback (FMPI2C_HandleTypeDef * hfmapi2c, HAL_FMPI2C_CallbackIDTypeDef CallbackID)

Function description

Unregister an FMPI2C Callback FMPI2C callback is redirected to the weak predefined callback.

Parameters

- **hfmapi2c**: Pointer to a FMPI2C_HandleTypeDef structure that contains the configuration information for the specified FMPI2C.
- **CallbackID**: ID of the callback to be unregistered This parameter can be one of the following values: This parameter can be one of the following values:
 - HAL_FMPI2C_MASTER_TX_COMPLETE_CB_ID Master Tx Transfer completed callback ID
 - HAL_FMPI2C_MASTER_RX_COMPLETE_CB_ID Master Rx Transfer completed callback ID
 - HAL_FMPI2C_SLAVE_TX_COMPLETE_CB_ID Slave Tx Transfer completed callback ID
 - HAL_FMPI2C_SLAVE_RX_COMPLETE_CB_ID Slave Rx Transfer completed callback ID
 - HAL_FMPI2C_LISTEN_COMPLETE_CB_ID Listen Complete callback ID
 - HAL_FMPI2C_MEM_TX_COMPLETE_CB_ID Memory Tx Transfer callback ID
 - HAL_FMPI2C_MEM_RX_COMPLETE_CB_ID Memory Rx Transfer completed callback ID
 - HAL_FMPI2C_ERROR_CB_ID Error callback ID
 - HAL_FMPI2C_ABORT_CB_ID Abort callback ID
 - HAL_FMPI2C_MSPINIT_CB_ID MspInit callback ID
 - HAL_FMPI2C_MSPDEINIT_CB_ID MspDeInit callback ID

Return values

- **HAL**: status

Notes

- The HAL_FMPI2C_UnRegisterCallback() may be called before HAL_FMPI2C_Init() in HAL_FMPI2C_STATE_RESET to un-register callbacks for HAL_FMPI2C_MSPINIT_CB_ID and HAL_FMPI2C_MSPDEINIT_CB_ID.

HAL_FMPI2C_RegisterAddrCallback

Function name

HAL_StatusTypeDef HAL_FMPI2C_RegisterAddrCallback (FMPI2C_HandleTypeDef * hfmapi2c, pFMPI2C_AddrCallbackTypeDef pCallback)

Function description

Register the Slave Address Match FMPI2C Callback To be used instead of the weak HAL_FMPI2C_AddrCallback() predefined callback.

Parameters

- **hfmapi2c**: Pointer to a FMPI2C_HandleTypeDef structure that contains the configuration information for the specified FMPI2C.
- **pCallback**: pointer to the Address Match Callback function

Return values

- **HAL**: status

HAL_FMPI2C_UnRegisterAddrCallback

Function name

HAL_StatusTypeDef HAL_FMPI2C_UnRegisterAddrCallback (FMPI2C_HandleTypeDef * hfmapi2c)

Function description

UnRegister the Slave Address Match FMPI2C Callback Info Ready FMPI2C Callback is redirected to the weak HAL_FMPI2C_AddrCallback() predefined callback.

Parameters

- **hfmapi2c:** Pointer to a FMPI2C_HandleTypeDef structure that contains the configuration information for the specified FMPI2C.

Return values

- **HAL:** status

HAL_FMPI2C_Master_Transmit

Function name

HAL_StatusTypeDef HAL_FMPI2C_Master_Transmit (FMPI2C_HandleTypeDef * hfmapi2c, uint16_t DevAddress, uint8_t * pData, uint16_t Size, uint32_t Timeout)

Function description

Transmits in master mode an amount of data in blocking mode.

Parameters

- **hfmapi2c:** Pointer to a FMPI2C_HandleTypeDef structure that contains the configuration information for the specified FMPI2C.
- **DevAddress:** Target device address: The device 7 bits address value in datasheet must be shifted to the left before calling the interface
- **pData:** Pointer to data buffer
- **Size:** Amount of data to be sent
- **Timeout:** Timeout duration

Return values

- **HAL:** status

HAL_FMPI2C_Master_Receive

Function name

HAL_StatusTypeDef HAL_FMPI2C_Master_Receive (FMPI2C_HandleTypeDef * hfmapi2c, uint16_t DevAddress, uint8_t * pData, uint16_t Size, uint32_t Timeout)

Function description

Receives in master mode an amount of data in blocking mode.

Parameters

- **hfmapi2c:** Pointer to a FMPI2C_HandleTypeDef structure that contains the configuration information for the specified FMPI2C.
- **DevAddress:** Target device address: The device 7 bits address value in datasheet must be shifted to the left before calling the interface
- **pData:** Pointer to data buffer
- **Size:** Amount of data to be sent
- **Timeout:** Timeout duration

Return values

- **HAL:** status

HAL_FMPI2C_Slave_Transmit

Function name

HAL_StatusTypeDef HAL_FMPI2C_Slave_Transmit (FMPI2C_HandleTypeDef * hfmapi2c, uint8_t * pData, uint16_t Size, uint32_t Timeout)

Function description

Transmits in slave mode an amount of data in blocking mode.

Parameters

- **hfmapi2c**: Pointer to a FMPI2C_HandleTypeDef structure that contains the configuration information for the specified FMPI2C.
- **pData**: Pointer to data buffer
- **Size**: Amount of data to be sent
- **Timeout**: Timeout duration

Return values

- **HAL**: status

HAL_FMPI2C_Slave_Receive

Function name

HAL_StatusTypeDef HAL_FMPI2C_Slave_Receive (FMPI2C_HandleTypeDef * hfmapi2c, uint8_t * pData, uint16_t Size, uint32_t Timeout)

Function description

Receive in slave mode an amount of data in blocking mode.

Parameters

- **hfmapi2c**: Pointer to a FMPI2C_HandleTypeDef structure that contains the configuration information for the specified FMPI2C.
- **pData**: Pointer to data buffer
- **Size**: Amount of data to be sent
- **Timeout**: Timeout duration

Return values

- **HAL**: status

HAL_FMPI2C_Mem_Write

Function name

HAL_StatusTypeDef HAL_FMPI2C_Mem_Write (FMPI2C_HandleTypeDef * hfmapi2c, uint16_t DevAddress, uint16_t MemAddress, uint16_t MemAddSize, uint8_t * pData, uint16_t Size, uint32_t Timeout)

Function description

Write an amount of data in blocking mode to a specific memory address.

Parameters

- **hfmapi2c:** Pointer to a FMPI2C_HandleTypeDef structure that contains the configuration information for the specified FMPI2C.
- **DevAddress:** Target device address: The device 7 bits address value in datasheet must be shifted to the left before calling the interface
- **MemAddress:** Internal memory address
- **MemAddSize:** Size of internal memory address
- **pData:** Pointer to data buffer
- **Size:** Amount of data to be sent
- **Timeout:** Timeout duration

Return values

- **HAL:** status

HAL_FMPI2C_Mem_Read

Function name

HAL_StatusTypeDef HAL_FMPI2C_Mem_Read (FMPI2C_HandleTypeDef * hfmapi2c, uint16_t DevAddress, uint16_t MemAddress, uint16_t MemAddSize, uint8_t * pData, uint16_t Size, uint32_t Timeout)

Function description

Read an amount of data in blocking mode from a specific memory address.

Parameters

- **hfmapi2c:** Pointer to a FMPI2C_HandleTypeDef structure that contains the configuration information for the specified FMPI2C.
- **DevAddress:** Target device address: The device 7 bits address value in datasheet must be shifted to the left before calling the interface
- **MemAddress:** Internal memory address
- **MemAddSize:** Size of internal memory address
- **pData:** Pointer to data buffer
- **Size:** Amount of data to be sent
- **Timeout:** Timeout duration

Return values

- **HAL:** status

HAL_FMPI2C_IsDeviceReady

Function name

HAL_StatusTypeDef HAL_FMPI2C_IsDeviceReady (FMPI2C_HandleTypeDef * hfmapi2c, uint16_t DevAddress, uint32_t Trials, uint32_t Timeout)

Function description

Checks if target device is ready for communication.

Parameters

- **hfmapi2c:** Pointer to a FMPI2C_HandleTypeDef structure that contains the configuration information for the specified FMPI2C.
- **DevAddress:** Target device address: The device 7 bits address value in datasheet must be shifted to the left before calling the interface
- **Trials:** Number of trials
- **Timeout:** Timeout duration

Return values

- **HAL:** status

Notes

- This function is used with Memory devices

HAL_FMPI2C_Master_Transmit_IT

Function name

HAL_StatusTypeDef HAL_FMPI2C_Master_Transmit_IT (FMPI2C_HandleTypeDef * hfmapi2c, uint16_t DevAddress, uint8_t * pData, uint16_t Size)

Function description

Transmit in master mode an amount of data in non-blocking mode with Interrupt.

Parameters

- **hfmapi2c**: Pointer to a FMPI2C_HandleTypeDef structure that contains the configuration information for the specified FMPI2C.
- **DevAddress**: Target device address: The device 7 bits address value in datasheet must be shifted to the left before calling the interface
- **pData**: Pointer to data buffer
- **Size**: Amount of data to be sent

Return values

- **HAL**: status

HAL_FMPI2C_Master_Receive_IT

Function name

HAL_StatusTypeDef HAL_FMPI2C_Master_Receive_IT (FMPI2C_HandleTypeDef * hfmapi2c, uint16_t DevAddress, uint8_t * pData, uint16_t Size)

Function description

Receive in master mode an amount of data in non-blocking mode with Interrupt.

Parameters

- **hfmapi2c**: Pointer to a FMPI2C_HandleTypeDef structure that contains the configuration information for the specified FMPI2C.
- **DevAddress**: Target device address: The device 7 bits address value in datasheet must be shifted to the left before calling the interface
- **pData**: Pointer to data buffer
- **Size**: Amount of data to be sent

Return values

- **HAL**: status

HAL_FMPI2C_Slave_Transmit_IT

Function name

HAL_StatusTypeDef HAL_FMPI2C_Slave_Transmit_IT (FMPI2C_HandleTypeDef * hfmapi2c, uint8_t * pData, uint16_t Size)

Function description

Transmit in slave mode an amount of data in non-blocking mode with Interrupt.

Parameters

- **hfmapi2c**: Pointer to a FMPI2C_HandleTypeDef structure that contains the configuration information for the specified FMPI2C.
- **pData**: Pointer to data buffer
- **Size**: Amount of data to be sent

Return values

- **HAL:** status

HAL_FMPI2C_Slave_Receive_IT

Function name

HAL_StatusTypeDef HAL_FMPI2C_Slave_Receive_IT (FMPI2C_HandleTypeDef * hfmapi2c, uint8_t * pData, uint16_t Size)

Function description

Receive in slave mode an amount of data in non-blocking mode with Interrupt.

Parameters

- **hfmapi2c:** Pointer to a FMPI2C_HandleTypeDef structure that contains the configuration information for the specified FMPI2C.
- **pData:** Pointer to data buffer
- **Size:** Amount of data to be sent

Return values

- **HAL:** status

HAL_FMPI2C_Mem_Write_IT

Function name

HAL_StatusTypeDef HAL_FMPI2C_Mem_Write_IT (FMPI2C_HandleTypeDef * hfmapi2c, uint16_t DevAddress, uint16_t MemAddress, uint16_t MemAddSize, uint8_t * pData, uint16_t Size)

Function description

Write an amount of data in non-blocking mode with Interrupt to a specific memory address.

Parameters

- **hfmapi2c:** Pointer to a FMPI2C_HandleTypeDef structure that contains the configuration information for the specified FMPI2C.
- **DevAddress:** Target device address: The device 7 bits address value in datasheet must be shifted to the left before calling the interface
- **MemAddress:** Internal memory address
- **MemAddSize:** Size of internal memory address
- **pData:** Pointer to data buffer
- **Size:** Amount of data to be sent

Return values

- **HAL:** status

HAL_FMPI2C_Mem_Read_IT

Function name

HAL_StatusTypeDef HAL_FMPI2C_Mem_Read_IT (FMPI2C_HandleTypeDef * hfmapi2c, uint16_t DevAddress, uint16_t MemAddress, uint16_t MemAddSize, uint8_t * pData, uint16_t Size)

Function description

Read an amount of data in non-blocking mode with Interrupt from a specific memory address.

Parameters

- **hfmapi2c**: Pointer to a FMPI2C_HandleTypeDef structure that contains the configuration information for the specified FMPI2C.
- **DevAddress**: Target device address: The device 7 bits address value in datasheet must be shifted to the left before calling the interface
- **MemAddress**: Internal memory address
- **MemAddSize**: Size of internal memory address
- **pData**: Pointer to data buffer
- **Size**: Amount of data to be sent

Return values

- **HAL**: status

HAL_FMPI2C_Master_Seq_Transmit_IT

Function name

HAL_StatusTypeDef HAL_FMPI2C_Master_Seq_Transmit_IT (FMPI2C_HandleTypeDef * hfmapi2c, uint16_t DevAddress, uint8_t * pData, uint16_t Size, uint32_t XferOptions)

Function description

Sequential transmit in master FMPI2C mode an amount of data in non-blocking mode with Interrupt.

Parameters

- **hfmapi2c**: Pointer to a FMPI2C_HandleTypeDef structure that contains the configuration information for the specified FMPI2C.
- **DevAddress**: Target device address: The device 7 bits address value in datasheet must be shifted to the left before calling the interface
- **pData**: Pointer to data buffer
- **Size**: Amount of data to be sent
- **XferOptions**: Options of Transfer, value of FMPI2C Sequential Transfer Options

Return values

- **HAL**: status

Notes

- This interface allow to manage repeated start condition when a direction change during transfer

HAL_FMPI2C_Master_Seq_Receive_IT

Function name

HAL_StatusTypeDef HAL_FMPI2C_Master_Seq_Receive_IT (FMPI2C_HandleTypeDef * hfmapi2c, uint16_t DevAddress, uint8_t * pData, uint16_t Size, uint32_t XferOptions)

Function description

Sequential receive in master FMPI2C mode an amount of data in non-blocking mode with Interrupt.

Parameters

- **hfmapi2c**: Pointer to a FMPI2C_HandleTypeDef structure that contains the configuration information for the specified FMPI2C.
- **DevAddress**: Target device address: The device 7 bits address value in datasheet must be shifted to the left before calling the interface
- **pData**: Pointer to data buffer
- **Size**: Amount of data to be sent
- **XferOptions**: Options of Transfer, value of FMPI2C Sequential Transfer Options

Return values

- **HAL:** status

Notes

- This interface allow to manage repeated start condition when a direction change during transfer

HAL_FMPI2C_Slave_Seq_Transmit_IT

Function name

HAL_StatusTypeDef HAL_FMPI2C_Slave_Seq_Transmit_IT (FMPI2C_HandleTypeDef * hfmapi2c, uint8_t * pData, uint16_t Size, uint32_t XferOptions)

Function description

Sequential transmit in slave/device FMPI2C mode an amount of data in non-blocking mode with Interrupt.

Parameters

- **hfmapi2c:** Pointer to a FMPI2C_HandleTypeDef structure that contains the configuration information for the specified FMPI2C.
- **pData:** Pointer to data buffer
- **Size:** Amount of data to be sent
- **XferOptions:** Options of Transfer, value of FMPI2C Sequential Transfer Options

Return values

- **HAL:** status

Notes

- This interface allow to manage repeated start condition when a direction change during transfer

HAL_FMPI2C_Slave_Seq_Receive_IT

Function name

HAL_StatusTypeDef HAL_FMPI2C_Slave_Seq_Receive_IT (FMPI2C_HandleTypeDef * hfmapi2c, uint8_t * pData, uint16_t Size, uint32_t XferOptions)

Function description

Sequential receive in slave/device FMPI2C mode an amount of data in non-blocking mode with Interrupt.

Parameters

- **hfmapi2c:** Pointer to a FMPI2C_HandleTypeDef structure that contains the configuration information for the specified FMPI2C.
- **pData:** Pointer to data buffer
- **Size:** Amount of data to be sent
- **XferOptions:** Options of Transfer, value of FMPI2C Sequential Transfer Options

Return values

- **HAL:** status

Notes

- This interface allow to manage repeated start condition when a direction change during transfer

HAL_FMPI2C_EnableListen_IT

Function name

HAL_StatusTypeDef HAL_FMPI2C_EnableListen_IT (FMPI2C_HandleTypeDef * hfmapi2c)

Function description

Enable the Address listen mode with Interrupt.

Parameters

- **hfmapi2c:** Pointer to a FMPI2C_HandleTypeDef structure that contains the configuration information for the specified FMPI2C.

Return values

- **HAL:** status

HAL_FMPI2C_DisableListen_IT

Function name

HAL_StatusTypeDef HAL_FMPI2C_DisableListen_IT (FMPI2C_HandleTypeDef * hfmapi2c)

Function description

Disable the Address listen mode with Interrupt.

Parameters

- **hfmapi2c:** Pointer to a FMPI2C_HandleTypeDef structure that contains the configuration information for the specified FMPI2C

Return values

- **HAL:** status

HAL_FMPI2C_Master_Abort_IT

Function name

HAL_StatusTypeDef HAL_FMPI2C_Master_Abort_IT (FMPI2C_HandleTypeDef * hfmapi2c, uint16_t DevAddress)

Function description

Abort a master FMPI2C IT or DMA process communication with Interrupt.

Parameters

- **hfmapi2c:** Pointer to a FMPI2C_HandleTypeDef structure that contains the configuration information for the specified FMPI2C.
- **DevAddress:** Target device address: The device 7 bits address value in datasheet must be shifted to the left before calling the interface

Return values

- **HAL:** status

HAL_FMPI2C_Master_Transmit_DMA

Function name

HAL_StatusTypeDef HAL_FMPI2C_Master_Transmit_DMA (FMPI2C_HandleTypeDef * hfmapi2c, uint16_t DevAddress, uint8_t * pData, uint16_t Size)

Function description

Transmit in master mode an amount of data in non-blocking mode with DMA.

Parameters

- **hfmapi2c:** Pointer to a FMPI2C_HandleTypeDef structure that contains the configuration information for the specified FMPI2C.
- **DevAddress:** Target device address: The device 7 bits address value in datasheet must be shifted to the left before calling the interface
- **pData:** Pointer to data buffer
- **Size:** Amount of data to be sent

Return values

- **HAL:** status

HAL_FMPI2C_Master_Receive_DMA

Function name

HAL_StatusTypeDef HAL_FMPI2C_Master_Receive_DMA (FMPI2C_HandleTypeDef * hfmapi2c, uint16_t DevAddress, uint8_t * pData, uint16_t Size)

Function description

Receive in master mode an amount of data in non-blocking mode with DMA.

Parameters

- **hfmapi2c:** Pointer to a FMPI2C_HandleTypeDef structure that contains the configuration information for the specified FMPI2C.
- **DevAddress:** Target device address: The device 7 bits address value in datasheet must be shifted to the left before calling the interface
- **pData:** Pointer to data buffer
- **Size:** Amount of data to be sent

Return values

- **HAL:** status

HAL_FMPI2C_Slave_Transmit_DMA

Function name

HAL_StatusTypeDef HAL_FMPI2C_Slave_Transmit_DMA (FMPI2C_HandleTypeDef * hfmapi2c, uint8_t * pData, uint16_t Size)

Function description

Transmit in slave mode an amount of data in non-blocking mode with DMA.

Parameters

- **hfmapi2c:** Pointer to a FMPI2C_HandleTypeDef structure that contains the configuration information for the specified FMPI2C.
- **pData:** Pointer to data buffer
- **Size:** Amount of data to be sent

Return values

- **HAL:** status

HAL_FMPI2C_Slave_Receive_DMA

Function name

HAL_StatusTypeDef HAL_FMPI2C_Slave_Receive_DMA (FMPI2C_HandleTypeDef * hfmapi2c, uint8_t * pData, uint16_t Size)

Function description

Receive in slave mode an amount of data in non-blocking mode with DMA.

Parameters

- **hfmapi2c:** Pointer to a FMPI2C_HandleTypeDef structure that contains the configuration information for the specified FMPI2C.
- **pData:** Pointer to data buffer
- **Size:** Amount of data to be sent

Return values

- **HAL:** status

HAL_FMPI2C_Mem_Write_DMA

Function name

HAL_StatusTypeDef HAL_FMPI2C_Mem_Write_DMA (FMPI2C_HandleTypeDef * hfmapi2c, uint16_t DevAddress, uint16_t MemAddress, uint16_t MemAddSize, uint8_t * pData, uint16_t Size)

Function description

Write an amount of data in non-blocking mode with DMA to a specific memory address.

Parameters

- **hfmapi2c:** Pointer to a FMPI2C_HandleTypeDef structure that contains the configuration information for the specified FMPI2C.
- **DevAddress:** Target device address: The device 7 bits address value in datasheet must be shifted to the left before calling the interface
- **MemAddress:** Internal memory address
- **MemAddSize:** Size of internal memory address
- **pData:** Pointer to data buffer
- **Size:** Amount of data to be sent

Return values

- **HAL:** status

HAL_FMPI2C_Mem_Read_DMA

Function name

HAL_StatusTypeDef HAL_FMPI2C_Mem_Read_DMA (FMPI2C_HandleTypeDef * hfmapi2c, uint16_t DevAddress, uint16_t MemAddress, uint16_t MemAddSize, uint8_t * pData, uint16_t Size)

Function description

Reads an amount of data in non-blocking mode with DMA from a specific memory address.

Parameters

- **hfmapi2c:** Pointer to a FMPI2C_HandleTypeDef structure that contains the configuration information for the specified FMPI2C.
- **DevAddress:** Target device address: The device 7 bits address value in datasheet must be shifted to the left before calling the interface
- **MemAddress:** Internal memory address
- **MemAddSize:** Size of internal memory address
- **pData:** Pointer to data buffer
- **Size:** Amount of data to be read

Return values

- **HAL:** status

HAL_FMPI2C_Master_Seq_Transmit_DMA

Function name

HAL_StatusTypeDef HAL_FMPI2C_Master_Seq_Transmit_DMA (FMPI2C_HandleTypeDef * hfmapi2c, uint16_t DevAddress, uint8_t * pData, uint16_t Size, uint32_t XferOptions)

Function description

Sequential transmit in master FMPI2C mode an amount of data in non-blocking mode with DMA.

Parameters

- **hfmapi2c:** Pointer to a FMPI2C_HandleTypeDef structure that contains the configuration information for the specified FMPI2C.
- **DevAddress:** Target device address: The device 7 bits address value in datasheet must be shifted to the left before calling the interface
- **pData:** Pointer to data buffer
- **Size:** Amount of data to be sent
- **XferOptions:** Options of Transfer, value of FMPI2C Sequential Transfer Options

Return values

- **HAL:** status

Notes

- This interface allow to manage repeated start condition when a direction change during transfer

HAL_FMPI2C_Master_Seq_Receive_DMA

Function name

HAL_StatusTypeDef HAL_FMPI2C_Master_Seq_Receive_DMA (FMPI2C_HandleTypeDef * hfmapi2c, uint16_t DevAddress, uint8_t * pData, uint16_t Size, uint32_t XferOptions)

Function description

Sequential receive in master FMPI2C mode an amount of data in non-blocking mode with DMA.

Parameters

- **hfmapi2c:** Pointer to a FMPI2C_HandleTypeDef structure that contains the configuration information for the specified FMPI2C.
- **DevAddress:** Target device address: The device 7 bits address value in datasheet must be shifted to the left before calling the interface
- **pData:** Pointer to data buffer
- **Size:** Amount of data to be sent
- **XferOptions:** Options of Transfer, value of FMPI2C Sequential Transfer Options

Return values

- **HAL:** status

Notes

- This interface allow to manage repeated start condition when a direction change during transfer

HAL_FMPI2C_Slave_Seq_Transmit_DMA

Function name

HAL_StatusTypeDef HAL_FMPI2C_Slave_Seq_Transmit_DMA (FMPI2C_HandleTypeDef * hfmapi2c, uint8_t * pData, uint16_t Size, uint32_t XferOptions)

Function description

Sequential transmit in slave/device FMPI2C mode an amount of data in non-blocking mode with DMA.

Parameters

- **hfmapi2c:** Pointer to a FMPI2C_HandleTypeDef structure that contains the configuration information for the specified FMPI2C.
- **pData:** Pointer to data buffer
- **Size:** Amount of data to be sent
- **XferOptions:** Options of Transfer, value of FMPI2C Sequential Transfer Options

Return values

- **HAL:** status

Notes

- This interface allow to manage repeated start condition when a direction change during transfer

HAL_FMPI2C_Slave_Seq_Receive_DMA

Function name

HAL_StatusTypeDef HAL_FMPI2C_Slave_Seq_Receive_DMA (FMPI2C_HandleTypeDef * hfmapi2c, uint8_t * pData, uint16_t Size, uint32_t XferOptions)

Function description

Sequential receive in slave/device FMPI2C mode an amount of data in non-blocking mode with DMA.

Parameters

- **hfmapi2c:** Pointer to a FMPI2C_HandleTypeDef structure that contains the configuration information for the specified FMPI2C.
- **pData:** Pointer to data buffer
- **Size:** Amount of data to be sent
- **XferOptions:** Options of Transfer, value of FMPI2C Sequential Transfer Options

Return values

- **HAL:** status

Notes

- This interface allow to manage repeated start condition when a direction change during transfer

HAL_FMPI2C_EV_IRQHandler

Function name

void HAL_FMPI2C_EV_IRQHandler (FMPI2C_HandleTypeDef * hfmapi2c)

Function description

This function handles FMPI2C event interrupt request.

Parameters

- **hfmapi2c:** Pointer to a FMPI2C_HandleTypeDef structure that contains the configuration information for the specified FMPI2C.

Return values

- **None:**

HAL_FMPI2C_ER_IRQHandler

Function name

void HAL_FMPI2C_ER_IRQHandler (FMPI2C_HandleTypeDef * hfmapi2c)

Function description

This function handles FMPI2C error interrupt request.

Parameters

- **hfmapi2c:** Pointer to a FMPI2C_HandleTypeDef structure that contains the configuration information for the specified FMPI2C.

Return values

- **None:**

HAL_FMPI2C_MasterTxCpltCallback

Function name

```
void HAL_FMPI2C_MasterTxCpltCallback (FMPI2C_HandleTypeDef * hfmpi2c)
```

Function description

Master Tx Transfer completed callback.

Parameters

- **hfmpi2c:** Pointer to a FMPI2C_HandleTypeDef structure that contains the configuration information for the specified FMPI2C.

Return values

- **None:**

HAL_FMPI2C_MasterRxCpltCallback

Function name

```
void HAL_FMPI2C_MasterRxCpltCallback (FMPI2C_HandleTypeDef * hfmpi2c)
```

Function description

Master Rx Transfer completed callback.

Parameters

- **hfmpi2c:** Pointer to a FMPI2C_HandleTypeDef structure that contains the configuration information for the specified FMPI2C.

Return values

- **None:**

HAL_FMPI2C_SlaveTxCpltCallback

Function name

```
void HAL_FMPI2C_SlaveTxCpltCallback (FMPI2C_HandleTypeDef * hfmpi2c)
```

Function description

Slave Tx Transfer completed callback.

Parameters

- **hfmpi2c:** Pointer to a FMPI2C_HandleTypeDef structure that contains the configuration information for the specified FMPI2C.

Return values

- **None:**

HAL_FMPI2C_SlaveRxCpltCallback

Function name

```
void HAL_FMPI2C_SlaveRxCpltCallback (FMPI2C_HandleTypeDef * hfmpi2c)
```

Function description

Slave Rx Transfer completed callback.

Parameters

- **hfmpi2c:** Pointer to a FMPI2C_HandleTypeDef structure that contains the configuration information for the specified FMPI2C.

Return values

- **None:**

HAL_FMPI2C_AddrCallback

Function name

void HAL_FMPI2C_AddrCallback (FMPI2C_HandleTypeDef * hfmapi2c, uint8_t TransferDirection, uint16_t AddrMatchCode)

Function description

Slave Address Match callback.

Parameters

- **hfmapi2c:** Pointer to a FMPI2C_HandleTypeDef structure that contains the configuration information for the specified FMPI2C.
- **TransferDirection:** Master request Transfer Direction (Write/Read), value of FMPI2C Transfer Direction Master Point of View
- **AddrMatchCode:** Address Match Code

Return values

- **None:**

HAL_FMPI2C_ListenCpltCallback

Function name

void HAL_FMPI2C_ListenCpltCallback (FMPI2C_HandleTypeDef * hfmapi2c)

Function description

Listen Complete callback.

Parameters

- **hfmapi2c:** Pointer to a FMPI2C_HandleTypeDef structure that contains the configuration information for the specified FMPI2C.

Return values

- **None:**

HAL_FMPI2C_MemTxCpltCallback

Function name

void HAL_FMPI2C_MemTxCpltCallback (FMPI2C_HandleTypeDef * hfmapi2c)

Function description

Memory Tx Transfer completed callback.

Parameters

- **hfmapi2c:** Pointer to a FMPI2C_HandleTypeDef structure that contains the configuration information for the specified FMPI2C.

Return values

- **None:**

HAL_FMPI2C_MemRxCpltCallback

Function name

void HAL_FMPI2C_MemRxCpltCallback (FMPI2C_HandleTypeDef * hfmapi2c)

Function description

Memory Rx Transfer completed callback.

Parameters

- **hfmapi2c:** Pointer to a FMPI2C_HandleTypeDef structure that contains the configuration information for the specified FMPI2C.

Return values

- **None:**

HAL_FMPI2C_ErrorCallback

Function name

void HAL_FMPI2C_ErrorCallback (FMPI2C_HandleTypeDef * hfmapi2c)

Function description

FMPI2C error callback.

Parameters

- **hfmapi2c:** Pointer to a FMPI2C_HandleTypeDef structure that contains the configuration information for the specified FMPI2C.

Return values

- **None:**

HAL_FMPI2C_AbortCpltCallback

Function name

void HAL_FMPI2C_AbortCpltCallback (FMPI2C_HandleTypeDef * hfmapi2c)

Function description

FMPI2C abort callback.

Parameters

- **hfmapi2c:** Pointer to a FMPI2C_HandleTypeDef structure that contains the configuration information for the specified FMPI2C.

Return values

- **None:**

HAL_FMPI2C_GetState

Function name

HAL_FMPI2C_StateTypeDef HAL_FMPI2C_GetState (const FMPI2C_HandleTypeDef * hfmapi2c)

Function description

Return the FMPI2C handle state.

Parameters

- **hfmapi2c:** Pointer to a FMPI2C_HandleTypeDef structure that contains the configuration information for the specified FMPI2C.

Return values

- **HAL:** state

HAL_FMPI2C_GetMode

Function name

HAL_FMPI2C_ModeTypeDef HAL_FMPI2C_GetMode (const FMPI2C_HandleTypeDef * hfmapi2c)

Function description

Returns the FMPI2C Master, Slave, Memory or no mode.

Parameters

- **hfmapi2c:** Pointer to a FMPI2C_HandleTypeDef structure that contains the configuration information for FMPI2C module

Return values

- **HAL:** mode

HAL_FMPI2C_GetError

Function name

uint32_t HAL_FMPI2C_GetError (const FMPI2C_HandleTypeDef * hfmapi2c)

Function description

Return the FMPI2C error code.

Parameters

- **hfmapi2c:** Pointer to a FMPI2C_HandleTypeDef structure that contains the configuration information for the specified FMPI2C.

Return values

- **FMPI2C:** Error Code

29.3 FMPI2C Firmware driver defines

The following section lists the various define and macros of the module.

29.3.1 FMPI2C

FMPI2C

FMPI2C Addressing Mode

FMPI2C_ADDRESSINGMODE_7BIT

FMPI2C_ADDRESSINGMODE_10BIT

FMPI2C Dual Addressing Mode

FMPI2C_DUALADDRESS_DISABLE

FMPI2C_DUALADDRESS_ENABLE

FMPI2C Error Code definition

HAL_FMPI2C_ERROR_NONE

No error

HAL_FMPI2C_ERROR_BERR

BERR error

HAL_FMPI2C_ERROR_ARLO

ARLO error

HAL_FMPI2C_ERROR_AF

ACKF error

HAL_FMPI2C_ERROR_OVR

OVR error

HAL_FMPI2C_ERROR_DMA

DMA transfer error

HAL_FMPI2C_ERROR_TIMEOUT

Timeout error

HAL_FMPI2C_ERROR_SIZE

Size Management error

HAL_FMPI2C_ERROR_DMA_PARAM

DMA Parameter Error

HAL_FMPI2C_ERROR_INVALID_CALLBACK

Invalid Callback error

HAL_FMPI2C_ERROR_INVALID_PARAM

Invalid Parameters error

FMPI2C Exported Macros

__HAL_FMPI2C_RESET_HANDLE_STATE

Description:

- Reset FMPI2C handle state.

Parameters:

- `__HANDLE__`: specifies the FMPI2C Handle.

Return value:

- None

__HAL_FMPI2C_ENABLE_IT

Description:

- Enable the specified FMPI2C interrupt.

Parameters:

- `__HANDLE__`: specifies the FMPI2C Handle.
- `__INTERRUPT__`: specifies the interrupt source to enable. This parameter can be one of the following values:
 - FMPI2C_IT_ERRI Errors interrupt enable
 - FMPI2C_IT_TCI Transfer complete interrupt enable
 - FMPI2C_IT_STOPI STOP detection interrupt enable
 - FMPI2C_IT_NACKI NACK received interrupt enable
 - FMPI2C_IT_ADDRI Address match interrupt enable
 - FMPI2C_IT_RXI RX interrupt enable
 - FMPI2C_IT_TXI TX interrupt enable

Return value:

- None

__HAL_FMPI2C_DISABLE_IT

Description:

- Disable the specified FMPI2C interrupt.

Parameters:

- `__HANDLE__`: specifies the FMPI2C Handle.
- `__INTERRUPT__`: specifies the interrupt source to disable. This parameter can be one of the following values:
 - FMPI2C_IT_ERRI Errors interrupt enable
 - FMPI2C_IT_TCI Transfer complete interrupt enable
 - FMPI2C_IT_STOPI STOP detection interrupt enable
 - FMPI2C_IT_NACKI NACK received interrupt enable
 - FMPI2C_IT_ADDRI Address match interrupt enable
 - FMPI2C_IT_RXI RX interrupt enable
 - FMPI2C_IT_TXI TX interrupt enable

Return value:

- None

__HAL_FMPI2C_GET_IT_SOURCE

Description:

- Check whether the specified FMPI2C interrupt source is enabled or not.

Parameters:

- `__HANDLE__`: specifies the FMPI2C Handle.
- `__INTERRUPT__`: specifies the FMPI2C interrupt source to check. This parameter can be one of the following values:
 - FMPI2C_IT_ERRI Errors interrupt enable
 - FMPI2C_IT_TCI Transfer complete interrupt enable
 - FMPI2C_IT_STOPI STOP detection interrupt enable
 - FMPI2C_IT_NACKI NACK received interrupt enable
 - FMPI2C_IT_ADDRI Address match interrupt enable
 - FMPI2C_IT_RXI RX interrupt enable
 - FMPI2C_IT_TXI TX interrupt enable

Return value:

- The: new state of `__INTERRUPT__` (SET or RESET).

FMPI2C_FLAG_MASK

Description:

- Check whether the specified FMPI2C flag is set or not.

Parameters:

- `__HANDLE__`: specifies the FMPI2C Handle.
- `__FLAG__`: specifies the flag to check. This parameter can be one of the following values:
 - FMPI2C_FLAG_TXE Transmit data register empty
 - FMPI2C_FLAG_TXIS Transmit interrupt status
 - FMPI2C_FLAG_RXNE Receive data register not empty
 - FMPI2C_FLAG_ADDR Address matched (slave mode)
 - FMPI2C_FLAG_AF Acknowledge failure received flag
 - FMPI2C_FLAG_STOPF STOP detection flag
 - FMPI2C_FLAG_TC Transfer complete (master mode)
 - FMPI2C_FLAG_TCR Transfer complete reload
 - FMPI2C_FLAG_BERR Bus error
 - FMPI2C_FLAG_ARLO Arbitration lost
 - FMPI2C_FLAG_OVR Overrun/Underrun
 - FMPI2C_FLAG_PECERR PEC error in reception
 - FMPI2C_FLAG_TIMEOUT Timeout or Tlow detection flag
 - FMPI2C_FLAG_ALERT SMBus alert
 - FMPI2C_FLAG_BUSY Bus busy
 - FMPI2C_FLAG_DIR Transfer direction (slave mode)

Return value:

- The: new state of `__FLAG__` (SET or RESET).

__HAL_FMPI2C_GET_FLAG

__HAL_FMPI2C_CLEAR_FLAG

Description:

- Clear the FMPI2C pending flags which are cleared by writing 1 in a specific bit.

Parameters:

- `__HANDLE__`: specifies the FMPI2C Handle.
- `__FLAG__`: specifies the flag to clear. This parameter can be any combination of the following values:
 - FMPI2C_FLAG_TXE Transmit data register empty
 - FMPI2C_FLAG_ADDR Address matched (slave mode)
 - FMPI2C_FLAG_AF Acknowledge failure received flag
 - FMPI2C_FLAG_STOPF STOP detection flag
 - FMPI2C_FLAG_BERR Bus error
 - FMPI2C_FLAG_ARLO Arbitration lost
 - FMPI2C_FLAG_OVR Overrun/Underrun
 - FMPI2C_FLAG_PECERR PEC error in reception
 - FMPI2C_FLAG_TIMEOUT Timeout or Tlow detection flag
 - FMPI2C_FLAG_ALERT SMBus alert

Return value:

- None

__HAL_FMPI2C_ENABLE

Description:

- Enable the specified FMPI2C peripheral.

Parameters:

- `__HANDLE__`: specifies the FMPI2C Handle.

Return value:

- None

__HAL_FMPI2C_DISABLE

Description:

- Disable the specified FMPI2C peripheral.

Parameters:

- `__HANDLE__`: specifies the FMPI2C Handle.

Return value:

- None

__HAL_FMPI2C_GENERATE_NACK

Description:

- Generate a Non-Acknowledge FMPI2C peripheral in Slave mode.

Parameters:

- `__HANDLE__`: specifies the FMPI2C Handle.

Return value:

- None

FMPI2C Flag definition

FMPI2C_FLAG_TXE

FMPI2C_FLAG_TXIS

FMPI2C_FLAG_RXNE

FMPI2C_FLAG_ADDR

FMPI2C_FLAG_AF

FMPI2C_FLAG_STOPF

FMPI2C_FLAG_TC

FMPI2C_FLAG_TCR

FMPI2C_FLAG_BERR

FMPI2C_FLAG_ARLO

FMPI2C_FLAG_OVR

FMPI2C_FLAG_PECERR

FMPI2C_FLAG_TIMEOUT

FMPI2C_FLAG_ALERT

FMPI2C_FLAG_BUSY

FMPI2C_FLAG_DIR

FMPI2C General Call Addressing Mode

FMPI2C_GENERALCALL_DISABLE

FMPI2C_GENERALCALL_ENABLE

FMPI2C Interrupt configuration definition

FMPI2C_IT_ERRI

FMPI2C_IT_TCI

FMPI2C_IT_STOPI

FMPI2C_IT_NACKI

FMPI2C_IT_ADDRI

FMPI2C_IT_RXI

FMPI2C_IT_TXI

FMPI2C Memory Address Size

FMPI2C_MEMADD_SIZE_8BIT

FMPI2C_MEMADD_SIZE_16BIT

FMPI2C No-Stretch Mode

FMPI2C_NOSTRETCH_DISABLE

FMPI2C_NOSTRETCH_ENABLE

FMPI2C Own Address2 Masks

FMPI2C_OA2_NOMASK

FMPI2C_OA2_MASK01

FMPI2C_OA2_MASK02

FMPI2C_OA2_MASK03

FMPI2C_OA2_MASK04

FMPI2C_OA2_MASK05

FMPI2C_OA2_MASK06

FMPI2C_OA2_MASK07

FMPI2C Reload End Mode

FMPI2C_RELOAD_MODE

FMPI2C_AUTOEND_MODE

FMPI2C_SOFTEND_MODE

FMPI2C Start or Stop Mode

FMPI2C_NO_STARTSTOP

FMPI2C_GENERATE_STOP

FMPI2C_GENERATE_START_READ

FMPI2C_GENERATE_START_WRITE

FMPI2C Transfer Direction Master Point of View

FMPI2C_DIRECTION_TRANSMIT

FMPI2C_DIRECTION_RECEIVE

FMPI2C Sequential Transfer Options

FMPI2C_FIRST_FRAME

FMPI2C_FIRST_AND_NEXT_FRAME

FMPI2C_NEXT_FRAME

FMPI2C_FIRST_AND_LAST_FRAME

FMPI2C_LAST_FRAME

FMPI2C_LAST_FRAME_NO_STOP

FMPI2C_OTHER_FRAME

FMPI2C_OTHER_AND_LAST_FRAME

30 HAL FMPI2C Extension Driver

30.1 FMPI2CEx Firmware driver API description

The following section lists the various functions of the FMPI2CEx library.

30.1.1 FMPI2C peripheral Extended features

Comparing to other previous devices, the FMPI2C interface for STM32F4xx devices contains the following additional features

- Possibility to disable or enable Analog Noise Filter
- Use of a configured Digital Noise Filter
- Disable or enable Fast Mode Plus

30.1.2 How to use this driver

This driver provides functions to:

1. Configure FMPI2C Analog noise filter using the function `HAL_FMPI2CEx_ConfigAnalogFilter()`
2. Configure FMPI2C Digital noise filter using the function `HAL_FMPI2CEx_ConfigDigitalFilter()`
3. Configure the enable or disable of fast mode plus driving capability using the functions :
 - `HAL_FMPI2CEx_EnableFastModePlus()`
 - `HAL_FMPI2CEx_DisableFastModePlus()`

30.1.3 Filter Mode Functions

This section provides functions allowing to:

- Configure Noise Filters

This section contains the following APIs:

- [`HAL\_FMPI2CEx\_ConfigAnalogFilter\(\)`](#)
- [`HAL\_FMPI2CEx\_ConfigDigitalFilter\(\)`](#)

30.1.4 Fast Mode Plus Functions

This section provides functions allowing to:

- Configure Fast Mode Plus

This section contains the following APIs:

- [`HAL\_FMPI2CEx\_EnableFastModePlus\(\)`](#)
- [`HAL\_FMPI2CEx\_DisableFastModePlus\(\)`](#)

30.1.5 Detailed description of functions

HAL_FMPI2CEx_ConfigAnalogFilter

Function name

HAL_StatusTypeDef HAL_FMPI2CEx_ConfigAnalogFilter (FMPI2C_HandleTypeDef * hfmapi2c, uint32_t AnalogFilter)

Function description

Configure FMPI2C Analog noise filter.

Parameters

- **hfmapi2c:** Pointer to a FMPI2C_HandleTypeDef structure that contains the configuration information for the specified FMPI2Cx peripheral.
- **AnalogFilter:** New state of the Analog filter.

Return values

- **HAL:** status

HAL_FMPI2CEx_ConfigDigitalFilter

Function name

HAL_StatusTypeDef HAL_FMPI2CEx_ConfigDigitalFilter (FMPI2C_HandleTypeDef * hfmapi2c, uint32_t DigitalFilter)

Function description

Configure FMPI2C Digital noise filter.

Parameters

- **hfmapi2c:** Pointer to a FMPI2C_HandleTypeDef structure that contains the configuration information for the specified FMPI2Cx peripheral.
- **DigitalFilter:** Coefficient of digital noise filter between Min_Data=0x00 and Max_Data=0x0F.

Return values

- **HAL:** status

HAL_FMPI2CEx_EnableFastModePlus

Function name

void HAL_FMPI2CEx_EnableFastModePlus (uint32_t ConfigFastModePlus)

Function description

Enable the FMPI2C fast mode plus driving capability.

Parameters

- **ConfigFastModePlus:** Selects the pin. This parameter can be one of the FMPI2C Extended Fast Mode Plus values

Return values

- **None:**

Notes

- For FMPI2C1, fast mode plus driving capability can be enabled on all selected FMPI2C1 pins using FMPI2C_FASTMODEPLUS_FMPI2C1 parameter or independently on each one of the following pins PB6, PB7, PB8 and PB9.
- For remaining FMPI2C1 pins (PA14, PA15...) fast mode plus driving capability can be enabled only by using FMPI2C_FASTMODEPLUS_FMPI2C1 parameter.

HAL_FMPI2CEx_DisableFastModePlus

Function name

void HAL_FMPI2CEx_DisableFastModePlus (uint32_t ConfigFastModePlus)

Function description

Disable the FMPI2C fast mode plus driving capability.

Parameters

- **ConfigFastModePlus:** Selects the pin. This parameter can be one of the FMPI2C Extended Fast Mode Plus values

Return values

- **None:**

Notes

- For FMPI2C1, fast mode plus driving capability can be disabled on all selected FMPI2C1 pins using FMPI2C_FASTMODEPLUS_FMPI2C1 parameter or independently on each one of the following pins PB6, PB7, PB8 and PB9.
- For remaining FMPI2C1 pins (PA14, PA15...) fast mode plus driving capability can be disabled only by using FMPI2C_FASTMODEPLUS_FMPI2C1 parameter.

30.2 FMPI2CEx Firmware driver defines

The following section lists the various define and macros of the module.

30.2.1 FMPI2CEx

FMPI2CEx

FMPI2C Extended Analog Filter

FMPI2C_ANALOGFILTER_ENABLE

FMPI2C_ANALOGFILTER_DISABLE

FMPI2C Extended Fast Mode Plus

FMPI2C_FASTMODEPLUS_SCL

Enable Fast Mode Plus on FMPI2C1 SCL pins

FMPI2C_FASTMODEPLUS_SDA

Enable Fast Mode Plus on FMPI2C1 SDA pins

31 HAL GPIO Generic Driver

31.1 GPIO Firmware driver registers structures

31.1.1 GPIO_InitTypeDef

GPIO_InitTypeDef is defined in the `stm32f4xx_hal_gpio.h`

Data Fields

- `uint32_t Pin`
- `uint32_t Mode`
- `uint32_t Pull`
- `uint32_t Speed`
- `uint32_t Alternate`

Field Documentation

- `uint32_t GPIO_InitTypeDef::Pin`
Specifies the GPIO pins to be configured. This parameter can be any value of [GPIO_pins_define](#)
- `uint32_t GPIO_InitTypeDef::Mode`
Specifies the operating mode for the selected pins. This parameter can be a value of [GPIO_mode_define](#)
- `uint32_t GPIO_InitTypeDef::Pull`
Specifies the Pull-up or Pull-Down activation for the selected pins. This parameter can be a value of [GPIO_pull_define](#)
- `uint32_t GPIO_InitTypeDef::Speed`
Specifies the speed for the selected pins. This parameter can be a value of [GPIO_speed_define](#)
- `uint32_t GPIO_InitTypeDef::Alternate`
Peripheral to be connected to the selected pins. This parameter can be a value of [GPIO_Alternate_function_selection](#)

31.2 GPIO Firmware driver API description

The following section lists the various functions of the GPIO library.

31.2.1 GPIO Peripheral features

Subject to the specific hardware characteristics of each I/O port listed in the datasheet, each port bit of the General Purpose IO (GPIO) Ports, can be individually configured by software in several modes:

- Input mode
- Analog mode
- Output mode
- Alternate function mode
- External interrupt/event lines

During and just after reset, the alternate functions and external interrupt lines are not active and the I/O ports are configured in input floating mode.

All GPIO pins have weak internal pull-up and pull-down resistors, which can be activated or not.

In Output or Alternate mode, each IO can be configured on open-drain or push-pull type and the IO speed can be selected depending on the VDD value.

All ports have external interrupt/event capability. To use external interrupt lines, the port must be configured in input mode. All available GPIO pins are connected to the 16 external interrupt/event lines from EXTI0 to EXTI15.

The external interrupt/event controller consists of up to 23 edge detectors (16 lines are connected to GPIO) for generating event/interrupt requests (each input line can be independently configured to select the type (interrupt or event) and the corresponding trigger event (rising or falling or both). Each line can also be masked independently.

31.2.2 How to use this driver

1. Enable the GPIO AHB clock using the following function: `__HAL_RCC_GPIOx_CLK_ENABLE()`.
2. Configure the GPIO pin(s) using `HAL_GPIO_Init()`.
 - Configure the IO mode using "Mode" member from `GPIO_InitTypeDef` structure
 - Activate Pull-up, Pull-down resistor using "Pull" member from `GPIO_InitTypeDef` structure.
 - In case of Output or alternate function mode selection: the speed is configured through "Speed" member from `GPIO_InitTypeDef` structure.
 - In alternate mode is selection, the alternate function connected to the IO is configured through "Alternate" member from `GPIO_InitTypeDef` structure.
 - Analog mode is required when a pin is to be used as ADC channel or DAC output.
 - In case of external interrupt/event selection the "Mode" member from `GPIO_InitTypeDef` structure select the type (interrupt or event) and the corresponding trigger event (rising or falling or both).
3. In case of external interrupt/event mode selection, configure NVIC IRQ priority mapped to the EXTI line using `HAL_NVIC_SetPriority()` and enable it using `HAL_NVIC_EnableIRQ()`.
4. To get the level of a pin configured in input mode use `HAL_GPIO_ReadPin()`.
5. To set/reset the level of a pin configured in output mode use `HAL_GPIO_WritePin()`/
`HAL_GPIO_TogglePin()`.
6. To lock pin configuration until next reset use `HAL_GPIO_LockPin()`.
7. During and just after reset, the alternate functions are not active and the GPIO pins are configured in input floating mode (except JTAG pins).
8. The LSE oscillator pins `OSC32_IN` and `OSC32_OUT` can be used as general purpose (PC14 and PC15, respectively) when the LSE oscillator is off. The LSE has priority over the GPIO function.
9. The HSE oscillator pins `OSC_IN/OSC_OUT` can be used as general purpose PH0 and PH1, respectively, when the HSE oscillator is off. The HSE has priority over the GPIO function.

31.2.3 Initialization and de-initialization functions

This section provides functions allowing to initialize and de-initialize the GPIOs to be ready for use.

This section contains the following APIs:

- [*HAL_GPIO_Init\(\)*](#)
- [*HAL_GPIO_DeInit\(\)*](#)

31.2.4 IO operation functions

This section contains the following APIs:

- [*HAL_GPIO_ReadPin\(\)*](#)
- [*HAL_GPIO_WritePin\(\)*](#)
- [*HAL_GPIO_TogglePin\(\)*](#)
- [*HAL_GPIO_LockPin\(\)*](#)
- [*HAL_GPIO_EXTI_IRQHandler\(\)*](#)
- [*HAL_GPIO_EXTI_Callback\(\)*](#)

31.2.5 Detailed description of functions

HAL_GPIO_Init

Function name

void HAL_GPIO_Init (GPIO_TypeDef * GPIOx, GPIO_InitTypeDef * GPIO_Init)

Function description

Initializes the GPIOx peripheral according to the specified parameters in the `GPIO_Init`.

Parameters

- **GPIOx:** where x can be (A..K) to select the GPIO peripheral for STM32F429X device or x can be (A..I) to select the GPIO peripheral for STM32F40XX and STM32F427X devices.
- **GPIO_Init:** pointer to a GPIO_InitTypeDef structure that contains the configuration information for the specified GPIO peripheral.

Return values

- **None:**

HAL_GPIO_DeInit

Function name

```
void HAL_GPIO_DeInit (GPIO_TypeDef * GPIOx, uint32_t GPIO_Pin)
```

Function description

De-initializes the GPIOx peripheral registers to their default reset values.

Parameters

- **GPIOx:** where x can be (A..K) to select the GPIO peripheral for STM32F429X device or x can be (A..I) to select the GPIO peripheral for STM32F40XX and STM32F427X devices.
- **GPIO_Pin:** specifies the port bit to be written. This parameter can be one of GPIO_PIN_x where x can be (0..15).

Return values

- **None:**

HAL_GPIO_ReadPin

Function name

```
GPIO_PinState HAL_GPIO_ReadPin (GPIO_TypeDef * GPIOx, uint16_t GPIO_Pin)
```

Function description

Reads the specified input port pin.

Parameters

- **GPIOx:** where x can be (A..K) to select the GPIO peripheral for STM32F429X device or x can be (A..I) to select the GPIO peripheral for STM32F40XX and STM32F427X devices.
- **GPIO_Pin:** specifies the port bit to read. This parameter can be GPIO_PIN_x where x can be (0..15).

Return values

- **The:** input port pin value.

HAL_GPIO_WritePin

Function name

```
void HAL_GPIO_WritePin (GPIO_TypeDef * GPIOx, uint16_t GPIO_Pin, GPIO_PinState PinState)
```

Function description

Sets or clears the selected data port bit.

Parameters

- **GPIOx:** where x can be (A..K) to select the GPIO peripheral for STM32F429X device or x can be (A..I) to select the GPIO peripheral for STM32F40XX and STM32F427X devices.
- **GPIO_Pin:** specifies the port bit to be written. This parameter can be one of GPIO_PIN_x where x can be (0..15).
- **PinState:** specifies the value to be written to the selected bit. This parameter can be one of the GPIO_PinState enum values:
 - GPIO_PIN_RESET: to clear the port pin
 - GPIO_PIN_SET: to set the port pin

Return values

- **None:**

Notes

- This function uses GPIOx_BSRR register to allow atomic read/modify accesses. In this way, there is no risk of an IRQ occurring between the read and the modify access.

HAL_GPIO_TogglePin

Function name

```
void HAL_GPIO_TogglePin (GPIO_TypeDef * GPIOx, uint16_t GPIO_Pin)
```

Function description

Toggles the specified GPIO pins.

Parameters

- **GPIOx:** Where x can be (A..K) to select the GPIO peripheral for STM32F429X device or x can be (A..I) to select the GPIO peripheral for STM32F40XX and STM32F427X devices.
- **GPIO_Pin:** Specifies the pins to be toggled.

Return values

- **None:**

HAL_GPIO_LockPin

Function name

```
HAL_StatusTypeDef HAL_GPIO_LockPin (GPIO_TypeDef * GPIOx, uint16_t GPIO_Pin)
```

Function description

Locks GPIO Pins configuration registers.

Parameters

- **GPIOx:** where x can be (A..F) to select the GPIO peripheral for STM32F4 family
- **GPIO_Pin:** specifies the port bit to be locked. This parameter can be any combination of GPIO_PIN_x where x can be (0..15).

Return values

- **None:**

Notes

- The locked registers are GPIOx_MODER, GPIOx_OTYPER, GPIOx_OSPEEDR, GPIOx_PUPDR, GPIOx_AFR1 and GPIOx_AFR2.
- The configuration of the locked GPIO pins can no longer be modified until the next reset.

HAL_GPIO_EXTI_IRQHandler

Function name

void HAL_GPIO_EXTI_IRQHandler (uint16_t GPIO_Pin)

Function description

This function handles EXTI interrupt request.

Parameters

- **GPIO_Pin:** Specifies the pins connected EXTI line

Return values

- **None:**

HAL_GPIO_EXTI_Callback

Function name

void HAL_GPIO_EXTI_Callback (uint16_t GPIO_Pin)

Function description

EXTI line detection callbacks.

Parameters

- **GPIO_Pin:** Specifies the pins connected EXTI line

Return values

- **None:**

31.3 GPIO Firmware driver defines

The following section lists the various define and macros of the module.

31.3.1 GPIO

GPIO

GPIO Alternate Function Selection

GPIO_AF0_RTC_50Hz

GPIO_AF0_MCO

GPIO_AF0_TAMPER

GPIO_AF0_SWJ

GPIO_AF0_TRACE

GPIO_AF1_TIM1

GPIO_AF1_TIM2

GPIO_AF2_TIM3

GPIO_AF2_TIM4

GPIO_AF2_TIM5

GPIO_AF3_TIM8

GPIO_AF3_TIM9

GPIO_AF3_TIM10

GPIO_AF3_TIM11

GPIO_AF4_I2C1

GPIO_AF4_I2C2

GPIO_AF4_I2C3

GPIO_AF5_SPI1

GPIO_AF5_SPI2

GPIO_AF5_SPI3

GPIO_AF5_SPI4

GPIO_AF5_SPI5

GPIO_AF5_SPI6

GPIO_AF5_I2S3ext

GPIO_AF6_SPI3

GPIO_AF6_I2S2ext

GPIO_AF6_SAI1

GPIO_AF7_USART1

GPIO_AF7_USART2

GPIO_AF7_USART3

GPIO_AF7_I2S3ext

GPIO_AF8_UART4

GPIO_AF8_UART5

GPIO_AF8_USART6

GPIO_AF8_UART7

GPIO_AF8_UART8

GPIO_AF9_CAN1

GPIO_AF9_CAN2

GPIO_AF9_TIM12

GPIO_AF9_TIM13

GPIO_AF9_TIM14

GPIO_AF9_LTDC

GPIO_AF9_QSPI

GPIO_AF10_OTG_FS

GPIO_AF10_OTG_HS

GPIO_AF10_QSPI

GPIO_AF11_ETH

GPIO_AF12_FMC

GPIO_AF12_OTG_HS_FS

GPIO_AF12_SDIO

GPIO_AF13_DCMI

GPIO_AF13_DSI

GPIO_AF14_LTDC

GPIO_AF15_EVENTOUT

GPIO Exported Macros

__HAL_GPIO_EXTI_GET_FLAG

Description:

- Checks whether the specified EXTI line flag is set or not.

Parameters:

- `__EXTI_LINE__`: specifies the EXTI line flag to check. This parameter can be GPIO_PIN_x where x can be (0..15)

Return value:

- The: new state of `__EXTI_LINE__` (SET or RESET).

__HAL_GPIO_EXTI_CLEAR_FLAG

Description:

- Clears the EXTI's line pending flags.

Parameters:

- `__EXTI_LINE__`: specifies the EXTI lines flags to clear. This parameter can be any combination of GPIO_PIN_x where x can be (0..15)

Return value:

- None

__HAL_GPIO_EXTI_GET_IT

Description:

- Checks whether the specified EXTI line is asserted or not.

Parameters:

- `__EXTI_LINE__`: specifies the EXTI line to check. This parameter can be `GPIO_PIN_x` where `x` can be (0..15)

Return value:

- The: new state of `__EXTI_LINE__` (SET or RESET).

__HAL_GPIO_EXTI_CLEAR_IT

Description:

- Clears the EXTI's line pending bits.

Parameters:

- `__EXTI_LINE__`: specifies the EXTI lines to clear. This parameter can be any combination of `GPIO_PIN_x` where `x` can be (0..15)

Return value:

- None

__HAL_GPIO_EXTI_GENERATE_SWIT

Description:

- Generates a Software interrupt on selected EXTI line.

Parameters:

- `__EXTI_LINE__`: specifies the EXTI line to check. This parameter can be `GPIO_PIN_x` where `x` can be (0..15)

Return value:

- None

GPIO mode define

GPIO_MODE_INPUT

Input Floating Mode

GPIO_MODE_OUTPUT_PP

Output Push Pull Mode

GPIO_MODE_OUTPUT_OD

Output Open Drain Mode

GPIO_MODE_AF_PP

Alternate Function Push Pull Mode

GPIO_MODE_AF_OD

Alternate Function Open Drain Mode

GPIO_MODE_ANALOG

Analog Mode

GPIO_MODE_IT_RISING

External Interrupt Mode with Rising edge trigger detection

GPIO_MODE_IT_FALLING

External Interrupt Mode with Falling edge trigger detection

GPIO_MODE_IT_RISING_FALLING

External Interrupt Mode with Rising/Falling edge trigger detection

GPIO_MODE_EVT_RISING

External Event Mode with Rising edge trigger detection

GPIO_MODE_EVT_FALLING

External Event Mode with Falling edge trigger detection

GPIO_MODE_EVT_RISING_FALLING

External Event Mode with Rising/Falling edge trigger detection

GPIO pins define**GPIO_PIN_0****GPIO_PIN_1****GPIO_PIN_2****GPIO_PIN_3****GPIO_PIN_4****GPIO_PIN_5****GPIO_PIN_6****GPIO_PIN_7****GPIO_PIN_8****GPIO_PIN_9****GPIO_PIN_10****GPIO_PIN_11****GPIO_PIN_12****GPIO_PIN_13****GPIO_PIN_14****GPIO_PIN_15****GPIO_PIN_AII****GPIO_PIN_MASK*****GPIO pull define*****GPIO_NOPULL**

No Pull-up or Pull-down activation

GPIO_PULLUP

Pull-up activation

GPIO_PULLDOWN

Pull-down activation

GPIO speed define**GPIO_SPEED_FREQ_LOW**

IO works at 2 MHz, please refer to the product datasheet

GPIO_SPEED_FREQ_MEDIUM

range 12,5 MHz to 50 MHz, please refer to the product datasheet

GPIO_SPEED_FREQ_HIGH

range 25 MHz to 100 MHz, please refer to the product datasheet

GPIO_SPEED_FREQ_VERY_HIGH

range 50 MHz to 200 MHz, please refer to the product datasheet

Notes

- Once the DMA transfer is finished, HAL_HASHEx_SHA256_Finish() API must be called to retrieve the computed digest.

HAL_HASHEx_SHA256_Finish

Function name

HAL_StatusTypeDef HAL_HASHEx_SHA256_Finish (HASH_HandleTypeDef * hhash, uint8_t * pOutBuffer, uint32_t Timeout)

Function description

Return the computed digest in SHA256 mode.

Parameters

- hhash**: HASH handle.
- pOutBuffer**: pointer to the computed digest. Digest size is 32 bytes.
- Timeout**: Timeout value.

Return values

- HAL**: status

Notes

- The API waits for DCIS to be set then reads the computed digest.
- HAL_HASHEx_SHA256_Finish() can be used as well to retrieve the digest in HMAC SHA256 mode.

HAL_HMACEx_SHA224_Start

Function name

HAL_StatusTypeDef HAL_HMACEx_SHA224_Start (HASH_HandleTypeDef * hhash, uint8_t * pInBuffer, uint32_t Size, uint8_t * pOutBuffer, uint32_t Timeout)

Function description

Initialize the HASH peripheral in HMAC SHA224 mode, next process pInBuffer then read the computed digest.

Parameters

- hhash**: HASH handle.
- pInBuffer**: pointer to the input buffer (buffer to be hashed).
- Size**: length of the input buffer in bytes.
- pOutBuffer**: pointer to the computed digest. Digest size is 28 bytes.
- Timeout**: Timeout value.

Return values

- HAL**: status

Notes

- Digest is available in pOutBuffer.
- Same key is used for the inner and the outer hash functions; pointer to key and key size are respectively stored in hhash->Init.pKey and hhash->Init.KeySize.

HAL_HMACEx_SHA256_Start

Function name

HAL_StatusTypeDef HAL_HMACEx_SHA256_Start (HASH_HandleTypeDef * hhash, uint8_t * pInBuffer, uint32_t Size, uint8_t * pOutBuffer, uint32_t Timeout)

Function description

Initialize the HASH peripheral in HMAC SHA256 mode, next process pInBuffer then read the computed digest.

Parameters

- **hhash**: HASH handle.
- **pInBuffer**: pointer to the input buffer (buffer to be hashed).
- **Size**: length of the input buffer in bytes.
- **pOutBuffer**: pointer to the computed digest. Digest size is 32 bytes.
- **Timeout**: Timeout value.

Return values

- **HAL**: status

Notes

- Digest is available in pOutBuffer.
- Same key is used for the inner and the outer hash functions; pointer to key and key size are respectively stored in hhash->Init.pKey and hhash->Init.KeySize.

HAL_HMACEx_SHA224_Start_IT

Function name

HAL_StatusTypeDef HAL_HMACEx_SHA224_Start_IT (HASH_HandleTypeDef * hhash, uint8_t * pInBuffer, uint32_t Size, uint8_t * pOutBuffer)

Function description

Initialize the HASH peripheral in HMAC SHA224 mode, next process pInBuffer then read the computed digest in interrupt mode.

Parameters

- **hhash**: HASH handle.
- **pInBuffer**: pointer to the input buffer (buffer to be hashed).
- **Size**: length of the input buffer in bytes.
- **pOutBuffer**: pointer to the computed digest. Digest size is 28 bytes.

Return values

- **HAL**: status

Notes

- Digest is available in pOutBuffer.
- Same key is used for the inner and the outer hash functions; pointer to key and key size are respectively stored in hhash->Init.pKey and hhash->Init.KeySize.

HAL_HMACEx_SHA256_Start_IT

Function name

HAL_StatusTypeDef HAL_HMACEx_SHA256_Start_IT (HASH_HandleTypeDef * hhash, uint8_t * pInBuffer, uint32_t Size, uint8_t * pOutBuffer)

Function description

Initialize the HASH peripheral in HMAC SHA256 mode, next process pInBuffer then read the computed digest in interrupt mode.

Parameters

- **hhash**: HASH handle.
- **pInBuffer**: pointer to the input buffer (buffer to be hashed).
- **Size**: length of the input buffer in bytes.
- **pOutBuffer**: pointer to the computed digest. Digest size is 32 bytes.

Return values

- **HAL:** status

Notes

- Digest is available in pOutBuffer.
- Same key is used for the inner and the outer hash functions; pointer to key and key size are respectively stored in hhash->Init.pKey and hhash->Init.KeySize.

HAL_HMACEx_SHA224_Start_DMA

Function name

HAL_StatusTypeDef HAL_HMACEx_SHA224_Start_DMA (HASH_HandleTypeDef * hhash, uint8_t * pInBuffer, uint32_t Size)

Function description

Initialize the HASH peripheral in HMAC SHA224 mode then initiate the required DMA transfers to feed the key and the input buffer to the Peripheral.

Parameters

- **hhash:** HASH handle.
- **pInBuffer:** pointer to the input buffer (buffer to be hashed).
- **Size:** length of the input buffer in bytes.

Return values

- **HAL:** status

Notes

- Once the DMA transfers are finished (indicated by hhash->State set back to HAL_HASH_STATE_READY), HAL_HASHEx_SHA224_Finish() API must be called to retrieve the computed digest.
- Same key is used for the inner and the outer hash functions; pointer to key and key size are respectively stored in hhash->Init.pKey and hhash->Init.KeySize.
- If MDMAT bit is set before calling this function (multi-buffer HASH processing case), the input buffer size (in bytes) must be a multiple of 4 otherwise, the HASH digest computation is corrupted. For the processing of the last buffer of the thread, MDMAT bit must be reset and the buffer length (in bytes) doesn't have to be a multiple of 4.

HAL_HMACEx_SHA256_Start_DMA

Function name

HAL_StatusTypeDef HAL_HMACEx_SHA256_Start_DMA (HASH_HandleTypeDef * hhash, uint8_t * pInBuffer, uint32_t Size)

Function description

Initialize the HASH peripheral in HMAC SHA224 mode then initiate the required DMA transfers to feed the key and the input buffer to the Peripheral.

Parameters

- **hhash:** HASH handle.
- **pInBuffer:** pointer to the input buffer (buffer to be hashed).
- **Size:** length of the input buffer in bytes.

Return values

- **HAL:** status

Notes

- Once the DMA transfers are finished (indicated by `hhash->State` set back to `HAL_HASH_STATE_READY`), `HAL_HASHEx_SHA256_Finish()` API must be called to retrieve the computed digest.
- Same key is used for the inner and the outer hash functions; pointer to key and key size are respectively stored in `hhash->Init.pKey` and `hhash->Init.KeySize`.
- If MDMAT bit is set before calling this function (multi-buffer HASH processing case), the input buffer size (in bytes) must be a multiple of 4 otherwise, the HASH digest computation is corrupted. For the processing of the last buffer of the thread, MDMAT bit must be reset and the buffer length (in bytes) doesn't have to be a multiple of 4.

HAL_HMACEx_MD5_Step1_2_DMA

Function name

```
HAL_StatusTypeDef HAL_HMACEx_MD5_Step1_2_DMA (HASH_HandleTypeDef * hhash, uint8_t *
pInBuffer, uint32_t Size)
```

Function description

MD5 HMAC step 1 completion and step 2 start in multi-buffer DMA mode.

Parameters

- **hhash**: HASH handle.
- **pInBuffer**: pointer to the input buffer (message buffer).
- **Size**: length of the input buffer in bytes.

Return values

- **HAL**: status

Notes

- Step 1 consists in writing the inner hash function key in the Peripheral, step 2 consists in writing the message text.
- The API carries out the HMAC step 1 then starts step 2 with the first buffer entered to the Peripheral. DCAL bit is not automatically set after the message buffer feeding, allowing other messages DMA transfers to occur.
- Same key is used for the inner and the outer hash functions; pointer to key and key size are respectively stored in `hhash->Init.pKey` and `hhash->Init.KeySize`.
- The input buffer size (in bytes) must be a multiple of 4 otherwise, the HASH digest computation is corrupted.

HAL_HMACEx_MD5_Step2_DMA

Function name

```
HAL_StatusTypeDef HAL_HMACEx_MD5_Step2_DMA (HASH_HandleTypeDef * hhash, uint8_t *
pInBuffer, uint32_t Size)
```

Function description

MD5 HMAC step 2 in multi-buffer DMA mode.

Parameters

- **hhash**: HASH handle.
- **pInBuffer**: pointer to the input buffer (message buffer).
- **Size**: length of the input buffer in bytes.

Return values

- **HAL**: status

Notes

- Step 2 consists in writing the message text in the Peripheral.
- The API carries on the HMAC step 2, applied to the buffer entered as input parameter. DCAL bit is not automatically set after the message buffer feeding, allowing other messages DMA transfers to occur.
- Same key is used for the inner and the outer hash functions; pointer to key and key size are respectively stored in `hhash->Init.pKey` and `hhash->Init.KeySize`.
- The input buffer size (in bytes) must be a multiple of 4 otherwise, the HASH digest computation is corrupted.

HAL_HMACEx_MD5_Step2_3_DMA

Function name

HAL_StatusTypeDef HAL_HMACEx_MD5_Step2_3_DMA (HASH_HandleTypeDef * hhash, uint8_t * pInBuffer, uint32_t Size)

Function description

MD5 HMAC step 2 wrap-up and step 3 completion in multi-buffer DMA mode.

Parameters

- **hhash**: HASH handle.
- **pInBuffer**: pointer to the input buffer (message buffer).
- **Size**: length of the input buffer in bytes.

Return values

- **HAL**: status

Notes

- Step 2 consists in writing the message text in the Peripheral, step 3 consists in writing the outer hash function key.
- The API wraps up the HMAC step 2 in processing the buffer entered as input parameter (the input buffer must be the last one of the multi-buffer thread) then carries out HMAC step 3.
- Same key is used for the inner and the outer hash functions; pointer to key and key size are respectively stored in `hhash->Init.pKey` and `hhash->Init.KeySize`.
- Once the DMA transfers are finished (indicated by `hhash->State` set back to `HAL_HASH_STATE_READY`), `HAL_HASHEx_SHA256_Finish()` API must be called to retrieve the computed digest.

HAL_HMACEx_SHA1_Step1_2_DMA

Function name

HAL_StatusTypeDef HAL_HMACEx_SHA1_Step1_2_DMA (HASH_HandleTypeDef * hhash, uint8_t * pInBuffer, uint32_t Size)

Function description

SHA1 HMAC step 1 completion and step 2 start in multi-buffer DMA mode.

Parameters

- **hhash**: HASH handle.
- **pInBuffer**: pointer to the input buffer (message buffer).
- **Size**: length of the input buffer in bytes.

Return values

- **HAL**: status

Notes

- Step 1 consists in writing the inner hash function key in the Peripheral, step 2 consists in writing the message text.
- The API carries out the HMAC step 1 then starts step 2 with the first buffer entered to the Peripheral. DCAL bit is not automatically set after the message buffer feeding, allowing other messages DMA transfers to occur.
- Same key is used for the inner and the outer hash functions; pointer to key and key size are respectively stored in `hhash->Init.pKey` and `hhash->Init.KeySize`.
- The input buffer size (in bytes) must be a multiple of 4 otherwise, the HASH digest computation is corrupted.

HAL_HMACEx_SHA1_Step2_DMA

Function name

HAL_StatusTypeDef HAL_HMACEx_SHA1_Step2_DMA (HASH_HandleTypeDef * hhash, uint8_t * pInBuffer, uint32_t Size)

Function description

SHA1 HMAC step 2 in multi-buffer DMA mode.

Parameters

- **hhash**: HASH handle.
- **pInBuffer**: pointer to the input buffer (message buffer).
- **Size**: length of the input buffer in bytes.

Return values

- **HAL**: status

Notes

- Step 2 consists in writing the message text in the Peripheral.
- The API carries on the HMAC step 2, applied to the buffer entered as input parameter. DCAL bit is not automatically set after the message buffer feeding, allowing other messages DMA transfers to occur.
- Same key is used for the inner and the outer hash functions; pointer to key and key size are respectively stored in `hhash->Init.pKey` and `hhash->Init.KeySize`.
- The input buffer size (in bytes) must be a multiple of 4 otherwise, the HASH digest computation is corrupted.

HAL_HMACEx_SHA1_Step2_3_DMA

Function name

HAL_StatusTypeDef HAL_HMACEx_SHA1_Step2_3_DMA (HASH_HandleTypeDef * hhash, uint8_t * pInBuffer, uint32_t Size)

Function description

SHA1 HMAC step 2 wrap-up and step 3 completion in multi-buffer DMA mode.

Parameters

- **hhash**: HASH handle.
- **pInBuffer**: pointer to the input buffer (message buffer).
- **Size**: length of the input buffer in bytes.

Return values

- **HAL**: status

Notes

- Step 2 consists in writing the message text in the Peripheral, step 3 consists in writing the outer hash function key.
- The API wraps up the HMAC step 2 in processing the buffer entered as input parameter (the input buffer must be the last one of the multi-buffer thread) then carries out HMAC step 3.
- Same key is used for the inner and the outer hash functions; pointer to key and key size are respectively stored in `hhash->Init.pKey` and `hhash->Init.KeySize`.
- Once the DMA transfers are finished (indicated by `hhash->State` set back to `HAL_HASH_STATE_READY`), `HAL_HASHEx_SHA256_Finish()` API must be called to retrieve the computed digest.

HAL_HMACEx_SHA224_Step1_2_DMA

Function name

HAL_StatusTypeDef HAL_HMACEx_SHA224_Step1_2_DMA (HASH_HandleTypeDef * hhash, uint8_t * pInBuffer, uint32_t Size)

Function description

SHA224 HMAC step 1 completion and step 2 start in multi-buffer DMA mode.

Parameters

- **hhash**: HASH handle.
- **pInBuffer**: pointer to the input buffer (message buffer).
- **Size**: length of the input buffer in bytes.

Return values

- **HAL**: status

Notes

- Step 1 consists in writing the inner hash function key in the Peripheral, step 2 consists in writing the message text.
- The API carries out the HMAC step 1 then starts step 2 with the first buffer entered to the Peripheral. DCAL bit is not automatically set after the message buffer feeding, allowing other messages DMA transfers to occur.
- Same key is used for the inner and the outer hash functions; pointer to key and key size are respectively stored in `hhash->Init.pKey` and `hhash->Init.KeySize`.
- The input buffer size (in bytes) must be a multiple of 4 otherwise, the HASH digest computation is corrupted.

HAL_HMACEx_SHA224_Step2_DMA

Function name

HAL_StatusTypeDef HAL_HMACEx_SHA224_Step2_DMA (HASH_HandleTypeDef * hhash, uint8_t * pInBuffer, uint32_t Size)

Function description

SHA224 HMAC step 2 in multi-buffer DMA mode.

Parameters

- **hhash**: HASH handle.
- **pInBuffer**: pointer to the input buffer (message buffer).
- **Size**: length of the input buffer in bytes.

Return values

- **HAL**: status

Notes

- Step 2 consists in writing the message text in the Peripheral.
- The API carries on the HMAC step 2, applied to the buffer entered as input parameter. DCAL bit is not automatically set after the message buffer feeding, allowing other messages DMA transfers to occur.
- Same key is used for the inner and the outer hash functions; pointer to key and key size are respectively stored in `hhash->Init.pKey` and `hhash->Init.KeySize`.
- The input buffer size (in bytes) must be a multiple of 4 otherwise, the HASH digest computation is corrupted.

HAL_HMACEx_SHA224_Step2_3_DMA

Function name

HAL_StatusTypeDef HAL_HMACEx_SHA224_Step2_3_DMA (HASH_HandleTypeDef * hhash, uint8_t * pInBuffer, uint32_t Size)

Function description

SHA224 HMAC step 2 wrap-up and step 3 completion in multi-buffer DMA mode.

Parameters

- **hhash**: HASH handle.
- **pInBuffer**: pointer to the input buffer (message buffer).
- **Size**: length of the input buffer in bytes.

Return values

- **HAL**: status

Notes

- Step 2 consists in writing the message text in the Peripheral, step 3 consists in writing the outer hash function key.
- The API wraps up the HMAC step 2 in processing the buffer entered as input parameter (the input buffer must be the last one of the multi-buffer thread) then carries out HMAC step 3.
- Same key is used for the inner and the outer hash functions; pointer to key and key size are respectively stored in `hhash->Init.pKey` and `hhash->Init.KeySize`.
- Once the DMA transfers are finished (indicated by `hhash->State` set back to `HAL_HASH_STATE_READY`), `HAL_HASHEx_SHA256_Finish()` API must be called to retrieve the computed digest.

HAL_HMACEx_SHA256_Step1_2_DMA

Function name

HAL_StatusTypeDef HAL_HMACEx_SHA256_Step1_2_DMA (HASH_HandleTypeDef * hhash, uint8_t * pInBuffer, uint32_t Size)

Function description

SHA256 HMAC step 1 completion and step 2 start in multi-buffer DMA mode.

Parameters

- **hhash**: HASH handle.
- **pInBuffer**: pointer to the input buffer (message buffer).
- **Size**: length of the input buffer in bytes.

Return values

- **HAL**: status

Notes

- Step 1 consists in writing the inner hash function key in the Peripheral, step 2 consists in writing the message text.
- The API carries out the HMAC step 1 then starts step 2 with the first buffer entered to the Peripheral. DCAL bit is not automatically set after the message buffer feeding, allowing other messages DMA transfers to occur.
- Same key is used for the inner and the outer hash functions; pointer to key and key size are respectively stored in `hhash->Init.pKey` and `hhash->Init.KeySize`.
- The input buffer size (in bytes) must be a multiple of 4 otherwise, the HASH digest computation is corrupted.

HAL_HMACEx_SHA256_Step2_DMA

Function name

HAL_StatusTypeDef HAL_HMACEx_SHA256_Step2_DMA (HASH_HandleTypeDef * hhash, uint8_t * pInBuffer, uint32_t Size)

Function description

SHA256 HMAC step 2 in multi-buffer DMA mode.

Parameters

- **hhash:** HASH handle.
- **pInBuffer:** pointer to the input buffer (message buffer).
- **Size:** length of the input buffer in bytes.

Return values

- **HAL:** status

Notes

- Step 2 consists in writing the message text in the Peripheral.
- The API carries on the HMAC step 2, applied to the buffer entered as input parameter. DCAL bit is not automatically set after the message buffer feeding, allowing other messages DMA transfers to occur.
- Same key is used for the inner and the outer hash functions; pointer to key and key size are respectively stored in `hhash->Init.pKey` and `hhash->Init.KeySize`.
- The input buffer size (in bytes) must be a multiple of 4 otherwise, the HASH digest computation is corrupted.

HAL_HMACEx_SHA256_Step2_3_DMA

Function name

HAL_StatusTypeDef HAL_HMACEx_SHA256_Step2_3_DMA (HASH_HandleTypeDef * hhash, uint8_t * pInBuffer, uint32_t Size)

Function description

SHA256 HMAC step 2 wrap-up and step 3 completion in multi-buffer DMA mode.

Parameters

- **hhash:** HASH handle.
- **pInBuffer:** pointer to the input buffer (message buffer).
- **Size:** length of the input buffer in bytes.

Return values

- **HAL:** status

Notes

- Step 2 consists in writing the message text in the Peripheral, step 3 consists in writing the outer hash function key.
- The API wraps up the HMAC step 2 in processing the buffer entered as input parameter (the input buffer must be the last one of the multi-buffer thread) then carries out HMAC step 3.
- Same key is used for the inner and the outer hash functions; pointer to key and key size are respectively stored in `hhash->Init.pKey` and `hhash->Init.KeySize`.
- Once the DMA transfers are finished (indicated by `hhash->State` set back to `HAL_HASH_STATE_READY`), `HAL_HASHEX_SHA256_Finish()` API must be called to retrieve the computed digest.

35 HAL HCD Generic Driver

35.1 HCD Firmware driver registers structures

35.1.1 HCD_HandleTypeDef

HCD_HandleTypeDef is defined in the stm32f4xx_hal_hcd.h

Data Fields

- **HCD_TypeDef* Instance**
- **HCD_InitTypeDef Init**
- **HCD_HCTypeDef hc**
- **HAL_LockTypeDef Lock**
- **__IO HCD_StateTypeDef State**
- **__IO uint32_t ErrorCode**
- **void* pData**

Field Documentation

- **HCD_TypeDef* HCD_HandleTypeDef::Instance**
Register base address
- **HCD_InitTypeDef HCD_HandleTypeDef::Init**
HCD required parameters
- **HCD_HCTypeDef HCD_HandleTypeDef::hc[16]**
Host channels parameters
- **HAL_LockTypeDef HCD_HandleTypeDef::Lock**
HCD peripheral status
- **__IO HCD_StateTypeDef HCD_HandleTypeDef::State**
HCD communication state
- **__IO uint32_t HCD_HandleTypeDef::ErrorCode**
HCD Error code
- **void* HCD_HandleTypeDef::pData**
Pointer Stack Handler

35.2 HCD Firmware driver API description

The following section lists the various functions of the HCD library.

35.2.1 How to use this driver

1. Declare a HCD_HandleTypeDef handle structure, for example: HCD_HandleTypeDef hhcd;
2. Fill parameters of Init structure in HCD handle
3. Call HAL_HCD_Init() API to initialize the HCD peripheral (Core, Host core, ...)
4. Initialize the HCD low level resources through the HAL_HCD_MspInit() API:
 - a. Enable the HCD/USB Low Level interface clock using the following macros
 - `__HAL_RCC_USB_OTG_FS_CLK_ENABLE();`
 - `__HAL_RCC_USB_OTG_HS_CLK_ENABLE();` (For High Speed Mode)
 - `__HAL_RCC_USB_OTG_HS_ULPI_CLK_ENABLE();` (For High Speed Mode)
 - b. Initialize the related GPIO clocks
 - c. Configure HCD pin-out
 - d. Configure HCD NVIC interrupt
5. Associate the Upper USB Host stack to the HAL HCD Driver:
 - a. `hhcd.pData = phost;`
6. Enable HCD transmission and reception:
 - a. `HAL_HCD_Start();`

35.2.2 Initialization and de-initialization functions

This section provides functions allowing to:

This section contains the following APIs:

- *HAL_HCD_Init()*
- *HAL_HCD_HC_Init()*
- *HAL_HCD_HC_Halt()*
- *HAL_HCD_DeInit()*
- *HAL_HCD_MspInit()*
- *HAL_HCD_MspDeInit()*
- *HAL_HCD_RegisterCallback()*
- *HAL_HCD_UnRegisterCallback()*
- *HAL_HCD_RegisterHC_NotifyURBChangeCallback()*
- *HAL_HCD_UnRegisterHC_NotifyURBChangeCallback()*

35.2.3 IO operation functions

This subsection provides a set of functions allowing to manage the USB Host Data Transfer

This section contains the following APIs:

- *HAL_HCD_HC_SubmitRequest()*
- *HAL_HCD_IRQHandler()*
- *HAL_HCD_WKUP_IRQHandler()*
- *HAL_HCD_SOF_Callback()*
- *HAL_HCD_Connect_Callback()*
- *HAL_HCD_Disconnect_Callback()*
- *HAL_HCD_PortEnabled_Callback()*
- *HAL_HCD_PortDisabled_Callback()*
- *HAL_HCD_HC_NotifyURBChange_Callback()*
- *HAL_HCD_RegisterCallback()*
- *HAL_HCD_UnRegisterCallback()*
- *HAL_HCD_RegisterHC_NotifyURBChangeCallback()*
- *HAL_HCD_UnRegisterHC_NotifyURBChangeCallback()*

35.2.4 Peripheral Control functions

This subsection provides a set of functions allowing to control the HCD data transfers.

This section contains the following APIs:

- *HAL_HCD_Start()*
- *HAL_HCD_Stop()*
- *HAL_HCD_ResetPort()*

35.2.5 Peripheral State functions

This subsection permits to get in run-time the status of the peripheral and the data flow.

This section contains the following APIs:

- *HAL_HCD_GetState()*
- *HAL_HCD_HC_GetURBState()*
- *HAL_HCD_HC_GetXferCount()*
- *HAL_HCD_HC_GetState()*
- *HAL_HCD_GetCurrentFrame()*
- *HAL_HCD_GetCurrentSpeed()*

35.2.6 Detailed description of functions

HAL_HCD_Init

Function name

```
HAL_StatusTypeDef HAL_HCD_Init (HCD_HandleTypeDef * hhcd)
```

Function description

Initialize the host driver.

Parameters

- **hhcd**: HCD handle

Return values

- **HAL**: status

HAL_HCD_DeInit

Function name

```
HAL_StatusTypeDef HAL_HCD_DeInit (HCD_HandleTypeDef * hhcd)
```

Function description

Deinitialize the host driver.

Parameters

- **hhcd**: HCD handle

Return values

- **HAL**: status

HAL_HCD_HC_Init

Function name

```
HAL_StatusTypeDef HAL_HCD_HC_Init (HCD_HandleTypeDef * hhcd, uint8_t ch_num, uint8_t epnum, uint8_t dev_address, uint8_t speed, uint8_t ep_type, uint16_t mps)
```

Function description

Initialize a host channel.

Parameters

- **hhcd**: HCD handle
- **ch_num**: Channel number. This parameter can be a value from 1 to 15
- **epnum**: Endpoint number. This parameter can be a value from 1 to 15
- **dev_address**: Current device address This parameter can be a value from 0 to 255
- **speed**: Current device speed. This parameter can be one of these values: HCD_DEVICE_SPEED_HIGH: High speed mode, HCD_DEVICE_SPEED_FULL: Full speed mode, HCD_DEVICE_SPEED_LOW: Low speed mode
- **ep_type**: Endpoint Type. This parameter can be one of these values: EP_TYPE_CTRL: Control type, EP_TYPE_ISOC: Isochronous type, EP_TYPE_BULK: Bulk type, EP_TYPE_INTR: Interrupt type
- **mps**: Max Packet Size. This parameter can be a value from 0 to 32K

Return values

- **HAL**: status

HAL_HCD_HC_Halt

Function name

HAL_StatusTypeDef HAL_HCD_HC_Halt (HCD_HandleTypeDef * hhcd, uint8_t ch_num)

Function description

Halt a host channel.

Parameters

- **hhcd**: HCD handle
- **ch_num**: Channel number. This parameter can be a value from 1 to 15

Return values

- **HAL**: status

HAL_HCD_Msplnit

Function name

void HAL_HCD_Msplnit (HCD_HandleTypeDef * hhcd)

Function description

Initialize the HCD MSP.

Parameters

- **hhcd**: HCD handle

Return values

- **None**:

HAL_HCD_MspDeInit

Function name

void HAL_HCD_MspDeInit (HCD_HandleTypeDef * hhcd)

Function description

Deinitialize the HCD MSP.

Parameters

- **hhcd**: HCD handle

Return values

- **None**:

HAL_HCD_RegisterCallback

Function name

HAL_StatusTypeDef HAL_HCD_RegisterCallback (HCD_HandleTypeDef * hhcd, HAL_HCD_CallbackIDTypeDef CallbackID, pHCD_CallbackTypeDef pCallback)

Function description

Register a User USB HCD Callback To be used instead of the weak predefined callback.

Parameters

- **hhcd**: USB HCD handle
- **CallbackID**: ID of the callback to be registered This parameter can be one of the following values:
 - HAL_HCD_SOF_CB_ID USB HCD SOF callback ID
 - HAL_HCD_CONNECT_CB_ID USB HCD Connect callback ID
 - HAL_HCD_DISCONNECT_CB_ID OTG HCD Disconnect callback ID
 - HAL_HCD_PORT_ENABLED_CB_ID USB HCD Port Enable callback ID
 - HAL_HCD_PORT_DISABLED_CB_ID USB HCD Port Disable callback ID
 - HAL_HCD_MSPINIT_CB_ID MspDeInit callback ID
 - HAL_HCD_MSPDEINIT_CB_ID MspDeInit callback ID
- **pCallback**: pointer to the Callback function

Return values

- **HAL**: status

HAL_HCD_UnRegisterCallback

Function name

HAL_StatusTypeDef HAL_HCD_UnRegisterCallback (HCD_HandleTypeDef * hhcd, HAL_HCD_CallbackIDTypeDef CallbackID)

Function description

Unregister an USB HCD Callback USB HCD callback is redirected to the weak predefined callback.

Parameters

- **hhcd**: USB HCD handle
- **CallbackID**: ID of the callback to be unregistered This parameter can be one of the following values:
 - HAL_HCD_SOF_CB_ID USB HCD SOF callback ID
 - HAL_HCD_CONNECT_CB_ID USB HCD Connect callback ID
 - HAL_HCD_DISCONNECT_CB_ID OTG HCD Disconnect callback ID
 - HAL_HCD_PORT_ENABLED_CB_ID USB HCD Port Enabled callback ID
 - HAL_HCD_PORT_DISABLED_CB_ID USB HCD Port Disabled callback ID
 - HAL_HCD_MSPINIT_CB_ID MspDeInit callback ID
 - HAL_HCD_MSPDEINIT_CB_ID MspDeInit callback ID

Return values

- **HAL**: status

HAL_HCD_RegisterHC_NotifyURBChangeCallback

Function name

HAL_StatusTypeDef HAL_HCD_RegisterHC_NotifyURBChangeCallback (HCD_HandleTypeDef * hhcd, pHCD_HC_NotifyURBChangeCallbackTypeDef pCallback)

Function description

Register USB HCD Host Channel Notify URB Change Callback To be used instead of the weak HAL_HCD_HC_NotifyURBChange_Callback() predefined callback.

Parameters

- **hhcd**: HCD handle
- **pCallback**: pointer to the USB HCD Host Channel Notify URB Change Callback function

Return values

- **HAL**: status

HAL_HCD_UnRegisterHC_NotifyURBChangeCallback

Function name

HAL_StatusTypeDef HAL_HCD_UnRegisterHC_NotifyURBChangeCallback (HCD_HandleTypeDef * hhcd)

Function description

Unregister the USB HCD Host Channel Notify URB Change Callback USB HCD Host Channel Notify URB Change Callback is redirected to the weak HAL_HCD_HC_NotifyURBChange_Callback() predefined callback.

Parameters

- **hhcd**: HCD handle

Return values

- **HAL**: status

HAL_HCD_HC_SubmitRequest

Function name

HAL_StatusTypeDef HAL_HCD_HC_SubmitRequest (HCD_HandleTypeDef * hhcd, uint8_t ch_num, uint8_t direction, uint8_t ep_type, uint8_t token, uint8_t * pbuff, uint16_t length, uint8_t do_ping)

Function description

Submit a new URB for processing.

Parameters

- **hhcd**: HCD handle
- **ch_num**: Channel number. This parameter can be a value from 1 to 15
- **direction**: Channel number. This parameter can be one of these values: 0 : Output / 1 : Input
- **ep_type**: Endpoint Type. This parameter can be one of these values: EP_TYPE_CTRL: Control type/ EP_TYPE_ISOC: Isochronous type/ EP_TYPE_BULK: Bulk type/ EP_TYPE_INTR: Interrupt type/
- **token**: Endpoint Type. This parameter can be one of these values: 0: HC_PID_SETUP / 1: HC_PID_DATA1
- **pbuff**: pointer to URB data
- **length**: Length of URB data
- **do_ping**: activate do ping protocol (for high speed only). This parameter can be one of these values: 0 : do ping inactive / 1 : do ping active

Return values

- **HAL**: status

HAL_HCD_IRQHandler

Function name

void HAL_HCD_IRQHandler (HCD_HandleTypeDef * hhcd)

Function description

Handle HCD interrupt request.

Parameters

- **hhcd**: HCD handle

Return values

- **None**:

HAL_HCD_WKUP_IRQHandler

Function name

void HAL_HCD_WKUP_IRQHandler (HCD_HandleTypeDef * hhcd)

Function description

Handles HCD Wakeup interrupt request.

Parameters

- **hhcd**: HCD handle

Return values

- **HAL**: status

HAL_HCD_SOF_Callback

Function name

void HAL_HCD_SOF_Callback (HCD_HandleTypeDef * hhcd)

Function description

SOF callback.

Parameters

- **hhcd**: HCD handle

Return values

- **None**:

HAL_HCD_Connect_Callback

Function name

void HAL_HCD_Connect_Callback (HCD_HandleTypeDef * hhcd)

Function description

Connection Event callback.

Parameters

- **hhcd**: HCD handle

Return values

- **None**:

HAL_HCD_Disconnect_Callback

Function name

void HAL_HCD_Disconnect_Callback (HCD_HandleTypeDef * hhcd)

Function description

Disconnection Event callback.

Parameters

- **hhcd**: HCD handle

Return values

- **None**:

HAL_HCD_PortEnabled_Callback

Function name

```
void HAL_HCD_PortEnabled_Callback (HCD_HandleTypeDef * hhcd)
```

Function description

Port Enabled Event callback.

Parameters

- **hhcd**: HCD handle

Return values

- **None**:

HAL_HCD_PortDisabled_Callback

Function name

```
void HAL_HCD_PortDisabled_Callback (HCD_HandleTypeDef * hhcd)
```

Function description

Port Disabled Event callback.

Parameters

- **hhcd**: HCD handle

Return values

- **None**:

HAL_HCD_HC_NotifyURBChange_Callback

Function name

```
void HAL_HCD_HC_NotifyURBChange_Callback (HCD_HandleTypeDef * hhcd, uint8_t chnum,
HCD_URBStateTypeDef urb_state)
```

Function description

Notify URB state change callback.

Parameters

- **hhcd**: HCD handle
- **chnum**: Channel number. This parameter can be a value from 1 to 15
- **urb_state**: This parameter can be one of these values: URB_IDLE/ URB_DONE/ URB_NOTREADY/ URB_NYET/ URB_ERROR/ URB_STALL/

Return values

- **None**:

HAL_HCD_ResetPort

Function name

```
HAL_StatusTypeDef HAL_HCD_ResetPort (HCD_HandleTypeDef * hhcd)
```

Function description

Reset the host port.

Parameters

- **hhcd**: HCD handle

Return values

- **HAL:** status

HAL_HCD_Start**Function name**

HAL_StatusTypeDef HAL_HCD_Start (HCD_HandleTypeDef * hhcd)

Function description

Start the host driver.

Parameters

- **hhcd:** HCD handle

Return values

- **HAL:** status

HAL_HCD_Stop**Function name**

HAL_StatusTypeDef HAL_HCD_Stop (HCD_HandleTypeDef * hhcd)

Function description

Stop the host driver.

Parameters

- **hhcd:** HCD handle

Return values

- **HAL:** status

HAL_HCD_GetState**Function name**

HCD_StateTypeDef HAL_HCD_GetState (HCD_HandleTypeDef * hhcd)

Function description

Return the HCD handle state.

Parameters

- **hhcd:** HCD handle

Return values

- **HAL:** state

HAL_HCD_HC_GetURBState**Function name**

HCD_URBStateTypeDef HAL_HCD_HC_GetURBState (HCD_HandleTypeDef * hhcd, uint8_t chnum)

Function description

Return URB state for a channel.

Parameters

- **hhcd:** HCD handle
- **chnum:** Channel number. This parameter can be a value from 1 to 15

Return values

- **URB:** state. This parameter can be one of these values: URB_IDLE/ URB_DONE/ URB_NOTREADY/ URB_NYET/ URB_ERROR/ URB_STALL

HAL_HCD_HC_GetState

Function name

HCD_HCStateTypeDef HAL_HCD_HC_GetState (HCD_HandleTypeDef * hhcd, uint8_t chnum)

Function description

Return the Host Channel state.

Parameters

- **hhcd:** HCD handle
- **chnum:** Channel number. This parameter can be a value from 1 to 15

Return values

- **Host:** channel state This parameter can be one of these values: HC_IDLE/ HC_XFRC/ HC_HALTED/ HC_NYET/ HC_NAK/ HC_STALL/ HC_XACTERR/ HC_BBLERR/ HC_DATATGLERR

HAL_HCD_HC_GetXferCount

Function name

uint32_t HAL_HCD_HC_GetXferCount (HCD_HandleTypeDef * hhcd, uint8_t chnum)

Function description

Return the last host transfer size.

Parameters

- **hhcd:** HCD handle
- **chnum:** Channel number. This parameter can be a value from 1 to 15

Return values

- **last:** transfer size in byte

HAL_HCD_GetCurrentFrame

Function name

uint32_t HAL_HCD_GetCurrentFrame (HCD_HandleTypeDef * hhcd)

Function description

Return the current Host frame number.

Parameters

- **hhcd:** HCD handle

Return values

- **Current:** Host frame number

HAL_HCD_GetCurrentSpeed

Function name

uint32_t HAL_HCD_GetCurrentSpeed (HCD_HandleTypeDef * hhcd)

Function description

Return the Host enumeration speed.

Parameters

- **hhcd:** HCD handle

Return values

- **Enumeration:** speed

35.3 HCD Firmware driver defines

The following section lists the various define and macros of the module.

35.3.1 HCD

HCD

HCD Device Speed

HCD_DEVICE_SPEED_HIGH

HCD_DEVICE_SPEED_FULL

HCD_DEVICE_SPEED_LOW

HCD Error Code definition

HAL_HCD_ERROR_INVALID_CALLBACK

Invalid Callback error

HCD Exported Macros

__HAL_HCD_ENABLE

__HAL_HCD_DISABLE

__HAL_HCD_GET_FLAG

__HAL_HCD_GET_CH_FLAG

__HAL_HCD_CLEAR_FLAG

__HAL_HCD_IS_INVALID_INTERRUPT

__HAL_HCD_CLEAR_HC_INT

__HAL_HCD_MASK_HALT_HC_INT

__HAL_HCD_UNMASK_HALT_HC_INT

__HAL_HCD_MASK_ACK_HC_INT

__HAL_HCD_UNMASK_ACK_HC_INT

HCD PHY Module

HCD_PHY_ULPI

HCD_PHY_EMBEDDED

HCD Speed



HCD_SPEED_HIGH

HCD_SPEED_FULL

HCD_SPEED_LOW

36 HAL I2C Generic Driver

36.1 I2C Firmware driver registers structures

36.1.1 I2C_InitTypeDef

I2C_InitTypeDef is defined in the stm32f4xx_hal_i2c.h

Data Fields

- *uint32_t* **ClockSpeed**
- *uint32_t* **DutyCycle**
- *uint32_t* **OwnAddress1**
- *uint32_t* **AddressingMode**
- *uint32_t* **DualAddressMode**
- *uint32_t* **OwnAddress2**
- *uint32_t* **GeneralCallMode**
- *uint32_t* **NoStretchMode**

Field Documentation

- *uint32_t* **I2C_InitTypeDef::ClockSpeed**
Specifies the clock frequency. This parameter must be set to a value lower than 400kHz
- *uint32_t* **I2C_InitTypeDef::DutyCycle**
Specifies the I2C fast mode duty cycle. This parameter can be a value of [I2C_duty_cycle_in_fast_mode](#)
- *uint32_t* **I2C_InitTypeDef::OwnAddress1**
Specifies the first device own address. This parameter can be a 7-bit or 10-bit address.
- *uint32_t* **I2C_InitTypeDef::AddressingMode**
Specifies if 7-bit or 10-bit addressing mode is selected. This parameter can be a value of [I2C_addressing_mode](#)
- *uint32_t* **I2C_InitTypeDef::DualAddressMode**
Specifies if dual addressing mode is selected. This parameter can be a value of [I2C_dual_addressing_mode](#)
- *uint32_t* **I2C_InitTypeDef::OwnAddress2**
Specifies the second device own address if dual addressing mode is selected This parameter can be a 7-bit address.
- *uint32_t* **I2C_InitTypeDef::GeneralCallMode**
Specifies if general call mode is selected. This parameter can be a value of [I2C_general_call_addressing_mode](#)
- *uint32_t* **I2C_InitTypeDef::NoStretchMode**
Specifies if nostretch mode is selected. This parameter can be a value of [I2C_nostretch_mode](#)

36.1.2 __I2C_HandleTypeDef

__I2C_HandleTypeDef is defined in the stm32f4xx_hal_i2c.h

Data Fields

- *I2C_TypeDef ** **Instance**
- *I2C_InitTypeDef* **Init**
- *uint8_t ** **pBuffPtr**
- *uint16_t* **XferSize**
- *__IO uint16_t* **XferCount**
- *__IO uint32_t* **XferOptions**
- *__IO uint32_t* **PreviousState**
- *DMA_HandleTypeDef ** **hdmatx**
- *DMA_HandleTypeDef ** **hdmarx**
- *HAL_LockTypeDef* **Lock**
- *__IO HAL_I2C_StateTypeDef* **State**

- **__IO HAL_I2C_ModeTypeDef Mode**
- **__IO uint32_t ErrorCode**
- **__IO uint32_t Devaddress**
- **__IO uint32_t Memaddress**
- **__IO uint32_t MemaddSize**
- **__IO uint32_t EventCount**
- **void(* MasterTxCpltCallback**
- **void(* MasterRxCpltCallback**
- **void(* SlaveTxCpltCallback**
- **void(* SlaveRxCpltCallback**
- **void(* ListenCpltCallback**
- **void(* MemTxCpltCallback**
- **void(* MemRxCpltCallback**
- **void(* ErrorCallback**
- **void(* AbortCpltCallback**
- **void(* AddrCallback**
- **void(* MspInitCallback**
- **void(* MspDeInitCallback**

Field Documentation

- **I2C_TypeDef* __I2C_HandleTypeDef::Instance**
I2C registers base address
- **I2C_InitTypeDef __I2C_HandleTypeDef::Init**
I2C communication parameters
- **uint8_t* __I2C_HandleTypeDef::pBuffPtr**
Pointer to I2C transfer buffer
- **uint16_t __I2C_HandleTypeDef::XferSize**
I2C transfer size
- **__IO uint16_t __I2C_HandleTypeDef::XferCount**
I2C transfer counter
- **__IO uint32_t __I2C_HandleTypeDef::XferOptions**
I2C transfer options
- **__IO uint32_t __I2C_HandleTypeDef::PreviousState**
I2C communication Previous state and mode context for internal usage
- **DMA_HandleTypeDef* __I2C_HandleTypeDef::hdmatx**
I2C Tx DMA handle parameters
- **DMA_HandleTypeDef* __I2C_HandleTypeDef::hdmarx**
I2C Rx DMA handle parameters
- **HAL_LockTypeDef __I2C_HandleTypeDef::Lock**
I2C locking object
- **__IO HAL_I2C_StateTypeDef __I2C_HandleTypeDef::State**
I2C communication state
- **__IO HAL_I2C_ModeTypeDef __I2C_HandleTypeDef::Mode**
I2C communication mode
- **__IO uint32_t __I2C_HandleTypeDef::ErrorCode**
I2C Error code
- **__IO uint32_t __I2C_HandleTypeDef::Devaddress**
I2C Target device address
- **__IO uint32_t __I2C_HandleTypeDef::Memaddress**
I2C Target memory address
- **__IO uint32_t __I2C_HandleTypeDef::MemaddSize**
I2C Target memory address size
- **__IO uint32_t __I2C_HandleTypeDef::EventCount**
I2C Event counter

- **`void(* __I2C_HandleTypeDef::MasterTxCpltCallback)(struct __I2C_HandleTypeDef *hi2c)`**
I2C Master Tx Transfer completed callback
- **`void(* __I2C_HandleTypeDef::MasterRxCpltCallback)(struct __I2C_HandleTypeDef *hi2c)`**
I2C Master Rx Transfer completed callback
- **`void(* __I2C_HandleTypeDef::SlaveTxCpltCallback)(struct __I2C_HandleTypeDef *hi2c)`**
I2C Slave Tx Transfer completed callback
- **`void(* __I2C_HandleTypeDef::SlaveRxCpltCallback)(struct __I2C_HandleTypeDef *hi2c)`**
I2C Slave Rx Transfer completed callback
- **`void(* __I2C_HandleTypeDef::ListenCpltCallback)(struct __I2C_HandleTypeDef *hi2c)`**
I2C Listen Complete callback
- **`void(* __I2C_HandleTypeDef::MemTxCpltCallback)(struct __I2C_HandleTypeDef *hi2c)`**
I2C Memory Tx Transfer completed callback
- **`void(* __I2C_HandleTypeDef::MemRxCpltCallback)(struct __I2C_HandleTypeDef *hi2c)`**
I2C Memory Rx Transfer completed callback
- **`void(* __I2C_HandleTypeDef::ErrorCallback)(struct __I2C_HandleTypeDef *hi2c)`**
I2C Error callback
- **`void(* __I2C_HandleTypeDef::AbortCpltCallback)(struct __I2C_HandleTypeDef *hi2c)`**
I2C Abort callback
- **`void(* __I2C_HandleTypeDef::AddrCallback)(struct __I2C_HandleTypeDef *hi2c, uint8_t TransferDirection, uint16_t AddrMatchCode)`**
I2C Slave Address Match callback
- **`void(* __I2C_HandleTypeDef::MspInitCallback)(struct __I2C_HandleTypeDef *hi2c)`**
I2C Msp Init callback
- **`void(* __I2C_HandleTypeDef::MspDeInitCallback)(struct __I2C_HandleTypeDef *hi2c)`**
I2C Msp DeInit callback

36.2 I2C Firmware driver API description

The following section lists the various functions of the I2C library.

36.2.1 How to use this driver

The I2C HAL driver can be used as follows:

1. Declare a `I2C_HandleTypeDef` handle structure, for example: `I2C_HandleTypeDef hi2c`;
2. Initialize the I2C low level resources by implementing the `HAL_I2C_MspInit()` API:
 - a. Enable the I2Cx interface clock
 - b. I2C pins configuration
 - Enable the clock for the I2C GPIOs
 - Configure I2C pins as alternate function open-drain
 - c. NVIC configuration if you need to use interrupt process
 - Configure the I2Cx interrupt priority
 - Enable the NVIC I2C IRQ Channel
 - d. DMA Configuration if you need to use DMA process
 - Declare a `DMA_HandleTypeDef` handle structure for the transmit or receive stream
 - Enable the DMAx interface clock using
 - Configure the DMA handle parameters
 - Configure the DMA Tx or Rx stream
 - Associate the initialized DMA handle to the hi2c DMA Tx or Rx handle
 - Configure the priority and enable the NVIC for the transfer complete interrupt on the DMA Tx or Rx stream
3. Configure the Communication Speed, Duty cycle, Addressing mode, Own Address1, Dual Addressing mode, Own Address2, General call and Nostretch mode in the `hi2c` Init structure.
4. Initialize the I2C registers by calling the `HAL_I2C_Init()`, configures also the low level Hardware (GPIO, CLOCK, NVIC...etc) by calling the customized `HAL_I2C_MspInit()` API.
5. To check if target device is ready for communication, use the function `HAL_I2C_IsDeviceReady()`

6. For I2C IO and IO MEM operations, three operation modes are available within this driver :

Polling mode IO operation

- Transmit in master mode an amount of data in blocking mode using HAL_I2C_Master_Transmit()
- Receive in master mode an amount of data in blocking mode using HAL_I2C_Master_Receive()
- Transmit in slave mode an amount of data in blocking mode using HAL_I2C_Slave_Transmit()
- Receive in slave mode an amount of data in blocking mode using HAL_I2C_Slave_Receive()

Polling mode IO MEM operation

- Write an amount of data in blocking mode to a specific memory address using HAL_I2C_Mem_Write()
- Read an amount of data in blocking mode from a specific memory address using HAL_I2C_Mem_Read()

Interrupt mode IO operation

- Transmit in master mode an amount of data in non-blocking mode using HAL_I2C_Master_Transmit_IT()
- At transmission end of transfer, HAL_I2C_MasterTxCpltCallback() is executed and user can add his own code by customization of function pointer HAL_I2C_MasterTxCpltCallback()
- Receive in master mode an amount of data in non-blocking mode using HAL_I2C_Master_Receive_IT()
- At reception end of transfer, HAL_I2C_MasterRxCpltCallback() is executed and user can add his own code by customization of function pointer HAL_I2C_MasterRxCpltCallback()
- Transmit in slave mode an amount of data in non-blocking mode using HAL_I2C_Slave_Transmit_IT()
- At transmission end of transfer, HAL_I2C_SlaveTxCpltCallback() is executed and user can add his own code by customization of function pointer HAL_I2C_SlaveTxCpltCallback()
- Receive in slave mode an amount of data in non-blocking mode using HAL_I2C_Slave_Receive_IT()
- At reception end of transfer, HAL_I2C_SlaveRxCpltCallback() is executed and user can add his own code by customization of function pointer HAL_I2C_SlaveRxCpltCallback()
- In case of transfer Error, HAL_I2C_ErrorCallback() function is executed and user can add his own code by customization of function pointer HAL_I2C_ErrorCallback()
- Abort a master I2C process communication with Interrupt using HAL_I2C_Master_Abort_IT()
- End of abort process, HAL_I2C_AbortCpltCallback() is executed and user can add his own code by customization of function pointer HAL_I2C_AbortCpltCallback()

Interrupt mode or DMA mode IO sequential operation

Note: These interfaces allow to manage a sequential transfer with a repeated start condition when a direction change during transfer

- A specific option field manage the different steps of a sequential transfer

- Option field values are defined through I2C_XferOptions_definition and are listed below:
 - I2C_FIRST_AND_LAST_FRAME: No sequential usage, functional is same as associated interfaces in no sequential mode
 - I2C_FIRST_FRAME: Sequential usage, this option allow to manage a sequence with start condition, address and data to transfer without a final stop condition
 - I2C_FIRST_AND_NEXT_FRAME: Sequential usage (Master only), this option allow to manage a sequence with start condition, address and data to transfer without a final stop condition, an then permit a call the same master sequential interface several times (like HAL_I2C_Master_Seq_Transmit_IT() then HAL_I2C_Master_Seq_Transmit_IT() or HAL_I2C_Master_Seq_Transmit_DMA() then HAL_I2C_Master_Seq_Transmit_DMA())
 - I2C_NEXT_FRAME: Sequential usage, this option allow to manage a sequence with a restart condition, address and with new data to transfer if the direction change or manage only the new data to transfer if no direction change and without a final stop condition in both cases
 - I2C_LAST_FRAME: Sequential usage, this option allow to manage a sequence with a restart condition, address and with new data to transfer if the direction change or manage only the new data to transfer if no direction change and with a final stop condition in both cases
 - I2C_LAST_FRAME_NO_STOP: Sequential usage (Master only), this option allow to manage a restart condition after several call of the same master sequential interface several times (link with option I2C_FIRST_AND_NEXT_FRAME). Usage can, transfer several bytes one by one using HAL_I2C_Master_Seq_Transmit_IT(option I2C_FIRST_AND_NEXT_FRAME then I2C_NEXT_FRAME) or HAL_I2C_Master_Seq_Receive_IT(option I2C_FIRST_AND_NEXT_FRAME then I2C_NEXT_FRAME) or HAL_I2C_Master_Seq_Transmit_DMA(option I2C_FIRST_AND_NEXT_FRAME then I2C_NEXT_FRAME) or HAL_I2C_Master_Seq_Receive_DMA(option I2C_FIRST_AND_NEXT_FRAME then I2C_NEXT_FRAME). Then usage of this option I2C_LAST_FRAME_NO_STOP at the last Transmit or Receive sequence permit to call the opposite interface Receive or Transmit without stopping the communication and so generate a restart condition.
 - I2C_OTHER_FRAME: Sequential usage (Master only), this option allow to manage a restart condition after each call of the same master sequential interface. Usage can, transfer several bytes one by one with a restart with slave address between each bytes using HAL_I2C_Master_Seq_Transmit_IT(option I2C_FIRST_FRAME then I2C_OTHER_FRAME) or HAL_I2C_Master_Seq_Receive_IT(option I2C_FIRST_FRAME then I2C_OTHER_FRAME) or HAL_I2C_Master_Seq_Transmit_DMA(option I2C_FIRST_FRAME then I2C_OTHER_FRAME) or HAL_I2C_Master_Seq_Receive_DMA(option I2C_FIRST_FRAME then I2C_OTHER_FRAME). Then usage of this option I2C_OTHER_AND_LAST_FRAME at the last frame to help automatic generation of STOP condition.

- Different sequential I2C interfaces are listed below:
 - Sequential transmit in master I2C mode an amount of data in non-blocking mode using HAL_I2C_Master_Seq_Transmit_IT() or using HAL_I2C_Master_Seq_Transmit_DMA()
 - At transmission end of current frame transfer, HAL_I2C_MasterTxCallback() is executed and user can add his own code by customization of function pointer HAL_I2C_MasterTxCallback()
 - Sequential receive in master I2C mode an amount of data in non-blocking mode using HAL_I2C_Master_Seq_Receive_IT() or using HAL_I2C_Master_Seq_Receive_DMA()
 - At reception end of current frame transfer, HAL_I2C_MasterRxCallback() is executed and user can add his own code by customization of function pointer HAL_I2C_MasterRxCallback()
 - Abort a master IT or DMA I2C process communication with Interrupt using HAL_I2C_Master_Abort_IT()
 - End of abort process, HAL_I2C_AbortCallback() is executed and user can add his own code by customization of function pointer HAL_I2C_AbortCallback()
 - Enable/disable the Address listen mode in slave I2C mode using HAL_I2C_EnableListen_IT() HAL_I2C_DisableListen_IT()
 - When address slave I2C match, HAL_I2C_AddrCallback() is executed and user can add his own code to check the Address Match Code and the transmission direction request by master (Write/Read).
 - At Listen mode end HAL_I2C_ListenCallback() is executed and user can add his own code by customization of function pointer HAL_I2C_ListenCallback()
 - Sequential transmit in slave I2C mode an amount of data in non-blocking mode using HAL_I2C_Slave_Seq_Transmit_IT() or using HAL_I2C_Slave_Seq_Transmit_DMA()
 - At transmission end of current frame transfer, HAL_I2C_SlaveTxCallback() is executed and user can add his own code by customization of function pointer HAL_I2C_SlaveTxCallback()
 - Sequential receive in slave I2C mode an amount of data in non-blocking mode using HAL_I2C_Slave_Seq_Receive_IT() or using HAL_I2C_Slave_Seq_Receive_DMA()
 - At reception end of current frame transfer, HAL_I2C_SlaveRxCallback() is executed and user can add his own code by customization of function pointer HAL_I2C_SlaveRxCallback()
 - In case of transfer Error, HAL_I2C_ErrorCallback() function is executed and user can add his own code by customization of function pointer HAL_I2C_ErrorCallback()

Interrupt mode IO MEM operation

- Write an amount of data in non-blocking mode with Interrupt to a specific memory address using HAL_I2C_Mem_Write_IT()
- At Memory end of write transfer, HAL_I2C_MemTxCallback() is executed and user can add his own code by customization of function pointer HAL_I2C_MemTxCallback()
- Read an amount of data in non-blocking mode with Interrupt from a specific memory address using HAL_I2C_Mem_Read_IT()
- At Memory end of read transfer, HAL_I2C_MemRxCallback() is executed and user can add his own code by customization of function pointer HAL_I2C_MemRxCallback()
- In case of transfer Error, HAL_I2C_ErrorCallback() function is executed and user can add his own code by customization of function pointer HAL_I2C_ErrorCallback()

DMA mode IO operation

- Transmit in master mode an amount of data in non-blocking mode (DMA) using HAL_I2C_Master_Transmit_DMA()
- At transmission end of transfer, HAL_I2C_MasterTxCallback() is executed and user can add his own code by customization of function pointer HAL_I2C_MasterTxCallback()
- Receive in master mode an amount of data in non-blocking mode (DMA) using HAL_I2C_Master_Receive_DMA()

- At reception end of transfer, HAL_I2C_MasterRxCpltCallback() is executed and user can add his own code by customization of function pointer HAL_I2C_MasterRxCpltCallback()
- Transmit in slave mode an amount of data in non-blocking mode (DMA) using HAL_I2C_Slave_Transmit_DMA()
- At transmission end of transfer, HAL_I2C_SlaveTxCpltCallback() is executed and user can add his own code by customization of function pointer HAL_I2C_SlaveTxCpltCallback()
- Receive in slave mode an amount of data in non-blocking mode (DMA) using HAL_I2C_Slave_Receive_DMA()
- At reception end of transfer, HAL_I2C_SlaveRxCpltCallback() is executed and user can add his own code by customization of function pointer HAL_I2C_SlaveRxCpltCallback()
- In case of transfer Error, HAL_I2C_ErrorCallback() function is executed and user can add his own code by customization of function pointer HAL_I2C_ErrorCallback()
- Abort a master I2C process communication with Interrupt using HAL_I2C_Master_Abort_IT()
- End of abort process, HAL_I2C_AbortCpltCallback() is executed and user can add his own code by customization of function pointer HAL_I2C_AbortCpltCallback()

DMA mode IO MEM operation

- Write an amount of data in non-blocking mode with DMA to a specific memory address using HAL_I2C_Mem_Write_DMA()
- At Memory end of write transfer, HAL_I2C_MemTxCpltCallback() is executed and user can add his own code by customization of function pointer HAL_I2C_MemTxCpltCallback()
- Read an amount of data in non-blocking mode with DMA from a specific memory address using HAL_I2C_Mem_Read_DMA()
- At Memory end of read transfer, HAL_I2C_MemRxCpltCallback() is executed and user can add his own code by customization of function pointer HAL_I2C_MemRxCpltCallback()
- In case of transfer Error, HAL_I2C_ErrorCallback() function is executed and user can add his own code by customization of function pointer HAL_I2C_ErrorCallback()

I2C HAL driver macros list

Below the list of most used macros in I2C HAL driver.

- `__HAL_I2C_ENABLE`: Enable the I2C peripheral
- `__HAL_I2C_DISABLE`: Disable the I2C peripheral
- `__HAL_I2C_GET_FLAG`: Checks whether the specified I2C flag is set or not
- `__HAL_I2C_CLEAR_FLAG`: Clear the specified I2C pending flag
- `__HAL_I2C_ENABLE_IT`: Enable the specified I2C interrupt
- `__HAL_I2C_DISABLE_IT`: Disable the specified I2C interrupt

Callback registration

The compilation flag `USE_HAL_I2C_REGISTER_CALLBACKS` when set to 1 allows the user to configure dynamically the driver callbacks. Use Functions `HAL_I2C_RegisterCallback()` or `HAL_I2C_RegisterAddrCallback()` to register an interrupt callback.

Function `HAL_I2C_RegisterCallback()` allows to register following callbacks:

- `MasterTxCpltCallback` : callback for Master transmission end of transfer.
- `MasterRxCpltCallback` : callback for Master reception end of transfer.
- `SlaveTxCpltCallback` : callback for Slave transmission end of transfer.
- `SlaveRxCpltCallback` : callback for Slave reception end of transfer.
- `ListenCpltCallback` : callback for end of listen mode.
- `MemTxCpltCallback` : callback for Memory transmission end of transfer.
- `MemRxCpltCallback` : callback for Memory reception end of transfer.
- `ErrorCallback` : callback for error detection.
- `AbortCpltCallback` : callback for abort completion process.
- `MspInitCallback` : callback for Msp Init.

- **MspDeInitCallback** : callback for Msp DeInit. This function takes as parameters the HAL peripheral handle, the Callback ID and a pointer to the user callback function.

For specific callback AddrCallback use dedicated register callbacks : `HAL_I2C_RegisterAddrCallback()`.

Use function `HAL_I2C_UnRegisterCallback` to reset a callback to the default weak function.

`HAL_I2C_UnRegisterCallback` takes as parameters the HAL peripheral handle, and the Callback ID. This function allows to reset following callbacks:

- **MasterTxCpltCallback** : callback for Master transmission end of transfer.
- **MasterRxCpltCallback** : callback for Master reception end of transfer.
- **SlaveTxCpltCallback** : callback for Slave transmission end of transfer.
- **SlaveRxCpltCallback** : callback for Slave reception end of transfer.
- **ListenCpltCallback** : callback for end of listen mode.
- **MemTxCpltCallback** : callback for Memory transmission end of transfer.
- **MemRxCpltCallback** : callback for Memory reception end of transfer.
- **ErrorCallback** : callback for error detection.
- **AbortCpltCallback** : callback for abort completion process.
- **MspInitCallback** : callback for Msp Init.
- **MspDeInitCallback** : callback for Msp DeInit.

For callback AddrCallback use dedicated register callbacks : `HAL_I2C_UnRegisterAddrCallback()`.

By default, after the `HAL_I2C_Init()` and when the state is `HAL_I2C_STATE_RESET` all callbacks are set to the corresponding weak functions: examples `HAL_I2C_MasterTxCpltCallback()`, `HAL_I2C_MasterRxCpltCallback()`.

Exception done for `MspInit` and `MspDeInit` functions that are reset to the legacy weak functions in the `HAL_I2C_Init()`/`HAL_I2C_DeInit()` only when these callbacks are null (not registered beforehand). If `MspInit` or `MspDeInit` are not null, the `HAL_I2C_Init()`/`HAL_I2C_DeInit()` keep and use the user `MspInit`/`MspDeInit` callbacks (registered beforehand) whatever the state.

Callbacks can be registered/unregistered in `HAL_I2C_STATE_READY` state only. Exception done `MspInit`/`MspDeInit` functions that can be registered/unregistered in `HAL_I2C_STATE_READY` or `HAL_I2C_STATE_RESET` state, thus registered (user) `MspInit`/`DeInit` callbacks can be used during the `Init`/`DeInit`. Then, the user first registers the `MspInit`/`MspDeInit` user callbacks using `HAL_I2C_RegisterCallback()` before calling `HAL_I2C_DeInit()` or `HAL_I2C_Init()` function.

When the compilation flag `USE_HAL_I2C_REGISTER_CALLBACKS` is set to 0 or not defined, the callback registration feature is not available and all callbacks are set to the corresponding weak functions.

Note: You can refer to the I2C HAL driver header file for more useful macros

36.2.2 Initialization and de-initialization functions

This subsection provides a set of functions allowing to initialize and deinitialize the I2Cx peripheral:

- User must Implement `HAL_I2C_MspInit()` function in which he configures all related peripherals resources (CLOCK, GPIO, DMA, IT and NVIC).
- Call the function `HAL_I2C_Init()` to configure the selected device with the selected configuration:
 - Communication Speed
 - Duty cycle
 - Addressing mode
 - Own Address 1
 - Dual Addressing mode
 - Own Address 2
 - General call mode
 - Nostretch mode
- Call the function `HAL_I2C_DeInit()` to restore the default configuration of the selected I2Cx peripheral.

This section contains the following APIs:

- **`HAL_I2C_Init()`**
- **`HAL_I2C_DeInit()`**
- **`HAL_I2C_MspInit()`**
- **`HAL_I2C_MspDeInit()`**

- *HAL_I2C_RegisterCallback()*
- *HAL_I2C_UnRegisterCallback()*
- *HAL_I2C_RegisterAddrCallback()*
- *HAL_I2C_UnRegisterAddrCallback()*

36.2.3 IO operation functions

This subsection provides a set of functions allowing to manage the I2C data transfers.

1. There are two modes of transfer:
 - Blocking mode : The communication is performed in the polling mode. The status of all data processing is returned by the same function after finishing transfer.
 - No-Blocking mode : The communication is performed using Interrupts or DMA. These functions return the status of the transfer startup. The end of the data processing will be indicated through the dedicated I2C IRQ when using Interrupt mode or the DMA IRQ when using DMA mode.
2. Blocking mode functions are :
 - HAL_I2C_Master_Transmit()
 - HAL_I2C_Master_Receive()
 - HAL_I2C_Slave_Transmit()
 - HAL_I2C_Slave_Receive()
 - HAL_I2C_Mem_Write()
 - HAL_I2C_Mem_Read()
 - HAL_I2C_IsDeviceReady()
3. No-Blocking mode functions with Interrupt are :
 - HAL_I2C_Master_Transmit_IT()
 - HAL_I2C_Master_Receive_IT()
 - HAL_I2C_Slave_Transmit_IT()
 - HAL_I2C_Slave_Receive_IT()
 - HAL_I2C_Mem_Write_IT()
 - HAL_I2C_Mem_Read_IT()
 - HAL_I2C_Master_Seq_Transmit_IT()
 - HAL_I2C_Master_Seq_Receive_IT()
 - HAL_I2C_Slave_Seq_Transmit_IT()
 - HAL_I2C_Slave_Seq_Receive_IT()
 - HAL_I2C_EnableListen_IT()
 - HAL_I2C_DisableListen_IT()
 - HAL_I2C_Master_Abort_IT()
4. No-Blocking mode functions with DMA are :
 - HAL_I2C_Master_Transmit_DMA()
 - HAL_I2C_Master_Receive_DMA()
 - HAL_I2C_Slave_Transmit_DMA()
 - HAL_I2C_Slave_Receive_DMA()
 - HAL_I2C_Mem_Write_DMA()
 - HAL_I2C_Mem_Read_DMA()
 - HAL_I2C_Master_Seq_Transmit_DMA()
 - HAL_I2C_Master_Seq_Receive_DMA()
 - HAL_I2C_Slave_Seq_Transmit_DMA()
 - HAL_I2C_Slave_Seq_Receive_DMA()

5. A set of Transfer Complete Callbacks are provided in non Blocking mode:

- HAL_I2C_MasterTxCpltCallback()
- HAL_I2C_MasterRxCpltCallback()
- HAL_I2C_SlaveTxCpltCallback()
- HAL_I2C_SlaveRxCpltCallback()
- HAL_I2C_MemTxCpltCallback()
- HAL_I2C_MemRxCpltCallback()
- HAL_I2C_AddrCallback()
- HAL_I2C_ListenCpltCallback()
- HAL_I2C_ErrorCallback()
- HAL_I2C_AbortCpltCallback()

This section contains the following APIs:

- *HAL_I2C_Master_Transmit()*
- *HAL_I2C_Master_Receive()*
- *HAL_I2C_Slave_Transmit()*
- *HAL_I2C_Slave_Receive()*
- *HAL_I2C_Master_Transmit_IT()*
- *HAL_I2C_Master_Receive_IT()*
- *HAL_I2C_Slave_Transmit_IT()*
- *HAL_I2C_Slave_Receive_IT()*
- *HAL_I2C_Master_Transmit_DMA()*
- *HAL_I2C_Master_Receive_DMA()*
- *HAL_I2C_Slave_Transmit_DMA()*
- *HAL_I2C_Slave_Receive_DMA()*
- *HAL_I2C_Mem_Write()*
- *HAL_I2C_Mem_Read()*
- *HAL_I2C_Mem_Write_IT()*
- *HAL_I2C_Mem_Read_IT()*
- *HAL_I2C_Mem_Write_DMA()*
- *HAL_I2C_Mem_Read_DMA()*
- *HAL_I2C_IsDeviceReady()*
- *HAL_I2C_Master_Seq_Transmit_IT()*
- *HAL_I2C_Master_Seq_Transmit_DMA()*
- *HAL_I2C_Master_Seq_Receive_IT()*
- *HAL_I2C_Master_Seq_Receive_DMA()*
- *HAL_I2C_Slave_Seq_Transmit_IT()*
- *HAL_I2C_Slave_Seq_Transmit_DMA()*
- *HAL_I2C_Slave_Seq_Receive_IT()*
- *HAL_I2C_Slave_Seq_Receive_DMA()*
- *HAL_I2C_EnableListen_IT()*
- *HAL_I2C_DisableListen_IT()*
- *HAL_I2C_Master_Abort_IT()*

36.2.4 Peripheral State, Mode and Error functions

This subsection permit to get in run-time the status of the peripheral and the data flow.

This section contains the following APIs:

- *HAL_I2C_GetState()*
- *HAL_I2C_GetMode()*
- *HAL_I2C_GetError()*

36.2.5 Detailed description of functions

HAL_I2C_Init

Function name

HAL_StatusTypeDef HAL_I2C_Init (I2C_HandleTypeDef * hi2c)

Function description

Initializes the I2C according to the specified parameters in the I2C_InitTypeDef and initialize the associated handle.

Parameters

- **hi2c**: Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.

Return values

- **HAL**: status

HAL_I2C_DeInit

Function name

HAL_StatusTypeDef HAL_I2C_DeInit (I2C_HandleTypeDef * hi2c)

Function description

DeInitialize the I2C peripheral.

Parameters

- **hi2c**: Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.

Return values

- **HAL**: status

HAL_I2C_MspInit

Function name

void HAL_I2C_MspInit (I2C_HandleTypeDef * hi2c)

Function description

Initialize the I2C MSP.

Parameters

- **hi2c**: Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.

Return values

- **None**:

HAL_I2C_MspDeInit

Function name

void HAL_I2C_MspDeInit (I2C_HandleTypeDef * hi2c)

Function description

DeInitialize the I2C MSP.

Parameters

- **hi2c:** Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.

Return values

- **None:**

HAL_I2C_RegisterCallback

Function name

HAL_StatusTypeDef HAL_I2C_RegisterCallback (I2C_HandleTypeDef * hi2c, HAL_I2C_CallbackIDTypeDef CallbackID, pI2C_CallbackTypeDef pCallback)

Function description

Register a User I2C Callback To be used instead of the weak predefined callback.

Parameters

- **hi2c:** Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.
- **CallbackID:** ID of the callback to be registered This parameter can be one of the following values:
 - HAL_I2C_MASTER_TX_COMPLETE_CB_ID Master Tx Transfer completed callback ID
 - HAL_I2C_MASTER_RX_COMPLETE_CB_ID Master Rx Transfer completed callback ID
 - HAL_I2C_SLAVE_TX_COMPLETE_CB_ID Slave Tx Transfer completed callback ID
 - HAL_I2C_SLAVE_RX_COMPLETE_CB_ID Slave Rx Transfer completed callback ID
 - HAL_I2C_LISTEN_COMPLETE_CB_ID Listen Complete callback ID
 - HAL_I2C_MEM_TX_COMPLETE_CB_ID Memory Tx Transfer callback ID
 - HAL_I2C_MEM_RX_COMPLETE_CB_ID Memory Rx Transfer completed callback ID
 - HAL_I2C_ERROR_CB_ID Error callback ID
 - HAL_I2C_ABORT_CB_ID Abort callback ID
 - HAL_I2C_MSPINIT_CB_ID MspInit callback ID
 - HAL_I2C_MSPDEINIT_CB_ID MspDeInit callback ID
- **pCallback:** pointer to the Callback function

Return values

- **HAL:** status

HAL_I2C_UnRegisterCallback

Function name

HAL_StatusTypeDef HAL_I2C_UnRegisterCallback (I2C_HandleTypeDef * hi2c, HAL_I2C_CallbackIDTypeDef CallbackID)

Function description

Unregister an I2C Callback I2C callback is redirected to the weak predefined callback.

Parameters

- **hi2c:** Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.
- **CallbackID:** ID of the callback to be unregistered This parameter can be one of the following values: This parameter can be one of the following values:
 - HAL_I2C_MASTER_TX_COMPLETE_CB_ID Master Tx Transfer completed callback ID
 - HAL_I2C_MASTER_RX_COMPLETE_CB_ID Master Rx Transfer completed callback ID
 - HAL_I2C_SLAVE_TX_COMPLETE_CB_ID Slave Tx Transfer completed callback ID
 - HAL_I2C_SLAVE_RX_COMPLETE_CB_ID Slave Rx Transfer completed callback ID
 - HAL_I2C_LISTEN_COMPLETE_CB_ID Listen Complete callback ID
 - HAL_I2C_MEM_TX_COMPLETE_CB_ID Memory Tx Transfer callback ID
 - HAL_I2C_MEM_RX_COMPLETE_CB_ID Memory Rx Transfer completed callback ID
 - HAL_I2C_ERROR_CB_ID Error callback ID
 - HAL_I2C_ABORT_CB_ID Abort callback ID
 - HAL_I2C_MSPINIT_CB_ID MspInit callback ID
 - HAL_I2C_MSPDEINIT_CB_ID MspDeInit callback ID

Return values

- **HAL:** status

HAL_I2C_RegisterAddrCallback

Function name

HAL_StatusTypeDef HAL_I2C_RegisterAddrCallback (I2C_HandleTypeDef * hi2c, I2C_AddrCallbackTypeDef pCallback)

Function description

Register the Slave Address Match I2C Callback To be used instead of the weak HAL_I2C_AddrCallback() predefined callback.

Parameters

- **hi2c:** Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.
- **pCallback:** pointer to the Address Match Callback function

Return values

- **HAL:** status

HAL_I2C_UnRegisterAddrCallback

Function name

HAL_StatusTypeDef HAL_I2C_UnRegisterAddrCallback (I2C_HandleTypeDef * hi2c)

Function description

UnRegister the Slave Address Match I2C Callback Info Ready I2C Callback is redirected to the weak HAL_I2C_AddrCallback() predefined callback.

Parameters

- **hi2c:** Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.

Return values

- **HAL:** status

HAL_I2C_Master_Transmit

Function name

HAL_StatusTypeDef HAL_I2C_Master_Transmit (I2C_HandleTypeDef * hi2c, uint16_t DevAddress, uint8_t * pData, uint16_t Size, uint32_t Timeout)

Function description

Transmits in master mode an amount of data in blocking mode.

Parameters

- **hi2c**: Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.
- **DevAddress**: Target device address: The device 7 bits address value in datasheet must be shifted to the left before calling the interface
- **pData**: Pointer to data buffer
- **Size**: Amount of data to be sent
- **Timeout**: Timeout duration

Return values

- **HAL**: status

HAL_I2C_Master_Receive

Function name

HAL_StatusTypeDef HAL_I2C_Master_Receive (I2C_HandleTypeDef * hi2c, uint16_t DevAddress, uint8_t * pData, uint16_t Size, uint32_t Timeout)

Function description

Receives in master mode an amount of data in blocking mode.

Parameters

- **hi2c**: Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.
- **DevAddress**: Target device address: The device 7 bits address value in datasheet must be shifted to the left before calling the interface
- **pData**: Pointer to data buffer
- **Size**: Amount of data to be sent
- **Timeout**: Timeout duration

Return values

- **HAL**: status

HAL_I2C_Slave_Transmit

Function name

HAL_StatusTypeDef HAL_I2C_Slave_Transmit (I2C_HandleTypeDef * hi2c, uint8_t * pData, uint16_t Size, uint32_t Timeout)

Function description

Transmits in slave mode an amount of data in blocking mode.

Parameters

- **hi2c:** Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.
- **pData:** Pointer to data buffer
- **Size:** Amount of data to be sent
- **Timeout:** Timeout duration

Return values

- **HAL:** status

HAL_I2C_Slave_Receive

Function name

HAL_StatusTypeDef HAL_I2C_Slave_Receive (I2C_HandleTypeDef * hi2c, uint8_t * pData, uint16_t Size, uint32_t Timeout)

Function description

Receive in slave mode an amount of data in blocking mode.

Parameters

- **hi2c:** Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.
- **pData:** Pointer to data buffer
- **Size:** Amount of data to be sent
- **Timeout:** Timeout duration

Return values

- **HAL:** status

HAL_I2C_Mem_Write

Function name

HAL_StatusTypeDef HAL_I2C_Mem_Write (I2C_HandleTypeDef * hi2c, uint16_t DevAddress, uint16_t MemAddress, uint16_t MemAddSize, uint8_t * pData, uint16_t Size, uint32_t Timeout)

Function description

Write an amount of data in blocking mode to a specific memory address.

Parameters

- **hi2c:** Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.
- **DevAddress:** Target device address: The device 7 bits address value in datasheet must be shifted to the left before calling the interface
- **MemAddress:** Internal memory address
- **MemAddSize:** Size of internal memory address
- **pData:** Pointer to data buffer
- **Size:** Amount of data to be sent
- **Timeout:** Timeout duration

Return values

- **HAL:** status

HAL_I2C_Mem_Read

Function name

HAL_StatusTypeDef HAL_I2C_Mem_Read (I2C_HandleTypeDef * hi2c, uint16_t DevAddress, uint16_t MemAddress, uint16_t MemAddSize, uint8_t * pData, uint16_t Size, uint32_t Timeout)

Function description

Read an amount of data in blocking mode from a specific memory address.

Parameters

- **hi2c**: Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.
- **DevAddress**: Target device address: The device 7 bits address value in datasheet must be shifted to the left before calling the interface
- **MemAddress**: Internal memory address
- **MemAddSize**: Size of internal memory address
- **pData**: Pointer to data buffer
- **Size**: Amount of data to be sent
- **Timeout**: Timeout duration

Return values

- **HAL**: status

HAL_I2C_IsDeviceReady

Function name

HAL_StatusTypeDef HAL_I2C_IsDeviceReady (I2C_HandleTypeDef * hi2c, uint16_t DevAddress, uint32_t Trials, uint32_t Timeout)

Function description

Checks if target device is ready for communication.

Parameters

- **hi2c**: Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.
- **DevAddress**: Target device address: The device 7 bits address value in datasheet must be shifted to the left before calling the interface
- **Trials**: Number of trials
- **Timeout**: Timeout duration

Return values

- **HAL**: status

Notes

- This function is used with Memory devices

HAL_I2C_Master_Transmit_IT

Function name

HAL_StatusTypeDef HAL_I2C_Master_Transmit_IT (I2C_HandleTypeDef * hi2c, uint16_t DevAddress, uint8_t * pData, uint16_t Size)

Function description

Transmit in master mode an amount of data in non-blocking mode with Interrupt.

Parameters

- **hi2c**: Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.
- **DevAddress**: Target device address: The device 7 bits address value in datasheet must be shifted to the left before calling the interface
- **pData**: Pointer to data buffer
- **Size**: Amount of data to be sent

Return values

- **HAL**: status

HAL_I2C_Master_Receive_IT

Function name

HAL_StatusTypeDef HAL_I2C_Master_Receive_IT (I2C_HandleTypeDef * hi2c, uint16_t DevAddress, uint8_t * pData, uint16_t Size)

Function description

Receive in master mode an amount of data in non-blocking mode with Interrupt.

Parameters

- **hi2c**: Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.
- **DevAddress**: Target device address: The device 7 bits address value in datasheet must be shifted to the left before calling the interface
- **pData**: Pointer to data buffer
- **Size**: Amount of data to be sent

Return values

- **HAL**: status

HAL_I2C_Slave_Transmit_IT

Function name

HAL_StatusTypeDef HAL_I2C_Slave_Transmit_IT (I2C_HandleTypeDef * hi2c, uint8_t * pData, uint16_t Size)

Function description

Transmit in slave mode an amount of data in non-blocking mode with Interrupt.

Parameters

- **hi2c**: Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.
- **pData**: Pointer to data buffer
- **Size**: Amount of data to be sent

Return values

- **HAL**: status

HAL_I2C_Slave_Receive_IT

Function name

HAL_StatusTypeDef HAL_I2C_Slave_Receive_IT (I2C_HandleTypeDef * hi2c, uint8_t * pData, uint16_t Size)

Function description

Receive in slave mode an amount of data in non-blocking mode with Interrupt.

Parameters

- **hi2c:** Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.
- **pData:** Pointer to data buffer
- **Size:** Amount of data to be sent

Return values

- **HAL:** status

HAL_I2C_Mem_Write_IT

Function name

HAL_StatusTypeDef HAL_I2C_Mem_Write_IT (I2C_HandleTypeDef * hi2c, uint16_t DevAddress, uint16_t MemAddress, uint16_t MemAddSize, uint8_t * pData, uint16_t Size)

Function description

Write an amount of data in non-blocking mode with Interrupt to a specific memory address.

Parameters

- **hi2c:** Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.
- **DevAddress:** Target device address: The device 7 bits address value in datasheet must be shifted to the left before calling the interface
- **MemAddress:** Internal memory address
- **MemAddSize:** Size of internal memory address
- **pData:** Pointer to data buffer
- **Size:** Amount of data to be sent

Return values

- **HAL:** status

HAL_I2C_Mem_Read_IT

Function name

HAL_StatusTypeDef HAL_I2C_Mem_Read_IT (I2C_HandleTypeDef * hi2c, uint16_t DevAddress, uint16_t MemAddress, uint16_t MemAddSize, uint8_t * pData, uint16_t Size)

Function description

Read an amount of data in non-blocking mode with Interrupt from a specific memory address.

Parameters

- **hi2c:** Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.
- **DevAddress:** Target device address
- **MemAddress:** Internal memory address
- **MemAddSize:** Size of internal memory address
- **pData:** Pointer to data buffer
- **Size:** Amount of data to be sent

Return values

- **HAL:** status

HAL_I2C_Master_Seq_Transmit_IT

Function name

HAL_StatusTypeDef HAL_I2C_Master_Seq_Transmit_IT (I2C_HandleTypeDef * hi2c, uint16_t DevAddress, uint8_t * pData, uint16_t Size, uint32_t XferOptions)

Function description

Sequential transmit in master I2C mode an amount of data in non-blocking mode with Interrupt.

Parameters

- **hi2c**: Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.
- **DevAddress**: Target device address: The device 7 bits address value in datasheet must be shifted to the left before calling the interface
- **pData**: Pointer to data buffer
- **Size**: Amount of data to be sent
- **XferOptions**: Options of Transfer, value of I2C XferOptions definition

Return values

- **HAL**: status

Notes

- This interface allow to manage repeated start condition when a direction change during transfer

HAL_I2C_Master_Seq_Receive_IT

Function name

HAL_StatusTypeDef HAL_I2C_Master_Seq_Receive_IT (I2C_HandleTypeDef * hi2c, uint16_t DevAddress, uint8_t * pData, uint16_t Size, uint32_t XferOptions)

Function description

Sequential receive in master I2C mode an amount of data in non-blocking mode with Interrupt.

Parameters

- **hi2c**: Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.
- **DevAddress**: Target device address: The device 7 bits address value in datasheet must be shifted to the left before calling the interface
- **pData**: Pointer to data buffer
- **Size**: Amount of data to be sent
- **XferOptions**: Options of Transfer, value of I2C XferOptions definition

Return values

- **HAL**: status

Notes

- This interface allow to manage repeated start condition when a direction change during transfer

HAL_I2C_Slave_Seq_Transmit_IT

Function name

HAL_StatusTypeDef HAL_I2C_Slave_Seq_Transmit_IT (I2C_HandleTypeDef * hi2c, uint8_t * pData, uint16_t Size, uint32_t XferOptions)

Function description

Sequential transmit in slave mode an amount of data in non-blocking mode with Interrupt.

Parameters

- **hi2c:** Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.
- **pData:** Pointer to data buffer
- **Size:** Amount of data to be sent
- **XferOptions:** Options of Transfer, value of I2C XferOptions definition

Return values

- **HAL:** status

Notes

- This interface allow to manage repeated start condition when a direction change during transfer

HAL_I2C_Slave_Seq_Receive_IT

Function name

HAL_StatusTypeDef HAL_I2C_Slave_Seq_Receive_IT (I2C_HandleTypeDef * hi2c, uint8_t * pData, uint16_t Size, uint32_t XferOptions)

Function description

Sequential receive in slave mode an amount of data in non-blocking mode with Interrupt.

Parameters

- **hi2c:** Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.
- **pData:** Pointer to data buffer
- **Size:** Amount of data to be sent
- **XferOptions:** Options of Transfer, value of I2C XferOptions definition

Return values

- **HAL:** status

Notes

- This interface allow to manage repeated start condition when a direction change during transfer

HAL_I2C_EnableListen_IT

Function name

HAL_StatusTypeDef HAL_I2C_EnableListen_IT (I2C_HandleTypeDef * hi2c)

Function description

Enable the Address listen mode with Interrupt.

Parameters

- **hi2c:** Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.

Return values

- **HAL:** status

HAL_I2C_DisableListen_IT

Function name

HAL_StatusTypeDef HAL_I2C_DisableListen_IT (I2C_HandleTypeDef * hi2c)

Function description

Disable the Address listen mode with Interrupt.

Parameters

- **hi2c:** Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.

Return values

- **HAL:** status

HAL_I2C_Master_Abort_IT

Function name

HAL_StatusTypeDef HAL_I2C_Master_Abort_IT (I2C_HandleTypeDef * hi2c, uint16_t DevAddress)

Function description

Abort a master I2C IT or DMA process communication with Interrupt.

Parameters

- **hi2c:** Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.
- **DevAddress:** Target device address: The device 7 bits address value in datasheet must be shifted to the left before calling the interface

Return values

- **HAL:** status

HAL_I2C_Master_Transmit_DMA

Function name

HAL_StatusTypeDef HAL_I2C_Master_Transmit_DMA (I2C_HandleTypeDef * hi2c, uint16_t DevAddress, uint8_t * pData, uint16_t Size)

Function description

Transmit in master mode an amount of data in non-blocking mode with DMA.

Parameters

- **hi2c:** Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.
- **DevAddress:** Target device address: The device 7 bits address value in datasheet must be shifted to the left before calling the interface
- **pData:** Pointer to data buffer
- **Size:** Amount of data to be sent

Return values

- **HAL:** status

HAL_I2C_Master_Receive_DMA

Function name

HAL_StatusTypeDef HAL_I2C_Master_Receive_DMA (I2C_HandleTypeDef * hi2c, uint16_t DevAddress, uint8_t * pData, uint16_t Size)

Function description

Receive in master mode an amount of data in non-blocking mode with DMA.

Parameters

- **hi2c:** Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.
- **DevAddress:** Target device address: The device 7 bits address value in datasheet must be shifted to the left before calling the interface
- **pData:** Pointer to data buffer
- **Size:** Amount of data to be sent

Return values

- **HAL:** status

HAL_I2C_Slave_Transmit_DMA

Function name

HAL_StatusTypeDef HAL_I2C_Slave_Transmit_DMA (I2C_HandleTypeDef * hi2c, uint8_t * pData, uint16_t Size)

Function description

Transmit in slave mode an amount of data in non-blocking mode with DMA.

Parameters

- **hi2c:** Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.
- **pData:** Pointer to data buffer
- **Size:** Amount of data to be sent

Return values

- **HAL:** status

HAL_I2C_Slave_Receive_DMA

Function name

HAL_StatusTypeDef HAL_I2C_Slave_Receive_DMA (I2C_HandleTypeDef * hi2c, uint8_t * pData, uint16_t Size)

Function description

Receive in slave mode an amount of data in non-blocking mode with DMA.

Parameters

- **hi2c:** Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.
- **pData:** Pointer to data buffer
- **Size:** Amount of data to be sent

Return values

- **HAL:** status

HAL_I2C_Mem_Write_DMA

Function name

HAL_StatusTypeDef HAL_I2C_Mem_Write_DMA (I2C_HandleTypeDef * hi2c, uint16_t DevAddress, uint16_t MemAddress, uint16_t MemAddSize, uint8_t * pData, uint16_t Size)

Function description

Write an amount of data in non-blocking mode with DMA to a specific memory address.

Parameters

- **hi2c**: Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.
- **DevAddress**: Target device address: The device 7 bits address value in datasheet must be shifted to the left before calling the interface
- **MemAddress**: Internal memory address
- **MemAddSize**: Size of internal memory address
- **pData**: Pointer to data buffer
- **Size**: Amount of data to be sent

Return values

- **HAL**: status

HAL_I2C_Mem_Read_DMA

Function name

HAL_StatusTypeDef HAL_I2C_Mem_Read_DMA (I2C_HandleTypeDef * hi2c, uint16_t DevAddress, uint16_t MemAddress, uint16_t MemAddSize, uint8_t * pData, uint16_t Size)

Function description

Reads an amount of data in non-blocking mode with DMA from a specific memory address.

Parameters

- **hi2c**: Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.
- **DevAddress**: Target device address: The device 7 bits address value in datasheet must be shifted to the left before calling the interface
- **MemAddress**: Internal memory address
- **MemAddSize**: Size of internal memory address
- **pData**: Pointer to data buffer
- **Size**: Amount of data to be read

Return values

- **HAL**: status

HAL_I2C_Master_Seq_Transmit_DMA

Function name

HAL_StatusTypeDef HAL_I2C_Master_Seq_Transmit_DMA (I2C_HandleTypeDef * hi2c, uint16_t DevAddress, uint8_t * pData, uint16_t Size, uint32_t XferOptions)

Function description

Sequential transmit in master I2C mode an amount of data in non-blocking mode with DMA.

Parameters

- **hi2c**: Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.
- **DevAddress**: Target device address: The device 7 bits address value in datasheet must be shifted to the left before calling the interface
- **pData**: Pointer to data buffer
- **Size**: Amount of data to be sent
- **XferOptions**: Options of Transfer, value of I2C XferOptions definition

Return values

- **HAL**: status

Notes

- This interface allow to manage repeated start condition when a direction change during transfer

HAL_I2C_Master_Seq_Receive_DMA

Function name

```
HAL_StatusTypeDef HAL_I2C_Master_Seq_Receive_DMA (I2C_HandleTypeDef * hi2c, uint16_t
DevAddress, uint8_t * pData, uint16_t Size, uint32_t XferOptions)
```

Function description

Sequential receive in master mode an amount of data in non-blocking mode with DMA.

Parameters

- **hi2c**: Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.
- **DevAddress**: Target device address: The device 7 bits address value in datasheet must be shifted to the left before calling the interface
- **pData**: Pointer to data buffer
- **Size**: Amount of data to be sent
- **XferOptions**: Options of Transfer, value of I2C XferOptions definition

Return values

- **HAL**: status

Notes

- This interface allow to manage repeated start condition when a direction change during transfer

HAL_I2C_Slave_Seq_Transmit_DMA

Function name

```
HAL_StatusTypeDef HAL_I2C_Slave_Seq_Transmit_DMA (I2C_HandleTypeDef * hi2c, uint8_t * pData,
uint16_t Size, uint32_t XferOptions)
```

Function description

Sequential transmit in slave mode an amount of data in non-blocking mode with DMA.

Parameters

- **hi2c**: Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.
- **pData**: Pointer to data buffer
- **Size**: Amount of data to be sent
- **XferOptions**: Options of Transfer, value of I2C XferOptions definition

Return values

- **HAL**: status

Notes

- This interface allow to manage repeated start condition when a direction change during transfer

HAL_I2C_Slave_Seq_Receive_DMA

Function name

```
HAL_StatusTypeDef HAL_I2C_Slave_Seq_Receive_DMA (I2C_HandleTypeDef * hi2c, uint8_t * pData,
uint16_t Size, uint32_t XferOptions)
```

Function description

Sequential receive in slave mode an amount of data in non-blocking mode with DMA.

Parameters

- **hi2c:** Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.
- **pData:** Pointer to data buffer
- **Size:** Amount of data to be sent
- **XferOptions:** Options of Transfer, value of I2C XferOptions definition

Return values

- **HAL:** status

Notes

- This interface allow to manage repeated start condition when a direction change during transfer

HAL_I2C_EV_IRQHandler

Function name

```
void HAL_I2C_EV_IRQHandler (I2C_HandleTypeDef * hi2c)
```

Function description

This function handles I2C event interrupt request.

Parameters

- **hi2c:** Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.

Return values

- **None:**

HAL_I2C_ER_IRQHandler

Function name

```
void HAL_I2C_ER_IRQHandler (I2C_HandleTypeDef * hi2c)
```

Function description

This function handles I2C error interrupt request.

Parameters

- **hi2c:** Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.

Return values

- **None:**

HAL_I2C_MasterTxCpltCallback

Function name

```
void HAL_I2C_MasterTxCpltCallback (I2C_HandleTypeDef * hi2c)
```

Function description

Master Tx Transfer completed callback.

Parameters

- **hi2c:** Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.

Return values

- **None:**

HAL_I2C_MasterRxCpltCallback

Function name

```
void HAL_I2C_MasterRxCpltCallback (I2C_HandleTypeDef * hi2c)
```

Function description

Master Rx Transfer completed callback.

Parameters

- **hi2c:** Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.

Return values

- **None:**

HAL_I2C_SlaveTxCpltCallback

Function name

```
void HAL_I2C_SlaveTxCpltCallback (I2C_HandleTypeDef * hi2c)
```

Function description

Slave Tx Transfer completed callback.

Parameters

- **hi2c:** Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.

Return values

- **None:**

HAL_I2C_SlaveRxCpltCallback

Function name

```
void HAL_I2C_SlaveRxCpltCallback (I2C_HandleTypeDef * hi2c)
```

Function description

Slave Rx Transfer completed callback.

Parameters

- **hi2c:** Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.

Return values

- **None:**

HAL_I2C_AddrCallback

Function name

```
void HAL_I2C_AddrCallback (I2C_HandleTypeDef * hi2c, uint8_t TransferDirection, uint16_t AddrMatchCode)
```

Function description

Slave Address Match callback.

Parameters

- **hi2c:** Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.
- **TransferDirection:** Master request Transfer Direction (Write/Read), value of I2C XferDirection definition
- **AddrMatchCode:** Address Match Code

Return values

- **None:**

HAL_I2C_ListenCpltCallback

Function name

```
void HAL_I2C_ListenCpltCallback (I2C_HandleTypeDef * hi2c)
```

Function description

Listen Complete callback.

Parameters

- **hi2c:** Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.

Return values

- **None:**

HAL_I2C_MemTxCpltCallback

Function name

```
void HAL_I2C_MemTxCpltCallback (I2C_HandleTypeDef * hi2c)
```

Function description

Memory Tx Transfer completed callback.

Parameters

- **hi2c:** Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.

Return values

- **None:**

HAL_I2C_MemRxCpltCallback

Function name

```
void HAL_I2C_MemRxCpltCallback (I2C_HandleTypeDef * hi2c)
```

Function description

Memory Rx Transfer completed callback.

Parameters

- **hi2c:** Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.

Return values

- **None:**

HAL_I2C_ErrorCallback

Function name

```
void HAL_I2C_ErrorCallback (I2C_HandleTypeDef * hi2c)
```

Function description

I2C error callback.

Parameters

- **hi2c:** Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.

Return values

- **None:**

HAL_I2C_AbortCpltCallback

Function name

void HAL_I2C_AbortCpltCallback (I2C_HandleTypeDef * hi2c)

Function description

I2C abort callback.

Parameters

- **hi2c:** Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.

Return values

- **None:**

HAL_I2C_GetState

Function name

HAL_I2C_StateTypeDef HAL_I2C_GetState (I2C_HandleTypeDef * hi2c)

Function description

Return the I2C handle state.

Parameters

- **hi2c:** Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.

Return values

- **HAL:** state

HAL_I2C_GetMode

Function name

HAL_I2C_ModeTypeDef HAL_I2C_GetMode (I2C_HandleTypeDef * hi2c)

Function description

Returns the I2C Master, Slave, Memory or no mode.

Parameters

- **hi2c:** Pointer to a I2C_HandleTypeDef structure that contains the configuration information for I2C module

Return values

- **HAL:** mode

HAL_I2C_GetError

Function name

uint32_t HAL_I2C_GetError (I2C_HandleTypeDef * hi2c)

Function description

Return the I2C error code.

Parameters

- **hi2c:** Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.

Return values

- **I2C:** Error Code

36.3 I2C Firmware driver defines

The following section lists the various define and macros of the module.

36.3.1 I2C

I2C

I2C addressing mode

I2C_ADDRESSINGMODE_7BIT

I2C_ADDRESSINGMODE_10BIT

I2C dual addressing mode

I2C_DUALADDRESS_DISABLE

I2C_DUALADDRESS_ENABLE

I2C duty cycle in fast mode

I2C_DUTYCYCLE_2

I2C_DUTYCYCLE_16_9

I2C Error Code definition

HAL_I2C_ERROR_NONE

No error

HAL_I2C_ERROR_BERR

BERR error

HAL_I2C_ERROR_ARLO

ARLO error

HAL_I2C_ERROR_AF

AF error

HAL_I2C_ERROR_OVR

OVR error

HAL_I2C_ERROR_DMA

DMA transfer error

HAL_I2C_ERROR_TIMEOUT

Timeout Error

HAL_I2C_ERROR_SIZE

Size Management error

HAL_I2C_ERROR_DMA_PARAM

DMA Parameter Error

HAL_I2C_WRONG_START

Wrong start Error

HAL_I2C_ERROR_INVALID_CALLBACK

Invalid Callback error

I2C Exported Macros

__HAL_I2C_RESET_HANDLE_STATE

Description:

- Reset I2C handle state.

Parameters:

- `__HANDLE__`: specifies the I2C Handle.

Return value:

- None

__HAL_I2C_ENABLE_IT

Description:

- Enable or disable the specified I2C interrupts.

Parameters:

- `__HANDLE__`: specifies the I2C Handle.
- `__INTERRUPT__`: specifies the interrupt source to enable or disable. This parameter can be one of the following values:
 - `I2C_IT_BUF`: Buffer interrupt enable
 - `I2C_IT_EVT`: Event interrupt enable
 - `I2C_IT_ERR`: Error interrupt enable

Return value:

- None

__HAL_I2C_DISABLE_IT

__HAL_I2C_GET_IT_SOURCE

Description:

- Checks if the specified I2C interrupt source is enabled or disabled.

Parameters:

- `__HANDLE__`: specifies the I2C Handle.
- `__INTERRUPT__`: specifies the I2C interrupt source to check. This parameter can be one of the following values:
 - `I2C_IT_BUF`: Buffer interrupt enable
 - `I2C_IT_EVT`: Event interrupt enable
 - `I2C_IT_ERR`: Error interrupt enable

Return value:

- The: new state of `__INTERRUPT__` (TRUE or FALSE).

__HAL_I2C_GET_FLAG

Description:

- Checks whether the specified I2C flag is set or not.

Parameters:

- `__HANDLE__`: specifies the I2C Handle.
- `__FLAG__`: specifies the flag to check. This parameter can be one of the following values:
 - `I2C_FLAG_OVR`: Overrun/Underrun flag
 - `I2C_FLAG_AF`: Acknowledge failure flag
 - `I2C_FLAG_ARLO`: Arbitration lost flag
 - `I2C_FLAG_BERR`: Bus error flag
 - `I2C_FLAG_TXE`: Data register empty flag
 - `I2C_FLAG_RXNE`: Data register not empty flag
 - `I2C_FLAG_STOPF`: Stop detection flag
 - `I2C_FLAG_ADD10`: 10-bit header sent flag
 - `I2C_FLAG_BTF`: Byte transfer finished flag
 - `I2C_FLAG_ADDR`: Address sent flag Address matched flag
 - `I2C_FLAG_SB`: Start bit flag
 - `I2C_FLAG_DUALF`: Dual flag
 - `I2C_FLAG_GENCALL`: General call header flag
 - `I2C_FLAG_TRA`: Transmitter/Receiver flag
 - `I2C_FLAG_BUSY`: Bus busy flag
 - `I2C_FLAG_MSL`: Master/Slave flag

Return value:

- The: new state of `__FLAG__` (TRUE or FALSE).

__HAL_I2C_CLEAR_FLAG

Description:

- Clears the I2C pending flags which are cleared by writing 0 in a specific bit.

Parameters:

- `__HANDLE__`: specifies the I2C Handle.
- `__FLAG__`: specifies the flag to clear. This parameter can be any combination of the following values:
 - `I2C_FLAG_OVR`: Overrun/Underrun flag (Slave mode)
 - `I2C_FLAG_AF`: Acknowledge failure flag
 - `I2C_FLAG_ARLO`: Arbitration lost flag (Master mode)
 - `I2C_FLAG_BERR`: Bus error flag

Return value:

- None

__HAL_I2C_CLEAR_ADDRFLAG

Description:

- Clears the I2C ADDR pending flag.

Parameters:

- `__HANDLE__`: specifies the I2C Handle. This parameter can be I2C where x: 1, 2, or 3 to select the I2C peripheral.

Return value:

- None

__HAL_I2C_CLEAR_STOPFLAG**Description:**

- Clears the I2C STOPF pending flag.

Parameters:

- `__HANDLE__`: specifies the I2C Handle.

Return value:

- None

__HAL_I2C_ENABLE**Description:**

- Enable the specified I2C peripheral.

Parameters:

- `__HANDLE__`: specifies the I2C Handle.

Return value:

- None

__HAL_I2C_DISABLE**Description:**

- Disable the specified I2C peripheral.

Parameters:

- `__HANDLE__`: specifies the I2C Handle.

Return value:

- None

I2C Flag definition**I2C_FLAG_OVR****I2C_FLAG_AF****I2C_FLAG_ARLO****I2C_FLAG_BERR****I2C_FLAG_TXE****I2C_FLAG_RXNE****I2C_FLAG_STOPF****I2C_FLAG_ADD10****I2C_FLAG_BTF****I2C_FLAG_ADDR****I2C_FLAG_SB****I2C_FLAG_DUALF****I2C_FLAG_GENCALL****I2C_FLAG_TRA**

I2C_FLAG_BUSY

I2C_FLAG_MSL

I2C general call addressing mode

I2C_GENERALCALL_DISABLE

I2C_GENERALCALL_ENABLE

I2C Interrupt configuration definition

I2C_IT_BUF

I2C_IT_EVT

I2C_IT_ERR

I2C Private macros to check input parameters

IS_I2C_DUTY_CYCLE

IS_I2C_ADDRESSING_MODE

IS_I2C_DUAL_ADDRESS

IS_I2C_GENERAL_CALL

IS_I2C_NO_STRETCH

IS_I2C_MEMADD_SIZE

IS_I2C_CLOCK_SPEED

IS_I2C_OWN_ADDRESS1

IS_I2C_OWN_ADDRESS2

IS_I2C_TRANSFER_OPTIONS_REQUEST

IS_I2C_TRANSFER_OTHER_OPTIONS_REQUEST

I2C_CHECK_FLAG

I2C_CHECK_IT_SOURCE

I2C Memory Address Size

I2C_MEMADD_SIZE_8BIT

I2C_MEMADD_SIZE_16BIT

I2C nostretch mode

I2C_NOSTRETCH_DISABLE

I2C_NOSTRETCH_ENABLE

I2C XferDirection definition

I2C_DIRECTION_RECEIVE

I2C_DIRECTION_TRANSMIT

I2C XferOptions definition

I2C_FIRST_FRAME

I2C_FIRST_AND_NEXT_FRAME

I2C_NEXT_FRAME

I2C_FIRST_AND_LAST_FRAME

I2C_LAST_FRAME_NO_STOP

I2C_LAST_FRAME

I2C_OTHER_FRAME

I2C_OTHER_AND_LAST_FRAME

37 HAL I2C Extension Driver

37.1 I2CEx Firmware driver API description

The following section lists the various functions of the I2CEx library.

37.1.1 I2C peripheral extension features

Comparing to other previous devices, the I2C interface for STM32F427xx/437xx/ 429xx/439xx devices contains the following additional features :

- Possibility to disable or enable Analog Noise Filter
- Use of a configured Digital Noise Filter

37.1.2 How to use this driver

This driver provides functions to configure Noise Filter

1. Configure I2C Analog noise filter using the function `HAL_I2C_AnalogFilter_Config()`
2. Configure I2C Digital noise filter using the function `HAL_I2C_DigitalFilter_Config()`

37.1.3 Extension features functions

This section provides functions allowing to:

- Configure Noise Filters

This section contains the following APIs:

- [*HAL_I2CEx_ConfigAnalogFilter\(\)*](#)
- [*HAL_I2CEx_ConfigDigitalFilter\(\)*](#)

37.1.4 Detailed description of functions

HAL_I2CEx_ConfigAnalogFilter

Function name

HAL_StatusTypeDef HAL_I2CEx_ConfigAnalogFilter (I2C_HandleTypeDef * hi2c, uint32_t AnalogFilter)

Function description

Configures I2C Analog noise filter.

Parameters

- **hi2c**: pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2Cx peripheral.
- **AnalogFilter**: new state of the Analog filter.

Return values

- **HAL**: status

HAL_I2CEx_ConfigDigitalFilter

Function name

HAL_StatusTypeDef HAL_I2CEx_ConfigDigitalFilter (I2C_HandleTypeDef * hi2c, uint32_t DigitalFilter)

Function description

Configures I2C Digital noise filter.

Parameters

- **hi2c**: pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2Cx peripheral.
- **DigitalFilter**: Coefficient of digital noise filter between 0x00 and 0x0F.

Return values

- **HAL**: status

37.2 I2CEX Firmware driver defines

The following section lists the various define and macros of the module.

37.2.1 I2CEX

I2CEX

I2C Analog Filter

I2C_ANALOGFILTER_ENABLE

I2C_ANALOGFILTER_DISABLE

38 HAL I2S Generic Driver

38.1 I2S Firmware driver registers structures

38.1.1 I2S_InitTypeDef

I2S_InitTypeDef is defined in the stm32f4xx_hal_i2s.h

Data Fields

- *uint32_t Mode*
- *uint32_t Standard*
- *uint32_t DataFormat*
- *uint32_t MCLKOutput*
- *uint32_t AudioFreq*
- *uint32_t CPOL*
- *uint32_t ClockSource*
- *uint32_t FullDuplexMode*

Field Documentation

- *uint32_t I2S_InitTypeDef::Mode*
Specifies the I2S operating mode. This parameter can be a value of *I2S_Mode*
- *uint32_t I2S_InitTypeDef::Standard*
Specifies the standard used for the I2S communication. This parameter can be a value of *I2S_Standard*
- *uint32_t I2S_InitTypeDef::DataFormat*
Specifies the data format for the I2S communication. This parameter can be a value of *I2S_Data_Format*
- *uint32_t I2S_InitTypeDef::MCLKOutput*
Specifies whether the I2S MCLK output is enabled or not. This parameter can be a value of *I2S_MCLK_Output*
- *uint32_t I2S_InitTypeDef::AudioFreq*
Specifies the frequency selected for the I2S communication. This parameter can be a value of *I2S_Audio_Frequency*
- *uint32_t I2S_InitTypeDef::CPOL*
Specifies the idle state of the I2S clock. This parameter can be a value of *I2S_Clock_Polarity*
- *uint32_t I2S_InitTypeDef::ClockSource*
Specifies the I2S Clock Source. This parameter can be a value of *I2S_Clock_Source*
- *uint32_t I2S_InitTypeDef::FullDuplexMode*
Specifies the I2S FullDuplex mode. This parameter can be a value of *I2S_FullDuplex_Mode*

38.1.2 __I2S_HandleTypeDef

__I2S_HandleTypeDef is defined in the stm32f4xx_hal_i2s.h

Data Fields

- *SPI_TypeDef * Instance*
- *I2S_InitTypeDef Init*
- *uint16_t * pTxBuffPtr*
- *__IO uint16_t TxXferSize*
- *__IO uint16_t TxXferCount*
- *uint16_t * pRxBuffPtr*
- *__IO uint16_t RxXferSize*
- *__IO uint16_t RxXferCount*
- *void(* IrqHandlerISR*
- *DMA_HandleTypeDef * hdmatx*
- *DMA_HandleTypeDef * hdmarx*
- *__IO HAL_LockTypeDef Lock*

- **__IO HAL_I2S_StateTypeDef State**
- **__IO uint32_t ErrorCode**
- **void(* TxCpltCallback**
- **void(* RxCpltCallback**
- **void(* TxRxCpltCallback**
- **void(* TxHalfCpltCallback**
- **void(* RxHalfCpltCallback**
- **void(* TxRxHalfCpltCallback**
- **void(* ErrorCallback**
- **void(* MspInitCallback**
- **void(* MspDeInitCallback**

Field Documentation

- **SPI_TypeDef* __I2S_HandleTypeDef::Instance**
I2S registers base address
- **I2S_InitTypeDef __I2S_HandleTypeDef::Init**
I2S communication parameters
- **uint16_t* __I2S_HandleTypeDef::pTxBuffPtr**
Pointer to I2S Tx transfer buffer
- **__IO uint16_t __I2S_HandleTypeDef::TxXferSize**
I2S Tx transfer size
- **__IO uint16_t __I2S_HandleTypeDef::TxXferCount**
I2S Tx transfer Counter
- **uint16_t* __I2S_HandleTypeDef::pRxBuffPtr**
Pointer to I2S Rx transfer buffer
- **__IO uint16_t __I2S_HandleTypeDef::RxXferSize**
I2S Rx transfer size
- **__IO uint16_t __I2S_HandleTypeDef::RxXferCount**
I2S Rx transfer counter (This field is initialized at the same value as transfer size at the beginning of the transfer and decremented when a sample is received NbSamplesReceived = RxBufferSize-RxBufferCount)
- **void(* __I2S_HandleTypeDef::IrqHandlerISR)(struct __I2S_HandleTypeDef *hi2s)**
I2S function pointer on IrqHandler
- **DMA_HandleTypeDef* __I2S_HandleTypeDef::hdmatx**
I2S Tx DMA handle parameters
- **DMA_HandleTypeDef* __I2S_HandleTypeDef::hdmarx**
I2S Rx DMA handle parameters
- **__IO HAL_LockTypeDef __I2S_HandleTypeDef::Lock**
I2S locking object
- **__IO HAL_I2S_StateTypeDef __I2S_HandleTypeDef::State**
I2S communication state
- **__IO uint32_t __I2S_HandleTypeDef::ErrorCode**
I2S Error code This parameter can be a value of [I2S_Error](#)
- **void(* __I2S_HandleTypeDef::TxCpltCallback)(struct __I2S_HandleTypeDef *hi2s)**
I2S Tx Completed callback
- **void(* __I2S_HandleTypeDef::RxCpltCallback)(struct __I2S_HandleTypeDef *hi2s)**
I2S Rx Completed callback
- **void(* __I2S_HandleTypeDef::TxRxCpltCallback)(struct __I2S_HandleTypeDef *hi2s)**
I2S TxRx Completed callback
- **void(* __I2S_HandleTypeDef::TxHalfCpltCallback)(struct __I2S_HandleTypeDef *hi2s)**
I2S Tx Half Completed callback
- **void(* __I2S_HandleTypeDef::RxHalfCpltCallback)(struct __I2S_HandleTypeDef *hi2s)**
I2S Rx Half Completed callback
- **void(* __I2S_HandleTypeDef::TxRxHalfCpltCallback)(struct __I2S_HandleTypeDef *hi2s)**
I2S TxRx Half Completed callback

- **`void(* __I2S_HandleTypeDef::ErrorCallback)(struct __I2S_HandleTypeDef *hi2s)`**
I2S Error callback
- **`void(* __I2S_HandleTypeDef::MspInitCallback)(struct __I2S_HandleTypeDef *hi2s)`**
I2S Msp Init callback
- **`void(* __I2S_HandleTypeDef::MspDeInitCallback)(struct __I2S_HandleTypeDef *hi2s)`**
I2S Msp DeInit callback

38.2 I2S Firmware driver API description

The following section lists the various functions of the I2S library.

38.2.1 How to use this driver

The I2S HAL driver can be used as follow:

1. Declare a `I2S_HandleTypeDef` handle structure.
2. Initialize the I2S low level resources by implement the `HAL_I2S_MspInit()` API:
 - a. Enable the SPIx interface clock.
 - b. I2S pins configuration:
 - Enable the clock for the I2S GPIOs.
 - Configure these I2S pins as alternate function pull-up.
 - c. NVIC configuration if you need to use interrupt process (`HAL_I2S_Transmit_IT()` and `HAL_I2S_Receive_IT()` APIs).
 - Configure the I2Sx interrupt priority.
 - Enable the NVIC I2S IRQ handle.
 - d. DMA Configuration if you need to use DMA process (`HAL_I2S_Transmit_DMA()` and `HAL_I2S_Receive_DMA()` APIs):
 - Declare a DMA handle structure for the Tx/Rx Stream/Channel.
 - Enable the DMAx interface clock.
 - Configure the declared DMA handle structure with the required Tx/Rx parameters.
 - Configure the DMA Tx/Rx Stream/Channel.
 - Associate the initialized DMA handle to the I2S DMA Tx/Rx handle.
 - Configure the priority and enable the NVIC for the transfer complete interrupt on the DMA Tx/Rx Stream/Channel.
3. Program the Mode, Standard, Data Format, MCLK Output, Audio frequency and Polarity using `HAL_I2S_Init()` function.

Note: *The specific I2S interrupts (Transmission complete interrupt, RXNE interrupt and Error Interrupts) will be managed using the macros `__HAL_I2S_ENABLE_IT()` and `__HAL_I2S_DISABLE_IT()` inside the transmit and receive process.*

Note: *Make sure that either:*

- *I2S PLL clock is configured or*
- *External clock source is configured after setting correctly the define constant `EXTERNAL_CLOCK_VALUE` in the `stm32f4xx_hal_conf.h` file.*

4. Three mode of operations are available within this driver :

Polling mode IO operation

- Send an amount of data in blocking mode using `HAL_I2S_Transmit()`
- Receive an amount of data in blocking mode using `HAL_I2S_Receive()`

Interrupt mode IO operation

- Send an amount of data in non blocking mode using `HAL_I2S_Transmit_IT()`
- At transmission end of half transfer `HAL_I2S_TxHalfCpltCallback` is executed and user can add his own code by customization of function pointer `HAL_I2S_TxHalfCpltCallback`
- At transmission end of transfer `HAL_I2S_TxCpltCallback` is executed and user can add his own code by customization of function pointer `HAL_I2S_TxCpltCallback`

- Receive an amount of data in non blocking mode using HAL_I2S_Receive_IT()
- At reception end of half transfer HAL_I2S_RxHalfCpltCallback is executed and user can add his own code by customization of function pointer HAL_I2S_RxHalfCpltCallback
- At reception end of transfer HAL_I2S_RxCpltCallback is executed and user can add his own code by customization of function pointer HAL_I2S_RxCpltCallback
- In case of transfer Error, HAL_I2S_ErrorCallback() function is executed and user can add his own code by customization of function pointer HAL_I2S_ErrorCallback

DMA mode IO operation

- Send an amount of data in non blocking mode (DMA) using HAL_I2S_Transmit_DMA()
- At transmission end of half transfer HAL_I2S_TxHalfCpltCallback is executed and user can add his own code by customization of function pointer HAL_I2S_TxHalfCpltCallback
- At transmission end of transfer HAL_I2S_TxCpltCallback is executed and user can add his own code by customization of function pointer HAL_I2S_TxCpltCallback
- Receive an amount of data in non blocking mode (DMA) using HAL_I2S_Receive_DMA()
- At reception end of half transfer HAL_I2S_RxHalfCpltCallback is executed and user can add his own code by customization of function pointer HAL_I2S_RxHalfCpltCallback
- At reception end of transfer HAL_I2S_RxCpltCallback is executed and user can add his own code by customization of function pointer HAL_I2S_RxCpltCallback
- In case of transfer Error, HAL_I2S_ErrorCallback() function is executed and user can add his own code by customization of function pointer HAL_I2S_ErrorCallback
- Pause the DMA Transfer using HAL_I2S_DMAPause()
- Resume the DMA Transfer using HAL_I2S_DMAResume()
- Stop the DMA Transfer using HAL_I2S_DMAStop() In Slave mode, if HAL_I2S_DMAStop is used to stop the communication, an error HAL_I2S_ERROR_BUSY_LINE_RX is raised as the master continue to transmit data. In this case __HAL_I2S_FLUSH_RX_DR macro must be used to flush the remaining data inside DR register and avoid using Delnit/Init process for the next transfer.

I2S HAL driver macros list

Below the list of most used macros in I2S HAL driver.

- __HAL_I2S_ENABLE: Enable the specified SPI peripheral (in I2S mode)
- __HAL_I2S_DISABLE: Disable the specified SPI peripheral (in I2S mode)
- __HAL_I2S_ENABLE_IT : Enable the specified I2S interrupts
- __HAL_I2S_DISABLE_IT : Disable the specified I2S interrupts
- __HAL_I2S_GET_FLAG: Check whether the specified I2S flag is set or not
- __HAL_I2S_FLUSH_RX_DR: Read DR Register to Flush RX Data

Note: You can refer to the I2S HAL driver header file for more useful macros

I2S HAL driver macros list

Callback registration:

1. The compilation flag USE_HAL_I2S_REGISTER_CALLBACKS when set to 1U allows the user to configure dynamically the driver callbacks. Use Functions HAL_I2S_RegisterCallback() to register an interrupt callback. Function HAL_I2S_RegisterCallback() allows to register following callbacks:
 - TxCpltCallback : I2S Tx Completed callback
 - RxCpltCallback : I2S Rx Completed callback
 - TxRxCpltCallback : I2S TxRx Completed callback
 - TxHalfCpltCallback : I2S Tx Half Completed callback
 - RxHalfCpltCallback : I2S Rx Half Completed callback
 - ErrorCallback : I2S Error callback
 - MspInitCallback : I2S Msp Init callback
 - MspDeInitCallback : I2S Msp DeInit callback This function takes as parameters the HAL peripheral handle, the Callback ID and a pointer to the user callback function.

2. Use function `HAL_I2S_UnRegisterCallback` to reset a callback to the default weak function. `HAL_I2S_UnRegisterCallback` takes as parameters the HAL peripheral handle, and the Callback ID. This function allows to reset following callbacks:
 - `TxCpltCallback` : I2S Tx Completed callback
 - `RxCpltCallback` : I2S Rx Completed callback
 - `TxRxCpltCallback` : I2S TxRx Completed callback
 - `TxHalfCpltCallback` : I2S Tx Half Completed callback
 - `RxHalfCpltCallback` : I2S Rx Half Completed callback
 - `ErrorCallback` : I2S Error callback
 - `MspInitCallback` : I2S Msp Init callback
 - `MspDeInitCallback` : I2S Msp DeInit callback

By default, after the `HAL_I2S_Init()` and when the state is `HAL_I2S_STATE_RESET` all callbacks are set to the corresponding weak functions: examples `HAL_I2S_MasterTxCpltCallback()`, `HAL_I2S_MasterRxCpltCallback()`. Exception done for `MspInit` and `MspDeInit` functions that are reset to the legacy weak functions in the `HAL_I2S_Init()`/`HAL_I2S_DeInit()` only when these callbacks are null (not registered beforehand). If `MspInit` or `MspDeInit` are not null, the `HAL_I2S_Init()`/`HAL_I2S_DeInit()` keep and use the user `MspInit`/`MspDeInit` callbacks (registered beforehand) whatever the state.

Callbacks can be registered/unregistered in `HAL_I2S_STATE_READY` state only. Exception done `MspInit`/`MspDeInit` functions that can be registered/unregistered in `HAL_I2S_STATE_READY` or `HAL_I2S_STATE_RESET` state, thus registered (user) `MspInit`/`DeInit` callbacks can be used during the `Init`/`DeInit`. Then, the user first registers the `MspInit`/`MspDeInit` user callbacks using `HAL_I2S_RegisterCallback()` before calling `HAL_I2S_DeInit()` or `HAL_I2S_Init()` function.

When the compilation define `USE_HAL_I2S_REGISTER_CALLBACKS` is set to 0 or not defined, the callback registering feature is not available and weak (surcharged) callbacks are used.

38.2.2

Initialization and de-initialization functions

This subsection provides a set of functions allowing to initialize and de-initialize the I2Sx peripheral in simplex mode:

- User must Implement `HAL_I2S_MspInit()` function in which he configures all related peripherals resources (CLOCK, GPIO, DMA, IT and NVIC).
- Call the function `HAL_I2S_Init()` to configure the selected device with the selected configuration:
 - Mode
 - Standard
 - Data Format
 - MCLK Output
 - Audio frequency
 - Polarity
 - Full duplex mode
- Call the function `HAL_I2S_DeInit()` to restore the default configuration of the selected I2Sx peripheral.

This section contains the following APIs:

- [***HAL_I2S_Init\(\)***](#)
- [***HAL_I2S_DeInit\(\)***](#)
- [***HAL_I2S_MspInit\(\)***](#)
- [***HAL_I2S_MspDeInit\(\)***](#)
- [***HAL_I2S_RegisterCallback\(\)***](#)
- [***HAL_I2S_UnRegisterCallback\(\)***](#)

38.2.3

IO operation functions

This subsection provides a set of functions allowing to manage the I2S data transfers.

1. There are two modes of transfer:
 - Blocking mode : The communication is performed in the polling mode. The status of all data processing is returned by the same function after finishing transfer.
 - No-Blocking mode : The communication is performed using Interrupts or DMA. These functions return the status of the transfer startup. The end of the data processing will be indicated through the dedicated I2S IRQ when using Interrupt mode or the DMA IRQ when using DMA mode.
2. Blocking mode functions are :
 - HAL_I2S_Transmit()
 - HAL_I2S_Receive()
3. No-Blocking mode functions with Interrupt are :
 - HAL_I2S_Transmit_IT()
 - HAL_I2S_Receive_IT()
4. No-Blocking mode functions with DMA are :
 - HAL_I2S_Transmit_DMA()
 - HAL_I2S_Receive_DMA()
5. A set of Transfer Complete Callbacks are provided in non Blocking mode:
 - HAL_I2S_TxCpltCallback()
 - HAL_I2S_RxCpltCallback()
 - HAL_I2S_ErrorCallback()

This section contains the following APIs:

- [*HAL_I2S_Transmit\(\)*](#)
- [*HAL_I2S_Receive\(\)*](#)
- [*HAL_I2S_Transmit_IT\(\)*](#)
- [*HAL_I2S_Receive_IT\(\)*](#)
- [*HAL_I2S_Transmit_DMA\(\)*](#)
- [*HAL_I2S_Receive_DMA\(\)*](#)
- [*HAL_I2S_DMAPause\(\)*](#)
- [*HAL_I2S_DMAResume\(\)*](#)
- [*HAL_I2S_DMAStop\(\)*](#)
- [*HAL_I2S_IRQHandler\(\)*](#)
- [*HAL_I2S_TxHalfCpltCallback\(\)*](#)
- [*HAL_I2S_TxCpltCallback\(\)*](#)
- [*HAL_I2S_RxHalfCpltCallback\(\)*](#)
- [*HAL_I2S_RxCpltCallback\(\)*](#)
- [*HAL_I2S_ErrorCallback\(\)*](#)

38.2.4 Peripheral State and Errors functions

This subsection permits to get in run-time the status of the peripheral and the data flow.

This section contains the following APIs:

- [*HAL_I2S_GetState\(\)*](#)
- [*HAL_I2S_GetError\(\)*](#)

38.2.5 Detailed description of functions

HAL_I2S_Init

Function name

HAL_StatusTypeDef HAL_I2S_Init (I2S_HandleTypeDef * hi2s)

Function description

Initializes the I2S according to the specified parameters in the I2S_InitTypeDef and create the associated handle.

Parameters

- **hi2s**: pointer to a I2S_HandleTypeDef structure that contains the configuration information for I2S module

Return values

- **HAL**: status

HAL_I2S_DeInit

Function name

HAL_StatusTypeDef HAL_I2S_DeInit (I2S_HandleTypeDef * hi2s)

Function description

DeInitializes the I2S peripheral.

Parameters

- **hi2s**: pointer to a I2S_HandleTypeDef structure that contains the configuration information for I2S module

Return values

- **HAL**: status

HAL_I2S_MspInit

Function name

void HAL_I2S_MspInit (I2S_HandleTypeDef * hi2s)

Function description

I2S MSP Init.

Parameters

- **hi2s**: pointer to a I2S_HandleTypeDef structure that contains the configuration information for I2S module

Return values

- **None**:

HAL_I2S_MspDeInit

Function name

void HAL_I2S_MspDeInit (I2S_HandleTypeDef * hi2s)

Function description

I2S MSP DeInit.

Parameters

- **hi2s**: pointer to a I2S_HandleTypeDef structure that contains the configuration information for I2S module

Return values

- **None**:

HAL_I2S_RegisterCallback

Function name

HAL_StatusTypeDef HAL_I2S_RegisterCallback (I2S_HandleTypeDef * hi2s, HAL_I2S_CallbackIDTypeDef CallbackID, pI2S_CallbackTypeDef pCallback)

Function description

Register a User I2S Callback To be used instead of the weak predefined callback.

Parameters

- **hi2s:** Pointer to a I2S_HandleTypeDef structure that contains the configuration information for the specified I2S.
- **CallbackID:** ID of the callback to be registered
- **pCallback:** pointer to the Callback function

Return values

- **HAL:** status

HAL_I2S_UnRegisterCallback

Function name

```
HAL_StatusTypeDef HAL_I2S_UnRegisterCallback (I2S_HandleTypeDef * hi2s,
HAL_I2S_CallbackIDTypeDef CallbackID)
```

Function description

Unregister an I2S Callback I2S callback is redirected to the weak predefined callback.

Parameters

- **hi2s:** Pointer to a I2S_HandleTypeDef structure that contains the configuration information for the specified I2S.
- **CallbackID:** ID of the callback to be unregistered

Return values

- **HAL:** status

HAL_I2S_Transmit

Function name

```
HAL_StatusTypeDef HAL_I2S_Transmit (I2S_HandleTypeDef * hi2s, uint16_t * pData, uint16_t Size,
uint32_t Timeout)
```

Function description

Transmit an amount of data in blocking mode.

Parameters

- **hi2s:** pointer to a I2S_HandleTypeDef structure that contains the configuration information for I2S module
- **pData:** a 16-bit pointer to data buffer.
- **Size:** number of data sample to be sent:
- **Timeout:** Timeout duration

Return values

- **HAL:** status

Notes

- When a 16-bit data frame or a 16-bit data frame extended is selected during the I2S configuration phase, the Size parameter means the number of 16-bit data length in the transaction and when a 24-bit data frame or a 32-bit data frame is selected the Size parameter means the number of 24-bit or 32-bit data length.
- The I2S is kept enabled at the end of transaction to avoid the clock de-synchronization between Master and Slave(example: audio streaming).

HAL_I2S_Receive

Function name

```
HAL_StatusTypeDef HAL_I2S_Receive (I2S_HandleTypeDef * hi2s, uint16_t * pData, uint16_t Size,
uint32_t Timeout)
```

Function description

Receive an amount of data in blocking mode.

Parameters

- **hi2s**: pointer to a I2S_HandleTypeDef structure that contains the configuration information for I2S module
- **pData**: a 16-bit pointer to data buffer.
- **Size**: number of data sample to be sent:
- **Timeout**: Timeout duration

Return values

- **HAL**: status

Notes

- When a 16-bit data frame or a 16-bit data frame extended is selected during the I2S configuration phase, the Size parameter means the number of 16-bit data length in the transaction and when a 24-bit data frame or a 32-bit data frame is selected the Size parameter means the number of 24-bit or 32-bit data length.
- The I2S is kept enabled at the end of transaction to avoid the clock de-synchronization between Master and Slave(example: audio streaming).
- In I2S Master Receiver mode, just after enabling the peripheral the clock will be generate in continuous way and as the I2S is not disabled at the end of the I2S transaction.

HAL_I2S_Transmit_IT

Function name

HAL_StatusTypeDef HAL_I2S_Transmit_IT (I2S_HandleTypeDef * hi2s, uint16_t * pData, uint16_t Size)

Function description

Transmit an amount of data in non-blocking mode with Interrupt.

Parameters

- **hi2s**: pointer to a I2S_HandleTypeDef structure that contains the configuration information for I2S module
- **pData**: a 16-bit pointer to data buffer.
- **Size**: number of data sample to be sent:

Return values

- **HAL**: status

Notes

- When a 16-bit data frame or a 16-bit data frame extended is selected during the I2S configuration phase, the Size parameter means the number of 16-bit data length in the transaction and when a 24-bit data frame or a 32-bit data frame is selected the Size parameter means the number of 24-bit or 32-bit data length.
- The I2S is kept enabled at the end of transaction to avoid the clock de-synchronization between Master and Slave(example: audio streaming).

HAL_I2S_Receive_IT

Function name

HAL_StatusTypeDef HAL_I2S_Receive_IT (I2S_HandleTypeDef * hi2s, uint16_t * pData, uint16_t Size)

Function description

Receive an amount of data in non-blocking mode with Interrupt.

Parameters

- **hi2s**: pointer to a I2S_HandleTypeDef structure that contains the configuration information for I2S module
- **pData**: a 16-bit pointer to the Receive data buffer.
- **Size**: number of data sample to be sent:

Return values

- **HAL**: status

Notes

- When a 16-bit data frame or a 16-bit data frame extended is selected during the I2S configuration phase, the Size parameter means the number of 16-bit data length in the transaction and when a 24-bit data frame or a 32-bit data frame is selected the Size parameter means the number of 24-bit or 32-bit data length.
- The I2S is kept enabled at the end of transaction to avoid the clock de-synchronization between Master and Slave(example: audio streaming).
- It is recommended to use DMA for the I2S receiver to avoid de-synchronization between Master and Slave otherwise the I2S interrupt should be optimized.

HAL_I2S_IRQHandler

Function name

void HAL_I2S_IRQHandler (I2S_HandleTypeDef * hi2s)

Function description

This function handles I2S interrupt request.

Parameters

- **hi2s**: pointer to a I2S_HandleTypeDef structure that contains the configuration information for I2S module

Return values

- **None**:

HAL_I2S_Transmit_DMA

Function name

HAL_StatusTypeDef HAL_I2S_Transmit_DMA (I2S_HandleTypeDef * hi2s, uint16_t * pData, uint16_t Size)

Function description

Transmit an amount of data in non-blocking mode with DMA.

Parameters

- **hi2s**: pointer to a I2S_HandleTypeDef structure that contains the configuration information for I2S module
- **pData**: a 16-bit pointer to the Transmit data buffer.
- **Size**: number of data sample to be sent:

Return values

- **HAL**: status

Notes

- When a 16-bit data frame or a 16-bit data frame extended is selected during the I2S configuration phase, the Size parameter means the number of 16-bit data length in the transaction and when a 24-bit data frame or a 32-bit data frame is selected the Size parameter means the number of 24-bit or 32-bit data length.
- The I2S is kept enabled at the end of transaction to avoid the clock de-synchronization between Master and Slave(example: audio streaming).

HAL_I2S_Receive_DMA

Function name

HAL_StatusTypeDef HAL_I2S_Receive_DMA (I2S_HandleTypeDef * hi2s, uint16_t * pData, uint16_t Size)

Function description

Receive an amount of data in non-blocking mode with DMA.

Parameters

- **hi2s**: pointer to a I2S_HandleTypeDef structure that contains the configuration information for I2S module
- **pData**: a 16-bit pointer to the Receive data buffer.
- **Size**: number of data sample to be sent:

Return values

- **HAL**: status

Notes

- When a 16-bit data frame or a 16-bit data frame extended is selected during the I2S configuration phase, the Size parameter means the number of 16-bit data length in the transaction and when a 24-bit data frame or a 32-bit data frame is selected the Size parameter means the number of 24-bit or 32-bit data length.
- The I2S is kept enabled at the end of transaction to avoid the clock de-synchronization between Master and Slave(example: audio streaming).

HAL_I2S_DMAPause

Function name

HAL_StatusTypeDef HAL_I2S_DMAPause (I2S_HandleTypeDef * hi2s)

Function description

Pauses the audio DMA Stream/Channel playing from the Media.

Parameters

- **hi2s**: pointer to a I2S_HandleTypeDef structure that contains the configuration information for I2S module

Return values

- **HAL**: status

HAL_I2S_DMAResume

Function name

HAL_StatusTypeDef HAL_I2S_DMAResume (I2S_HandleTypeDef * hi2s)

Function description

Resumes the audio DMA Stream/Channel playing from the Media.

Parameters

- **hi2s**: pointer to a I2S_HandleTypeDef structure that contains the configuration information for I2S module

Return values

- **HAL**: status

HAL_I2S_DMAStop

Function name

HAL_StatusTypeDef HAL_I2S_DMAStop (I2S_HandleTypeDef * hi2s)

Function description

Stops the audio DMA Stream/Channel playing from the Media.

Parameters

- **hi2s:** pointer to a I2S_HandleTypeDef structure that contains the configuration information for I2S module

Return values

- **HAL:** status

HAL_I2S_TxHalfCpltCallback

Function name

void HAL_I2S_TxHalfCpltCallback (I2S_HandleTypeDef * hi2s)

Function description

Tx Transfer Half completed callbacks.

Parameters

- **hi2s:** pointer to a I2S_HandleTypeDef structure that contains the configuration information for I2S module

Return values

- **None:**

HAL_I2S_TxCpltCallback

Function name

void HAL_I2S_TxCpltCallback (I2S_HandleTypeDef * hi2s)

Function description

Tx Transfer completed callbacks.

Parameters

- **hi2s:** pointer to a I2S_HandleTypeDef structure that contains the configuration information for I2S module

Return values

- **None:**

HAL_I2S_RxHalfCpltCallback

Function name

void HAL_I2S_RxHalfCpltCallback (I2S_HandleTypeDef * hi2s)

Function description

Rx Transfer half completed callbacks.

Parameters

- **hi2s:** pointer to a I2S_HandleTypeDef structure that contains the configuration information for I2S module

Return values

- **None:**

HAL_I2S_RxCpltCallback

Function name

void HAL_I2S_RxCpltCallback (I2S_HandleTypeDef * hi2s)

Function description

Rx Transfer completed callbacks.

Parameters

- **hi2s**: pointer to a I2S_HandleTypeDef structure that contains the configuration information for I2S module

Return values

- **None**:

HAL_I2S_ErrorCallback

Function name

void HAL_I2S_ErrorCallback (I2S_HandleTypeDef * hi2s)

Function description

I2S error callbacks.

Parameters

- **hi2s**: pointer to a I2S_HandleTypeDef structure that contains the configuration information for I2S module

Return values

- **None**:

HAL_I2S_GetState

Function name

HAL_I2S_StateTypeDef HAL_I2S_GetState (I2S_HandleTypeDef * hi2s)

Function description

Return the I2S state.

Parameters

- **hi2s**: pointer to a I2S_HandleTypeDef structure that contains the configuration information for I2S module

Return values

- **HAL**: state

HAL_I2S_GetError

Function name

uint32_t HAL_I2S_GetError (I2S_HandleTypeDef * hi2s)

Function description

Return the I2S error code.

Parameters

- **hi2s**: pointer to a I2S_HandleTypeDef structure that contains the configuration information for I2S module

Return values

- **I2S**: Error Code

38.3 I2S Firmware driver defines

The following section lists the various define and macros of the module.

38.3.1

I2S

I2S

I2S Audio Frequency

I2S_AUDIOFREQ_192K

I2S_AUDIOFREQ_96K

I2S_AUDIOFREQ_48K

I2S_AUDIOFREQ_44K

I2S_AUDIOFREQ_32K

I2S_AUDIOFREQ_22K

I2S_AUDIOFREQ_16K

I2S_AUDIOFREQ_11K

I2S_AUDIOFREQ_8K

I2S_AUDIOFREQ_DEFAULT

I2S Clock Polarity

I2S_CPOL_LOW

I2S_CPOL_HIGH

I2S Clock Source Definition

I2S_CLOCK_PLL

I2S_CLOCK_EXTERNAL

I2S Data Format

I2S_DATAFORMAT_16B

I2S_DATAFORMAT_16B_EXTENDED

I2S_DATAFORMAT_24B

I2S_DATAFORMAT_32B

I2S Error

HAL_I2S_ERROR_NONE

No error

HAL_I2S_ERROR_TIMEOUT

Timeout error

HAL_I2S_ERROR_OVR

OVR error

HAL_I2S_ERROR_UDR

UDR error

HAL_I2S_ERROR_DMA

DMA transfer error

HAL_I2S_ERROR_PRESCALER

Prescaler Calculation error

HAL_I2S_ERROR_INVALID_CALLBACK

Invalid Callback error

HAL_I2S_ERROR_BUSY_LINE_RX

Busy Rx Line error

I2S Exported Macros

__HAL_I2S_RESET_HANDLE_STATE

Description:

- Reset I2S handle state.

Parameters:

- `__HANDLE__`: specifies the I2S Handle.

Return value:

- None

__HAL_I2S_ENABLE

Description:

- Enable the specified SPI peripheral (in I2S mode).

Parameters:

- `__HANDLE__`: specifies the I2S Handle.

Return value:

- None

__HAL_I2S_DISABLE

Description:

- Disable the specified SPI peripheral (in I2S mode).

Parameters:

- `__HANDLE__`: specifies the I2S Handle.

Return value:

- None

__HAL_I2S_ENABLE_IT

Description:

- Enable the specified I2S interrupts.

Parameters:

- `__HANDLE__`: specifies the I2S Handle.
- `__INTERRUPT__`: specifies the interrupt source to enable or disable. This parameter can be one of the following values:
 - I2S_IT_TXE: Tx buffer empty interrupt enable
 - I2S_IT_RXNE: RX buffer not empty interrupt enable
 - I2S_IT_ERR: Error interrupt enable

Return value:

- None

__HAL_I2S_DISABLE_IT

Description:

- Disable the specified I2S interrupts.

Parameters:

- `__HANDLE__`: specifies the I2S Handle.
- `__INTERRUPT__`: specifies the interrupt source to enable or disable. This parameter can be one of the following values:
 - `I2S_IT_TXE`: Tx buffer empty interrupt enable
 - `I2S_IT_RXNE`: RX buffer not empty interrupt enable
 - `I2S_IT_ERR`: Error interrupt enable

Return value:

- None

__HAL_I2S_GET_IT_SOURCE

Description:

- Checks if the specified I2S interrupt source is enabled or disabled.

Parameters:

- `__HANDLE__`: specifies the I2S Handle. This parameter can be `I2S` where `x`: 1, 2, or 3 to select the I2S peripheral.
- `__INTERRUPT__`: specifies the I2S interrupt source to check. This parameter can be one of the following values:
 - `I2S_IT_TXE`: Tx buffer empty interrupt enable
 - `I2S_IT_RXNE`: RX buffer not empty interrupt enable
 - `I2S_IT_ERR`: Error interrupt enable

Return value:

- The: new state of `__IT__` (TRUE or FALSE).

__HAL_I2S_GET_FLAG

Description:

- Checks whether the specified I2S flag is set or not.

Parameters:

- `__HANDLE__`: specifies the I2S Handle.
- `__FLAG__`: specifies the flag to check. This parameter can be one of the following values:
 - `I2S_FLAG_RXNE`: Receive buffer not empty flag
 - `I2S_FLAG_TXE`: Transmit buffer empty flag
 - `I2S_FLAG_UDR`: Underrun flag
 - `I2S_FLAG_OVR`: Overrun flag
 - `I2S_FLAG_FRE`: Frame error flag
 - `I2S_FLAG_CHSIDE`: Channel Side flag
 - `I2S_FLAG_BSY`: Busy flag

Return value:

- The: new state of `__FLAG__` (TRUE or FALSE).

__HAL_I2S_CLEAR_OVRFLAG

Description:

- Clears the I2S OVR pending flag.

Parameters:

- `__HANDLE__`: specifies the I2S Handle.

Return value:

- None

__HAL_I2S_CLEAR_UDRFLAG

Description:

- Clears the I2S UDR pending flag.

Parameters:

- __HANDLE__: specifies the I2S Handle.

Return value:

- None

__HAL_I2S_FLUSH_RX_DR

Description:

- Flush the I2S DR Register.

Parameters:

- __HANDLE__: specifies the I2S Handle.

Return value:

- None

I2S Flags Definition

I2S_FLAG_TXE

I2S_FLAG_RXNE

I2S_FLAG_UDR

I2S_FLAG_OVR

I2S_FLAG_FRE

I2S_FLAG_CHSIDE

I2S_FLAG_BSY

I2S_FLAG_MASK

I2S FullDuplex Mode

I2S_FULLDUPLEXMODE_DISABLE

I2S_FULLDUPLEXMODE_ENABLE

I2S Interrupts Definition

I2S_IT_TXE

I2S_IT_RXNE

I2S_IT_ERR

I2S MCLK Output

I2S_MCLKOUTPUT_ENABLE

I2S_MCLKOUTPUT_DISABLE

I2S Mode

I2S_MODE_SLAVE_TX

I2S_MODE_SLAVE_RX

I2S_MODE_MASTER_TX

I2S_MODE_MASTER_RX

I2S Standard

I2S_STANDARD_PHILIPS

I2S_STANDARD_MSB

I2S_STANDARD_LSB

I2S_STANDARD_PCM_SHORT

I2S_STANDARD_PCM_LONG

39 HAL I2S Extension Driver

39.1 I2SEx Firmware driver API description

The following section lists the various functions of the I2SEx library.

39.1.1 I2S Extension features

1. In I2S full duplex mode, each SPI peripheral is able to manage sending and receiving data simultaneously using two data lines. Each SPI peripheral has an extended block called I2Sxext (i.e I2S2ext for SPI2 and I2S3ext for SPI3).
2. The extension block is not a full SPI IP, it is used only as I2S slave to implement full duplex mode. The extension block uses the same clock sources as its master.
3. Both I2Sx and I2Sx_ext can be configured as transmitters or receivers.

Note: Only I2Sx can deliver SCK and WS to I2Sx_ext in full duplex mode, where I2Sx can be I2S2 or I2S3.

39.1.2 How to use this driver

Three operation modes are available within this driver :

Polling mode IO operation

- Send and receive in the same time an amount of data in blocking mode using `HAL_I2SEx_TransmitReceive()`

Interrupt mode IO operation

- Send and receive in the same time an amount of data in non blocking mode using `HAL_I2SEx_TransmitReceive_IT()`
- At transmission/reception end of transfer `HAL_I2SEx_TxRxCpltCallback` is executed and user can add his own code by customization of function pointer `HAL_I2SEx_TxRxCpltCallback`
- In case of transfer Error, `HAL_I2S_ErrorCallback()` function is executed and user can add his own code by customization of function pointer `HAL_I2S_ErrorCallback`

DMA mode IO operation

- Send and receive an amount of data in non blocking mode (DMA) using `HAL_I2SEx_TransmitReceive_DMA()`
- At transmission/reception end of transfer `HAL_I2SEx_TxRxCpltCallback` is executed and user can add his own code by customization of function pointer `HAL_I2S_TxRxCpltCallback`
- In case of transfer Error, `HAL_I2S_ErrorCallback()` function is executed and user can add his own code by customization of function pointer `HAL_I2S_ErrorCallback`
- `__HAL_I2SEXT_FLUSH_RX_DR`: In Full-Duplex Slave mode, if `HAL_I2S_DMAStop` is used to stop the communication, an error `HAL_I2S_ERROR_BUSY_LINE_RX` is raised as the master continue to transmit data. In this case `__HAL_I2SEXT_FLUSH_RX_DR` macro must be used to flush the remaining data inside I2Sx and I2Sx_ext DR registers and avoid using Delnit/Init process for the next transfer.

39.1.3 IO operation functions

This subsection provides a set of functions allowing to manage the I2S data transfers.

1. There are two modes of transfer:
 - Blocking mode : The communication is performed in the polling mode. The status of all data processing is returned by the same function after finishing transfer.
 - No-Blocking mode : The communication is performed using Interrupts or DMA. These functions return the status of the transfer startup. The end of the data processing will be indicated through the dedicated I2S IRQ when using Interrupt mode or the DMA IRQ when using DMA mode.
2. Blocking mode functions are :
 - `HAL_I2SEx_TransmitReceive()`

3. No-Blocking mode functions with Interrupt are :
 - HAL_I2SEx_TransmitReceive_IT()
 - HAL_I2SEx_FullDuplex_IRQHandler()
4. No-Blocking mode functions with DMA are :
 - HAL_I2SEx_TransmitReceive_DMA()
5. A set of Transfer Complete Callback are provided in non Blocking mode:
 - HAL_I2SEx_TxRxCpltCallback()

This section contains the following APIs:

- [HAL_I2SEx_TransmitReceive\(\)](#)
- [HAL_I2SEx_TransmitReceive_IT\(\)](#)
- [HAL_I2SEx_TransmitReceive_DMA\(\)](#)
- [HAL_I2SEx_FullDuplex_IRQHandler\(\)](#)
- [HAL_I2SEx_TxRxHalfCpltCallback\(\)](#)
- [HAL_I2SEx_TxRxCpltCallback\(\)](#)

39.1.4 Detailed description of functions

HAL_I2SEx_TransmitReceive

Function name

HAL_StatusTypeDef HAL_I2SEx_TransmitReceive (I2S_HandleTypeDef * hi2s, uint16_t * pTxData, uint16_t * pRxData, uint16_t Size, uint32_t Timeout)

Function description

Full-Duplex Transmit/Receive data in blocking mode.

Parameters

- **hi2s**: pointer to a I2S_HandleTypeDef structure that contains the configuration information for I2S module
- **pTxData**: a 16-bit pointer to the Transmit data buffer.
- **pRxData**: a 16-bit pointer to the Receive data buffer.
- **Size**: number of data sample to be sent:
- **Timeout**: Timeout duration

Return values

- **HAL**: status

Notes

- When a 16-bit data frame or a 16-bit data frame extended is selected during the I2S configuration phase, the Size parameter means the number of 16-bit data length in the transaction and when a 24-bit data frame or a 32-bit data frame is selected the Size parameter means the number of 16-bit data length.
- The I2S is kept enabled at the end of transaction to avoid the clock de-synchronization between Master and Slave(example: audio streaming).

HAL_I2SEx_TransmitReceive_IT

Function name

HAL_StatusTypeDef HAL_I2SEx_TransmitReceive_IT (I2S_HandleTypeDef * hi2s, uint16_t * pTxData, uint16_t * pRxData, uint16_t Size)

Function description

Full-Duplex Transmit/Receive data in non-blocking mode using Interrupt.

Parameters

- **hi2s**: pointer to a I2S_HandleTypeDef structure that contains the configuration information for I2S module
- **pTxData**: a 16-bit pointer to the Transmit data buffer.
- **pRxData**: a 16-bit pointer to the Receive data buffer.
- **Size**: number of data sample to be sent:

Return values

- **HAL**: status

Notes

- When a 16-bit data frame or a 16-bit data frame extended is selected during the I2S configuration phase, the Size parameter means the number of 16-bit data length in the transaction and when a 24-bit data frame or a 32-bit data frame is selected the Size parameter means the number of 16-bit data length.
- The I2S is kept enabled at the end of transaction to avoid the clock de-synchronization between Master and Slave(example: audio streaming).

HAL_I2SEx_TransmitReceive_DMA

Function name

HAL_StatusTypeDef HAL_I2SEx_TransmitReceive_DMA (I2S_HandleTypeDef * hi2s, uint16_t * pTxData, uint16_t * pRxData, uint16_t Size)

Function description

Full-Duplex Transmit/Receive data in non-blocking mode using DMA.

Parameters

- **hi2s**: pointer to a I2S_HandleTypeDef structure that contains the configuration information for I2S module
- **pTxData**: a 16-bit pointer to the Transmit data buffer.
- **pRxData**: a 16-bit pointer to the Receive data buffer.
- **Size**: number of data sample to be sent:

Return values

- **HAL**: status

Notes

- When a 16-bit data frame or a 16-bit data frame extended is selected during the I2S configuration phase, the Size parameter means the number of 16-bit data length in the transaction and when a 24-bit data frame or a 32-bit data frame is selected the Size parameter means the number of 16-bit data length.
- The I2S is kept enabled at the end of transaction to avoid the clock de-synchronization between Master and Slave(example: audio streaming).

HAL_I2SEx_FullDuplex_IRQHandler

Function name

void HAL_I2SEx_FullDuplex_IRQHandler (I2S_HandleTypeDef * hi2s)

Function description

This function handles I2S/I2Sext interrupt requests in full-duplex mode.

Parameters

- **hi2s**: I2S handle

Return values

- **HAL**: status

HAL_I2SEx_TxRxHalfCpltCallback

Function name

void HAL_I2SEx_TxRxHalfCpltCallback (I2S_HandleTypeDef * hi2s)

Function description

Tx and Rx Transfer half completed callback.

Parameters

- **hi2s:** I2S handle

Return values

- **None:**

HAL_I2SEx_TxRxCpltCallback

Function name

void HAL_I2SEx_TxRxCpltCallback (I2S_HandleTypeDef * hi2s)

Function description

Tx and Rx Transfer completed callback.

Parameters

- **hi2s:** I2S handle

Return values

- **None:**

39.2 I2SEx Firmware driver defines

The following section lists the various define and macros of the module.

39.2.1

I2SEx

I2SEx

I2S Extended Exported Macros

I2SxEXT

__HAL_I2SEXT_ENABLE

Description:

- Enable or disable the specified I2SExt peripheral.

Parameters:

- **__HANDLE__:** specifies the I2S Handle.

Return value:

- None

__HAL_I2SEXT_DISABLE

__HAL_I2SEXT_ENABLE_IT

Description:

- Enable or disable the specified I2SExt interrupts.

Parameters:

- `__HANDLE__`: specifies the I2S Handle.
- `__INTERRUPT__`: specifies the interrupt source to enable or disable. This parameter can be one of the following values:
 - `I2S_IT_TXE`: Tx buffer empty interrupt enable
 - `I2S_IT_RXNE`: RX buffer not empty interrupt enable
 - `I2S_IT_ERR`: Error interrupt enable

Return value:

- None

__HAL_I2SEXT_DISABLE_IT

__HAL_I2SEXT_GET_IT_SOURCE

Description:

- Checks if the specified I2SExt interrupt source is enabled or disabled.

Parameters:

- `__HANDLE__`: specifies the I2S Handle. This parameter can be I2S where x: 1, 2, or 3 to select the I2S peripheral.
- `__INTERRUPT__`: specifies the I2S interrupt source to check. This parameter can be one of the following values:
 - `I2S_IT_TXE`: Tx buffer empty interrupt enable
 - `I2S_IT_RXNE`: RX buffer not empty interrupt enable
 - `I2S_IT_ERR`: Error interrupt enable

Return value:

- The: new state of `__IT__` (TRUE or FALSE).

__HAL_I2SEXT_GET_FLAG

Description:

- Checks whether the specified I2SExt flag is set or not.

Parameters:

- `__HANDLE__`: specifies the I2S Handle.
- `__FLAG__`: specifies the flag to check. This parameter can be one of the following values:
 - `I2S_FLAG_RXNE`: Receive buffer not empty flag
 - `I2S_FLAG_TXE`: Transmit buffer empty flag
 - `I2S_FLAG_UDR`: Underrun flag
 - `I2S_FLAG_OVR`: Overrun flag
 - `I2S_FLAG_FRE`: Frame error flag
 - `I2S_FLAG_CHSIDE`: Channel Side flag
 - `I2S_FLAG_BSY`: Busy flag

Return value:

- The: new state of `__FLAG__` (TRUE or FALSE).

__HAL_I2SEXT_CLEAR_OVRFLAG**Description:**

- Clears the I2SExt OVR pending flag.

Parameters:

- `__HANDLE__`: specifies the I2S Handle.

Return value:

- None

__HAL_I2SEXT_CLEAR_UDRFLAG**Description:**

- Clears the I2SExt UDR pending flag.

Parameters:

- `__HANDLE__`: specifies the I2S Handle.

Return value:

- None

__HAL_I2SEXT_FLUSH_RX_DR**Description:**

- Flush the I2S and I2SExt DR Registers.

Parameters:

- `__HANDLE__`: specifies the I2S Handle.

Return value:

- None

40 HAL IRDA Generic Driver

40.1 IRDA Firmware driver registers structures

40.1.1 IRDA_InitTypeDef

IRDA_InitTypeDef is defined in the stm32f4xx_hal_irda.h

Data Fields

- *uint32_t BaudRate*
- *uint32_t WordLength*
- *uint32_t Parity*
- *uint32_t Mode*
- *uint8_t Prescaler*
- *uint32_t IrDAMode*

Field Documentation

- *uint32_t IRDA_InitTypeDef::BaudRate*
This member configures the IRDA communication baud rate. The baud rate is computed using the following formula:
 - IntegerDivider = ((PCLKx) / (8 * (hirda->Init.BaudRate)))
 - FractionalDivider = ((IntegerDivider - ((uint32_t) IntegerDivider)) * 8) + 0.5
- *uint32_t IRDA_InitTypeDef::WordLength*
Specifies the number of data bits transmitted or received in a frame. This parameter can be a value of [IRDA_Word_Length](#)
- *uint32_t IRDA_InitTypeDef::Parity*
Specifies the parity mode. This parameter can be a value of [IRDA_Parity](#)
Note:
 - When parity is enabled, the computed parity is inserted at the MSB position of the transmitted data (9th bit when the word length is set to 9 data bits; 8th bit when the word length is set to 8 data bits).
- *uint32_t IRDA_InitTypeDef::Mode*
Specifies whether the Receive or Transmit mode is enabled or disabled. This parameter can be a value of [IRDA_Mode](#)
- *uint8_t IRDA_InitTypeDef::Prescaler*
Specifies the Prescaler value to be programmed in the IrDA low-power Baud Register, for defining pulse width on which burst acceptance/rejection will be decided. This value is used as divisor of system clock to achieve required pulse width.
- *uint32_t IRDA_InitTypeDef::IrDAMode*
Specifies the IrDA mode This parameter can be a value of [IRDA_Low_Power](#)

40.1.2 IRDA_HandleTypeDef

IRDA_HandleTypeDef is defined in the stm32f4xx_hal_irda.h

Data Fields

- *USART_TypeDef * Instance*
- *IRDA_InitTypeDef Init*
- *uint8_t * pTxBuffPtr*
- *uint16_t TxXferSize*
- *__IO uint16_t TxXferCount*
- *uint8_t * pRxBuffPtr*
- *uint16_t RxXferSize*
- *__IO uint16_t RxXferCount*
- *DMA_HandleTypeDef * hdmatx*
- *DMA_HandleTypeDef * hdmarx*
- *HAL_LockTypeDef Lock*

- `__IO HAL_IRDA_StateTypeDef gState`
- `__IO HAL_IRDA_StateTypeDef RxState`
- `__IO uint32_t ErrorCode`

Field Documentation

- **`USART_TypeDef* IRDA_HandleTypeDef::Instance`**
USART registers base address
- **`IRDA_InitTypeDef IRDA_HandleTypeDef::Init`**
IRDA communication parameters
- **`uint8_t* IRDA_HandleTypeDef::pTxBuffPtr`**
Pointer to IRDA Tx transfer Buffer
- **`uint16_t IRDA_HandleTypeDef::TxXferSize`**
IRDA Tx Transfer size
- **`__IO uint16_t IRDA_HandleTypeDef::TxXferCount`**
IRDA Tx Transfer Counter
- **`uint8_t* IRDA_HandleTypeDef::pRxBuffPtr`**
Pointer to IRDA Rx transfer Buffer
- **`uint16_t IRDA_HandleTypeDef::RxXferSize`**
IRDA Rx Transfer size
- **`__IO uint16_t IRDA_HandleTypeDef::RxXferCount`**
IRDA Rx Transfer Counter
- **`DMA_HandleTypeDef* IRDA_HandleTypeDef::hdmatx`**
IRDA Tx DMA Handle parameters
- **`DMA_HandleTypeDef* IRDA_HandleTypeDef::hdmarx`**
IRDA Rx DMA Handle parameters
- **`HAL_LockTypeDef IRDA_HandleTypeDef::Lock`**
Locking object
- **`__IO HAL_IRDA_StateTypeDef IRDA_HandleTypeDef::gState`**
IRDA state information related to global Handle management and also related to Tx operations. This parameter can be a value of `HAL_IRDA_StateTypeDef`
- **`__IO HAL_IRDA_StateTypeDef IRDA_HandleTypeDef::RxState`**
IRDA state information related to Rx operations. This parameter can be a value of `HAL_IRDA_StateTypeDef`
- **`__IO uint32_t IRDA_HandleTypeDef::ErrorCode`**
IRDA Error code

40.2 IRDA Firmware driver API description

The following section lists the various functions of the IRDA library.

40.2.1 How to use this driver

The IRDA HAL driver can be used as follows:

1. Declare a `IRDA_HandleTypeDef` handle structure (eg. `IRDA_HandleTypeDef hirda`).

2. Initialize the IRDA low level resources by implementing the HAL_IRDA_MspInit() API:
 - a. Enable the USARTx interface clock.
 - b. IRDA pins configuration:
 - Enable the clock for the IRDA GPIOs.
 - Configure IRDA pins as alternate function pull-up.
 - c. NVIC configuration if you need to use interrupt process (HAL_IRDA_Transmit_IT() and HAL_IRDA_Receive_IT() APIs):
 - Configure the USARTx interrupt priority.
 - Enable the NVIC USART IRQ handle.
 - d. DMA Configuration if you need to use DMA process (HAL_IRDA_Transmit_DMA() and HAL_IRDA_Receive_DMA() APIs):
 - Declare a DMA handle structure for the Tx/Rx stream.
 - Enable the DMAx interface clock.
 - Configure the declared DMA handle structure with the required Tx/Rx parameters.
 - Configure the DMA Tx/Rx stream.
 - Associate the initialized DMA handle to the IRDA DMA Tx/Rx handle.
 - Configure the priority and enable the NVIC for the transfer complete interrupt on the DMA Tx/Rx stream.
 - Configure the IRDAx interrupt priority and enable the NVIC USART IRQ handle (used for last byte sending completion detection in DMA non circular mode)
 3. Program the Baud Rate, Word Length, Parity, IrDA Mode, Prescaler and Mode(Receiver/Transmitter) in the hirda Init structure.
 4. Initialize the IRDA registers by calling the HAL_IRDA_Init() API:
 - This API configures also the low level Hardware GPIO, CLOCK, CORTEX...etc) by calling the customized HAL_IRDA_MspInit() API.
- Note: The specific IRDA interrupts (Transmission complete interrupt, RXNE interrupt and Error Interrupts) will be managed using the macros __HAL_IRDA_ENABLE_IT() and __HAL_IRDA_DISABLE_IT() inside the transmit and receive process.*
5. Three operation modes are available within this driver :

Polling mode IO operation

- Send an amount of data in blocking mode using HAL_IRDA_Transmit()
- Receive an amount of data in blocking mode using HAL_IRDA_Receive()

Interrupt mode IO operation

- Send an amount of data in non blocking mode using HAL_IRDA_Transmit_IT()
- At transmission end of transfer HAL_IRDA_TxCpltCallback is executed and user can add his own code by customization of function pointer HAL_IRDA_TxCpltCallback
- Receive an amount of data in non blocking mode using HAL_IRDA_Receive_IT()
- At reception end of transfer HAL_IRDA_RxCpltCallback is executed and user can add his own code by customization of function pointer HAL_IRDA_RxCpltCallback
- In case of transfer Error, HAL_IRDA_ErrorCallback() function is executed and user can add his own code by customization of function pointer HAL_IRDA_ErrorCallback

DMA mode IO operation

- Send an amount of data in non blocking mode (DMA) using HAL_IRDA_Transmit_DMA()
- At transmission end of half transfer HAL_IRDA_TxHalfCpltCallback is executed and user can add his own code by customization of function pointer HAL_IRDA_TxHalfCpltCallback
- At transmission end of transfer HAL_IRDA_TxCpltCallback is executed and user can add his own code by customization of function pointer HAL_IRDA_TxCpltCallback
- Receive an amount of data in non blocking mode (DMA) using HAL_IRDA_Receive_DMA()

- At reception end of half transfer HAL_IRDA_RxHalfCpltCallback is executed and user can add his own code by customization of function pointer HAL_IRDA_RxHalfCpltCallback
- At reception end of transfer HAL_IRDA_RxCpltCallback is executed and user can add his own code by customization of function pointer HAL_IRDA_RxCpltCallback
- In case of transfer Error, HAL_IRDA_ErrorCallback() function is executed and user can add his own code by customization of function pointer HAL_IRDA_ErrorCallback
- Pause the DMA Transfer using HAL_IRDA_DMAPause()
- Resume the DMA Transfer using HAL_IRDA_DMAResume()
- Stop the DMA Transfer using HAL_IRDA_DMAStop()

IRDA HAL driver macros list

Below the list of most used macros in IRDA HAL driver.

- `__HAL_IRDA_ENABLE`: Enable the IRDA peripheral
- `__HAL_IRDA_DISABLE`: Disable the IRDA peripheral
- `__HAL_IRDA_GET_FLAG`: Check whether the specified IRDA flag is set or not
- `__HAL_IRDA_CLEAR_FLAG`: Clear the specified IRDA pending flag
- `__HAL_IRDA_ENABLE_IT`: Enable the specified IRDA interrupt
- `__HAL_IRDA_DISABLE_IT`: Disable the specified IRDA interrupt
- `__HAL_IRDA_GET_IT_SOURCE`: Check whether the specified IRDA interrupt has occurred or not

Note: You can refer to the IRDA HAL driver header file for more useful macros

40.2.2 Callback registration

The compilation define `USE_HAL_IRDA_REGISTER_CALLBACKS` when set to 1 allows the user to configure dynamically the driver callbacks.

Use Function `HAL_IRDA_RegisterCallback()` to register a user callback. Function `HAL_IRDA_RegisterCallback()` allows to register following callbacks:

- `TxHalfCpltCallback`: Tx Half Complete Callback.
- `TxCpltCallback`: Tx Complete Callback.
- `RxHalfCpltCallback`: Rx Half Complete Callback.
- `RxCpltCallback`: Rx Complete Callback.
- `ErrorCallback`: Error Callback.
- `AbortCpltCallback`: Abort Complete Callback.
- `AbortTransmitCpltCallback`: Abort Transmit Complete Callback.
- `AbortReceiveCpltCallback`: Abort Receive Complete Callback.
- `MspInitCallback`: IRDA MspInit.
- `MspDeInitCallback`: IRDA MspDeInit. This function takes as parameters the HAL peripheral handle, the Callback ID and a pointer to the user callback function.

Use function `HAL_IRDA_UnRegisterCallback()` to reset a callback to the default weak (surcharged) function. `HAL_IRDA_UnRegisterCallback()` takes as parameters the HAL peripheral handle, and the Callback ID. This function allows to reset following callbacks:

- `TxHalfCpltCallback`: Tx Half Complete Callback.
- `TxCpltCallback`: Tx Complete Callback.
- `RxHalfCpltCallback`: Rx Half Complete Callback.
- `RxCpltCallback`: Rx Complete Callback.
- `ErrorCallback`: Error Callback.
- `AbortCpltCallback`: Abort Complete Callback.
- `AbortTransmitCpltCallback`: Abort Transmit Complete Callback.
- `AbortReceiveCpltCallback`: Abort Receive Complete Callback.
- `MspInitCallback`: IRDA MspInit.
- `MspDeInitCallback`: IRDA MspDeInit.

By default, after the HAL_IRDA_Init() and when the state is HAL_IRDA_STATE_RESET all callbacks are set to the corresponding weak (surcharged) functions: examples HAL_IRDA_TxCpltCallback(), HAL_IRDA_RxCpltCallback(). Exception done for MspInit and MspDeInit functions that are respectively reset to the legacy weak (surcharged) functions in the HAL_IRDA_Init() and HAL_IRDA_DeInit() only when these callbacks are null (not registered beforehand). If not, MspInit or MspDeInit are not null, the HAL_IRDA_Init() and HAL_IRDA_DeInit() keep and use the user MspInit/MspDeInit callbacks (registered beforehand).

Callbacks can be registered/unregistered in HAL_IRDA_STATE_READY state only. Exception done MspInit/MspDeInit that can be registered/unregistered in HAL_IRDA_STATE_READY or HAL_IRDA_STATE_RESET state, thus registered (user) MspInit/DeInit callbacks can be used during the Init/DeInit. In that case first register the MspInit/MspDeInit user callbacks using HAL_IRDA_RegisterCallback() before calling HAL_IRDA_DeInit() or HAL_IRDA_Init() function.

When The compilation define USE_HAL_IRDA_REGISTER_CALLBACKS is set to 0 or not defined, the callback registration feature is not available and weak (surcharged) callbacks are used.

40.2.3 Initialization and Configuration functions

This subsection provides a set of functions allowing to initialize the USARTx or the UARTy in asynchronous IrDA mode.

- For the asynchronous mode only these parameters can be configured:
 - BaudRate
 - WordLength
 - Parity: If the parity is enabled, then the MSB bit of the data written in the data register is transmitted but is changed by the parity bit. Depending on the frame length defined by the M bit (8-bits or 9-bits), please refer to Reference manual for possible IrDA frame formats.
 - Prescaler: A pulse of width less than two and greater than one PSC period(s) may or may not be rejected. The receiver set up time should be managed by software. The IrDA physical layer specification specifies a minimum of 10 ms delay between transmission and reception (IrDA is a half duplex protocol).
 - Mode: Receiver/transmitter modes
 - IrDAMode: the IrDA can operate in the Normal mode or in the Low power mode.

The HAL_IRDA_Init() API follows IrDA configuration procedures (details for the procedures are available in reference manual).

This section contains the following APIs:

- [HAL_IRDA_Init\(\)](#)
- [HAL_IRDA_DeInit\(\)](#)
- [HAL_IRDA_MspInit\(\)](#)
- [HAL_IRDA_MspDeInit\(\)](#)
- [HAL_IRDA_RegisterCallback\(\)](#)
- [HAL_IRDA_UnRegisterCallback\(\)](#)

40.2.4 IO operation functions

This subsection provides a set of functions allowing to manage the IrDA data transfers. IrDA is a half duplex communication protocol. If the Transmitter is busy, any data on the IrDA receive line will be ignored by the IrDA decoder and if the Receiver is busy, data on the TX from the USART to IrDA will not be encoded by IrDA. While receiving data, transmission should be avoided as the data to be transmitted could be corrupted.

1. There are two modes of transfer:
 - Blocking mode: The communication is performed in polling mode. The HAL status of all data processing is returned by the same function after finishing transfer.
 - Non-Blocking mode: The communication is performed using Interrupts or DMA, these API's return the HAL status. The end of the data processing will be indicated through the dedicated IrDA IRQ when using Interrupt mode or the DMA IRQ when using DMA mode. The HAL_IRDA_TxCpltCallback(), HAL_IRDA_RxCpltCallback() user callbacks will be executed respectively at the end of the Transmit or Receive process. The HAL_IRDA_ErrorCallback() user callback will be executed when a communication error is detected.

2. Blocking mode APIs are :
 - HAL_IRDA_Transmit()
 - HAL_IRDA_Receive()
3. Non Blocking mode APIs with Interrupt are :
 - HAL_IRDA_Transmit_IT()
 - HAL_IRDA_Receive_IT()
 - HAL_IRDA_IRQHandler()
4. Non Blocking mode functions with DMA are :
 - HAL_IRDA_Transmit_DMA()
 - HAL_IRDA_Receive_DMA()
 - HAL_IRDA_DMAPause()
 - HAL_IRDA_DMAResume()
 - HAL_IRDA_DMAStop()
5. A set of Transfer Complete Callbacks are provided in Non Blocking mode:
 - HAL_IRDA_TxHalfCpltCallback()
 - HAL_IRDA_TxCpltCallback()
 - HAL_IRDA_RxHalfCpltCallback()
 - HAL_IRDA_RxCpltCallback()
 - HAL_IRDA_ErrorCallback()
6. Non-Blocking mode transfers could be aborted using Abort API's : (+) HAL_IRDA_Abort()
(+) HAL_IRDA_AbortTransmit() (+) HAL_IRDA_AbortReceive() (+) HAL_IRDA_Abort_IT() (+)
HAL_IRDA_AbortTransmit_IT() (+) HAL_IRDA_AbortReceive_IT()
7. For Abort services based on interrupts (HAL_IRDA_Abortxxx_IT), a set of Abort Complete Callbacks are provided: (+) HAL_IRDA_AbortCpltCallback() (+) HAL_IRDA_AbortTransmitCpltCallback() (+)
HAL_IRDA_AbortReceiveCpltCallback()
8. In Non-Blocking mode transfers, possible errors are split into 2 categories. Errors are handled as follows :
(+) Error is considered as Recoverable and non blocking : Transfer could go till end, but error severity is to be evaluated by user : this concerns Frame Error, Parity Error or Noise Error in Interrupt mode reception . Received character is then retrieved and stored in Rx buffer, Error code is set to allow user to identify error type, and HAL_IRDA_ErrorCallback() user callback is executed. Transfer is kept ongoing on IRDA side. If user wants to abort it, Abort services should be called by user. (+) Error is considered as Blocking : Transfer could not be completed properly and is aborted. This concerns Overrun Error In Interrupt mode reception and all errors in DMA mode. Error code is set to allow user to identify error type, and HAL_IRDA_ErrorCallback() user callback is executed.

This subsection provides a set of functions allowing to manage the IRDA data transfers. IrDA is a half duplex communication protocol. If the Transmitter is busy, any data on the IrDA receive line will be ignored by the IrDA decoder and if the Receiver is busy, data on the TX from the USART to IrDA will not be encoded by IrDA. While receiving data, transmission should be avoided as the data to be transmitted could be corrupted. (#) There are two modes of transfer: (++) Blocking mode: The communication is performed in polling mode. The HAL status of all data processing is returned by the same function after finishing transfer. (++) Non-Blocking mode: The communication is performed using Interrupts or DMA, these API's return the HAL status. The end of the data processing will be indicated through the dedicated IRDA IRQ when using Interrupt mode or the DMA IRQ when using DMA mode. The HAL_IRDA_TxCpltCallback(), HAL_IRDA_RxCpltCallback() user callbacks will be executed respectively at the end of the Transmit or Receive process The HAL_IRDA_ErrorCallback() user callback will be executed when a communication error is detected (#) Blocking mode APIs are : (++) HAL_IRDA_Transmit() (++) HAL_IRDA_Receive() (#) Non Blocking mode APIs with Interrupt are : (++) HAL_IRDA_Transmit_IT() (++) HAL_IRDA_Receive_IT() (++) HAL_IRDA_IRQHandler() (#) Non Blocking mode functions with DMA are : (++) HAL_IRDA_Transmit_DMA() (++) HAL_IRDA_Receive_DMA() (++) HAL_IRDA_DMAPause() (++) HAL_IRDA_DMAResume() (++) HAL_IRDA_DMAStop() (#) A set of Transfer Complete Callbacks are provided in Non Blocking mode: (++) HAL_IRDA_TxHalfCpltCallback() (+) HAL_IRDA_TxCpltCallback() (++) HAL_IRDA_RxHalfCpltCallback() (++) HAL_IRDA_RxCpltCallback() (++) HAL_IRDA_ErrorCallback() (#) Non-Blocking mode transfers could be aborted using Abort API's :

- HAL_IRDA_Abort()
- HAL_IRDA_AbortTransmit()
- HAL_IRDA_AbortReceive()
- HAL_IRDA_Abort_IT()

- HAL_IRDA_AbortTransmit_IT()
- HAL_IRDA_AbortReceive_IT() (#) For Abort services based on interrupts (HAL_IRDA_Abortxxx_IT), a set of Abort Complete Callbacks are provided:
- HAL_IRDA_AbortCpltCallback()
- HAL_IRDA_AbortTransmitCpltCallback()
- HAL_IRDA_AbortReceiveCpltCallback() (#) In Non-Blocking mode transfers, possible errors are split into 2 categories. Errors are handled as follows :
- Error is considered as Recoverable and non blocking : Transfer could go till end, but error severity is to be evaluated by user : this concerns Frame Error, Parity Error or Noise Error in Interrupt mode reception . Received character is then retrieved and stored in Rx buffer, Error code is set to allow user to identify error type, and HAL_IRDA_ErrorCallback() user callback is executed. Transfer is kept ongoing on IRDA side. If user wants to abort it, Abort services should be called by user.
- Error is considered as Blocking : Transfer could not be completed properly and is aborted. This concerns Overrun Error In Interrupt mode reception and all errors in DMA mode. Error code is set to allow user to identify error type, and HAL_IRDA_ErrorCallback() user callback is executed.

This section contains the following APIs:

- **HAL_IRDA_Transmit()**
- **HAL_IRDA_Receive()**
- **HAL_IRDA_Transmit_IT()**
- **HAL_IRDA_Receive_IT()**
- **HAL_IRDA_Transmit_DMA()**
- **HAL_IRDA_Receive_DMA()**
- **HAL_IRDA_DMABuffer()**
- **HAL_IRDA_DMABufferSize()**
- **HAL_IRDA_DMAStop()**
- **HAL_IRDA_Abort()**
- **HAL_IRDA_AbortTransmit()**
- **HAL_IRDA_AbortReceive()**
- **HAL_IRDA_Abort_IT()**
- **HAL_IRDA_AbortTransmit_IT()**
- **HAL_IRDA_AbortReceive_IT()**
- **HAL_IRDA_IRQHandler()**
- **HAL_IRDA_TxCpltCallback()**
- **HAL_IRDA_TxHalfCpltCallback()**
- **HAL_IRDA_RxCpltCallback()**
- **HAL_IRDA_RxHalfCpltCallback()**
- **HAL_IRDA_ErrorCallback()**
- **HAL_IRDA_AbortCpltCallback()**
- **HAL_IRDA_AbortTransmitCpltCallback()**
- **HAL_IRDA_AbortReceiveCpltCallback()**

40.2.5 Peripheral State and Errors functions

This subsection provides a set of functions allowing to return the State of IrDA communication process and also return Peripheral Errors occurred during communication process

- HAL_IRDA_GetState() API can be helpful to check in run-time the state of the IrDA peripheral.
- HAL_IRDA_GetError() check in run-time errors that could be occurred during communication.

This section contains the following APIs:

- **HAL_IRDA_GetState()**
- **HAL_IRDA_GetError()**

40.2.6 Detailed description of functions

HAL_IRDA_Init

Function name

HAL_StatusTypeDef HAL_IRDA_Init (IRDA_HandleTypeDef * hirda)

Function description

Initializes the IRDA mode according to the specified parameters in the IRDA_InitTypeDef and create the associated handle.

Parameters

- **hirda**: Pointer to a IRDA_HandleTypeDef structure that contains the configuration information for the specified IRDA module.

Return values

- **HAL**: status

HAL_IRDA_DeInit

Function name

HAL_StatusTypeDef HAL_IRDA_DeInit (IRDA_HandleTypeDef * hirda)

Function description

DeInitializes the IRDA peripheral.

Parameters

- **hirda**: Pointer to a IRDA_HandleTypeDef structure that contains the configuration information for the specified IRDA module.

Return values

- **HAL**: status

HAL_IRDA_MspInit

Function name

void HAL_IRDA_MspInit (IRDA_HandleTypeDef * hirda)

Function description

IRDA MSP Init.

Parameters

- **hirda**: Pointer to a IRDA_HandleTypeDef structure that contains the configuration information for the specified IRDA module.

Return values

- **None**:

HAL_IRDA_MspDeInit

Function name

void HAL_IRDA_MspDeInit (IRDA_HandleTypeDef * hirda)

Function description

IRDA MSP DeInit.

Parameters

- **hirda:** Pointer to a `IRDA_HandleTypeDef` structure that contains the configuration information for the specified IRDA module.

Return values

- **None:**

HAL_IRDA_RegisterCallback

Function name

HAL_StatusTypeDef HAL_IRDA_RegisterCallback (IRDA_HandleTypeDef * hirda, HAL_IRDA_CallbackIDTypeDef CallbackID, pIRDA_CallbackTypeDef pCallback)

Function description

Register a User IRDA Callback To be used instead of the weak predefined callback.

Parameters

- **hirda:** irda handle
- **CallbackID:** ID of the callback to be registered This parameter can be one of the following values:
 - `HAL_IRDA_TX_HALFCOMPLETE_CB_ID` Tx Half Complete Callback ID
 - `HAL_IRDA_TX_COMPLETE_CB_ID` Tx Complete Callback ID
 - `HAL_IRDA_RX_HALFCOMPLETE_CB_ID` Rx Half Complete Callback ID
 - `HAL_IRDA_RX_COMPLETE_CB_ID` Rx Complete Callback ID
 - `HAL_IRDA_ERROR_CB_ID` Error Callback ID
 - `HAL_IRDA_ABORT_COMPLETE_CB_ID` Abort Complete Callback ID
 - `HAL_IRDA_ABORT_TRANSMIT_COMPLETE_CB_ID` Abort Transmit Complete Callback ID
 - `HAL_IRDA_ABORT_RECEIVE_COMPLETE_CB_ID` Abort Receive Complete Callback ID
 - `HAL_IRDA_MSPINIT_CB_ID` MspInit Callback ID
 - `HAL_IRDA_MSPDEINIT_CB_ID` MspDeInit Callback ID
- **pCallback:** pointer to the Callback function

Return values

- **HAL:** status

Notes

- The `HAL_IRDA_RegisterCallback()` may be called before `HAL_IRDA_Init()` in `HAL_IRDA_STATE_RESET` to register callbacks for `HAL_IRDA_MSPINIT_CB_ID` and `HAL_IRDA_MSPDEINIT_CB_ID`

HAL_IRDA_UnRegisterCallback

Function name

HAL_StatusTypeDef HAL_IRDA_UnRegisterCallback (IRDA_HandleTypeDef * hirda, HAL_IRDA_CallbackIDTypeDef CallbackID)

Function description

Unregister an IRDA callback IRDA callback is redirected to the weak predefined callback.

Parameters

- **hirda:** irda handle
- **CallbackID:** ID of the callback to be unregistered This parameter can be one of the following values:
 - HAL_IRDA_TX_HALFCOMPLETE_CB_ID Tx Half Complete Callback ID
 - HAL_IRDA_TX_COMPLETE_CB_ID Tx Complete Callback ID
 - HAL_IRDA_RX_HALFCOMPLETE_CB_ID Rx Half Complete Callback ID
 - HAL_IRDA_RX_COMPLETE_CB_ID Rx Complete Callback ID
 - HAL_IRDA_ERROR_CB_ID Error Callback ID
 - HAL_IRDA_ABORT_COMPLETE_CB_ID Abort Complete Callback ID
 - HAL_IRDA_ABORT_TRANSMIT_COMPLETE_CB_ID Abort Transmit Complete Callback ID
 - HAL_IRDA_ABORT_RECEIVE_COMPLETE_CB_ID Abort Receive Complete Callback ID
 - HAL_IRDA_MSPINIT_CB_ID MspInit Callback ID
 - HAL_IRDA_MSPDEINIT_CB_ID MspDeInit Callback ID

Return values

- **HAL:** status

Notes

- The HAL_IRDA_UnRegisterCallback() may be called before HAL_IRDA_Init() in HAL_IRDA_STATE_RESET to un-register callbacks for HAL_IRDA_MSPINIT_CB_ID and HAL_IRDA_MSPDEINIT_CB_ID

HAL_IRDA_Transmit

Function name

HAL_StatusTypeDef HAL_IRDA_Transmit (IRDA_HandleTypeDef * hirda, const uint8_t * pData, uint16_t Size, uint32_t Timeout)

Function description

Sends an amount of data in blocking mode.

Parameters

- **hirda:** Pointer to a IRDA_HandleTypeDef structure that contains the configuration information for the specified IRDA module.
- **pData:** Pointer to data buffer (u8 or u16 data elements).
- **Size:** Amount of data elements (u8 or u16) to be sent.
- **Timeout:** Specify timeout value.

Return values

- **HAL:** status

Notes

- When UART parity is not enabled (PCE = 0), and Word Length is configured to 9 bits (M1-M0 = 01), the sent data is handled as a set of u16. In this case, Size must reflect the number of u16 available through pData.

HAL_IRDA_Receive

Function name

HAL_StatusTypeDef HAL_IRDA_Receive (IRDA_HandleTypeDef * hirda, uint8_t * pData, uint16_t Size, uint32_t Timeout)

Function description

Receive an amount of data in blocking mode.

Parameters

- **hirda**: Pointer to a `IRDA_HandleTypeDef` structure that contains the configuration information for the specified IRDA module.
- **pData**: Pointer to data buffer (u8 or u16 data elements).
- **Size**: Amount of data elements (u8 or u16) to be received.
- **Timeout**: Specify timeout value

Return values

- **HAL**: status

Notes

- When UART parity is not enabled ($PCE = 0$), and Word Length is configured to 9 bits ($M1-M0 = 01$), the received data is handled as a set of u16. In this case, Size must reflect the number of u16 available through pData.

HAL_IRDA_Transmit_IT

Function name

HAL_StatusTypeDef HAL_IRDA_Transmit_IT (`IRDA_HandleTypeDef * hirda`, `const uint8_t * pData`, `uint16_t Size`)

Function description

Send an amount of data in non blocking mode.

Parameters

- **hirda**: Pointer to a `IRDA_HandleTypeDef` structure that contains the configuration information for the specified IRDA module.
- **pData**: Pointer to data buffer (u8 or u16 data elements).
- **Size**: Amount of data elements (u8 or u16) to be sent.

Return values

- **HAL**: status

Notes

- When UART parity is not enabled ($PCE = 0$), and Word Length is configured to 9 bits ($M1-M0 = 01$), the sent data is handled as a set of u16. In this case, Size must reflect the number of u16 available through pData.

HAL_IRDA_Receive_IT

Function name

HAL_StatusTypeDef HAL_IRDA_Receive_IT (`IRDA_HandleTypeDef * hirda`, `uint8_t * pData`, `uint16_t Size`)

Function description

Receive an amount of data in non blocking mode.

Parameters

- **hirda**: Pointer to a `IRDA_HandleTypeDef` structure that contains the configuration information for the specified IRDA module.
- **pData**: Pointer to data buffer (u8 or u16 data elements).
- **Size**: Amount of data elements (u8 or u16) to be received.

Return values

- **HAL**: status

Notes

- When UART parity is not enabled (PCE = 0), and Word Length is configured to 9 bits (M1-M0 = 01), the received data is handled as a set of u16. In this case, Size must reflect the number of u16 available through pData.

HAL_IRDA_Transmit_DMA

Function name

HAL_StatusTypeDef HAL_IRDA_Transmit_DMA (IRDA_HandleTypeDef * hirda, const uint8_t * pData, uint16_t Size)

Function description

Send an amount of data in DMA mode.

Parameters

- hirda**: Pointer to a IRDA_HandleTypeDef structure that contains the configuration information for the specified IRDA module.
- pData**: Pointer to data buffer (u8 or u16 data elements).
- Size**: Amount of data elements (u8 or u16) to be sent.

Return values

- HAL**: status

Notes

- When UART parity is not enabled (PCE = 0), and Word Length is configured to 9 bits (M1-M0 = 01), the sent data is handled as a set of u16. In this case, Size must reflect the number of u16 available through pData.

HAL_IRDA_Receive_DMA

Function name

HAL_StatusTypeDef HAL_IRDA_Receive_DMA (IRDA_HandleTypeDef * hirda, uint8_t * pData, uint16_t Size)

Function description

Receives an amount of data in DMA mode.

Parameters

- hirda**: Pointer to a IRDA_HandleTypeDef structure that contains the configuration information for the specified IRDA module.
- pData**: Pointer to data buffer (u8 or u16 data elements).
- Size**: Amount of data elements (u8 or u16) to be received.

Return values

- HAL**: status

Notes

- When UART parity is not enabled (PCE = 0), and Word Length is configured to 9 bits (M1-M0 = 01), the received data is handled as a set of u16. In this case, Size must reflect the number of u16 available through pData.
- When the IRDA parity is enabled (PCE = 1) the data received contain the parity bit.

HAL_IRDA_DMAPause

Function name

HAL_StatusTypeDef HAL_IRDA_DMAPause (IRDA_HandleTypeDef * hirda)

Function description

Pauses the DMA Transfer.

Parameters

- **hirda**: Pointer to a `IRDA_HandleTypeDef` structure that contains the configuration information for the specified IRDA module.

Return values

- **HAL**: status

HAL_IRDA_DMAResume

Function name

`HAL_StatusTypeDef HAL_IRDA_DMAResume (IRDA_HandleTypeDef * hirda)`

Function description

Resumes the DMA Transfer.

Parameters

- **hirda**: Pointer to a `IRDA_HandleTypeDef` structure that contains the configuration information for the specified IRDA module.

Return values

- **HAL**: status

HAL_IRDA_DMAStop

Function name

`HAL_StatusTypeDef HAL_IRDA_DMAStop (IRDA_HandleTypeDef * hirda)`

Function description

Stops the DMA Transfer.

Parameters

- **hirda**: Pointer to a `IRDA_HandleTypeDef` structure that contains the configuration information for the specified IRDA module.

Return values

- **HAL**: status

HAL_IRDA_Abort

Function name

`HAL_StatusTypeDef HAL_IRDA_Abort (IRDA_HandleTypeDef * hirda)`

Function description

Abort ongoing transfers (blocking mode).

Parameters

- **hirda**: IRDA handle.

Return values

- **HAL**: status

Notes

- This procedure could be used for aborting any ongoing transfer started in Interrupt or DMA mode. This procedure performs following operations : Disable PPP InterruptsDisable the DMA transfer in the peripheral register (if enabled)Abort DMA transfer by calling HAL_DMA_Abort (in case of transfer in DMA mode)Set handle State to READY
- This procedure is executed in blocking mode : when exiting function, Abort is considered as completed.

HAL_IRDA_AbortTransmit

Function name

HAL_StatusTypeDef HAL_IRDA_AbortTransmit (IRDA_HandleTypeDef * hirda)

Function description

Abort ongoing Transmit transfer (blocking mode).

Parameters

- **hirda**: IRDA handle.

Return values

- **HAL**: status

Notes

- This procedure could be used for aborting any ongoing transfer started in Interrupt or DMA mode. This procedure performs following operations : Disable PPP InterruptsDisable the DMA transfer in the peripheral register (if enabled)Abort DMA transfer by calling HAL_DMA_Abort (in case of transfer in DMA mode)Set handle State to READY
- This procedure is executed in blocking mode : when exiting function, Abort is considered as completed.

HAL_IRDA_AbortReceive

Function name

HAL_StatusTypeDef HAL_IRDA_AbortReceive (IRDA_HandleTypeDef * hirda)

Function description

Abort ongoing Receive transfer (blocking mode).

Parameters

- **hirda**: IRDA handle.

Return values

- **HAL**: status

Notes

- This procedure could be used for aborting any ongoing transfer started in Interrupt or DMA mode. This procedure performs following operations : Disable PPP InterruptsDisable the DMA transfer in the peripheral register (if enabled)Abort DMA transfer by calling HAL_DMA_Abort (in case of transfer in DMA mode)Set handle State to READY
- This procedure is executed in blocking mode : when exiting function, Abort is considered as completed.

HAL_IRDA_Abort_IT

Function name

HAL_StatusTypeDef HAL_IRDA_Abort_IT (IRDA_HandleTypeDef * hirda)

Function description

Abort ongoing transfers (Interrupt mode).

Parameters

- **hirda:** IRDA handle.

Return values

- **HAL:** status

Notes

- This procedure could be used for aborting any ongoing transfer started in Interrupt or DMA mode. This procedure performs following operations : Disable PPP Interrupts
Disable the DMA transfer in the peripheral register (if enabled)
Abort DMA transfer by calling HAL_DMA_Abort_IT (in case of transfer in DMA mode)
Set handle State to READY
At abort completion, call user abort complete callback
- This procedure is executed in Interrupt mode, meaning that abort procedure could be considered as completed only when user abort complete callback is executed (not when exiting function).

HAL_IRDA_AbortTransmit_IT

Function name

HAL_StatusTypeDef HAL_IRDA_AbortTransmit_IT (IRDA_HandleTypeDef * hirda)

Function description

Abort ongoing Transmit transfer (Interrupt mode).

Parameters

- **hirda:** IRDA handle.

Return values

- **HAL:** status

Notes

- This procedure could be used for aborting any ongoing transfer started in Interrupt or DMA mode. This procedure performs following operations : Disable IRDA Interrupts (Tx)
Disable the DMA transfer in the peripheral register (if enabled)
Abort DMA transfer by calling HAL_DMA_Abort_IT (in case of transfer in DMA mode)
Set handle State to READY
At abort completion, call user abort complete callback
- This procedure is executed in Interrupt mode, meaning that abort procedure could be considered as completed only when user abort complete callback is executed (not when exiting function).

HAL_IRDA_AbortReceive_IT

Function name

HAL_StatusTypeDef HAL_IRDA_AbortReceive_IT (IRDA_HandleTypeDef * hirda)

Function description

Abort ongoing Receive transfer (Interrupt mode).

Parameters

- **hirda:** IRDA handle.

Return values

- **HAL:** status

Notes

- This procedure could be used for aborting any ongoing transfer started in Interrupt or DMA mode. This procedure performs following operations : Disable PPP Interrupts
Disable the DMA transfer in the peripheral register (if enabled)
Abort DMA transfer by calling HAL_DMA_Abort_IT (in case of transfer in DMA mode)
Set handle State to READY
At abort completion, call user abort complete callback
- This procedure is executed in Interrupt mode, meaning that abort procedure could be considered as completed only when user abort complete callback is executed (not when exiting function).

HAL_IRDA_IRQHandler

Function name

void HAL_IRDA_IRQHandler (IRDA_HandleTypeDef * hirda)

Function description

This function handles IRDA interrupt request.

Parameters

- **hirda:** Pointer to a IRDA_HandleTypeDef structure that contains the configuration information for the specified IRDA module.

Return values

- **None:**

HAL_IRDA_TxCpltCallback

Function name

void HAL_IRDA_TxCpltCallback (IRDA_HandleTypeDef * hirda)

Function description

Tx Transfer complete callback.

Parameters

- **hirda:** Pointer to a IRDA_HandleTypeDef structure that contains the configuration information for the specified IRDA module.

Return values

- **None:**

HAL_IRDA_RxCpltCallback

Function name

void HAL_IRDA_RxCpltCallback (IRDA_HandleTypeDef * hirda)

Function description

Rx Transfer complete callback.

Parameters

- **hirda:** Pointer to a IRDA_HandleTypeDef structure that contains the configuration information for the specified IRDA module.

Return values

- **None:**

HAL_IRDA_TxHalfCpltCallback

Function name

void HAL_IRDA_TxHalfCpltCallback (IRDA_HandleTypeDef * hirda)

Function description

Tx Half Transfer completed callback.

Parameters

- **hirda:** Pointer to a IRDA_HandleTypeDef structure that contains the configuration information for the specified USART module.

Return values

- **None:**

HAL_IRDA_RxHalfCpltCallback**Function name****void HAL_IRDA_RxHalfCpltCallback (IRDA_HandleTypeDef * hirda)****Function description**

Rx Half Transfer complete callback.

Parameters

- **hirda:** Pointer to a IRDA_HandleTypeDef structure that contains the configuration information for the specified IRDA module.

Return values

- **None:**

HAL_IRDA_ErrorCallback**Function name****void HAL_IRDA_ErrorCallback (IRDA_HandleTypeDef * hirda)****Function description**

IRDA error callback.

Parameters

- **hirda:** Pointer to a IRDA_HandleTypeDef structure that contains the configuration information for the specified IRDA module.

Return values

- **None:**

HAL_IRDA_AbortCpltCallback**Function name****void HAL_IRDA_AbortCpltCallback (IRDA_HandleTypeDef * hirda)****Function description**

IRDA Abort Complete callback.

Parameters

- **hirda:** Pointer to a IRDA_HandleTypeDef structure that contains the configuration information for the specified IRDA module.

Return values

- **None:**

HAL_IRDA_AbortTransmitCpltCallback**Function name****void HAL_IRDA_AbortTransmitCpltCallback (IRDA_HandleTypeDef * hirda)****Function description**

IRDA Abort Transmit Complete callback.

Parameters

- **hirda**: Pointer to a `IRDA_HandleTypeDef` structure that contains the configuration information for the specified IRDA module.

Return values

- **None**:

HAL_IRDA_AbortReceiveCpltCallback

Function name

void HAL_IRDA_AbortReceiveCpltCallback (IRDA_HandleTypeDef * hirda)

Function description

IRDA Abort Receive Complete callback.

Parameters

- **hirda**: Pointer to a `IRDA_HandleTypeDef` structure that contains the configuration information for the specified IRDA module.

Return values

- **None**:

HAL_IRDA_GetState

Function name

HAL_IRDA_StateTypeDef HAL_IRDA_GetState (const IRDA_HandleTypeDef * hirda)

Function description

Return the IRDA state.

Parameters

- **hirda**: Pointer to a `IRDA_HandleTypeDef` structure that contains the configuration information for the specified IRDA.

Return values

- **HAL**: state

HAL_IRDA_GetError

Function name

uint32_t HAL_IRDA_GetError (const IRDA_HandleTypeDef * hirda)

Function description

Return the IRDA error code.

Parameters

- **hirda**: Pointer to a `IRDA_HandleTypeDef` structure that contains the configuration information for the specified IRDA.

Return values

- **IRDA**: Error Code

40.3 IRDA Firmware driver defines

The following section lists the various define and macros of the module.

40.3.1 IRDA

IRDA

IRDA Error Code

HAL_IRDA_ERROR_NONE

No error

HAL_IRDA_ERROR_PE

Parity error

HAL_IRDA_ERROR_NE

Noise error

HAL_IRDA_ERROR_FE

Frame error

HAL_IRDA_ERROR_ORE

Overrun error

HAL_IRDA_ERROR_DMA

DMA transfer error

HAL_IRDA_ERROR_INVALID_CALLBACK

Invalid Callback error

IRDA Exported Macros

__HAL_IRDA_RESET_HANDLE_STATE

Description:

- Reset IRDA handle gstate & RxState.

Parameters:

- `__HANDLE__`: specifies the IRDA Handle. IRDA Handle selects the USARTx or UARTy peripheral (USART,UART availability and x,y values depending on device).

Return value:

- None

__HAL_IRDA_FLUSH_DRREGISTER

Description:

- Flush the IRDA DR register.

Parameters:

- `__HANDLE__`: specifies the IRDA Handle. IRDA Handle selects the USARTx or UARTy peripheral (USART,UART availability and x,y values depending on device).

Return value:

- None

__HAL_IRDA_GET_FLAG

Description:

- Check whether the specified IRDA flag is set or not.

Parameters:

- `__HANDLE__`: specifies the IRDA Handle. IRDA Handle selects the USARTx or UARTy peripheral (USART,UART availability and x,y values depending on device).
- `__FLAG__`: specifies the flag to check. This parameter can be one of the following values:
 - `IRDA_FLAG_TXE`: Transmit data register empty flag
 - `IRDA_FLAG_TC`: Transmission Complete flag
 - `IRDA_FLAG_RXNE`: Receive data register not empty flag
 - `IRDA_FLAG_IDLE`: Idle Line detection flag
 - `IRDA_FLAG_ORE`: OverRun Error flag
 - `IRDA_FLAG_NE`: Noise Error flag
 - `IRDA_FLAG_FE`: Framing Error flag
 - `IRDA_FLAG_PE`: Parity Error flag

Return value:

- The: new state of `__FLAG__` (TRUE or FALSE).

__HAL_IRDA_CLEAR_FLAG

Description:

- Clear the specified IRDA pending flag.

Parameters:

- `__HANDLE__`: specifies the IRDA Handle. IRDA Handle selects the USARTx or UARTy peripheral (USART,UART availability and x,y values depending on device).
- `__FLAG__`: specifies the flag to check. This parameter can be any combination of the following values:
 - `IRDA_FLAG_TC`: Transmission Complete flag.
 - `IRDA_FLAG_RXNE`: Receive data register not empty flag.

Return value:

- None

Notes:

- PE (Parity error), FE (Framing error), NE (Noise error), ORE (OverRun error) and IDLE (Idle line detected) flags are cleared by software sequence: a read operation to USART_SR register followed by a read operation to USART_DR register. RXNE flag can be also cleared by a read to the USART_DR register. TC flag can be also cleared by software sequence: a read operation to USART_SR register followed by a write operation to USART_DR register. TXE flag is cleared only by a write to the USART_DR register.

__HAL_IRDA_CLEAR_PFLAG

Description:

- Clear the IRDA PE pending flag.

Parameters:

- `__HANDLE__`: specifies the IRDA Handle. IRDA Handle selects the USARTx or UARTy peripheral (USART,UART availability and x,y values depending on device).

Return value:

- None

__HAL_IRDA_CLEAR_FEFLAG**Description:**

- Clear the IRDA FE pending flag.

Parameters:

- `__HANDLE__`: specifies the IRDA Handle. IRDA Handle selects the USARTx or UARTy peripheral (USART,UART availability and x,y values depending on device).

Return value:

- None

__HAL_IRDA_CLEAR_NEFLAG**Description:**

- Clear the IRDA NE pending flag.

Parameters:

- `__HANDLE__`: specifies the IRDA Handle. IRDA Handle selects the USARTx or UARTy peripheral (USART,UART availability and x,y values depending on device).

Return value:

- None

__HAL_IRDA_CLEAR_OREFLAG**Description:**

- Clear the IRDA ORE pending flag.

Parameters:

- `__HANDLE__`: specifies the IRDA Handle. IRDA Handle selects the USARTx or UARTy peripheral (USART,UART availability and x,y values depending on device).

Return value:

- None

__HAL_IRDA_CLEAR_IDLEFLAG**Description:**

- Clear the IRDA IDLE pending flag.

Parameters:

- `__HANDLE__`: specifies the IRDA Handle. IRDA Handle selects the USARTx or UARTy peripheral (USART,UART availability and x,y values depending on device).

Return value:

- None

__HAL_IRDA_ENABLE_IT

Description:

- Enable the specified IRDA interrupt.

Parameters:

- `__HANDLE__`: specifies the IRDA Handle. IRDA Handle selects the USARTx or UARTy peripheral (USART,UART availability and x,y values depending on device).
- `__INTERRUPT__`: specifies the IRDA interrupt source to enable. This parameter can be one of the following values:
 - `IRDA_IT_TXE`: Transmit Data Register empty interrupt
 - `IRDA_IT_TC`: Transmission complete interrupt
 - `IRDA_IT_RXNE`: Receive Data register not empty interrupt
 - `IRDA_IT_IDLE`: Idle line detection interrupt
 - `IRDA_IT_PE`: Parity Error interrupt
 - `IRDA_IT_ERR`: Error interrupt(Frame error, noise error, overrun error)

Return value:

- None

__HAL_IRDA_DISABLE_IT

Description:

- Disable the specified IRDA interrupt.

Parameters:

- `__HANDLE__`: specifies the IRDA Handle. IRDA Handle selects the USARTx or UARTy peripheral (USART,UART availability and x,y values depending on device).
- `__INTERRUPT__`: specifies the IRDA interrupt source to disable. This parameter can be one of the following values:
 - `IRDA_IT_TXE`: Transmit Data Register empty interrupt
 - `IRDA_IT_TC`: Transmission complete interrupt
 - `IRDA_IT_RXNE`: Receive Data register not empty interrupt
 - `IRDA_IT_IDLE`: Idle line detection interrupt
 - `IRDA_IT_PE`: Parity Error interrupt
 - `IRDA_IT_ERR`: Error interrupt(Frame error, noise error, overrun error)

Return value:

- None

__HAL_IRDA_GET_IT_SOURCE

Description:

- Check whether the specified IRDA interrupt has occurred or not.

Parameters:

- `__HANDLE__`: specifies the IRDA Handle. IRDA Handle selects the USARTx or UARTy peripheral (USART,UART availability and x,y values depending on device).
- `__IT__`: specifies the IRDA interrupt source to check. This parameter can be one of the following values:
 - `IRDA_IT_TXE`: Transmit Data Register empty interrupt
 - `IRDA_IT_TC`: Transmission complete interrupt
 - `IRDA_IT_RXNE`: Receive Data register not empty interrupt
 - `IRDA_IT_IDLE`: Idle line detection interrupt
 - `IRDA_IT_ERR`: Error interrupt
 - `IRDA_IT_PE`: Parity Error interrupt

Return value:

- The: new state of `__IT__` (TRUE or FALSE).

__HAL_IRDA_ONE_BIT_SAMPLE_ENABLE**Description:**

- Macro to enable the IRDA's one bit sample method.

Parameters:

- `__HANDLE__`: specifies the IRDA Handle.

Return value:

- None

__HAL_IRDA_ONE_BIT_SAMPLE_DISABLE**Description:**

- Macro to disable the IRDA's one bit sample method.

Parameters:

- `__HANDLE__`: specifies the IRDA Handle.

Return value:

- None

__HAL_IRDA_ENABLE**Description:**

- Enable UART/USART associated to IRDA Handle.

Parameters:

- `__HANDLE__`: specifies the IRDA Handle. IRDA Handle selects the USARTx or UARTy peripheral (USART, UART availability and x,y values depending on device).

Return value:

- None

__HAL_IRDA_DISABLE**Description:**

- Disable UART/USART associated to IRDA Handle.

Parameters:

- `__HANDLE__`: specifies the IRDA Handle. IRDA Handle selects the USARTx or UARTy peripheral (USART, UART availability and x,y values depending on device).

Return value:

- None

IRDA Flags**IRDA_FLAG_TXE****IRDA_FLAG_TC****IRDA_FLAG_RXNE****IRDA_FLAG_IDLE****IRDA_FLAG_ORE****IRDA_FLAG_NE****IRDA_FLAG_FE****IRDA_FLAG_PE****IRDA Interrupt Definitions**

IRDA_IT_PE

IRDA_IT_TXE

IRDA_IT_TC

IRDA_IT_RXNE

IRDA_IT_IDLE

IRDA_IT_LBD

IRDA_IT_CTS

IRDA_IT_ERR

IRDA Low Power

IRDA_POWERMODE_LOWPOWER

IRDA_POWERMODE_NORMAL

IRDA Transfer Mode

IRDA_MODE_RX

IRDA_MODE_TX

IRDA_MODE_TX_RX

IRDA Parity

IRDA_PARITY_NONE

IRDA_PARITY_EVEN

IRDA_PARITY_ODD

IRDA Word Length

IRDA_WORDLENGTH_8B

IRDA_WORDLENGTH_9B

41 HAL IWDG Generic Driver

41.1 IWDG Firmware driver registers structures

41.1.1 IWDG_InitTypeDef

IWDG_InitTypeDef is defined in the stm32f4xx_hal_iwdg.h

Data Fields

- *uint32_t Prescaler*
- *uint32_t Reload*

Field Documentation

- *uint32_t IWDG_InitTypeDef::Prescaler*
Select the prescaler of the IWDG. This parameter can be a value of *IWDG_Prescaler*
- *uint32_t IWDG_InitTypeDef::Reload*
Specifies the IWDG down-counter reload value. This parameter must be a number between Min_Data = 0 and Max_Data = 0x0FFF

41.1.2 IWDG_HandleTypeDef

IWDG_HandleTypeDef is defined in the stm32f4xx_hal_iwdg.h

Data Fields

- *IWDG_TypeDef * Instance*
- *IWDG_InitTypeDef Init*

Field Documentation

- *IWDG_TypeDef* IWDG_HandleTypeDef::Instance*
Register base address
- *IWDG_InitTypeDef IWDG_HandleTypeDef::Init*
IWDG required parameters

41.2 IWDG Firmware driver API description

The following section lists the various functions of the IWDG library.

41.2.1 IWDG Generic features

- The IWDG can be started by either software or hardware (configurable through option byte).
- The IWDG is clocked by the Low-Speed Internal clock (LSI) and thus stays active even if the main clock fails.
- Once the IWDG is started, the LSI is forced ON and both cannot be disabled. The counter starts counting down from the reset value (0xFFFF). When it reaches the end of count value (0x000) a reset signal is generated (IWDG reset).
- Whenever the key value 0x0000 AAAA is written in the IWDG_KR register, the IWDG_RLR value is reloaded into the counter and the watchdog reset is prevented.
- The IWDG is implemented in the VDD voltage domain that is still functional in STOP and STANDBY mode (IWDG reset can wake up the CPU from STANDBY). IWDGRST flag in RCC_CSR register can be used to inform when an IWDG reset occurs.
- Debug mode: When the microcontroller enters debug mode (core halted), the IWDG counter either continues to work normally or stops, depending on DBG_IWDG_STOP configuration bit in DBG module, accessible through `__HAL_DBGMCU_FREEZE_IWDG()` and `__HAL_DBGMCU_UNFREEZE_IWDG()` macros.

Min-max timeout value @32KHz (LSI): ~125us / ~32.7s The IWDG timeout may vary due to LSI clock frequency dispersion. STM32F4xx devices provide the capability to measure the LSI clock frequency (LSI clock is internally connected to TIM5 CH4 input capture). The measured value can be used to have an IWDG timeout with an acceptable accuracy.

Default timeout value (necessary for IWDG_SR status register update): Constant LSI_VALUE is defined based on the nominal LSI clock frequency. This frequency being subject to variations as mentioned above, the default timeout value (defined through constant HAL_IWDG_DEFAULT_TIMEOUT below) may become too short or too long. In such cases, this default timeout value can be tuned by redefining the constant LSI_VALUE at user-application level (based, for instance, on the measured LSI clock frequency as explained above).

41.2.2 How to use this driver

1. Use IWDG using HAL_IWDG_Init() function to :
 - Enable instance by writing Start keyword in IWDG_KEY register. LSI clock is forced ON and IWDG counter starts counting down.
 - Enable write access to configuration registers: IWDG_PR and IWDG_RLR.
 - Configure the IWDG prescaler and counter reload value. This reload value will be loaded in the IWDG counter each time the watchdog is reloaded, then the IWDG will start counting down from this value.
 - Wait for status flags to be reset.
2. Then the application program must refresh the IWDG counter at regular intervals during normal operation to prevent an MCU reset, using HAL_IWDG_Refresh() function.

IWDG HAL driver macros list

Below the list of most used macros in IWDG HAL driver:

- `__HAL_IWDG_START`: Enable the IWDG peripheral
- `__HAL_IWDG_RELOAD_COUNTER`: Reloads IWDG counter with value defined in the reload register

41.2.3 Initialization and Start functions

This section provides functions allowing to:

- Initialize the IWDG according to the specified parameters in the IWDG_InitTypeDef of associated handle.
- Once initialization is performed in HAL_IWDG_Init function, Watchdog is reloaded in order to exit function with correct time base.

This section contains the following APIs:

- [*HAL_IWDG_Init\(\)*](#)

41.2.4 IO operation functions

This section provides functions allowing to:

- Refresh the IWDG.

This section contains the following APIs:

- [*HAL_IWDG_Refresh\(\)*](#)

41.2.5 Detailed description of functions

HAL_IWDG_Init

Function name

HAL_StatusTypeDef HAL_IWDG_Init (IWDG_HandleTypeDef * hiwdg)

Function description

Initialize the IWDG according to the specified parameters in the IWDG_InitTypeDef and start watchdog.

Parameters

- **hiwdg**: pointer to a IWDG_HandleTypeDef structure that contains the configuration information for the specified IWDG module.

Return values

- **HAL**: status

HAL_IWDG_Refresh

Function name

HAL_StatusTypeDef HAL_IWDG_Refresh (IWDG_HandleTypeDef * hiwdg)

Function description

Refresh the IWDG.

Parameters

- **hiwdg**: pointer to a IWDG_HandleTypeDef structure that contains the configuration information for the specified IWDG module.

Return values

- **HAL**: status

41.3 IWDG Firmware driver defines

The following section lists the various define and macros of the module.

41.3.1 IWDG

IWDG

IWDG Exported Macros

__HAL_IWDG_START

Description:

- Enable the IWDG peripheral.

Parameters:

- **__HANDLE__**: IWDG handle

Return value:

- None

__HAL_IWDG_RELOAD_COUNTER

Description:

- Reload IWDG counter with value defined in the reload register (write access to IWDG_PR and IWDG_RLR registers disabled).

Parameters:

- **__HANDLE__**: IWDG handle

Return value:

- None

IWDG Prescaler

IWDG_PRESCALER_4

IWDG prescaler set to 4

IWDG_PRESCALER_8

IWDG prescaler set to 8

IWDG_PRESCALER_16

IWDG prescaler set to 16

IWDG_PRESCALER_32

IWDG prescaler set to 32

IWDG_PRESCALER_64

IWDG prescaler set to 64

IWDG_PRESCALER_128

IWDG prescaler set to 128

IWDG_PRESCALER_256

IWDG prescaler set to 256

42 HAL LPTIM Generic Driver

42.1 LPTIM Firmware driver registers structures

42.1.1 LPTIM_ClockConfigTypeDef

LPTIM_ClockConfigTypeDef is defined in the stm32f4xx_hal_lptim.h

Data Fields

- **uint32_t Source**
- **uint32_t Prescaler**

Field Documentation

- **uint32_t LPTIM_ClockConfigTypeDef::Source**
Selects the clock source. This parameter can be a value of **LPTIM_Clock_Source**
- **uint32_t LPTIM_ClockConfigTypeDef::Prescaler**
Specifies the counter clock Prescaler. This parameter can be a value of **LPTIM_Clock_Prescaler**

42.1.2 LPTIM_ULPClockConfigTypeDef

LPTIM_ULPClockConfigTypeDef is defined in the stm32f4xx_hal_lptim.h

Data Fields

- **uint32_t Polarity**
- **uint32_t SampleTime**

Field Documentation

- **uint32_t LPTIM_ULPClockConfigTypeDef::Polarity**
Selects the polarity of the active edge for the counter unit if the ULPTIM input is selected. Note: This parameter is used only when Ultra low power clock source is used. Note: If the polarity is configured on 'both edges', an auxiliary clock (one of the Low power oscillator) must be active. This parameter can be a value of **LPTIM_Clock_Polarity**
- **uint32_t LPTIM_ULPClockConfigTypeDef::SampleTime**
Selects the clock sampling time to configure the clock glitch filter. Note: This parameter is used only when Ultra low power clock source is used. This parameter can be a value of **LPTIM_Clock_Sample_Time**

42.1.3 LPTIM_TriggerConfigTypeDef

LPTIM_TriggerConfigTypeDef is defined in the stm32f4xx_hal_lptim.h

Data Fields

- **uint32_t Source**
- **uint32_t ActiveEdge**
- **uint32_t SampleTime**

Field Documentation

- **uint32_t LPTIM_TriggerConfigTypeDef::Source**
Selects the Trigger source. This parameter can be a value of **LPTIM_Trigger_Source**
- **uint32_t LPTIM_TriggerConfigTypeDef::ActiveEdge**
Selects the Trigger active edge. Note: This parameter is used only when an external trigger is used. This parameter can be a value of **LPTIM_External_Trigger_Polarity**
- **uint32_t LPTIM_TriggerConfigTypeDef::SampleTime**
Selects the trigger sampling time to configure the clock glitch filter. Note: This parameter is used only when an external trigger is used. This parameter can be a value of **LPTIM_Trigger_Sample_Time**

42.1.4 LPTIM_InitTypeDef

LPTIM_InitTypeDef is defined in the stm32f4xx_hal_lptim.h

Data Fields

- **LPTIM_ClockConfigTypeDef Clock**
- **LPTIM_ULPClockConfigTypeDef UltraLowPowerClock**

49 HAL PCD Generic Driver

49.1 PCD Firmware driver registers structures

49.1.1 PCD_HandleTypeDef

PCD_HandleTypeDef is defined in the stm32f4xx_hal_pcd.h

Data Fields

- **PCD_TypeDef* Instance**
- **PCD_InitTypeDef Init**
- **__IO uint8_t USB_Address**
- **PCD_EPTypeDef IN_ep**
- **PCD_EPTypeDef OUT_ep**
- **HAL_LockTypeDef Lock**
- **__IO PCD_StateTypeDef State**
- **__IO uint32_t ErrorCode**
- **uint32_t Setup**
- **PCD_LPM_StateTypeDef LPM_State**
- **uint32_t BESL**
- **uint32_t lpm_active**
- **uint32_t battery_charging_active**
- **void* pData**

Field Documentation

- **PCD_TypeDef* PCD_HandleTypeDef::Instance**
Register base address
- **PCD_InitTypeDef PCD_HandleTypeDef::Init**
PCD required parameters
- **__IO uint8_t PCD_HandleTypeDef::USB_Address**
USB Address
- **PCD_EPTypeDef PCD_HandleTypeDef::IN_ep[16]**
IN endpoint parameters
- **PCD_EPTypeDef PCD_HandleTypeDef::OUT_ep[16]**
OUT endpoint parameters
- **HAL_LockTypeDef PCD_HandleTypeDef::Lock**
PCD peripheral status
- **__IO PCD_StateTypeDef PCD_HandleTypeDef::State**
PCD communication state
- **__IO uint32_t PCD_HandleTypeDef::ErrorCode**
PCD Error code
- **uint32_t PCD_HandleTypeDef::Setup[12]**
Setup packet buffer
- **PCD_LPM_StateTypeDef PCD_HandleTypeDef::LPM_State**
LPM State
- **uint32_t PCD_HandleTypeDef::BESL**
- **uint32_t PCD_HandleTypeDef::lpm_active**
Enable or disable the Link Power Management . This parameter can be set to ENABLE or DISABLE
- **uint32_t PCD_HandleTypeDef::battery_charging_active**
Enable or disable Battery charging. This parameter can be set to ENABLE or DISABLE
- **void* PCD_HandleTypeDef::pData**
Pointer to upper stack Handler

49.2 PCD Firmware driver API description

The following section lists the various functions of the PCD library.

49.2.1 How to use this driver

The PCD HAL driver can be used as follows:

1. Declare a PCD_HandleTypeDef handle structure, for example: PCD_HandleTypeDef hpcd;
2. Fill parameters of Init structure in HCD handle
3. Call HAL_PCD_Init() API to initialize the PCD peripheral (Core, Device core, ...)
4. Initialize the PCD low level resources through the HAL_PCD_MspInit() API:
 - a. Enable the PCD/USB Low Level interface clock using
 - __HAL_RCC_USB_OTG_FS_CLK_ENABLE();
 - __HAL_RCC_USB_OTG_HS_CLK_ENABLE(); (For High Speed Mode)
 - b. Initialize the related GPIO clocks
 - c. Configure PCD pin-out
 - d. Configure PCD NVIC interrupt
5. Associate the Upper USB device stack to the HAL PCD Driver:
 - a. hpcd.pData = pdev;
6. Enable PCD transmission and reception:
 - a. HAL_PCD_Start();

49.2.2 Initialization and de-initialization functions

This section provides functions allowing to:

This section contains the following APIs:

- [*HAL_PCD_Init\(\)*](#)
- [*HAL_PCD_DeInit\(\)*](#)
- [*HAL_PCD_MspInit\(\)*](#)
- [*HAL_PCD_MspDeInit\(\)*](#)
- [*HAL_PCD_RegisterCallback\(\)*](#)
- [*HAL_PCD_UnRegisterCallback\(\)*](#)
- [*HAL_PCD_RegisterDataOutStageCallback\(\)*](#)
- [*HAL_PCD_UnRegisterDataOutStageCallback\(\)*](#)
- [*HAL_PCD_RegisterDataInStageCallback\(\)*](#)
- [*HAL_PCD_UnRegisterDataInStageCallback\(\)*](#)
- [*HAL_PCD_RegisterIsoOutInCpltCallback\(\)*](#)
- [*HAL_PCD_UnRegisterIsoOutInCpltCallback\(\)*](#)
- [*HAL_PCD_RegisterIsoInInCpltCallback\(\)*](#)
- [*HAL_PCD_UnRegisterIsoInInCpltCallback\(\)*](#)
- [*HAL_PCD_RegisterBcdCallback\(\)*](#)
- [*HAL_PCD_UnRegisterBcdCallback\(\)*](#)
- [*HAL_PCD_RegisterLpmCallback\(\)*](#)
- [*HAL_PCD_UnRegisterLpmCallback\(\)*](#)

49.2.3 IO operation functions

This subsection provides a set of functions allowing to manage the PCD data transfers.

This section contains the following APIs:

- [*HAL_PCD_Start\(\)*](#)
- [*HAL_PCD_Stop\(\)*](#)
- [*HAL_PCD_IRQHandler\(\)*](#)
- [*HAL_PCD_WKUP_IRQHandler\(\)*](#)

- *HAL_PCD_DataOutStageCallback()*
- *HAL_PCD_DataInStageCallback()*
- *HAL_PCD_SetupStageCallback()*
- *HAL_PCD_SOFCallback()*
- *HAL_PCD_ResetCallback()*
- *HAL_PCD_SuspendCallback()*
- *HAL_PCD_ResumeCallback()*
- *HAL_PCD_ISOOUTIncompleteCallback()*
- *HAL_PCD_ISOINIncompleteCallback()*
- *HAL_PCD_ConnectCallback()*
- *HAL_PCD_DisconnectCallback()*

49.2.4 Peripheral Control functions

This subsection provides a set of functions allowing to control the PCD data transfers.

This section contains the following APIs:

- *HAL_PCD_DevConnect()*
- *HAL_PCD_DevDisconnect()*
- *HAL_PCD_SetAddress()*
- *HAL_PCD_EP_Open()*
- *HAL_PCD_EP_Close()*
- *HAL_PCD_EP_Receive()*
- *HAL_PCD_EP_GetRxCount()*
- *HAL_PCD_EP_Transmit()*
- *HAL_PCD_EP_SetStall()*
- *HAL_PCD_EP_ClrStall()*
- *HAL_PCD_EP_Abort()*
- *HAL_PCD_EP_Flush()*
- *HAL_PCD_ActivateRemoteWakeup()*
- *HAL_PCD_DeActivateRemoteWakeup()*
- *HAL_PCD_SetTestMode()*

49.2.5 Peripheral State functions

This subsection permits to get in run-time the status of the peripheral and the data flow.

This section contains the following APIs:

- *HAL_PCD_GetState()*
- *HAL_PCD_SetTestMode()*

49.2.6 Detailed description of functions

HAL_PCD_Init

Function name

HAL_StatusTypeDef HAL_PCD_Init (PCD_HandleTypeDef * hpcd)

Function description

Initializes the PCD according to the specified parameters in the PCD_InitTypeDef and initialize the associated handle.

Parameters

- **hpcd**: PCD handle

Return values

- **HAL:** status

HAL_PCD_DeInit**Function name****HAL_StatusTypeDef HAL_PCD_DeInit (PCD_HandleTypeDef * hpcd)****Function description**

DeInitializes the PCD peripheral.

Parameters

- **hpcd:** PCD handle

Return values

- **HAL:** status

HAL_PCD_MspInit**Function name****void HAL_PCD_MspInit (PCD_HandleTypeDef * hpcd)****Function description**

Initializes the PCD MSP.

Parameters

- **hpcd:** PCD handle

Return values

- **None:**

HAL_PCD_MspDeInit**Function name****void HAL_PCD_MspDeInit (PCD_HandleTypeDef * hpcd)****Function description**

DeInitializes PCD MSP.

Parameters

- **hpcd:** PCD handle

Return values

- **None:**

HAL_PCD_RegisterCallback**Function name****HAL_StatusTypeDef HAL_PCD_RegisterCallback (PCD_HandleTypeDef * hpcd,
HAL_PCD_CallbackIDTypeDef CallbackID, pPCD_CallbackTypeDef pCallback)****Function description**

Register a User USB PCD Callback To be used instead of the weak predefined callback.

Parameters

- **hpcd**: USB PCD handle
- **CallbackID**: ID of the callback to be registered This parameter can be one of the following values:
 - HAL_PCD_SOF_CB_ID USB PCD SOF callback ID
 - HAL_PCD_SETUPSTAGE_CB_ID USB PCD Setup callback ID
 - HAL_PCD_RESET_CB_ID USB PCD Reset callback ID
 - HAL_PCD_SUSPEND_CB_ID USB PCD Suspend callback ID
 - HAL_PCD_RESUME_CB_ID USB PCD Resume callback ID
 - HAL_PCD_CONNECT_CB_ID USB PCD Connect callback ID
 - HAL_PCD_DISCONNECT_CB_ID USB PCD Disconnect callback ID
 - HAL_PCD_MSPINIT_CB_ID MspDeInit callback ID
 - HAL_PCD_MSPDEINIT_CB_ID MspDeInit callback ID
- **pCallback**: pointer to the Callback function

Return values

- **HAL**: status

HAL_PCD_UnRegisterCallback

Function name

HAL_StatusTypeDef HAL_PCD_UnRegisterCallback (PCD_HandleTypeDef * hpcd, HAL_PCD_CallbackIDTypeDef CallbackID)

Function description

Unregister an USB PCD Callback USB PCD callback is redirected to the weak predefined callback.

Parameters

- **hpcd**: USB PCD handle
- **CallbackID**: ID of the callback to be unregistered This parameter can be one of the following values:
 - HAL_PCD_SOF_CB_ID USB PCD SOF callback ID
 - HAL_PCD_SETUPSTAGE_CB_ID USB PCD Setup callback ID
 - HAL_PCD_RESET_CB_ID USB PCD Reset callback ID
 - HAL_PCD_SUSPEND_CB_ID USB PCD Suspend callback ID
 - HAL_PCD_RESUME_CB_ID USB PCD Resume callback ID
 - HAL_PCD_CONNECT_CB_ID USB PCD Connect callback ID
 - HAL_PCD_DISCONNECT_CB_ID USB PCD Disconnect callback ID
 - HAL_PCD_MSPINIT_CB_ID MspDeInit callback ID
 - HAL_PCD_MSPDEINIT_CB_ID MspDeInit callback ID

Return values

- **HAL**: status

HAL_PCD_RegisterDataOutStageCallback

Function name

HAL_StatusTypeDef HAL_PCD_RegisterDataOutStageCallback (PCD_HandleTypeDef * hpcd, pPCD_DataOutStageCallbackTypeDef pCallback)

Function description

Register USB PCD Data OUT Stage Callback To be used instead of the weak HAL_PCD_DataOutStageCallback() predefined callback.

Parameters

- **hpcd**: PCD handle
- **pCallback**: pointer to the USB PCD Data OUT Stage Callback function

Return values

- **HAL**: status

HAL_PCD_UnRegisterDataOutStageCallback

Function name

HAL_StatusTypeDef HAL_PCD_UnRegisterDataOutStageCallback (PCD_HandleTypeDef * hpcd)

Function description

Unregister the USB PCD Data OUT Stage Callback USB PCD Data OUT Stage Callback is redirected to the weak HAL_PCD_DataOutStageCallback() predefined callback.

Parameters

- **hpcd**: PCD handle

Return values

- **HAL**: status

HAL_PCD_RegisterDataInStageCallback

Function name

HAL_StatusTypeDef HAL_PCD_RegisterDataInStageCallback (PCD_HandleTypeDef * hpcd, pPCD_DataInStageCallbackTypeDef pCallback)

Function description

Register USB PCD Data IN Stage Callback To be used instead of the weak HAL_PCD_DataInStageCallback() predefined callback.

Parameters

- **hpcd**: PCD handle
- **pCallback**: pointer to the USB PCD Data IN Stage Callback function

Return values

- **HAL**: status

HAL_PCD_UnRegisterDataInStageCallback

Function name

HAL_StatusTypeDef HAL_PCD_UnRegisterDataInStageCallback (PCD_HandleTypeDef * hpcd)

Function description

Unregister the USB PCD Data IN Stage Callback USB PCD Data OUT Stage Callback is redirected to the weak HAL_PCD_DataInStageCallback() predefined callback.

Parameters

- **hpcd**: PCD handle

Return values

- **HAL**: status

HAL_PCD_RegisterIsoOutIncpItCallback

Function name

```
HAL_StatusTypeDef HAL_PCD_RegisterIsoOutIncpItCallback (PCD_HandleTypeDef * hpcd,
pPCD_IsoOutIncpItCallbackTypeDef pCallback)
```

Function description

Register USB PCD Iso OUT incomplete Callback To be used instead of the weak HAL_PCD_ISOOUTIncompleteCallback() predefined callback.

Parameters

- **hpcd**: PCD handle
- **pCallback**: pointer to the USB PCD Iso OUT incomplete Callback function

Return values

- **HAL**: status

HAL_PCD_UnRegisterIsoOutIncpItCallback

Function name

```
HAL_StatusTypeDef HAL_PCD_UnRegisterIsoOutIncpItCallback (PCD_HandleTypeDef * hpcd)
```

Function description

Unregister the USB PCD Iso OUT incomplete Callback USB PCD Iso OUT incomplete Callback is redirected to the weak HAL_PCD_ISOOUTIncompleteCallback() predefined callback.

Parameters

- **hpcd**: PCD handle

Return values

- **HAL**: status

HAL_PCD_RegisterIsoInIncpItCallback

Function name

```
HAL_StatusTypeDef HAL_PCD_RegisterIsoInIncpItCallback (PCD_HandleTypeDef * hpcd,
pPCD_IsoInIncpItCallbackTypeDef pCallback)
```

Function description

Register USB PCD Iso IN incomplete Callback To be used instead of the weak HAL_PCD_ISOINIncompleteCallback() predefined callback.

Parameters

- **hpcd**: PCD handle
- **pCallback**: pointer to the USB PCD Iso IN incomplete Callback function

Return values

- **HAL**: status

HAL_PCD_UnRegisterIsoInIncpItCallback

Function name

```
HAL_StatusTypeDef HAL_PCD_UnRegisterIsoInIncpItCallback (PCD_HandleTypeDef * hpcd)
```

Function description

Unregister the USB PCD Iso IN incomplete Callback USB PCD Iso IN incomplete Callback is redirected to the weak HAL_PCD_ISOINIncompleteCallback() predefined callback.

Parameters

- **hpcd**: PCD handle

Return values

- **HAL**: status

HAL_PCD_RegisterBcdCallback

Function name

HAL_StatusTypeDef HAL_PCD_RegisterBcdCallback (PCD_HandleTypeDef * hpcd, pPCD_BcdCallbackTypeDef pCallback)

Function description

Register USB PCD BCD Callback To be used instead of the weak HAL_PCDEx_BCD_Callback() predefined callback.

Parameters

- **hpcd**: PCD handle
- **pCallback**: pointer to the USB PCD BCD Callback function

Return values

- **HAL**: status

HAL_PCD_UnRegisterBcdCallback

Function name

HAL_StatusTypeDef HAL_PCD_UnRegisterBcdCallback (PCD_HandleTypeDef * hpcd)

Function description

Unregister the USB PCD BCD Callback USB BCD Callback is redirected to the weak HAL_PCDEx_BCD_Callback() predefined callback.

Parameters

- **hpcd**: PCD handle

Return values

- **HAL**: status

HAL_PCD_RegisterLpmCallback

Function name

HAL_StatusTypeDef HAL_PCD_RegisterLpmCallback (PCD_HandleTypeDef * hpcd, pPCD_LpmCallbackTypeDef pCallback)

Function description

Register USB PCD LPM Callback To be used instead of the weak HAL_PCDEx_LPM_Callback() predefined callback.

Parameters

- **hpcd**: PCD handle
- **pCallback**: pointer to the USB PCD LPM Callback function

Return values

- **HAL**: status

HAL_PCD_UnRegisterLpmCallback

Function name

HAL_StatusTypeDef HAL_PCD_UnRegisterLpmCallback (PCD_HandleTypeDef * hpcd)

Function description

Unregister the USB PCD LPM Callback USB LPM Callback is redirected to the weak HAL_PCDEx_LPM_Callback() predefined callback.

Parameters

- **hpcd**: PCD handle

Return values

- **HAL**: status

HAL_PCD_Start

Function name

HAL_StatusTypeDef HAL_PCD_Start (PCD_HandleTypeDef * hpcd)

Function description

Start the USB device.

Parameters

- **hpcd**: PCD handle

Return values

- **HAL**: status

HAL_PCD_Stop

Function name

HAL_StatusTypeDef HAL_PCD_Stop (PCD_HandleTypeDef * hpcd)

Function description

Stop the USB device.

Parameters

- **hpcd**: PCD handle

Return values

- **HAL**: status

HAL_PCD_IRQHandler

Function name

void HAL_PCD_IRQHandler (PCD_HandleTypeDef * hpcd)

Function description

Handles PCD interrupt request.

Parameters

- **hpcd**: PCD handle

Return values

- **HAL**: status

HAL_PCD_WKUP_IRQHandler

Function name

void HAL_PCD_WKUP_IRQHandler (PCD_HandleTypeDef * hpcd)

Function description

Handles PCD Wakeup interrupt request.

Parameters

- **hpcd**: PCD handle

Return values

- **HAL**: status

HAL_PCD_SOFCallback

Function name

void HAL_PCD_SOFCallback (PCD_HandleTypeDef * hpcd)

Function description

USB Start Of Frame callback.

Parameters

- **hpcd**: PCD handle

Return values

- **None**:

HAL_PCD_SetupStageCallback

Function name

void HAL_PCD_SetupStageCallback (PCD_HandleTypeDef * hpcd)

Function description

Setup stage callback.

Parameters

- **hpcd**: PCD handle

Return values

- **None**:

HAL_PCD_ResetCallback

Function name

void HAL_PCD_ResetCallback (PCD_HandleTypeDef * hpcd)

Function description

USB Reset callback.

Parameters

- **hpcd**: PCD handle

Return values

- **None**:

HAL_PCD_SuspendCallback

Function name

void HAL_PCD_SuspendCallback (PCD_HandleTypeDef * hpcd)

Function description

Suspend event callback.

Parameters

- **hpcd**: PCD handle

Return values

- **None**:

HAL_PCD_ResumeCallback

Function name

void HAL_PCD_ResumeCallback (PCD_HandleTypeDef * hpcd)

Function description

Resume event callback.

Parameters

- **hpcd**: PCD handle

Return values

- **None**:

HAL_PCD_ConnectCallback

Function name

void HAL_PCD_ConnectCallback (PCD_HandleTypeDef * hpcd)

Function description

Connection event callback.

Parameters

- **hpcd**: PCD handle

Return values

- **None**:

HAL_PCD_DisconnectCallback

Function name

void HAL_PCD_DisconnectCallback (PCD_HandleTypeDef * hpcd)

Function description

Disconnection event callback.

Parameters

- **hpcd**: PCD handle

Return values

- **None**:

HAL_PCD_DataOutStageCallback

Function name

```
void HAL_PCD_DataOutStageCallback (PCD_HandleTypeDef * hpcd, uint8_t epnum)
```

Function description

Data OUT stage callback.

Parameters

- **hpcd**: PCD handle
- **epnum**: endpoint number

Return values

- **None**:

HAL_PCD_DataInStageCallback

Function name

```
void HAL_PCD_DataInStageCallback (PCD_HandleTypeDef * hpcd, uint8_t epnum)
```

Function description

Data IN stage callback.

Parameters

- **hpcd**: PCD handle
- **epnum**: endpoint number

Return values

- **None**:

HAL_PCD_ISOOUTIncompleteCallback

Function name

```
void HAL_PCD_ISOOUTIncompleteCallback (PCD_HandleTypeDef * hpcd, uint8_t epnum)
```

Function description

Incomplete ISO OUT callback.

Parameters

- **hpcd**: PCD handle
- **epnum**: endpoint number

Return values

- **None**:

HAL_PCD_ISOINIncompleteCallback

Function name

```
void HAL_PCD_ISOINIncompleteCallback (PCD_HandleTypeDef * hpcd, uint8_t epnum)
```

Function description

Incomplete ISO IN callback.

Parameters

- **hpcd**: PCD handle
- **epnum**: endpoint number

Return values

- **None:**

HAL_PCD_DevConnect

Function name

HAL_StatusTypeDef HAL_PCD_DevConnect (PCD_HandleTypeDef * hpcd)

Function description

Connect the USB device.

Parameters

- **hpcd:** PCD handle

Return values

- **HAL:** status

HAL_PCD_DevDisconnect

Function name

HAL_StatusTypeDef HAL_PCD_DevDisconnect (PCD_HandleTypeDef * hpcd)

Function description

Disconnect the USB device.

Parameters

- **hpcd:** PCD handle

Return values

- **HAL:** status

HAL_PCD_SetAddress

Function name

HAL_StatusTypeDef HAL_PCD_SetAddress (PCD_HandleTypeDef * hpcd, uint8_t address)

Function description

Set the USB Device address.

Parameters

- **hpcd:** PCD handle
- **address:** new device address

Return values

- **HAL:** status

HAL_PCD_EP_Open

Function name

HAL_StatusTypeDef HAL_PCD_EP_Open (PCD_HandleTypeDef * hpcd, uint8_t ep_addr, uint16_t ep_mps, uint8_t ep_type)

Function description

Open and configure an endpoint.

Parameters

- **hpcd**: PCD handle
- **ep_addr**: endpoint address
- **ep_mps**: endpoint max packet size
- **ep_type**: endpoint type

Return values

- **HAL**: status

HAL_PCD_EP_Close

Function name

HAL_StatusTypeDef HAL_PCD_EP_Close (PCD_HandleTypeDef * hpcd, uint8_t ep_addr)

Function description

Deactivate an endpoint.

Parameters

- **hpcd**: PCD handle
- **ep_addr**: endpoint address

Return values

- **HAL**: status

HAL_PCD_EP_Receive

Function name

HAL_StatusTypeDef HAL_PCD_EP_Receive (PCD_HandleTypeDef * hpcd, uint8_t ep_addr, uint8_t * pBuf, uint32_t len)

Function description

Receive an amount of data.

Parameters

- **hpcd**: PCD handle
- **ep_addr**: endpoint address
- **pBuf**: pointer to the reception buffer
- **len**: amount of data to be received

Return values

- **HAL**: status

HAL_PCD_EP_Transmit

Function name

HAL_StatusTypeDef HAL_PCD_EP_Transmit (PCD_HandleTypeDef * hpcd, uint8_t ep_addr, uint8_t * pBuf, uint32_t len)

Function description

Send an amount of data.

Parameters

- **hpcd**: PCD handle
- **ep_addr**: endpoint address
- **pBuf**: pointer to the transmission buffer
- **len**: amount of data to be sent

Return values

- **HAL:** status

HAL_PCD_EP_SetStall

Function name

HAL_StatusTypeDef HAL_PCD_EP_SetStall (PCD_HandleTypeDef * hpcd, uint8_t ep_addr)

Function description

Set a STALL condition over an endpoint.

Parameters

- **hpcd:** PCD handle
- **ep_addr:** endpoint address

Return values

- **HAL:** status

HAL_PCD_EP_ClrStall

Function name

HAL_StatusTypeDef HAL_PCD_EP_ClrStall (PCD_HandleTypeDef * hpcd, uint8_t ep_addr)

Function description

Clear a STALL condition over in an endpoint.

Parameters

- **hpcd:** PCD handle
- **ep_addr:** endpoint address

Return values

- **HAL:** status

HAL_PCD_EP_Flush

Function name

HAL_StatusTypeDef HAL_PCD_EP_Flush (PCD_HandleTypeDef * hpcd, uint8_t ep_addr)

Function description

Flush an endpoint.

Parameters

- **hpcd:** PCD handle
- **ep_addr:** endpoint address

Return values

- **HAL:** status

HAL_PCD_EP_Abort

Function name

HAL_StatusTypeDef HAL_PCD_EP_Abort (PCD_HandleTypeDef * hpcd, uint8_t ep_addr)

Function description

Abort an USB EP transaction.

Parameters

- **hpcd**: PCD handle
- **ep_addr**: endpoint address

Return values

- **HAL**: status

HAL_PCD_ActivateRemoteWakeup

Function name

HAL_StatusTypeDef HAL_PCD_ActivateRemoteWakeup (PCD_HandleTypeDef * hpcd)

Function description

Activate remote wakeup signalling.

Parameters

- **hpcd**: PCD handle

Return values

- **HAL**: status

HAL_PCD_DeActivateRemoteWakeup

Function name

HAL_StatusTypeDef HAL_PCD_DeActivateRemoteWakeup (PCD_HandleTypeDef * hpcd)

Function description

De-activate remote wakeup signalling.

Parameters

- **hpcd**: PCD handle

Return values

- **HAL**: status

HAL_PCD_SetTestMode

Function name

HAL_StatusTypeDef HAL_PCD_SetTestMode (PCD_HandleTypeDef * hpcd, uint8_t testmode)

Function description

Set the USB Device high speed test mode.

Parameters

- **hpcd**: PCD handle
- **testmode**: USB Device high speed test mode

Return values

- **HAL**: status

HAL_PCD_EP_GetRxCount

Function name

uint32_t HAL_PCD_EP_GetRxCount (PCD_HandleTypeDef * hpcd, uint8_t ep_addr)

Function description

Get Received Data Size.

Parameters

- **hpcd**: PCD handle
- **ep_addr**: endpoint address

Return values

- **Data**: Size

HAL_PCD_GetState

Function name

PCD_StateTypeDef HAL_PCD_GetState (PCD_HandleTypeDef * hpcd)

Function description

Return the PCD handle state.

Parameters

- **hpcd**: PCD handle

Return values

- **HAL**: state

49.3 PCD Firmware driver defines

The following section lists the various define and macros of the module.

49.3.1 PCD

PCD

PCD Error Code definition

HAL_PCD_ERROR_INVALID_CALLBACK

Invalid Callback error

PCD Exported Macros

__HAL_PCD_ENABLE

__HAL_PCD_DISABLE

__HAL_PCD_GET_FLAG

__HAL_PCD_CLEAR_FLAG

__HAL_PCD_IS_INVALID_INTERRUPT

__HAL_PCD_UNGATE_PHYCLOCK

__HAL_PCD_GATE_PHYCLOCK

__HAL_PCD_IS_PHY_SUSPENDED

__HAL_USB_OTG_HS_WAKEUP_EXTI_ENABLE_IT

__HAL_USB_OTG_HS_WAKEUP_EXTI_DISABLE_IT

__HAL_USB_OTG_HS_WAKEUP_EXTI_GET_FLAG

__HAL_USB_OTG_HS_WAKEUP_EXTI_CLEAR_FLAG

__HAL_USB_OTG_HS_WAKEUP_EXTI_ENABLE_RISING_EDGE

__HAL_USB_OTG_FS_WAKEUP_EXTI_ENABLE_IT

__HAL_USB_OTG_FS_WAKEUP_EXTI_DISABLE_IT

__HAL_USB_OTG_FS_WAKEUP_EXTI_GET_FLAG

__HAL_USB_OTG_FS_WAKEUP_EXTI_CLEAR_FLAG

__HAL_USB_OTG_FS_WAKEUP_EXTI_ENABLE_RISING_EDGE

PCD PHY Module

PCD_PHY_ULPI

PCD_PHY_EMBEDDED

PCD_PHY_UTMI

PCD Speed

PCD_SPEED_HIGH

PCD_SPEED_HIGH_IN_FULL

PCD_SPEED_FULL

50 HAL PCD Extension Driver

50.1 PCDEx Firmware driver API description

The following section lists the various functions of the PCDEx library.

50.1.1 Extended features functions

This section provides functions allowing to:

- Update FIFO configuration

This section contains the following APIs:

- [*HAL_PCDEx_SetTxFiFo\(\)*](#)
- [*HAL_PCDEx_SetRxFiFo\(\)*](#)
- [*HAL_PCDEx_ActivateLPM\(\)*](#)
- [*HAL_PCDEx_DeActivateLPM\(\)*](#)
- [*HAL_PCDEx_LPM_Callback\(\)*](#)
- [*HAL_PCDEx_BCD_Callback\(\)*](#)

50.1.2 Detailed description of functions

HAL_PCDEx_SetTxFiFo

Function name

HAL_StatusTypeDef HAL_PCDEx_SetTxFiFo (PCD_HandleTypeDef * hpcd, uint8_t fifo, uint16_t size)

Function description

Set Tx FIFO.

Parameters

- **hpcd**: PCD handle
- **fifo**: The number of Tx fifo
- **size**: Fifo size

Return values

- **HAL**: status

HAL_PCDEx_SetRxFiFo

Function name

HAL_StatusTypeDef HAL_PCDEx_SetRxFiFo (PCD_HandleTypeDef * hpcd, uint16_t size)

Function description

Set Rx FIFO.

Parameters

- **hpcd**: PCD handle
- **size**: Size of Rx fifo

Return values

- **HAL**: status

HAL_PCDEx_ActivateLPM

Function name

HAL_StatusTypeDef HAL_PCDEx_ActivateLPM (PCD_HandleTypeDef * hpcd)

Function description

Activate LPM feature.

Parameters

- **hpcd**: PCD handle

Return values

- **HAL**: status

HAL_PCDEx_DeActivateLPM

Function name

HAL_StatusTypeDef HAL_PCDEx_DeActivateLPM (PCD_HandleTypeDef * hpcd)

Function description

Deactivate LPM feature.

Parameters

- **hpcd**: PCD handle

Return values

- **HAL**: status

HAL_PCDEx_LPM_Callback

Function name

void HAL_PCDEx_LPM_Callback (PCD_HandleTypeDef * hpcd, PCD_LPM_MsgTypeDef msg)

Function description

Send LPM message to user layer callback.

Parameters

- **hpcd**: PCD handle
- **msg**: LPM message

Return values

- **HAL**: status

HAL_PCDEx_BCD_Callback

Function name

void HAL_PCDEx_BCD_Callback (PCD_HandleTypeDef * hpcd, PCD_BCD_MsgTypeDef msg)

Function description

Send BatteryCharging message to user layer callback.

Parameters

- **hpcd**: PCD handle
- **msg**: LPM message

Return values

- **HAL**: status

51 HAL PWR Generic Driver

51.1 PWR Firmware driver registers structures

51.1.1 PWR_PVDTypeDef

PWR_PVDTypeDef is defined in the stm32f4xx_hal_pwr.h

Data Fields

- **uint32_t PVDLevel**
- **uint32_t Mode**

Field Documentation

- **uint32_t PWR_PVDTypeDef::PVDLevel**
PVDLevel: Specifies the PVD detection level. This parameter can be a value of [PWR_PVD_detection_level](#)
- **uint32_t PWR_PVDTypeDef::Mode**
Mode: Specifies the operating mode for the selected pins. This parameter can be a value of [PWR_PVD_Mode](#)

51.2 PWR Firmware driver API description

The following section lists the various functions of the PWR library.

51.2.1 Initialization and de-initialization functions

After reset, the backup domain (RTC registers, RTC backup data registers and backup SRAM) is protected against possible unwanted write accesses. To enable access to the RTC Domain and RTC registers, proceed as follows:

- Enable the Power Controller (PWR) APB1 interface clock using the `__HAL_RCC_PWR_CLK_ENABLE()` macro.
- Enable access to RTC domain using the `HAL_PWR_EnableBkUpAccess()` function.

This section contains the following APIs:

- [HAL_PWR_DeInit\(\)](#)
- [HAL_PWR_EnableBkUpAccess\(\)](#)
- [HAL_PWR_DisableBkUpAccess\(\)](#)

51.2.2 Peripheral Control functions

PVD configuration

- The PVD is used to monitor the VDD power supply by comparing it to a threshold selected by the PVD Level (PLS[2:0] bits in the PWR_CR).
- A PVDO flag is available to indicate if VDD/VDDA is higher or lower than the PVD threshold. This event is internally connected to the EXTI line16 and can generate an interrupt if enabled. This is done through `__HAL_PWR_PVD_EXTI_ENABLE_IT()` macro.
- The PVD is stopped in Standby mode.

Wake-up pin configuration

- Wake-up pin is used to wake up the system from Standby mode. This pin is forced in input pull-down configuration and is active on rising edges.
- There is one Wake-up pin: Wake-up Pin 1 on PA.00.
 - For STM32F446xx there are two Wake-Up pins: Pin1 on PA.00 and Pin2 on PC.13
 - For STM32F410xx/STM32F412xx/STM32F413xx/STM32F423xx there are three Wake-Up pins: Pin1 on PA.00, Pin2 on PC.00 and Pin3 on PC.01

Low Power modes configuration

The devices feature 3 low-power modes:

- Sleep mode: Cortex-M4 core stopped, peripherals kept running.
- Stop mode: all clocks are stopped, regulator running, regulator in low power mode
- Standby mode: 1.2V domain powered off.

Sleep mode

- Entry: The Sleep mode is entered by using the `HAL_PWR_EnterSLEEPMode(PWR_MAINREGULATOR_ON, PWR_SLEEPENTRY_WFI)` functions with
 - `PWR_SLEEPENTRY_WFI`: enter SLEEP mode with WFI instruction
 - `PWR_SLEEPENTRY_WFE`: enter SLEEP mode with WFE instruction

Note: *The Regulator parameter is not used for the STM32F4 family and is kept as parameter just to maintain compatibility with the lower power families (STM32L).*

- Exit: Any peripheral interrupt acknowledged by the nested vectored interrupt controller (NVIC) can wake up the device from Sleep mode.

Stop mode

In Stop mode, all clocks in the 1.2V domain are stopped, the PLL, the HSI, and the HSE RC oscillators are disabled. Internal SRAM and register contents are preserved. The voltage regulator can be configured either in normal or low-power mode. To minimize the consumption In Stop mode, FLASH can be powered off before entering the Stop mode using the `HAL_PWREx_EnableFlashPowerDown()` function. It can be switched on again by software after exiting the Stop mode using the `HAL_PWREx_DisableFlashPowerDown()` function.

- Entry: The Stop mode is entered using the `HAL_PWR_EnterSTOPMode(PWR_MAINREGULATOR_ON)` function with:
 - Main regulator ON.
 - Low Power regulator ON.
- Exit: Any EXTI Line (Internal or External) configured in Interrupt/Event mode.

Standby mode

- The Standby mode allows to achieve the lowest power consumption. It is based on the Cortex-M4 deep sleep mode, with the voltage regulator disabled. The 1.2V domain is consequently powered off. The PLL, the HSI oscillator and the HSE oscillator are also switched off. SRAM and register contents are lost except for the RTC registers, RTC backup registers, backup SRAM and Standby circuitry. The voltage regulator is OFF.
 - Entry:
 - The Standby mode is entered using the `HAL_PWR_EnterSTANDBYMode()` function.
 - Exit:
 - WKUP pin rising edge, RTC alarm (Alarm A and Alarm B), RTC wake-up, tamper event, time-stamp event, external reset in NRST pin, IWDG reset.

Auto-wake-up (AWU) from low-power mode

- The MCU can be woken up from low-power mode by an RTC Alarm event, an RTC Wake-up event, a tamper event or a time-stamp event, without depending on an external interrupt (Auto-wake-up mode).
- RTC auto-wake-up (AWU) from the Stop and Standby modes
 - To wake up from the Stop mode with an RTC alarm event, it is necessary to configure the RTC to generate the RTC alarm using the `HAL_RTC_SetAlarm_IT()` function.
 - To wake up from the Stop mode with an RTC Tamper or time stamp event, it is necessary to configure the RTC to detect the tamper or time stamp event using the `HAL_RTCEx_SetTimeStamp_IT()` or `HAL_RTCEx_SetTamper_IT()` functions.
 - To wake up from the Stop mode with an RTC Wake-up event, it is necessary to configure the RTC to generate the RTC Wake-up event using the `HAL_RTCEx_SetWakeUpTimer_IT()` function.

This section contains the following APIs:

- *HAL_PWR_ConfigPVD()*
- *HAL_PWR_EnablePVD()*
- *HAL_PWR_DisablePVD()*
- *HAL_PWR_EnableWakeUpPin()*
- *HAL_PWR_DisableWakeUpPin()*
- *HAL_PWR_EnterSLEEPMode()*
- *HAL_PWR_EnterSTOPMode()*
- *HAL_PWR_EnterSTANDBYMode()*
- *HAL_PWR_PVD_IRQHandler()*
- *HAL_PWR_PVDCallback()*
- *HAL_PWR_EnableSleepOnExit()*
- *HAL_PWR_DisableSleepOnExit()*
- *HAL_PWR_EnableSEVOnPend()*
- *HAL_PWR_DisableSEVOnPend()*

51.2.3 Detailed description of functions

HAL_PWR_DeInit

Function name

void HAL_PWR_DeInit (void)

Function description

Deinitializes the HAL PWR peripheral registers to their default reset values.

Return values

- **None:**

HAL_PWR_EnableBkUpAccess

Function name

void HAL_PWR_EnableBkUpAccess (void)

Function description

Enables access to the backup domain (RTC registers, RTC backup data registers and backup SRAM).

Return values

- **None:**

Notes

- If the HSE divided by 2, 3, ..31 is used as the RTC clock, the Backup Domain Access should be kept enabled.
- The following sequence is required to bypass the delay between DBP bit programming and the effective enabling of the backup domain. Please check the Errata Sheet for more details under "Possible delay in backup domain protection disabling/enabling after programming the DBP bit" section.

HAL_PWR_DisableBkUpAccess

Function name

void HAL_PWR_DisableBkUpAccess (void)

Function description

Disables access to the backup domain (RTC registers, RTC backup data registers and backup SRAM).

Return values

- **None:**

Notes

- If the HSE divided by 2, 3, ..31 is used as the RTC clock, the Backup Domain Access should be kept enabled.
- The following sequence is required to bypass the delay between DBP bit programming and the effective disabling of the backup domain. Please check the Errata Sheet for more details under "Possible delay in backup domain protection disabling/enabling after programming the DBP bit" section.

HAL_PWR_ConfigPVD

Function name

```
void HAL_PWR_ConfigPVD (PWR_PVDTypeDef * sConfigPVD)
```

Function description

Configures the voltage threshold detected by the Power Voltage Detector(PVD).

Parameters

- **sConfigPVD:** pointer to an PWR_PVDTypeDef structure that contains the configuration information for the PVD.

Return values

- **None:**

Notes

- Refer to the electrical characteristics of your device datasheet for more details about the voltage threshold corresponding to each detection level.

HAL_PWR_EnablePVD

Function name

```
void HAL_PWR_EnablePVD (void )
```

Function description

Enables the Power Voltage Detector(PVD).

Return values

- **None:**

HAL_PWR_DisablePVD

Function name

```
void HAL_PWR_DisablePVD (void )
```

Function description

Disables the Power Voltage Detector(PVD).

Return values

- **None:**

HAL_PWR_EnableWakeUpPin

Function name

```
void HAL_PWR_EnableWakeUpPin (uint32_t WakeUpPinx)
```

Function description

Enables the Wake-up PINx functionality.

Parameters

- **WakeUpPinx:** Specifies the Power Wake-Up pin to enable. This parameter can be one of the following values:
 - PWR_WAKEUP_PIN1
 - PWR_WAKEUP_PIN2 available only on STM32F410xx/STM32F446xx/STM32F412xx/STM32F413xx/STM32F423xx devices
 - PWR_WAKEUP_PIN3 available only on STM32F410xx/STM32F412xx/STM32F413xx/STM32F423xx devices

Return values

- **None:**

HAL_PWR_DisableWakeUpPin

Function name

void HAL_PWR_DisableWakeUpPin (uint32_t WakeUpPinx)

Function description

Disables the Wake-up PINx functionality.

Parameters

- **WakeUpPinx:** Specifies the Power Wake-Up pin to disable. This parameter can be one of the following values:
 - PWR_WAKEUP_PIN1
 - PWR_WAKEUP_PIN2 available only on STM32F410xx/STM32F446xx/STM32F412xx/STM32F413xx/STM32F423xx devices
 - PWR_WAKEUP_PIN3 available only on STM32F410xx/STM32F412xx/STM32F413xx/STM32F423xx devices

Return values

- **None:**

HAL_PWR_EnterSTOPMode

Function name

void HAL_PWR_EnterSTOPMode (uint32_t Regulator, uint8_t STOPEntry)

Function description

Enters Stop mode.

Parameters

- **Regulator:** Specifies the regulator state in Stop mode. This parameter can be one of the following values:
 - PWR_MAINREGULATOR_ON: Stop mode with regulator ON
 - PWR_LOWPOWERREGULATOR_ON: Stop mode with low power regulator ON
- **STOPEntry:** Specifies if Stop mode is entered with WFI or WFE instruction. This parameter can be one of the following values:
 - PWR_STOPENTRY_WFI: Enter Stop mode with WFI instruction
 - PWR_STOPENTRY_WFE: Enter Stop mode with WFE instruction

Return values

- **None:**

Notes

- In Stop mode, all I/O pins keep the same state as in Run mode.
- When exiting Stop mode by issuing an interrupt or a wake-up event, the HSI RC oscillator is selected as system clock.
- When the voltage regulator operates in low power mode, an additional startup delay is incurred when waking up from Stop mode. By keeping the internal regulator ON during Stop mode, the consumption is higher although the startup time is reduced.

HAL_PWR_EnterSLEEPMode

Function name

```
void HAL_PWR_EnterSLEEPMode (uint32_t Regulator, uint8_t SLEEPEntry)
```

Function description

Enters Sleep mode.

Parameters

- **Regulator:** Specifies the regulator state in SLEEP mode. This parameter can be one of the following values:
 - PWR_MAINREGULATOR_ON: SLEEP mode with regulator ON
 - PWR_LOWPOWERREGULATOR_ON: SLEEP mode with low power regulator ON
- **SLEEPEntry:** Specifies if SLEEP mode is entered with WFI or WFE instruction. This parameter can be one of the following values:
 - PWR_SLEEPENTRY_WFI: enter SLEEP mode with WFI instruction
 - PWR_SLEEPENTRY_WFE: enter SLEEP mode with WFE instruction

Return values

- **None:**

Notes

- In Sleep mode, all I/O pins keep the same state as in Run mode.
- In Sleep mode, the systick is stopped to avoid exit from this mode with systick interrupt when used as time base for Timeout
- This parameter is not used for the STM32F4 family and is kept as parameter just to maintain compatibility with the lower power families.

HAL_PWR_EnterSTANDBYMode

Function name

```
void HAL_PWR_EnterSTANDBYMode (void )
```

Function description

Enters Standby mode.

Return values

- **None:**

Notes

- In Standby mode, all I/O pins are high impedance except for: Reset pad (still available) RTC_AF1 pin (PC13) if configured for tamper, time-stamp, RTC Alarm out, or RTC clock calibration out. RTC_AF2 pin (PI8) if configured for tamper or time-stamp. WKUP pin 1 (PA0) if enabled.

HAL_PWR_PVD_IRQHandler

Function name

```
void HAL_PWR_PVD_IRQHandler (void )
```

Function description

This function handles the PWR PVD interrupt request.

Return values

- **None:**

Notes

- This API should be called under the PVD_IRQHandler().

HAL_PWR_PVDCallback

Function name

void HAL_PWR_PVDCallback (void)

Function description

PWR PVD interrupt callback.

Return values

- **None:**

HAL_PWR_EnableSleepOnExit

Function name

void HAL_PWR_EnableSleepOnExit (void)

Function description

Indicates Sleep-On-Exit when returning from Handler mode to Thread mode.

Return values

- **None:**

Notes

- Set SLEEPONEXIT bit of SCR register. When this bit is set, the processor re-enters SLEEP mode when an interruption handling is over. Setting this bit is useful when the processor is expected to run only on interruptions handling.

HAL_PWR_DisableSleepOnExit

Function name

void HAL_PWR_DisableSleepOnExit (void)

Function description

Disables Sleep-On-Exit feature when returning from Handler mode to Thread mode.

Return values

- **None:**

Notes

- Clears SLEEPONEXIT bit of SCR register. When this bit is set, the processor re-enters SLEEP mode when an interruption handling is over.

HAL_PWR_EnableSEVOnPend

Function name

void HAL_PWR_EnableSEVOnPend (void)

Function description

Enables CORTEX M4 SEVONPEND bit.

Return values

- **None:**

Notes

- Sets SEVONPEND bit of SCR register. When this bit is set, this causes WFE to wake up when an interrupt moves from inactive to pended.

HAL_PWR_DisableSEVOnPend

Function name

void HAL_PWR_DisableSEVOnPend (void)

Function description

Disables CORTEX M4 SEVONPEND bit.

Return values

- **None:**

Notes

- Clears SEVONPEND bit of SCR register. When this bit is set, this causes WFE to wake up when an interrupt moves from inactive to pended.

51.3 PWR Firmware driver defines

The following section lists the various define and macros of the module.

51.3.1 PWR

PWR

PWR CR Register alias address

DBP_BIT_NUMBER

CR_DBP_BB

PVDE_BIT_NUMBER

CR_PVDE_BB

VOS_BIT_NUMBER

CR_VOS_BB

PWR CSR Register alias address

EWUP_BIT_NUMBER

CSR_EWUP_BB

PWR Exported Macro

__HAL_PWR_GET_FLAG

Description:

- Check PWR flag is set or not.

Parameters:

- `__FLAG__`: specifies the flag to check. This parameter can be one of the following values:
 - `PWR_FLAG_WU`: Wake Up flag. This flag indicates that a wakeup event was received from the WKUP pin or from the RTC alarm (Alarm A or Alarm B), RTC Tamper event, RTC TimeStamp event or RTC Wakeup. An additional wakeup event is detected if the WKUP pin is enabled (by setting the EWUP bit) when the WKUP pin level is already high.
 - `PWR_FLAG_SB`: StandBy flag. This flag indicates that the system was resumed from StandBy mode.
 - `PWR_FLAG_PVDO`: PVD Output. This flag is valid only if PVD is enabled by the `HAL_PWR_EnablePVD()` function. The PVD is stopped by Standby mode. For this reason, this bit is equal to 0 after Standby or reset until the PVDE bit is set.
 - `PWR_FLAG_BRR`: Backup regulator ready flag. This bit is not reset when the device wakes up from Standby mode or by a system reset or power reset.
 - `PWR_FLAG_VOSRDY`: This flag indicates that the Regulator voltage scaling output selection is ready.

Return value:

- The: new state of `__FLAG__` (TRUE or FALSE).

__HAL_PWR_CLEAR_FLAG

Description:

- Clear the PWR's pending flags.

Parameters:

- `__FLAG__`: specifies the flag to clear. This parameter can be one of the following values:
 - `PWR_FLAG_WU`: Wake Up flag
 - `PWR_FLAG_SB`: StandBy flag

__HAL_PWR_PVD_EXTI_ENABLE_IT

Description:

- Enable the PVD Exti Line 16.

Return value:

- None.

__HAL_PWR_PVD_EXTI_DISABLE_IT

Description:

- Disable the PVD EXTI Line 16.

Return value:

- None.

__HAL_PWR_PVD_EXTI_ENABLE_EVENT

Description:

- Enable event on PVD Exti Line 16.

Return value:

- None.

__HAL_PWR_PVD_EXTI_DISABLE_EVENT

Description:

- Disable event on PVD Exti Line 16.

Return value:

- None.

__HAL_PWR_PVD_EXTI_ENABLE_RISING_EDGE**Description:**

- Enable the PVD Extended Interrupt Rising Trigger.

Return value:

- None.

__HAL_PWR_PVD_EXTI_DISABLE_RISING_EDGE**Description:**

- Disable the PVD Extended Interrupt Rising Trigger.

Return value:

- None.

__HAL_PWR_PVD_EXTI_ENABLE_FALLING_EDGE**Description:**

- Enable the PVD Extended Interrupt Falling Trigger.

Return value:

- None.

__HAL_PWR_PVD_EXTI_DISABLE_FALLING_EDGE**Description:**

- Disable the PVD Extended Interrupt Falling Trigger.

Return value:

- None.

__HAL_PWR_PVD_EXTI_ENABLE_RISING_FALLING_EDGE**Description:**

- PVD EXTI line configuration: set rising & falling edge trigger.

Return value:

- None.

__HAL_PWR_PVD_EXTI_DISABLE_RISING_FALLING_EDGE**Description:**

- Disable the PVD Extended Interrupt Rising & Falling Trigger.

Return value:

- None.

__HAL_PWR_PVD_EXTI_GET_FLAG**Description:**

- checks whether the specified PVD Exti interrupt flag is set or not.

Return value:

- EXTI: PVD Line Status.

__HAL_PWR_PVD_EXTI_CLEAR_FLAG**Description:**

- Clear the PVD Exti flag.

Return value:

- None.

__HAL_PWR_PVD_EXTI_GENERATE_SWIT**Description:**

- Generates a Software interrupt on PVD EXTI line.

Return value:

- None

PWR Flag**PWR_FLAG_WU****PWR_FLAG_SB****PWR_FLAG_PVDO****PWR_FLAG_BRR****PWR_FLAG_VOSRDY****PWR Private macros to check input parameters****IS_PWR_PVD_LEVEL****IS_PWR_PVD_MODE****IS_PWR_REGULATOR****IS_PWR_SLEEP_ENTRY****IS_PWR_STOP_ENTRY****PWR PVD detection level****PWR_PVDLEVEL_0****PWR_PVDLEVEL_1****PWR_PVDLEVEL_2****PWR_PVDLEVEL_3****PWR_PVDLEVEL_4****PWR_PVDLEVEL_5****PWR_PVDLEVEL_6****PWR_PVDLEVEL_7****PWR PVD EXTI Line****PWR_EXTI_LINE_PVD**

External interrupt line 16 Connected to the PVD EXTI Line

PWR PVD Mode

PWR_PVD_MODE_NORMAL

basic mode is used

PWR_PVD_MODE_IT_RISING

External Interrupt Mode with Rising edge trigger detection

PWR_PVD_MODE_IT_FALLING

External Interrupt Mode with Falling edge trigger detection

PWR_PVD_MODE_IT_RISING_FALLING

External Interrupt Mode with Rising/Falling edge trigger detection

PWR_PVD_MODE_EVENT_RISING

Event Mode with Rising edge trigger detection

PWR_PVD_MODE_EVENT_FALLING

Event Mode with Falling edge trigger detection

PWR_PVD_MODE_EVENT_RISING_FALLING

Event Mode with Rising/Falling edge trigger detection

PWR PVD Mode Mask**PVD_MODE_IT****PVD_MODE_EVT****PVD_RISING_EDGE****PVD_FALLING_EDGE*****PWR Register alias address*****PWR_OFFSET****PWR_CR_OFFSET****PWR_CSR_OFFSET****PWR_CR_OFFSET_BB****PWR_CSR_OFFSET_BB*****PWR Regulator state in SLEEP/STOP mode*****PWR_MAINREGULATOR_ON****PWR_LOWPOWERREGULATOR_ON*****PWR SLEEP mode entry*****PWR_SLEEPENTRY_WFI****PWR_SLEEPENTRY_WFE*****PWR STOP mode entry***



PWR_STOPENTRY_WFI

PWR_STOPENTRY_WFE

PWR WakeUp Pins

PWR_WAKEUP_PIN1

52 HAL PWR Extension Driver

52.1 PWREx Firmware driver API description

The following section lists the various functions of the PWREx library.

52.1.1 Peripheral extended features functions

Main and Backup Regulators configuration

- The backup domain includes 4 Kbytes of backup SRAM accessible only from the CPU, and address in 32-bit, 16-bit or 8-bit mode. Its content is retained even in Standby or VBAT mode when the low power backup regulator is enabled. It can be considered as an internal EEPROM when VBAT is always present. You can use the `HAL_PWREx_EnableBkUpReg()` function to enable the low power backup regulator.
- When the backup domain is supplied by VDD (analog switch connected to VDD) the backup SRAM is powered from VDD which replaces the VBAT power supply to save battery life.
- The backup SRAM is not mass erased by a tamper event. It is read protected to prevent confidential data, such as cryptographic private key, from being accessed. The backup SRAM can be erased only through the Flash interface when a protection level change from level 1 to level 0 is requested.

Note: Refer to the description of Read protection (RDP) in the Flash programming manual.

- The main internal regulator can be configured to have a tradeoff between performance and power consumption when the device does not operate at the maximum frequency. This is done through `__HAL_PWR_MAINREGULATORMODE_CONFIG()` macro which configure VOS bit in `PWR_CR` register. Refer to the product datasheets for more details.

FLASH Power Down configuration

- By setting the `FPDS` bit in the `PWR_CR` register by using the `HAL_PWREx_EnableFlashPowerDown()` function, the Flash memory also enters power down mode when the device enters Stop mode. When the Flash memory is in power down mode, an additional startup delay is incurred when waking up from Stop mode.
- For STM32F42xxx/43xxx/446xx/469xx/479xx Devices, the scale can be modified only when the PLL is OFF and the HSI or HSE clock source is selected as system clock. The new value programmed is active only when the PLL is ON. When the PLL is OFF, the voltage scale 3 is automatically selected. Refer to the datasheets for more details.

Over-Drive and Under-Drive configuration

- For STM32F42xxx/43xxx/446xx/469xx/479xx Devices, in Run mode: the main regulator has 2 operating modes available:
 - Normal mode: The CPU and core logic operate at maximum frequency at a given voltage scaling (scale 1, scale 2 or scale 3)
 - Over-drive mode: This mode allows the CPU and the core logic to operate at a higher frequency than the normal mode for a given voltage scaling (scale 1, scale 2 or scale 3). This mode is enabled through `HAL_PWREx_EnableOverDrive()` function and disabled by `HAL_PWREx_DisableOverDrive()` function, to enter or exit from Over-drive mode please follow the sequence described in Reference manual.
- For STM32F42xxx/43xxx/446xx/469xx/479xx Devices, in Stop mode: the main regulator or low power regulator supplies a low power voltage to the 1.2V domain, thus preserving the content of registers and internal SRAM. 2 operating modes are available:
 - Normal mode: the 1.2V domain is preserved in nominal leakage mode. This mode is only available when the main regulator or the low power regulator is used in Scale 3 or low voltage mode.
 - Under-drive mode: the 1.2V domain is preserved in reduced leakage mode. This mode is only available when the main regulator or the low power regulator is in low voltage mode.

This section contains the following APIs:

- [`HAL\_PWREx\_EnableBkUpReg\(\)`](#)
- [`HAL\_PWREx\_DisableBkUpReg\(\)`](#)

- *HAL_PWREx_EnableFlashPowerDown()*
- *HAL_PWREx_DisableFlashPowerDown()*
- *HAL_PWREx_GetVoltageRange()*
- *HAL_PWREx_ControlVoltageScaling()*
- *HAL_PWREx_EnableOverDrive()*
- *HAL_PWREx_DisableOverDrive()*
- *HAL_PWREx_EnterUnderDriveSTOPMode()*

52.1.2 Detailed description of functions

HAL_PWREx_EnableFlashPowerDown

Function name

void HAL_PWREx_EnableFlashPowerDown (void)

Function description

Enables the Flash Power Down in Stop mode.

Return values

- **None:**

HAL_PWREx_DisableFlashPowerDown

Function name

void HAL_PWREx_DisableFlashPowerDown (void)

Function description

Disables the Flash Power Down in Stop mode.

Return values

- **None:**

HAL_PWREx_EnableBkUpReg

Function name

HAL_StatusTypeDef HAL_PWREx_EnableBkUpReg (void)

Function description

Enables the Backup Regulator.

Return values

- **HAL:** status

HAL_PWREx_DisableBkUpReg

Function name

HAL_StatusTypeDef HAL_PWREx_DisableBkUpReg (void)

Function description

Disables the Backup Regulator.

Return values

- **HAL:** status

HAL_PWREx_GetVoltageRange

Function name

uint32_t HAL_PWREx_GetVoltageRange (void)

Function description

Return Voltage Scaling Range.

Return values

- **The:** configured scale for the regulator voltage(VOS bit field). The returned value can be one of the following:
 - ◦ PWR_REGULATOR_VOLTAGE_SCALE1: Regulator voltage output Scale 1 mode
 - ◦ PWR_REGULATOR_VOLTAGE_SCALE2: Regulator voltage output Scale 2 mode
 - ◦ PWR_REGULATOR_VOLTAGE_SCALE3: Regulator voltage output Scale 3 mode

HAL_PWREx_ControlVoltageScaling

Function name

HAL_StatusTypeDef HAL_PWREx_ControlVoltageScaling (uint32_t VoltageScaling)

Function description

Configures the main internal regulator output voltage.

Parameters

- **VoltageScaling:** specifies the regulator output voltage to achieve a tradeoff between performance and power consumption. This parameter can be one of the following values:
 - PWR_REGULATOR_VOLTAGE_SCALE1: Regulator voltage output range 1 mode, the maximum value of fHCLK is 168 MHz. It can be extended to 180 MHz by activating the over-drive mode.
 - PWR_REGULATOR_VOLTAGE_SCALE2: Regulator voltage output range 2 mode, the maximum value of fHCLK is 144 MHz. It can be extended to, 168 MHz by activating the over-drive mode.
 - PWR_REGULATOR_VOLTAGE_SCALE3: Regulator voltage output range 3 mode, the maximum value of fHCLK is 120 MHz.

Return values

- **HAL:** Status

Notes

- To update the system clock frequency(SYSCLK): Set the HSI or HSE as system clock frequency using the HAL_RCC_ClockConfig(). Call the HAL_RCC_OscConfig() to configure the PLL. Call HAL_PWREx_ConfigVoltageScaling() API to adjust the voltage scale. Set the new system clock frequency using the HAL_RCC_ClockConfig().
- The scale can be modified only when the HSI or HSE clock source is selected as system clock source, otherwise the API returns HAL_ERROR.
- When the PLL is OFF, the voltage scale 3 is automatically selected and the VOS bits value in the PWR_CR1 register are not taken in account.
- This API forces the PLL state ON to allow the possibility to configure the voltage scale 1 or 2.
- The new voltage scale is active only when the PLL is ON.

HAL_PWREx_EnableOverDrive

Function name

HAL_StatusTypeDef HAL_PWREx_EnableOverDrive (void)

Function description

Activates the Over-Drive mode.

Return values

- **HAL:** status

Notes

- This function can be used only for STM32F42xx/STM32F43xx/STM32F446xx/STM32F469xx/STM32F479xx devices. This mode allows the CPU and the core logic to operate at a higher frequency than the normal mode for a given voltage scaling (scale 1, scale 2 or scale 3).
- It is recommended to enter or exit Over-drive mode when the application is not running critical tasks and when the system clock source is either HSI or HSE. During the Over-drive switch activation, no peripheral clocks should be enabled. The peripheral clocks must be enabled once the Over-drive mode is activated.

HAL_PWREx_DisableOverDrive

Function name

HAL_StatusTypeDef HAL_PWREx_DisableOverDrive (void)

Function description

Deactivates the Over-Drive mode.

Return values

- **HAL:** status

Notes

- This function can be used only for STM32F42xx/STM32F43xx/STM32F446xx/STM32F469xx/STM32F479xx devices. This mode allows the CPU and the core logic to operate at a higher frequency than the normal mode for a given voltage scaling (scale 1, scale 2 or scale 3).
- It is recommended to enter or exit Over-drive mode when the application is not running critical tasks and when the system clock source is either HSI or HSE. During the Over-drive switch activation, no peripheral clocks should be enabled. The peripheral clocks must be enabled once the Over-drive mode is activated.

HAL_PWREx_EnterUnderDriveSTOPMode

Function name

HAL_StatusTypeDef HAL_PWREx_EnterUnderDriveSTOPMode (uint32_t Regulator, uint8_t STOPEntry)

Function description

Enters in Under-Drive STOP mode.

Parameters

- **Regulator:** specifies the regulator state in STOP mode. This parameter can be one of the following values:
 - PWR_MAINREGULATOR_UNDERDRIVE_ON: Main Regulator in under-drive mode and Flash memory in power-down when the device is in Stop under-drive mode
 - PWR_LOWPOWERREGULATOR_UNDERDRIVE_ON: Low Power Regulator in under-drive mode and Flash memory in power-down when the device is in Stop under-drive mode
- **STOPEntry:** specifies if STOP mode in entered with WFI or WFE instruction. This parameter can be one of the following values:
 - PWR_SLEEPENTRY_WFI: enter STOP mode with WFI instruction
 - PWR_SLEEPENTRY_WFE: enter STOP mode with WFE instruction

Return values

- **None:**

Notes

- This mode is only available for STM32F42xxx/STM32F43xxx/STM32F446xx/STM32F469xx/STM32F479xx devices.
- This mode can be selected only when the Under-Drive is already active
- This mode is enabled only with STOP low power mode. In this mode, the 1.2V domain is preserved in reduced leakage mode. This mode is only available when the main regulator or the low power regulator is in low voltage mode
- If the Under-drive mode was enabled, it is automatically disabled after exiting Stop mode. When the voltage regulator operates in Under-drive mode, an additional startup delay is induced when waking up from Stop mode.
- In Stop mode, all I/O pins keep the same state as in Run mode.
- When exiting Stop mode by issuing an interrupt or a wake-up event, the HSI RC oscillator is selected as system clock.
- When the voltage regulator operates in low power mode, an additional startup delay is incurred when waking up from Stop mode. By keeping the internal regulator ON during Stop mode, the consumption is higher although the startup time is reduced.

52.2 PWREx Firmware driver defines

The following section lists the various define and macros of the module.

52.2.1 PWREx

PWREx

PWRx CSR Register alias address

BRE_BIT_NUMBER

CSR_BRE_BB

PWREx Exported Constants

__HAL_PWR_VOLTAGESCALING_CONFIG

Description:

- macros configure the main internal regulator output voltage.

Parameters:

- **__REGULATOR__**: specifies the regulator output voltage to achieve a tradeoff between performance and power consumption when the device does not operate at the maximum frequency (refer to the datasheets for more details). This parameter can be one of the following values:
 - PWR_REGULATOR_VOLTAGE_SCALE1: Regulator voltage output Scale 1 mode
 - PWR_REGULATOR_VOLTAGE_SCALE2: Regulator voltage output Scale 2 mode
 - PWR_REGULATOR_VOLTAGE_SCALE3: Regulator voltage output Scale 3 mode

Return value:

- None

__HAL_PWR_OVERDRIVE_ENABLE

Notes:

- These macros can be used only for STM32F42xx/STM3243xx devices.

__HAL_PWR_OVERDRIVE_DISABLE

__HAL_PWR_OVERDRIVESWITCHING_ENABLE

Notes:

- These macros can be used only for STM32F42xx/STM3243xx devices.

__HAL_PWR_OVERDRIVESWITCHING_DISABLE

__HAL_PWR_UNDERDRIVE_ENABLE

Notes:

- This mode is enabled only with STOP low power mode. In this mode, the 1.2V domain is preserved in reduced leakage mode. This mode is only available when the main regulator or the low power regulator is in low voltage mode. If the Under-drive mode was enabled, it is automatically disabled after exiting Stop mode. When the voltage regulator operates in Under-drive mode, an additional startup delay is induced when waking up from Stop mode.

__HAL_PWR_UNDERDRIVE_DISABLE

__HAL_PWR_GET_ODRUDR_FLAG

Description:

- Check PWR flag is set or not.

Parameters:

- __FLAG__: specifies the flag to check. This parameter can be one of the following values:
 - PWR_FLAG_ODRDY: This flag indicates that the Over-drive mode is ready
 - PWR_FLAG_ODSWRDY: This flag indicates that the Over-drive mode switching is ready
 - PWR_FLAG_UDRDY: This flag indicates that the Under-drive mode is enabled in Stop mode

Return value:

- The: new state of __FLAG__ (TRUE or FALSE).

Notes:

- These macros can be used only for STM32F42xx/STM3243xx devices.

__HAL_PWR_CLEAR_ODRUDR_FLAG

Notes:

- These macros can be used only for STM32F42xx/STM3243xx devices.

PWREx Private macros to check input parameters

IS_PWR_REGULATOR_UNDERDRIVE

IS_PWR_VOLTAGE_SCALING_RANGE

IS_PWR_WAKEUP_PIN

PWREx Over Under Drive Flag

PWR_FLAG_ODRDY

PWR_FLAG_ODSWRDY

PWR_FLAG_UDRDY

PWREx Register alias address

FPDS_BIT_NUMBER

CR_FPDS_BB

ODEN_BIT_NUMBER

CR_ODEN_BB

ODSWEN_BIT_NUMBER

CR_ODSWEN_BB

MRLVDS_BIT_NUMBER

CR_MRLVDS_BB

LPLVDS_BIT_NUMBER

CR_LPLVDS_BB

PWREx Regulator state in UnderDrive mode

PWR_MAINREGULATOR_UNDERDRIVE_ON

PWR_LOWPOWERREGULATOR_UNDERDRIVE_ON

PWREx Regulator Voltage Scale

PWR_REGULATOR_VOLTAGE_SCALE1

PWR_REGULATOR_VOLTAGE_SCALE2

PWR_REGULATOR_VOLTAGE_SCALE3

54 HAL RCC Generic Driver

54.1 RCC Firmware driver registers structures

54.1.1 RCC_OscInitTypeDef

RCC_OscInitTypeDef is defined in the stm32f4xx_hal_rcc.h

Data Fields

- **uint32_t OscillatorType**
- **uint32_t HSEState**
- **uint32_t LSEState**
- **uint32_t HSIState**
- **uint32_t HSICalibrationValue**
- **uint32_t LSIState**
- **RCC_PLLInitTypeDef PLL**

Field Documentation

- **uint32_t RCC_OscInitTypeDef::OscillatorType**
The oscillators to be configured. This parameter can be a value of [RCC_Oscillator_Type](#)
- **uint32_t RCC_OscInitTypeDef::HSEState**
The new state of the HSE. This parameter can be a value of [RCC_HSE_Config](#)
- **uint32_t RCC_OscInitTypeDef::LSEState**
The new state of the LSE. This parameter can be a value of [RCC_LSE_Config](#)
- **uint32_t RCC_OscInitTypeDef::HSIState**
The new state of the HSI. This parameter can be a value of [RCC_HSI_Config](#)
- **uint32_t RCC_OscInitTypeDef::HSICalibrationValue**
The HSI calibration trimming value (default is RCC_HSICALIBRATION_DEFAULT). This parameter must be a number between Min_Data = 0x00 and Max_Data = 0x1F
- **uint32_t RCC_OscInitTypeDef::LSIState**
The new state of the LSI. This parameter can be a value of [RCC_LSI_Config](#)
- **RCC_PLLInitTypeDef RCC_OscInitTypeDef::PLL**
PLL structure parameters

54.1.2 RCC_ClkInitTypeDef

RCC_ClkInitTypeDef is defined in the stm32f4xx_hal_rcc.h

Data Fields

- **uint32_t ClockType**
- **uint32_t SYSCLKSource**
- **uint32_t AHBCLKDivider**
- **uint32_t APB1CLKDivider**
- **uint32_t APB2CLKDivider**

Field Documentation

- **uint32_t RCC_ClkInitTypeDef::ClockType**
The clock to be configured. This parameter can be a value of [RCC_System_Clock_Type](#)
- **uint32_t RCC_ClkInitTypeDef::SYSCLKSource**
The clock source (SYSCLKS) used as system clock. This parameter can be a value of [RCC_System_Clock_Source](#)
- **uint32_t RCC_ClkInitTypeDef::AHBCLKDivider**
The AHB clock (HCLK) divider. This clock is derived from the system clock (SYSCLK). This parameter can be a value of [RCC_AHB_Clock_Source](#)
- **uint32_t RCC_ClkInitTypeDef::APB1CLKDivider**
The APB1 clock (PCLK1) divider. This clock is derived from the AHB clock (HCLK). This parameter can be a value of [RCC_APB1_APB2_Clock_Source](#)

- **`uint32_t RCC_ClkInitTypeDef::APB2CLKDivider`**
The APB2 clock (PCLK2) divider. This clock is derived from the AHB clock (HCLK). This parameter can be a value of **`RCC_APB1_APB2_Clock_Source`**

54.2 RCC Firmware driver API description

The following section lists the various functions of the RCC library.

54.2.1 RCC specific features

After reset the device is running from Internal High Speed oscillator (HSI 16MHz) with Flash 0 wait state, Flash prefetch buffer, D-Cache and I-Cache are disabled, and all peripherals are off except internal SRAM, Flash and JTAG.

- There is no prescaler on High speed (AHB) and Low speed (APB) busses; all peripherals mapped on these busses are running at HSI speed.
- The clock for all peripherals is switched off, except the SRAM and FLASH.
- All GPIOs are in input floating state, except the JTAG pins which are assigned to be used for debug purpose.

Once the device started from reset, the user application has to:

- Configure the clock source to be used to drive the System clock (if the application needs higher frequency/performance)
- Configure the System clock frequency and Flash settings
- Configure the AHB and APB busses prescalers
- Enable the clock for the peripheral(s) to be used
- Configure the clock source(s) for peripherals which clocks are not derived from the System clock (I2S, RTC, ADC, USB OTG FS/SDIO/RNG)

54.2.2 RCC Limitations

A delay between an RCC peripheral clock enable and the effective peripheral enabling should be taken into account in order to manage the peripheral read/write from/to registers.

- This delay depends on the peripheral mapping.
- If peripheral is mapped on AHB: the delay is 2 AHB clock cycle after the clock enable bit is set on the hardware register
- If peripheral is mapped on APB: the delay is 2 APB clock cycle after the clock enable bit is set on the hardware register

Implemented Workaround:

- For AHB & APB peripherals, a dummy read to the peripheral register has been inserted in each `__HAL_RCC_PPP_CLK_ENABLE()` macro.

54.2.3 Initialization and de-initialization functions

This section provides functions allowing to configure the internal/external oscillators (HSE, HSI, LSE, LSI, PLL, CSS and MCO) and the System busses clocks (SYSCLK, AHB, APB1 and APB2).

Internal/external clock and PLL configuration

1. HSI (high-speed internal), 16 MHz factory-trimmed RC used directly or through the PLL as System clock source.
2. LSI (low-speed internal), 32 KHz low consumption RC used as IWDG and/or RTC clock source.
3. HSE (high-speed external), 4 to 26 MHz crystal oscillator used directly or through the PLL as System clock source. Can be used also as RTC clock source.
4. LSE (low-speed external), 32 KHz oscillator used as RTC clock source.
5. PLL (clocked by HSI or HSE), featuring two different output clocks:
 - The first output is used to generate the high speed system clock (up to 168 MHz)
 - The second output is used to generate the clock for the USB OTG FS (48 MHz), the random analog generator (<=48 MHz) and the SDIO (<= 48 MHz).

6. CSS (Clock security system), once enable using the macro `__HAL_RCC_CSS_ENABLE()` and if a HSE clock failure occurs (HSE used directly or through PLL as System clock source), the System clocks automatically switched to HSI and an interrupt is generated if enabled. The interrupt is linked to the Cortex-M4 NMI (Non-Maskable Interrupt) exception vector.
7. MCO1 (microcontroller clock output), used to output HSI, LSE, HSE or PLL clock (through a configurable prescaler) on PA8 pin.
8. MCO2 (microcontroller clock output), used to output HSE, PLL, SYSCLK or PLLI2S clock (through a configurable prescaler) on PC9 pin.

System, AHB and APB busses clocks configuration

1. Several clock sources can be used to drive the System clock (SYSCLK): HSI, HSE and PLL. The AHB clock (HCLK) is derived from System clock through configurable prescaler and used to clock the CPU, memory and peripherals mapped on AHB bus (DMA, GPIO...). APB1 (PCLK1) and APB2 (PCLK2) clocks are derived from AHB clock through configurable prescalers and used to clock the peripherals mapped on these busses. You can use "`HAL_RCC_GetSysClockFreq()`" function to retrieve the frequencies of these clocks.
2. For the STM32F405xx/07xx and STM32F415xx/17xx devices, the maximum frequency of the SYSCLK and HCLK is 168 MHz, PCLK2 84 MHz and PCLK1 42 MHz. Depending on the device voltage range, the maximum frequency should be adapted accordingly (refer to the product datasheets for more details).
3. For the STM32F42xxx, STM32F43xxx, STM32F446xx, STM32F469xx and STM32F479xx devices, the maximum frequency of the SYSCLK and HCLK is 180 MHz, PCLK2 90 MHz and PCLK1 45 MHz. Depending on the device voltage range, the maximum frequency should be adapted accordingly (refer to the product datasheets for more details).
4. For the STM32F401xx, the maximum frequency of the SYSCLK and HCLK is 84 MHz, PCLK2 84 MHz and PCLK1 42 MHz. Depending on the device voltage range, the maximum frequency should be adapted accordingly (refer to the product datasheets for more details).
5. For the STM32F41xxx, the maximum frequency of the SYSCLK and HCLK is 100 MHz, PCLK2 100 MHz and PCLK1 50 MHz. Depending on the device voltage range, the maximum frequency should be adapted accordingly (refer to the product datasheets for more details).

This section contains the following APIs:

- [`HAL\_RCC\_DeInit\(\)`](#)
- [`HAL\_RCC\_OscConfig\(\)`](#)
- [`HAL\_RCC\_ClockConfig\(\)`](#)

54.2.4 Peripheral Control functions

This subsection provides a set of functions allowing to control the RCC Clocks frequencies.

This section contains the following APIs:

- [`HAL\_RCC\_MCOConfig\(\)`](#)
- [`HAL\_RCC\_EnableCSS\(\)`](#)
- [`HAL\_RCC\_DisableCSS\(\)`](#)
- [`HAL\_RCC\_GetSysClockFreq\(\)`](#)
- [`HAL\_RCC\_GetHCLKFreq\(\)`](#)
- [`HAL\_RCC\_GetPCLK1Freq\(\)`](#)
- [`HAL\_RCC\_GetPCLK2Freq\(\)`](#)
- [`HAL\_RCC\_GetOscConfig\(\)`](#)
- [`HAL\_RCC\_GetClockConfig\(\)`](#)
- [`HAL\_RCC\_NMI\_IRQHandler\(\)`](#)
- [`HAL\_RCC\_CSSCallback\(\)`](#)

54.2.5 Detailed description of functions

HAL_RCC_DeInit

Function name

`HAL_StatusTypeDef HAL_RCC_DeInit (void )`

Function description

Resets the RCC clock configuration to the default reset state.

Return values

- **HAL:** status

Notes

- The default reset state of the clock configuration is given below: HSI ON and used as system clock source HSE and PLL OFF AHB, APB1 and APB2 prescaler set to 1. CSS, MCO1 and MCO2 OFF All interrupts disabled
- This function doesn't modify the configuration of the Peripheral clocks LSI, LSE and RTC clocks

HAL_RCC_OscConfig

Function name

HAL_StatusTypeDef HAL_RCC_OscConfig (RCC_OscInitTypeDef * RCC_OscInitStruct)

Function description

Initializes the RCC Oscillators according to the specified parameters in the RCC_OscInitTypeDef.

Parameters

- **RCC_OscInitStruct:** pointer to an RCC_OscInitTypeDef structure that contains the configuration information for the RCC Oscillators.

Return values

- **HAL:** status

Notes

- The PLL is not disabled when used as system clock.
- Transitions LSE Bypass to LSE On and LSE On to LSE Bypass are not supported by this API. User should request a transition to LSE Off first and then LSE On or LSE Bypass.
- Transition HSE Bypass to HSE On and HSE On to HSE Bypass are not supported by this API. User should request a transition to HSE Off first and then HSE On or HSE Bypass.

HAL_RCC_ClockConfig

Function name

HAL_StatusTypeDef HAL_RCC_ClockConfig (RCC_ClkInitTypeDef * RCC_ClkInitStruct, uint32_t FLatency)

Function description

Initializes the CPU, AHB and APB busses clocks according to the specified parameters in the RCC_ClkInitStruct.

Parameters

- **RCC_ClkInitStruct:** pointer to an RCC_OscInitTypeDef structure that contains the configuration information for the RCC peripheral.
- **FLatency:** FLASH Latency, this parameter depend on device selected

Return values

- **None:**

Notes

- The SystemCoreClock CMSIS variable is used to store System Clock Frequency and updated by HAL_RCC_GetHCLKFreq() function called within this function
- The HSI is used (enabled by hardware) as system clock source after startup from Reset, wake-up from STOP and STANDBY mode, or in case of failure of the HSE used directly or indirectly as system clock (if the Clock Security System CSS is enabled).
- A switch from one clock source to another occurs only if the target clock source is ready (clock stable after startup delay or PLL locked). If a clock source which is not yet ready is selected, the switch will occur when the clock source will be ready.
- Depending on the device voltage range, the software has to set correctly HPRE[3:0] bits to ensure that HCLK not exceed the maximum allowed frequency (for more details refer to section above "Initialization/de-initialization functions")

HAL_RCC_MCOConfig

Function name

void HAL_RCC_MCOConfig (uint32_t RCC_MCOx, uint32_t RCC_MCOSource, uint32_t RCC_MCODiv)

Function description

Selects the clock source to output on MCO1 pin(PA8) or on MCO2 pin(PC9).

Parameters

- **RCC_MCOx**: specifies the output direction for the clock source. This parameter can be one of the following values:
 - RCC_MCO1: Clock source to output on MCO1 pin(PA8).
 - RCC_MCO2: Clock source to output on MCO2 pin(PC9).
- **RCC_MCOSource**: specifies the clock source to output. This parameter can be one of the following values:
 - RCC_MCO1SOURCE_HSI: HSI clock selected as MCO1 source
 - RCC_MCO1SOURCE_LSE: LSE clock selected as MCO1 source
 - RCC_MCO1SOURCE_HSE: HSE clock selected as MCO1 source
 - RCC_MCO1SOURCE_PLLCLK: main PLL clock selected as MCO1 source
 - RCC_MCO2SOURCE_SYSCLK: System clock (SYSCLK) selected as MCO2 source
 - RCC_MCO2SOURCE_PLLI2SCLK: PLLI2S clock selected as MCO2 source, available for all STM32F4 devices except STM32F410xx
 - RCC_MCO2SOURCE_I2SCLK: I2SCLK clock selected as MCO2 source, available only for STM32F410Rx devices
 - RCC_MCO2SOURCE_HSE: HSE clock selected as MCO2 source
 - RCC_MCO2SOURCE_PLLCLK: main PLL clock selected as MCO2 source
- **RCC_MCODiv**: specifies the MCOx prescaler. This parameter can be one of the following values:
 - RCC_MCODIV_1: no division applied to MCOx clock
 - RCC_MCODIV_2: division by 2 applied to MCOx clock
 - RCC_MCODIV_3: division by 3 applied to MCOx clock
 - RCC_MCODIV_4: division by 4 applied to MCOx clock
 - RCC_MCODIV_5: division by 5 applied to MCOx clock

Return values

- **None:**

Notes

- PA8/PC9 should be configured in alternate function mode.
- For STM32F410Rx devices to output I2SCLK clock on MCO2 you should have at least one of the SPI clocks enabled (SPI1, SPI2 or SPI5).

HAL_RCC_EnableCSS

Function name

void HAL_RCC_EnableCSS (void)

Function description

Enables the Clock Security System.

Return values

- **None:**

Notes

- If a failure is detected on the HSE oscillator clock, this oscillator is automatically disabled and an interrupt is generated to inform the software about the failure (Clock Security System Interrupt, CSSI), allowing the MCU to perform rescue operations. The CSSI is linked to the Cortex-M4 NMI (Non-Maskable Interrupt) exception vector.

HAL_RCC_DisableCSS

Function name

void HAL_RCC_DisableCSS (void)

Function description

Disables the Clock Security System.

Return values

- **None:**

HAL_RCC_GetSysClockFreq

Function name

uint32_t HAL_RCC_GetSysClockFreq (void)

Function description

Returns the SYSCLK frequency.

Return values

- **SYSCLK:** frequency

Notes

- The system frequency computed by this function is not the real frequency in the chip. It is calculated based on the predefined constant and the selected clock source:
- If SYSCLK source is HSI, function returns values based on HSI_VALUE(*)
- If SYSCLK source is HSE, function returns values based on HSE_VALUE(**)
- If SYSCLK source is PLL, function returns values based on HSE_VALUE(**) or HSI_VALUE(*) multiplied/divided by the PLL factors.
- (*) HSI_VALUE is a constant defined in stm32f4xx_hal_conf.h file (default value 16 MHz) but the real value may vary depending on the variations in voltage and temperature.
- (**) HSE_VALUE is a constant defined in stm32f4xx_hal_conf.h file (default value 25 MHz), user has to ensure that HSE_VALUE is same as the real frequency of the crystal used. Otherwise, this function may have wrong result.
- The result of this function could be not correct when using fractional value for HSE crystal.
- This function can be used by the user application to compute the baudrate for the communication peripherals or configure other parameters.
- Each time SYSCLK changes, this function must be called to update the right SYSCLK value. Otherwise, any configuration based on this function will be incorrect.

HAL_RCC_GetHCLKFreq

Function name

uint32_t HAL_RCC_GetHCLKFreq (void)

Function description

Returns the HCLK frequency.

Return values

- **HCLK:** frequency

Notes

- Each time HCLK changes, this function must be called to update the right HCLK value. Otherwise, any configuration based on this function will be incorrect.
- The SystemCoreClock CMSIS variable is used to store System Clock Frequency and updated within this function

HAL_RCC_GetPCLK1Freq

Function name

uint32_t HAL_RCC_GetPCLK1Freq (void)

Function description

Returns the PCLK1 frequency.

Return values

- **PCLK1:** frequency

Notes

- Each time PCLK1 changes, this function must be called to update the right PCLK1 value. Otherwise, any configuration based on this function will be incorrect.

HAL_RCC_GetPCLK2Freq

Function name

uint32_t HAL_RCC_GetPCLK2Freq (void)

Function description

Returns the PCLK2 frequency.

Return values

- **PCLK2:** frequency

Notes

- Each time PCLK2 changes, this function must be called to update the right PCLK2 value. Otherwise, any configuration based on this function will be incorrect.

HAL_RCC_GetOscConfig

Function name

void HAL_RCC_GetOscConfig (RCC_OscInitTypeDef * RCC_OscInitStruct)

Function description

Configures the RCC_OscInitStruct according to the internal RCC configuration registers.

Parameters

- **RCC_OscInitStruct:** pointer to an RCC_OscInitTypeDef structure that will be configured.

Return values

- **None:**

HAL_RCC_GetClockConfig

Function name

void HAL_RCC_GetClockConfig (RCC_ClkInitTypeDef * RCC_ClkInitStruct, uint32_t * pFLatency)

Function description

Configures the RCC_ClkInitStruct according to the internal RCC configuration registers.

Parameters

- **RCC_ClkInitStruct:** pointer to an RCC_ClkInitTypeDef structure that will be configured.
- **pFLatency:** Pointer on the Flash Latency.

Return values

- **None:**

HAL_RCC_NMI_IRQHandler

Function name

void HAL_RCC_NMI_IRQHandler (void)

Function description

This function handles the RCC CSS interrupt request.

Return values

- **None:**

Notes

- This API should be called under the NMI_Handler().

HAL_RCC_CSSCallback

Function name

void HAL_RCC_CSSCallback (void)

Function description

RCC Clock Security System interrupt callback.

Return values

- **None:**

54.3 RCC Firmware driver defines

The following section lists the various define and macros of the module.

54.3.1 RCC

RCC

AHB1 Peripheral Clock Enable Disable

__HAL_RCC_GPIOA_CLK_ENABLE

__HAL_RCC_GPIOB_CLK_ENABLE

__HAL_RCC_GPIOC_CLK_ENABLE

__HAL_RCC_GPIOH_CLK_ENABLE

__HAL_RCC_DMA1_CLK_ENABLE

__HAL_RCC_DMA2_CLK_ENABLE

__HAL_RCC_GPIOA_CLK_DISABLE

__HAL_RCC_GPIOB_CLK_DISABLE

__HAL_RCC_GPIOC_CLK_DISABLE

__HAL_RCC_GPIOH_CLK_DISABLE

__HAL_RCC_DMA1_CLK_DISABLE

__HAL_RCC_DMA2_CLK_DISABLE

AHB1 Force Release Reset

__HAL_RCC_AHB1_FORCE_RESET

__HAL_RCC_GPIOA_FORCE_RESET

__HAL_RCC_GPIOB_FORCE_RESET

__HAL_RCC_GPIOC_FORCE_RESET

__HAL_RCC_GPIOH_FORCE_RESET

__HAL_RCC_DMA1_FORCE_RESET

__HAL_RCC_DMA2_FORCE_RESET

__HAL_RCC_AHB1_RELEASE_RESET

__HAL_RCC_GPIOA_RELEASE_RESET

__HAL_RCC_GPIOB_RELEASE_RESET

__HAL_RCC_GPIOC_RELEASE_RESET

__HAL_RCC_GPIOH_RELEASE_RESET

__HAL_RCC_DMA1_RELEASE_RESET

__HAL_RCC_DMA2_RELEASE_RESET

AHB1 Peripheral Low Power Enable Disable

__HAL_RCC_GPIOA_CLK_SLEEP_ENABLE

__HAL_RCC_GPIOB_CLK_SLEEP_ENABLE

__HAL_RCC_GPIOC_CLK_SLEEP_ENABLE

__HAL_RCC_GPIOH_CLK_SLEEP_ENABLE
__HAL_RCC_DMA1_CLK_SLEEP_ENABLE
__HAL_RCC_DMA2_CLK_SLEEP_ENABLE
__HAL_RCC_GPIOA_CLK_SLEEP_DISABLE
__HAL_RCC_GPIOB_CLK_SLEEP_DISABLE
__HAL_RCC_GPIOC_CLK_SLEEP_DISABLE
__HAL_RCC_GPIOH_CLK_SLEEP_DISABLE
__HAL_RCC_DMA1_CLK_SLEEP_DISABLE
__HAL_RCC_DMA2_CLK_SLEEP_DISABLE

AHB1 Peripheral Clock Enable Disable Status

__HAL_RCC_GPIOA_IS_CLK_ENABLED
__HAL_RCC_GPIOB_IS_CLK_ENABLED
__HAL_RCC_GPIOC_IS_CLK_ENABLED
__HAL_RCC_GPIOH_IS_CLK_ENABLED
__HAL_RCC_DMA1_IS_CLK_ENABLED
__HAL_RCC_DMA2_IS_CLK_ENABLED
__HAL_RCC_GPIOA_IS_CLK_DISABLED
__HAL_RCC_GPIOB_IS_CLK_DISABLED
__HAL_RCC_GPIOC_IS_CLK_DISABLED
__HAL_RCC_GPIOH_IS_CLK_DISABLED
__HAL_RCC_DMA1_IS_CLK_DISABLED
__HAL_RCC_DMA2_IS_CLK_DISABLED

AHB Clock Source

RCC_SYSCLK_DIV1
RCC_SYSCLK_DIV2
RCC_SYSCLK_DIV4
RCC_SYSCLK_DIV8
RCC_SYSCLK_DIV16

RCC_SYSCLK_DIV64

RCC_SYSCLK_DIV128

RCC_SYSCLK_DIV256

RCC_SYSCLK_DIV512

APB1/APB2 Clock Source

RCC_HCLK_DIV1

RCC_HCLK_DIV2

RCC_HCLK_DIV4

RCC_HCLK_DIV8

RCC_HCLK_DIV16

APB1 Peripheral Clock Enable Disable

__HAL_RCC_TIM5_CLK_ENABLE

__HAL_RCC_WWDG_CLK_ENABLE

__HAL_RCC_SPI2_CLK_ENABLE

__HAL_RCC_USART2_CLK_ENABLE

__HAL_RCC_I2C1_CLK_ENABLE

__HAL_RCC_I2C2_CLK_ENABLE

__HAL_RCC_PWR_CLK_ENABLE

__HAL_RCC_TIM5_CLK_DISABLE

__HAL_RCC_WWDG_CLK_DISABLE

__HAL_RCC_SPI2_CLK_DISABLE

__HAL_RCC_USART2_CLK_DISABLE

__HAL_RCC_I2C1_CLK_DISABLE

__HAL_RCC_I2C2_CLK_DISABLE

__HAL_RCC_PWR_CLK_DISABLE

APB1 Force Release Reset

__HAL_RCC_APB1_FORCE_RESET

__HAL_RCC_TIM5_FORCE_RESET

__HAL_RCC_WWDG_FORCE_RESET

__HAL_RCC_SPI2_FORCE_RESET

__HAL_RCC_USART2_FORCE_RESET

__HAL_RCC_I2C1_FORCE_RESET

__HAL_RCC_I2C2_FORCE_RESET

__HAL_RCC_PWR_FORCE_RESET

__HAL_RCC_APB1_RELEASE_RESET

__HAL_RCC_TIM5_RELEASE_RESET

__HAL_RCC_WWDG_RELEASE_RESET

__HAL_RCC_SPI2_RELEASE_RESET

__HAL_RCC_USART2_RELEASE_RESET

__HAL_RCC_I2C1_RELEASE_RESET

__HAL_RCC_I2C2_RELEASE_RESET

__HAL_RCC_PWR_RELEASE_RESET

APB1 Peripheral Low Power Enable Disable

__HAL_RCC_TIM5_CLK_SLEEP_ENABLE

__HAL_RCC_WWDG_CLK_SLEEP_ENABLE

__HAL_RCC_SPI2_CLK_SLEEP_ENABLE

__HAL_RCC_USART2_CLK_SLEEP_ENABLE

__HAL_RCC_I2C1_CLK_SLEEP_ENABLE

__HAL_RCC_I2C2_CLK_SLEEP_ENABLE

__HAL_RCC_PWR_CLK_SLEEP_ENABLE

__HAL_RCC_TIM5_CLK_SLEEP_DISABLE

__HAL_RCC_WWDG_CLK_SLEEP_DISABLE

__HAL_RCC_SPI2_CLK_SLEEP_DISABLE

__HAL_RCC_USART2_CLK_SLEEP_DISABLE

__HAL_RCC_I2C1_CLK_SLEEP_DISABLE

__HAL_RCC_I2C2_CLK_SLEEP_DISABLE

__HAL_RCC_PWR_CLK_SLEEP_DISABLE

APB1 Peripheral Clock Enable Disable Status

__HAL_RCC_TIM5_IS_CLK_ENABLED

__HAL_RCC_WWDG_IS_CLK_ENABLED

__HAL_RCC_SPI2_IS_CLK_ENABLED

__HAL_RCC_USART2_IS_CLK_ENABLED

__HAL_RCC_I2C1_IS_CLK_ENABLED

__HAL_RCC_I2C2_IS_CLK_ENABLED

__HAL_RCC_PWR_IS_CLK_ENABLED

__HAL_RCC_TIM5_IS_CLK_DISABLED

__HAL_RCC_WWDG_IS_CLK_DISABLED

__HAL_RCC_SPI2_IS_CLK_DISABLED

__HAL_RCC_USART2_IS_CLK_DISABLED

__HAL_RCC_I2C1_IS_CLK_DISABLED

__HAL_RCC_I2C2_IS_CLK_DISABLED

__HAL_RCC_PWR_IS_CLK_DISABLED

APB2 Peripheral Clock Enable Disable

__HAL_RCC_TIM1_CLK_ENABLE

__HAL_RCC_USART1_CLK_ENABLE

__HAL_RCC_USART6_CLK_ENABLE

__HAL_RCC_ADC1_CLK_ENABLE

__HAL_RCC_SPI1_CLK_ENABLE

__HAL_RCC_SYSCFG_CLK_ENABLE

__HAL_RCC_TIM9_CLK_ENABLE

__HAL_RCC_TIM11_CLK_ENABLE

__HAL_RCC_TIM1_CLK_DISABLE

__HAL_RCC_USART1_CLK_DISABLE

__HAL_RCC_USART6_CLK_DISABLE

__HAL_RCC_ADC1_CLK_DISABLE

__HAL_RCC_SPI1_CLK_DISABLE

__HAL_RCC_SYSCFG_CLK_DISABLE

__HAL_RCC_TIM9_CLK_DISABLE

__HAL_RCC_TIM11_CLK_DISABLE

APB2 Force Release Reset

__HAL_RCC_APB2_FORCE_RESET

__HAL_RCC_TIM1_FORCE_RESET

__HAL_RCC_USART1_FORCE_RESET

__HAL_RCC_USART6_FORCE_RESET

__HAL_RCC_ADC_FORCE_RESET

__HAL_RCC_SPI1_FORCE_RESET

__HAL_RCC_SYSCFG_FORCE_RESET

__HAL_RCC_TIM9_FORCE_RESET

__HAL_RCC_TIM11_FORCE_RESET

__HAL_RCC_APB2_RELEASE_RESET

__HAL_RCC_TIM1_RELEASE_RESET

__HAL_RCC_USART1_RELEASE_RESET

__HAL_RCC_USART6_RELEASE_RESET

__HAL_RCC_ADC_RELEASE_RESET

__HAL_RCC_SPI1_RELEASE_RESET

__HAL_RCC_SYSCFG_RELEASE_RESET

__HAL_RCC_TIM9_RELEASE_RESET

__HAL_RCC_TIM11_RELEASE_RESET

APB2 Peripheral Low Power Enable Disable

__HAL_RCC_TIM1_CLK_SLEEP_ENABLE

__HAL_RCC_USART1_CLK_SLEEP_ENABLE

__HAL_RCC_USART6_CLK_SLEEP_ENABLE

`__HAL_RCC_ADC1_CLK_SLEEP_ENABLE`
`__HAL_RCC_SPI1_CLK_SLEEP_ENABLE`
`__HAL_RCC_SYSCFG_CLK_SLEEP_ENABLE`
`__HAL_RCC_TIM9_CLK_SLEEP_ENABLE`
`__HAL_RCC_TIM11_CLK_SLEEP_ENABLE`
`__HAL_RCC_TIM1_CLK_SLEEP_DISABLE`
`__HAL_RCC_USART1_CLK_SLEEP_DISABLE`
`__HAL_RCC_USART6_CLK_SLEEP_DISABLE`
`__HAL_RCC_ADC1_CLK_SLEEP_DISABLE`
`__HAL_RCC_SPI1_CLK_SLEEP_DISABLE`
`__HAL_RCC_SYSCFG_CLK_SLEEP_DISABLE`
`__HAL_RCC_TIM9_CLK_SLEEP_DISABLE`
`__HAL_RCC_TIM11_CLK_SLEEP_DISABLE`

APB2 Peripheral Clock Enable Disable Status

`__HAL_RCC_TIM1_IS_CLK_ENABLED`
`__HAL_RCC_USART1_IS_CLK_ENABLED`
`__HAL_RCC_USART6_IS_CLK_ENABLED`
`__HAL_RCC_ADC1_IS_CLK_ENABLED`
`__HAL_RCC_SPI1_IS_CLK_ENABLED`
`__HAL_RCC_SYSCFG_IS_CLK_ENABLED`
`__HAL_RCC_TIM9_IS_CLK_ENABLED`
`__HAL_RCC_TIM11_IS_CLK_ENABLED`
`__HAL_RCC_TIM1_IS_CLK_DISABLED`
`__HAL_RCC_USART1_IS_CLK_DISABLED`
`__HAL_RCC_USART6_IS_CLK_DISABLED`
`__HAL_RCC_ADC1_IS_CLK_DISABLED`
`__HAL_RCC_SPI1_IS_CLK_DISABLED`
`__HAL_RCC_SYSCFG_IS_CLK_DISABLED`

__HAL_RCC_TIM9_IS_CLK_DISABLED

__HAL_RCC_TIM11_IS_CLK_DISABLED

RCC BitAddress AliasRegion

RCC_OFFSET

RCC_CR_OFFSET

RCC_HSION_BIT_NUMBER

RCC_CR_HSION_BB

RCC_CSSON_BIT_NUMBER

RCC_CR_CSSON_BB

RCC_PLLON_BIT_NUMBER

RCC_CR_PLLON_BB

RCC_BDCR_OFFSET

RCC_RTCEN_BIT_NUMBER

RCC_BDCR_RTCEN_BB

RCC_BDRST_BIT_NUMBER

RCC_BDCR_BDRST_BB

RCC_CSR_OFFSET

RCC_LSION_BIT_NUMBER

RCC_CSR_LSION_BB

RCC_CR_BYTE2_ADDRESS

RCC_CIR_BYTE1_ADDRESS

RCC_CIR_BYTE2_ADDRESS

RCC_BDCR_BYTE0_ADDRESS

RCC_DBP_TIMEOUT_VALUE

RCC_LSE_TIMEOUT_VALUE

HSE_TIMEOUT_VALUE

HSI_TIMEOUT_VALUE

LSI_TIMEOUT_VALUE

CLOCKSWITCH_TIMEOUT_VALUE

Flags

RCC_FLAG_HSIRDY

RCC_FLAG_HSERDY

RCC_FLAG_PLLRDY

RCC_FLAG_PLLI2SRDY

RCC_FLAG_LSERDY

RCC_FLAG_LSIRDY

RCC_FLAG BORRST

RCC_FLAG_PINRST

RCC_FLAG_PORRST

RCC_FLAG_SFTRST

RCC_FLAG_IWDGRST

RCC_FLAG_WWDGRST

RCC_FLAG_LPWRRST

Flags Interrupts Management

__HAL_RCC_ENABLE_IT

Description:

- Enable RCC interrupt (Perform Byte access to RCC_CIR[14:8] bits to enable the selected interrupts).

Parameters:

- __INTERRUPT__: specifies the RCC interrupt sources to be enabled. This parameter can be any combination of the following values:
 - RCC_IT_LSIRDY: LSI ready interrupt.
 - RCC_IT_LSERDY: LSE ready interrupt.
 - RCC_IT_HSIRDY: HSI ready interrupt.
 - RCC_IT_HSERDY: HSE ready interrupt.
 - RCC_IT_PLLRDY: Main PLL ready interrupt.
 - RCC_IT_PLLI2SRDY: PLLI2S ready interrupt.

__HAL_RCC_DISABLE_IT

Description:

- Disable RCC interrupt (Perform Byte access to RCC_CIR[14:8] bits to disable the selected interrupts).

Parameters:

- `__INTERRUPT__`: specifies the RCC interrupt sources to be disabled. This parameter can be any combination of the following values:
 - `RCC_IT_LSIRDY`: LSI ready interrupt.
 - `RCC_IT_LSERDY`: LSE ready interrupt.
 - `RCC_IT_HSIRDY`: HSI ready interrupt.
 - `RCC_IT_HSERDY`: HSE ready interrupt.
 - `RCC_IT_PLLRDY`: Main PLL ready interrupt.
 - `RCC_IT_PLLI2SRDY`: PLLI2S ready interrupt.

__HAL_RCC_CLEAR_IT

Description:

- Clear the RCC's interrupt pending bits (Perform Byte access to RCC_CIR[23:16] bits to clear the selected interrupt pending bits).

Parameters:

- `__INTERRUPT__`: specifies the interrupt pending bit to clear. This parameter can be any combination of the following values:
 - `RCC_IT_LSIRDY`: LSI ready interrupt.
 - `RCC_IT_LSERDY`: LSE ready interrupt.
 - `RCC_IT_HSIRDY`: HSI ready interrupt.
 - `RCC_IT_HSERDY`: HSE ready interrupt.
 - `RCC_IT_PLLRDY`: Main PLL ready interrupt.
 - `RCC_IT_PLLI2SRDY`: PLLI2S ready interrupt.
 - `RCC_IT_CSS`: Clock Security System interrupt

__HAL_RCC_GET_IT

Description:

- Check the RCC's interrupt has occurred or not.

Parameters:

- `__INTERRUPT__`: specifies the RCC interrupt source to check. This parameter can be one of the following values:
 - `RCC_IT_LSIRDY`: LSI ready interrupt.
 - `RCC_IT_LSERDY`: LSE ready interrupt.
 - `RCC_IT_HSIRDY`: HSI ready interrupt.
 - `RCC_IT_HSERDY`: HSE ready interrupt.
 - `RCC_IT_PLLRDY`: Main PLL ready interrupt.
 - `RCC_IT_PLLI2SRDY`: PLLI2S ready interrupt.
 - `RCC_IT_CSS`: Clock Security System interrupt

Return value:

- The: new state of `__INTERRUPT__` (TRUE or FALSE).

__HAL_RCC_CLEAR_RESET_FLAGS

RCC_FLAG_MASK

Description:

- Check RCC flag is set or not.

Parameters:

- `__FLAG__`: specifies the flag to check. This parameter can be one of the following values:
 - `RCC_FLAG_HSIIRDY`: HSI oscillator clock ready.
 - `RCC_FLAG_HSERDY`: HSE oscillator clock ready.
 - `RCC_FLAG_PLLRDY`: Main PLL clock ready.
 - `RCC_FLAG_PLLI2SRDY`: PLLI2S clock ready.
 - `RCC_FLAG_LSERDY`: LSE oscillator clock ready.
 - `RCC_FLAG_LSIRDY`: LSI oscillator clock ready.
 - `RCC_FLAG_BORRST`: POR/PDR or BOR reset.
 - `RCC_FLAG_PINRST`: Pin reset.
 - `RCC_FLAG_PORRST`: POR/PDR reset.
 - `RCC_FLAG_SFTRST`: Software reset.
 - `RCC_FLAG_IWDGRST`: Independent Watchdog reset.
 - `RCC_FLAG_WWDGRST`: Window Watchdog reset.
 - `RCC_FLAG_LPWRST`: Low Power reset.

Return value:

- The: new state of `__FLAG__` (TRUE or FALSE).

__HAL_RCC_GET_FLAG

Get Clock source

__HAL_RCC_SYSCCLK_CONFIG

Description:

- Macro to configure the system clock source.

Parameters:

- `__RCC_SYSCCLKSOURCE__`: specifies the system clock source. This parameter can be one of the following values:
 - `RCC_SYSCCLKSOURCE_HSI`: HSI oscillator is used as system clock source.
 - `RCC_SYSCCLKSOURCE_HSE`: HSE oscillator is used as system clock source.
 - `RCC_SYSCCLKSOURCE_PLLCLK`: PLL output is used as system clock source.
 - `RCC_SYSCCLKSOURCE_PLLRCLK`: PLLR output is used as system clock source. This parameter is available only for STM32F446xx devices.

__HAL_RCC_GET_SYSCCLK_SOURCE

Description:

- Macro to get the clock source used as system clock.

Return value:

- The: clock source used as system clock. The returned value can be one of the following:
 - `RCC_SYSCCLKSOURCE_STATUS_HSI`: HSI used as system clock.
 - `RCC_SYSCCLKSOURCE_STATUS_HSE`: HSE used as system clock.
 - `RCC_SYSCCLKSOURCE_STATUS_PLLCLK`: PLL used as system clock.
 - `RCC_SYSCCLKSOURCE_STATUS_PLLRCLK`: PLLR used as system clock. This parameter is available only for STM32F446xx devices.

__HAL_RCC_GET_PLL_OSCSOURCE

Description:

- Macro to get the oscillator used as PLL clock source.

Return value:

- The: oscillator used as PLL clock source. The returned value can be one of the following:
 - RCC_PLLSOURCE_HSI: HSI oscillator is used as PLL clock source.
 - RCC_PLLSOURCE_HSE: HSE oscillator is used as PLL clock source.

HSE Config

RCC_HSE_OFF

RCC_HSE_ON

RCC_HSE_BYPASS

HSE Configuration

__HAL_RCC_HSE_CONFIG

Description:

- Macro to configure the External High Speed oscillator (HSE).

Parameters:

- __STATE__: specifies the new state of the HSE. This parameter can be one of the following values:
 - RCC_HSE_OFF: turn OFF the HSE oscillator, HSERDY flag goes low after 6 HSE oscillator clock cycles.
 - RCC_HSE_ON: turn ON the HSE oscillator.
 - RCC_HSE_BYPASS: HSE oscillator bypassed with external clock.

Notes:

- Transition HSE Bypass to HSE On and HSE On to HSE Bypass are not supported by this macro. User should request a transition to HSE Off first and then HSE On or HSE Bypass. After enabling the HSE (RCC_HSE_ON or RCC_HSE_Bypass), the application software should wait on HSERDY flag to be set indicating that HSE clock is stable and can be used to clock the PLL and/or system clock. HSE state can not be changed if it is used directly or through the PLL as system clock. In this case, you have to select another source of the system clock then change the HSE state (ex. disable it). The HSE is stopped by hardware when entering STOP and STANDBY modes. This function reset the CSSON bit, so if the clock security system(CSS) was previously enabled you have to enable it again after calling this function.

HSI Config

RCC_HSI_OFF

RCC_HSI_ON

RCC_HSICALIBRATION_DEFAULT

HSI Configuration

__HAL_RCC_HSI_ENABLE

Notes:

- The HSI is stopped by hardware when entering STOP and STANDBY modes. It is used (enabled by hardware) as system clock source after startup from Reset, wake-up from STOP and STANDBY mode, or in case of failure of the HSE used directly or indirectly as system clock (if the Clock Security System CSS is enabled). HSI can not be stopped if it is used as system clock source. In this case, you have to select another source of the system clock then stop the HSI. After enabling the HSI, the application software should wait on HSIRDY flag to be set indicating that HSI clock is stable and can be used as system clock source. This parameter can be: ENABLE or DISABLE. When the HSI is stopped, HSIRDY flag goes low after 6 HSI oscillator clock cycles.

__HAL_RCC_HSI_DISABLE

__HAL_RCC_HSI_CALIBRATIONVALUE_ADJUST

Description:

- Macro to adjust the Internal High Speed oscillator (HSI) calibration value.

Parameters:

- `__HSICalibrationValue__`: specifies the calibration trimming value. (default is `RCC_HSICALIBRATION_DEFAULT`). This parameter must be a number between 0 and 0x1F.

Notes:

- The calibration is used to compensate for the variations in voltage and temperature that influence the frequency of the internal HSI RC.

RTC Clock Configuration

__HAL_RCC_RTC_ENABLE

Notes:

- These macros must be used only after the RTC clock source was selected.

__HAL_RCC_RTC_DISABLE

__HAL_RCC_RTC_CLKPRESCALER

Description:

- Macros to configure the RTC clock (RTCCLK).

Parameters:

- `__RTCCLKSource__`: specifies the RTC clock source. This parameter can be one of the following values:
 - `RCC_RTCCLKSOURCE_NO_CLK`: No clock selected as RTC clock.
 - `RCC_RTCCLKSOURCE_LSE`: LSE selected as RTC clock.
 - `RCC_RTCCLKSOURCE_LSI`: LSI selected as RTC clock.
 - `RCC_RTCCLKSOURCE_HSE_DIVX`: HSE divided by X selected as RTC clock (X can be retrieved thanks to `__HAL_RCC_GET_RTC_HSE_PRESCALER()`)

Notes:

- As the RTC clock configuration bits are in the Backup domain and write access is denied to this domain after reset, you have to enable write access using the Power Backup Access macro before to configure the RTC clock source (to be done once after reset). Once the RTC clock is configured it can't be changed unless the Backup domain is reset using `__HAL_RCC_BackupReset_RELEASE()` macro, or by a Power On Reset (POR).
- If the LSE or LSI is used as RTC clock source, the RTC continues to work in STOP and STANDBY modes, and can be used as wake-up source. However, when the HSE clock is used as RTC clock source, the RTC cannot be used in STOP and STANDBY modes. The maximum input clock frequency for RTC is 1MHz (when using HSE as RTC clock source).

__HAL_RCC_RTC_CONFIG

__HAL_RCC_GET_RTC_SOURCE

Description:

- Macro to get the RTC clock source.

Return value:

- The clock source can be one of the following values:
 - RCC_RTCCLKSOURCE_NO_CLK No clock selected as RTC clock
 - RCC_RTCCLKSOURCE_LSE LSE selected as RTC clock
 - RCC_RTCCLKSOURCE_LSI LSI selected as RTC clock
 - RCC_RTCCLKSOURCE_HSE_DIVX HSE divided by X selected as RTC clock (X can be retrieved thanks to __HAL_RCC_GET_RTC_HSE_PRESCALER())

__HAL_RCC_GET_RTC_HSE_PRESCALER

Description:

- Get the RTC and HSE clock divider (RTCPRE).

Return value:

- Returned: value can be one of the following values:
 - RCC_RTCCLKSOURCE_HSE_DIVX HSE divided by X selected as RTC clock (X can be retrieved thanks to __HAL_RCC_GET_RTC_HSE_PRESCALER())

__HAL_RCC_BACKUPRESET_FORCE

Notes:

- This function resets the RTC peripheral (including the backup registers) and the RTC clock source selection in RCC_CSR register. The BKPSRAM is not affected by this reset.

__HAL_RCC_BACKUPRESET_RELEASE

Interrupts

RCC_IT_LSIRDY

RCC_IT_LSERDY

RCC_IT_HSIRDY

RCC_IT_HSERDY

RCC_IT_PLLRDY

RCC_IT_PLLI2SRDY

RCC_IT_CSS

RCC Private macros to check input parameters

IS_RCC_OSCILLATORTYPE

IS_RCC_HSE

IS_RCC_LSE

IS_RCC_HSI

IS_RCC_LSI

IS_RCC_PLL

IS_RCC_PLLSOURCE

IS_RCC_SYSCLKSOURCE

IS_RCC_RTCCLKSOURCE

IS_RCC_PLLM_VALUE

IS_RCC_PLLP_VALUE

IS_RCC_PLLQ_VALUE

IS_RCC_HCLK

IS_RCC_CLOCKTYPE

IS_RCC_PCLK

IS_RCC_MCO

IS_RCC_MCO1SOURCE

IS_RCC_MCODIV

IS_RCC_CALIBRATION_VALUE

LSE Config

RCC_LSE_OFF

RCC_LSE_ON

RCC_LSE_BYPASS

LSE Configuration

__HAL_RCC_LSE_CONFIG

Description:

- Macro to configure the External Low Speed oscillator (LSE).

Parameters:

- `__STATE__`: specifies the new state of the LSE. This parameter can be one of the following values:
 - `RCC_LSE_OFF`: turn OFF the LSE oscillator, LSERDY flag goes low after 6 LSE oscillator clock cycles.
 - `RCC_LSE_ON`: turn ON the LSE oscillator.
 - `RCC_LSE_BYPASS`: LSE oscillator bypassed with external clock.

Notes:

- Transition LSE Bypass to LSE On and LSE On to LSE Bypass are not supported by this macro. User should request a transition to LSE Off first and then LSE On or LSE Bypass. As the LSE is in the Backup domain and write access is denied to this domain after reset, you have to enable write access using `HAL_PWR_EnableBkUpAccess()` function before to configure the LSE (to be done once after reset). After enabling the LSE (`RCC_LSE_ON` or `RCC_LSE_BYPASS`), the application software should wait on LSERDY flag to be set indicating that LSE clock is stable and can be used to clock the RTC.

LSI Config

RCC_LSI_OFF

RCC_LSI_ON

LSI Configuration

__HAL_RCC_LSI_ENABLE

Notes:

- After enabling the LSI, the application software should wait on LSIRDY flag to be set indicating that LSI clock is stable and can be used to clock the IWDG and/or the RTC. LSI can not be disabled if the IWDG is running. When the LSI is stopped, LSIRDY flag goes low after 6 LSI oscillator clock cycles.

__HAL_RCC_LSI_DISABLE

MCO1 Clock Source

RCC_MCO1SOURCE_HSI

RCC_MCO1SOURCE_LSE

RCC_MCO1SOURCE_HSE

RCC_MCO1SOURCE_PLLCLK

MCO2 Clock Source

RCC_MCO2SOURCE_SYSCLK

RCC_MCO2SOURCE_PLLI2SCLK

RCC_MCO2SOURCE_HSE

RCC_MCO2SOURCE_PLLCLK

MCOx Clock Prescaler

RCC_MCODIV_1

RCC_MCODIV_2

RCC_MCODIV_3

RCC_MCODIV_4

RCC_MCODIV_5

MCO Index

RCC_MCO1

RCC_MCO2

Oscillator Type

RCC_OSCILLATORTYPE_NONE

RCC_OSCILLATORTYPE_HSE

RCC_OSCILLATORTYPE_HSI

RCC_OSCILLATORTYPE_LSE

RCC_OSCILLATORTYPE_LSI

PLL Clock Divider

RCC_PLLP_DIV2

RCC_PLLP_DIV4

RCC_PLLP_DIV6

RCC_PLLP_DIV8

PLL Clock Source

RCC_PLLSOURCE_HSI

RCC_PLLSOURCE_HSE

PLL Config

RCC_PLL_NONE

RCC_PLL_OFF

RCC_PLL_ON

PLL Configuration

__HAL_RCC_PLL_ENABLE

Notes:

- After enabling the main PLL, the application software should wait on PLLRDY flag to be set indicating that PLL clock is stable and can be used as system clock source. The main PLL can not be disabled if it is used as system clock source. The main PLL is disabled by hardware when entering STOP and STANDBY modes.

__HAL_RCC_PLL_DISABLE

__HAL_RCC_PLL_PLLSOURCE_CONFIG

Description:

- Macro to configure the PLL clock source.

Parameters:

- `__PLLSOURCE__`: specifies the PLL entry clock source. This parameter can be one of the following values:
 - RCC_PLLSOURCE_HSI: HSI oscillator clock selected as PLL clock entry
 - RCC_PLLSOURCE_HSE: HSE oscillator clock selected as PLL clock entry

Notes:

- This function must be used only when the main PLL is disabled.

__HAL_RCC_PLL_PLLM_CONFIG

Description:

- Macro to configure the PLL multiplication factor.

Parameters:

- `__PLLM__`: specifies the division factor for PLL VCO input clock This parameter must be a number between Min_Data = 2 and Max_Data = 63.

Notes:

- This function must be used only when the main PLL is disabled.
- You have to set the PLLM parameter correctly to ensure that the VCO input frequency ranges from 1 to 2 MHz. It is recommended to select a frequency of 2 MHz to limit PLL jitter.

RTC Clock Source

RCC_RTCCLKSOURCE_NO_CLK

RCC_RTCCLKSOURCE_LSE

RCC_RTCCLKSOURCE_LSI

RCC_RTCCLKSOURCE_HSE_DIVX

RCC_RTCCLKSOURCE_HSE_DIV2

RCC_RTCCLKSOURCE_HSE_DIV3

RCC_RTCCLKSOURCE_HSE_DIV4

RCC_RTCCLKSOURCE_HSE_DIV5

RCC_RTCCLKSOURCE_HSE_DIV6

RCC_RTCCLKSOURCE_HSE_DIV7

RCC_RTCCLKSOURCE_HSE_DIV8

RCC_RTCCLKSOURCE_HSE_DIV9

RCC_RTCCLKSOURCE_HSE_DIV10

RCC_RTCCLKSOURCE_HSE_DIV11

RCC_RTCCLKSOURCE_HSE_DIV12

RCC_RTCCLKSOURCE_HSE_DIV13

RCC_RTCCLKSOURCE_HSE_DIV14

RCC_RTCCLKSOURCE_HSE_DIV15

RCC_RTCCLKSOURCE_HSE_DIV16

RCC_RTCCLKSOURCE_HSE_DIV17

RCC_RTCCLKSOURCE_HSE_DIV18

RCC_RTCCLKSOURCE_HSE_DIV19

RCC_RTCCLKSOURCE_HSE_DIV20

RCC_RTCCLKSOURCE_HSE_DIV21

RCC_RTCCLKSOURCE_HSE_DIV22

RCC_RTCCLKSOURCE_HSE_DIV23

RCC_RTCCLKSOURCE_HSE_DIV24

RCC_RTCCLKSOURCE_HSE_DIV25

RCC_RTCCLKSOURCE_HSE_DIV26

RCC_RTCCLKSOURCE_HSE_DIV27

RCC_RTCCLKSOURCE_HSE_DIV28

RCC_RTCCLKSOURCE_HSE_DIV29

RCC_RTCCLKSOURCE_HSE_DIV30

RCC_RTCCLKSOURCE_HSE_DIV31

System Clock Source

RCC_SYSCLKSOURCE_HSI

RCC_SYSCLKSOURCE_HSE

RCC_SYSCLKSOURCE_PLLCLK

RCC_SYSCLKSOURCE_PLLRCLK

System Clock Source Status

RCC_SYSCLKSOURCE_STATUS_HSI

HSI used as system clock

RCC_SYSCLKSOURCE_STATUS_HSE

HSE used as system clock

RCC_SYSCLKSOURCE_STATUS_PLLCLK

PLL used as system clock

RCC_SYSCLKSOURCE_STATUS_PLLRCLK

PLLR used as system clock

System Clock Type

RCC_CLOCKTYPE_SYSCLK

RCC_CLOCKTYPE_HCLK

RCC_CLOCKTYPE_PCLK1



RCC_CLOCKTYPE_PCLK2

55 HAL RCC Extension Driver

55.1 RCCEX Firmware driver registers structures

55.1.1 RCC_PLLInitTypeDef

RCC_PLLInitTypeDef is defined in the stm32f4xx_hal_rcc_ex.h

Data Fields

- **uint32_t PLLState**
- **uint32_t PLLSource**
- **uint32_t PLLM**
- **uint32_t PLLN**
- **uint32_t PLLP**
- **uint32_t PLLQ**
- **uint32_t PLLR**

Field Documentation

- **uint32_t RCC_PLLInitTypeDef::PLLState**
The new state of the PLL. This parameter can be a value of [RCC_PLL_Config](#)
- **uint32_t RCC_PLLInitTypeDef::PLLSource**
RCC_PLLSource: PLL entry clock source. This parameter must be a value of [RCC_PLL_Clock_Source](#)
- **uint32_t RCC_PLLInitTypeDef::PLLM**
PLLM: Division factor for PLL VCO input clock. This parameter must be a number between Min_Data = 0 and Max_Data = 63
- **uint32_t RCC_PLLInitTypeDef::PLLN**
PLLN: Multiplication factor for PLL VCO output clock. This parameter must be a number between Min_Data = 50 and Max_Data = 432 except for STM32F411xE devices where the Min_Data = 192
- **uint32_t RCC_PLLInitTypeDef::PLLP**
PLLP: Division factor for main system clock (SYSCLK). This parameter must be a value of [RCC_PLLP_Clock_Divider](#)
- **uint32_t RCC_PLLInitTypeDef::PLLQ**
PLLQ: Division factor for OTG FS, SDIO and RNG clocks. This parameter must be a number between Min_Data = 2 and Max_Data = 15
- **uint32_t RCC_PLLInitTypeDef::PLLR**
PLLR: PLL division factor for I2S, SAI, SYSTEM, SPDIFRX clocks. This parameter is only available in STM32F410xx/STM32F446xx/STM32F469xx/STM32F479xx and STM32F412Zx/STM32F412Vx/STM32F412Rx/STM32F412Cx/STM32F413xx/STM32F423xx devices. This parameter must be a number between Min_Data = 2 and Max_Data = 7

55.1.2 RCC_PLLI2SInitTypeDef

RCC_PLLI2SInitTypeDef is defined in the stm32f4xx_hal_rcc_ex.h

Data Fields

- **uint32_t PLLI2SN**
- **uint32_t PLLI2SR**
- **uint32_t PLLI2SQ**

Field Documentation

- **uint32_t RCC_PLLI2SInitTypeDef::PLLI2SN**
Specifies the multiplication factor for PLLI2S VCO output clock. This parameter must be a number between Min_Data = 50 and Max_Data = 432. This parameter will be used only when PLLI2S is selected as Clock Source I2S or SAI
- **uint32_t RCC_PLLI2SInitTypeDef::PLLI2SR**
Specifies the division factor for I2S clock. This parameter must be a number between Min_Data = 2 and Max_Data = 7. This parameter will be used only when PLLI2S is selected as Clock Source I2S or SAI

- **`uint32_t RCC_PLLI2SInitTypeDef::PLLI2SQ`**
Specifies the division factor for SAI1 clock. This parameter must be a number between Min_Data = 2 and Max_Data = 15. This parameter will be used only when PLLI2S is selected as Clock Source SAI

55.1.3

RCC_PLLSAIInitTypeDef

`RCC_PLLSAIInitTypeDef` is defined in the `stm32f4xx_hal_rcc_ex.h`

Data Fields

- **`uint32_t PLLSAIN`**
- **`uint32_t PLLSAIP`**
- **`uint32_t PLLSAIQ`**
- **`uint32_t PLLSAIR`**

Field Documentation

- **`uint32_t RCC_PLLSAIInitTypeDef::PLLSAIN`**
Specifies the multiplication factor for PLLI2S VCO output clock. This parameter must be a number between Min_Data = 50 and Max_Data = 432. This parameter will be used only when PLLSAI is selected as Clock Source SAI or LTDC
- **`uint32_t RCC_PLLSAIInitTypeDef::PLLSAIP`**
Specifies division factor for OTG FS and SDIO clocks. This parameter is only available in STM32F469xx/STM32F479xx devices. This parameter must be a value of [RCCEX_PLLSAIP_Clock_Divider](#)
- **`uint32_t RCC_PLLSAIInitTypeDef::PLLSAIQ`**
Specifies the division factor for SAI1 clock. This parameter must be a number between Min_Data = 2 and Max_Data = 15. This parameter will be used only when PLLSAI is selected as Clock Source SAI or LTDC
- **`uint32_t RCC_PLLSAIInitTypeDef::PLLSAIR`**
specifies the division factor for LTDC clock This parameter must be a number between Min_Data = 2 and Max_Data = 7. This parameter will be used only when PLLSAI is selected as Clock Source LTDC

55.1.4

RCC_PeriphCLKInitTypeDef

`RCC_PeriphCLKInitTypeDef` is defined in the `stm32f4xx_hal_rcc_ex.h`

Data Fields

- **`uint32_t PeriphClockSelection`**
- **`RCC_PLLI2SInitTypeDef PLLI2S`**
- **`RCC_PLLSAIInitTypeDef PLLSAI`**
- **`uint32_t PLLI2SDivQ`**
- **`uint32_t PLLSAIDivQ`**
- **`uint32_t PLLSAIDivR`**
- **`uint32_t RTCClockSelection`**
- **`uint8_t TIMPresSelection`**
- **`uint32_t Clk48ClockSelection`**
- **`uint32_t SdioClockSelection`**

Field Documentation

- **`uint32_t RCC_PeriphCLKInitTypeDef::PeriphClockSelection`**
The Extended Clock to be configured. This parameter can be a value of [RCCEX_Periph_Clock_Selection](#)
- **`RCC_PLLI2SInitTypeDef RCC_PeriphCLKInitTypeDef::PLLI2S`**
PLL I2S structure parameters. This parameter will be used only when PLLI2S is selected as Clock Source I2S or SAI
- **`RCC_PLLSAIInitTypeDef RCC_PeriphCLKInitTypeDef::PLLSAI`**
PLL SAI structure parameters. This parameter will be used only when PLLI2S is selected as Clock Source SAI or LTDC
- **`uint32_t RCC_PeriphCLKInitTypeDef::PLLI2SDivQ`**
Specifies the PLLI2S division factor for SAI1 clock. This parameter must be a number between Min_Data = 1 and Max_Data = 32 This parameter will be used only when PLLI2S is selected as Clock Source SAI
- **`uint32_t RCC_PeriphCLKInitTypeDef::PLLSAIDivQ`**
Specifies the PLLI2S division factor for SAI1 clock. This parameter must be a number between Min_Data = 1 and Max_Data = 32 This parameter will be used only when PLLSAI is selected as Clock Source SAI

- **`uint32_t RCC_PeriphCLKInitTypeDef::PLLSAIDivR`**
Specifies the PLLSAI division factor for LTDC clock. This parameter must be one value of [RCCEX_PLLSAI_DIVR](#)
- **`uint32_t RCC_PeriphCLKInitTypeDef::RTCClockSelection`**
Specifies RTC Clock Prescalers Selection. This parameter can be a value of [RCC_RTC_Clock_Source](#)
- **`uint8_t RCC_PeriphCLKInitTypeDef::TIMPresSelection`**
Specifies TIM Clock Prescalers Selection. This parameter can be a value of [RCCEX_TIM_PRescaler_Selection](#)
- **`uint32_t RCC_PeriphCLKInitTypeDef::Clk48ClockSelection`**
Specifies CLK48 Clock Selection this clock used OTG FS, SDIO and RNG clocks. This parameter can be a value of [RCCEX_CLK48_Clock_Source](#)
- **`uint32_t RCC_PeriphCLKInitTypeDef::SdioClockSelection`**
Specifies SDIO Clock Source Selection. This parameter can be a value of [RCCEX_SDIO_Clock_Source](#)

55.2 RCCEX Firmware driver API description

The following section lists the various functions of the RCCEX library.

55.2.1 Extended Peripheral Control functions

This subsection provides a set of functions allowing to control the RCC Clocks frequencies.

Note: *Important note: Care must be taken when `HAL_RCCEX_PeriphCLKConfig()` is used to select the RTC clock source; in this case the Backup domain will be reset in order to modify the RTC Clock source, as consequence RTC registers (including the backup registers) and `RCC_BDCR` register are set to their reset values.*

This section contains the following APIs:

- [HAL_RCCEX_PeriphCLKConfig\(\)](#)
- [HAL_RCCEX_GetPeriphCLKConfig\(\)](#)
- [HAL_RCCEX_GetPeriphCLKFreq\(\)](#)
- [HAL_RCCEX_SelectLSEMode\(\)](#)
- [HAL_RCCEX_EnablePLLI2S\(\)](#)
- [HAL_RCCEX_DisablePLLI2S\(\)](#)
- [HAL_RCCEX_EnablePLLSAI\(\)](#)
- [HAL_RCCEX_DisablePLLSAI\(\)](#)

55.2.2 Detailed description of functions

HAL_RCCEX_PeriphCLKConfig

Function name

`HAL_StatusTypeDef HAL_RCCEX_PeriphCLKConfig (RCC_PeriphCLKInitTypeDef * PeriphClkInit)`

Function description

Initializes the RCC extended peripherals clocks according to the specified parameters in the `RCC_PeriphCLKInitTypeDef`.

Parameters

- **PeriphClkInit:** pointer to an `RCC_PeriphCLKInitTypeDef` structure that contains the configuration information for the Extended Peripherals clocks(I2S, SAI, LTDC, RTC and TIM).

Return values

- **HAL:** status

Notes

- Care must be taken when `HAL_RCCEX_PeriphCLKConfig()` is used to select the RTC clock source; in this case the Backup domain will be reset in order to modify the RTC Clock source, as consequence RTC registers (including the backup registers) and `RCC_BDCR` register are set to their reset values.

HAL_RCCEx_GetPeriphCLKConfig

Function name

```
void HAL_RCCEx_GetPeriphCLKConfig (RCC_PeriphCLKInitTypeDef * PeriphClkInit)
```

Function description

Configures the RCC_PeriphCLKInitTypeDef according to the internal RCC configuration registers.

Parameters

- **PeriphClkInit:** pointer to an RCC_PeriphCLKInitTypeDef structure that will be configured.

Return values

- **None:**

HAL_RCCEx_GetPeriphCLKFreq

Function name

```
uint32_t HAL_RCCEx_GetPeriphCLKFreq (uint32_t PeriphClk)
```

Function description

Return the peripheral clock frequency for a given peripheral(SAI..)

Parameters

- **PeriphClk:** Peripheral clock identifier This parameter can be one of the following values:
 - RCC_PERIPHCLK_I2S: I2S peripheral clock

Return values

- **Frequency:** in KHz

Notes

- Return 0 if peripheral clock identifier not managed by this API

HAL_RCCEx_SelectLSEMode

Function name

```
void HAL_RCCEx_SelectLSEMode (uint8_t Mode)
```

Function description

Select LSE mode.

Parameters

- **Mode:** specifies the LSE mode. This parameter can be one of the following values:
 - RCC_LSE_LOWPOWER_MODE: LSE oscillator in low power mode selection
 - RCC_LSE_HIGHDRIVE_MODE: LSE oscillator in High Drive mode selection

Return values

- **None:**

Notes

- This mode is only available for STM32F410xx/STM32F411xx/STM32F446xx/STM32F469xx/STM32F479xx/STM32F412Zx/STM32F412Vx/STM32F412Rx/STM32F412Cx devices.

HAL_RCCEx_EnablePLLI2S

Function name

```
HAL_StatusTypeDef HAL_RCCEx_EnablePLLI2S (RCC_PLLI2SInitTypeDef * PLLI2SInit)
```

Function description

Enable PLLI2S.

Parameters

- **PLLI2SInit**: pointer to an RCC_PLLI2SInitTypeDef structure that contains the configuration information for the PLLI2S

Return values

- **HAL**: status

HAL_RCCEX_DisablePLLI2S

Function name

HAL_StatusTypeDef HAL_RCCEX_DisablePLLI2S (void)

Function description

Disable PLLI2S.

Return values

- **HAL**: status

HAL_RCCEX_EnablePLLSAI

Function name

HAL_StatusTypeDef HAL_RCCEX_EnablePLLSAI (RCC_PLLSAIInitTypeDef * PLLSAIInit)

Function description

Enable PLLSAI.

Parameters

- **PLLSAIInit**: pointer to an RCC_PLLSAIInitTypeDef structure that contains the configuration information for the PLLSAI

Return values

- **HAL**: status

HAL_RCCEX_DisablePLLSAI

Function name

HAL_StatusTypeDef HAL_RCCEX_DisablePLLSAI (void)

Function description

Disable PLLSAI.

Return values

- **HAL**: status

55.3 RCCEX Firmware driver defines

The following section lists the various define and macros of the module.

55.3.1 RCCEX

RCCEX

AHB1 Peripheral Clock Enable Disable

__HAL_RCC_BKPSRAM_CLK_ENABLE

__HAL_RCC_CCMDATARAMEN_CLK_ENABLE

```
__HAL_RCC_CRC_CLK_ENABLE
__HAL_RCC_GPIOD_CLK_ENABLE
__HAL_RCC_GPIOE_CLK_ENABLE
__HAL_RCC_GPIOI_CLK_ENABLE
__HAL_RCC_GPIOF_CLK_ENABLE
__HAL_RCC_GPIOG_CLK_ENABLE
__HAL_RCC_GPIOJ_CLK_ENABLE
__HAL_RCC_GPIOK_CLK_ENABLE
__HAL_RCC_DMA2D_CLK_ENABLE
__HAL_RCC_ETHMAC_CLK_ENABLE
__HAL_RCC_ETHMACTX_CLK_ENABLE
__HAL_RCC_ETHMACRX_CLK_ENABLE
__HAL_RCC_ETHMACPTP_CLK_ENABLE
__HAL_RCC_USB_OTG_HS_CLK_ENABLE
__HAL_RCC_USB_OTG_HS_ULPI_CLK_ENABLE
__HAL_RCC_GPIOD_CLK_DISABLE
__HAL_RCC_GPIOE_CLK_DISABLE
__HAL_RCC_GPIOF_CLK_DISABLE
__HAL_RCC_GPIOG_CLK_DISABLE
__HAL_RCC_GPIOI_CLK_DISABLE
__HAL_RCC_GPIOJ_CLK_DISABLE
__HAL_RCC_GPIOK_CLK_DISABLE
__HAL_RCC_DMA2D_CLK_DISABLE
__HAL_RCC_ETHMAC_CLK_DISABLE
__HAL_RCC_ETHMACTX_CLK_DISABLE
__HAL_RCC_ETHMACRX_CLK_DISABLE
__HAL_RCC_ETHMACPTP_CLK_DISABLE
__HAL_RCC_USB_OTG_HS_CLK_DISABLE
```

__HAL_RCC_USB_OTG_HS_ULPI_CLK_DISABLE

__HAL_RCC_BKPSRAM_CLK_DISABLE

__HAL_RCC_CCMDATARAMEN_CLK_DISABLE

__HAL_RCC_CRC_CLK_DISABLE

__HAL_RCC_ETH_CLK_ENABLE

__HAL_RCC_ETH_CLK_DISABLE

AHB1 Force Release Reset

__HAL_RCC_GPIOD_FORCE_RESET

__HAL_RCC_GPIOE_FORCE_RESET

__HAL_RCC_GPIOF_FORCE_RESET

__HAL_RCC_GPIOG_FORCE_RESET

__HAL_RCC_GPIOI_FORCE_RESET

__HAL_RCC_ETHMAC_FORCE_RESET

__HAL_RCC_USB_OTG_HS_FORCE_RESET

__HAL_RCC_GPIOJ_FORCE_RESET

__HAL_RCC_GPIOK_FORCE_RESET

__HAL_RCC_DMA2D_FORCE_RESET

__HAL_RCC_CRC_FORCE_RESET

__HAL_RCC_GPIOD_RELEASE_RESET

__HAL_RCC_GPIOE_RELEASE_RESET

__HAL_RCC_GPIOF_RELEASE_RESET

__HAL_RCC_GPIOG_RELEASE_RESET

__HAL_RCC_GPIOI_RELEASE_RESET

__HAL_RCC_ETHMAC_RELEASE_RESET

__HAL_RCC_USB_OTG_HS_RELEASE_RESET

__HAL_RCC_GPIOJ_RELEASE_RESET

__HAL_RCC_GPIOK_RELEASE_RESET

__HAL_RCC_DMA2D_RELEASE_RESET

__HAL_RCC_CRC_RELEASE_RESET

AHB1 Peripheral Low Power Enable Disable

__HAL_RCC_GPIOD_CLK_SLEEP_ENABLE

__HAL_RCC_GPIOE_CLK_SLEEP_ENABLE

__HAL_RCC_GPIOF_CLK_SLEEP_ENABLE

__HAL_RCC_GPIOG_CLK_SLEEP_ENABLE

__HAL_RCC_GPIOI_CLK_SLEEP_ENABLE

__HAL_RCC_SRAM2_CLK_SLEEP_ENABLE

__HAL_RCC_ETHMAC_CLK_SLEEP_ENABLE

__HAL_RCC_ETHMACTX_CLK_SLEEP_ENABLE

__HAL_RCC_ETHMACRX_CLK_SLEEP_ENABLE

__HAL_RCC_ETHMACPTP_CLK_SLEEP_ENABLE

__HAL_RCC_USB_OTG_HS_CLK_SLEEP_ENABLE

__HAL_RCC_USB_OTG_HS_ULPI_CLK_SLEEP_ENABLE

__HAL_RCC_GPIOJ_CLK_SLEEP_ENABLE

__HAL_RCC_GPIOK_CLK_SLEEP_ENABLE

__HAL_RCC_SRAM3_CLK_SLEEP_ENABLE

__HAL_RCC_DMA2D_CLK_SLEEP_ENABLE

__HAL_RCC_CRC_CLK_SLEEP_ENABLE

__HAL_RCC_FLITF_CLK_SLEEP_ENABLE

__HAL_RCC_SRAM1_CLK_SLEEP_ENABLE

__HAL_RCC_BKPSRAM_CLK_SLEEP_ENABLE

__HAL_RCC_GPIOD_CLK_SLEEP_DISABLE

__HAL_RCC_GPIOE_CLK_SLEEP_DISABLE

__HAL_RCC_GPIOF_CLK_SLEEP_DISABLE

__HAL_RCC_GPIOG_CLK_SLEEP_DISABLE

__HAL_RCC_GPIOI_CLK_SLEEP_DISABLE

__HAL_RCC_SRAM2_CLK_SLEEP_DISABLE

`__HAL_RCC_ETHMAC_CLK_SLEEP_DISABLE`
`__HAL_RCC_ETHMACTX_CLK_SLEEP_DISABLE`
`__HAL_RCC_ETHMACRX_CLK_SLEEP_DISABLE`
`__HAL_RCC_ETHMACPTP_CLK_SLEEP_DISABLE`
`__HAL_RCC_USB_OTG_HS_CLK_SLEEP_DISABLE`
`__HAL_RCC_USB_OTG_HS_ULPI_CLK_SLEEP_DISABLE`
`__HAL_RCC_GPIOJ_CLK_SLEEP_DISABLE`
`__HAL_RCC_GPIOK_CLK_SLEEP_DISABLE`
`__HAL_RCC_DMA2D_CLK_SLEEP_DISABLE`
`__HAL_RCC_CRC_CLK_SLEEP_DISABLE`
`__HAL_RCC_FLITF_CLK_SLEEP_DISABLE`
`__HAL_RCC_SRAM1_CLK_SLEEP_DISABLE`
`__HAL_RCC_BKPSRAM_CLK_SLEEP_DISABLE`

AHB1 Peripheral Clock Enable Disable Status

`__HAL_RCC_GPIOD_IS_CLK_ENABLED`
`__HAL_RCC_GPIOE_IS_CLK_ENABLED`
`__HAL_RCC_GPIOF_IS_CLK_ENABLED`
`__HAL_RCC_GPIOG_IS_CLK_ENABLED`
`__HAL_RCC_GPIOI_IS_CLK_ENABLED`
`__HAL_RCC_GPIOJ_IS_CLK_ENABLED`
`__HAL_RCC_GPIOK_IS_CLK_ENABLED`
`__HAL_RCC_DMA2D_IS_CLK_ENABLED`
`__HAL_RCC_ETHMAC_IS_CLK_ENABLED`
`__HAL_RCC_ETHMACTX_IS_CLK_ENABLED`
`__HAL_RCC_ETHMACRX_IS_CLK_ENABLED`
`__HAL_RCC_ETHMACPTP_IS_CLK_ENABLED`
`__HAL_RCC_USB_OTG_HS_IS_CLK_ENABLED`
`__HAL_RCC_USB_OTG_HS_ULPI_IS_CLK_ENABLED`

__HAL_RCC_BKPSRAM_IS_CLK_ENABLED
__HAL_RCC_CCMDATARAMEN_IS_CLK_ENABLED
__HAL_RCC_CRC_IS_CLK_ENABLED
__HAL_RCC_ETH_IS_CLK_ENABLED
__HAL_RCC_GPIOD_IS_CLK_DISABLED
__HAL_RCC_GPIOE_IS_CLK_DISABLED
__HAL_RCC_GPIOF_IS_CLK_DISABLED
__HAL_RCC_GPIOG_IS_CLK_DISABLED
__HAL_RCC_GPIOI_IS_CLK_DISABLED
__HAL_RCC_GPIOJ_IS_CLK_DISABLED
__HAL_RCC_GPIOK_IS_CLK_DISABLED
__HAL_RCC_DMA2D_IS_CLK_DISABLED
__HAL_RCC_ETHMAC_IS_CLK_DISABLED
__HAL_RCC_ETHMACTX_IS_CLK_DISABLED
__HAL_RCC_ETHMACRX_IS_CLK_DISABLED
__HAL_RCC_ETHMACPTP_IS_CLK_DISABLED
__HAL_RCC_USB_OTG_HS_IS_CLK_DISABLED
__HAL_RCC_USB_OTG_HS_ULPI_IS_CLK_DISABLED
__HAL_RCC_BKPSRAM_IS_CLK_DISABLED
__HAL_RCC_CCMDATARAMEN_IS_CLK_DISABLED
__HAL_RCC_CRC_IS_CLK_DISABLED
__HAL_RCC_ETH_IS_CLK_DISABLED

AHB2 Peripheral Clock Enable Disable

__HAL_RCC_DCMI_CLK_ENABLE
__HAL_RCC_DCMI_CLK_DISABLE
__HAL_RCC_Cryp_CLK_ENABLE
__HAL_RCC_HASH_CLK_ENABLE
__HAL_RCC_Cryp_CLK_DISABLE

__HAL_RCC_HASH_CLK_DISABLE

__HAL_RCC_USB_OTG_FS_CLK_ENABLE

__HAL_RCC_USB_OTG_FS_CLK_DISABLE

__HAL_RCC_RNG_CLK_ENABLE

__HAL_RCC_RNG_CLK_DISABLE

AHB2 Force Release Reset

__HAL_RCC_AHB2_FORCE_RESET

__HAL_RCC_USB_OTG_FS_FORCE_RESET

__HAL_RCC_RNG_FORCE_RESET

__HAL_RCC_DCMI_FORCE_RESET

__HAL_RCC_AHB2_RELEASE_RESET

__HAL_RCC_USB_OTG_FS_RELEASE_RESET

__HAL_RCC_RNG_RELEASE_RESET

__HAL_RCC_DCMI_RELEASE_RESET

__HAL_RCC_Cryp_FORCE_RESET

__HAL_RCC_HASH_FORCE_RESET

__HAL_RCC_Cryp_RELEASE_RESET

__HAL_RCC_HASH_RELEASE_RESET

AHB2 Peripheral Low Power Enable Disable

__HAL_RCC_USB_OTG_FS_CLK_SLEEP_ENABLE

__HAL_RCC_USB_OTG_FS_CLK_SLEEP_DISABLE

__HAL_RCC_RNG_CLK_SLEEP_ENABLE

__HAL_RCC_RNG_CLK_SLEEP_DISABLE

__HAL_RCC_DCMI_CLK_SLEEP_ENABLE

__HAL_RCC_DCMI_CLK_SLEEP_DISABLE

__HAL_RCC_Cryp_CLK_SLEEP_ENABLE

__HAL_RCC_HASH_CLK_SLEEP_ENABLE

__HAL_RCC_Cryp_CLK_SLEEP_DISABLE

__HAL_RCC_HASH_CLK_SLEEP_DISABLE

AHB2 Peripheral Clock Enable Disable Status

__HAL_RCC_DCMI_IS_CLK_ENABLED

__HAL_RCC_DCMI_IS_CLK_DISABLED

__HAL_RCC_Cryp_IS_CLK_ENABLED

__HAL_RCC_Cryp_IS_CLK_DISABLED

__HAL_RCC_HASH_IS_CLK_ENABLED

__HAL_RCC_HASH_IS_CLK_DISABLED

__HAL_RCC_USB_OTG_FS_IS_CLK_ENABLED

__HAL_RCC_USB_OTG_FS_IS_CLK_DISABLED

__HAL_RCC_RNG_IS_CLK_ENABLED

__HAL_RCC_RNG_IS_CLK_DISABLED

AHB3 Peripheral Clock Enable Disable

__HAL_RCC_FMC_CLK_ENABLE

__HAL_RCC_FMC_CLK_DISABLE

__HAL_RCC_QSPI_CLK_ENABLE

__HAL_RCC_QSPI_CLK_DISABLE

AHB3 Force Release Reset

__HAL_RCC_AHB3_FORCE_RESET

__HAL_RCC_AHB3_RELEASE_RESET

__HAL_RCC_FMC_FORCE_RESET

__HAL_RCC_FMC_RELEASE_RESET

__HAL_RCC_QSPI_FORCE_RESET

__HAL_RCC_QSPI_RELEASE_RESET

AHB3 Peripheral Low Power Enable Disable

__HAL_RCC_FMC_CLK_SLEEP_ENABLE

__HAL_RCC_FMC_CLK_SLEEP_DISABLE

__HAL_RCC_QSPI_CLK_SLEEP_ENABLE

__HAL_RCC_QSPI_CLK_SLEEP_DISABLE

AHB3 Peripheral Clock Enable Disable Status

__HAL_RCC_FMC_IS_CLK_ENABLED

__HAL_RCC_FMC_IS_CLK_DISABLED

__HAL_RCC_QSPI_IS_CLK_ENABLED

__HAL_RCC_QSPI_IS_CLK_DISABLED

APB1 Peripheral Clock Enable Disable

__HAL_RCC_TIM6_CLK_ENABLE

__HAL_RCC_TIM7_CLK_ENABLE

__HAL_RCC_TIM12_CLK_ENABLE

__HAL_RCC_TIM13_CLK_ENABLE

__HAL_RCC_TIM14_CLK_ENABLE

__HAL_RCC_TIM14_CLK_ENABLE

__HAL_RCC_USART3_CLK_ENABLE

__HAL_RCC_UART4_CLK_ENABLE

__HAL_RCC_UART5_CLK_ENABLE

__HAL_RCC_CAN1_CLK_ENABLE

__HAL_RCC_CAN2_CLK_ENABLE

__HAL_RCC_DAC_CLK_ENABLE

__HAL_RCC_UART7_CLK_ENABLE

__HAL_RCC_UART8_CLK_ENABLE

__HAL_RCC_TIM2_CLK_ENABLE

__HAL_RCC_TIM3_CLK_ENABLE

__HAL_RCC_TIM4_CLK_ENABLE

__HAL_RCC_SPI3_CLK_ENABLE

__HAL_RCC_I2C3_CLK_ENABLE

__HAL_RCC_TIM2_CLK_DISABLE

__HAL_RCC_TIM3_CLK_DISABLE

__HAL_RCC_TIM4_CLK_DISABLE
__HAL_RCC_SPI3_CLK_DISABLE
__HAL_RCC_I2C3_CLK_DISABLE
__HAL_RCC_TIM6_CLK_DISABLE
__HAL_RCC_TIM7_CLK_DISABLE
__HAL_RCC_TIM12_CLK_DISABLE
__HAL_RCC_TIM13_CLK_DISABLE
__HAL_RCC_TIM14_CLK_DISABLE
__HAL_RCC_USART3_CLK_DISABLE
__HAL_RCC_UART4_CLK_DISABLE
__HAL_RCC_UART5_CLK_DISABLE
__HAL_RCC_CAN1_CLK_DISABLE
__HAL_RCC_CAN2_CLK_DISABLE
__HAL_RCC_DAC_CLK_DISABLE
__HAL_RCC_UART7_CLK_DISABLE
__HAL_RCC_UART8_CLK_DISABLE

APB1 Force Release Reset

__HAL_RCC_TIM6_FORCE_RESET
__HAL_RCC_TIM7_FORCE_RESET
__HAL_RCC_TIM12_FORCE_RESET
__HAL_RCC_TIM13_FORCE_RESET
__HAL_RCC_TIM14_FORCE_RESET
__HAL_RCC_USART3_FORCE_RESET
__HAL_RCC_UART4_FORCE_RESET
__HAL_RCC_UART5_FORCE_RESET
__HAL_RCC_CAN1_FORCE_RESET
__HAL_RCC_CAN2_FORCE_RESET
__HAL_RCC_DAC_FORCE_RESET

__HAL_RCC_UART7_FORCE_RESET

__HAL_RCC_UART8_FORCE_RESET

__HAL_RCC_TIM2_FORCE_RESET

__HAL_RCC_TIM3_FORCE_RESET

__HAL_RCC_TIM4_FORCE_RESET

__HAL_RCC_SPI3_FORCE_RESET

__HAL_RCC_I2C3_FORCE_RESET

__HAL_RCC_TIM2_RELEASE_RESET

__HAL_RCC_TIM3_RELEASE_RESET

__HAL_RCC_TIM4_RELEASE_RESET

__HAL_RCC_SPI3_RELEASE_RESET

__HAL_RCC_I2C3_RELEASE_RESET

__HAL_RCC_TIM6_RELEASE_RESET

__HAL_RCC_TIM7_RELEASE_RESET

__HAL_RCC_TIM12_RELEASE_RESET

__HAL_RCC_TIM13_RELEASE_RESET

__HAL_RCC_TIM14_RELEASE_RESET

__HAL_RCC_USART3_RELEASE_RESET

__HAL_RCC_UART4_RELEASE_RESET

__HAL_RCC_UART5_RELEASE_RESET

__HAL_RCC_CAN1_RELEASE_RESET

__HAL_RCC_CAN2_RELEASE_RESET

__HAL_RCC_DAC_RELEASE_RESET

__HAL_RCC_UART7_RELEASE_RESET

__HAL_RCC_UART8_RELEASE_RESET

APB1 Peripheral Low Power Enable Disable

__HAL_RCC_TIM6_CLK_SLEEP_ENABLE

__HAL_RCC_TIM7_CLK_SLEEP_ENABLE

`__HAL_RCC_TIM12_CLK_SLEEP_ENABLE`
`__HAL_RCC_TIM13_CLK_SLEEP_ENABLE`
`__HAL_RCC_TIM14_CLK_SLEEP_ENABLE`
`__HAL_RCC_USART3_CLK_SLEEP_ENABLE`
`__HAL_RCC_UART4_CLK_SLEEP_ENABLE`
`__HAL_RCC_UART5_CLK_SLEEP_ENABLE`
`__HAL_RCC_CAN1_CLK_SLEEP_ENABLE`
`__HAL_RCC_CAN2_CLK_SLEEP_ENABLE`
`__HAL_RCC_DAC_CLK_SLEEP_ENABLE`
`__HAL_RCC_UART7_CLK_SLEEP_ENABLE`
`__HAL_RCC_UART8_CLK_SLEEP_ENABLE`
`__HAL_RCC_TIM2_CLK_SLEEP_ENABLE`
`__HAL_RCC_TIM3_CLK_SLEEP_ENABLE`
`__HAL_RCC_TIM4_CLK_SLEEP_ENABLE`
`__HAL_RCC_SPI3_CLK_SLEEP_ENABLE`
`__HAL_RCC_I2C3_CLK_SLEEP_ENABLE`
`__HAL_RCC_TIM2_CLK_SLEEP_DISABLE`
`__HAL_RCC_TIM3_CLK_SLEEP_DISABLE`
`__HAL_RCC_TIM4_CLK_SLEEP_DISABLE`
`__HAL_RCC_SPI3_CLK_SLEEP_DISABLE`
`__HAL_RCC_I2C3_CLK_SLEEP_DISABLE`
`__HAL_RCC_TIM6_CLK_SLEEP_DISABLE`
`__HAL_RCC_TIM7_CLK_SLEEP_DISABLE`
`__HAL_RCC_TIM12_CLK_SLEEP_DISABLE`
`__HAL_RCC_TIM13_CLK_SLEEP_DISABLE`
`__HAL_RCC_TIM14_CLK_SLEEP_DISABLE`
`__HAL_RCC_USART3_CLK_SLEEP_DISABLE`
`__HAL_RCC_UART4_CLK_SLEEP_DISABLE`

__HAL_RCC_UART5_CLK_SLEEP_DISABLE

__HAL_RCC_CAN1_CLK_SLEEP_DISABLE

__HAL_RCC_CAN2_CLK_SLEEP_DISABLE

__HAL_RCC_DAC_CLK_SLEEP_DISABLE

__HAL_RCC_UART7_CLK_SLEEP_DISABLE

__HAL_RCC_UART8_CLK_SLEEP_DISABLE

APB1 Peripheral Clock Enable Disable Status

__HAL_RCC_TIM2_IS_CLK_ENABLED

__HAL_RCC_TIM3_IS_CLK_ENABLED

__HAL_RCC_TIM4_IS_CLK_ENABLED

__HAL_RCC_SPI3_IS_CLK_ENABLED

__HAL_RCC_I2C3_IS_CLK_ENABLED

__HAL_RCC_TIM6_IS_CLK_ENABLED

__HAL_RCC_TIM7_IS_CLK_ENABLED

__HAL_RCC_TIM12_IS_CLK_ENABLED

__HAL_RCC_TIM13_IS_CLK_ENABLED

__HAL_RCC_TIM14_IS_CLK_ENABLED

__HAL_RCC_USART3_IS_CLK_ENABLED

__HAL_RCC_UART4_IS_CLK_ENABLED

__HAL_RCC_UART5_IS_CLK_ENABLED

__HAL_RCC_CAN1_IS_CLK_ENABLED

__HAL_RCC_CAN2_IS_CLK_ENABLED

__HAL_RCC_DAC_IS_CLK_ENABLED

__HAL_RCC_UART7_IS_CLK_ENABLED

__HAL_RCC_UART8_IS_CLK_ENABLED

__HAL_RCC_TIM2_IS_CLK_DISABLED

__HAL_RCC_TIM3_IS_CLK_DISABLED

__HAL_RCC_TIM4_IS_CLK_DISABLED

__HAL_RCC_SPI3_IS_CLK_DISABLED
__HAL_RCC_I2C3_IS_CLK_DISABLED
__HAL_RCC_TIM6_IS_CLK_DISABLED
__HAL_RCC_TIM7_IS_CLK_DISABLED
__HAL_RCC_TIM12_IS_CLK_DISABLED
__HAL_RCC_TIM13_IS_CLK_DISABLED
__HAL_RCC_TIM14_IS_CLK_DISABLED
__HAL_RCC_USART3_IS_CLK_DISABLED
__HAL_RCC_UART4_IS_CLK_DISABLED
__HAL_RCC_UART5_IS_CLK_DISABLED
__HAL_RCC_CAN1_IS_CLK_DISABLED
__HAL_RCC_CAN2_IS_CLK_DISABLED
__HAL_RCC_DAC_IS_CLK_DISABLED
__HAL_RCC_UART7_IS_CLK_DISABLED
__HAL_RCC_UART8_IS_CLK_DISABLED

APB2 Peripheral Clock Enable Disable

__HAL_RCC_TIM8_CLK_ENABLE
__HAL_RCC_ADC2_CLK_ENABLE
__HAL_RCC_ADC3_CLK_ENABLE
__HAL_RCC_SPI5_CLK_ENABLE
__HAL_RCC_SPI6_CLK_ENABLE
__HAL_RCC_SAI1_CLK_ENABLE
__HAL_RCC_SDIO_CLK_ENABLE
__HAL_RCC_SPI4_CLK_ENABLE
__HAL_RCC_TIM10_CLK_ENABLE
__HAL_RCC_SDIO_CLK_DISABLE
__HAL_RCC_SPI4_CLK_DISABLE
__HAL_RCC_TIM10_CLK_DISABLE

__HAL_RCC_TIM8_CLK_DISABLE
__HAL_RCC_ADC2_CLK_DISABLE
__HAL_RCC_ADC3_CLK_DISABLE
__HAL_RCC_SPI5_CLK_DISABLE
__HAL_RCC_SPI6_CLK_DISABLE
__HAL_RCC_SAI1_CLK_DISABLE
__HAL_RCC_LTDC_CLK_ENABLE
__HAL_RCC_LTDC_CLK_DISABLE
__HAL_RCC_DSI_CLK_ENABLE
__HAL_RCC_DSI_CLK_DISABLE

APB2 Force Release Reset

__HAL_RCC_TIM8_FORCE_RESET
__HAL_RCC_SPI5_FORCE_RESET
__HAL_RCC_SPI6_FORCE_RESET
__HAL_RCC_SAI1_FORCE_RESET
__HAL_RCC_SDIO_FORCE_RESET
__HAL_RCC_SPI4_FORCE_RESET
__HAL_RCC_TIM10_FORCE_RESET
__HAL_RCC_SDIO_RELEASE_RESET
__HAL_RCC_SPI4_RELEASE_RESET
__HAL_RCC_TIM10_RELEASE_RESET
__HAL_RCC_TIM8_RELEASE_RESET
__HAL_RCC_SPI5_RELEASE_RESET
__HAL_RCC_SPI6_RELEASE_RESET
__HAL_RCC_SAI1_RELEASE_RESET
__HAL_RCC_LTDC_FORCE_RESET
__HAL_RCC_LTDC_RELEASE_RESET
__HAL_RCC_DSI_FORCE_RESET

__HAL_RCC_DSI_RELEASE_RESET

APB2 Peripheral Low Power Enable Disable

__HAL_RCC_TIM8_CLK_SLEEP_ENABLE

__HAL_RCC_ADC2_CLK_SLEEP_ENABLE

__HAL_RCC_ADC3_CLK_SLEEP_ENABLE

__HAL_RCC_SPI5_CLK_SLEEP_ENABLE

__HAL_RCC_SPI6_CLK_SLEEP_ENABLE

__HAL_RCC_SAI1_CLK_SLEEP_ENABLE

__HAL_RCC_SDIO_CLK_SLEEP_ENABLE

__HAL_RCC_SPI4_CLK_SLEEP_ENABLE

__HAL_RCC_TIM10_CLK_SLEEP_ENABLE

__HAL_RCC_SDIO_CLK_SLEEP_DISABLE

__HAL_RCC_SPI4_CLK_SLEEP_DISABLE

__HAL_RCC_TIM10_CLK_SLEEP_DISABLE

__HAL_RCC_TIM8_CLK_SLEEP_DISABLE

__HAL_RCC_ADC2_CLK_SLEEP_DISABLE

__HAL_RCC_ADC3_CLK_SLEEP_DISABLE

__HAL_RCC_SPI5_CLK_SLEEP_DISABLE

__HAL_RCC_SPI6_CLK_SLEEP_DISABLE

__HAL_RCC_SAI1_CLK_SLEEP_DISABLE

__HAL_RCC_LTDC_CLK_SLEEP_ENABLE

__HAL_RCC_LTDC_CLK_SLEEP_DISABLE

__HAL_RCC_DSI_CLK_SLEEP_ENABLE

__HAL_RCC_DSI_CLK_SLEEP_DISABLE

APB2 Peripheral Clock Enable Disable Status

__HAL_RCC_TIM8_IS_CLK_ENABLED

__HAL_RCC_ADC2_IS_CLK_ENABLED

__HAL_RCC_ADC3_IS_CLK_ENABLED

__HAL_RCC_SPI5_IS_CLK_ENABLED
__HAL_RCC_SPI6_IS_CLK_ENABLED
__HAL_RCC_SAI1_IS_CLK_ENABLED
__HAL_RCC_SDIO_IS_CLK_ENABLED
__HAL_RCC_SPI4_IS_CLK_ENABLED
__HAL_RCC_TIM10_IS_CLK_ENABLED
__HAL_RCC_SDIO_IS_CLK_DISABLED
__HAL_RCC_SPI4_IS_CLK_DISABLED
__HAL_RCC_TIM10_IS_CLK_DISABLED
__HAL_RCC_TIM8_IS_CLK_DISABLED
__HAL_RCC_ADC2_IS_CLK_DISABLED
__HAL_RCC_ADC3_IS_CLK_DISABLED
__HAL_RCC_SPI5_IS_CLK_DISABLED
__HAL_RCC_SPI6_IS_CLK_DISABLED
__HAL_RCC_SAI1_IS_CLK_DISABLED
__HAL_RCC_LTDC_IS_CLK_ENABLED
__HAL_RCC_LTDC_IS_CLK_DISABLED
__HAL_RCC_DSI_IS_CLK_ENABLED
__HAL_RCC_DSI_IS_CLK_DISABLED

RCC BitAddress AliasRegion

RCC_PLLSAION_BIT_NUMBER
RCC_CR_PLLSAION_BB
PLLSAI_TIMEOUT_VALUE
RCC_PLLI2SON_BIT_NUMBER
RCC_CR_PLLI2SON_BB
RCC_DCKCFGR_OFFSET
RCC_TIMPRE_BIT_NUMBER
RCC_DCKCFGR_TIMPRE_BB

RCC_CFGR_OFFSET

RCC_I2SSRC_BIT_NUMBER

RCC_CFGR_I2SSRC_BB

PLLI2S_TIMEOUT_VALUE

PLL_TIMEOUT_VALUE

RCC CLK48 Clock Source

RCC_CLK48CLKSOURCE_PLLQ

RCC_CLK48CLKSOURCE_PLLSAIP

RCC DSI Clock Source

RCC_DSICLKSOURCE_DSIPHY

RCC_DSICLKSOURCE_PLLR

RCCEX Exported Macros

__HAL_RCC_PLL_CONFIG

Description:

- Macro to configure the main PLL clock source, multiplication and division factors.

Parameters:

- `__RCC_PLLSource__`: specifies the PLL entry clock source. This parameter can be one of the following values:
 - `RCC_PLLSOURCE_HSI`: HSI oscillator clock selected as PLL clock entry
 - `RCC_PLLSOURCE_HSE`: HSE oscillator clock selected as PLL clock entry
- `__PLLM__`: specifies the division factor for PLL VCO input clock This parameter must be a number between `Min_Data = 2` and `Max_Data = 63`.
- `__PLLN__`: specifies the multiplication factor for PLL VCO output clock This parameter must be a number between `Min_Data = 50` and `Max_Data = 432`.
- `__PLLP__`: specifies the division factor for main system clock (SYSCLK) This parameter must be a number in the range {2, 4, 6, or 8}.
- `__PLLQ__`: specifies the division factor for OTG FS, SDIO and RNG clocks This parameter must be a number between `Min_Data = 2` and `Max_Data = 15`.
- `__PLLR__`: PLL division factor for I2S, SAI, SYSTEM, SPDIFRX clocks. This parameter must be a number between `Min_Data = 2` and `Max_Data = 7`.

Notes:

- This function must be used only when the main PLL is disabled.
- This clock source (`RCC_PLLSource`) is common for the main PLL and PLLI2S.
- You have to set the `PLLM` parameter correctly to ensure that the VCO input frequency ranges from 1 to 2 MHz. It is recommended to select a frequency of 2 MHz to limit PLL jitter.
- You have to set the `PLLN` parameter correctly to ensure that the VCO output frequency is between 100 and 432 MHz.
- If the USB OTG FS is used in your application, you have to set the `PLLQ` parameter correctly to have 48 MHz clock for the USB. However, the SDIO and RNG need a frequency lower than or equal to 48 MHz to work correctly.
- This parameter is only available in STM32F446xx/STM32F469xx/STM32F479xx/ STM32F412Zx/ STM32F412Vx/STM32F412Rx/STM32F412Cx/STM32F413xx/STM32F423xx devices.

__HAL_RCC_PLLI2S_ENABLE

Notes:

- The PLLI2S is disabled by hardware when entering STOP and STANDBY modes.

__HAL_RCC_PLLI2S_DISABLE

__HAL_RCC_PLLI2S_CONFIG

Description:

- Macro to configure the PLLI2S clock multiplication and division factors .

Parameters:

- `__PLLI2SN__`: specifies the multiplication factor for PLLI2S VCO output clock This parameter must be a number between Min_Data = 50 and Max_Data = 432.
- `__PLLI2SR__`: specifies the division factor for I2S clock This parameter must be a number between Min_Data = 2 and Max_Data = 7.

Notes:

- This macro must be used only when the PLLI2S is disabled. PLLI2S clock source is common with the main PLL (configured in HAL_RCC_ClockConfig() API).
- You have to set the PLLI2SN parameter correctly to ensure that the VCO output frequency is between Min_Data = 100 and Max_Data = 432 MHz.
- You have to set the PLLI2SR parameter correctly to not exceed 192 MHz on the I2S clock frequency.

__HAL_RCC_PLLI2S_SAICLK_CONFIG

Description:

- Macro used by the SAI HAL driver to configure the PLLI2S clock multiplication and division factors.

Parameters:

- `__PLLI2SN__`: specifies the multiplication factor for PLLI2S VCO output clock. This parameter must be a number between Min_Data = 50 and Max_Data = 432.
- `__PLLI2SQ__`: specifies the division factor for SAI1 clock. This parameter must be a number between Min_Data = 2 and Max_Data = 15.
- `__PLLI2SR__`: specifies the division factor for I2S clock This parameter must be a number between Min_Data = 2 and Max_Data = 7.

Notes:

- This macro must be used only when the PLLI2S is disabled. PLLI2S clock source is common with the main PLL (configured in HAL_RCC_ClockConfig() API)
- You have to set the PLLI2SN parameter correctly to ensure that the VCO output frequency is between Min_Data = 100 and Max_Data = 432 MHz.
- the PLLI2SQ parameter is only available with STM32F427xx/437xx/429xx/439xx/469xx/479xx Devices and can be configured using the `__HAL_RCC_PLLI2S_PLLSAICLK_CONFIG()` macro
- You have to set the PLLI2SR parameter correctly to not exceed 192 MHz on the I2S clock frequency.

__HAL_RCC_PLLSAI_ENABLE

Notes:

- The PLLSAI is only available with STM32F429x/439x Devices. The PLLSAI is disabled by hardware when entering STOP and STANDBY modes.

__HAL_RCC_PLLSAI_DISABLE

__HAL_RCC_PLLSAI_CONFIG

Description:

- Macro to configure the PLLSAI clock multiplication and division factors.

Parameters:

- `__PLLSAIN__`: specifies the multiplication factor for PLLSAI VCO output clock. This parameter must be a number between `Min_Data = 50` and `Max_Data = 432`.
- `__PLLSAIP__`: specifies division factor for SDIO and CLK48 clocks. This parameter must be a number in the range {2, 4, 6, or 8}.
- `__PLLSAIQ__`: specifies the division factor for SAI clock This parameter must be a number between `Min_Data = 2` and `Max_Data = 15`.
- `__PLLSAIR__`: specifies the division factor for LTDC clock This parameter must be a number between `Min_Data = 2` and `Max_Data = 7`.

Notes:

- You have to set the PLLSAIN parameter correctly to ensure that the VCO output frequency is between `Min_Data = 100` and `Max_Data = 432` MHz.

__HAL_RCC_PLLI2S_PLLSAICLKDIVQ_CONFIG

Description:

- Macro to configure the SAI clock Divider coming from PLLI2S.

Parameters:

- `__PLLI2SDivQ__`: specifies the PLLI2S division factor for SAI1 clock. This parameter must be a number between 1 and 32. SAI1 clock frequency = $f(\text{PLLI2SQ}) / \text{__PLLI2SDivQ__}$

Notes:

- This function must be called before enabling the PLLI2S.

__HAL_RCC_PLLSAI_PLLSAICLKDIVQ_CONFIG

Description:

- Macro to configure the SAI clock Divider coming from PLLSAI.

Parameters:

- `__PLLSAIDivQ__`: specifies the PLLSAI division factor for SAI1 clock . This parameter must be a number between `Min_Data = 1` and `Max_Data = 32`. SAI1 clock frequency = $f(\text{PLLSAIQ}) / \text{__PLLSAIDivQ__}$

Notes:

- This function must be called before enabling the PLLSAI.

__HAL_RCC_PLLSAI_PLLSAICLKDIVR_CONFIG

Description:

- Macro to configure the LTDC clock Divider coming from PLLSAI.

Parameters:

- `__PLLSAIDivR__`: specifies the PLLSAI division factor for LTDC clock . This parameter must be a number between `Min_Data = 2` and `Max_Data = 16`. LTDC clock frequency = $f(\text{PLLSAIR}) / \text{__PLLSAIDivR__}$

Notes:

- The LTDC peripheral is only available with STM32F427/437/429/439/469/479xx Devices. This function must be called before enabling the PLLSAI.

__HAL_RCC_I2S_CONFIG

Description:

- Macro to configure the I2S clock source (I2SCLK).

Parameters:

- `__SOURCE__`: specifies the I2S clock source. This parameter can be one of the following values:
 - `RCC_I2SCLKSOURCE_PLLI2S`: PLLI2S clock used as I2S clock source.
 - `RCC_I2SCLKSOURCE_EXT`: External clock mapped on the I2S_CKIN pin used as I2S clock source.

Notes:

- This function must be called before enabling the I2S APB clock.

__HAL_RCC_GET_I2S_SOURCE

Description:

- Macro to get the I2S clock source (I2SCLK).

Return value:

- The: clock source can be one of the following values:
 - `RCC_I2SCLKSOURCE_PLLI2S`: PLLI2S clock used as I2S clock source.
 - `RCC_I2SCLKSOURCE_EXT`: External clock mapped on the I2S_CKIN pin used as I2S clock source

__HAL_RCC_SAI_BLOCKACKLSOURCE_CONFIG

Description:

- Macro to configure SAI1BlockA clock source selection.

Parameters:

- `__SOURCE__`: specifies the SAI Block A clock source. This parameter can be one of the following values:
 - `RCC_SAIACKLSOURCE_PLLI2S`: PLLI2S_Q clock divided by PLLI2SDIVQ used as SAI1 Block A clock.
 - `RCC_SAIACKLSOURCE_PLLSAI`: PLLISAI_Q clock divided by PLLSAIDIVQ used as SAI1 Block A clock.
 - `RCC_SAIACKLSOURCE_Ext`: External clock mapped on the I2S_CKIN pin used as SAI1 Block A clock.

Notes:

- The SAI peripheral is only available with STM32F427/437/429/439/469/479xx Devices. This function must be called before enabling PLLSAI, PLLI2S and the SAI clock.

__HAL_RCC_SAI_BLOCKBCLKSOURCE_CONFIG

Description:

- Macro to configure SAI1BlockB clock source selection.

Parameters:

- `__SOURCE__`: specifies the SAI Block B clock source. This parameter can be one of the following values:
 - `RCC_SAIACKLSOURCE_PLLI2S`: PLLI2S_Q clock divided by PLLI2SDIVQ used as SAI1 Block B clock.
 - `RCC_SAIACKLSOURCE_PLLSAI`: PLLISAI_Q clock divided by PLLSAIDIVQ used as SAI1 Block B clock.
 - `RCC_SAIACKLSOURCE_Ext`: External clock mapped on the I2S_CKIN pin used as SAI1 Block B clock.

Notes:

- The SAI peripheral is only available with STM32F427/437/429/439/469/479xx Devices. This function must be called before enabling PLLSAI, PLLI2S and the SAI clock.

__HAL_RCC_CLK48_CONFIG

Description:

- Macro to configure the CLK48 clock.

Parameters:

- `__SOURCE__`: specifies the CLK48 clock source. This parameter can be one of the following values:
 - `RCC_CLK48CLKSOURCE_PLLQ`: PLL VCO Output divided by PLLQ used as CLK48 clock.
 - `RCC_CLK48CLKSOURCE_PLLSAIP`: PLLSAI VCO Output divided by PLLSAIP used as CLK48 clock.

__HAL_RCC_GET_CLK48_SOURCE

Description:

- Macro to Get the CLK48 clock.

Return value:

- The: clock source can be one of the following values:
 - `RCC_CLK48CLKSOURCE_PLLQ`: PLL VCO Output divided by PLLQ used as CLK48 clock.
 - `RCC_CLK48CLKSOURCE_PLLSAIP`: PLLSAI VCO Output divided by PLLSAIP used as CLK48 clock.

__HAL_RCC_SDIO_CONFIG

Description:

- Macro to configure the SDIO clock.

Parameters:

- `__SOURCE__`: specifies the SDIO clock source. This parameter can be one of the following values:
 - `RCC_SDIOLCKSOURCE_CLK48`: CLK48 output used as SDIO clock.
 - `RCC_SDIOLCKSOURCE_SYSCLK`: System clock output used as SDIO clock.

__HAL_RCC_GET_SDIO_SOURCE

Description:

- Macro to Get the SDIO clock.

Return value:

- The: clock source can be one of the following values:
 - `RCC_SDIOLCKSOURCE_CLK48`: CLK48 output used as SDIO clock.
 - `RCC_SDIOLCKSOURCE_SYSCLK`: System clock output used as SDIO clock.

__HAL_RCC_DSI_CONFIG

Description:

- Macro to configure the DSI clock.

Parameters:

- `__SOURCE__`: specifies the DSI clock source. This parameter can be one of the following values:
 - `RCC_DSICLKSOURCE_PLLR`: PLLR output used as DSI clock.
 - `RCC_DSICLKSOURCE_DSIPHY`: DSI-PHY output used as DSI clock.

__HAL_RCC_GET_DSI_SOURCE

Description:

- Macro to Get the DSI clock.

Return value:

- The: clock source can be one of the following values:
 - `RCC_DSICLKSOURCE_PLLR`: PLLR output used as DSI clock.
 - `RCC_DSICLKSOURCE_DSIPHY`: DSI-PHY output used as DSI clock.

__HAL_RCC_TIMCLKPRESCALER

Description:

- Macro to configure the Timers clocks prescalers.

Parameters:

- `__PRESC__`: specifies the Timers clocks prescalers selection This parameter can be one of the following values:
 - `RCC_TIMPRES_DESACTIVATED`: The Timers kernels clocks prescaler is equal to HPRE if PPREx is corresponding to division by 1 or 2, else it is equal to $[(HPRE * PPREx) / 2]$ if PPREx is corresponding to division by 4 or more.
 - `RCC_TIMPRES_ACTIVATED`: The Timers kernels clocks prescaler is equal to HPRE if PPREx is corresponding to division by 1, 2 or 4, else it is equal to $[(HPRE * PPREx) / 4]$ if PPREx is corresponding to division by 8 or more.

Notes:

- This feature is only available with STM32F429x/439x Devices.

__HAL_RCC_PLLSAI_ENABLE_IT

__HAL_RCC_PLLSAI_DISABLE_IT

__HAL_RCC_PLLSAI_CLEAR_IT

__HAL_RCC_PLLSAI_GET_IT

Description:

- Check the PLLSAI RDY interrupt has occurred or not.

Return value:

- The: new state (TRUE or FALSE).

__HAL_RCC_PLLSAI_GET_FLAG

Description:

- Check PLLSAI RDY flag is set or not.

Return value:

- The: new state (TRUE or FALSE).

I2S Clock Source

RCC_I2SCLKSOURCE_PLLI2S

RCC_I2SCLKSOURCE_EXT

RCC Private macros to check input parameters

IS_RCC_PLLN_VALUE

IS_RCC_PLLI2SN_VALUE

IS_RCC_PERIPHCLK

IS_RCC_PLLI2SR_VALUE

IS_RCC_PLLI2SQ_VALUE

IS_RCC_PLLSAIN_VALUE

IS_RCC_PLLSAIQ_VALUE

IS_RCC_PLLSAIR_VALUE

IS_RCC_PLLSAI_DIVQ_VALUE

IS_RCC_PLLI2S_DIVQ_VALUE

IS_RCC_PLLSAI_DIVR_VALUE

IS_RCC_PLLR_VALUE

IS_RCC_PLLSAIP_VALUE

IS_RCC_CLK48CLKSOURCE

IS_RCC_SDIOCLKSOURCE

IS_RCC_DSIBYTELANECLKSOURCE

IS_RCC_LSE_MODE

IS_RCC_MCO2SOURCE

RCC LSE Dual Mode Selection

RCC_LSE_LOWPOWER_MODE

RCC_LSE_HIGHDRIVE_MODE

RCC Extended MCOx Clock Config

__HAL_RCC_MCO1_CONFIG

Description:

- Macro to configure the MCO1 clock.

Parameters:

- `__MCOCLKSOURCE__`: specifies the MCO clock source. This parameter can be one of the following values:
 - `RCC_MCO1SOURCE_HSI`: HSI clock selected as MCO1 source
 - `RCC_MCO1SOURCE_LSE`: LSE clock selected as MCO1 source
 - `RCC_MCO1SOURCE_HSE`: HSE clock selected as MCO1 source
 - `RCC_MCO1SOURCE_PLLCLK`: main PLL clock selected as MCO1 source
- `__MCODIV__`: specifies the MCO clock prescaler. This parameter can be one of the following values:
 - `RCC_MCODIV_1`: no division applied to MCOx clock
 - `RCC_MCODIV_2`: division by 2 applied to MCOx clock
 - `RCC_MCODIV_3`: division by 3 applied to MCOx clock
 - `RCC_MCODIV_4`: division by 4 applied to MCOx clock
 - `RCC_MCODIV_5`: division by 5 applied to MCOx clock

__HAL_RCC_MCO2_CONFIG

Description:

- Macro to configure the MCO2 clock.

Parameters:

- __MCOCLKSOURCE__: specifies the MCO clock source. This parameter can be one of the following values:
 - RCC_MCO2SOURCE_SYSCLK: System clock (SYSCLK) selected as MCO2 source
 - RCC_MCO2SOURCE_PLLI2SCLK: PLLI2S clock selected as MCO2 source, available for all STM32F4 devices except STM32F410xx
 - RCC_MCO2SOURCE_I2SCLK: I2SCLK clock selected as MCO2 source, available only for STM32F410Rx devices
 - RCC_MCO2SOURCE_HSE: HSE clock selected as MCO2 source
 - RCC_MCO2SOURCE_PLLCLK: main PLL clock selected as MCO2 source
- __MCO DIV__: specifies the MCO clock prescaler. This parameter can be one of the following values:
 - RCC_MCO DIV_1: no division applied to MCOx clock
 - RCC_MCO DIV_2: division by 2 applied to MCOx clock
 - RCC_MCO DIV_3: division by 3 applied to MCOx clock
 - RCC_MCO DIV_4: division by 4 applied to MCOx clock
 - RCC_MCO DIV_5: division by 5 applied to MCOx clock

Notes:

- For STM32F410Rx devices, to output I2SCLK clock on MCO2, you should have at least one of the SPI clocks enabled (SPI1, SPI2 or SPI5).

RCC Periph Clock Selection

RCC_PERIPHCLK_I2S

RCC_PERIPHCLK_SAI_PLLI2S

RCC_PERIPHCLK_SAI_PLLSAI

RCC_PERIPHCLK_LTDC

RCC_PERIPHCLK_TIM

RCC_PERIPHCLK_RTC

RCC_PERIPHCLK_PLLI2S

RCC_PERIPHCLK_CLK48

RCC_PERIPHCLK_SDIO

RCC PLLSAIP Clock Divider

RCC_PLLSAIP_DIV2

RCC_PLLSAIP_DIV4

RCC_PLLSAIP_DIV6

RCC_PLLSAIP_DIV8

RCC PLLSAI DIVR

RCC_PLLSAIDIVR_2

RCC_PLLSAIDIVR_4

RCC_PLLSAIDIVR_8

RCC_PLLSAIDIVR_16

RCC SAI BlockA Clock Source

RCC_SAIACLKSOURCE_PLLSAI

RCC_SAIACLKSOURCE_PLLI2S

RCC_SAIACLKSOURCE_EXT

RCC SAI BlockB Clock Source

RCC_SAIBCLKSOURCE_PLLSAI

RCC_SAIBCLKSOURCE_PLLI2S

RCC_SAIBCLKSOURCE_EXT

RCC SDIO Clock Source

RCC_SDIOCLKSOURCE_CLK48

RCC_SDIOCLKSOURCE_SYSCLK

RCC TIM PRescaler Selection

RCC_TIMPRES_DESACTIVATED

RCC_TIMPRES_ACTIVATED

56 HAL RNG Generic Driver

56.1 RNG Firmware driver registers structures

56.1.1 RNG_HandleTypeDef

RNG_HandleTypeDef is defined in the stm32f4xx_hal_rng.h

Data Fields

- **RNG_TypeDef * Instance**
- **HAL_LockTypeDef Lock**
- **__IO HAL_RNG_StateTypeDef State**
- **__IO uint32_t ErrorCode**
- **uint32_t RandomNumber**

Field Documentation

- **RNG_TypeDef* RNG_HandleTypeDef::Instance**
Register base address
- **HAL_LockTypeDef RNG_HandleTypeDef::Lock**
RNG locking object
- **__IO HAL_RNG_StateTypeDef RNG_HandleTypeDef::State**
RNG communication state
- **__IO uint32_t RNG_HandleTypeDef::ErrorCode**
RNG Error code
- **uint32_t RNG_HandleTypeDef::RandomNumber**
Last Generated RNG Data

56.2 RNG Firmware driver API description

The following section lists the various functions of the RNG library.

56.2.1 How to use this driver

The RNG HAL driver can be used as follows:

1. Enable the RNG controller clock using `__HAL_RCC_RNG_CLK_ENABLE()` macro in `HAL_RNG_MspInit()`.
2. Activate the RNG peripheral using `HAL_RNG_Init()` function.
3. Wait until the 32 bit Random Number Generator contains a valid random data using (polling/interrupt) mode.
4. Get the 32 bit random number using `HAL_RNG_GenerateRandomNumber()` function.

56.2.2 Callback registration

The compilation define `USE_HAL_RNG_REGISTER_CALLBACKS` when set to 1 allows the user to configure dynamically the driver callbacks.

Use Function `HAL_RNG_RegisterCallback()` to register a user callback. Function `HAL_RNG_RegisterCallback()` allows to register following callbacks:

- **ErrorCallback** : RNG Error Callback.
- **MspInitCallback** : RNG MspInit.
- **MspDeInitCallback** : RNG MspDeInit. This function takes as parameters the HAL peripheral handle, the Callback ID and a pointer to the user callback function.

Use function `HAL_RNG_UnRegisterCallback()` to reset a callback to the default weak (surcharged) function. `HAL_RNG_UnRegisterCallback()` takes as parameters the HAL peripheral handle, and the Callback ID. This function allows to reset following callbacks:

- **ErrorCallback** : RNG Error Callback.
- **MspInitCallback** : RNG MspInit.

- MspDeInitCallback : RNG MspDeInit.

For specific callback ReadyDataCallback, use dedicated register callbacks: respectively HAL_RNG_RegisterReadyDataCallback() , HAL_RNG_UnRegisterReadyDataCallback().

By default, after the HAL_RNG_Init() and when the state is HAL_RNG_STATE_RESET all callbacks are set to the corresponding weak (surcharged) functions: example HAL_RNG_ErrorCallback(). Exception done for MspInit and MspDeInit functions that are respectively reset to the legacy weak (surcharged) functions in the HAL_RNG_Init() and HAL_RNG_DeInit() only when these callbacks are null (not registered beforehand). If not, MspInit or MspDeInit are not null, the HAL_RNG_Init() and HAL_RNG_DeInit() keep and use the user MspInit/ MspDeInit callbacks (registered beforehand).

Callbacks can be registered/unregistered in HAL_RNG_STATE_READY state only. Exception done MspInit/ MspDeInit that can be registered/unregistered in HAL_RNG_STATE_READY or HAL_RNG_STATE_RESET state, thus registered (user) MspInit/DeInit callbacks can be used during the Init/DeInit. In that case first register the MspInit/MspDeInit user callbacks using HAL_RNG_RegisterCallback() before calling HAL_RNG_DeInit() or HAL_RNG_Init() function.

When The compilation define USE_HAL_RNG_REGISTER_CALLBACKS is set to 0 or not defined, the callback registration feature is not available and weak (surcharged) callbacks are used.

56.2.3 Initialization and configuration functions

This section provides functions allowing to:

- Initialize the RNG according to the specified parameters in the RNG_InitTypeDef and create the associated handle
- DeInitialize the RNG peripheral
- Initialize the RNG MSP
- DeInitialize RNG MSP

This section contains the following APIs:

- ***HAL_RNG_Init()***
- ***HAL_RNG_DeInit()***
- ***HAL_RNG_MspInit()***
- ***HAL_RNG_MspDeInit()***
- ***HAL_RNG_RegisterCallback()***
- ***HAL_RNG_UnRegisterCallback()***
- ***HAL_RNG_RegisterReadyDataCallback()***
- ***HAL_RNG_UnRegisterReadyDataCallback()***

56.2.4 Peripheral Control functions

This section provides functions allowing to:

- Get the 32 bit Random number
- Get the 32 bit Random number with interrupt enabled
- Handle RNG interrupt request

This section contains the following APIs:

- ***HAL_RNG_GenerateRandomNumber()***
- ***HAL_RNG_GenerateRandomNumber_IT()***
- ***HAL_RNG_GetRandomNumber()***
- ***HAL_RNG_GetRandomNumber_IT()***
- ***HAL_RNG_IRQHandler()***
- ***HAL_RNG_ReadLastRandomNumber()***
- ***HAL_RNG_ReadyDataCallback()***
- ***HAL_RNG_ErrorCallback()***

56.2.5 Peripheral State functions

This subsection permits to get in run-time the status of the peripheral and the data flow.

This section contains the following APIs:

- `HAL_RNG_GetState()`
- `HAL_RNG_GetError()`

56.2.6 Detailed description of functions

HAL_RNG_Init

Function name

HAL_StatusTypeDef HAL_RNG_Init (RNG_HandleTypeDef * hrng)

Function description

Initializes the RNG peripheral and creates the associated handle.

Parameters

- **hrng**: pointer to a RNG_HandleTypeDef structure that contains the configuration information for RNG.

Return values

- **HAL**: status

HAL_RNG_DeInit

Function name

HAL_StatusTypeDef HAL_RNG_DeInit (RNG_HandleTypeDef * hrng)

Function description

DeInitializes the RNG peripheral.

Parameters

- **hrng**: pointer to a RNG_HandleTypeDef structure that contains the configuration information for RNG.

Return values

- **HAL**: status

HAL_RNG_MspInit

Function name

void HAL_RNG_MspInit (RNG_HandleTypeDef * hrng)

Function description

Initializes the RNG MSP.

Parameters

- **hrng**: pointer to a RNG_HandleTypeDef structure that contains the configuration information for RNG.

Return values

- **None**:

HAL_RNG_MspDeInit

Function name

void HAL_RNG_MspDeInit (RNG_HandleTypeDef * hrng)

Function description

DeInitializes the RNG MSP.

Parameters

- **hrng**: pointer to a RNG_HandleTypeDef structure that contains the configuration information for RNG.

Return values

- **None:**

HAL_RNG_RegisterCallback

Function name

```
HAL_StatusTypeDef HAL_RNG_RegisterCallback (RNG_HandleTypeDef * hrng,
HAL_RNG_CallbackIDTypeDef CallbackID, pRNG_CallbackTypeDef pCallback)
```

Function description

Register a User RNG Callback To be used instead of the weak predefined callback.

Parameters

- **hrng:** RNG handle
- **CallbackID:** ID of the callback to be registered This parameter can be one of the following values:
 - HAL_RNG_ERROR_CB_ID Error callback ID
 - HAL_RNG_MSPINIT_CB_ID MspInit callback ID
 - HAL_RNG_MSPDEINIT_CB_ID MspDeInit callback ID
- **pCallback:** pointer to the Callback function

Return values

- **HAL:** status

HAL_RNG_UnRegisterCallback

Function name

```
HAL_StatusTypeDef HAL_RNG_UnRegisterCallback (RNG_HandleTypeDef * hrng,
HAL_RNG_CallbackIDTypeDef CallbackID)
```

Function description

Unregister an RNG Callback RNG callback is redirected to the weak predefined callback.

Parameters

- **hrng:** RNG handle
- **CallbackID:** ID of the callback to be unregistered This parameter can be one of the following values:
 - HAL_RNG_ERROR_CB_ID Error callback ID
 - HAL_RNG_MSPINIT_CB_ID MspInit callback ID
 - HAL_RNG_MSPDEINIT_CB_ID MspDeInit callback ID

Return values

- **HAL:** status

HAL_RNG_RegisterReadyDataCallback

Function name

```
HAL_StatusTypeDef HAL_RNG_RegisterReadyDataCallback (RNG_HandleTypeDef * hrng,
pRNG_ReadyDataCallbackTypeDef pCallback)
```

Function description

Register Data Ready RNG Callback To be used instead of the weak HAL_RNG_ReadyDataCallback() predefined callback.

Parameters

- **hrng:** RNG handle
- **pCallback:** pointer to the Data Ready Callback function

Return values

- **HAL:** status

HAL_RNG_UnRegisterReadyDataCallback

Function name

HAL_StatusTypeDef HAL_RNG_UnRegisterReadyDataCallback (RNG_HandleTypeDef * hrng)

Function description

UnRegister the Data Ready RNG Callback Data Ready RNG Callback is redirected to the weak HAL_RNG_ReadyDataCallback() predefined callback.

Parameters

- **hrng:** RNG handle

Return values

- **HAL:** status

HAL_RNG_GetRandomNumber

Function name

uint32_t HAL_RNG_GetRandomNumber (RNG_HandleTypeDef * hrng)

Function description

Returns generated random number in polling mode (Obsolete) Use HAL_RNG_GenerateRandomNumber() API instead.

Parameters

- **hrng:** pointer to a RNG_HandleTypeDef structure that contains the configuration information for RNG.

Return values

- **Random:** value

HAL_RNG_GetRandomNumber_IT

Function name

uint32_t HAL_RNG_GetRandomNumber_IT (RNG_HandleTypeDef * hrng)

Function description

Returns a 32-bit random number with interrupt enabled (Obsolete), Use HAL_RNG_GenerateRandomNumber_IT() API instead.

Parameters

- **hrng:** pointer to a RNG_HandleTypeDef structure that contains the configuration information for RNG.

Return values

- **32-bit:** random number

HAL_RNG_GenerateRandomNumber

Function name

HAL_StatusTypeDef HAL_RNG_GenerateRandomNumber (RNG_HandleTypeDef * hrng, uint32_t * random32bit)

Function description

Generates a 32-bit random number.

Parameters

- **hrng**: pointer to a RNG_HandleTypeDef structure that contains the configuration information for RNG.
- **random32bit**: pointer to generated random number variable if successful.

Return values

- **HAL**: status

Notes

- Each time the random number data is read the RNG_FLAG_DRDY flag is automatically cleared.

HAL_RNG_GenerateRandomNumber_IT

Function name

HAL_StatusTypeDef HAL_RNG_GenerateRandomNumber_IT (RNG_HandleTypeDef * hrng)

Function description

Generates a 32-bit random number in interrupt mode.

Parameters

- **hrng**: pointer to a RNG_HandleTypeDef structure that contains the configuration information for RNG.

Return values

- **HAL**: status

HAL_RNG_ReadLastRandomNumber

Function name

uint32_t HAL_RNG_ReadLastRandomNumber (RNG_HandleTypeDef * hrng)

Function description

Read latest generated random number.

Parameters

- **hrng**: pointer to a RNG_HandleTypeDef structure that contains the configuration information for RNG.

Return values

- **random**: value

HAL_RNG_IRQHandler

Function name

void HAL_RNG_IRQHandler (RNG_HandleTypeDef * hrng)

Function description

Handles RNG interrupt request.

Parameters

- **hrng**: pointer to a RNG_HandleTypeDef structure that contains the configuration information for RNG.

Return values

- **None**:

Notes

- In the case of a clock error, the RNG is no more able to generate random numbers because the PLL48CLK clock is not correct. User has to check that the clock controller is correctly configured to provide the RNG clock and clear the CEIS bit using `__HAL_RNG_CLEAR_IT()`. The clock error has no impact on the previously generated random numbers, and the RNG_DR register contents can be used.
- In the case of a seed error, the generation of random numbers is interrupted as long as the SECS bit is '1'. If a number is available in the RNG_DR register, it must not be used because it may not have enough entropy. In this case, it is recommended to clear the SEIS bit using `__HAL_RNG_CLEAR_IT()`, then disable and enable the RNG peripheral to reinitialize and restart the RNG.
- User-written `HAL_RNG_ErrorCallback()` API is called once whether SEIS or CEIS are set.

HAL_RNG_ErrorCallback

Function name

```
void HAL_RNG_ErrorCallback (RNG_HandleTypeDef * hrng)
```

Function description

RNG error callbacks.

Parameters

- **hrng**: pointer to a `RNG_HandleTypeDef` structure that contains the configuration information for RNG.

Return values

- **None**:

HAL_RNG_ReadyDataCallback

Function name

```
void HAL_RNG_ReadyDataCallback (RNG_HandleTypeDef * hrng, uint32_t random32bit)
```

Function description

Data Ready callback in non-blocking mode.

Parameters

- **hrng**: pointer to a `RNG_HandleTypeDef` structure that contains the configuration information for RNG.
- **random32bit**: generated random number.

Return values

- **None**:

HAL_RNG_GetState

Function name

```
HAL_RNG_StateTypeDef HAL_RNG_GetState (RNG_HandleTypeDef * hrng)
```

Function description

Returns the RNG state.

Parameters

- **hrng**: pointer to a `RNG_HandleTypeDef` structure that contains the configuration information for RNG.

Return values

- **HAL**: state

HAL_RNG_GetError

Function name

```
uint32_t HAL_RNG_GetError (RNG_HandleTypeDef * hrng)
```

Function description

Return the RNG handle error code.

Parameters

- **hrng**: pointer to a RNG_HandleTypeDef structure.

Return values

- **RNG**: Error Code

56.3 RNG Firmware driver defines

The following section lists the various define and macros of the module.

56.3.1 RNG

RNG

RNG Error Definition

HAL_RNG_ERROR_NONE

No error

HAL_RNG_ERROR_INVALID_CALLBACK

Invalid Callback error

HAL_RNG_ERROR_TIMEOUT

Timeout error

HAL_RNG_ERROR_BUSY

Busy error

HAL_RNG_ERROR_SEED

Seed error

HAL_RNG_ERROR_CLOCK

Clock error

RNG Interrupt definition

RNG_IT_DRDY

Data Ready interrupt

RNG_IT_CEI

Clock error interrupt

RNG_IT_SEI

Seed error interrupt

RNG Flag definition

RNG_FLAG_DRDY

Data ready

RNG_FLAG_CECS

Clock error current status

RNG_FLAG_SECS

Seed error current status

RNG Exported Macros

__HAL_RNG_RESET_HANDLE_STATE

Description:

- Reset RNG handle state.

Parameters:

- `__HANDLE__`: RNG Handle

Return value:

- None

__HAL_RNG_ENABLE

Description:

- Enables the RNG peripheral.

Parameters:

- `__HANDLE__`: RNG Handle

Return value:

- None

__HAL_RNG_DISABLE

Description:

- Disables the RNG peripheral.

Parameters:

- `__HANDLE__`: RNG Handle

Return value:

- None

__HAL_RNG_GET_FLAG

Description:

- Check the selected RNG flag status.

Parameters:

- `__HANDLE__`: RNG Handle
- `__FLAG__`: RNG flag This parameter can be one of the following values:
 - `RNG_FLAG_DRDY`: Data ready
 - `RNG_FLAG_CECS`: Clock error current status
 - `RNG_FLAG_SECS`: Seed error current status

Return value:

- The: new state of `__FLAG__` (SET or RESET).

__HAL_RNG_CLEAR_FLAG

Description:

- Clears the selected RNG flag status.

Parameters:

- `__HANDLE__`: RNG handle
- `__FLAG__`: RNG flag to clear

Return value:

- None

Notes:

- WARNING: This is a dummy macro for HAL code alignment, flags `RNG_FLAG_DRDY`, `RNG_FLAG_CECS` and `RNG_FLAG_SECS` are read-only.

__HAL_RNG_ENABLE_IT

Description:

- Enables the RNG interrupts.

Parameters:

- `__HANDLE__`: RNG Handle

Return value:

- None

__HAL_RNG_DISABLE_IT

Description:

- Disables the RNG interrupts.

Parameters:

- `__HANDLE__`: RNG Handle

Return value:

- None

__HAL_RNG_GET_IT

Description:

- Checks whether the specified RNG interrupt has occurred or not.

Parameters:

- `__HANDLE__`: RNG Handle
- `__INTERRUPT__`: specifies the RNG interrupt status flag to check. This parameter can be one of the following values:
 - `RNG_IT_DRDY`: Data ready interrupt
 - `RNG_IT_CEI`: Clock error interrupt
 - `RNG_IT_SEI`: Seed error interrupt

Return value:

- The: new state of `__INTERRUPT__` (SET or RESET).

__HAL_RNG_CLEAR_IT

Description:

- Clear the RNG interrupt status flags.

Parameters:

- `__HANDLE__`: RNG Handle
- `__INTERRUPT__`: specifies the RNG interrupt status flag to clear. This parameter can be one of the following values:
 - `RNG_IT_CEI`: Clock error interrupt
 - `RNG_IT_SEI`: Seed error interrupt

Return value:

- None

Notes:

- `RNG_IT_DRDY` flag is read-only, reading `RNG_DR` register automatically clears `RNG_IT_DRDY`.

57 HAL RTC Generic Driver

57.1 RTC Firmware driver registers structures

57.1.1 RTC_InitTypeDef

RTC_InitTypeDef is defined in the stm32f4xx_hal_rtc.h

Data Fields

- **uint32_t HourFormat**
- **uint32_t AsynchPrediv**
- **uint32_t SynchPrediv**
- **uint32_t OutPut**
- **uint32_t OutPutPolarity**
- **uint32_t OutPutType**

Field Documentation

- **uint32_t RTC_InitTypeDef::HourFormat**
Specifies the RTC Hour Format. This parameter can be a value of [RTC_Hour_Formats](#)
- **uint32_t RTC_InitTypeDef::AsynchPrediv**
Specifies the RTC Asynchronous Predivider value. This parameter must be a number between Min_Data = 0x00 and Max_Data = 0x7F
- **uint32_t RTC_InitTypeDef::SynchPrediv**
Specifies the RTC Synchronous Predivider value. This parameter must be a number between Min_Data = 0x0000 and Max_Data = 0x7FFF
- **uint32_t RTC_InitTypeDef::OutPut**
Specifies which signal will be routed to the RTC output. This parameter can be a value of [RTC_Output_selection_Definitions](#)
- **uint32_t RTC_InitTypeDef::OutPutPolarity**
Specifies the polarity of the output signal. This parameter can be a value of [RTC_Output_Polarity_Definitions](#)
- **uint32_t RTC_InitTypeDef::OutPutType**
Specifies the RTC Output Pin mode. This parameter can be a value of [RTC_Output_Type_ALARM_OUT](#)

57.1.2 RTC_TimeTypeDef

RTC_TimeTypeDef is defined in the stm32f4xx_hal_rtc.h

Data Fields

- **uint8_t Hours**
- **uint8_t Minutes**
- **uint8_t Seconds**
- **uint8_t TimeFormat**
- **uint32_t SubSeconds**
- **uint32_t SecondFraction**
- **uint32_t DayLightSaving**
- **uint32_t StoreOperation**

Field Documentation

- **uint8_t RTC_TimeTypeDef::Hours**
Specifies the RTC Time Hour. This parameter must be a number between Min_Data = 0 and Max_Data = 12 if the RTC_HourFormat_12 is selected This parameter must be a number between Min_Data = 0 and Max_Data = 23 if the RTC_HourFormat_24 is selected
- **uint8_t RTC_TimeTypeDef::Minutes**
Specifies the RTC Time Minutes. This parameter must be a number between Min_Data = 0 and Max_Data = 59

- **`uint8_t RTC_TimeTypeDef::Seconds`**
Specifies the RTC Time Seconds. This parameter must be a number between Min_Data = 0 and Max_Data = 59
- **`uint8_t RTC_TimeTypeDef::TimeFormat`**
Specifies the RTC AM/PM Time. This parameter can be a value of [RTC_AM_PM_Definitions](#)
- **`uint32_t RTC_TimeTypeDef::SubSeconds`**
Specifies the RTC_SSR RTC Sub Second register content. This parameter corresponds to a time unit range between [0-1] Second with [1 Sec / SecondFraction + 1] granularity
- **`uint32_t RTC_TimeTypeDef::SecondFraction`**
Specifies the range or granularity of Sub Second register content corresponding to Synchronous prescaler factor value (PREDIV_S) This parameter corresponds to a time unit range between [0-1] Second with [1 Sec / SecondFraction + 1] granularity. This field will be used only by HAL_RTC_GetTime function
- **`uint32_t RTC_TimeTypeDef::DayLightSaving`**
This interface is deprecated. To manage Daylight Saving Time, please use HAL_RTC_DST_xxx functions
- **`uint32_t RTC_TimeTypeDef::StoreOperation`**
This interface is deprecated. To manage Daylight Saving Time, please use HAL_RTC_DST_xxx functions

57.1.3

RTC_DateTypeDef

RTC_DateTypeDef is defined in the stm32f4xx_hal_rtc.h

Data Fields

- **`uint8_t WeekDay`**
- **`uint8_t Month`**
- **`uint8_t Date`**
- **`uint8_t Year`**

Field Documentation

- **`uint8_t RTC_DateTypeDef::WeekDay`**
Specifies the RTC Date WeekDay. This parameter can be a value of [RTC_WeekDay_Definitions](#)
- **`uint8_t RTC_DateTypeDef::Month`**
Specifies the RTC Date Month (in BCD format). This parameter can be a value of [RTC_Month_Date_Definitions](#)
- **`uint8_t RTC_DateTypeDef::Date`**
Specifies the RTC Date. This parameter must be a number between Min_Data = 1 and Max_Data = 31
- **`uint8_t RTC_DateTypeDef::Year`**
Specifies the RTC Date Year. This parameter must be a number between Min_Data = 0 and Max_Data = 99

57.1.4

RTC_AlarmTypeDef

RTC_AlarmTypeDef is defined in the stm32f4xx_hal_rtc.h

Data Fields

- **`RTC_TimeTypeDef AlarmTime`**
- **`uint32_t AlarmMask`**
- **`uint32_t AlarmSubSecondMask`**
- **`uint32_t AlarmDateWeekDaySel`**
- **`uint8_t AlarmDateWeekDay`**
- **`uint32_t Alarm`**

Field Documentation

- **`RTC_TimeTypeDef RTC_AlarmTypeDef::AlarmTime`**
Specifies the RTC Alarm Time members
- **`uint32_t RTC_AlarmTypeDef::AlarmMask`**
Specifies the RTC Alarm Masks. This parameter can be a value of [RTC_AlarmMask_Definitions](#)
- **`uint32_t RTC_AlarmTypeDef::AlarmSubSecondMask`**
Specifies the RTC Alarm SubSeconds Masks. This parameter can be a value of [RTC_Alarm_Sub_Seconds_Masks_Definitions](#)

- **`uint32_t RTC_AlarmTypeDef::AlarmDateWeekDaySel`**
Specifies the RTC Alarm is on Date or WeekDay. This parameter can be a value of [RTC_AlarmDateWeekDay_Definitions](#)
- **`uint8_t RTC_AlarmTypeDef::AlarmDateWeekDay`**
Specifies the RTC Alarm Date/WeekDay. If the Alarm Date is selected, this parameter must be set to a value in the 1-31 range. If the Alarm WeekDay is selected, this parameter can be a value of [RTC_WeekDay_Definitions](#)
- **`uint32_t RTC_AlarmTypeDef::Alarm`**
Specifies the alarm . This parameter can be a value of [RTC_Alarms_Definitions](#)

57.1.5

RTC_HandleTypeDef

`RTC_HandleTypeDef` is defined in the `stm32f4xx_hal_rtc.h`

Data Fields

- **`RTC_TypeDef * Instance`**
- **`RTC_InitTypeDef Init`**
- **`HAL_LockTypeDef Lock`**
- **`__IO HAL_RTCStateTypeDef State`**

Field Documentation

- **`RTC_TypeDef* RTC_HandleTypeDef::Instance`**
Register base address
- **`RTC_InitTypeDef RTC_HandleTypeDef::Init`**
RTC required parameters
- **`HAL_LockTypeDef RTC_HandleTypeDef::Lock`**
RTC locking object
- **`__IO HAL_RTCStateTypeDef RTC_HandleTypeDef::State`**
Time communication state

57.2

RTC Firmware driver API description

The following section lists the various functions of the RTC library.

57.2.1

Backup Domain Operating Condition

The real-time clock (RTC), the RTC backup registers, and the backup SRAM (BKP SRAM) can be powered from the VBAT voltage when the main VDD supply is powered off. To retain the content of the RTC backup registers, backup SRAM, and supply the RTC when VDD is turned off, VBAT pin can be connected to an optional standby voltage supplied by a battery or by another source.

To allow the RTC operating even when the main digital supply (VDD) is turned off, the VBAT pin powers the following blocks:

1. The RTC
2. The LSE oscillator
3. The backup SRAM when the low power backup regulator is enabled
4. PC13 to PC15 I/Os, plus PI8 I/O (when available)

When the backup domain is supplied by VDD (analog switch connected to VDD), the following pins are available:

1. PC14 and PC15 can be used as either GPIO or LSE pins
2. PC13 can be used as a GPIO or as the RTC_AF1 pin
3. PI8 can be used as a GPIO or as the RTC_AF2 pin

When the backup domain is supplied by VBAT (analog switch connected to VBAT because VDD is not present), the following pins are available:

1. PC14 and PC15 can be used as LSE pins only
2. PC13 can be used as the RTC_AF1 pin
3. PI8 can be used as the RTC_AF2 pin

57.2.2

Backup Domain Reset

The backup domain reset sets all RTC registers and the RCC_BDCR register to their reset values. The BKP SRAM is not affected by this reset. The only way to reset the BKP SRAM is through the Flash interface by requesting a protection level change from 1 to 0.

A backup domain reset is generated when one of the following events occurs:

1. Software reset, triggered by setting the BDRST bit in the RCC Backup domain control register (RCC_BDCR).
2. VDD or VBAT power on, if both supplies have previously been powered off.

57.2.3 Backup Domain Access

After reset, the backup domain (RTC registers, RTC backup data registers and BKP SRAM) is protected against possible unwanted write accesses.

To enable access to the RTC Domain and RTC registers, proceed as follows:

- Enable the Power Controller (PWR) APB1 interface clock using the `__HAL_RCC_PWR_CLK_ENABLE()` macro.
- Enable access to RTC domain using the `HAL_PWR_EnableBkUpAccess()` function.
- Select the RTC clock source using the `__HAL_RCC_RTC_CONFIG()` macro.
- Enable RTC Clock using the `__HAL_RCC_RTC_ENABLE()` macro.

57.2.4 How to use this driver

- Enable the RTC domain access (see description in the section above).
- Configure the RTC Prescaler (Asynchronous and Synchronous) and RTC hour format using the `HAL_RTC_Init()` function.

Time and Date configuration

- To configure the RTC Calendar (Time and Date) use the `HAL_RTC_SetTime()` and `HAL_RTC_SetDate()` functions.
- To read the RTC Calendar, use the `HAL_RTC_GetTime()` and `HAL_RTC_GetDate()` functions.
- To manage the RTC summer or winter time change, use the following functions:
 - `HAL_RTC_DST_Add1Hour()` or `HAL_RTC_DST_Sub1Hour` to add or subtract 1 hour from the calendar time.
 - `HAL_RTC_DST_SetStoreOperation()` or `HAL_RTC_DST_ClearStoreOperation` to memorize whether the time change has been performed or not.

Alarm configuration

- To configure the RTC Alarm use the `HAL_RTC_SetAlarm()` function. You can also configure the RTC Alarm with interrupt mode using the `HAL_RTC_SetAlarm_IT()` function.
- To read the RTC Alarm, use the `HAL_RTC_GetAlarm()` function.

57.2.5 RTC and low power modes

The MCU can be woken up from a low power mode by an RTC alternate function.

The RTC alternate functions are the RTC alarms (Alarm A and Alarm B), RTC wakeup, RTC tamper event detection and RTC timestamp event detection. These RTC alternate functions can wake up the system from the Stop and Standby low power modes.

The system can also wake up from low power modes without depending on an external interrupt (Auto-wakeup mode), by using the RTC alarm or the RTC wakeup events.

The RTC provides a programmable time base for waking up from the Stop or Standby mode at regular intervals. Wakeup from STOP and STANDBY modes is possible only when the RTC clock source is LSE or LSI.

Callback registration

The compilation define `USE_HAL_RTC_REGISTER_CALLBACKS` when set to 1 allows the user to configure dynamically the driver callbacks. Use Function `HAL_RTC_RegisterCallback()` to register an interrupt callback.

Function `HAL_RTC_RegisterCallback()` allows to register following callbacks:

- AlarmAEventCallback : RTC Alarm A Event callback.
- AlarmBEventCallback : RTC Alarm B Event callback.
- TimeStampEventCallback : RTC Timestamp Event callback.
- WakeUpTimerEventCallback : RTC WakeUpTimer Event callback.
- Tamper1EventCallback : RTC Tamper 1 Event callback.
- Tamper2EventCallback : RTC Tamper 2 Event callback.
- MspInitCallback : RTC MspInit callback.
- MspDeInitCallback : RTC MspDeInit callback.

This function takes as parameters the HAL peripheral handle, the Callback ID and a pointer to the user callback function.

Use function HAL_RTC_UnRegisterCallback() to reset a callback to the default weak function.

HAL_RTC_UnRegisterCallback() takes as parameters the HAL peripheral handle, and the Callback ID. This function allows to reset following callbacks:

- AlarmAEventCallback : RTC Alarm A Event callback.
- AlarmBEventCallback : RTC Alarm B Event callback.
- TimeStampEventCallback : RTC Timestamp Event callback.
- WakeUpTimerEventCallback : RTC WakeUpTimer Event callback.
- Tamper1EventCallback : RTC Tamper 1 Event callback.
- Tamper2EventCallback : RTC Tamper 2 Event callback.
- MspInitCallback : RTC MspInit callback.
- MspDeInitCallback : RTC MspDeInit callback.

By default, after the HAL_RTC_Init() and when the state is HAL_RTC_STATE_RESET, all callbacks are set to the corresponding weak functions: examples AlarmAEventCallback(), WakeUpTimerEventCallback(). Exception done for MspInit() and MspDeInit() callbacks that are reset to the legacy weak function in the HAL_RTC_Init()/HAL_RTC_DeInit() only when these callbacks are null (not registered beforehand). If not, MspInit() or MspDeInit() are not null, HAL_RTC_Init()/HAL_RTC_DeInit() keep and use the user MspInit()/MspDeInit() callbacks (registered beforehand).

Callbacks can be registered/unregistered in HAL_RTC_STATE_READY state only. Exception done MspInit()/MspDeInit() that can be registered/unregistered in HAL_RTC_STATE_READY or HAL_RTC_STATE_RESET state. Thus registered (user) MspInit()/MspDeInit() callbacks can be used during the Init/DeInit. In that case first register the MspInit()/MspDeInit() user callbacks using HAL_RTC_RegisterCallback() before calling HAL_RTC_DeInit() or HAL_RTC_Init() functions.

When The compilation define USE_HAL_RTC_REGISTER_CALLBACKS is set to 0 or not defined, the callback registration feature is not available and all callbacks are set to the corresponding weak functions.

57.2.6 Initialization and de-initialization functions

This section provides functions allowing to initialize and configure the RTC Prescaler (Synchronous and Asynchronous), RTC Hour format, disable RTC registers Write protection, enter and exit the RTC initialization mode, RTC registers synchronization check and reference clock detection enable.

1. The RTC Prescaler is programmed to generate the RTC 1Hz time base. It is split into 2 programmable prescalers to minimize power consumption.
 - A 7-bit asynchronous prescaler and a 15-bit synchronous prescaler.
 - When both prescalers are used, it is recommended to configure the asynchronous prescaler to a high value to minimize power consumption.
2. All RTC registers are Write protected. Writing to the RTC registers is enabled by writing a key into the Write Protection register, RTC_WPR.
3. To configure the RTC Calendar, user application should enter initialization mode. In this mode, the calendar counter is stopped and its value can be updated. When the initialization sequence is complete, the calendar restarts counting after 4 RTCCLK cycles.
4. To read the calendar through the shadow registers after Calendar initialization, calendar update or after wakeup from low power modes the software must first clear the RSF flag. The software must then wait until it is set again before reading the calendar, which means that the calendar registers have been correctly copied into the RTC_TR and RTC_DR shadow registers. The HAL_RTC_WaitForSynchro() function implements the above software sequence (RSF clear and RSF check).

This section contains the following APIs:

- *HAL_RTC_Init()*
- *HAL_RTC_DeInit()*
- *HAL_RTC_RegisterCallback()*
- *HAL_RTC_UnRegisterCallback()*
- *HAL_RTC_MspInit()*
- *HAL_RTC_MspDeInit()*

57.2.7 RTC Time and Date functions

This section provides functions allowing to configure Time and Date features

This section contains the following APIs:

- *HAL_RTC_SetTime()*
- *HAL_RTC_GetTime()*
- *HAL_RTC_SetDate()*
- *HAL_RTC_GetDate()*

57.2.8 RTC Alarm functions

This section provides functions allowing to configure Alarm feature

This section contains the following APIs:

- *HAL_RTC_SetAlarm()*
- *HAL_RTC_SetAlarm_IT()*
- *HAL_RTC_DeactivateAlarm()*
- *HAL_RTC_GetAlarm()*
- *HAL_RTC_AlarmIRQHandler()*
- *HAL_RTC_AlarmAEventCallback()*
- *HAL_RTC_PollForAlarmAEvent()*

57.2.9 Peripheral Control functions

This subsection provides functions allowing to

- Wait for RTC Time and Date Synchronization
- Manage RTC Summer or Winter time change

This section contains the following APIs:

- *HAL_RTC_WaitForSynchro()*
- *HAL_RTC_DST_Add1Hour()*
- *HAL_RTC_DST_Sub1Hour()*
- *HAL_RTC_DST_SetStoreOperation()*
- *HAL_RTC_DST_ClearStoreOperation()*
- *HAL_RTC_DST_ReadStoreOperation()*

57.2.10 Peripheral State functions

This subsection provides functions allowing to

- Get RTC state

This section contains the following APIs:

- *HAL_RTC_GetState()*

57.2.11 Detailed description of functions

HAL_RTC_Init

Function name

HAL_StatusTypeDef HAL_RTC_Init (RTC_HandleTypeDef * hrtc)

Function description

Initializes the RTC peripheral.

Parameters

- **hrtc**: pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.

Return values

- **HAL**: status

HAL_RTC_DeInit

Function name

HAL_StatusTypeDef HAL_RTC_DeInit (RTC_HandleTypeDef * hrtc)

Function description

Deinitializes the RTC peripheral.

Parameters

- **hrtc**: pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.

Return values

- **HAL**: status

Notes

- This function does not reset the RTC Backup Data registers.

HAL_RTC_MspltInit

Function name

void HAL_RTC_MspltInit (RTC_HandleTypeDef * hrtc)

Function description

Initializes the RTC MSP.

Parameters

- **hrtc**: pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.

Return values

- **None**:

HAL_RTC_MspDeInit

Function name

void HAL_RTC_MspDeInit (RTC_HandleTypeDef * hrtc)

Function description

Deinitializes the RTC MSP.

Parameters

- **hrtc**: pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.

Return values

- **None:**

HAL_RTC_RegisterCallback

Function name

HAL_StatusTypeDef HAL_RTC_RegisterCallback (RTC_HandleTypeDef * hrtc, HAL_RTC_CallbackIDTypeDef CallbackID, pRTC_CallbackTypeDef pCallback)

Function description

Registers a User RTC Callback To be used instead of the weak predefined callback.

Parameters

- **hrtc:** pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.
- **CallbackID:** ID of the callback to be registered This parameter can be one of the following values:
 - HAL_RTC_ALARM_A_EVENT_CB_ID Alarm A Event Callback ID
 - HAL_RTC_ALARM_B_EVENT_CB_ID Alarm B Event Callback ID
 - HAL_RTC_TIMESTAMP_EVENT_CB_ID Timestamp Event Callback ID
 - HAL_RTC_WAKEUPTIMER_EVENT_CB_ID Wakeup Timer Event Callback ID
 - HAL_RTC_TAMPER1_EVENT_CB_ID Tamper 1 Callback ID
 - HAL_RTC_TAMPER2_EVENT_CB_ID Tamper 2 Callback ID
 - HAL_RTC_MSPINIT_CB_ID Msp Init callback ID
 - HAL_RTC_MSPDEINIT_CB_ID Msp Delnit callback ID
- **pCallback:** pointer to the Callback function

Return values

- **HAL:** status

Notes

- HAL_RTC_TAMPER2_EVENT_CB_ID is not applicable to all devices.

HAL_RTC_UnRegisterCallback

Function name

HAL_StatusTypeDef HAL_RTC_UnRegisterCallback (RTC_HandleTypeDef * hrtc, HAL_RTC_CallbackIDTypeDef CallbackID)

Function description

Unregisters an RTC Callback RTC callback is redirected to the weak predefined callback.

Parameters

- **hrtc:** pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.
- **CallbackID:** ID of the callback to be unregistered This parameter can be one of the following values:
 - HAL_RTC_ALARM_A_EVENT_CB_ID Alarm A Event Callback ID
 - HAL_RTC_ALARM_B_EVENT_CB_ID Alarm B Event Callback ID
 - HAL_RTC_TIMESTAMP_EVENT_CB_ID Timestamp Event Callback ID
 - HAL_RTC_WAKEUPTIMER_EVENT_CB_ID Wakeup Timer Event Callback ID
 - HAL_RTC_TAMPER1_EVENT_CB_ID Tamper 1 Callback ID
 - HAL_RTC_TAMPER2_EVENT_CB_ID Tamper 2 Callback ID
 - HAL_RTC_MSPINIT_CB_ID Msp Init callback ID
 - HAL_RTC_MSPDEINIT_CB_ID Msp Delnit callback ID

Return values

- **HAL:** status

Notes

- HAL_RTC_TAMPER2_EVENT_CB_ID is not applicable to all devices.

HAL_RTC_SetTime

Function name

HAL_StatusTypeDef HAL_RTC_SetTime (RTC_HandleTypeDef * hrtc, RTC_TimeTypeDef * sTime, uint32_t Format)

Function description

Sets RTC current time.

Parameters

- **hrtc**: pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.
- **sTime**: Pointer to Time structure
- **Format**: Specifies the format of the entered parameters. This parameter can be one of the following values:
 - RTC_FORMAT_BIN: Binary data format
 - RTC_FORMAT_BCD: BCD data format

Return values

- **HAL**: status

Notes

- DayLightSaving and StoreOperation interfaces are deprecated. To manage Daylight Saving Time, please use HAL_RTC_DST_xxx functions.

HAL_RTC_GetTime

Function name

HAL_StatusTypeDef HAL_RTC_GetTime (RTC_HandleTypeDef * hrtc, RTC_TimeTypeDef * sTime, uint32_t Format)

Function description

Gets RTC current time.

Parameters

- **hrtc**: pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.
- **sTime**: Pointer to Time structure
- **Format**: Specifies the format of the entered parameters. This parameter can be one of the following values:
 - RTC_FORMAT_BIN: Binary data format
 - RTC_FORMAT_BCD: BCD data format

Return values

- **HAL**: status

Notes

- You can use SubSeconds and SecondFraction (sTime structure fields returned) to convert SubSeconds value in second fraction ratio with time unit following generic formula: Second fraction ratio * time_unit = [(SecondFraction - SubSeconds) / (SecondFraction + 1)] * time_unit This conversion can be performed only if no shift operation is pending (ie. SHFP=0) when PREDIV_S >= SS
- You must call HAL_RTC_GetDate() after HAL_RTC_GetTime() to unlock the values in the higher-order calendar shadow registers to ensure consistency between the time and date values. Reading RTC current time locks the values in calendar shadow registers until current date is read to ensure consistency between the time and date values.

HAL_RTC_SetDate

Function name

HAL_StatusTypeDef HAL_RTC_SetDate (RTC_HandleTypeDef * hrtc, RTC_DateTypeDef * sDate, uint32_t Format)

Function description

Sets RTC current date.

Parameters

- **hrtc**: pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.
- **sDate**: Pointer to date structure
- **Format**: specifies the format of the entered parameters. This parameter can be one of the following values:
 - RTC_FORMAT_BIN: Binary data format
 - RTC_FORMAT_BCD: BCD data format

Return values

- **HAL**: status

HAL_RTC_GetDate

Function name

HAL_StatusTypeDef HAL_RTC_GetDate (RTC_HandleTypeDef * hrtc, RTC_DateTypeDef * sDate, uint32_t Format)

Function description

Gets RTC current date.

Parameters

- **hrtc**: pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.
- **sDate**: Pointer to Date structure
- **Format**: Specifies the format of the entered parameters. This parameter can be one of the following values:
 - RTC_FORMAT_BIN: Binary data format
 - RTC_FORMAT_BCD: BCD data format

Return values

- **HAL**: status

Notes

- You must call HAL_RTC_GetDate() after HAL_RTC_GetTime() to unlock the values in the higher-order calendar shadow registers to ensure consistency between the time and date values. Reading RTC current time locks the values in calendar shadow registers until current date is read to ensure consistency between the time and date values.

HAL_RTC_SetAlarm

Function name

HAL_StatusTypeDef HAL_RTC_SetAlarm (RTC_HandleTypeDef * hrtc, RTC_AlarmTypeDef * sAlarm, uint32_t Format)

Function description

Sets the specified RTC Alarm.

Parameters

- **hrtc**: pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.
- **sAlarm**: Pointer to Alarm structure
- **Format**: Specifies the format of the entered parameters. This parameter can be one of the following values:
 - RTC_FORMAT_BIN: Binary data format
 - RTC_FORMAT_BCD: BCD data format

Return values

- **HAL**: status

Notes

- The Alarm register can only be written when the corresponding Alarm is disabled (Use the HAL_RTC_DeactivateAlarm()).
- The HAL_RTC_SetTime() must be called before enabling the Alarm feature.

HAL_RTC_SetAlarm_IT

Function name

HAL_StatusTypeDef HAL_RTC_SetAlarm_IT (RTC_HandleTypeDef * hrtc, RTC_AlarmTypeDef * sAlarm, uint32_t Format)

Function description

Sets the specified RTC Alarm with Interrupt.

Parameters

- **hrtc**: pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.
- **sAlarm**: Pointer to Alarm structure
- **Format**: Specifies the format of the entered parameters. This parameter can be one of the following values:
 - RTC_FORMAT_BIN: Binary data format
 - RTC_FORMAT_BCD: BCD data format

Return values

- **HAL**: status

Notes

- The Alarm register can only be written when the corresponding Alarm is disabled (Use the HAL_RTC_DeactivateAlarm()).
- The HAL_RTC_SetTime() must be called before enabling the Alarm feature.

HAL_RTC_DeactivateAlarm

Function name

HAL_StatusTypeDef HAL_RTC_DeactivateAlarm (RTC_HandleTypeDef * hrtc, uint32_t Alarm)

Function description

Deactivates the specified RTC Alarm.

Parameters

- **hrtc**: pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.
- **Alarm**: Specifies the Alarm. This parameter can be one of the following values:
 - RTC_ALARM_A: Alarm A
 - RTC_ALARM_B: Alarm B

Return values

- **HAL:** status

HAL_RTC_GetAlarm

Function name

HAL_StatusTypeDef HAL_RTC_GetAlarm (RTC_HandleTypeDef * hrtc, RTC_AlarmTypeDef * sAlarm, uint32_t Alarm, uint32_t Format)

Function description

Gets the RTC Alarm value and masks.

Parameters

- **hrtc:** pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.
- **sAlarm:** Pointer to Date structure
- **Alarm:** Specifies the Alarm. This parameter can be one of the following values:
 - RTC_ALARM_A: Alarm A
 - RTC_ALARM_B: Alarm B
- **Format:** Specifies the format of the entered parameters. This parameter can be one of the following values:
 - RTC_FORMAT_BIN: Binary data format
 - RTC_FORMAT_BCD: BCD data format

Return values

- **HAL:** status

HAL_RTC_AlarmIRQHandler

Function name

void HAL_RTC_AlarmIRQHandler (RTC_HandleTypeDef * hrtc)

Function description

Handles Alarm interrupt request.

Parameters

- **hrtc:** pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.

Return values

- **None:**

HAL_RTC_PollForAlarmAEvent

Function name

HAL_StatusTypeDef HAL_RTC_PollForAlarmAEvent (RTC_HandleTypeDef * hrtc, uint32_t Timeout)

Function description

Handles Alarm A Polling request.

Parameters

- **hrtc:** pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.
- **Timeout:** Timeout duration

Return values

- **HAL:** status

HAL_RTC_AlarmAEventCallback

Function name

void HAL_RTC_AlarmAEventCallback (RTC_HandleTypeDef * hrtc)

Function description

Alarm A callback.

Parameters

- **hrtc**: pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.

Return values

- **None**:

HAL_RTC_WaitForSynchro

Function name

HAL_StatusTypeDef HAL_RTC_WaitForSynchro (RTC_HandleTypeDef * hrtc)

Function description

Waits until the RTC Time and Date registers (RTC_TR and RTC_DR) are synchronized with RTC APB clock.

Parameters

- **hrtc**: pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.

Return values

- **HAL**: status

Notes

- The RTC Resynchronization mode is write protected, use the `__HAL_RTC_WRITEPROTECTION_DISABLE()` before calling this function.
- To read the calendar through the shadow registers after Calendar initialization, calendar update or after wakeup from low power modes the software must first clear the RSF flag. The software must then wait until it is set again before reading the calendar, which means that the calendar registers have been correctly copied into the RTC_TR and RTC_DR shadow registers.

HAL_RTC_DST_Add1Hour

Function name

void HAL_RTC_DST_Add1Hour (RTC_HandleTypeDef * hrtc)

Function description

Daylight Saving Time, adds one hour to the calendar in one single operation without going through the initialization procedure.

Parameters

- **hrtc**: pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.

Return values

- **None**:

HAL_RTC_DST_Sub1Hour

Function name

void HAL_RTC_DST_Sub1Hour (RTC_HandleTypeDef * hrtc)

Function description

Daylight Saving Time, subtracts one hour from the calendar in one single operation without going through the initialization procedure.

Parameters

- **hrtc**: pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.

Return values

- **None**:

HAL_RTC_DST_SetStoreOperation

Function name

```
void HAL_RTC_DST_SetStoreOperation (RTC_HandleTypeDef * hrtc)
```

Function description

Daylight Saving Time, sets the store operation bit.

Parameters

- **hrtc**: pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.

Return values

- **None**:

Notes

- It can be used by the software in order to memorize the DST status.

HAL_RTC_DST_ClearStoreOperation

Function name

```
void HAL_RTC_DST_ClearStoreOperation (RTC_HandleTypeDef * hrtc)
```

Function description

Daylight Saving Time, clears the store operation bit.

Parameters

- **hrtc**: pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.

Return values

- **None**:

HAL_RTC_DST_ReadStoreOperation

Function name

```
uint32_t HAL_RTC_DST_ReadStoreOperation (RTC_HandleTypeDef * hrtc)
```

Function description

Daylight Saving Time, reads the store operation bit.

Parameters

- **hrtc**: RTC handle

Return values

- **operation**: see RTC_StoreOperation_Definitions

HAL_RTC_GetState

Function name

HAL_RTCStateTypeDef HAL_RTC_GetState (RTC_HandleTypeDef * hrtc)

Function description

Returns the RTC state.

Parameters

- **hrtc**: pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.

Return values

- **HAL**: state

RTC_EnterInitMode

Function name

HAL_StatusTypeDef RTC_EnterInitMode (RTC_HandleTypeDef * hrtc)

Function description

Enters the RTC Initialization mode.

Parameters

- **hrtc**: pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.

Return values

- **HAL**: status

Notes

- The RTC Initialization mode is write protected, use the `__HAL_RTC_WRITEPROTECTION_DISABLE()` before calling this function.

RTC_ExitInitMode

Function name

HAL_StatusTypeDef RTC_ExitInitMode (RTC_HandleTypeDef * hrtc)

Function description

Exits the RTC Initialization mode.

Parameters

- **hrtc**: pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.

Return values

- **HAL**: status

RTC_ByteToBcd2

Function name

uint8_t RTC_ByteToBcd2 (uint8_t number)

Function description

Converts a 2-digit number from decimal to BCD format.

Parameters

- **number**: decimal-formatted number (from 0 to 99) to be converted

Return values

- **Converted:** byte

RTC_Bcd2ToByte

Function name

uint8_t RTC_Bcd2ToByte (uint8_t number)

Function description

Converts a 2-digit number from BCD to decimal format.

Parameters

- **number:** BCD-formatted number (from 00 to 99) to be converted

Return values

- **Converted:** word

57.3 RTC Firmware driver defines

The following section lists the various define and macros of the module.

57.3.1 RTC

RTC

RTC Alarm Date WeekDay Definitions

RTC_ALARMDATEWEEKDAYSEL_DATE

RTC_ALARMDATEWEEKDAYSEL_WEEKDAY

RTC Alarm Mask Definitions

RTC_ALARM_MASK_NONE

RTC_ALARM_MASK_DATEWEEKDAY

RTC_ALARM_MASK_HOURS

RTC_ALARM_MASK_MINUTES

RTC_ALARM_MASK_SECONDS

RTC_ALARM_MASK_ALL

RTC Alarms Definitions

RTC_ALARM_A

RTC_ALARM_B

RTC Alarm Sub Seconds Masks Definitions

RTC_ALARMSSUBSECONDMASK_ALL

< All Alarm SS fields are masked. There is no comparison on sub seconds for Alarm SS[14:1] are don't care in Alarm comparison. Only SS[0] is compared.

RTC_ALARMSSUBSECONDMASK_SS14_1

SS[14:2] are don't care in Alarm comparison. Only SS[1:0] are compared.

RTC_ALARMSSUBSECONDMASK_SS14_2

SS[14:3] are don't care in Alarm comparison. Only SS[2:0] are compared.

RTC_ALARMSSUBSECONDMASK_SS14_3

SS[14:4] are don't care in Alarm comparison. Only SS[3:0] are compared.

RTC_ALARMSSUBSECONDMASK_SS14_4

SS[14:5] are don't care in Alarm comparison. Only SS[4:0] are compared.

RTC_ALARMSSUBSECONDMASK_SS14_5

SS[14:6] are don't care in Alarm comparison. Only SS[5:0] are compared.

RTC_ALARMSSUBSECONDMASK_SS14_6

SS[14:7] are don't care in Alarm comparison. Only SS[6:0] are compared.

RTC_ALARMSSUBSECONDMASK_SS14_7

SS[14:8] are don't care in Alarm comparison. Only SS[7:0] are compared.

RTC_ALARMSSUBSECONDMASK_SS14_8

SS[14:9] are don't care in Alarm comparison. Only SS[8:0] are compared.

RTC_ALARMSSUBSECONDMASK_SS14_9

SS[14:10] are don't care in Alarm comparison. Only SS[9:0] are compared.

RTC_ALARMSSUBSECONDMASK_SS14_10

SS[14:11] are don't care in Alarm comparison. Only SS[10:0] are compared.

RTC_ALARMSSUBSECONDMASK_SS14_11

SS[14:12] are don't care in Alarm comparison. Only SS[11:0] are compared.

RTC_ALARMSSUBSECONDMASK_SS14_12

SS[14:13] are don't care in Alarm comparison. Only SS[12:0] are compared.

RTC_ALARMSSUBSECONDMASK_SS14_13

SS[14] is don't care in Alarm comparison. Only SS[13:0] are compared.

RTC_ALARMSSUBSECONDMASK_SS14

SS[14:0] are compared and must match to activate alarm.

RTC_ALARMSSUBSECONDMASK_NONE***RTC AM PM Definitions*****RTC_HOURFORMAT12_AM****RTC_HOURFORMAT12_PM*****RTC DayLight Saving Definitions*****RTC_DAYLIGHTSAVING_SUB1H****RTC_DAYLIGHTSAVING_ADD1H****RTC_DAYLIGHTSAVING_NONE*****RTC Exported Macros***

__HAL_RTC_RESET_HANDLE_STATE

Description:

- Reset RTC handle state.

Parameters:

- `__HANDLE__`: specifies the RTC handle.

Return value:

- None

__HAL_RTC_WRITEPROTECTION_DISABLE

Description:

- Disable the write protection for RTC registers.

Parameters:

- `__HANDLE__`: specifies the RTC handle.

Return value:

- None

__HAL_RTC_WRITEPROTECTION_ENABLE

Description:

- Enable the write protection for RTC registers.

Parameters:

- `__HANDLE__`: specifies the RTC handle.

Return value:

- None

__HAL_RTC_IS_CALENDAR_INITIALIZED

Description:

- Check whether the RTC Calendar is initialized.

Parameters:

- `__HANDLE__`: specifies the RTC handle.

Return value:

- None

__HAL_RTC_ALARMA_ENABLE

Description:

- Enable the RTC ALARMA peripheral.

Parameters:

- `__HANDLE__`: specifies the RTC handle.

Return value:

- None

__HAL_RTC_ALARMA_DISABLE

Description:

- Disable the RTC ALARMA peripheral.

Parameters:

- `__HANDLE__`: specifies the RTC handle.

Return value:

- None

__HAL_RTC_ALARMB_ENABLE

Description:

- Enable the RTC ALARMB peripheral.

Parameters:

- `__HANDLE__`: specifies the RTC handle.

Return value:

- None

__HAL_RTC_ALARMB_DISABLE

Description:

- Disable the RTC ALARMB peripheral.

Parameters:

- `__HANDLE__`: specifies the RTC handle.

Return value:

- None

__HAL_RTC_ALARM_ENABLE_IT

Description:

- Enable the RTC Alarm interrupt.

Parameters:

- `__HANDLE__`: specifies the RTC handle.
- `__INTERRUPT__`: specifies the RTC Alarm interrupt sources to be enabled or disabled. This parameter can be any combination of the following values:
 - `RTC_IT_ALRA`: Alarm A interrupt
 - `RTC_IT_ALRB`: Alarm B interrupt

Return value:

- None

__HAL_RTC_ALARM_DISABLE_IT

Description:

- Disable the RTC Alarm interrupt.

Parameters:

- `__HANDLE__`: specifies the RTC handle.
- `__INTERRUPT__`: specifies the RTC Alarm interrupt sources to be enabled or disabled. This parameter can be any combination of the following values:
 - `RTC_IT_ALRA`: Alarm A interrupt
 - `RTC_IT_ALRB`: Alarm B interrupt

Return value:

- None

__HAL_RTC_ALARM_GET_IT

Description:

- Check whether the specified RTC Alarm interrupt has occurred or not.

Parameters:

- `__HANDLE__`: specifies the RTC handle.
- `__INTERRUPT__`: specifies the RTC Alarm interrupt to check. This parameter can be:
 - `RTC_IT_ALRA`: Alarm A interrupt
 - `RTC_IT_ALRB`: Alarm B interrupt

Return value:

- None

__HAL_RTC_ALARM_GET_FLAG

Description:

- Get the selected RTC Alarm's flag status.

Parameters:

- __HANDLE__: specifies the RTC handle.
- __FLAG__: specifies the RTC Alarm Flag to check. This parameter can be:
 - RTC_FLAG_ALRAF: Alarm A interrupt flag
 - RTC_FLAG_ALRAWF: Alarm A 'write allowed' flag
 - RTC_FLAG_ALRBF: Alarm B interrupt flag
 - RTC_FLAG_ALRBWF: Alarm B 'write allowed' flag

Return value:

- None

__HAL_RTC_ALARM_CLEAR_FLAG

Description:

- Clear the RTC Alarm's pending flags.

Parameters:

- __HANDLE__: specifies the RTC handle.
- __FLAG__: specifies the RTC Alarm flag to be cleared. This parameter can be:
 - RTC_FLAG_ALRAF
 - RTC_FLAG_ALRBF

Return value:

- None

__HAL_RTC_ALARM_GET_IT_SOURCE

Description:

- Check whether the specified RTC Alarm interrupt has been enabled or not.

Parameters:

- __HANDLE__: specifies the RTC handle.
- __INTERRUPT__: specifies the RTC Alarm interrupt sources to check. This parameter can be:
 - RTC_IT_ALRA: Alarm A interrupt
 - RTC_IT_ALRB: Alarm B interrupt

Return value:

- None

__HAL_RTC_ALARM_EXTI_ENABLE_IT

Description:

- Enable interrupt on the RTC Alarm associated EXTI line.

Return value:

- None

__HAL_RTC_ALARM_EXTI_DISABLE_IT

Description:

- Disable interrupt on the RTC Alarm associated EXTI line.

Return value:

- None

__HAL_RTC_ALARM_EXTI_ENABLE_EVENT**Description:**

- Enable event on the RTC Alarm associated EXTI line.

Return value:

- None.

__HAL_RTC_ALARM_EXTI_DISABLE_EVENT**Description:**

- Disable event on the RTC Alarm associated EXTI line.

Return value:

- None.

__HAL_RTC_ALARM_EXTI_ENABLE_FALLING_EDGE**Description:**

- Enable falling edge trigger on the RTC Alarm associated EXTI line.

Return value:

- None.

__HAL_RTC_ALARM_EXTI_DISABLE_FALLING_EDGE**Description:**

- Disable falling edge trigger on the RTC Alarm associated EXTI line.

Return value:

- None.

__HAL_RTC_ALARM_EXTI_ENABLE_RISING_EDGE**Description:**

- Enable rising edge trigger on the RTC Alarm associated EXTI line.

Return value:

- None.

__HAL_RTC_ALARM_EXTI_DISABLE_RISING_EDGE**Description:**

- Disable rising edge trigger on the RTC Alarm associated EXTI line.

Return value:

- None.

__HAL_RTC_ALARM_EXTI_ENABLE_RISING_FALLING_EDGE**Description:**

- Enable rising & falling edge trigger on the RTC Alarm associated EXTI line.

Return value:

- None.

__HAL_RTC_ALARM_EXTI_DISABLE_RISING_FALLING_EDGE**Description:**

- Disable rising & falling edge trigger on the RTC Alarm associated EXTI line.

Return value:

- None.

__HAL_RTC_ALARM_EXTI_GET_FLAG

Description:

- Check whether the RTC Alarm associated EXTI line interrupt flag is set or not.

Return value:

- Line: Status.

__HAL_RTC_ALARM_EXTI_CLEAR_FLAG

Description:

- Clear the RTC Alarm associated EXTI line flag.

Return value:

- None.

__HAL_RTC_ALARM_EXTI_GENERATE_SWIT

Description:

- Generate a Software interrupt on RTC Alarm associated EXTI line.

Return value:

- None.

RTC Flags Definitions

RTC_FLAG_RECALPF

Recalibration pending flag

RTC_FLAG_TAMP2F

Tamper 2 event flag

RTC_FLAG_TAMP1F

Tamper 1 event flag

RTC_FLAG_TSOVF

Timestamp overflow flag

RTC_FLAG_TSF

Timestamp event flag

RTC_FLAG_WUTF

Wakeup timer event flag

RTC_FLAG_ALRBF

Alarm B event flag

RTC_FLAG_ALRAF

Alarm A event flag

RTC_FLAG_INITF

RTC in initialization mode flag

RTC_FLAG_RSF

Register synchronization flag

RTC_FLAG_INITS

RTC initialization status flag

RTC_FLAG_SHPF

Shift operation pending flag

RTC_FLAG_WUTWF

WUTR register write allowance flag

RTC_FLAG_ALRBWF

ALRMBR register write allowance flag

RTC_FLAG_ALRAWF

ALRMAR register write allowance flag

RTC Hour Formats**RTC_HOURFORMAT_24****RTC_HOURFORMAT_12*****RTC Input Parameter Format Definitions*****RTC_FORMAT_BIN****RTC_FORMAT_BCD*****RTC Interrupts Definitions*****RTC_IT_TS**

Enable Timestamp Interrupt

RTC_IT_WUT

Enable Wakeup timer Interrupt

RTC_IT_ALRB

Enable Alarm B Interrupt

RTC_IT_ALRA

Enable Alarm A Interrupt

RTC Private macros to check input parameters**IS_RTC_HOUR_FORMAT****IS_RTC_OUTPUT****IS_RTC_OUTPUT_POL****IS_RTC_OUTPUT_TYPE****IS_RTC_ASYNC_PREDIV****IS_RTC_SYNC_PREDIV****IS_RTC_HOUR12****IS_RTC_HOUR24****IS_RTC_MINUTES****IS_RTC_SECONDS**

IS_RTC_HOURFORMAT12

IS_RTC_DAYLIGHT_SAVING

IS_RTC_STORE_OPERATION

IS_RTC_FORMAT

IS_RTC_YEAR

IS_RTC_MONTH

IS_RTC_DATE

IS_RTC_WEEKDAY

IS_RTC_ALARM_DATE_WEEKDAY_DATE

IS_RTC_ALARM_DATE_WEEKDAY_WEEKDAY

IS_RTC_ALARM_DATE_WEEKDAY_SEL

IS_RTC_ALARM_MASK

IS_RTC_ALARM

IS_RTC_ALARM_SUB_SECOND_VALUE

IS_RTC_ALARM_SUB_SECOND_MASK

RTC Month Date Definitions (in BCD format)

RTC_MONTH_JANUARY

RTC_MONTH_FEBRUARY

RTC_MONTH_MARCH

RTC_MONTH_APRIL

RTC_MONTH_MAY

RTC_MONTH_JUNE

RTC_MONTH_JULY

RTC_MONTH_AUGUST

RTC_MONTH_SEPTEMBER

RTC_MONTH_OCTOBER

RTC_MONTH_NOVEMBER

RTC_MONTH_DECEMBER

RTC Output Polarity Definitions**RTC_OUTPUT_POLARITY_HIGH****RTC_OUTPUT_POLARITY_LOW*****RTC Output Selection Definitions*****RTC_OUTPUT_DISABLE****RTC_OUTPUT_ALARM_A****RTC_OUTPUT_ALARM_B****RTC_OUTPUT_WAKEUP*****RTC Output Type ALARM OUT*****RTC_OUTPUT_TYPE_OPENDRAIN****RTC_OUTPUT_TYPE_PUSH_PULL*****RTC Store Operation Definitions*****RTC_STORE_OPERATION_RESET****RTC_STORE_OPERATION_SET*****RTC WeekDay Definitions*****RTC_WEEKDAY_MONDAY****RTC_WEEKDAY_TUESDAY****RTC_WEEKDAY_WEDNESDAY****RTC_WEEKDAY_THURSDAY****RTC_WEEKDAY_FRIDAY****RTC_WEEKDAY_SATURDAY****RTC_WEEKDAY_SUNDAY**

58 HAL RTC Extension Driver

58.1 RTCEX Firmware driver registers structures

58.1.1 RTC_TamperTypeDef

RTC_TamperTypeDef is defined in the `stm32f4xx_hal_rtc_ex.h`

Data Fields

- *uint32_t Tamper*
- *uint32_t PinSelection*
- *uint32_t Trigger*
- *uint32_t Filter*
- *uint32_t SamplingFrequency*
- *uint32_t PrechargeDuration*
- *uint32_t TamperPullUp*
- *uint32_t TimeStampOnTamperDetection*

Field Documentation

- *uint32_t RTC_TamperTypeDef::Tamper*
Specifies the Tamper Pin. This parameter can be a value of *RTCEX_Tamper_Pin_Definitions*
- *uint32_t RTC_TamperTypeDef::PinSelection*
Specifies the Tamper Pin. This parameter can be a value of *RTCEX_Tamper_Pin_Selection*
- *uint32_t RTC_TamperTypeDef::Trigger*
Specifies the Tamper Trigger. This parameter can be a value of *RTCEX_Tamper_Trigger_Definitions*
- *uint32_t RTC_TamperTypeDef::Filter*
Specifies the RTC Filter Tamper. This parameter can be a value of *RTCEX_Tamper_Filter_Definitions*
- *uint32_t RTC_TamperTypeDef::SamplingFrequency*
Specifies the sampling frequency. This parameter can be a value of *RTCEX_Tamper_Sampling_Frequencies_Definitions*
- *uint32_t RTC_TamperTypeDef::PrechargeDuration*
Specifies the Precharge Duration . This parameter can be a value of *RTCEX_Tamper_Pin_Precharge_Duration_Definitions*
- *uint32_t RTC_TamperTypeDef::TamperPullUp*
Specifies the Tamper PullUp . This parameter can be a value of *RTCEX_Tamper_Pull_Up_Definitions*
- *uint32_t RTC_TamperTypeDef::TimeStampOnTamperDetection*
Specifies the TimeStampOnTamperDetection. This parameter can be a value of *RTCEX_Tamper_TimeStampOnTamperDetection_Definitions*

58.2 RTCEX Firmware driver API description

The following section lists the various functions of the RTCEX library.

58.2.1 How to use this driver

- Enable the RTC domain access.
- Configure the RTC Prescaler (Asynchronous and Synchronous) and RTC hour format using the `HAL_RTC_Init()` function.

RTC Wakeup configuration

- To configure the RTC Wakeup Clock source and Counter use the `HAL_RTCEX_SetWakeUpTimer()` function. You can also configure the RTC Wakeup timer in interrupt mode using the `HAL_RTCEX_SetWakeUpTimer_IT()` function.
- To read the RTC Wakeup Counter register, use the `HAL_RTCEX_GetWakeUpTimer()` function.

Timestamp configuration

- To configure the RTC Timestamp use the `HAL_RTCEX_SetTimeStamp()` function. You can also configure the RTC Timestamp with interrupt mode using the `HAL_RTCEX_SetTimeStamp_IT()` function.
- To read the RTC Timestamp Time and Date register, use the `HAL_RTCEX_GetTimeStamp()` function.
- The Timestamp alternate function can be mapped either to RTC_AF1 (PC13) or RTC_AF2 (PI8) depending on the value of TSINSEL bit in RTC_TAFCR register. For STM32F446xx devices RTC_AF2 corresponds to pin PA0 and not to pin PI8. The corresponding pin is also selected by `HAL_RTCEX_SetTimeStamp()` or `HAL_RTCEX_SetTimeStamp_IT()` functions.

Tamper configuration

- To Enable the RTC Tamper and configure the Tamper filter count, trigger Edge or Level according to the Tamper filter value (if equal to 0 Edge else Level), sampling frequency, precharge or discharge and Pull-UP use the `HAL_RTCEX_SetTamper()` function. You can configure RTC Tamper in interrupt mode using `HAL_RTCEX_SetTamper_IT()` function.
- The TAMPER1 alternate function can be mapped either to RTC_AF1 (PC13) or RTC_AF2 (PI8) depending on the value of TAMP1INSEL bit in RTC_TAFCR register. The corresponding pin is also selected by `HAL_RTCEX_SetTamper()` or `HAL_RTCEX_SetTamper_IT()` functions.
- The TAMPER2 alternate function is mapped to RTC_AF2 (PI8). For STM32F446xx devices RTC_AF2 corresponds to pin PA0 and not to pin PI8.

Backup Data Registers configuration

- To write to the RTC Backup Data registers, use the `HAL_RTCEX_BKUPWrite()` function.
- To read the RTC Backup Data registers, use the `HAL_RTCEX_BKUPRead()` function.

Coarse Digital Calibration configuration

- The Coarse Digital Calibration can be used to compensate crystal inaccuracy by setting the DCS bit in RTC_CALIBR register.
- When positive calibration is enabled (DCS = '0'), 2 asynchronous prescaler clock cycles are added every minute during 2xDC minutes. This causes the calendar to be updated sooner, thereby adjusting the effective RTC frequency to be a bit higher.
- When negative calibration is enabled (DCS = '1'), 1 asynchronous prescaler clock cycle is removed every minute during 2xDC minutes. This causes the calendar to be updated later, thereby adjusting the effective RTC frequency to be a bit lower.
- DC is configured through bits DC[4:0] of RTC_CALIBR register. This number ranges from 0 to 31 corresponding to a time interval (2xDC) ranging from 0 to 62.
- In order to measure the clock deviation, a 512 Hz clock is output for calibration.
- The RTC Coarse Digital Calibration value and sign can be calibrated using the `HAL_RTCEX_SetCoarseCalib()` function.

Smooth Digital Calibration configuration

- RTC frequency can be digitally calibrated with a resolution of about 0.954 ppm with a range from -487.1 ppm to +488.5 ppm. The correction of the frequency is performed using a series of small adjustments (adding and/or subtracting individual RTCCLK pulses).
- The smooth digital calibration is performed during a cycle of about 2²⁰ RTCCLK pulses (or 32 seconds) when the input frequency is 32,768 Hz. This cycle is maintained by a 20-bit counter clocked by RTCCLK.
- The smooth calibration register (RTC_CALR) specifies the number of RTCCLK clock cycles to be masked during the 32-second cycle.
- The RTC Smooth Digital Calibration value and the corresponding calibration cycle period (32s, 16s, or 8s) can be calibrated using the `HAL_RTCEX_SetSmoothCalib()` function.

58.2.2

RTC Timestamp and Tamper functions

This section provides functions allowing to configure Timestamp feature

This section contains the following APIs:

- *HAL_RTCEx_SetTimeStamp()*
- *HAL_RTCEx_SetTimeStamp_IT()*
- *HAL_RTCEx_DeactivateTimeStamp()*
- *HAL_RTCEx_GetTimeStamp()*
- *HAL_RTCEx_SetTamper()*
- *HAL_RTCEx_SetTamper_IT()*
- *HAL_RTCEx_DeactivateTamper()*
- *HAL_RTCEx_TamperTimeStampIRQHandler()*
- *HAL_RTCEx_TimeStampEventCallback()*
- *HAL_RTCEx_Tamper1EventCallback()*
- *HAL_RTCEx_Tamper2EventCallback()*
- *HAL_RTCEx_PollForTimeStampEvent()*
- *HAL_RTCEx_PollForTamper1Event()*
- *HAL_RTCEx_PollForTamper2Event()*

58.2.3 RTC Wake-up functions

This section provides functions allowing to configure Wake-up feature

This section contains the following APIs:

- *HAL_RTCEx_SetWakeUpTimer()*
- *HAL_RTCEx_SetWakeUpTimer_IT()*
- *HAL_RTCEx_DeactivateWakeUpTimer()*
- *HAL_RTCEx_GetWakeUpTimer()*
- *HAL_RTCEx_WakeUpTimerIRQHandler()*
- *HAL_RTCEx_WakeUpTimerEventCallback()*
- *HAL_RTCEx_PollForWakeUpTimerEvent()*

58.2.4 Extension Peripheral Control functions

This subsection provides functions allowing to

- Write a data in a specified RTC Backup data register
- Read a data in a specified RTC Backup data register
- Set the Coarse calibration parameters.
- Deactivate the Coarse calibration parameters
- Set the Smooth calibration parameters.
- Configure the Synchronization Shift Control Settings.
- Configure the Calibration Pinout (RTC_CALIB) Selection (1Hz or 512Hz).
- Deactivate the Calibration Pinout (RTC_CALIB) Selection (1Hz or 512Hz).
- Enable the RTC reference clock detection.
- Disable the RTC reference clock detection.
- Enable the Bypass Shadow feature.
- Disable the Bypass Shadow feature.

This section contains the following APIs:

- *HAL_RTCEx_BKUPWrite()*
- *HAL_RTCEx_BKUPRead()*
- *HAL_RTCEx_SetCoarseCalib()*
- *HAL_RTCEx_DeactivateCoarseCalib()*
- *HAL_RTCEx_SetSmoothCalib()*
- *HAL_RTCEx_SetSynchroShift()*
- *HAL_RTCEx_SetCalibrationOutPut()*
- *HAL_RTCEx_DeactivateCalibrationOutPut()*
- *HAL_RTCEx_SetRefClock()*

- **HAL_RTCEx_DeactivateRefClock()**
- **HAL_RTCEx_EnableBypassShadow()**
- **HAL_RTCEx_DisableBypassShadow()**

58.2.5 Extended features functions

This section provides functions allowing to:

- RTC Alarm B callback
- RTC Poll for Alarm B request

This section contains the following APIs:

- **HAL_RTCEx_AlarmBEventCallback()**
- **HAL_RTCEx_PollForAlarmBEvent()**

58.2.6 Detailed description of functions

HAL_RTCEx_SetTimeStamp

Function name

HAL_StatusTypeDef HAL_RTCEx_SetTimeStamp (RTC_HandleTypeDef * hrtc, uint32_t RTC_TimeStampEdge, uint32_t RTC_TimeStampPin)

Function description

Sets Timestamp.

Parameters

- **hrtc**: pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.
- **RTC_TimeStampEdge**: Specifies the pin edge on which the Timestamp is activated. This parameter can be one of the following values:
 - **RTC_TIMESTAMPEdge_RISING**: the Timestamp event occurs on the rising edge of the related pin.
 - **RTC_TIMESTAMPEdge_FALLING**: the Timestamp event occurs on the falling edge of the related pin.
- **RTC_TimeStampPin**: Specifies the RTC Timestamp Pin. This parameter can be one of the following values:
 - **RTC_TIMESTAMPPIN_DEFAULT**: PC13 is selected as RTC Timestamp Pin.
 - **RTC_TIMESTAMPPIN_POS1**: PI8 is selected as RTC Timestamp Pin.

Return values

- **HAL**: status

Notes

- This API must be called before enabling the Timestamp feature.
- **RTC_TIMESTAMPPIN_POS1** corresponds to pin PA0 in the case of STM32F446xx devices.
- **RTC_TIMESTAMPPIN_POS1** is not applicable to the following list of devices: STM32F412xx, STM32F413xx and STM32F423xx.

HAL_RTCEx_SetTimeStamp_IT

Function name

HAL_StatusTypeDef HAL_RTCEx_SetTimeStamp_IT (RTC_HandleTypeDef * hrtc, uint32_t RTC_TimeStampEdge, uint32_t RTC_TimeStampPin)

Function description

Sets Timestamp with Interrupt.

Parameters

- **hrtc**: pointer to a `RTC_HandleTypeDef` structure that contains the configuration information for RTC.
- **RTC_TimeStampEdge**: Specifies the pin edge on which the Timestamp is activated. This parameter can be one of the following values:
 - `RTC_TIMESTAMPEdge_RISING`: the Timestamp event occurs on the rising edge of the related pin.
 - `RTC_TIMESTAMPEdge_FALLING`: the Timestamp event occurs on the falling edge of the related pin.
- **RTC_TimeStampPin**: Specifies the RTC Timestamp Pin. This parameter can be one of the following values:
 - `RTC_TIMESTAMPPIN_DEFAULT`: PC13 is selected as RTC Timestamp Pin.
 - `RTC_TIMESTAMPPIN_POS1`: PI8 is selected as RTC Timestamp Pin.

Return values

- **HAL**: status

Notes

- This API must be called before enabling the Timestamp feature.
- `RTC_TIMESTAMPPIN_POS1` corresponds to pin PA0 in the case of STM32F446xx devices.
- `RTC_TIMESTAMPPIN_POS1` is not applicable to the following list of devices: STM32F412xx, STM32F413xx and STM32F423xx.

HAL_RTCEx_DeactivateTimeStamp

Function name

HAL_StatusTypeDef HAL_RTCEx_DeactivateTimeStamp (RTC_HandleTypeDef * hrtc)

Function description

Deactivates Timestamp.

Parameters

- **hrtc**: pointer to a `RTC_HandleTypeDef` structure that contains the configuration information for RTC.

Return values

- **HAL**: status

HAL_RTCEx_GetTimeStamp

Function name

HAL_StatusTypeDef HAL_RTCEx_GetTimeStamp (RTC_HandleTypeDef * hrtc, RTC_TimeTypeDef * sTimeStamp, RTC_DateTypeDef * sTimeStampDate, uint32_t Format)

Function description

Gets the RTC Timestamp value.

Parameters

- **hrtc**: pointer to a `RTC_HandleTypeDef` structure that contains the configuration information for RTC.
- **sTimeStamp**: Pointer to Time structure
- **sTimeStampDate**: Pointer to Date structure
- **Format**: specifies the format of the entered parameters. This parameter can be one of the following values:
 - `RTC_FORMAT_BIN`: Binary data format
 - `RTC_FORMAT_BCD`: BCD data format

Return values

- **HAL**: status

HAL_RTCEX_SetTamper

Function name

HAL_StatusTypeDef HAL_RTCEX_SetTamper (RTC_HandleTypeDef * hrtc, RTC_TamperTypeDef * sTamper)

Function description

Sets Tamper.

Parameters

- **hrtc**: pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.
- **sTamper**: Pointer to Tamper Structure.

Return values

- **HAL**: status

Notes

- By calling this API the tamper global interrupt will be disabled.

HAL_RTCEX_SetTamper_IT

Function name

HAL_StatusTypeDef HAL_RTCEX_SetTamper_IT (RTC_HandleTypeDef * hrtc, RTC_TamperTypeDef * sTamper)

Function description

Sets Tamper with interrupt.

Parameters

- **hrtc**: pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.
- **sTamper**: Pointer to RTC Tamper.

Return values

- **HAL**: status

Notes

- By calling this API the tamper global interrupt will be enabled.

HAL_RTCEX_DeactivateTamper

Function name

HAL_StatusTypeDef HAL_RTCEX_DeactivateTamper (RTC_HandleTypeDef * hrtc, uint32_t Tamper)

Function description

Deactivates Tamper.

Parameters

- **hrtc**: pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.
- **Tamper**: Selected tamper pin. This parameter can be any combination of the following values:
 - RTC_TAMPER_1: Tamper 1
 - RTC_TAMPER_2: Tamper 2

Return values

- **HAL**: status

Notes

- The tamper global interrupt bit will remain unchanged.
- RTC_TAMPER_2 is not applicable to all devices.

HAL_RTCEX_TamperTimeStampIRQHandler

Function name

```
void HAL_RTCEX_TamperTimeStampIRQHandler (RTC_HandleTypeDef * hrtc)
```

Function description

Handles Timestamp and Tamper interrupt request.

Parameters

- **hrtc**: pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.

Return values

- **None**:

HAL_RTCEX_Tamper1EventCallback

Function name

```
void HAL_RTCEX_Tamper1EventCallback (RTC_HandleTypeDef * hrtc)
```

Function description

Tamper 1 callback.

Parameters

- **hrtc**: pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.

Return values

- **None**:

HAL_RTCEX_Tamper2EventCallback

Function name

```
void HAL_RTCEX_Tamper2EventCallback (RTC_HandleTypeDef * hrtc)
```

Function description

Tamper 2 callback.

Parameters

- **hrtc**: pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.

Return values

- **None**:

HAL_RTCEX_TimeStampEventCallback

Function name

```
void HAL_RTCEX_TimeStampEventCallback (RTC_HandleTypeDef * hrtc)
```

Function description

Timestamp callback.

Parameters

- **hrtc**: pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.

Return values

- **None:**

HAL_RTCEx_PollForTimeStampEvent

Function name

HAL_StatusTypeDef HAL_RTCEx_PollForTimeStampEvent (RTC_HandleTypeDef * hrtc, uint32_t Timeout)

Function description

Handles Timestamp polling request.

Parameters

- **hrtc:** pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.
- **Timeout:** Timeout duration

Return values

- **HAL:** status

HAL_RTCEx_PollForTamper1Event

Function name

HAL_StatusTypeDef HAL_RTCEx_PollForTamper1Event (RTC_HandleTypeDef * hrtc, uint32_t Timeout)

Function description

Handles Tamper 1 Polling.

Parameters

- **hrtc:** pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.
- **Timeout:** Timeout duration

Return values

- **HAL:** status

HAL_RTCEx_PollForTamper2Event

Function name

HAL_StatusTypeDef HAL_RTCEx_PollForTamper2Event (RTC_HandleTypeDef * hrtc, uint32_t Timeout)

Function description

Handles Tamper 2 Polling.

Parameters

- **hrtc:** pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.
- **Timeout:** Timeout duration

Return values

- **HAL:** status

HAL_RTCEx_SetWakeUpTimer

Function name

HAL_StatusTypeDef HAL_RTCEx_SetWakeUpTimer (RTC_HandleTypeDef * hrtc, uint32_t WakeUpCounter, uint32_t WakeUpClock)

Function description

Sets wakeup timer.

Parameters

- **hrtc**: pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.
- **WakeUpCounter**: Wakeup counter
- **WakeUpClock**: Wakeup clock

Return values

- **HAL**: status

HAL_RTCEX_SetWakeUpTimer_IT

Function name

HAL_StatusTypeDef HAL_RTCEX_SetWakeUpTimer_IT (RTC_HandleTypeDef * hrtc, uint32_t WakeUpCounter, uint32_t WakeUpClock)

Function description

Sets wakeup timer with interrupt.

Parameters

- **hrtc**: pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.
- **WakeUpCounter**: Wakeup counter
- **WakeUpClock**: Wakeup clock

Return values

- **HAL**: status

HAL_RTCEX_DeactivateWakeUpTimer

Function name

HAL_StatusTypeDef HAL_RTCEX_DeactivateWakeUpTimer (RTC_HandleTypeDef * hrtc)

Function description

Deactivates wakeup timer counter.

Parameters

- **hrtc**: pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.

Return values

- **HAL**: status

HAL_RTCEX_GetWakeUpTimer

Function name

uint32_t HAL_RTCEX_GetWakeUpTimer (RTC_HandleTypeDef * hrtc)

Function description

Gets wakeup timer counter.

Parameters

- **hrtc**: pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.

Return values

- **Counter**: value

HAL_RTCEX_WakeUpTimerIRQHandler

Function name

void HAL_RTCEX_WakeUpTimerIRQHandler (RTC_HandleTypeDef * hrtc)

Function description

Handles Wakeup Timer interrupt request.

Parameters

- **hrtc**: pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.

Return values

- **None**:

Notes

- Unlike alarm interrupt line (shared by Alarms A and B) or tamper interrupt line (shared by timestamp and tampers) wakeup timer interrupt line is exclusive to the wakeup timer. There is no need in this case to check on the interrupt enable status via __HAL_RTC_WAKEUPTIMER_GET_IT_SOURCE().

HAL_RTCEx_WakeUpTimerEventCallback

Function name

```
void HAL_RTCEx_WakeUpTimerEventCallback (RTC_HandleTypeDef * hrtc)
```

Function description

Wakeup Timer callback.

Parameters

- **hrtc**: pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.

Return values

- **None**:

HAL_RTCEx_PollForWakeUpTimerEvent

Function name

```
HAL_StatusTypeDef HAL_RTCEx_PollForWakeUpTimerEvent (RTC_HandleTypeDef * hrtc, uint32_t Timeout)
```

Function description

Handles Wakeup Timer Polling.

Parameters

- **hrtc**: pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.
- **Timeout**: Timeout duration

Return values

- **HAL**: status

HAL_RTCEx_BKUPWrite

Function name

```
void HAL_RTCEx_BKUPWrite (RTC_HandleTypeDef * hrtc, uint32_t BackupRegister, uint32_t Data)
```

Function description

Writes a data in a specified RTC Backup data register.

Parameters

- **hrtc**: pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.
- **BackupRegister**: RTC Backup data Register number. This parameter can be: RTC_BKP_DRx (where x can be from 0 to 19) to specify the register.
- **Data**: Data to be written in the specified RTC Backup data register.

Return values

- **None:**

HAL_RTCEX_BKUPRead

Function name

uint32_t HAL_RTCEX_BKUPRead (RTC_HandleTypeDef * hrtc, uint32_t BackupRegister)

Function description

Reads data from the specified RTC Backup data Register.

Parameters

- **hrtc:** pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.
- **BackupRegister:** RTC Backup data Register number. This parameter can be: RTC_BKP_DRx (where x can be from 0 to 19) to specify the register.

Return values

- **Read:** value

HAL_RTCEX_SetCoarseCalib

Function name

HAL_StatusTypeDef HAL_RTCEX_SetCoarseCalib (RTC_HandleTypeDef * hrtc, uint32_t CalibSign, uint32_t Value)

Function description

Sets the Coarse calibration parameters.

Parameters

- **hrtc:** pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.
- **CalibSign:** Specifies the sign of the coarse calibration value. This parameter can be one of the following values:
 - RTC_CALIBSIGN_POSITIVE: The value sign is positive
 - RTC_CALIBSIGN_NEGATIVE: The value sign is negative
- **Value:** value of coarse calibration expressed in ppm (coded on 5 bits).

Return values

- **HAL:** status

Notes

- This Calibration value should be between 0 and 63 when using negative sign with a 2-ppm step.
- This Calibration value should be between 0 and 126 when using positive sign with a 4-ppm step.

HAL_RTCEX_DeactivateCoarseCalib

Function name

HAL_StatusTypeDef HAL_RTCEX_DeactivateCoarseCalib (RTC_HandleTypeDef * hrtc)

Function description

Deactivates the Coarse calibration parameters.

Parameters

- **hrtc:** pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.

Return values

- **HAL:** status

HAL_RTCEX_SetSmoothCalib

Function name

HAL_StatusTypeDef HAL_RTCEX_SetSmoothCalib (RTC_HandleTypeDef * hrtc, uint32_t SmoothCalibPeriod, uint32_t SmoothCalibPlusPulses, uint32_t SmoothCalibMinusPulsesValue)

Function description

Sets the Smooth calibration parameters.

Parameters

- **hrtc**: pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.
- **SmoothCalibPeriod**: Select the Smooth Calibration Period. This parameter can be one of the following values:
 - RTC_SMOOTHCALIB_PERIOD_32SEC: The smooth calibration period is 32s.
 - RTC_SMOOTHCALIB_PERIOD_16SEC: The smooth calibration period is 16s.
 - RTC_SMOOTHCALIB_PERIOD_8SEC: The smooth calibration period is 8s.
- **SmoothCalibPlusPulses**: Select to Set or reset the CALP bit. This parameter can be one of the following values:
 - RTC_SMOOTHCALIB_PLUSPULSES_SET: Add one RTCCLK pulse every 2*11 pulses.
 - RTC_SMOOTHCALIB_PLUSPULSES_RESET: No RTCCLK pulses are added.
- **SmoothCalibMinusPulsesValue**: Select the value of CALM[8:0] bits. This parameter can be any value from 0 to 0x000001FF.

Return values

- **HAL**: status

Notes

- To deactivate the smooth calibration, the field SmoothCalibPlusPulses must be equal to SMOOTHCALIB_PLUSPULSES_RESET and the field SmoothCalibMinusPulsesValue must be equal to 0.

HAL_RTCEX_SetSynchroShift

Function name

HAL_StatusTypeDef HAL_RTCEX_SetSynchroShift (RTC_HandleTypeDef * hrtc, uint32_t ShiftAdd1S, uint32_t ShiftSubFS)

Function description

Configures the Synchronization Shift Control Settings.

Parameters

- **hrtc**: pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.
- **ShiftAdd1S**: Select to add or not 1 second to the time calendar. This parameter can be one of the following values:
 - RTC_SHIFTADD1S_SET: Add one second to the clock calendar.
 - RTC_SHIFTADD1S_RESET: No effect.
- **ShiftSubFS**: Select the number of Second Fractions to substitute. This parameter can be any value from 0 to 0x7FFF.

Return values

- **HAL**: status

Notes

- When REFCKON is set, firmware must not write to Shift control register.

HAL_RTCEX_SetCalibrationOutPut

Function name

HAL_StatusTypeDef HAL_RTCEX_SetCalibrationOutPut (RTC_HandleTypeDef * hrtc, uint32_t CalibOutput)

Function description

Configures the Calibration Pinout (RTC_CALIB) Selection (1Hz or 512Hz).

Parameters

- **hrtc**: pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.
- **CalibOutput**: Select the Calibration output Selection. This parameter can be one of the following values:
 - RTC_CALIBOUTPUT_512HZ: A signal has a regular waveform at 512Hz.
 - RTC_CALIBOUTPUT_1HZ: A signal has a regular waveform at 1Hz.

Return values

- **HAL**: status

HAL_RTCEX_DeactivateCalibrationOutPut

Function name

HAL_StatusTypeDef HAL_RTCEX_DeactivateCalibrationOutPut (RTC_HandleTypeDef * hrtc)

Function description

Deactivates the Calibration Pinout (RTC_CALIB) Selection (1Hz or 512Hz).

Parameters

- **hrtc**: pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.

Return values

- **HAL**: status

HAL_RTCEX_SetRefClock

Function name

HAL_StatusTypeDef HAL_RTCEX_SetRefClock (RTC_HandleTypeDef * hrtc)

Function description

Enables the RTC reference clock detection.

Parameters

- **hrtc**: pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.

Return values

- **HAL**: status

HAL_RTCEX_DeactivateRefClock

Function name

HAL_StatusTypeDef HAL_RTCEX_DeactivateRefClock (RTC_HandleTypeDef * hrtc)

Function description

Disable the RTC reference clock detection.

Parameters

- **hrtc**: pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.

Return values

- **HAL:** status

HAL_RTCEX_EnableBypassShadow

Function name

HAL_StatusTypeDef HAL_RTCEX_EnableBypassShadow (RTC_HandleTypeDef * hrtc)

Function description

Enables the Bypass Shadow feature.

Parameters

- **hrtc:** pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.

Return values

- **HAL:** status

Notes

- When the Bypass Shadow is enabled the calendar value are taken directly from the Calendar counter.

HAL_RTCEX_DisableBypassShadow

Function name

HAL_StatusTypeDef HAL_RTCEX_DisableBypassShadow (RTC_HandleTypeDef * hrtc)

Function description

Disables the Bypass Shadow feature.

Parameters

- **hrtc:** pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.

Return values

- **HAL:** status

Notes

- When the Bypass Shadow is enabled the calendar value are taken directly from the Calendar counter.

HAL_RTCEX_AlarmBEventCallback

Function name

void HAL_RTCEX_AlarmBEventCallback (RTC_HandleTypeDef * hrtc)

Function description

Alarm B callback.

Parameters

- **hrtc:** pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.

Return values

- **None:**

HAL_RTCEX_PollForAlarmBEvent

Function name

HAL_StatusTypeDef HAL_RTCEX_PollForAlarmBEvent (RTC_HandleTypeDef * hrtc, uint32_t Timeout)

Function description

Handles Alarm B Polling request.

Parameters

- **hrtc**: pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.
- **Timeout**: Timeout duration

Return values

- **HAL**: status

58.3 RTCEX Firmware driver defines

The following section lists the various define and macros of the module.

58.3.1 RTCEX

RTCEX

RTCEX Add 1 Second Parameter Definitions

RTC_SHIFTADD1S_RESET

RTC_SHIFTADD1S_SET

RTCEX Backup Registers Definitions

RTC_BKP_DR0

RTC_BKP_DR1

RTC_BKP_DR2

RTC_BKP_DR3

RTC_BKP_DR4

RTC_BKP_DR5

RTC_BKP_DR6

RTC_BKP_DR7

RTC_BKP_DR8

RTC_BKP_DR9

RTC_BKP_DR10

RTC_BKP_DR11

RTC_BKP_DR12

RTC_BKP_DR13

RTC_BKP_DR14

RTC_BKP_DR15

RTC_BKP_DR16

RTC_BKP_DR17

RTC_BKP_DR18

RTC_BKP_DR19

*RTCEX Calibration***__HAL_RTC_COARSE_CALIB_ENABLE****Description:**

- Enable the Coarse calibration process.

Parameters:

- `__HANDLE__`: specifies the RTC handle.

Return value:

- None

__HAL_RTC_COARSE_CALIB_DISABLE**Description:**

- Disable the Coarse calibration process.

Parameters:

- `__HANDLE__`: specifies the RTC handle.

Return value:

- None

__HAL_RTC_CALIBRATION_OUTPUT_ENABLE**Description:**

- Enable the RTC calibration output.

Parameters:

- `__HANDLE__`: specifies the RTC handle.

Return value:

- None

__HAL_RTC_CALIBRATION_OUTPUT_DISABLE**Description:**

- Disable the calibration output.

Parameters:

- `__HANDLE__`: specifies the RTC handle.

Return value:

- None

__HAL_RTC_CLOCKREF_DETECTION_ENABLE**Description:**

- Enable the clock reference detection.

Parameters:

- `__HANDLE__`: specifies the RTC handle.

Return value:

- None

__HAL_RTC_CLOCKREF_DETECTION_DISABLE

Description:

- Disable the clock reference detection.

Parameters:

- **__HANDLE__**: specifies the RTC handle.

Return value:

- None

__HAL_RTC_SHIFT_GET_FLAG

Description:

- Get the selected RTC shift operation's flag status.

Parameters:

- **__HANDLE__**: specifies the RTC handle.
- **__FLAG__**: specifies the RTC shift operation Flag is pending or not. This parameter can be:
 - **RTC_FLAG_SHPF**: Shift pending flag

Return value:

- None

RTCEX Calib Output Selection Definitions

RTC_CALIBOUTPUT_512HZ

RTC_CALIBOUTPUT_1HZ

RTCEX Coarse Calib Definitions

RTC_CALIBSIGN_POSITIVE

RTC_CALIBSIGN_NEGATIVE

Private macros to check input parameters

IS_RTC_BKP

IS_TIMESTAMP_EDGE

IS_RTC_TAMPER

IS_RTC_TAMPER_PIN

IS_RTC_TIMESTAMP_PIN

IS_RTC_TAMPER_TRIGGER

IS_RTC_TAMPER_FILTER

IS_RTC_TAMPER_FILTER_CONFIG_CORRECT

IS_RTC_TAMPER_SAMPLING_FREQ

IS_RTC_TAMPER_PRECHARGE_DURATION

IS_RTC_TAMPER_PULLUP_STATE

IS_RTC_TAMPER_TIMESTAMPONTAMPER_DETECTION

IS_RTC_WAKEUP_CLOCK

IS_RTC_WAKEUP_COUNTER

IS_RTC_CALIB_SIGN

IS_RTC_CALIB_VALUE

IS_RTC_SMOOTH_CALIB_PERIOD

IS_RTC_SMOOTH_CALIB_PLUS

IS_RTC_SMOOTH_CALIB_MINUS

IS_RTC_SHIFT_ADD1S

IS_RTC_SHIFT_SUBFS

IS_RTC_CALIB_OUTPUT

RTCEX Smooth Calib Period Definitions

RTC_SMOOTHCALIB_PERIOD_32SEC

If RTCCLK = 32768 Hz, smooth calibration period is 32s, otherwise 2^{20} RTCCLK pulses

RTC_SMOOTHCALIB_PERIOD_16SEC

If RTCCLK = 32768 Hz, smooth calibration period is 16s, otherwise 2^{19} RTCCLK pulses

RTC_SMOOTHCALIB_PERIOD_8SEC

If RTCCLK = 32768 Hz, smooth calibration period is 8s, otherwise 2^{18} RTCCLK pulses

RTCEX Smooth Calib Plus Pulses Definitions

RTC_SMOOTHCALIB_PLUSPULSES_SET

The number of RTCCLK pulses added during a X -second window = $Y - \text{CALM}[8:0]$ with $Y = 512, 256, 128$ when $X = 32, 16, 8$

RTC_SMOOTHCALIB_PLUSPULSES_RESET

The number of RTCCLK pulses subbstituted during a 32-second window = $\text{CALM}[8:0]$

RTCEX Tamper

__HAL_RTC_TAMPER1_ENABLE

Description:

- Enable the RTC Tamper1 input detection.

Parameters:

- __HANDLE__: specifies the RTC handle.

Return value:

- None

__HAL_RTC_TAMPER1_DISABLE

Description:

- Disable the RTC Tamper1 input detection.

Parameters:

- `__HANDLE__`: specifies the RTC handle.

Return value:

- None

__HAL_RTC_TAMPER2_ENABLE

Description:

- Enable the RTC Tamper2 input detection.

Parameters:

- `__HANDLE__`: specifies the RTC handle.

Return value:

- None

__HAL_RTC_TAMPER2_DISABLE

Description:

- Disable the RTC Tamper2 input detection.

Parameters:

- `__HANDLE__`: specifies the RTC handle.

Return value:

- None

__HAL_RTC_TAMPER_ENABLE_IT

Description:

- Enable the RTC Tamper interrupt.

Parameters:

- `__HANDLE__`: specifies the RTC handle.
- `__INTERRUPT__`: specifies the RTC Tamper interrupt sources to be enabled. This parameter can be any combination of the following values:
 - `RTC_IT_TAMP`: Tamper global interrupt

Return value:

- None

__HAL_RTC_TAMPER_DISABLE_IT

Description:

- Disable the RTC Tamper interrupt.

Parameters:

- `__HANDLE__`: specifies the RTC handle.
- `__INTERRUPT__`: specifies the RTC Tamper interrupt sources to be disabled. This parameter can be any combination of the following values:
 - `RTC_IT_TAMP`: Tamper global interrupt

Return value:

- None

__HAL_RTC_TAMPER_GET_IT

Description:

- Check whether the specified RTC Tamper interrupt has occurred or not.

Parameters:

- `__HANDLE__`: specifies the RTC handle.
- `__INTERRUPT__`: specifies the RTC Tamper interrupt to check. This parameter can be:
 - `RTC_IT_TAMP1`: Tamper 1 interrupt
 - `RTC_IT_TAMP2`: Tamper 2 interrupt

Return value:

- None

Notes:

- `RTC_IT_TAMP2` is not applicable to all devices.

__HAL_RTC_TAMPER_GET_IT_SOURCE

Description:

- Check whether the specified RTC Tamper interrupt has been enabled or not.

Parameters:

- `__HANDLE__`: specifies the RTC handle.
- `__INTERRUPT__`: specifies the RTC Tamper interrupt source to check. This parameter can be:
 - `RTC_IT_TAMP`: Tamper global interrupt

Return value:

- None

__HAL_RTC_TAMPER_GET_FLAG

Description:

- Get the selected RTC Tamper's flag status.

Parameters:

- `__HANDLE__`: specifies the RTC handle.
- `__FLAG__`: specifies the RTC Tamper flag to be checked. This parameter can be:
 - `RTC_FLAG_TAMP1F`: Tamper 1 interrupt flag
 - `RTC_FLAG_TAMP2F`: Tamper 2 interrupt flag

Return value:

- None

Notes:

- `RTC_FLAG_TAMP2F` is not applicable to all devices.

__HAL_RTC_TAMPER_CLEAR_FLAG

Description:

- Clear the RTC Tamper's pending flags.

Parameters:

- `__HANDLE__`: specifies the RTC handle.
- `__FLAG__`: specifies the RTC Tamper Flag to clear. This parameter can be:
 - `RTC_FLAG_TAMP1F`: Tamper 1 interrupt flag
 - `RTC_FLAG_TAMP2F`: Tamper 2 interrupt flag

Return value:

- None

Notes:

- `RTC_FLAG_TAMP2F` is not applicable to all devices.

RTCEX Tamper Filter Definitions

RTC_TAMPERFILTER_DISABLE

Tamper filter is disabled

RTC_TAMPERFILTER_2SAMPLE

Tamper is activated after 2 consecutive samples at the active level

RTC_TAMPERFILTER_4SAMPLE

Tamper is activated after 4 consecutive samples at the active level

RTC_TAMPERFILTER_8SAMPLE

Tamper is activated after 8 consecutive samples at the active level

RTC_TAMPERFILTER_MASK

Masking all bits except those of field TAMPFLT

RTCEX Tamper Interrupt Definitions**RTC_IT_TAMP**

Enable global Tamper Interrupt

RTCEX Tamper Pins Definitions**RTC_TAMPER_1****RTC_TAMPER_2*****RTCEX Tamper Pin Precharge Duration Definitions*****RTC_TAMPERPRECHARGEDURATION_1RTCCLK**

Tamper pins are pre-charged before sampling during 1 RTCCLK cycle

RTC_TAMPERPRECHARGEDURATION_2RTCCLK

Tamper pins are pre-charged before sampling during 2 RTCCLK cycles

RTC_TAMPERPRECHARGEDURATION_4RTCCLK

Tamper pins are pre-charged before sampling during 4 RTCCLK cycles

RTC_TAMPERPRECHARGEDURATION_8RTCCLK

Tamper pins are pre-charged before sampling during 8 RTCCLK cycles

RTC_TAMPERPRECHARGEDURATION_MASK

Masking all bits except those of field TAMPPRCH

RTC tamper Pins Selection**RTC_TAMPERPIN_DEFAULT****RTC_TAMPERPIN_POS1*****RTCEX Tamper Pull Up Definitions*****RTC_TAMPER_PULLUP_ENABLE**

Tamper pins are pre-charged before sampling

RTC_TAMPER_PULLUP_DISABLE

Tamper pins are not pre-charged before sampling

RTC_TAMPER_PULLUP_MASK

Masking all bits except bit TAMPPUDIS

RTCEX Tamper Sampling Frequencies Definitions

RTC_TAMPERSAMPLINGFREQ_RTCCLK_DIV32768

Each of the tamper inputs are sampled with a frequency = $RTCCLK / 32768$

RTC_TAMPERSAMPLINGFREQ_RTCCLK_DIV16384

Each of the tamper inputs are sampled with a frequency = $RTCCLK / 16384$

RTC_TAMPERSAMPLINGFREQ_RTCCLK_DIV8192

Each of the tamper inputs are sampled with a frequency = $RTCCLK / 8192$

RTC_TAMPERSAMPLINGFREQ_RTCCLK_DIV4096

Each of the tamper inputs are sampled with a frequency = $RTCCLK / 4096$

RTC_TAMPERSAMPLINGFREQ_RTCCLK_DIV2048

Each of the tamper inputs are sampled with a frequency = $RTCCLK / 2048$

RTC_TAMPERSAMPLINGFREQ_RTCCLK_DIV1024

Each of the tamper inputs are sampled with a frequency = $RTCCLK / 1024$

RTC_TAMPERSAMPLINGFREQ_RTCCLK_DIV512

Each of the tamper inputs are sampled with a frequency = $RTCCLK / 512$

RTC_TAMPERSAMPLINGFREQ_RTCCLK_DIV256

Each of the tamper inputs are sampled with a frequency = $RTCCLK / 256$

RTC_TAMPERSAMPLINGFREQ_RTCCLK_MASK

Masking all bits except those of field TAMPFREQ

EXTI RTC Tamper Timestamp EXTI

__HAL_RTC_TAMPER_TIMESTAMP_EXTI_ENABLE_IT

Description:

- Enable interrupt on the RTC Tamper and Timestamp associated EXTI line.

Return value:

- None

__HAL_RTC_TAMPER_TIMESTAMP_EXTI_DISABLE_IT

Description:

- Disable interrupt on the RTC Tamper and Timestamp associated EXTI line.

Return value:

- None

__HAL_RTC_TAMPER_TIMESTAMP_EXTI_ENABLE_EVENT

Description:

- Enable event on the RTC Tamper and Timestamp associated EXTI line.

Return value:

- None.

__HAL_RTC_TAMPER_TIMESTAMP_EXTI_DISABLE_EVENT**Description:**

- Disable event on the RTC Tamper and Timestamp associated EXTI line.

Return value:

- None.

__HAL_RTC_TAMPER_TIMESTAMP_EXTI_ENABLE_FALLING_EDGE**Description:**

- Enable falling edge trigger on the RTC Tamper and Timestamp associated EXTI line.

Return value:

- None.

__HAL_RTC_TAMPER_TIMESTAMP_EXTI_DISABLE_FALLING_EDGE**Description:**

- Disable falling edge trigger on the RTC Tamper and Timestamp associated EXTI line.

Return value:

- None.

__HAL_RTC_TAMPER_TIMESTAMP_EXTI_ENABLE_RISING_EDGE**Description:**

- Enable rising edge trigger on the RTC Tamper and Timestamp associated EXTI line.

Return value:

- None.

__HAL_RTC_TAMPER_TIMESTAMP_EXTI_DISABLE_RISING_EDGE**Description:**

- Disable rising edge trigger on the RTC Tamper and Timestamp associated EXTI line.

Return value:

- None.

__HAL_RTC_TAMPER_TIMESTAMP_EXTI_ENABLE_RISING_FALLING_EDGE**Description:**

- Enable rising & falling edge trigger on the RTC Tamper and Timestamp associated EXTI line.

Return value:

- None.

__HAL_RTC_TAMPER_TIMESTAMP_EXTI_DISABLE_RISING_FALLING_EDGE**Description:**

- Disable rising & falling edge trigger on the RTC Tamper and Timestamp associated EXTI line.

Return value:

- None.

__HAL_RTC_TAMPER_TIMESTAMP_EXTI_GET_FLAG**Description:**

- Check whether the RTC Tamper and Timestamp associated EXTI line interrupt flag is set or not.

Return value:

- Line: Status.

__HAL_RTC_TAMPER_TIMESTAMP_EXTI_CLEAR_FLAG

Description:

- Clear the RTC Tamper and Timestamp associated EXTI line flag.

Return value:

- None.

__HAL_RTC_TAMPER_TIMESTAMP_EXTI_GENERATE_SWIT

Description:

- Generate a Software interrupt on the RTC Tamper and Timestamp associated EXTI line.

Return value:

- None.

RTCEX Tamper TimeStamp On Tamper Detection Definitions

RTC_TIMESTAMPONTAMPERDETECTION_ENABLE

TimeStamp on Tamper Detection event saved

RTC_TIMESTAMPONTAMPERDETECTION_DISABLE

TimeStamp on Tamper Detection event is not saved

RTC_TIMESTAMPONTAMPERDETECTION_MASK

Masking all bits except bit TAMPTS

RTCEX Tamper Triggers Definitions

RTC_TAMPERTRIGGER_RISINGEDGE

RTC_TAMPERTRIGGER_FALLINGEDGE

RTC_TAMPERTRIGGER_LOWLEVEL

RTC_TAMPERTRIGGER_HIGHLEVEL

RTCEX Timestamp

__HAL_RTC_TIMESTAMP_ENABLE

Description:

- Enable the RTC Timestamp peripheral.

Parameters:

- `__HANDLE__`: specifies the RTC handle.

Return value:

- None

__HAL_RTC_TIMESTAMP_DISABLE

Description:

- Disable the RTC Timestamp peripheral.

Parameters:

- `__HANDLE__`: specifies the RTC handle.

Return value:

- None

__HAL_RTC_TIMESTAMP_ENABLE_IT

Description:

- Enable the RTC Timestamp interrupt.

Parameters:

- `__HANDLE__`: specifies the RTC handle.
- `__INTERRUPT__`: specifies the RTC Timestamp interrupt sources to be enabled or disabled. This parameter can be:
 - `RTC_IT_TS`: TimeStamp interrupt

Return value:

- None

__HAL_RTC_TIMESTAMP_DISABLE_IT

Description:

- Disable the RTC Timestamp interrupt.

Parameters:

- `__HANDLE__`: specifies the RTC handle.
- `__INTERRUPT__`: specifies the RTC Timestamp interrupt sources to be enabled or disabled. This parameter can be:
 - `RTC_IT_TS`: TimeStamp interrupt

Return value:

- None

__HAL_RTC_TIMESTAMP_GET_IT

Description:

- Check whether the specified RTC Timestamp interrupt has occurred or not.

Parameters:

- `__HANDLE__`: specifies the RTC handle.
- `__INTERRUPT__`: specifies the RTC Timestamp interrupt to check. This parameter can be:
 - `RTC_IT_TS`: TimeStamp interrupt

Return value:

- None

__HAL_RTC_TIMESTAMP_GET_IT_SOURCE

Description:

- Check whether the specified RTC Timestamp interrupt has been enabled or not.

Parameters:

- `__HANDLE__`: specifies the RTC handle.
- `__INTERRUPT__`: specifies the RTC Timestamp interrupt source to check. This parameter can be:
 - `RTC_IT_TS`: TimeStamp interrupt

Return value:

- None

__HAL_RTC_TIMESTAMP_GET_FLAG

Description:

- Get the selected RTC Timestamp's flag status.

Parameters:

- __HANDLE__: specifies the RTC handle.
- __FLAG__: specifies the RTC Timestamp flag to check. This parameter can be:
 - RTC_FLAG_TSF: Timestamp interrupt flag
 - RTC_FLAG_TSOVF: Timestamp overflow flag

Return value:

- None

__HAL_RTC_TIMESTAMP_CLEAR_FLAG

Description:

- Clear the RTC Timestamp's pending flags.

Parameters:

- __HANDLE__: specifies the RTC handle.
- __FLAG__: specifies the RTC Timestamp flag to clear. This parameter can be:
 - RTC_FLAG_TSF: Timestamp interrupt flag
 - RTC_FLAG_TSOVF: Timestamp overflow flag

Return value:

- None

RTCEX Timestamp Edges Definitions

RTC_TIMESTAMPEDGE_RISING

RTC_TIMESTAMPEDGE_FALLING

RTC Timestamp Pin Selection

RTC_TIMESTAMPPIN_DEFAULT

RTC_TIMESTAMPPIN_POS1

RTCEX WakeUp Timer

__HAL_RTC_WAKEUPTIMER_ENABLE

Description:

- Enable the RTC WakeUp Timer peripheral.

Parameters:

- __HANDLE__: specifies the RTC handle.

Return value:

- None

__HAL_RTC_WAKEUPTIMER_DISABLE

Description:

- Disable the RTC Wakeup Timer peripheral.

Parameters:

- __HANDLE__: specifies the RTC handle.

Return value:

- None

__HAL_RTC_WAKEUPTIMER_ENABLE_IT

Description:

- Enable the RTC Wakeup Timer interrupt.

Parameters:

- `__HANDLE__`: specifies the RTC handle.
- `__INTERRUPT__`: specifies the RTC Wakeup Timer interrupt sources to be enabled or disabled. This parameter can be:
 - `RTC_IT_WUT`: Wakeup Timer interrupt

Return value:

- None

__HAL_RTC_WAKEUPTIMER_DISABLE_IT

Description:

- Disable the RTC Wakeup Timer interrupt.

Parameters:

- `__HANDLE__`: specifies the RTC handle.
- `__INTERRUPT__`: specifies the RTC Wakeup Timer interrupt sources to be enabled or disabled. This parameter can be:
 - `RTC_IT_WUT`: Wakeup Timer interrupt

Return value:

- None

__HAL_RTC_WAKEUPTIMER_GET_IT

Description:

- Check whether the specified RTC Wakeup Timer interrupt has occurred or not.

Parameters:

- `__HANDLE__`: specifies the RTC handle.
- `__INTERRUPT__`: specifies the RTC Wakeup Timer interrupt to check. This parameter can be:
 - `RTC_IT_WUT`: Wakeup Timer interrupt

Return value:

- None

__HAL_RTC_WAKEUPTIMER_GET_IT_SOURCE

Description:

- Check whether the specified RTC Wakeup timer interrupt has been enabled or not.

Parameters:

- `__HANDLE__`: specifies the RTC handle.
- `__INTERRUPT__`: specifies the RTC Wakeup timer interrupt sources to check. This parameter can be:
 - `RTC_IT_WUT`: WakeUpTimer interrupt

Return value:

- None

__HAL_RTC_WAKEUPTIMER_GET_FLAG

Description:

- Get the selected RTC Wakeup Timer's flag status.

Parameters:

- `__HANDLE__`: specifies the RTC handle.
- `__FLAG__`: specifies the RTC Wakeup Timer flag to check. This parameter can be:
 - `RTC_FLAG_WUTF`: Wakeup Timer interrupt flag
 - `RTC_FLAG_WUTWF`: Wakeup Timer 'write allowed' flag

Return value:

- None

__HAL_RTC_WAKEUPTIMER_CLEAR_FLAG

Description:

- Clear the RTC Wakeup timer's pending flags.

Parameters:

- `__HANDLE__`: specifies the RTC handle.
- `__FLAG__`: specifies the RTC Wakeup Timer Flag to clear. This parameter can be:
 - `RTC_FLAG_WUTF`: Wakeup Timer interrupt Flag

Return value:

- None

__HAL_RTC_WAKEUPTIMER_EXTI_ENABLE_IT

Description:

- Enable interrupt on the RTC Wakeup Timer associated EXTI line.

Return value:

- None

__HAL_RTC_WAKEUPTIMER_EXTI_DISABLE_IT

Description:

- Disable interrupt on the RTC Wakeup Timer associated EXTI line.

Return value:

- None

__HAL_RTC_WAKEUPTIMER_EXTI_ENABLE_EVENT

Description:

- Enable event on the RTC Wakeup Timer associated EXTI line.

Return value:

- None.

__HAL_RTC_WAKEUPTIMER_EXTI_DISABLE_EVENT

Description:

- Disable event on the RTC Wakeup Timer associated EXTI line.

Return value:

- None.

__HAL_RTC_WAKEUPTIMER_EXTI_ENABLE_FALLING_EDGE

Description:

- Enable falling edge trigger on the RTC Wakeup Timer associated EXTI line.

Return value:

- None.

__HAL_RTC_WAKEUPTIMER_EXTI_DISABLE_FALLING_EDGE

Description:

- Disable falling edge trigger on the RTC Wakeup Timer associated EXTI line.

Return value:

- None.

__HAL_RTC_WAKEUPTIMER_EXTI_ENABLE_RISING_EDGE

Description:

- Enable rising edge trigger on the RTC Wakeup Timer associated EXTI line.

Return value:

- None.

__HAL_RTC_WAKEUPTIMER_EXTI_DISABLE_RISING_EDGE

Description:

- Disable rising edge trigger on the RTC Wakeup Timer associated EXTI line.

Return value:

- None.

__HAL_RTC_WAKEUPTIMER_EXTI_ENABLE_RISING_FALLING_EDGE

Description:

- Enable rising & falling edge trigger on the RTC Wakeup Timer associated EXTI line.

Return value:

- None.

__HAL_RTC_WAKEUPTIMER_EXTI_DISABLE_RISING_FALLING_EDGE

Description:

- Disable rising & falling edge trigger on the RTC Wakeup Timer associated EXTI line.

Return value:

- None.

__HAL_RTC_WAKEUPTIMER_EXTI_GET_FLAG

Description:

- Check whether the RTC Wakeup Timer associated EXTI line interrupt flag is set or not.

Return value:

- Line: Status.

__HAL_RTC_WAKEUPTIMER_EXTI_CLEAR_FLAG

Description:

- Clear the RTC Wakeup Timer associated EXTI line flag.

Return value:

- None.

__HAL_RTC_WAKEUPTIMER_EXTI_GENERATE_SWIT

Description:

- Generate a Software interrupt on the RTC Wakeup Timer associated EXTI line.

Return value:

- None.

RTCEX Wakeup Timer Definitions

RTC_WAKEUPCLOCK_RTCCLK_DIV16

RTC_WAKEUPCLOCK_RTCCLK_DIV8

RTC_WAKEUPCLOCK_RTCCLK_DIV4

RTC_WAKEUPCLOCK_RTCCLK_DIV2

RTC_WAKEUPCLOCK_CK_SPRE_16BITS

RTC_WAKEUPCLOCK_CK_SPRE_17BITS

63 HAL SMARTCARD Generic Driver

63.1 SMARTCARD Firmware driver registers structures

63.1.1 SMARTCARD_InitTypeDef

SMARTCARD_InitTypeDef is defined in the `stm32f4xx_hal_smartcard.h`

Data Fields

- **uint32_t BaudRate**
- **uint32_t WordLength**
- **uint32_t StopBits**
- **uint32_t Parity**
- **uint32_t Mode**
- **uint32_t CLKPolarity**
- **uint32_t CLKPhase**
- **uint32_t CLKLastBit**
- **uint32_t Prescaler**
- **uint32_t GuardTime**
- **uint32_t NACKState**

Field Documentation

- **uint32_t SMARTCARD_InitTypeDef::BaudRate**
This member configures the SmartCard communication baud rate. The baud rate is computed using the following formula:
 - $\text{IntegerDivider} = ((\text{PCLKx}) / (16 * (\text{hsc->Init.BaudRate})))$
 - $\text{FractionalDivider} = ((\text{IntegerDivider} - ((\text{uint32_t}) \text{IntegerDivider})) * 16) + 0.5$
- **uint32_t SMARTCARD_InitTypeDef::WordLength**
Specifies the number of data bits transmitted or received in a frame. This parameter can be a value of [SMARTCARD_Word_Length](#)
- **uint32_t SMARTCARD_InitTypeDef::StopBits**
Specifies the number of stop bits transmitted. This parameter can be a value of [SMARTCARD_Stop_Bits](#)
- **uint32_t SMARTCARD_InitTypeDef::Parity**
Specifies the parity mode. This parameter can be a value of [SMARTCARD_Parity](#)
Note:
 - When parity is enabled, the computed parity is inserted at the MSB position of the transmitted data (9th bit when the word length is set to 9 data bits; 8th bit when the word length is set to 8 data bits).
- **uint32_t SMARTCARD_InitTypeDef::Mode**
Specifies whether the Receive or Transmit mode is enabled or disabled. This parameter can be a value of [SMARTCARD_Mode](#)
- **uint32_t SMARTCARD_InitTypeDef::CLKPolarity**
Specifies the steady state of the serial clock. This parameter can be a value of [SMARTCARD_Clock_Polarity](#)
- **uint32_t SMARTCARD_InitTypeDef::CLKPhase**
Specifies the clock transition on which the bit capture is made. This parameter can be a value of [SMARTCARD_Clock_Phase](#)
- **uint32_t SMARTCARD_InitTypeDef::CLKLastBit**
Specifies whether the clock pulse corresponding to the last transmitted data bit (MSB) has to be output on the SCLK pin in synchronous mode. This parameter can be a value of [SMARTCARD_Last_Bit](#)
- **uint32_t SMARTCARD_InitTypeDef::Prescaler**
Specifies the SmartCard Prescaler value used for dividing the system clock to provide the smartcard clock. The value given in the register (5 significant bits) is multiplied by 2 to give the division factor of the source clock frequency. This parameter can be a value of [SMARTCARD_Prescaler](#)
- **uint32_t SMARTCARD_InitTypeDef::GuardTime**
Specifies the SmartCard Guard Time value in terms of number of baud clocks

- **`uint32_t SMARTCARD_InitTypeDef::NACKState`**
Specifies the SmartCard NACK Transmission state. This parameter can be a value of **`SMARTCARD_NACK_State`**

63.1.2 **`__SMARTCARD_HandleTypeDef`**

`__SMARTCARD_HandleTypeDef` is defined in the `stm32f4xx_hal_smartcard.h`

Data Fields

- **`USART_TypeDef * Instance`**
- **`SMARTCARD_InitTypeDef Init`**
- **`const uint8_t * pTxBuffPtr`**
- **`uint16_t TxXferSize`**
- **`__IO uint16_t TxXferCount`**
- **`uint8_t * pRxBuffPtr`**
- **`uint16_t RxXferSize`**
- **`__IO uint16_t RxXferCount`**
- **`DMA_HandleTypeDef * hdmatx`**
- **`DMA_HandleTypeDef * hdmarx`**
- **`HAL_LockTypeDef Lock`**
- **`__IO HAL_SMARTCARD_StateTypeDef gState`**
- **`__IO HAL_SMARTCARD_StateTypeDef RxState`**
- **`__IO uint32_t ErrorCode`**
- **`void(* TxCpltCallback`**
- **`void(* RxCpltCallback`**
- **`void(* ErrorCallback`**
- **`void(* AbortCpltCallback`**
- **`void(* AbortTransmitCpltCallback`**
- **`void(* AbortReceiveCpltCallback`**
- **`void(* MspInitCallback`**
- **`void(* MspDeInitCallback`**

Field Documentation

- **`USART_TypeDef* __SMARTCARD_HandleTypeDef::Instance`**
USART registers base address
- **`SMARTCARD_InitTypeDef __SMARTCARD_HandleTypeDef::Init`**
SmartCard communication parameters
- **`const uint8_t* __SMARTCARD_HandleTypeDef::pTxBuffPtr`**
Pointer to SmartCard Tx transfer Buffer
- **`uint16_t __SMARTCARD_HandleTypeDef::TxXferSize`**
SmartCard Tx Transfer size
- **`__IO uint16_t __SMARTCARD_HandleTypeDef::TxXferCount`**
SmartCard Tx Transfer Counter
- **`uint8_t* __SMARTCARD_HandleTypeDef::pRxBuffPtr`**
Pointer to SmartCard Rx transfer Buffer
- **`uint16_t __SMARTCARD_HandleTypeDef::RxXferSize`**
SmartCard Rx Transfer size
- **`__IO uint16_t __SMARTCARD_HandleTypeDef::RxXferCount`**
SmartCard Rx Transfer Counter
- **`DMA_HandleTypeDef* __SMARTCARD_HandleTypeDef::hdmatx`**
SmartCard Tx DMA Handle parameters
- **`DMA_HandleTypeDef* __SMARTCARD_HandleTypeDef::hdmarx`**
SmartCard Rx DMA Handle parameters
- **`HAL_LockTypeDef __SMARTCARD_HandleTypeDef::Lock`**
Locking object

- **__IO HAL_SMARTCARD_StateTypeDef __SMARTCARD_HandleTypeDef::gState**
SmartCard state information related to global Handle management and also related to Tx operations. This parameter can be a value of **HAL_SMARTCARD_StateTypeDef**
- **__IO HAL_SMARTCARD_StateTypeDef __SMARTCARD_HandleTypeDef::RxState**
SmartCard state information related to Rx operations. This parameter can be a value of **HAL_SMARTCARD_StateTypeDef**
- **__IO uint32_t __SMARTCARD_HandleTypeDef::ErrorCode**
SmartCard Error code
- **void(* __SMARTCARD_HandleTypeDef::TxCpltCallback)(struct __SMARTCARD_HandleTypeDef *hsc)**
SMARTCARD Tx Complete Callback
- **void(* __SMARTCARD_HandleTypeDef::RxCpltCallback)(struct __SMARTCARD_HandleTypeDef *hsc)**
SMARTCARD Rx Complete Callback
- **void(* __SMARTCARD_HandleTypeDef::ErrorCallback)(struct __SMARTCARD_HandleTypeDef *hsc)**
SMARTCARD Error Callback
- **void(* __SMARTCARD_HandleTypeDef::AbortCpltCallback)(struct __SMARTCARD_HandleTypeDef *hsc)**
SMARTCARD Abort Complete Callback
- **void(* __SMARTCARD_HandleTypeDef::AbortTransmitCpltCallback)(struct __SMARTCARD_HandleTypeDef *hsc)**
SMARTCARD Abort Transmit Complete Callback
- **void(* __SMARTCARD_HandleTypeDef::AbortReceiveCpltCallback)(struct __SMARTCARD_HandleTypeDef *hsc)**
SMARTCARD Abort Receive Complete Callback
- **void(* __SMARTCARD_HandleTypeDef::MspInitCallback)(struct __SMARTCARD_HandleTypeDef *hsc)**
SMARTCARD Msp Init callback
- **void(* __SMARTCARD_HandleTypeDef::MspDeInitCallback)(struct __SMARTCARD_HandleTypeDef *hsc)**
SMARTCARD Msp DeInit callback

63.2 SMARTCARD Firmware driver API description

The following section lists the various functions of the SMARTCARD library.

63.2.1 How to use this driver

The SMARTCARD HAL driver can be used as follows:

1. Declare a **SMARTCARD_HandleTypeDef** handle structure.

2. Initialize the SMARTCARD low level resources by implementing the HAL_SMARTCARD_MspInit() API:
 - a. Enable the interface clock of the USARTx associated to the SMARTCARD.
 - b. SMARTCARD pins configuration:
 - Enable the clock for the SMARTCARD GPIOs.
 - Configure SMARTCARD pins as alternate function pull-up.
 - c. NVIC configuration if you need to use interrupt process (HAL_SMARTCARD_Transmit_IT() and HAL_SMARTCARD_Receive_IT() APIs):
 - Configure the USARTx interrupt priority.
 - Enable the NVIC USART IRQ handle.
 - d. DMA Configuration if you need to use DMA process (HAL_SMARTCARD_Transmit_DMA() and HAL_SMARTCARD_Receive_DMA() APIs):
 - Declare a DMA handle structure for the Tx/Rx stream.
 - Enable the DMAx interface clock.
 - Configure the declared DMA handle structure with the required Tx/Rx parameters.
 - Configure the DMA Tx/Rx stream.
 - Associate the initialized DMA handle to the SMARTCARD DMA Tx/Rx handle.
 - Configure the priority and enable the NVIC for the transfer complete interrupt on the DMA Tx/Rx stream.
 - Configure the USARTx interrupt priority and enable the NVIC USART IRQ handle (used for last byte sending completion detection in DMA non circular mode)
3. Program the Baud Rate, Word Length, Stop Bit, Parity, Hardware flow control and Mode(Receiver/Transmitter) in the SMARTCARD Init structure.
4. Initialize the SMARTCARD registers by calling the HAL_SMARTCARD_Init() API:
 - These APIs configure also the low level Hardware GPIO, CLOCK, CORTEX...etc) by calling the customized HAL_SMARTCARD_MspInit() API.

Note: *The specific SMARTCARD interrupts (Transmission complete interrupt, RXNE interrupt and Error Interrupts) will be managed using the macros __HAL_SMARTCARD_ENABLE_IT() and __HAL_SMARTCARD_DISABLE_IT() inside the transmit and receive process.*

Three operation modes are available within this driver :

Polling mode IO operation

- Send an amount of data in blocking mode using HAL_SMARTCARD_Transmit()
- Receive an amount of data in blocking mode using HAL_SMARTCARD_Receive()

Interrupt mode IO operation

- Send an amount of data in non blocking mode using HAL_SMARTCARD_Transmit_IT()
- At transmission end of transfer HAL_SMARTCARD_TxCpltCallback is executed and user can add his own code by customization of function pointer HAL_SMARTCARD_TxCpltCallback
- Receive an amount of data in non blocking mode using HAL_SMARTCARD_Receive_IT()
- At reception end of transfer HAL_SMARTCARD_RxCpltCallback is executed and user can add his own code by customization of function pointer HAL_SMARTCARD_RxCpltCallback
- In case of transfer Error, HAL_SMARTCARD_ErrorCallback() function is executed and user can add his own code by customization of function pointer HAL_SMARTCARD_ErrorCallback

DMA mode IO operation

- Send an amount of data in non blocking mode (DMA) using HAL_SMARTCARD_Transmit_DMA()
- At transmission end of transfer HAL_SMARTCARD_TxCpltCallback is executed and user can add his own code by customization of function pointer HAL_SMARTCARD_TxCpltCallback
- Receive an amount of data in non blocking mode (DMA) using HAL_SMARTCARD_Receive_DMA()
- At reception end of transfer HAL_SMARTCARD_RxCpltCallback is executed and user can add his own code by customization of function pointer HAL_SMARTCARD_RxCpltCallback

- In case of transfer Error, HAL_SMARTCARD_ErrorCallback() function is executed and user can add his own code by customization of function pointer HAL_SMARTCARD_ErrorCallback

SMARTCARD HAL driver macros list

Below the list of most used macros in SMARTCARD HAL driver.

- __HAL_SMARTCARD_ENABLE: Enable the SMARTCARD peripheral
- __HAL_SMARTCARD_DISABLE: Disable the SMARTCARD peripheral
- __HAL_SMARTCARD_GET_FLAG : Check whether the specified SMARTCARD flag is set or not
- __HAL_SMARTCARD_CLEAR_FLAG : Clear the specified SMARTCARD pending flag
- __HAL_SMARTCARD_ENABLE_IT: Enable the specified SMARTCARD interrupt
- __HAL_SMARTCARD_DISABLE_IT: Disable the specified SMARTCARD interrupt

Note: You can refer to the SMARTCARD HAL driver header file for more useful macros

63.2.2 Callback registration

The compilation define USE_HAL_SMARTCARD_REGISTER_CALLBACKS when set to 1 allows the user to configure dynamically the driver callbacks.

Use Function HAL_SMARTCARD_RegisterCallback() to register a user callback. Function HAL_SMARTCARD_RegisterCallback() allows to register following callbacks:

- TxCpltCallback : Tx Complete Callback.
- RxCpltCallback : Rx Complete Callback.
- ErrorCallback : Error Callback.
- AbortCpltCallback : Abort Complete Callback.
- AbortTransmitCpltCallback : Abort Transmit Complete Callback.
- AbortReceiveCpltCallback : Abort Receive Complete Callback.
- MspInitCallback : SMARTCARD MspInit.
- MspDeInitCallback : SMARTCARD MspDeInit. This function takes as parameters the HAL peripheral handle, the Callback ID and a pointer to the user callback function.

Use function HAL_SMARTCARD_UnRegisterCallback() to reset a callback to the default weak (surcharged) function. HAL_SMARTCARD_UnRegisterCallback() takes as parameters the HAL peripheral handle, and the Callback ID. This function allows to reset following callbacks:

- TxCpltCallback : Tx Complete Callback.
- RxCpltCallback : Rx Complete Callback.
- ErrorCallback : Error Callback.
- AbortCpltCallback : Abort Complete Callback.
- AbortTransmitCpltCallback : Abort Transmit Complete Callback.
- AbortReceiveCpltCallback : Abort Receive Complete Callback.
- MspInitCallback : SMARTCARD MspInit.
- MspDeInitCallback : SMARTCARD MspDeInit.

By default, after the HAL_SMARTCARD_Init() and when the state is HAL_SMARTCARD_STATE_RESET all callbacks are set to the corresponding weak (surcharged) functions: examples HAL_SMARTCARD_TxCpltCallback(), HAL_SMARTCARD_RxCpltCallback(). Exception done for MspInit and MspDeInit functions that are respectively reset to the legacy weak (surcharged) functions in the HAL_SMARTCARD_Init() and HAL_SMARTCARD_DeInit() only when these callbacks are null (not registered beforehand). If not, MspInit or MspDeInit are not null, the HAL_SMARTCARD_Init() and HAL_SMARTCARD_DeInit() keep and use the user MspInit/MspDeInit callbacks (registered beforehand).

Callbacks can be registered/unregistered in HAL_SMARTCARD_STATE_READY state only. Exception done MspInit/MspDeInit that can be registered/unregistered in HAL_SMARTCARD_STATE_READY or HAL_SMARTCARD_STATE_RESET state, thus registered (user) MspInit/DeInit callbacks can be used during the Init/DeInit. In that case first register the MspInit/MspDeInit user callbacks using HAL_SMARTCARD_RegisterCallback() before calling HAL_SMARTCARD_DeInit() or HAL_SMARTCARD_Init() function.

When The compilation define USE_HAL_SMARTCARD_REGISTER_CALLBACKS is set to 0 or not defined, the callback registration feature is not available and weak (surcharged) callbacks are used.

63.2.3 Initialization and Configuration functions

This subsection provides a set of functions allowing to initialize the USART in Smartcard mode.

The Smartcard interface is designed to support asynchronous protocol Smartcards as defined in the ISO 7816-3 standard.

The USART can provide a clock to the smartcard through the SCLK output. In smartcard mode, SCLK is not associated to the communication but is simply derived from the internal peripheral input clock through a 5-bit prescaler.

- For the Smartcard mode only these parameters can be configured:
 - Baud Rate
 - Word Length => Should be 9 bits (8 bits + parity)
 - Stop Bit
 - Parity: => Should be enabled
 - USART polarity
 - USART phase
 - USART LastBit
 - Receiver/transmitter modes
 - Prescaler
 - GuardTime
 - NACKState: The Smartcard NACK state
- Recommended SmartCard interface configuration to get the Answer to Reset from the Card:
 - Word Length = 9 Bits
 - 1.5 Stop Bit
 - Even parity
 - BaudRate = 12096 baud
 - Tx and Rx enabled

Please refer to the ISO 7816-3 specification for more details.

Note: It is also possible to choose 0.5 stop bit for receiving but it is recommended to use 1.5 stop bits for both transmitting and receiving to avoid switching between the two configurations.

The HAL_SMARTCARD_Init() function follows the USART SmartCard configuration procedures (details for the procedures are available in reference manual (RM0430 for STM32F4X3xx MCUs and RM0402 for STM32F412xx MCUs RM0383 for STM32F411xC/E MCUs and RM0401 for STM32F410xx MCUs RM0090 for STM32F4X5xx/STM32F4X7xx/STM32F429xx/STM32F439xx MCUs RM0390 for STM32F446xx MCUs and RM0386 for STM32F469xx/STM32F479xx MCUs)).

This section contains the following APIs:

- [**HAL_SMARTCARD_Init\(\)**](#)
- [**HAL_SMARTCARD_DeInit\(\)**](#)
- [**HAL_SMARTCARD_MspInit\(\)**](#)
- [**HAL_SMARTCARD_MspDeInit\(\)**](#)
- [**HAL_SMARTCARD_RegisterCallback\(\)**](#)
- [**HAL_SMARTCARD_UnRegisterCallback\(\)**](#)
- [**HAL_SMARTCARD_Relnit\(\)**](#)

63.2.4 IO operation functions

This subsection provides a set of functions allowing to manage the SMARTCARD data transfers.

1. Smartcard is a single wire half duplex communication protocol. The Smartcard interface is designed to support asynchronous protocol Smartcards as defined in the ISO 7816-3 standard.
2. The USART should be configured as:
 - 8 bits plus parity: where M=1 and PCE=1 in the USART_CR1 register
 - 1.5 stop bits when transmitting and receiving: where STOP=11 in the USART_CR2 register.

3. There are two modes of transfer:
 - Blocking mode: The communication is performed in polling mode. The HAL status of all data processing is returned by the same function after finishing transfer.
 - Non Blocking mode: The communication is performed using Interrupts or DMA, These APIs return the HAL status. The end of the data processing will be indicated through the dedicated SMARTCARD IRQ when using Interrupt mode or the DMA IRQ when using DMA mode. The HAL_SMARTCARD_TxCpltCallback(), HAL_SMARTCARD_RxCpltCallback() user callbacks will be executed respectively at the end of the Transmit or Receive process The HAL_SMARTCARD_ErrorCallback() user callback will be executed when a communication error is detected
4. Blocking mode APIs are :
 - HAL_SMARTCARD_Transmit()
 - HAL_SMARTCARD_Receive()
5. Non Blocking mode APIs with Interrupt are :
 - HAL_SMARTCARD_Transmit_IT()
 - HAL_SMARTCARD_Receive_IT()
 - HAL_SMARTCARD_IRQHandler()
6. Non Blocking mode functions with DMA are :
 - HAL_SMARTCARD_Transmit_DMA()
 - HAL_SMARTCARD_Receive_DMA()
7. A set of Transfer Complete Callbacks are provided in non Blocking mode:
 - HAL_SMARTCARD_TxCpltCallback()
 - HAL_SMARTCARD_RxCpltCallback()
 - HAL_SMARTCARD_ErrorCallback()
8. Non-Blocking mode transfers could be aborted using Abort API's : (+) HAL_SMARTCARD_Abort() (+) HAL_SMARTCARD_AbortTransmit() (+) HAL_SMARTCARD_AbortReceive() (+) HAL_SMARTCARD_Abort_IT() (+) HAL_SMARTCARD_AbortTransmit_IT() (+) HAL_SMARTCARD_AbortReceive_IT()
9. For Abort services based on interrupts (HAL_SMARTCARD_Abortxxx_IT), a set of Abort Complete Callbacks are provided: (+) HAL_SMARTCARD_AbortCpltCallback() (+) HAL_SMARTCARD_AbortTransmitCpltCallback() (+) HAL_SMARTCARD_AbortReceiveCpltCallback()
10. In Non-Blocking mode transfers, possible errors are split into 2 categories. Errors are handled as follows :
 - (+) Error is considered as Recoverable and non blocking : Transfer could go till end, but error severity is to be evaluated by user : this concerns Frame Error, Parity Error or Noise Error in Interrupt mode reception . Received character is then retrieved and stored in Rx buffer, Error code is set to allow user to identify error type, and HAL_SMARTCARD_ErrorCallback() user callback is executed. Transfer is kept ongoing on SMARTCARD side. If user wants to abort it, Abort services should be called by user. (+) Error is considered as Blocking : Transfer could not be completed properly and is aborted. This concerns Frame Error in Interrupt mode transmission, Overrun Error in Interrupt mode reception and all errors in DMA mode. Error code is set to allow user to identify error type, and HAL_SMARTCARD_ErrorCallback() user callback is executed.

(#) Smartcard is a single wire half duplex communication protocol. The Smartcard interface is designed to support asynchronous protocol Smartcards as defined in the ISO 7816-3 standard. (#) The USART should be configured as: (++) 8 bits plus parity: where M=1 and PCE=1 in the USART_CR1 register (++) 1.5 stop bits when transmitting and receiving: where STOP=11 in the USART_CR2 register. (#) There are two modes of transfer: (++) Blocking mode: The communication is performed in polling mode. The HAL status of all data processing is returned by the same function after finishing transfer. (++) Non Blocking mode: The communication is performed using Interrupts or DMA, These APIs return the HAL status. The end of the data processing will be indicated through the dedicated SMARTCARD IRQ when using Interrupt mode or the DMA IRQ when using DMA mode. The HAL_SMARTCARD_TxCpltCallback(), HAL_SMARTCARD_RxCpltCallback() user callbacks will be executed respectively at the end of the Transmit or Receive process The HAL_SMARTCARD_ErrorCallback() user callback will be executed when a communication error is detected (#) Blocking mode APIs are : (++) HAL_SMARTCARD_Transmit() (++) HAL_SMARTCARD_Receive() (#) Non Blocking mode APIs with Interrupt are : (++) HAL_SMARTCARD_Transmit_IT() (++) HAL_SMARTCARD_Receive_IT() (++) HAL_SMARTCARD_IRQHandler() (#) Non Blocking mode functions with DMA are : (++) HAL_SMARTCARD_Transmit_DMA() (++) HAL_SMARTCARD_Receive_DMA() (#) A set of Transfer Complete Callbacks are provided in non Blocking mode: (++) HAL_SMARTCARD_TxCpltCallback() (++) HAL_SMARTCARD_RxCpltCallback() (++) HAL_SMARTCARD_ErrorCallback() (#) Non-Blocking mode transfers could be aborted using Abort API's :

- HAL_SMARTCARD_Abort()
- HAL_SMARTCARD_AbortTransmit()
- HAL_SMARTCARD_AbortReceive()
- HAL_SMARTCARD_Abort_IT()
- HAL_SMARTCARD_AbortTransmit_IT()
- HAL_SMARTCARD_AbortReceive_IT() (#) For Abort services based on interrupts (HAL_SMARTCARD_Abortxxx_IT), a set of Abort Complete Callbacks are provided:
- HAL_SMARTCARD_AbortCpltCallback()
- HAL_SMARTCARD_AbortTransmitCpltCallback()
- HAL_SMARTCARD_AbortReceiveCpltCallback() (#) In Non-Blocking mode transfers, possible errors are split into 2 categories. Errors are handled as follows :
- Error is considered as Recoverable and non blocking : Transfer could go till end, but error severity is to be evaluated by user : this concerns Frame Error, Parity Error or Noise Error in Interrupt mode reception . Received character is then retrieved and stored in Rx buffer, Error code is set to allow user to identify error type, and HAL_SMARTCARD_ErrorCallback() user callback is executed. Transfer is kept ongoing on SMARTCARD side. If user wants to abort it, Abort services should be called by user.
- Error is considered as Blocking : Transfer could not be completed properly and is aborted. This concerns Frame Error in Interrupt mode transmission, Overrun Error in Interrupt mode reception and all errors in DMA mode. Error code is set to allow user to identify error type, and HAL_SMARTCARD_ErrorCallback() user callback is executed.

This section contains the following APIs:

- [HAL_SMARTCARD_Transmit\(\)](#)
- [HAL_SMARTCARD_Receive\(\)](#)
- [HAL_SMARTCARD_Transmit_IT\(\)](#)
- [HAL_SMARTCARD_Receive_IT\(\)](#)
- [HAL_SMARTCARD_Transmit_DMA\(\)](#)
- [HAL_SMARTCARD_Receive_DMA\(\)](#)
- [HAL_SMARTCARD_Abort\(\)](#)
- [HAL_SMARTCARD_AbortTransmit\(\)](#)
- [HAL_SMARTCARD_AbortReceive\(\)](#)
- [HAL_SMARTCARD_Abort_IT\(\)](#)
- [HAL_SMARTCARD_AbortTransmit_IT\(\)](#)
- [HAL_SMARTCARD_AbortReceive_IT\(\)](#)
- [HAL_SMARTCARD_IRQHandler\(\)](#)
- [HAL_SMARTCARD_TxCpltCallback\(\)](#)
- [HAL_SMARTCARD_RxCpltCallback\(\)](#)
- [HAL_SMARTCARD_ErrorCallback\(\)](#)

- [*HAL_SMARTCARD_AbortCpltCallback\(\)*](#)
- [*HAL_SMARTCARD_AbortTransmitCpltCallback\(\)*](#)
- [*HAL_SMARTCARD_AbortReceiveCpltCallback\(\)*](#)

63.2.5 Peripheral State and Errors functions

This subsection provides a set of functions allowing to control the SmartCard.

- [*HAL_SMARTCARD_GetState\(\)*](#) API can be helpful to check in run-time the state of the SmartCard peripheral.
- [*HAL_SMARTCARD_GetError\(\)*](#) check in run-time errors that could be occurred during communication.

This section contains the following APIs:

- [*HAL_SMARTCARD_GetState\(\)*](#)
- [*HAL_SMARTCARD_GetError\(\)*](#)

63.2.6 Detailed description of functions

HAL_SMARTCARD_Init

Function name

HAL_StatusTypeDef HAL_SMARTCARD_Init (SMARTCARD_HandleTypeDef * hsc)

Function description

Initializes the SmartCard mode according to the specified parameters in the SMARTCARD_InitTypeDef and create the associated handle.

Parameters

- **hsc:** Pointer to a SMARTCARD_HandleTypeDef structure that contains the configuration information for SMARTCARD module.

Return values

- **HAL:** status

HAL_SMARTCARD_Relnit

Function name

HAL_StatusTypeDef HAL_SMARTCARD_Relnit (SMARTCARD_HandleTypeDef * hsc)

Function description

HAL_SMARTCARD_DeInit

Function name

HAL_StatusTypeDef HAL_SMARTCARD_DeInit (SMARTCARD_HandleTypeDef * hsc)

Function description

DeInitializes the USART SmartCard peripheral.

Parameters

- **hsc:** Pointer to a SMARTCARD_HandleTypeDef structure that contains the configuration information for SMARTCARD module.

Return values

- **HAL:** status

HAL_SMARTCARD_MspInit

Function name

void HAL_SMARTCARD_MspInit (SMARTCARD_HandleTypeDef * hsc)

Function description

SMARTCARD MSP Init.

Parameters

- **hsc:** Pointer to a SMARTCARD_HandleTypeDef structure that contains the configuration information for SMARTCARD module.

Return values

- **None:**

HAL_SMARTCARD_MspDeInit

Function name

void HAL_SMARTCARD_MspDeInit (SMARTCARD_HandleTypeDef * hsc)

Function description

SMARTCARD MSP DeInit.

Parameters

- **hsc:** Pointer to a SMARTCARD_HandleTypeDef structure that contains the configuration information for SMARTCARD module.

Return values

- **None:**

HAL_SMARTCARD_RegisterCallback

Function name

HAL_StatusTypeDef HAL_SMARTCARD_RegisterCallback (SMARTCARD_HandleTypeDef * hsc, HAL_SMARTCARD_CallbackIDTypeDef CallbackID, pSMARTCARD_CallbackTypeDef pCallback)

Function description

Register a User SMARTCARD Callback To be used instead of the weak predefined callback.

Parameters

- **hsc:** smartcard handle
- **CallbackID:** ID of the callback to be registered This parameter can be one of the following values:
 - HAL_SMARTCARD_TX_COMPLETE_CB_ID Tx Complete Callback ID
 - HAL_SMARTCARD_RX_COMPLETE_CB_ID Rx Complete Callback ID
 - HAL_SMARTCARD_ERROR_CB_ID Error Callback ID
 - HAL_SMARTCARD_ABORT_COMPLETE_CB_ID Abort Complete Callback ID
 - HAL_SMARTCARD_ABORT_TRANSMIT_COMPLETE_CB_ID Abort Transmit Complete Callback ID
 - HAL_SMARTCARD_ABORT_RECEIVE_COMPLETE_CB_ID Abort Receive Complete Callback ID
 - HAL_SMARTCARD_MSPINIT_CB_ID MspInit Callback ID
 - HAL_SMARTCARD_MSPDEINIT_CB_ID MspDeInit Callback ID
- **pCallback:** pointer to the Callback function

Return values

- **HAL:** status

Notes

- The HAL_SMARTCARD_RegisterCallback() may be called before HAL_SMARTCARD_Init() in HAL_SMARTCARD_STATE_RESET to register callbacks for HAL_SMARTCARD_MSPINIT_CB_ID and HAL_SMARTCARD_MSPDEINIT_CB_ID

HAL_SMARTCARD_UnRegisterCallback

Function name

HAL_StatusTypeDef HAL_SMARTCARD_UnRegisterCallback (SMARTCARD_HandleTypeDef * hsc, HAL_SMARTCARD_CallbackIDTypeDef CallbackID)

Function description

Unregister an SMARTCARD callback SMARTCARD callback is redirected to the weak predefined callback.

Parameters

- hsc:** smartcard handle
- CallbackID:** ID of the callback to be unregistered This parameter can be one of the following values:
 - HAL_SMARTCARD_TX_COMPLETE_CB_ID Tx Complete Callback ID
 - HAL_SMARTCARD_RX_COMPLETE_CB_ID Rx Complete Callback ID
 - HAL_SMARTCARD_ERROR_CB_ID Error Callback ID
 - HAL_SMARTCARD_ABORT_COMPLETE_CB_ID Abort Complete Callback ID
 - HAL_SMARTCARD_ABORT_TRANSMIT_COMPLETE_CB_ID Abort Transmit Complete Callback ID
 - HAL_SMARTCARD_ABORT_RECEIVE_COMPLETE_CB_ID Abort Receive Complete Callback ID
 - HAL_SMARTCARD_MSPINIT_CB_ID MspInit Callback ID
 - HAL_SMARTCARD_MSPDEINIT_CB_ID MspDeInit Callback ID

Return values

- HAL:** status

Notes

- The HAL_SMARTCARD_UnRegisterCallback() may be called before HAL_SMARTCARD_Init() in HAL_SMARTCARD_STATE_RESET to un-register callbacks for HAL_SMARTCARD_MSPINIT_CB_ID and HAL_SMARTCARD_MSPDEINIT_CB_ID

HAL_SMARTCARD_Transmit

Function name

HAL_StatusTypeDef HAL_SMARTCARD_Transmit (SMARTCARD_HandleTypeDef * hsc, const uint8_t * pData, uint16_t Size, uint32_t Timeout)

Function description

Send an amount of data in blocking mode.

Parameters

- hsc:** Pointer to a SMARTCARD_HandleTypeDef structure that contains the configuration information for SMARTCARD module.
- pData:** Pointer to data buffer
- Size:** Amount of data to be sent
- Timeout:** Timeout duration

Return values

- HAL:** status

HAL_SMARTCARD_Receive

Function name

HAL_StatusTypeDef HAL_SMARTCARD_Receive (SMARTCARD_HandleTypeDef * hsc, uint8_t * pData, uint16_t Size, uint32_t Timeout)

Function description

Receive an amount of data in blocking mode.

Parameters

- **hsc:** Pointer to a SMARTCARD_HandleTypeDef structure that contains the configuration information for SMARTCARD module.
- **pData:** Pointer to data buffer
- **Size:** Amount of data to be received
- **Timeout:** Timeout duration

Return values

- **HAL:** status

HAL_SMARTCARD_Transmit_IT

Function name

HAL_StatusTypeDef HAL_SMARTCARD_Transmit_IT (SMARTCARD_HandleTypeDef * hsc, const uint8_t * pData, uint16_t Size)

Function description

Send an amount of data in non blocking mode.

Parameters

- **hsc:** Pointer to a SMARTCARD_HandleTypeDef structure that contains the configuration information for SMARTCARD module.
- **pData:** Pointer to data buffer
- **Size:** Amount of data to be sent

Return values

- **HAL:** status

HAL_SMARTCARD_Receive_IT

Function name

HAL_StatusTypeDef HAL_SMARTCARD_Receive_IT (SMARTCARD_HandleTypeDef * hsc, uint8_t * pData, uint16_t Size)

Function description

Receive an amount of data in non blocking mode.

Parameters

- **hsc:** Pointer to a SMARTCARD_HandleTypeDef structure that contains the configuration information for SMARTCARD module.
- **pData:** Pointer to data buffer
- **Size:** Amount of data to be received

Return values

- **HAL:** status

HAL_SMARTCARD_Transmit_DMA

Function name

HAL_StatusTypeDef HAL_SMARTCARD_Transmit_DMA (SMARTCARD_HandleTypeDef * hsc, const uint8_t * pData, uint16_t Size)

Function description

Send an amount of data in non blocking mode.

Parameters

- **hsc**: Pointer to a SMARTCARD_HandleTypeDef structure that contains the configuration information for SMARTCARD module.
- **pData**: Pointer to data buffer
- **Size**: Amount of data to be sent

Return values

- **HAL**: status

HAL_SMARTCARD_Receive_DMA

Function name

HAL_StatusTypeDef HAL_SMARTCARD_Receive_DMA (SMARTCARD_HandleTypeDef * hsc, uint8_t * pData, uint16_t Size)

Function description

Receive an amount of data in non blocking mode.

Parameters

- **hsc**: Pointer to a SMARTCARD_HandleTypeDef structure that contains the configuration information for SMARTCARD module.
- **pData**: Pointer to data buffer
- **Size**: Amount of data to be received

Return values

- **HAL**: status

Notes

- When the SMARTCARD parity is enabled (PCE = 1) the data received contain the parity bit.s

HAL_SMARTCARD_Abort

Function name

HAL_StatusTypeDef HAL_SMARTCARD_Abort (SMARTCARD_HandleTypeDef * hsc)

Function description

Abort ongoing transfers (blocking mode).

Parameters

- **hsc**: SMARTCARD handle.

Return values

- **HAL**: status

Notes

- This procedure could be used for aborting any ongoing transfer started in Interrupt or DMA mode. This procedure performs following operations : Disable PPP InterruptsDisable the DMA transfer in the peripheral register (if enabled)Abort DMA transfer by calling HAL_DMA_Abort (in case of transfer in DMA mode)Set handle State to READY
- This procedure is executed in blocking mode : when exiting function, Abort is considered as completed.

HAL_SMARTCARD_AbortTransmit

Function name

HAL_StatusTypeDef HAL_SMARTCARD_AbortTransmit (SMARTCARD_HandleTypeDef * hsc)

Function description

Abort ongoing Transmit transfer (blocking mode).

Parameters

- **hsc:** SMARTCARD handle.

Return values

- **HAL:** status

Notes

- This procedure could be used for aborting any ongoing transfer started in Interrupt or DMA mode. This procedure performs following operations : Disable SMARTCARD Interrupts (Tx)Disable the DMA transfer in the peripheral register (if enabled)Abort DMA transfer by calling HAL_DMA_Abort (in case of transfer in DMA mode)Set handle State to READY
- This procedure is executed in blocking mode : when exiting function, Abort is considered as completed.

HAL_SMARTCARD_AbortReceive

Function name

HAL_StatusTypeDef HAL_SMARTCARD_AbortReceive (SMARTCARD_HandleTypeDef * hsc)

Function description

Abort ongoing Receive transfer (blocking mode).

Parameters

- **hsc:** SMARTCARD handle.

Return values

- **HAL:** status

Notes

- This procedure could be used for aborting any ongoing transfer started in Interrupt or DMA mode. This procedure performs following operations : Disable PPP InterruptsDisable the DMA transfer in the peripheral register (if enabled)Abort DMA transfer by calling HAL_DMA_Abort (in case of transfer in DMA mode)Set handle State to READY
- This procedure is executed in blocking mode : when exiting function, Abort is considered as completed.

HAL_SMARTCARD_Abort_IT

Function name

HAL_StatusTypeDef HAL_SMARTCARD_Abort_IT (SMARTCARD_HandleTypeDef * hsc)

Function description

Abort ongoing transfers (Interrupt mode).

Parameters

- **hsc:** SMARTCARD handle.

Return values

- **HAL:** status

Notes

- This procedure could be used for aborting any ongoing transfer started in Interrupt or DMA mode. This procedure performs following operations : Disable PPP Interrupts
Disable the DMA transfer in the peripheral register (if enabled)
Abort DMA transfer by calling HAL_DMA_Abort_IT (in case of transfer in DMA mode)
Set handle State to READY
At abort completion, call user abort complete callback
- This procedure is executed in Interrupt mode, meaning that abort procedure could be considered as completed only when user abort complete callback is executed (not when exiting function).

HAL_SMARTCARD_AbortTransmit_IT

Function name

HAL_StatusTypeDef HAL_SMARTCARD_AbortTransmit_IT (SMARTCARD_HandleTypeDef * hsc)

Function description

Abort ongoing Transmit transfer (Interrupt mode).

Parameters

- **hsc:** SMARTCARD handle.

Return values

- **HAL:** status

Notes

- This procedure could be used for aborting any ongoing transfer started in Interrupt or DMA mode. This procedure performs following operations : Disable SMARTCARD Interrupts (Tx)
Disable the DMA transfer in the peripheral register (if enabled)
Abort DMA transfer by calling HAL_DMA_Abort_IT (in case of transfer in DMA mode)
Set handle State to READY
At abort completion, call user abort complete callback
- This procedure is executed in Interrupt mode, meaning that abort procedure could be considered as completed only when user abort complete callback is executed (not when exiting function).

HAL_SMARTCARD_AbortReceive_IT

Function name

HAL_StatusTypeDef HAL_SMARTCARD_AbortReceive_IT (SMARTCARD_HandleTypeDef * hsc)

Function description

Abort ongoing Receive transfer (Interrupt mode).

Parameters

- **hsc:** SMARTCARD handle.

Return values

- **HAL:** status

Notes

- This procedure could be used for aborting any ongoing transfer started in Interrupt or DMA mode. This procedure performs following operations : Disable SMARTCARD Interrupts (Rx)
Disable the DMA transfer in the peripheral register (if enabled)
Abort DMA transfer by calling HAL_DMA_Abort_IT (in case of transfer in DMA mode)
Set handle State to READY
At abort completion, call user abort complete callback
- This procedure is executed in Interrupt mode, meaning that abort procedure could be considered as completed only when user abort complete callback is executed (not when exiting function).

HAL_SMARTCARD_IRQHandler

Function name

void HAL_SMARTCARD_IRQHandler (SMARTCARD_HandleTypeDef * hsc)

Function description

This function handles SMARTCARD interrupt request.

Parameters

- **hsc:** Pointer to a SMARTCARD_HandleTypeDef structure that contains the configuration information for SMARTCARD module.

Return values

- **None:**

HAL_SMARTCARD_TxCpltCallback

Function name

void HAL_SMARTCARD_TxCpltCallback (SMARTCARD_HandleTypeDef * hsc)

Function description

Tx Transfer completed callbacks.

Parameters

- **hsc:** Pointer to a SMARTCARD_HandleTypeDef structure that contains the configuration information for SMARTCARD module.

Return values

- **None:**

HAL_SMARTCARD_RxCpltCallback

Function name

void HAL_SMARTCARD_RxCpltCallback (SMARTCARD_HandleTypeDef * hsc)

Function description

Rx Transfer completed callback.

Parameters

- **hsc:** Pointer to a SMARTCARD_HandleTypeDef structure that contains the configuration information for SMARTCARD module.

Return values

- **None:**

HAL_SMARTCARD_ErrorCallback

Function name

void HAL_SMARTCARD_ErrorCallback (SMARTCARD_HandleTypeDef * hsc)

Function description

SMARTCARD error callback.

Parameters

- **hsc:** Pointer to a SMARTCARD_HandleTypeDef structure that contains the configuration information for SMARTCARD module.

Return values

- **None:**

HAL_SMARTCARD_AbortCpltCallback

Function name

void HAL_SMARTCARD_AbortCpltCallback (SMARTCARD_HandleTypeDef * hsc)

Function description

SMARTCARD Abort Complete callback.

Parameters

- **hsc:** SMARTCARD handle.

Return values

- **None:**

HAL_SMARTCARD_AbortTransmitCpltCallback

Function name

void HAL_SMARTCARD_AbortTransmitCpltCallback (SMARTCARD_HandleTypeDef * hsc)

Function description

SMARTCARD Abort Transmit Complete callback.

Parameters

- **hsc:** SMARTCARD handle.

Return values

- **None:**

HAL_SMARTCARD_AbortReceiveCpltCallback

Function name

void HAL_SMARTCARD_AbortReceiveCpltCallback (SMARTCARD_HandleTypeDef * hsc)

Function description

SMARTCARD Abort Receive Complete callback.

Parameters

- **hsc:** SMARTCARD handle.

Return values

- **None:**

HAL_SMARTCARD_GetState

Function name

HAL_SMARTCARD_StateTypeDef HAL_SMARTCARD_GetState (const SMARTCARD_HandleTypeDef * hsc)

Function description

Return the SMARTCARD handle state.

Parameters

- **hsc:** Pointer to a SMARTCARD_HandleTypeDef structure that contains the configuration information for SMARTCARD module.

Return values

- **HAL:** state

HAL_SMARTCARD_GetError

Function name

uint32_t HAL_SMARTCARD_GetError (const SMARTCARD_HandleTypeDef * hsc)

Function description

Return the SMARTCARD error code.

Parameters

- **hsc:** Pointer to a SMARTCARD_HandleTypeDef structure that contains the configuration information for the specified SMARTCARD.

Return values

- **SMARTCARD:** Error Code

63.3 SMARTCARD Firmware driver defines

The following section lists the various define and macros of the module.

63.3.1 SMARTCARD

SMARTCARD

SMARTCARD Clock Phase

SMARTCARD_PHASE_1EDGE

SMARTCARD_PHASE_2EDGE

SMARTCARD Clock Polarity

SMARTCARD_POLARITY_LOW

SMARTCARD_POLARITY_HIGH

SMARTCARD DMA requests

SMARTCARD_DMAREQ_TX

SMARTCARD_DMAREQ_RX

SMARTCARD Error Code

HAL_SMARTCARD_ERROR_NONE

No error

HAL_SMARTCARD_ERROR_PE

Parity error

HAL_SMARTCARD_ERROR_NE

Noise error

HAL_SMARTCARD_ERROR_FE

Frame error

HAL_SMARTCARD_ERROR_ORE

Overrun error

HAL_SMARTCARD_ERROR_DMA

DMA transfer error

HAL_SMARTCARD_ERROR_INVALID_CALLBACK

Invalid Callback error

SMARTCARD Exported Macros

__HAL_SMARTCARD_RESET_HANDLE_STATE

Description:

- Reset SMARTCARD handle gstate & RxState.

Parameters:

- `__HANDLE__`: specifies the SMARTCARD Handle. SMARTCARD Handle selects the USARTx peripheral (USART availability and x value depending on device).

Return value:

- None

__HAL_SMARTCARD_FLUSH_DRREGISTER

Description:

- Flush the Smartcard DR register.

Parameters:

- `__HANDLE__`: specifies the SMARTCARD Handle. SMARTCARD Handle selects the USARTx peripheral (USART availability and x value depending on device).

Return value:

- None

__HAL_SMARTCARD_GET_FLAG

Description:

- Check whether the specified Smartcard flag is set or not.

Parameters:

- `__HANDLE__`: specifies the SMARTCARD Handle. SMARTCARD Handle selects the USARTx peripheral (USART availability and x value depending on device).
- `__FLAG__`: specifies the flag to check. This parameter can be one of the following values:
 - SMARTCARD_FLAG_TXE: Transmit data register empty flag
 - SMARTCARD_FLAG_TC: Transmission Complete flag
 - SMARTCARD_FLAG_RXNE: Receive data register not empty flag
 - SMARTCARD_FLAG_IDLE: Idle Line detection flag
 - SMARTCARD_FLAG_ORE: Overrun Error flag
 - SMARTCARD_FLAG_NE: Noise Error flag
 - SMARTCARD_FLAG_FE: Framing Error flag
 - SMARTCARD_FLAG_PE: Parity Error flag

Return value:

- The: new state of `__FLAG__` (TRUE or FALSE).

__HAL_SMARTCARD_CLEAR_FLAG

Description:

- Clear the specified Smartcard pending flags.

Parameters:

- `__HANDLE__`: specifies the SMARTCARD Handle. SMARTCARD Handle selects the USARTx peripheral (USART availability and x value depending on device).
- `__FLAG__`: specifies the flag to check. This parameter can be any combination of the following values:
 - SMARTCARD_FLAG_TC: Transmission Complete flag.
 - SMARTCARD_FLAG_RXNE: Receive data register not empty flag.

Return value:

- None

Notes:

- PE (Parity error), FE (Framing error), NE (Noise error) and ORE (Overrun error) flags are cleared by software sequence: a read operation to USART_SR register followed by a read operation to USART_DR register. RXNE flag can be also cleared by a read to the USART_DR register. TC flag can be also cleared by software sequence: a read operation to USART_SR register followed by a write operation to USART_DR register. TXE flag is cleared only by a write to the USART_DR register.

__HAL_SMARTCARD_CLEAR_PFLAG

Description:

- Clear the SMARTCARD PE pending flag.

Parameters:

- `__HANDLE__`: specifies the USART Handle. SMARTCARD Handle selects the USARTx peripheral (USART availability and x value depending on device).

Return value:

- None

__HAL_SMARTCARD_CLEAR_FEFLAG

Description:

- Clear the SMARTCARD FE pending flag.

Parameters:

- `__HANDLE__`: specifies the USART Handle. SMARTCARD Handle selects the USARTx peripheral (USART availability and x value depending on device).

Return value:

- None

__HAL_SMARTCARD_CLEAR_NEFLAG

Description:

- Clear the SMARTCARD NE pending flag.

Parameters:

- `__HANDLE__`: specifies the USART Handle. SMARTCARD Handle selects the USARTx peripheral (USART availability and x value depending on device).

Return value:

- None

__HAL_SMARTCARD_CLEAR_OREFLAG

Description:

- Clear the SMARTCARD ORE pending flag.

Parameters:

- `__HANDLE__`: specifies the USART Handle. SMARTCARD Handle selects the USARTx peripheral (USART availability and x value depending on device).

Return value:

- None

__HAL_SMARTCARD_CLEAR_IDLEFLAG

Description:

- Clear the SMARTCARD IDLE pending flag.

Parameters:

- `__HANDLE__`: specifies the USART Handle. SMARTCARD Handle selects the USARTx peripheral (USART availability and x value depending on device).

Return value:

- None

__HAL_SMARTCARD_ENABLE_IT

Description:

- Enable the specified SmartCard interrupt.

Parameters:

- `__HANDLE__`: specifies the SMARTCARD Handle. SMARTCARD Handle selects the USARTx peripheral (USART availability and x value depending on device).
- `__INTERRUPT__`: specifies the SMARTCARD interrupt to enable. This parameter can be one of the following values:
 - SMARTCARD_IT_TXE: Transmit Data Register empty interrupt
 - SMARTCARD_IT_TC: Transmission complete interrupt
 - SMARTCARD_IT_RXNE: Receive Data register not empty interrupt
 - SMARTCARD_IT_IDLE: Idle line detection interrupt
 - SMARTCARD_IT_PE: Parity Error interrupt
 - SMARTCARD_IT_ERR: Error interrupt(Frame error, noise error, overrun error)

Return value:

- None

__HAL_SMARTCARD_DISABLE_IT

Description:

- Disable the specified SmartCard interrupt.

Parameters:

- `__HANDLE__`: specifies the SMARTCARD Handle. SMARTCARD Handle selects the USARTx peripheral (USART availability and x value depending on device).
- `__INTERRUPT__`: specifies the SMARTCARD interrupt to disable. This parameter can be one of the following values:
 - SMARTCARD_IT_TXE: Transmit Data Register empty interrupt
 - SMARTCARD_IT_TC: Transmission complete interrupt
 - SMARTCARD_IT_RXNE: Receive Data register not empty interrupt
 - SMARTCARD_IT_IDLE: Idle line detection interrupt
 - SMARTCARD_IT_PE: Parity Error interrupt
 - SMARTCARD_IT_ERR: Error interrupt(Frame error, noise error, overrun error)

Return value:

- None

__HAL_SMARTCARD_GET_IT_SOURCE

Description:

- Checks whether the specified SmartCard interrupt has occurred or not.

Parameters:

- `__HANDLE__`: specifies the SmartCard Handle.
- `__IT__`: specifies the SMARTCARD interrupt source to check. This parameter can be one of the following values:
 - SMARTCARD_IT_TXE: Transmit Data Register empty interrupt
 - SMARTCARD_IT_TC: Transmission complete interrupt
 - SMARTCARD_IT_RXNE: Receive Data register not empty interrupt
 - SMARTCARD_IT_IDLE: Idle line detection interrupt
 - SMARTCARD_IT_ERR: Error interrupt
 - SMARTCARD_IT_PE: Parity Error interrupt

Return value:

- The: new state of `__IT__` (TRUE or FALSE).

__HAL_SMARTCARD_ONE_BIT_SAMPLE_ENABLE

Description:

- Macro to enable the SMARTCARD's one bit sample method.

Parameters:

- `__HANDLE__`: specifies the SMARTCARD Handle.

Return value:

- None

__HAL_SMARTCARD_ONE_BIT_SAMPLE_DISABLE

Description:

- Macro to disable the SMARTCARD's one bit sample method.

Parameters:

- `__HANDLE__`: specifies the SMARTCARD Handle.

Return value:

- None

__HAL_SMARTCARD_ENABLE

Description:

- Enable the USART associated to the SMARTCARD Handle.

Parameters:

- `__HANDLE__`: specifies the SMARTCARD Handle. SMARTCARD Handle selects the USARTx peripheral (USART availability and x value depending on device).

Return value:

- None

__HAL_SMARTCARD_DISABLE

Description:

- Disable the USART associated to the SMARTCARD Handle.

Parameters:

- `__HANDLE__`: specifies the SMARTCARD Handle. SMARTCARD Handle selects the USARTx peripheral (USART availability and x value depending on device).

Return value:

- None

__HAL_SMARTCARD_DMA_REQUEST_ENABLE

Description:

- Macros to enable the SmartCard DMA request.

Parameters:

- __HANDLE__: specifies the SmartCard Handle.
- __REQUEST__: specifies the SmartCard DMA request. This parameter can be one of the following values:
 - SMARTCARD_DMAREQ_TX: SmartCard DMA transmit request
 - SMARTCARD_DMAREQ_RX: SmartCard DMA receive request

Return value:

- None

__HAL_SMARTCARD_DMA_REQUEST_DISABLE

Description:

- Macros to disable the SmartCard DMA request.

Parameters:

- __HANDLE__: specifies the SmartCard Handle.
- __REQUEST__: specifies the SmartCard DMA request. This parameter can be one of the following values:
 - SMARTCARD_DMAREQ_TX: SmartCard DMA transmit request
 - SMARTCARD_DMAREQ_RX: SmartCard DMA receive request

Return value:

- None

SMARTCARD Last Bit

SMARTCARD_LASTBIT_DISABLE

SMARTCARD_LASTBIT_ENABLE

SMARTCARD Mode

SMARTCARD_MODE_RX

SMARTCARD_MODE_TX

SMARTCARD_MODE_TX_RX

SMARTCARD NACK State

SMARTCARD_NACK_ENABLE

SMARTCARD_NACK_DISABLE

SMARTCARD Parity

SMARTCARD_PARITY_EVEN

SMARTCARD_PARITY_ODD

SMARTCARD Prescaler

SMARTCARD_PRESCALER_SYSCLK_DIV2

SYSCLK divided by 2

SMARTCARD_PRESCALER_SYSCLK_DIV4

SYSCLK divided by 4

SMARTCARD_PRESCALER_SYSCLK_DIV6

SYSCLK divided by 6

SMARTCARD_PRESCALER_SYSCLK_DIV8

SYSCLK divided by 8

SMARTCARD_PRESCALER_SYSCLK_DIV10

SYSCLK divided by 10

SMARTCARD_PRESCALER_SYSCLK_DIV12

SYSCLK divided by 12

SMARTCARD_PRESCALER_SYSCLK_DIV14

SYSCLK divided by 14

SMARTCARD_PRESCALER_SYSCLK_DIV16

SYSCLK divided by 16

SMARTCARD_PRESCALER_SYSCLK_DIV18

SYSCLK divided by 18

SMARTCARD_PRESCALER_SYSCLK_DIV20

SYSCLK divided by 20

SMARTCARD_PRESCALER_SYSCLK_DIV22

SYSCLK divided by 22

SMARTCARD_PRESCALER_SYSCLK_DIV24

SYSCLK divided by 24

SMARTCARD_PRESCALER_SYSCLK_DIV26

SYSCLK divided by 26

SMARTCARD_PRESCALER_SYSCLK_DIV28

SYSCLK divided by 28

SMARTCARD_PRESCALER_SYSCLK_DIV30

SYSCLK divided by 30

SMARTCARD_PRESCALER_SYSCLK_DIV32

SYSCLK divided by 32

SMARTCARD_PRESCALER_SYSCLK_DIV34

SYSCLK divided by 34

SMARTCARD_PRESCALER_SYSCLK_DIV36

SYSCLK divided by 36

SMARTCARD_PRESCALER_SYSCLK_DIV38

SYSCLK divided by 38

SMARTCARD_PRESCALER_SYSCLK_DIV40

SYSCLK divided by 40

SMARTCARD_PRESCALER_SYSCLK_DIV42

SYSCLK divided by 42

SMARTCARD_PRESCALER_SYSCLK_DIV44

SYSCLK divided by 44

SMARTCARD_PRESCALER_SYSCLK_DIV46

SYSCLK divided by 46

SMARTCARD_PRESCALER_SYSCLK_DIV48

SYSCLK divided by 48

SMARTCARD_PRESCALER_SYSCLK_DIV50

SYSCLK divided by 50

SMARTCARD_PRESCALER_SYSCLK_DIV52

SYSCLK divided by 52

SMARTCARD_PRESCALER_SYSCLK_DIV54

SYSCLK divided by 54

SMARTCARD_PRESCALER_SYSCLK_DIV56

SYSCLK divided by 56

SMARTCARD_PRESCALER_SYSCLK_DIV58

SYSCLK divided by 58

SMARTCARD_PRESCALER_SYSCLK_DIV60

SYSCLK divided by 60

SMARTCARD_PRESCALER_SYSCLK_DIV62

SYSCLK divided by 62

SMARTCARD Number of Stop Bits**SMARTCARD_STOPBITS_0_5****SMARTCARD_STOPBITS_1_5*****SMARTCARD Word Length*****SMARTCARD_WORDLENGTH_9B**

64 HAL SMBUS Generic Driver

64.1 SMBUS Firmware driver registers structures

64.1.1 SMBUS_InitTypeDef

SMBUS_InitTypeDef is defined in the `stm32f4xx_hal_smbus.h`

Data Fields

- `uint32_t ClockSpeed`
- `uint32_t AnalogFilter`
- `uint32_t OwnAddress1`
- `uint32_t AddressingMode`
- `uint32_t DualAddressMode`
- `uint32_t OwnAddress2`
- `uint32_t GeneralCallMode`
- `uint32_t NoStretchMode`
- `uint32_t PacketErrorCheckMode`
- `uint32_t PeripheralMode`

Field Documentation

- `uint32_t SMBUS_InitTypeDef::ClockSpeed`
Specifies the clock frequency. This parameter must be set to a value lower than 100kHz
- `uint32_t SMBUS_InitTypeDef::AnalogFilter`
Specifies if Analog Filter is enable or not. This parameter can be a value of [SMBUS_Analog_Filter](#)
- `uint32_t SMBUS_InitTypeDef::OwnAddress1`
Specifies the first device own address. This parameter can be a 7-bit or 10-bit address.
- `uint32_t SMBUS_InitTypeDef::AddressingMode`
Specifies if 7-bit or 10-bit addressing mode is selected. This parameter can be a value of [SMBUS_addressing_mode](#)
- `uint32_t SMBUS_InitTypeDef::DualAddressMode`
Specifies if dual addressing mode is selected. This parameter can be a value of [SMBUS_dual_addressing_mode](#)
- `uint32_t SMBUS_InitTypeDef::OwnAddress2`
Specifies the second device own address if dual addressing mode is selected. This parameter can be a 7-bit address.
- `uint32_t SMBUS_InitTypeDef::GeneralCallMode`
Specifies if general call mode is selected. This parameter can be a value of [SMBUS_general_call_addressing_mode](#)
- `uint32_t SMBUS_InitTypeDef::NoStretchMode`
Specifies if nostretch mode is selected. This parameter can be a value of [SMBUS_nostretch_mode](#)
- `uint32_t SMBUS_InitTypeDef::PacketErrorCheckMode`
Specifies if Packet Error Check mode is selected. This parameter can be a value of [SMBUS_packet_error_check_mode](#)
- `uint32_t SMBUS_InitTypeDef::PeripheralMode`
Specifies which mode of Periph is selected. This parameter can be a value of [SMBUS_peripheral_mode](#)

64.1.2 __SMBUS_HandleTypeDef

__SMBUS_HandleTypeDef is defined in the `stm32f4xx_hal_smbus.h`

Data Fields

- `I2C_TypeDef * Instance`
- `SMBUS_InitTypeDef Init`
- `uint8_t * pBuffPtr`
- `uint16_t XferSize`

- `__IO uint16_t XferCount`
- `__IO uint32_t XferOptions`
- `__IO uint32_t PreviousState`
- `HAL_LockTypeDef Lock`
- `__IO HAL_SMBUS_StateTypeDef State`
- `__IO HAL_SMBUS_ModeTypeDef Mode`
- `__IO uint32_t ErrorCode`
- `__IO uint32_t Devaddress`
- `__IO uint32_t EventCount`
- `uint8_t XferPEC`
- `void(* MasterTxCpltCallback`
- `void(* MasterRxCpltCallback`
- `void(* SlaveTxCpltCallback`
- `void(* SlaveRxCpltCallback`
- `void(* ListenCpltCallback`
- `void(* MemTxCpltCallback`
- `void(* MemRxCpltCallback`
- `void(* ErrorCallback`
- `void(* AbortCpltCallback`
- `void(* AddrCallback`
- `void(* MspInitCallback`
- `void(* MspDeInitCallback`

Field Documentation

- **`I2C_TypeDef* __SMBUS_HandleTypeDef::Instance`**
SMBUS registers base address
- **`SMBUS_InitTypeDef __SMBUS_HandleTypeDef::Init`**
SMBUS communication parameters
- **`uint8_t* __SMBUS_HandleTypeDef::pBuffPtr`**
Pointer to SMBUS transfer buffer
- **`uint16_t __SMBUS_HandleTypeDef::XferSize`**
SMBUS transfer size
- **`__IO uint16_t __SMBUS_HandleTypeDef::XferCount`**
SMBUS transfer counter
- **`__IO uint32_t __SMBUS_HandleTypeDef::XferOptions`**
SMBUS transfer options this parameter can be a value of [SMBUS_XferOptions_definition](#)
- **`__IO uint32_t __SMBUS_HandleTypeDef::PreviousState`**
SMBUS communication Previous state and mode context for internal usage
- **`HAL_LockTypeDef __SMBUS_HandleTypeDef::Lock`**
SMBUS locking object
- **`__IO HAL_SMBUS_StateTypeDef __SMBUS_HandleTypeDef::State`**
SMBUS communication state
- **`__IO HAL_SMBUS_ModeTypeDef __SMBUS_HandleTypeDef::Mode`**
SMBUS communication mode
- **`__IO uint32_t __SMBUS_HandleTypeDef::ErrorCode`**
SMBUS Error code
- **`__IO uint32_t __SMBUS_HandleTypeDef::Devaddress`**
SMBUS Target device address
- **`__IO uint32_t __SMBUS_HandleTypeDef::EventCount`**
SMBUS Event counter
- **`uint8_t __SMBUS_HandleTypeDef::XferPEC`**
SMBUS PEC data in reception mode

- **`void(* __SMBUS_HandleTypeDef::MasterTxCpltCallback)(struct __SMBUS_HandleTypeDef *hsmbus)`**
SMBUS Master Tx Transfer completed callback
- **`void(* __SMBUS_HandleTypeDef::MasterRxCpltCallback)(struct __SMBUS_HandleTypeDef *hsmbus)`**
SMBUS Master Rx Transfer completed callback
- **`void(* __SMBUS_HandleTypeDef::SlaveTxCpltCallback)(struct __SMBUS_HandleTypeDef *hsmbus)`**
SMBUS Slave Tx Transfer completed callback
- **`void(* __SMBUS_HandleTypeDef::SlaveRxCpltCallback)(struct __SMBUS_HandleTypeDef *hsmbus)`**
SMBUS Slave Rx Transfer completed callback
- **`void(* __SMBUS_HandleTypeDef::ListenCpltCallback)(struct __SMBUS_HandleTypeDef *hsmbus)`**
SMBUS Listen Complete callback
- **`void(* __SMBUS_HandleTypeDef::MemTxCpltCallback)(struct __SMBUS_HandleTypeDef *hsmbus)`**
SMBUS Memory Tx Transfer completed callback
- **`void(* __SMBUS_HandleTypeDef::MemRxCpltCallback)(struct __SMBUS_HandleTypeDef *hsmbus)`**
SMBUS Memory Rx Transfer completed callback
- **`void(* __SMBUS_HandleTypeDef::ErrorCallback)(struct __SMBUS_HandleTypeDef *hsmbus)`**
SMBUS Error callback
- **`void(* __SMBUS_HandleTypeDef::AbortCpltCallback)(struct __SMBUS_HandleTypeDef *hsmbus)`**
SMBUS Abort callback
- **`void(* __SMBUS_HandleTypeDef::AddrCallback)(struct __SMBUS_HandleTypeDef *hsmbus, uint8_t TransferDirection, uint16_t AddrMatchCode)`**
SMBUS Slave Address Match callback
- **`void(* __SMBUS_HandleTypeDef::MspInitCallback)(struct __SMBUS_HandleTypeDef *hsmbus)`**
SMBUS Msp Init callback
- **`void(* __SMBUS_HandleTypeDef::MspDeInitCallback)(struct __SMBUS_HandleTypeDef *hsmbus)`**
SMBUS Msp DeInit callback

64.2 SMBUS Firmware driver API description

The following section lists the various functions of the SMBUS library.

64.2.1 How to use this driver

The SMBUS HAL driver can be used as follows:

1. Declare a `SMBUS_HandleTypeDef` handle structure, for example: `SMBUS_HandleTypeDef hsmbus;`
2. Initialize the SMBUS low level resources by implementing the `HAL_SMBUS_MspInit()` API:
 - a. Enable the SMBUSx interface clock
 - b. SMBUS pins configuration
 - Enable the clock for the SMBUS GPIOs
 - Configure SMBUS pins as alternate function open-drain
 - c. NVIC configuration if you need to use interrupt process
 - Configure the SMBUSx interrupt priority
 - Enable the NVIC SMBUS IRQ Channel
3. Configure the Communication Speed, Duty cycle, Addressing mode, Own Address1, Dual Addressing mode, Own Address2, General call and Nostretch mode in the `hsmbus Init` structure.
4. Initialize the SMBUS registers by calling the `HAL_SMBUS_Init()`, configures also the low level Hardware (GPIO, CLOCK, NVIC...etc) by calling the customized `HAL_SMBUS_MspInit(&hsmbus)` API.
5. To check if target device is ready for communication, use the function `HAL_SMBUS_IsDeviceReady()`
6. For SMBUS IO operations, only one mode of operations is available within this driver :

Interrupt mode IO operation

- Transmit in master/host SMBUS mode an amount of data in non blocking mode using `HAL_SMBUS_Master_Transmit_IT()`
 - At transmission end of transfer `HAL_SMBUS_MasterTxCpltCallback()` is executed and user can add his own code by customization of function pointer `HAL_SMBUS_MasterTxCpltCallback()`
- Receive in master/host SMBUS mode an amount of data in non blocking mode using `HAL_SMBUS_Master_Receive_IT()`
 - At reception end of transfer `HAL_SMBUS_MasterRxCpltCallback()` is executed and user can add his own code by customization of function pointer `HAL_SMBUS_MasterRxCpltCallback()`
- Abort a master/Host SMBUS process communication with Interrupt using `HAL_SMBUS_Master_Abort_IT()`
 - End of abort process, `HAL_SMBUS_AbortCpltCallback()` is executed and user can add his own code by customization of function pointer `HAL_SMBUS_AbortCpltCallback()`
- Enable/disable the Address listen mode in slave/device or host/slave SMBUS mode using `HAL_SMBUS_EnableListen_IT()` `HAL_SMBUS_DisableListen_IT()`
 - When address slave/device SMBUS match, `HAL_SMBUS_AddrCallback()` is executed and user can add his own code to check the Address Match Code and the transmission direction request by master/host (Write/Read).
 - At Listen mode end `HAL_SMBUS_ListenCpltCallback()` is executed and user can add his own code by customization of function pointer `HAL_SMBUS_ListenCpltCallback()`
- Transmit in slave/device SMBUS mode an amount of data in non blocking mode using `HAL_SMBUS_Slave_Transmit_IT()`
 - At transmission end of transfer `HAL_SMBUS_SlaveTxCpltCallback()` is executed and user can add his own code by customization of function pointer `HAL_SMBUS_SlaveTxCpltCallback()`
- Receive in slave/device SMBUS mode an amount of data in non blocking mode using `HAL_SMBUS_Slave_Receive_IT()`
 - At reception end of transfer `HAL_SMBUS_SlaveRxCpltCallback()` is executed and user can add his own code by customization of function pointer `HAL_SMBUS_SlaveRxCpltCallback()`
- Enable/Disable the SMBUS alert mode using `HAL_SMBUS_EnableAlert_IT()` and `HAL_SMBUS_DisableAlert_IT()`
 - When SMBUS Alert is generated `HAL_SMBUS_ErrorCallback()` is executed and user can add his own code by customization of function pointer `HAL_SMBUS_ErrorCallback()` to check the Alert Error Code using function `HAL_SMBUS_GetError()`
- Get HAL state machine or error values using `HAL_SMBUS_GetState()` or `HAL_SMBUS_GetError()`
- In case of transfer Error, `HAL_SMBUS_ErrorCallback()` function is executed and user can add his own code by customization of function pointer `HAL_SMBUS_ErrorCallback()` to check the Error Code using function `HAL_SMBUS_GetError()`

SMBUS HAL driver macros list

Below the list of most used macros in SMBUS HAL driver.

- `__HAL_SMBUS_ENABLE` : Enable the SMBUS peripheral
- `__HAL_SMBUS_DISABLE` : Disable the SMBUS peripheral
- `__HAL_SMBUS_GET_FLAG` : Checks whether the specified SMBUS flag is set or not
- `__HAL_SMBUS_CLEAR_FLAG`: Clear the specified SMBUS pending flag
- `__HAL_SMBUS_ENABLE_IT` : Enable the specified SMBUS interrupt
- `__HAL_SMBUS_DISABLE_IT`: Disable the specified SMBUS interrupt

Note: You can refer to the SMBUS HAL driver header file for more useful macros

Callback registration

The compilation flag `USE_HAL_SMBUS_REGISTER_CALLBACKS` when set to 1 allows the user to configure dynamically the driver callbacks. Use Functions `HAL_SMBUS_RegisterCallback()` or `HAL_SMBUS_RegisterXXXCallback()` to register an interrupt callback. Function `HAL_SMBUS_RegisterCallback()` allows to register following callbacks:

- MasterTxCpltCallback : callback for Master transmission end of transfer.
- MasterRxCpltCallback : callback for Master reception end of transfer.
- SlaveTxCpltCallback : callback for Slave transmission end of transfer.
- SlaveRxCpltCallback : callback for Slave reception end of transfer.
- ListenCpltCallback : callback for end of listen mode.
- ErrorCallback : callback for error detection.
- AbortCpltCallback : callback for abort completion process.
- MspInitCallback : callback for Msp Init.
- MspDeInitCallback : callback for Msp DeInit. This function takes as parameters the HAL peripheral handle, the Callback ID and a pointer to the user callback function.

For specific callback AddrCallback use dedicated register callbacks : HAL_SMBUS_RegisterAddrCallback().

Use function HAL_SMBUS_UnRegisterCallback to reset a callback to the default weak function.

HAL_SMBUS_UnRegisterCallback takes as parameters the HAL peripheral handle, and the Callback ID. This function allows to reset following callbacks:

- MasterTxCpltCallback : callback for Master transmission end of transfer.
- MasterRxCpltCallback : callback for Master reception end of transfer.
- SlaveTxCpltCallback : callback for Slave transmission end of transfer.
- SlaveRxCpltCallback : callback for Slave reception end of transfer.
- ListenCpltCallback : callback for end of listen mode.
- ErrorCallback : callback for error detection.
- AbortCpltCallback : callback for abort completion process.
- MspInitCallback : callback for Msp Init.
- MspDeInitCallback : callback for Msp DeInit.

For callback AddrCallback use dedicated register callbacks : HAL_SMBUS_UnRegisterAddrCallback().

By default, after the HAL_SMBUS_Init() and when the state is HAL_SMBUS_STATE_RESET all callbacks are set to the corresponding weak functions: examples HAL_SMBUS_MasterTxCpltCallback(), HAL_SMBUS_MasterRxCpltCallback(). Exception done for MspInit and MspDeInit functions that are reset to the legacy weak functions in the HAL_SMBUS_Init()/ HAL_SMBUS_DeInit() only when these callbacks are null (not registered beforehand). If MspInit or MspDeInit are not null, the HAL_SMBUS_Init()/ HAL_SMBUS_DeInit() keep and use the user MspInit/MspDeInit callbacks (registered beforehand) whatever the state.

Callbacks can be registered/unregistered in HAL_SMBUS_STATE_READY state only. Exception done MspInit/MspDeInit functions that can be registered/unregistered in HAL_SMBUS_STATE_READY or HAL_SMBUS_STATE_RESET state, thus registered (user) MspInit/DeInit callbacks can be used during the Init/DeInit. Then, the user first registers the MspInit/MspDeInit user callbacks using HAL_SMBUS_RegisterCallback() before calling HAL_SMBUS_DeInit() or HAL_SMBUS_Init() function.

When the compilation flag USE_HAL_SMBUS_REGISTER_CALLBACKS is set to 0 or not defined, the callback registration feature is not available and all callbacks are set to the corresponding weak functions.

64.2.2 Initialization and de-initialization functions

This subsection provides a set of functions allowing to initialize and deinitialize the SMBUSx peripheral:

- User must Implement HAL_SMBUS_MspInit() function in which he configures all related peripherals resources (CLOCK, GPIO, IT and NVIC).
- Call the function HAL_SMBUS_Init() to configure the selected device with the selected configuration:
 - Communication Speed
 - Addressing mode
 - Own Address 1
 - Dual Addressing mode
 - Own Address 2
 - General call mode
 - Nostretch mode
 - Packet Error Check mode
 - Peripheral mode

- Call the function `HAL_SMBUS_DeInit()` to restore the default configuration of the selected SMBUSx peripheral.

This section contains the following APIs:

- `HAL_SMBUS_Init()`
- `HAL_SMBUS_DeInit()`
- `HAL_SMBUS_MspInit()`
- `HAL_SMBUS_MspDeInit()`
- `HAL_SMBUS_ConfigAnalogFilter()`
- `HAL_SMBUS_ConfigDigitalFilter()`
- `HAL_SMBUS_RegisterCallback()`
- `HAL_SMBUS_UnRegisterCallback()`
- `HAL_SMBUS_RegisterAddrCallback()`
- `HAL_SMBUS_UnRegisterAddrCallback()`

64.2.3 IO operation functions

This subsection provides a set of functions allowing to manage the SMBUS data transfers.

1. Blocking mode function to check if device is ready for usage is :
 - `HAL_SMBUS_IsDeviceReady()`
2. There is only one mode of transfer:
 - Non Blocking mode : The communication is performed using Interrupts. These functions return the status of the transfer startup. The end of the data processing will be indicated through the dedicated SMBUS IRQ when using Interrupt mode.
3. Non Blocking mode functions with Interrupt are :
 - `HAL_SMBUS_Master_Transmit_IT()`
 - `HAL_SMBUS_Master_Receive_IT()`
 - `HAL_SMBUS_Master_Abort_IT()`
 - `HAL_SMBUS_Slave_Transmit_IT()`
 - `HAL_SMBUS_Slave_Receive_IT()`
 - `HAL_SMBUS_EnableAlert_IT()`
 - `HAL_SMBUS_DisableAlert_IT()`
4. A set of Transfer Complete Callbacks are provided in No_Blocking mode:
 - `HAL_SMBUS_MasterTxCpltCallback()`
 - `HAL_SMBUS_MasterRxCpltCallback()`
 - `HAL_SMBUS_SlaveTxCpltCallback()`
 - `HAL_SMBUS_SlaveRxCpltCallback()`
 - `HAL_SMBUS_AddrCallback()`
 - `HAL_SMBUS_ListenCpltCallback()`
 - `HAL_SMBUS_ErrorCallback()`
 - `HAL_SMBUS_AbortCpltCallback()`

This section contains the following APIs:

- `HAL_SMBUS_Master_Transmit_IT()`
- `HAL_SMBUS_Master_Receive_IT()`
- `HAL_SMBUS_Master_Abort_IT()`
- `HAL_SMBUS_Slave_Transmit_IT()`
- `HAL_SMBUS_Slave_Receive_IT()`
- `HAL_SMBUS_EnableListen_IT()`
- `HAL_SMBUS_DisableListen_IT()`
- `HAL_SMBUS_EnableAlert_IT()`
- `HAL_SMBUS_DisableAlert_IT()`
- `HAL_SMBUS_IsDeviceReady()`

- `HAL_SMBUS_EV_IRQHandler()`
- `HAL_SMBUS_ER_IRQHandler()`
- `HAL_SMBUS_MasterTxCpltCallback()`
- `HAL_SMBUS_MasterRxCpltCallback()`
- `HAL_SMBUS_SlaveTxCpltCallback()`
- `HAL_SMBUS_SlaveRxCpltCallback()`
- `HAL_SMBUS_AddrCallback()`
- `HAL_SMBUS_ListenCpltCallback()`
- `HAL_SMBUS_ErrorCallback()`
- `HAL_SMBUS_AbortCpltCallback()`

64.2.4 Peripheral State, Mode and Error functions

This subsection permits to get in run-time the status of the peripheral and the data flow.

This section contains the following APIs:

- `HAL_SMBUS_GetState()`
- `HAL_SMBUS_GetMode()`
- `HAL_SMBUS_GetError()`

64.2.5 Detailed description of functions

HAL_SMBUS_Init

Function name

HAL_StatusTypeDef HAL_SMBUS_Init (SMBUS_HandleTypeDef * hsmbus)

Function description

Initializes the SMBUS according to the specified parameters in the SMBUS_InitTypeDef and initialize the associated handle.

Parameters

- **hsmbus:** pointer to a SMBUS_HandleTypeDef structure that contains the configuration information for the specified SMBUS

Return values

- **HAL:** status

HAL_SMBUS_DeInit

Function name

HAL_StatusTypeDef HAL_SMBUS_DeInit (SMBUS_HandleTypeDef * hsmbus)

Function description

Deinitializes the SMBUS peripheral.

Parameters

- **hsmbus:** pointer to a SMBUS_HandleTypeDef structure that contains the configuration information for the specified SMBUS.

Return values

- **HAL:** status

HAL_SMBUS_Msplnit

Function name

void HAL_SMBUS_Msplnit (SMBUS_HandleTypeDef * hsmbus)

Function description

Initialize the SMBUS MSP.

Parameters

- **hsmbus:** pointer to a SMBUS_HandleTypeDef structure that contains the configuration information for the specified SMBUS

Return values

- **None:**

HAL_SMBUS_MspDeInit

Function name

void HAL_SMBUS_MspDeInit (SMBUS_HandleTypeDef * hsmbus)

Function description

Deinitialize the SMBUS MSP.

Parameters

- **hsmbus:** pointer to a SMBUS_HandleTypeDef structure that contains the configuration information for the specified SMBUS

Return values

- **None:**

HAL_SMBUS_RegisterCallback

Function name

HAL_StatusTypeDef HAL_SMBUS_RegisterCallback (SMBUS_HandleTypeDef * hsmbus, HAL_SMBUS_CallbackIDTypeDef CallbackID, pSMBUS_CallbackTypeDef pCallback)

Function description

Register a User SMBUS Callback To be used instead of the weak predefined callback.

Parameters

- **hsmbus:** Pointer to a SMBUS_HandleTypeDef structure that contains the configuration information for the specified SMBUS.
- **CallbackID:** ID of the callback to be registered This parameter can be one of the following values:
 - HAL_SMBUS_MASTER_TX_COMPLETE_CB_ID Master Tx Transfer completed callback ID
 - HAL_SMBUS_MASTER_RX_COMPLETE_CB_ID Master Rx Transfer completed callback ID
 - HAL_SMBUS_SLAVE_TX_COMPLETE_CB_ID Slave Tx Transfer completed callback ID
 - HAL_SMBUS_SLAVE_RX_COMPLETE_CB_ID Slave Rx Transfer completed callback ID
 - HAL_SMBUS_LISTEN_COMPLETE_CB_ID Listen Complete callback ID
 - HAL_SMBUS_ERROR_CB_ID Error callback ID
 - HAL_SMBUS_ABORT_CB_ID Abort callback ID
 - HAL_SMBUS_MSPINIT_CB_ID MspInit callback ID
 - HAL_SMBUS_MSPDEINIT_CB_ID MspDeInit callback ID
- **pCallback:** pointer to the Callback function

Return values

- **HAL:** status

HAL_SMBUS_UnRegisterCallback

Function name

HAL_StatusTypeDef HAL_SMBUS_UnRegisterCallback (SMBUS_HandleTypeDef * hsmbus, HAL_SMBUS_CallbackIDTypeDef CallbackID)

Function description

Unregister an SMBUS Callback SMBUS callback is redirected to the weak predefined callback.

Parameters

- **hsmbus:** Pointer to a SMBUS_HandleTypeDef structure that contains the configuration information for the specified SMBUS.
- **CallbackID:** ID of the callback to be unregistered This parameter can be one of the following values: This parameter can be one of the following values:
 - HAL_SMBUS_MASTER_TX_COMPLETE_CB_ID Master Tx Transfer completed callback ID
 - HAL_SMBUS_MASTER_RX_COMPLETE_CB_ID Master Rx Transfer completed callback ID
 - HAL_SMBUS_SLAVE_TX_COMPLETE_CB_ID Slave Tx Transfer completed callback ID
 - HAL_SMBUS_SLAVE_RX_COMPLETE_CB_ID Slave Rx Transfer completed callback ID
 - HAL_SMBUS_LISTEN_COMPLETE_CB_ID Listen Complete callback ID
 - HAL_SMBUS_ERROR_CB_ID Error callback ID
 - HAL_SMBUS_ABORT_CB_ID Abort callback ID
 - HAL_SMBUS_MSPINIT_CB_ID MspInit callback ID
 - HAL_SMBUS_MSPDEINIT_CB_ID MspDeInit callback ID

Return values

- **HAL:** status

HAL_SMBUS_RegisterAddrCallback

Function name

HAL_StatusTypeDef HAL_SMBUS_RegisterAddrCallback (SMBUS_HandleTypeDef * hsmbus, pSMBUS_AddrCallbackTypeDef pCallback)

Function description

Register the Slave Address Match SMBUS Callback To be used instead of the weak HAL_SMBUS_AddrCallback() predefined callback.

Parameters

- **hsmbus:** Pointer to a SMBUS_HandleTypeDef structure that contains the configuration information for the specified SMBUS.
- **pCallback:** pointer to the Address Match Callback function

Return values

- **HAL:** status

HAL_SMBUS_UnRegisterAddrCallback

Function name

HAL_StatusTypeDef HAL_SMBUS_UnRegisterAddrCallback (SMBUS_HandleTypeDef * hsmbus)

Function description

UnRegister the Slave Address Match SMBUS Callback Info Ready SMBUS Callback is redirected to the weak HAL_SMBUS_AddrCallback() predefined callback.

Parameters

- **hsmbus:** Pointer to a SMBUS_HandleTypeDef structure that contains the configuration information for the specified SMBUS.

Return values

- **HAL:** status

HAL_SMBUS_IsDeviceReady

Function name

HAL_StatusTypeDef HAL_SMBUS_IsDeviceReady (SMBUS_HandleTypeDef * hsmbus, uint16_t DevAddress, uint32_t Trials, uint32_t Timeout)

Function description

Check if target device is ready for communication.

Parameters

- **hsmbus:** Pointer to a SMBUS_HandleTypeDef structure that contains the configuration information for the specified SMBUS.
- **DevAddress:** Target device address The device 7 bits address value in datasheet must be shifted to the left before calling the interface
- **Trials:** Number of trials
- **Timeout:** Timeout duration

Return values

- **HAL:** status

HAL_SMBUS_Master_Transmit_IT

Function name

HAL_StatusTypeDef HAL_SMBUS_Master_Transmit_IT (SMBUS_HandleTypeDef * hsmbus, uint16_t DevAddress, uint8_t * pData, uint16_t Size, uint32_t XferOptions)

Function description

Transmits in master mode an amount of data in blocking mode.

Parameters

- **hsmbus:** Pointer to a SMBUS_HandleTypeDef structure that contains the configuration information for the specified SMBUS.
- **DevAddress:** Target device address The device 7 bits address value in datasheet must be shifted to the left before calling the interface
- **pData:** Pointer to data buffer
- **Size:** Amount of data to be sent
- **XferOptions:** Options of Transfer

Return values

- **HAL:** status

HAL_SMBUS_Master_Receive_IT

Function name

HAL_StatusTypeDef HAL_SMBUS_Master_Receive_IT (SMBUS_HandleTypeDef * hsmbus, uint16_t DevAddress, uint8_t * pData, uint16_t Size, uint32_t XferOptions)

Function description

Receive in master/host SMBUS mode an amount of data in non blocking mode with Interrupt.

Parameters

- **hsmbus:** Pointer to a SMBUS_HandleTypeDef structure that contains the configuration information for the specified SMBUS.
- **DevAddress:** Target device address The device 7 bits address value in datasheet must be shifted to the left before calling the interface
- **pData:** Pointer to data buffer
- **Size:** Amount of data to be sent
- **XferOptions:** Options of Transfer, value of SMBUS XferOptions definition

Return values

- **HAL:** status

HAL_SMBUS_Master_Abort_IT

Function name

HAL_StatusTypeDef HAL_SMBUS_Master_Abort_IT (SMBUS_HandleTypeDef * hsmbus, uint16_t DevAddress)

Function description

Abort a master/host SMBUS process communication with Interrupt.

Parameters

- **hsmbus:** Pointer to a SMBUS_HandleTypeDef structure that contains the configuration information for the specified SMBUS.
- **DevAddress:** Target device address The device 7 bits address value in datasheet must be shifted to the left before calling the interface

Return values

- **HAL:** status

Notes

- This abort can be called only if state is ready

HAL_SMBUS_Slave_Transmit_IT

Function name

HAL_StatusTypeDef HAL_SMBUS_Slave_Transmit_IT (SMBUS_HandleTypeDef * hsmbus, uint8_t * pData, uint16_t Size, uint32_t XferOptions)

Function description

Transmit in slave/device SMBUS mode an amount of data in non blocking mode with Interrupt.

Parameters

- **hsmbus:** Pointer to a SMBUS_HandleTypeDef structure that contains the configuration information for the specified SMBUS.
- **pData:** Pointer to data buffer
- **Size:** Amount of data to be sent
- **XferOptions:** Options of Transfer, value of SMBUS XferOptions definition

Return values

- **HAL:** status

HAL_SMBUS_Slave_Receive_IT

Function name

HAL_StatusTypeDef HAL_SMBUS_Slave_Receive_IT (SMBUS_HandleTypeDef * hsmbus, uint8_t * pData, uint16_t Size, uint32_t XferOptions)

Function description

Enable the Address listen mode with Interrupt.

Parameters

- **hsmbus**: Pointer to a SMBUS_HandleTypeDef structure that contains the configuration information for the specified SMBUS.
- **pData**: Pointer to data buffer
- **Size**: Amount of data to be sent
- **XferOptions**: Options of Transfer, value of SMBUS XferOptions definition

Return values

- **HAL**: status

HAL_SMBUS_EnableAlert_IT

Function name

HAL_StatusTypeDef HAL_SMBUS_EnableAlert_IT (SMBUS_HandleTypeDef * hsmbus)

Function description

Enable the SMBUS alert mode with Interrupt.

Parameters

- **hsmbus**: Pointer to a SMBUS_HandleTypeDef structure that contains the configuration information for the specified SMBUSx peripheral.

Return values

- **HAL**: status

HAL_SMBUS_DisableAlert_IT

Function name

HAL_StatusTypeDef HAL_SMBUS_DisableAlert_IT (SMBUS_HandleTypeDef * hsmbus)

Function description

Disable the SMBUS alert mode with Interrupt.

Parameters

- **hsmbus**: Pointer to a SMBUS_HandleTypeDef structure that contains the configuration information for the specified SMBUSx peripheral.

Return values

- **HAL**: status

HAL_SMBUS_EnableListen_IT

Function name

HAL_StatusTypeDef HAL_SMBUS_EnableListen_IT (SMBUS_HandleTypeDef * hsmbus)

Function description

Enable the Address listen mode with Interrupt.

Parameters

- **hsmbus**: Pointer to a SMBUS_HandleTypeDef structure that contains the configuration information for the specified SMBUS.

Return values

- **HAL**: status

HAL_SMBUS_DisableListen_IT

Function name

HAL_StatusTypeDef HAL_SMBUS_DisableListen_IT (SMBUS_HandleTypeDef * hsmbus)

Function description

Disable the Address listen mode with Interrupt.

Parameters

- **hsmbus:** Pointer to a SMBUS_HandleTypeDef structure that contains the configuration information for the specified SMBUS.

Return values

- **HAL:** status

HAL_SMBUS_ConfigAnalogFilter

Function name

HAL_StatusTypeDef HAL_SMBUS_ConfigAnalogFilter (SMBUS_HandleTypeDef * hsmbus, uint32_t AnalogFilter)

Function description

Configures SMBUS Analog noise filter.

Parameters

- **hsmbus:** pointer to a SMBUS_HandleTypeDef structure that contains the configuration information for the specified SMBUSx peripheral.
- **AnalogFilter:** new state of the Analog filter.

Return values

- **HAL:** status

HAL_SMBUS_ConfigDigitalFilter

Function name

HAL_StatusTypeDef HAL_SMBUS_ConfigDigitalFilter (SMBUS_HandleTypeDef * hsmbus, uint32_t DigitalFilter)

Function description

Configures SMBUS Digital noise filter.

Parameters

- **hsmbus:** pointer to a SMBUS_HandleTypeDef structure that contains the configuration information for the specified SMBUSx peripheral.
- **DigitalFilter:** Coefficient of digital noise filter between 0x00 and 0x0F.

Return values

- **HAL:** status

HAL_SMBUS_EV_IRQHandler

Function name

void HAL_SMBUS_EV_IRQHandler (SMBUS_HandleTypeDef * hsmbus)

Function description

This function handles SMBUS event interrupt request.

Parameters

- **hsmbus:** Pointer to a SMBUS_HandleTypeDef structure that contains the configuration information for the specified SMBUS.

Return values

- **None:**

HAL_SMBUS_ER_IRQHandler

Function name

void HAL_SMBUS_ER_IRQHandler (SMBUS_HandleTypeDef * hsmbus)

Function description

This function handles SMBUS error interrupt request.

Parameters

- **hsmbus:** Pointer to a SMBUS_HandleTypeDef structure that contains the configuration information for the specified SMBUS.

Return values

- **None:**

HAL_SMBUS_MasterTxCpltCallback

Function name

void HAL_SMBUS_MasterTxCpltCallback (SMBUS_HandleTypeDef * hsmbus)

Function description

Master Tx Transfer completed callback.

Parameters

- **hsmbus:** Pointer to a SMBUS_HandleTypeDef structure that contains the configuration information for the specified SMBUS.

Return values

- **None:**

HAL_SMBUS_MasterRxCpltCallback

Function name

void HAL_SMBUS_MasterRxCpltCallback (SMBUS_HandleTypeDef * hsmbus)

Function description

Master Rx Transfer completed callback.

Parameters

- **hsmbus:** Pointer to a SMBUS_HandleTypeDef structure that contains the configuration information for the specified SMBUS.

Return values

- **None:**

HAL_SMBUS_SlaveTxCpltCallback

Function name

void HAL_SMBUS_SlaveTxCpltCallback (SMBUS_HandleTypeDef * hsmbus)

Function description

Slave Tx Transfer completed callback.

Parameters

- **hsmbus:** Pointer to a SMBUS_HandleTypeDef structure that contains the configuration information for the specified SMBUS.

Return values

- **None:**

HAL_SMBUS_SlaveRxCpltCallback

Function name

void HAL_SMBUS_SlaveRxCpltCallback (SMBUS_HandleTypeDef * hsmbus)

Function description

Slave Rx Transfer completed callback.

Parameters

- **hsmbus:** Pointer to a SMBUS_HandleTypeDef structure that contains the configuration information for the specified SMBUS.

Return values

- **None:**

HAL_SMBUS_AddrCallback

Function name

void HAL_SMBUS_AddrCallback (SMBUS_HandleTypeDef * hsmbus, uint8_t TransferDirection, uint16_t AddrMatchCode)

Function description

Slave Address Match callback.

Parameters

- **hsmbus:** Pointer to a SMBUS_HandleTypeDef structure that contains the configuration information for the specified SMBUS.
- **TransferDirection:** Master request Transfer Direction (Write/Read), value of SMBUS XferOptions definition
- **AddrMatchCode:** Address Match Code

Return values

- **None:**

HAL_SMBUS_ListenCpltCallback

Function name

void HAL_SMBUS_ListenCpltCallback (SMBUS_HandleTypeDef * hsmbus)

Function description

Listen Complete callback.

Parameters

- **hsmbus:** Pointer to a SMBUS_HandleTypeDef structure that contains the configuration information for the specified SMBUS.

Return values

- **None:**

HAL_SMBUS_ErrorCallback

Function name

```
void HAL_SMBUS_ErrorCallback (SMBUS_HandleTypeDef * hsmbus)
```

Function description

SMBUS error callback.

Parameters

- **hsmbus:** Pointer to a SMBUS_HandleTypeDef structure that contains the configuration information for the specified SMBUS.

Return values

- **None:**

HAL_SMBUS_AbortCpltCallback

Function name

```
void HAL_SMBUS_AbortCpltCallback (SMBUS_HandleTypeDef * hsmbus)
```

Function description

SMBUS abort callback.

Parameters

- **hsmbus:** Pointer to a SMBUS_HandleTypeDef structure that contains the configuration information for the specified SMBUS.

Return values

- **None:**

HAL_SMBUS_GetState

Function name

```
HAL_SMBUS_StateTypeDef HAL_SMBUS_GetState (SMBUS_HandleTypeDef * hsmbus)
```

Function description

Return the SMBUS handle state.

Parameters

- **hsmbus:** Pointer to a SMBUS_HandleTypeDef structure that contains the configuration information for the specified SMBUS.

Return values

- **HAL:** state

HAL_SMBUS_GetMode

Function name

```
HAL_SMBUS_ModeTypeDef HAL_SMBUS_GetMode (SMBUS_HandleTypeDef * hsmbus)
```

Function description

Return the SMBUS Master, Slave or no mode.

Parameters

- **hsmbus:** Pointer to a SMBUS_HandleTypeDef structure that contains the configuration information for SMBUS module

Return values

- **HAL:** mode

HAL_SMBUS_GetError

Function name

uint32_t HAL_SMBUS_GetError (SMBUS_HandleTypeDef * hsmbus)

Function description

Return the SMBUS error code.

Parameters

- **hsmbus:** Pointer to a SMBUS_HandleTypeDef structure that contains the configuration information for the specified SMBUS.

Return values

- **SMBUS:** Error Code

64.3 SMBUS Firmware driver defines

The following section lists the various define and macros of the module.

64.3.1 SMBUS

SMBUS

SMBUS addressing mode

SMBUS_ADDRESSINGMODE_7BIT

SMBUS_ADDRESSINGMODE_10BIT

SMBUS Analog Filter

SMBUS_ANALOGFILTER_ENABLE

SMBUS_ANALOGFILTER_DISABLE

SMBUS dual addressing mode

SMBUS_DUALADDRESS_DISABLE

SMBUS_DUALADDRESS_ENABLE

SMBUS Error Code

HAL_SMBUS_ERROR_NONE

No error

HAL_SMBUS_ERROR_BERR

BERR error

HAL_SMBUS_ERROR_ARLO

ARLO error

HAL_SMBUS_ERROR_AF

AF error

HAL_SMBUS_ERROR_OVR

OVR error

HAL_SMBUS_ERROR_TIMEOUT

Timeout Error

HAL_SMBUS_ERROR_ALERT

Alert error

HAL_SMBUS_ERROR_PECERR

PEC error

HAL_SMBUS_ERROR_INVALID_CALLBACK

Invalid Callback error

SMBUS Exported Macros

__HAL_SMBUS_RESET_HANDLE_STATE

Description:

- Reset SMBUS handle state.

Parameters:

- `__HANDLE__`: specifies the SMBUS Handle. This parameter can be SMBUS where x: 1, 2, or 3 to select the SMBUS peripheral.

Return value:

- None

__HAL_SMBUS_ENABLE_IT

Description:

- Enable or disable the specified SMBUS interrupts.

Parameters:

- `__HANDLE__`: specifies the SMBUS Handle. This parameter can be SMBUS where x: 1, 2, or 3 to select the SMBUS peripheral.
- `__INTERRUPT__`: specifies the interrupt source to enable or disable. This parameter can be one of the following values:
 - SMBUS_IT_BUF: Buffer interrupt enable
 - SMBUS_IT_EVT: Event interrupt enable
 - SMBUS_IT_ERR: Error interrupt enable

Return value:

- None

__HAL_SMBUS_DISABLE_IT

__HAL_SMBUS_GET_IT_SOURCE

Description:

- Checks if the specified SMBUS interrupt source is enabled or disabled.

Parameters:

- `__HANDLE__`: specifies the SMBUS Handle. This parameter can be SMBUS where x: 1, 2, or 3 to select the SMBUS peripheral.
- `__INTERRUPT__`: specifies the SMBUS interrupt source to check. This parameter can be one of the following values:
 - `SMBUS_IT_BUF`: Buffer interrupt enable
 - `SMBUS_IT_EVT`: Event interrupt enable
 - `SMBUS_IT_ERR`: Error interrupt enable

Return value:

- The: new state of `__INTERRUPT__` (TRUE or FALSE).

__HAL_SMBUS_GET_FLAG

Description:

- Checks whether the specified SMBUS flag is set or not.

Parameters:

- `__HANDLE__`: specifies the SMBUS Handle. This parameter can be SMBUS where x: 1, 2, or 3 to select the SMBUS peripheral.
- `__FLAG__`: specifies the flag to check. This parameter can be one of the following values:
 - `SMBUS_FLAG_SMBALERT`: SMBus Alert flag
 - `SMBUS_FLAG_TIMEOUT`: Timeout or Tlow error flag
 - `SMBUS_FLAG_PECERR`: PEC error in reception flag
 - `SMBUS_FLAG_OVR`: Overrun/Underrun flag
 - `SMBUS_FLAG_AF`: Acknowledge failure flag
 - `SMBUS_FLAG_ARLO`: Arbitration lost flag
 - `SMBUS_FLAG_BERR`: Bus error flag
 - `SMBUS_FLAG_TXE`: Data register empty flag
 - `SMBUS_FLAG_RXNE`: Data register not empty flag
 - `SMBUS_FLAG_STOPF`: Stop detection flag
 - `SMBUS_FLAG_ADD10`: 10-bit header sent flag
 - `SMBUS_FLAG_BTF`: Byte transfer finished flag
 - `SMBUS_FLAG_ADDR`: Address sent flag Address matched flag
 - `SMBUS_FLAG_SB`: Start bit flag
 - `SMBUS_FLAG_DUALF`: Dual flag
 - `SMBUS_FLAG_SMBHOST`: SMBus host header
 - `SMBUS_FLAG_SMBDEFAULT`: SMBus default header
 - `SMBUS_FLAG_GENCALL`: General call header flag
 - `SMBUS_FLAG_TRA`: Transmitter/Receiver flag
 - `SMBUS_FLAG_BUSY`: Bus busy flag
 - `SMBUS_FLAG_MSL`: Master/Slave flag

Return value:

- The: new state of `__FLAG__` (TRUE or FALSE).

__HAL_SMBUS_CLEAR_FLAG

Description:

- Clears the SMBUS pending flags which are cleared by writing 0 in a specific bit.

Parameters:

- `__HANDLE__`: specifies the SMBUS Handle. This parameter can be SMBUS where x: 1, 2, or 3 to select the SMBUS peripheral.
- `__FLAG__`: specifies the flag to clear. This parameter can be any combination of the following values:
 - `SMBUS_FLAG_SMBALERT`: Smbus Alert flag
 - `SMBUS_FLAG_TIMEOUT`: Timeout or Tlow error flag
 - `SMBUS_FLAG_PECERR`: PEC error in reception flag
 - `SMBUS_FLAG_OVR`: Overrun/Underrun flag (Slave mode)
 - `SMBUS_FLAG_AF`: Acknowledge failure flag
 - `SMBUS_FLAG_ARLO`: Arbitration lost flag (Master mode)
 - `SMBUS_FLAG_BERR`: Bus error flag

Return value:

- None

__HAL_SMBUS_CLEAR_ADDRFLAG

Description:

- Clears the SMBUS ADDR pending flag.

Parameters:

- `__HANDLE__`: specifies the SMBUS Handle. This parameter can be SMBUS where x: 1, 2, or 3 to select the SMBUS peripheral.

Return value:

- None

__HAL_SMBUS_CLEAR_STOPFLAG

Description:

- Clears the SMBUS STOPF pending flag.

Parameters:

- `__HANDLE__`: specifies the SMBUS Handle. This parameter can be SMBUS where x: 1, 2, or 3 to select the SMBUS peripheral.

Return value:

- None

__HAL_SMBUS_ENABLE

Description:

- Enable the SMBUS peripheral.

Parameters:

- `__HANDLE__`: specifies the SMBUS Handle. This parameter can be SMBUSx where x: 1 or 2 to select the SMBUS peripheral.

Return value:

- None

__HAL_SMBUS_DISABLE

Description:

- Disable the SMBUS peripheral.

Parameters:

- __HANDLE__: specifies the SMBUS Handle. This parameter can be SMBUSx where x: 1 or 2 to select the SMBUS peripheral.

Return value:

- None

__HAL_SMBUS_GENERATE_NACK

Description:

- Generate a Non-Acknowledge SMBUS peripheral in Slave mode.

Parameters:

- __HANDLE__: specifies the SMBUS Handle.

Return value:

- None

SMBUS Flag definition

SMBUS_FLAG_SMBALERT

SMBUS_FLAG_TIMEOUT

SMBUS_FLAG_PECERR

SMBUS_FLAG_OVR

SMBUS_FLAG_AF

SMBUS_FLAG_ARLO

SMBUS_FLAG_BERR

SMBUS_FLAG_TXE

SMBUS_FLAG_RXNE

SMBUS_FLAG_STOPF

SMBUS_FLAG_ADD10

SMBUS_FLAG_BTF

SMBUS_FLAG_ADDR

SMBUS_FLAG_SB

SMBUS_FLAG_DUALF

SMBUS_FLAG_SMBHOST

SMBUS_FLAG_SMBDEFAULT

SMBUS_FLAG_GENCALL

SMBUS_FLAG_TRA

SMBUS_FLAG_BUSY

SMBUS_FLAG_MSL

SMBUS general call addressing mode

SMBUS_GENERALCALL_DISABLE

SMBUS_GENERALCALL_ENABLE

SMBUS Interrupt configuration definition

SMBUS_IT_BUF

SMBUS_IT_EVT

SMBUS_IT_ERR

SMBUS nostretch mode

SMBUS_NOSTRETCH_DISABLE

SMBUS_NOSTRETCH_ENABLE

SMBUS packet error check mode

SMBUS_PEC_DISABLE

SMBUS_PEC_ENABLE

SMBUS peripheral mode

SMBUS_PERIPHERAL_MODE_SMBUS_HOST

SMBUS_PERIPHERAL_MODE_SMBUS_SLAVE

SMBUS_PERIPHERAL_MODE_SMBUS_SLAVE_ARP

SMBUS XferDirection definition

SMBUS_DIRECTION_RECEIVE

SMBUS_DIRECTION_TRANSMIT

SMBUS XferOptions definition

SMBUS_FIRST_FRAME

SMBUS_NEXT_FRAME

SMBUS_FIRST_AND_LAST_FRAME_NO_PEC

SMBUS_LAST_FRAME_NO_PEC



SMBUS_FIRST_AND_LAST_FRAME_WITH_PEC

SMBUS_LAST_FRAME_WITH_PEC

66 HAL SPI Generic Driver

66.1 SPI Firmware driver registers structures

66.1.1 SPI_InitTypeDef

SPI_InitTypeDef is defined in the stm32f4xx_hal_spi.h

Data Fields

- **uint32_t Mode**
- **uint32_t Direction**
- **uint32_t DataSize**
- **uint32_t CLKPolarity**
- **uint32_t CLKPhase**
- **uint32_t NSS**
- **uint32_t BaudRatePrescaler**
- **uint32_t FirstBit**
- **uint32_t TIMode**
- **uint32_t CRCCalculation**
- **uint32_t CRCPolynomial**

Field Documentation

- **uint32_t SPI_InitTypeDef::Mode**
Specifies the SPI operating mode. This parameter can be a value of [SPI_Mode](#)
- **uint32_t SPI_InitTypeDef::Direction**
Specifies the SPI bidirectional mode state. This parameter can be a value of [SPI_Direction](#)
- **uint32_t SPI_InitTypeDef::DataSize**
Specifies the SPI data size. This parameter can be a value of [SPI_Data_Size](#)
- **uint32_t SPI_InitTypeDef::CLKPolarity**
Specifies the serial clock steady state. This parameter can be a value of [SPI_Clock_Polarity](#)
- **uint32_t SPI_InitTypeDef::CLKPhase**
Specifies the clock active edge for the bit capture. This parameter can be a value of [SPI_Clock_Phase](#)
- **uint32_t SPI_InitTypeDef::NSS**
Specifies whether the NSS signal is managed by hardware (NSS pin) or by software using the SSI bit. This parameter can be a value of [SPI_Slave_Select_management](#)
- **uint32_t SPI_InitTypeDef::BaudRatePrescaler**
Specifies the Baud Rate prescaler value which will be used to configure the transmit and receive SCK clock. This parameter can be a value of [SPI_BaudRate_Prescaler](#)
Note:
– The communication clock is derived from the master clock. The slave clock does not need to be set.
- **uint32_t SPI_InitTypeDef::FirstBit**
Specifies whether data transfers start from MSB or LSB bit. This parameter can be a value of [SPI_MSB_LSB_transmission](#)
- **uint32_t SPI_InitTypeDef::TIMode**
Specifies if the TI mode is enabled or not. This parameter can be a value of [SPI_TI_mode](#)
- **uint32_t SPI_InitTypeDef::CRCCalculation**
Specifies if the CRC calculation is enabled or not. This parameter can be a value of [SPI_CRC_Calculation](#)
- **uint32_t SPI_InitTypeDef::CRCPolynomial**
Specifies the polynomial used for the CRC calculation. This parameter must be an odd number between Min_Data = 1 and Max_Data = 65535

66.1.2 __SPI_HandleTypeDef

__SPI_HandleTypeDef is defined in the stm32f4xx_hal_spi.h

Data Fields

- ***SPI_TypeDef * Instance***
- ***SPI_InitTypeDef Init***
- ***uint8_t * pTxBuffPtr***
- ***uint16_t TxXferSize***
- ***__IO uint16_t TxXferCount***
- ***uint8_t * pRxBuffPtr***
- ***uint16_t RxXferSize***
- ***__IO uint16_t RxXferCount***
- ***void(* RxISR***
- ***void(* TxISR***
- ***DMA_HandleTypeDef * hdmatrix***
- ***DMA_HandleTypeDef * hdmatrix***
- ***HAL_LockTypeDef Lock***
- ***__IO HAL_SPI_StateTypeDef State***
- ***__IO uint32_t ErrorCode***
- ***void(* TxCpltCallback***
- ***void(* RxCpltCallback***
- ***void(* TxRxCpltCallback***
- ***void(* TxHalfCpltCallback***
- ***void(* RxHalfCpltCallback***
- ***void(* TxRxHalfCpltCallback***
- ***void(* ErrorCallback***
- ***void(* AbortCpltCallback***
- ***void(* MspInitCallback***
- ***void(* MspDeInitCallback***

Field Documentation

- ***SPI_TypeDef* __SPI_HandleTypeDef::Instance***
SPI registers base address
- ***SPI_InitTypeDef __SPI_HandleTypeDef::Init***
SPI communication parameters
- ***uint8_t* __SPI_HandleTypeDef::pTxBuffPtr***
Pointer to SPI Tx transfer Buffer
- ***uint16_t __SPI_HandleTypeDef::TxXferSize***
SPI Tx Transfer size
- ***__IO uint16_t __SPI_HandleTypeDef::TxXferCount***
SPI Tx Transfer Counter
- ***uint8_t* __SPI_HandleTypeDef::pRxBuffPtr***
Pointer to SPI Rx transfer Buffer
- ***uint16_t __SPI_HandleTypeDef::RxXferSize***
SPI Rx Transfer size
- ***__IO uint16_t __SPI_HandleTypeDef::RxXferCount***
SPI Rx Transfer Counter
- ***void(* __SPI_HandleTypeDef::RxISR)(struct __SPI_HandleTypeDef *hspl)***
function pointer on Rx ISR
- ***void(* __SPI_HandleTypeDef::TxISR)(struct __SPI_HandleTypeDef *hspl)***
function pointer on Tx ISR
- ***DMA_HandleTypeDef* __SPI_HandleTypeDef::hdmatrix***
SPI Tx DMA Handle parameters
- ***DMA_HandleTypeDef* __SPI_HandleTypeDef::hdmatrix***
SPI Rx DMA Handle parameters
- ***HAL_LockTypeDef __SPI_HandleTypeDef::Lock***
Locking object

- **__IO HAL_SPI_StateTypeDef __SPI_HandleTypeDef::State**
SPI communication state
- **__IO uint32_t __SPI_HandleTypeDef::ErrorCode**
SPI Error code
- **void(* __SPI_HandleTypeDef::TxCpltCallback)(struct __SPI_HandleTypeDef *hspi)**
SPI Tx Completed callback
- **void(* __SPI_HandleTypeDef::RxCpltCallback)(struct __SPI_HandleTypeDef *hspi)**
SPI Rx Completed callback
- **void(* __SPI_HandleTypeDef::TxRxCpltCallback)(struct __SPI_HandleTypeDef *hspi)**
SPI TxRx Completed callback
- **void(* __SPI_HandleTypeDef::TxHalfCpltCallback)(struct __SPI_HandleTypeDef *hspi)**
SPI Tx Half Completed callback
- **void(* __SPI_HandleTypeDef::RxHalfCpltCallback)(struct __SPI_HandleTypeDef *hspi)**
SPI Rx Half Completed callback
- **void(* __SPI_HandleTypeDef::TxRxHalfCpltCallback)(struct __SPI_HandleTypeDef *hspi)**
SPI TxRx Half Completed callback
- **void(* __SPI_HandleTypeDef::ErrorCallback)(struct __SPI_HandleTypeDef *hspi)**
SPI Error callback
- **void(* __SPI_HandleTypeDef::AbortCpltCallback)(struct __SPI_HandleTypeDef *hspi)**
SPI Abort callback
- **void(* __SPI_HandleTypeDef::MspInitCallback)(struct __SPI_HandleTypeDef *hspi)**
SPI Msp Init callback
- **void(* __SPI_HandleTypeDef::MspDeInitCallback)(struct __SPI_HandleTypeDef *hspi)**
SPI Msp DeInit callback

66.2 SPI Firmware driver API description

The following section lists the various functions of the SPI library.

66.2.1 How to use this driver

The SPI HAL driver can be used as follows:

1. Declare a SPI_HandleTypeDef handle structure, for example: SPI_HandleTypeDef hspi;
2. Initialize the SPI low level resources by implementing the HAL_SPI_MspInit() API:
 - a. Enable the SPIx interface clock
 - b. SPI pins configuration
 - Enable the clock for the SPI GPIOs
 - Configure these SPI pins as alternate function push-pull
 - c. NVIC configuration if you need to use interrupt process
 - Configure the SPIx interrupt priority
 - Enable the NVIC SPI IRQ handle
 - d. DMA Configuration if you need to use DMA process
 - Declare a DMA_HandleTypeDef handle structure for the transmit or receive Stream/Channel
 - Enable the DMAx clock
 - Configure the DMA handle parameters
 - Configure the DMA Tx or Rx Stream/Channel
 - Associate the initialized hdma_tx(or _rx) handle to the hspi DMA Tx or Rx handle
 - Configure the priority and enable the NVIC for the transfer complete interrupt on the DMA Tx or Rx Stream/Channel
3. Program the Mode, BidirectionalMode, Data size, Baudrate Prescaler, NSS management, Clock polarity and phase, FirstBit and CRC configuration in the hspi Init structure.
4. Initialize the SPI registers by calling the HAL_SPI_Init() API:
 - This API configures also the low level Hardware GPIO, CLOCK, CORTEX...etc) by calling the customized HAL_SPI_MspInit() API.

Circular mode restriction:

1. The DMA circular mode cannot be used when the SPI is configured in these modes:
 - a. Master 2Lines RxOnly
 - b. Master 1Line Rx
2. The CRC feature is not managed when the DMA circular mode is enabled
3. When the SPI DMA Pause/Stop features are used, we must use the following APIs the HAL_SPI_DMALPause()/ HAL_SPI_DMALStop() only under the SPI callbacks

Master Receive mode restriction:

1. In Master unidirectional receive-only mode (MSTR =1, BIDIMODE=0, RXONLY=1) or bidirectional receive mode (MSTR=1, BIDIMODE=1, BIDIOE=0), to ensure that the SPI does not initiate a new transfer the following procedure has to be respected:
 - a. HAL_SPI_DeInit()
 - b. HAL_SPI_Init()

Callback registration:

1. The compilation flag USE_HAL_SPI_REGISTER_CALLBACKS when set to 1U allows the user to configure dynamically the driver callbacks. Use Functions HAL_SPI_RegisterCallback() to register an interrupt callback. Function HAL_SPI_RegisterCallback() allows to register following callbacks:
 - TxCpltCallback : SPI Tx Completed callback
 - RxCpltCallback : SPI Rx Completed callback
 - TxCpltCallback : SPI TxRx Completed callback
 - TxHalfCpltCallback : SPI Tx Half Completed callback
 - RxHalfCpltCallback : SPI Rx Half Completed callback
 - TxRxHalfCpltCallback : SPI TxRx Half Completed callback
 - ErrorCallback : SPI Error callback
 - AbortCpltCallback : SPI Abort callback
 - MspInitCallback : SPI Msp Init callback
 - MspDeInitCallback : SPI Msp DeInit callback This function takes as parameters the HAL peripheral handle, the Callback ID and a pointer to the user callback function.
2. Use function HAL_SPI_UnRegisterCallback to reset a callback to the default weak function. HAL_SPI_UnRegisterCallback takes as parameters the HAL peripheral handle, and the Callback ID. This function allows to reset following callbacks:
 - TxCpltCallback : SPI Tx Completed callback
 - RxCpltCallback : SPI Rx Completed callback
 - TxCpltCallback : SPI TxRx Completed callback
 - TxHalfCpltCallback : SPI Tx Half Completed callback
 - RxHalfCpltCallback : SPI Rx Half Completed callback
 - TxRxHalfCpltCallback : SPI TxRx Half Completed callback
 - ErrorCallback : SPI Error callback
 - AbortCpltCallback : SPI Abort callback
 - MspInitCallback : SPI Msp Init callback
 - MspDeInitCallback : SPI Msp DeInit callback

By default, after the HAL_SPI_Init() and when the state is HAL_SPI_STATE_RESET all callbacks are set to the corresponding weak functions: examples HAL_SPI_MasterTxCpltCallback(), HAL_SPI_MasterRxCpltCallback(). Exception done for MspInit and MspDeInit functions that are reset to the legacy weak functions in the HAL_SPI_Init()/ HAL_SPI_DeInit() only when these callbacks are null (not registered beforehand). If MspInit or MspDeInit are not null, the HAL_SPI_Init()/ HAL_SPI_DeInit() keep and use the user MspInit/MspDeInit callbacks (registered beforehand) whatever the state.

Callbacks can be registered/unregistered in HAL_SPI_STATE_READY state only. Exception done MspInit/MspDeInit functions that can be registered/unregistered in HAL_SPI_STATE_READY or HAL_SPI_STATE_RESET state, thus registered (user) MspInit/DeInit callbacks can be used during the Init/DeInit. Then, the user first registers the MspInit/MspDeInit user callbacks using HAL_SPI_RegisterCallback() before calling HAL_SPI_DeInit() or HAL_SPI_Init() function.

When the compilation define USE_HAL_PPP_REGISTER_CALLBACKS is set to 0 or not defined, the callback registering feature is not available and weak (surcharged) callbacks are used.

Using the HAL it is not possible to reach all supported SPI frequency with the different SPI Modes, the following table resume the max SPI frequency reached with data size 8bits/16bits, according to frequency of the APBx Peripheral Clock (fPCLK) used by the SPI instance.

66.2.2 Initialization and de-initialization functions

This subsection provides a set of functions allowing to initialize and de-initialize the SPIx peripheral:

- User must implement HAL_SPI_MspInit() function in which he configures all related peripherals resources (CLOCK, GPIO, DMA, IT and NVIC).
- Call the function HAL_SPI_Init() to configure the selected device with the selected configuration:
 - Mode
 - Direction
 - Data Size
 - Clock Polarity and Phase
 - NSS Management
 - BaudRate Prescaler
 - FirstBit
 - TMode
 - CRC Calculation
 - CRC Polynomial if CRC enabled
- Call the function HAL_SPI_DeInit() to restore the default configuration of the selected SPIx peripheral.

This section contains the following APIs:

- [*HAL_SPI_Init\(\)*](#)
- [*HAL_SPI_DeInit\(\)*](#)
- [*HAL_SPI_MspInit\(\)*](#)
- [*HAL_SPI_MspDeInit\(\)*](#)
- [*HAL_SPI_RegisterCallback\(\)*](#)
- [*HAL_SPI_UnRegisterCallback\(\)*](#)

66.2.3 IO operation functions

This subsection provides a set of functions allowing to manage the SPI data transfers.

The SPI supports master and slave mode :

1. There are two modes of transfer:
 - Blocking mode: The communication is performed in polling mode. The HAL status of all data processing is returned by the same function after finishing transfer.
 - No-Blocking mode: The communication is performed using Interrupts or DMA, These APIs return the HAL status. The end of the data processing will be indicated through the dedicated SPI IRQ when using Interrupt mode or the DMA IRQ when using DMA mode. The HAL_SPI_TxCpltCallback(), HAL_SPI_RxCpltCallback() and HAL_SPI_TxRxCpltCallback() user callbacks will be executed respectively at the end of the transmit or Receive process The HAL_SPI_ErrorCallback()user callback will be executed when a communication error is detected
2. APIs provided for these 2 transfer modes (Blocking mode or Non blocking mode using either Interrupt or DMA) exist for 1Line (simplex) and 2Lines (full duplex) modes.

This section contains the following APIs:

- [*HAL_SPI_Transmit\(\)*](#)
- [*HAL_SPI_Receive\(\)*](#)
- [*HAL_SPI_TransmitReceive\(\)*](#)
- [*HAL_SPI_Transmit_IT\(\)*](#)
- [*HAL_SPI_Receive_IT\(\)*](#)
- [*HAL_SPI_TransmitReceive_IT\(\)*](#)
- [*HAL_SPI_Transmit_DMA\(\)*](#)
- [*HAL_SPI_Receive_DMA\(\)*](#)

- *HAL_SPI_TransmitReceive_DMA()*
- *HAL_SPI_Abort()*
- *HAL_SPI_Abort_IT()*
- *HAL_SPI_DMABase()*
- *HAL_SPI_DMAResume()*
- *HAL_SPI_DMAStop()*
- *HAL_SPI_IRQHandler()*
- *HAL_SPI_TxCpltCallback()*
- *HAL_SPI_RxCpltCallback()*
- *HAL_SPI_TxRxTxRxCpltCallback()*
- *HAL_SPI_TxHalfCpltCallback()*
- *HAL_SPI_RxHalfCpltCallback()*
- *HAL_SPI_TxRxHalfCpltCallback()*
- *HAL_SPI_ErrorCallback()*
- *HAL_SPI_AbortCpltCallback()*

66.2.4 Peripheral State and Errors functions

This subsection provides a set of functions allowing to control the SPI.

- *HAL_SPI_GetState()* API can be helpful to check in run-time the state of the SPI peripheral
- *HAL_SPI_GetError()* check in run-time Errors occurring during communication

This section contains the following APIs:

- *HAL_SPI_GetState()*
- *HAL_SPI_GetError()*

66.2.5 Detailed description of functions

HAL_SPI_Init

Function name

HAL_StatusTypeDef HAL_SPI_Init (SPI_HandleTypeDef * hspi)

Function description

Initialize the SPI according to the specified parameters in the SPI_InitTypeDef and initialize the associated handle.

Parameters

- **hspi**: pointer to a SPI_HandleTypeDef structure that contains the configuration information for SPI module.

Return values

- **HAL**: status

HAL_SPI_DeInit

Function name

HAL_StatusTypeDef HAL_SPI_DeInit (SPI_HandleTypeDef * hspi)

Function description

De-Initialize the SPI peripheral.

Parameters

- **hspi**: pointer to a SPI_HandleTypeDef structure that contains the configuration information for SPI module.

Return values

- **HAL:** status

HAL_SPI_MspInit

Function name

```
void HAL_SPI_MspInit (SPI_HandleTypeDef * hspi)
```

Function description

Initialize the SPI MSP.

Parameters

- **hspi:** pointer to a SPI_HandleTypeDef structure that contains the configuration information for SPI module.

Return values

- **None:**

HAL_SPI_MspDeInit

Function name

```
void HAL_SPI_MspDeInit (SPI_HandleTypeDef * hspi)
```

Function description

De-Initialize the SPI MSP.

Parameters

- **hspi:** pointer to a SPI_HandleTypeDef structure that contains the configuration information for SPI module.

Return values

- **None:**

HAL_SPI_RegisterCallback

Function name

```
HAL_StatusTypeDef HAL_SPI_RegisterCallback (SPI_HandleTypeDef * hspi,  
HAL_SPI_CallbackIDTypeDef CallbackID, pSPI_CallbackTypeDef pCallback)
```

Function description

Register a User SPI Callback To be used instead of the weak predefined callback.

Parameters

- **hspi:** Pointer to a SPI_HandleTypeDef structure that contains the configuration information for the specified SPI.
- **CallbackID:** ID of the callback to be registered
- **pCallback:** pointer to the Callback function

Return values

- **HAL:** status

HAL_SPI_UnRegisterCallback

Function name

```
HAL_StatusTypeDef HAL_SPI_UnRegisterCallback (SPI_HandleTypeDef * hspi,  
HAL_SPI_CallbackIDTypeDef CallbackID)
```

Function description

Unregister an SPI Callback SPI callback is redirected to the weak predefined callback.

Parameters

- **hspi**: Pointer to a SPI_HandleTypeDef structure that contains the configuration information for the specified SPI.
- **CallbackID**: ID of the callback to be unregistered

Return values

- **HAL**: status

HAL_SPI_Transmit

Function name

HAL_StatusTypeDef HAL_SPI_Transmit (SPI_HandleTypeDef * hspi, uint8_t * pData, uint16_t Size, uint32_t Timeout)

Function description

Transmit an amount of data in blocking mode.

Parameters

- **hspi**: pointer to a SPI_HandleTypeDef structure that contains the configuration information for SPI module.
- **pData**: pointer to data buffer
- **Size**: amount of data to be sent
- **Timeout**: Timeout duration

Return values

- **HAL**: status

HAL_SPI_Receive

Function name

HAL_StatusTypeDef HAL_SPI_Receive (SPI_HandleTypeDef * hspi, uint8_t * pData, uint16_t Size, uint32_t Timeout)

Function description

Receive an amount of data in blocking mode.

Parameters

- **hspi**: pointer to a SPI_HandleTypeDef structure that contains the configuration information for SPI module.
- **pData**: pointer to data buffer
- **Size**: amount of data to be received
- **Timeout**: Timeout duration

Return values

- **HAL**: status

HAL_SPI_TransmitReceive

Function name

HAL_StatusTypeDef HAL_SPI_TransmitReceive (SPI_HandleTypeDef * hspi, uint8_t * pTxData, uint8_t * pRxData, uint16_t Size, uint32_t Timeout)

Function description

Transmit and Receive an amount of data in blocking mode.

Parameters

- **hspi**: pointer to a SPI_HandleTypeDef structure that contains the configuration information for SPI module.
- **pTxData**: pointer to transmission data buffer
- **pRxData**: pointer to reception data buffer
- **Size**: amount of data to be sent and received
- **Timeout**: Timeout duration

Return values

- **HAL**: status

HAL_SPI_Transmit_IT

Function name

HAL_StatusTypeDef HAL_SPI_Transmit_IT (SPI_HandleTypeDef * hspi, uint8_t * pData, uint16_t Size)

Function description

Transmit an amount of data in non-blocking mode with Interrupt.

Parameters

- **hspi**: pointer to a SPI_HandleTypeDef structure that contains the configuration information for SPI module.
- **pData**: pointer to data buffer
- **Size**: amount of data to be sent

Return values

- **HAL**: status

HAL_SPI_Receive_IT

Function name

HAL_StatusTypeDef HAL_SPI_Receive_IT (SPI_HandleTypeDef * hspi, uint8_t * pData, uint16_t Size)

Function description

Receive an amount of data in non-blocking mode with Interrupt.

Parameters

- **hspi**: pointer to a SPI_HandleTypeDef structure that contains the configuration information for SPI module.
- **pData**: pointer to data buffer
- **Size**: amount of data to be sent

Return values

- **HAL**: status

HAL_SPI_TransmitReceive_IT

Function name

HAL_StatusTypeDef HAL_SPI_TransmitReceive_IT (SPI_HandleTypeDef * hspi, uint8_t * pTxData, uint8_t * pRxData, uint16_t Size)

Function description

Transmit and Receive an amount of data in non-blocking mode with Interrupt.

Parameters

- **hspi**: pointer to a SPI_HandleTypeDef structure that contains the configuration information for SPI module.
- **pTxData**: pointer to transmission data buffer
- **pRxData**: pointer to reception data buffer
- **Size**: amount of data to be sent and received

Return values

- **HAL**: status

HAL_SPI_Transmit_DMA

Function name

HAL_StatusTypeDef HAL_SPI_Transmit_DMA (SPI_HandleTypeDef * hspi, uint8_t * pData, uint16_t Size)

Function description

Transmit an amount of data in non-blocking mode with DMA.

Parameters

- **hspi**: pointer to a SPI_HandleTypeDef structure that contains the configuration information for SPI module.
- **pData**: pointer to data buffer
- **Size**: amount of data to be sent

Return values

- **HAL**: status

HAL_SPI_Receive_DMA

Function name

HAL_StatusTypeDef HAL_SPI_Receive_DMA (SPI_HandleTypeDef * hspi, uint8_t * pData, uint16_t Size)

Function description

Receive an amount of data in non-blocking mode with DMA.

Parameters

- **hspi**: pointer to a SPI_HandleTypeDef structure that contains the configuration information for SPI module.
- **pData**: pointer to data buffer
- **Size**: amount of data to be sent

Return values

- **HAL**: status

Notes

- In case of MASTER mode and SPI_DIRECTION_2LINES direction, hdmatx shall be defined.
- When the CRC feature is enabled the pData Length must be Size + 1.

HAL_SPI_TransmitReceive_DMA

Function name

HAL_StatusTypeDef HAL_SPI_TransmitReceive_DMA (SPI_HandleTypeDef * hspi, uint8_t * pTxData, uint8_t * pRxData, uint16_t Size)

Function description

Transmit and Receive an amount of data in non-blocking mode with DMA.

Parameters

- **hspi**: pointer to a SPI_HandleTypeDef structure that contains the configuration information for SPI module.
- **pTxData**: pointer to transmission data buffer
- **pRxData**: pointer to reception data buffer
- **Size**: amount of data to be sent

Return values

- **HAL**: status

Notes

- When the CRC feature is enabled the pRxData Length must be Size + 1

HAL_SPI_DMAPause

Function name

HAL_StatusTypeDef HAL_SPI_DMAPause (SPI_HandleTypeDef * hspi)

Function description

Pause the DMA Transfer.

Parameters

- **hspi**: pointer to a SPI_HandleTypeDef structure that contains the configuration information for the specified SPI module.

Return values

- **HAL**: status

HAL_SPI_DMAResume

Function name

HAL_StatusTypeDef HAL_SPI_DMAResume (SPI_HandleTypeDef * hspi)

Function description

Resume the DMA Transfer.

Parameters

- **hspi**: pointer to a SPI_HandleTypeDef structure that contains the configuration information for the specified SPI module.

Return values

- **HAL**: status

HAL_SPI_DMAStop

Function name

HAL_StatusTypeDef HAL_SPI_DMAStop (SPI_HandleTypeDef * hspi)

Function description

Stop the DMA Transfer.

Parameters

- **hspi**: pointer to a SPI_HandleTypeDef structure that contains the configuration information for the specified SPI module.

Return values

- **HAL**: status

HAL_SPI_Abort

Function name

HAL_StatusTypeDef HAL_SPI_Abort (SPI_HandleTypeDef * hspi)

Function description

Abort ongoing transfer (blocking mode).

Parameters

- **hspi**: SPI handle.

Return values

- **HAL**: status

Notes

- This procedure could be used for aborting any ongoing transfer (Tx and Rx), started in Interrupt or DMA mode. This procedure performs following operations : Disable SPI Interrupts (depending of transfer direction)Disable the DMA transfer in the peripheral register (if enabled)Abort DMA transfer by calling HAL_DMA_Abort (in case of transfer in DMA mode)Set handle State to READY
- This procedure is executed in blocking mode : when exiting function, Abort is considered as completed.

HAL_SPI_Abort_IT

Function name

HAL_StatusTypeDef HAL_SPI_Abort_IT (SPI_HandleTypeDef * hspi)

Function description

Abort ongoing transfer (Interrupt mode).

Parameters

- **hspi**: SPI handle.

Return values

- **HAL**: status

Notes

- This procedure could be used for aborting any ongoing transfer (Tx and Rx), started in Interrupt or DMA mode. This procedure performs following operations : Disable SPI Interrupts (depending of transfer direction)Disable the DMA transfer in the peripheral register (if enabled)Abort DMA transfer by calling HAL_DMA_Abort_IT (in case of transfer in DMA mode)Set handle State to READYAt abort completion, call user abort complete callback
- This procedure is executed in Interrupt mode, meaning that abort procedure could be considered as completed only when user abort complete callback is executed (not when exiting function).

HAL_SPI_IRQHandler

Function name

void HAL_SPI_IRQHandler (SPI_HandleTypeDef * hspi)

Function description

Handle SPI interrupt request.

Parameters

- **hspi**: pointer to a SPI_HandleTypeDef structure that contains the configuration information for the specified SPI module.

Return values

- **None:**

HAL_SPI_TxCpltCallback

Function name

void HAL_SPI_TxCpltCallback (SPI_HandleTypeDef * hspi)

Function description

Tx Transfer completed callback.

Parameters

- **hspi:** pointer to a SPI_HandleTypeDef structure that contains the configuration information for SPI module.

Return values

- **None:**

HAL_SPI_RxCpltCallback

Function name

void HAL_SPI_RxCpltCallback (SPI_HandleTypeDef * hspi)

Function description

Rx Transfer completed callback.

Parameters

- **hspi:** pointer to a SPI_HandleTypeDef structure that contains the configuration information for SPI module.

Return values

- **None:**

HAL_SPI_TxRxCpltCallback

Function name

void HAL_SPI_TxRxCpltCallback (SPI_HandleTypeDef * hspi)

Function description

Tx and Rx Transfer completed callback.

Parameters

- **hspi:** pointer to a SPI_HandleTypeDef structure that contains the configuration information for SPI module.

Return values

- **None:**

HAL_SPI_TxHalfCpltCallback

Function name

void HAL_SPI_TxHalfCpltCallback (SPI_HandleTypeDef * hspi)

Function description

Tx Half Transfer completed callback.

Parameters

- **hspi**: pointer to a SPI_HandleTypeDef structure that contains the configuration information for SPI module.

Return values

- **None**:

HAL_SPI_RxHalfCpltCallback

Function name

void HAL_SPI_RxHalfCpltCallback (SPI_HandleTypeDef * hspi)

Function description

Rx Half Transfer completed callback.

Parameters

- **hspi**: pointer to a SPI_HandleTypeDef structure that contains the configuration information for SPI module.

Return values

- **None**:

HAL_SPI_TxRxHalfCpltCallback

Function name

void HAL_SPI_TxRxHalfCpltCallback (SPI_HandleTypeDef * hspi)

Function description

Tx and Rx Half Transfer callback.

Parameters

- **hspi**: pointer to a SPI_HandleTypeDef structure that contains the configuration information for SPI module.

Return values

- **None**:

HAL_SPI_ErrorCallback

Function name

void HAL_SPI_ErrorCallback (SPI_HandleTypeDef * hspi)

Function description

SPI error callback.

Parameters

- **hspi**: pointer to a SPI_HandleTypeDef structure that contains the configuration information for SPI module.

Return values

- **None**:

HAL_SPI_AbortCpltCallback

Function name

void HAL_SPI_AbortCpltCallback (SPI_HandleTypeDef * hspi)

Function description

SPI Abort Complete callback.

Parameters

- **hspl:** SPI handle.

Return values

- **None:**

HAL_SPI_GetState

Function name

HAL_SPI_StateTypeDef HAL_SPI_GetState (SPI_HandleTypeDef * hspl)

Function description

Return the SPI handle state.

Parameters

- **hspl:** pointer to a SPI_HandleTypeDef structure that contains the configuration information for SPI module.

Return values

- **SPI:** state

HAL_SPI_GetError

Function name

uint32_t HAL_SPI_GetError (SPI_HandleTypeDef * hspl)

Function description

Return the SPI error code.

Parameters

- **hspl:** pointer to a SPI_HandleTypeDef structure that contains the configuration information for SPI module.

Return values

- **SPI:** error code in bitmap format

66.3 SPI Firmware driver defines

The following section lists the various define and macros of the module.

66.3.1

SPI

SPI

SPI BaudRate Prescaler

SPI_BAUDRATEPRESCALER_2

SPI_BAUDRATEPRESCALER_4

SPI_BAUDRATEPRESCALER_8

SPI_BAUDRATEPRESCALER_16

SPI_BAUDRATEPRESCALER_32

SPI_BAUDRATEPRESCALER_64

SPI_BAUDRATEPRESCALER_128

SPI_BAUDRATEPRESCALER_256

SPI Clock Phase

SPI_PHASE_1EDGE

SPI_PHASE_2EDGE

SPI Clock Polarity

SPI_POLARITY_LOW

SPI_POLARITY_HIGH

SPI CRC Calculation

SPI_CRCCALCULATION_DISABLE

SPI_CRCCALCULATION_ENABLE

SPI Data Size

SPI_DATASIZE_8BIT

SPI_DATASIZE_16BIT

SPI Direction Mode

SPI_DIRECTION_2LINES

SPI_DIRECTION_2LINES_RXONLY

SPI_DIRECTION_1LINE

SPI Error Code

HAL_SPI_ERROR_NONE

No error

HAL_SPI_ERROR_MODF

MODF error

HAL_SPI_ERROR_CRC

CRC error

HAL_SPI_ERROR_OVR

OVR error

HAL_SPI_ERROR_FRE

FRE error

HAL_SPI_ERROR_DMA

DMA transfer error

HAL_SPI_ERROR_FLAG

Error on RXNE/TXE/BSY Flag

HAL_SPI_ERROR_ABORT

Error during SPI Abort procedure

HAL_SPI_ERROR_INVALID_CALLBACK

Invalid Callback error

SPI Exported Macros

__HAL_SPI_RESET_HANDLE_STATE

Description:

- Reset SPI handle state.

Parameters:

- `__HANDLE__`: specifies the SPI Handle. This parameter can be SPI where x: 1, 2, or 3 to select the SPI peripheral.

Return value:

- None

__HAL_SPI_ENABLE_IT

Description:

- Enable the specified SPI interrupts.

Parameters:

- `__HANDLE__`: specifies the SPI Handle. This parameter can be SPI where x: 1, 2, or 3 to select the SPI peripheral.
- `__INTERRUPT__`: specifies the interrupt source to enable. This parameter can be one of the following values:
 - `SPI_IT_TXE`: Tx buffer empty interrupt enable
 - `SPI_IT_RXNE`: RX buffer not empty interrupt enable
 - `SPI_IT_ERR`: Error interrupt enable

Return value:

- None

__HAL_SPI_DISABLE_IT

Description:

- Disable the specified SPI interrupts.

Parameters:

- `__HANDLE__`: specifies the SPI handle. This parameter can be SPIx where x: 1, 2, or 3 to select the SPI peripheral.
- `__INTERRUPT__`: specifies the interrupt source to disable. This parameter can be one of the following values:
 - `SPI_IT_TXE`: Tx buffer empty interrupt enable
 - `SPI_IT_RXNE`: RX buffer not empty interrupt enable
 - `SPI_IT_ERR`: Error interrupt enable

Return value:

- None

__HAL_SPI_GET_IT_SOURCE

Description:

- Check whether the specified SPI interrupt source is enabled or not.

Parameters:

- `__HANDLE__`: specifies the SPI Handle. This parameter can be SPI where x: 1, 2, or 3 to select the SPI peripheral.
- `__INTERRUPT__`: specifies the SPI interrupt source to check. This parameter can be one of the following values:
 - `SPI_IT_TXE`: Tx buffer empty interrupt enable
 - `SPI_IT_RXNE`: RX buffer not empty interrupt enable
 - `SPI_IT_ERR`: Error interrupt enable

Return value:

- The: new state of `__IT__` (TRUE or FALSE).

__HAL_SPI_GET_FLAG

Description:

- Check whether the specified SPI flag is set or not.

Parameters:

- `__HANDLE__`: specifies the SPI Handle. This parameter can be SPI where x: 1, 2, or 3 to select the SPI peripheral.
- `__FLAG__`: specifies the flag to check. This parameter can be one of the following values:
 - `SPI_FLAG_RXNE`: Receive buffer not empty flag
 - `SPI_FLAG_TXE`: Transmit buffer empty flag
 - `SPI_FLAG_CRCERR`: CRC error flag
 - `SPI_FLAG_MODF`: Mode fault flag
 - `SPI_FLAG_OVR`: Overrun flag
 - `SPI_FLAG_BSY`: Busy flag
 - `SPI_FLAG_FRE`: Frame format error flag

Return value:

- The: new state of `__FLAG__` (TRUE or FALSE).

__HAL_SPI_CLEAR_CRCERRFLAG

Description:

- Clear the SPI CRCERR pending flag.

Parameters:

- `__HANDLE__`: specifies the SPI Handle. This parameter can be SPI where x: 1, 2, or 3 to select the SPI peripheral.

Return value:

- None

__HAL_SPI_CLEAR_MODFFLAG

Description:

- Clear the SPI MODF pending flag.

Parameters:

- `__HANDLE__`: specifies the SPI Handle. This parameter can be SPI where x: 1, 2, or 3 to select the SPI peripheral.

Return value:

- None

__HAL_SPI_CLEAR_OVRFLAG

Description:

- Clear the SPI OVR pending flag.

Parameters:

- `__HANDLE__`: specifies the SPI Handle. This parameter can be SPI where x: 1, 2, or 3 to select the SPI peripheral.

Return value:

- None

__HAL_SPI_CLEAR_FREFLAG

Description:

- Clear the SPI FRE pending flag.

Parameters:

- `__HANDLE__`: specifies the SPI Handle. This parameter can be SPI where x: 1, 2, or 3 to select the SPI peripheral.

Return value:

- None

__HAL_SPI_ENABLE

Description:

- Enable the SPI peripheral.

Parameters:

- `__HANDLE__`: specifies the SPI Handle. This parameter can be SPI where x: 1, 2, or 3 to select the SPI peripheral.

Return value:

- None

__HAL_SPI_DISABLE

Description:

- Disable the SPI peripheral.

Parameters:

- `__HANDLE__`: specifies the SPI Handle. This parameter can be SPI where x: 1, 2, or 3 to select the SPI peripheral.

Return value:

- None

SPI Flags Definition

SPI_FLAG_RXNE

SPI_FLAG_TXE

SPI_FLAG_BSY

SPI_FLAG_CRCERR

SPI_FLAG_MODF

SPI_FLAG_OVR

SPI_FLAG_FRE

SPI_FLAG_MASK

SPI Interrupt Definition

SPI_IT_TXE

SPI_IT_RXNE

SPI_IT_ERR

SPI Mode

SPI_MODE_SLAVE

SPI_MODE_MASTER

SPI MSB LSB Transmission

SPI_FIRSTBIT_MSB

SPI_FIRSTBIT_LSB

SPI Slave Select Management

SPI_NSS_SOFT

SPI_NSS_HARD_INPUT

SPI_NSS_HARD_OUTPUT

SPI TI Mode

SPI_TIMODE_DISABLE

SPI_TIMODE_ENABLE

67 HAL SRAM Generic Driver

67.1 SRAM Firmware driver registers structures

67.1.1 SRAM_HandleTypeDef

SRAM_HandleTypeDef is defined in the stm32f4xx_hal_sram.h

Data Fields

- **FMC_NORSRAM_TypeDef * Instance**
- **FMC_NORSRAM_EXTENDED_TypeDef * Extended**
- **FMC_NORSRAM_InitTypeDef Init**
- **HAL_LockTypeDef Lock**
- **__IO HAL_SRAM_StateTypeDef State**
- **DMA_HandleTypeDef * hdma**

Field Documentation

- **FMC_NORSRAM_TypeDef* SRAM_HandleTypeDef::Instance**
Register base address
- **FMC_NORSRAM_EXTENDED_TypeDef* SRAM_HandleTypeDef::Extended**
Extended mode register base address
- **FMC_NORSRAM_InitTypeDef SRAM_HandleTypeDef::Init**
SRAM device control configuration parameters
- **HAL_LockTypeDef SRAM_HandleTypeDef::Lock**
SRAM locking object
- **__IO HAL_SRAM_StateTypeDef SRAM_HandleTypeDef::State**
SRAM device access state
- **DMA_HandleTypeDef* SRAM_HandleTypeDef::hdma**
Pointer DMA handler

67.2 SRAM Firmware driver API description

The following section lists the various functions of the SRAM library.

67.2.1 How to use this driver

This driver is a generic layered driver which contains a set of APIs used to control SRAM memories. It uses the FMC layer functions to interface with SRAM devices. The following sequence should be followed to configure the FMC/FSMC to interface with SRAM/PSRAM memories:

1. Declare a SRAM_HandleTypeDef handle structure, for example: SRAM_HandleTypeDef hsram; and:
 - Fill the SRAM_HandleTypeDef handle "Init" field with the allowed values of the structure member.
 - Fill the SRAM_HandleTypeDef handle "Instance" field with a predefined base register instance for NOR or SRAM device
 - Fill the SRAM_HandleTypeDef handle "Extended" field with a predefined base register instance for NOR or SRAM extended mode
2. Declare two FMC_NORSRAM_TimingTypeDef structures, for both normal and extended mode timings; for example: FMC_NORSRAM_TimingTypeDef Timing and FMC_NORSRAM_TimingTypeDef ExTiming; and fill its fields with the allowed values of the structure member.

3. Initialize the SRAM Controller by calling the function `HAL_SRAM_Init()`. This function performs the following sequence:
 - a. MSP hardware layer configuration using the function `HAL_SRAM_MspInit()`
 - b. Control register configuration using the FMC NORSRAM interface function `FMC_NORSRAM_Init()`
 - c. Timing register configuration using the FMC NORSRAM interface function `FMC_NORSRAM_Timing_Init()`
 - d. Extended mode Timing register configuration using the FMC NORSRAM interface function `FMC_NORSRAM_Extended_Timing_Init()`
 - e. Enable the SRAM device using the macro `__FMC_NORSRAM_ENABLE()`
4. At this stage you can perform read/write accesses from/to the memory connected to the NOR/SRAM Bank. You can perform either polling or DMA transfer using the following APIs:
 - `HAL_SRAM_Read()/HAL_SRAM_Write()` for polling read/write access
 - `HAL_SRAM_Read_DMA()/HAL_SRAM_Write_DMA()` for DMA read/write transfer
5. You can also control the SRAM device by calling the control APIs `HAL_SRAM_WriteOperation_Enable()/HAL_SRAM_WriteOperation_Disable()` to respectively enable/disable the SRAM write operation
6. You can continuously monitor the SRAM device HAL state by calling the function `HAL_SRAM_GetState()`

Callback registration

The compilation define `USE_HAL_SRAM_REGISTER_CALLBACKS` when set to 1 allows the user to configure dynamically the driver callbacks. Use Functions `HAL_SRAM_RegisterCallback()` to register a user callback, it allows to register following callbacks:

- `MspInitCallback` : SRAM `MspInit`.
- `MspDeInitCallback` : SRAM `MspDeInit`. This function takes as parameters the HAL peripheral handle, the Callback ID and a pointer to the user callback function. Use function `HAL_SRAM_UnRegisterCallback()` to reset a callback to the default weak (surcharged) function. It allows to reset following callbacks:
- `MspInitCallback` : SRAM `MspInit`.
- `MspDeInitCallback` : SRAM `MspDeInit`. This function) takes as parameters the HAL peripheral handle and the Callback ID. By default, after the `HAL_SRAM_Init` and if the state is `HAL_SRAM_STATE_RESET` all callbacks are reset to the corresponding legacy weak (surcharged) functions. Exception done for `MspInit` and `MspDeInit` callbacks that are respectively reset to the legacy weak (surcharged) functions in the `HAL_SRAM_Init` and `HAL_SRAM_DeInit` only when these callbacks are null (not registered beforehand). If not, `MspInit` or `MspDeInit` are not null, the `HAL_SRAM_Init` and `HAL_SRAM_DeInit` keep and use the user `MspInit`/`MspDeInit` callbacks (registered beforehand) Callbacks can be registered/unregistered in READY state only. Exception done for `MspInit`/`MspDeInit` callbacks that can be registered/unregistered in READY or RESET state, thus registered (user) `MspInit`/`DeInit` callbacks can be used during the `Init`/`DeInit`. In that case first register the `MspInit`/`MspDeInit` user callbacks using `HAL_SRAM_RegisterCallback` before calling `HAL_SRAM_DeInit` or `HAL_SRAM_Init` function. When The compilation define `USE_HAL_SRAM_REGISTER_CALLBACKS` is set to 0 or not defined, the callback registering feature is not available and weak (surcharged) callbacks are used.

67.2.2 SRAM Initialization and de_initialization functions

This section provides functions allowing to initialize/de-initialize the SRAM memory

This section contains the following APIs:

- [`HAL\_SRAM\_Init\(\)`](#)
- [`HAL\_SRAM\_DeInit\(\)`](#)
- [`HAL\_SRAM\_MspInit\(\)`](#)
- [`HAL\_SRAM\_MspDeInit\(\)`](#)
- [`HAL\_SRAM\_DMA\_XferCpltCallback\(\)`](#)
- [`HAL\_SRAM\_DMA\_XferErrorCallback\(\)`](#)

67.2.3 SRAM Input and Output functions

This section provides functions allowing to use and control the SRAM memory

This section contains the following APIs:

- *HAL_SRAM_Read_8b()*
- *HAL_SRAM_Write_8b()*
- *HAL_SRAM_Read_16b()*
- *HAL_SRAM_Write_16b()*
- *HAL_SRAM_Read_32b()*
- *HAL_SRAM_Write_32b()*
- *HAL_SRAM_Read_DMA()*
- *HAL_SRAM_Write_DMA()*
- *HAL_SRAM_RegisterCallback()*
- *HAL_SRAM_UnRegisterCallback()*
- *HAL_SRAM_RegisterDmaCallback()*
- *HAL_SRAM_DMA_XferCpltCallback()*
- *HAL_SRAM_DMA_XferErrorCallback()*

67.2.4 SRAM Control functions

This subsection provides a set of functions allowing to control dynamically the SRAM interface.

This section contains the following APIs:

- *HAL_SRAM_WriteOperation_Enable()*
- *HAL_SRAM_WriteOperation_Disable()*

67.2.5 SRAM State functions

This subsection permits to get in run-time the status of the SRAM controller and the data flow.

This section contains the following APIs:

- *HAL_SRAM_GetState()*

67.2.6 Detailed description of functions

HAL_SRAM_Init

Function name

HAL_StatusTypeDef HAL_SRAM_Init (SRAM_HandleTypeDef * hsram, FMC_NORSRAM_TimingTypeDef * Timing, FMC_NORSRAM_TimingTypeDef * ExtTiming)

Function description

Performs the SRAM device initialization sequence.

Parameters

- **hsram:** pointer to a SRAM_HandleTypeDef structure that contains the configuration information for SRAM module.
- **Timing:** Pointer to SRAM control timing structure
- **ExtTiming:** Pointer to SRAM extended mode timing structure

Return values

- **HAL:** status

HAL_SRAM_DeInit

Function name

HAL_StatusTypeDef HAL_SRAM_DeInit (SRAM_HandleTypeDef * hsram)

Function description

Performs the SRAM device De-initialization sequence.

Parameters

- **hsram:** pointer to a `SRAM_HandleTypeDef` structure that contains the configuration information for SRAM module.

Return values

- **HAL:** status

HAL_SRAM_MspInit

Function name

void HAL_SRAM_MspInit (SRAM_HandleTypeDef * hsram)

Function description

SRAM MSP Init.

Parameters

- **hsram:** pointer to a `SRAM_HandleTypeDef` structure that contains the configuration information for SRAM module.

Return values

- **None:**

HAL_SRAM_MspDeInit

Function name

void HAL_SRAM_MspDeInit (SRAM_HandleTypeDef * hsram)

Function description

SRAM MSP DeInit.

Parameters

- **hsram:** pointer to a `SRAM_HandleTypeDef` structure that contains the configuration information for SRAM module.

Return values

- **None:**

HAL_SRAM_Read_8b

Function name

HAL_StatusTypeDef HAL_SRAM_Read_8b (SRAM_HandleTypeDef * hsram, uint32_t * pAddress, uint8_t * pDstBuffer, uint32_t BufferSize)

Function description

Reads 8-bit buffer from SRAM memory.

Parameters

- **hsram:** pointer to a `SRAM_HandleTypeDef` structure that contains the configuration information for SRAM module.
- **pAddress:** Pointer to read start address
- **pDstBuffer:** Pointer to destination buffer
- **BufferSize:** Size of the buffer to read from memory

Return values

- **HAL:** status

HAL_SRAM_Write_8b

Function name

HAL_StatusTypeDef HAL_SRAM_Write_8b (SRAM_HandleTypeDef * hsram, uint32_t * pAddress, uint8_t * pSrcBuffer, uint32_t BufferSize)

Function description

Writes 8-bit buffer to SRAM memory.

Parameters

- **hsram**: pointer to a SRAM_HandleTypeDef structure that contains the configuration information for SRAM module.
- **pAddress**: Pointer to write start address
- **pSrcBuffer**: Pointer to source buffer to write
- **BufferSize**: Size of the buffer to write to memory

Return values

- **HAL**: status

HAL_SRAM_Read_16b

Function name

HAL_StatusTypeDef HAL_SRAM_Read_16b (SRAM_HandleTypeDef * hsram, uint32_t * pAddress, uint16_t * pDstBuffer, uint32_t BufferSize)

Function description

Reads 16-bit buffer from SRAM memory.

Parameters

- **hsram**: pointer to a SRAM_HandleTypeDef structure that contains the configuration information for SRAM module.
- **pAddress**: Pointer to read start address
- **pDstBuffer**: Pointer to destination buffer
- **BufferSize**: Size of the buffer to read from memory

Return values

- **HAL**: status

HAL_SRAM_Write_16b

Function name

HAL_StatusTypeDef HAL_SRAM_Write_16b (SRAM_HandleTypeDef * hsram, uint32_t * pAddress, uint16_t * pSrcBuffer, uint32_t BufferSize)

Function description

Writes 16-bit buffer to SRAM memory.

Parameters

- **hsram**: pointer to a SRAM_HandleTypeDef structure that contains the configuration information for SRAM module.
- **pAddress**: Pointer to write start address
- **pSrcBuffer**: Pointer to source buffer to write
- **BufferSize**: Size of the buffer to write to memory

Return values

- **HAL**: status

HAL_SRAM_Read_32b

Function name

HAL_StatusTypeDef HAL_SRAM_Read_32b (SRAM_HandleTypeDef * hsram, uint32_t * pAddress, uint32_t * pDstBuffer, uint32_t BufferSize)

Function description

Reads 32-bit buffer from SRAM memory.

Parameters

- **hsram**: pointer to a SRAM_HandleTypeDef structure that contains the configuration information for SRAM module.
- **pAddress**: Pointer to read start address
- **pDstBuffer**: Pointer to destination buffer
- **BufferSize**: Size of the buffer to read from memory

Return values

- **HAL**: status

HAL_SRAM_Write_32b

Function name

HAL_StatusTypeDef HAL_SRAM_Write_32b (SRAM_HandleTypeDef * hsram, uint32_t * pAddress, uint32_t * pSrcBuffer, uint32_t BufferSize)

Function description

Writes 32-bit buffer to SRAM memory.

Parameters

- **hsram**: pointer to a SRAM_HandleTypeDef structure that contains the configuration information for SRAM module.
- **pAddress**: Pointer to write start address
- **pSrcBuffer**: Pointer to source buffer to write
- **BufferSize**: Size of the buffer to write to memory

Return values

- **HAL**: status

HAL_SRAM_Read_DMA

Function name

HAL_StatusTypeDef HAL_SRAM_Read_DMA (SRAM_HandleTypeDef * hsram, uint32_t * pAddress, uint32_t * pDstBuffer, uint32_t BufferSize)

Function description

Reads a Words data from the SRAM memory using DMA transfer.

Parameters

- **hsram**: pointer to a SRAM_HandleTypeDef structure that contains the configuration information for SRAM module.
- **pAddress**: Pointer to read start address
- **pDstBuffer**: Pointer to destination buffer
- **BufferSize**: Size of the buffer to read from memory

Return values

- **HAL**: status

HAL_SRAM_Write_DMA

Function name

```
HAL_StatusTypeDef HAL_SRAM_Write_DMA (SRAM_HandleTypeDef * hsram, uint32_t * pAddress,
uint32_t * pSrcBuffer, uint32_t BufferSize)
```

Function description

Writes a Words data buffer to SRAM memory using DMA transfer.

Parameters

- **hsram**: pointer to a SRAM_HandleTypeDef structure that contains the configuration information for SRAM module.
- **pAddress**: Pointer to write start address
- **pSrcBuffer**: Pointer to source buffer to write
- **BufferSize**: Size of the buffer to write to memory

Return values

- **HAL**: status

HAL_SRAM_DMA_XferCpltCallback

Function name

```
void HAL_SRAM_DMA_XferCpltCallback (DMA_HandleTypeDef * hdma)
```

Function description

DMA transfer complete callback.

Parameters

- **hdma**: pointer to a SRAM_HandleTypeDef structure that contains the configuration information for SRAM module.

Return values

- **None**:

HAL_SRAM_DMA_XferErrorCallback

Function name

```
void HAL_SRAM_DMA_XferErrorCallback (DMA_HandleTypeDef * hdma)
```

Function description

DMA transfer complete error callback.

Parameters

- **hdma**: pointer to a SRAM_HandleTypeDef structure that contains the configuration information for SRAM module.

Return values

- **None**:

HAL_SRAM_RegisterCallback

Function name

```
HAL_StatusTypeDef HAL_SRAM_RegisterCallback (SRAM_HandleTypeDef * hsram,
HAL_SRAM_CallbackIDTypeDef CallbackId, pSRAM_CallbackTypeDef pCallback)
```

Function description

Register a User SRAM Callback To be used instead of the weak (surcharged) predefined callback.

Parameters

- **hsram** : SRAM handle
- **CallbackId** : ID of the callback to be registered This parameter can be one of the following values:
 - HAL_SRAM_MSP_INIT_CB_ID SRAM MspInit callback ID
 - HAL_SRAM_MSP_DEINIT_CB_ID SRAM MspDeInit callback ID
- **pCallback** : pointer to the Callback function

Return values

- **status**:

HAL_SRAM_UnRegisterCallback

Function name

HAL_StatusTypeDef HAL_SRAM_UnRegisterCallback (SRAM_HandleTypeDef * hsram, HAL_SRAM_CallbackIDTypeDef CallbackId)

Function description

Unregister a User SRAM Callback SRAM Callback is redirected to the weak (surcharged) predefined callback.

Parameters

- **hsram** : SRAM handle
- **CallbackId** : ID of the callback to be unregistered This parameter can be one of the following values:
 - HAL_SRAM_MSP_INIT_CB_ID SRAM MspInit callback ID
 - HAL_SRAM_MSP_DEINIT_CB_ID SRAM MspDeInit callback ID
 - HAL_SRAM_DMA_XFER_CPLT_CB_ID SRAM DMA Xfer Complete callback ID
 - HAL_SRAM_DMA_XFER_ERR_CB_ID SRAM DMA Xfer Error callback ID

Return values

- **status**:

HAL_SRAM_RegisterDmaCallback

Function name

HAL_StatusTypeDef HAL_SRAM_RegisterDmaCallback (SRAM_HandleTypeDef * hsram, HAL_SRAM_CallbackIDTypeDef CallbackId, pSRAM_DmaCallbackTypeDef pCallback)

Function description

Register a User SRAM Callback for DMA transfers To be used instead of the weak (surcharged) predefined callback.

Parameters

- **hsram** : SRAM handle
- **CallbackId** : ID of the callback to be registered This parameter can be one of the following values:
 - HAL_SRAM_DMA_XFER_CPLT_CB_ID SRAM DMA Xfer Complete callback ID
 - HAL_SRAM_DMA_XFER_ERR_CB_ID SRAM DMA Xfer Error callback ID
- **pCallback** : pointer to the Callback function

Return values

- **status**:

HAL_SRAM_WriteOperation_Enable

Function name

HAL_StatusTypeDef HAL_SRAM_WriteOperation_Enable (SRAM_HandleTypeDef * hsram)

Function description

Enables dynamically SRAM write operation.

Parameters

- **hsram**: pointer to a SRAM_HandleTypeDef structure that contains the configuration information for SRAM module.

Return values

- **HAL**: status

HAL_SRAM_WriteOperation_Disable

Function name

HAL_StatusTypeDef HAL_SRAM_WriteOperation_Disable (SRAM_HandleTypeDef * hsram)

Function description

Disables dynamically SRAM write operation.

Parameters

- **hsram**: pointer to a SRAM_HandleTypeDef structure that contains the configuration information for SRAM module.

Return values

- **HAL**: status

HAL_SRAM_GetState

Function name

HAL_SRAM_StateTypeDef HAL_SRAM_GetState (SRAM_HandleTypeDef * hsram)

Function description

Returns the SRAM controller state.

Parameters

- **hsram**: pointer to a SRAM_HandleTypeDef structure that contains the configuration information for SRAM module.

Return values

- **HAL**: state

67.3 SRAM Firmware driver defines

The following section lists the various define and macros of the module.

67.3.1 SRAM

SRAM

SRAM Exported Macros

__HAL_SRAM_RESET_HANDLE_STATE

Description:

- Reset SRAM handle state.

Parameters:

- **__HANDLE__**: SRAM handle

Return value:

- None

68 HAL TIM Generic Driver

68.1 TIM Firmware driver registers structures

68.1.1 TIM_Base_InitTypeDef

TIM_Base_InitTypeDef is defined in the stm32f4xx_hal_tim.h

Data Fields

- **uint32_t Prescaler**
- **uint32_t CounterMode**
- **uint32_t Period**
- **uint32_t ClockDivision**
- **uint32_t RepetitionCounter**
- **uint32_t AutoReloadPreload**

Field Documentation

- **uint32_t TIM_Base_InitTypeDef::Prescaler**
Specifies the prescaler value used to divide the TIM clock. This parameter can be a number between Min_Data = 0x0000 and Max_Data = 0xFFFF
- **uint32_t TIM_Base_InitTypeDef::CounterMode**
Specifies the counter mode. This parameter can be a value of [TIM_Counter_Mode](#)
- **uint32_t TIM_Base_InitTypeDef::Period**
Specifies the period value to be loaded into the active Auto-Reload Register at the next update event. This parameter can be a number between Min_Data = 0x0000 and Max_Data = 0xFFFF.
- **uint32_t TIM_Base_InitTypeDef::ClockDivision**
Specifies the clock division. This parameter can be a value of [TIM_ClockDivision](#)
- **uint32_t TIM_Base_InitTypeDef::RepetitionCounter**
Specifies the repetition counter value. Each time the RCR downcounter reaches zero, an update event is generated and counting restarts from the RCR value (N). This means in PWM mode that (N+1) corresponds to:
 - the number of PWM periods in edge-aligned mode
 - the number of half PWM period in center-aligned mode GP timers: this parameter must be a number between Min_Data = 0x00 and Max_Data = 0xFF. Advanced timers: this parameter must be a number between Min_Data = 0x0000 and Max_Data = 0xFFFF.
- **uint32_t TIM_Base_InitTypeDef::AutoReloadPreload**
Specifies the auto-reload preload. This parameter can be a value of [TIM_AutoReloadPreload](#)

68.1.2 TIM_OC_InitTypeDef

TIM_OC_InitTypeDef is defined in the stm32f4xx_hal_tim.h

Data Fields

- **uint32_t OCMODE**
- **uint32_t Pulse**
- **uint32_t OCPolarity**
- **uint32_t OCNPolarity**
- **uint32_t OCFastMode**
- **uint32_t OCIdleState**
- **uint32_t OCNIdleState**

Field Documentation

- **uint32_t TIM_OC_InitTypeDef::OCMode**
Specifies the TIM mode. This parameter can be a value of [TIM_Output_Compare_and_PWM_modes](#)
- **uint32_t TIM_OC_InitTypeDef::Pulse**
Specifies the pulse value to be loaded into the Capture Compare Register. This parameter can be a number between Min_Data = 0x0000 and Max_Data = 0xFFFF

- **`uint32_t TIM_OC_InitTypeDef::OCpolarity`**
Specifies the output polarity. This parameter can be a value of [TIM_Output_Compare_Polarity](#)
- **`uint32_t TIM_OC_InitTypeDef::OCNPolarity`**
Specifies the complementary output polarity. This parameter can be a value of [TIM_Output_Compare_N_Polarity](#)
Note:
 - This parameter is valid only for timer instances supporting break feature.
- **`uint32_t TIM_OC_InitTypeDef::OCFastMode`**
Specifies the Fast mode state. This parameter can be a value of [TIM_Output_Fast_State](#)
Note:
 - This parameter is valid only in PWM1 and PWM2 mode.
- **`uint32_t TIM_OC_InitTypeDef::OCIdleState`**
Specifies the TIM Output Compare pin state during Idle state. This parameter can be a value of [TIM_Output_Compare_Idle_State](#)
Note:
 - This parameter is valid only for timer instances supporting break feature.
- **`uint32_t TIM_OC_InitTypeDef::OCNIdleState`**
Specifies the TIM Output Compare pin state during Idle state. This parameter can be a value of [TIM_Output_Compare_N_Idle_State](#)
Note:
 - This parameter is valid only for timer instances supporting break feature.

68.1.3

TIM_OnePulse_InitTypeDef

TIM_OnePulse_InitTypeDef is defined in the `stm32f4xx_hal_tim.h`

Data Fields

- **`uint32_t OCMODE`**
- **`uint32_t Pulse`**
- **`uint32_t OCPolarity`**
- **`uint32_t OCNPolarity`**
- **`uint32_t OCIdleState`**
- **`uint32_t OCNIdleState`**
- **`uint32_t ICPolarity`**
- **`uint32_t ICSelection`**
- **`uint32_t ICFILTER`**

Field Documentation

- **`uint32_t TIM_OnePulse_InitTypeDef::OCMODE`**
Specifies the TIM mode. This parameter can be a value of [TIM_Output_Compare_and_PWM_modes](#)
- **`uint32_t TIM_OnePulse_InitTypeDef::Pulse`**
Specifies the pulse value to be loaded into the Capture Compare Register. This parameter can be a number between `Min_Data = 0x0000` and `Max_Data = 0xFFFF`
- **`uint32_t TIM_OnePulse_InitTypeDef::OCPolarity`**
Specifies the output polarity. This parameter can be a value of [TIM_Output_Compare_Polarity](#)
- **`uint32_t TIM_OnePulse_InitTypeDef::OCNPolarity`**
Specifies the complementary output polarity. This parameter can be a value of [TIM_Output_Compare_N_Polarity](#)
Note:
 - This parameter is valid only for timer instances supporting break feature.
- **`uint32_t TIM_OnePulse_InitTypeDef::OCIdleState`**
Specifies the TIM Output Compare pin state during Idle state. This parameter can be a value of [TIM_Output_Compare_Idle_State](#)
Note:
 - This parameter is valid only for timer instances supporting break feature.

- **`uint32_t TIM_OnePulse_InitTypeDef::OCNIdleState`**
Specifies the TIM Output Compare pin state during Idle state. This parameter can be a value of [TIM_Output_Compare_N_Idle_State](#)
Note:
 - This parameter is valid only for timer instances supporting break feature.
- **`uint32_t TIM_OnePulse_InitTypeDef::ICPolarity`**
Specifies the active edge of the input signal. This parameter can be a value of [TIM_Input_Capture_Polarity](#)
- **`uint32_t TIM_OnePulse_InitTypeDef::ICSelection`**
Specifies the input. This parameter can be a value of [TIM_Input_Capture_Selection](#)
- **`uint32_t TIM_OnePulse_InitTypeDef::ICFilter`**
Specifies the input capture filter. This parameter can be a number between Min_Data = 0x0 and Max_Data = 0xF

68.1.4

TIM_IC_InitTypeDef

TIM_IC_InitTypeDef is defined in the `stm32f4xx_hal_tim.h`

Data Fields

- **`uint32_t ICPolarity`**
- **`uint32_t ICSelection`**
- **`uint32_t ICPrescaler`**
- **`uint32_t ICFilter`**

Field Documentation

- **`uint32_t TIM_IC_InitTypeDef::ICPolarity`**
Specifies the active edge of the input signal. This parameter can be a value of [TIM_Input_Capture_Polarity](#)
- **`uint32_t TIM_IC_InitTypeDef::ICSelection`**
Specifies the input. This parameter can be a value of [TIM_Input_Capture_Selection](#)
- **`uint32_t TIM_IC_InitTypeDef::ICPrescaler`**
Specifies the Input Capture Prescaler. This parameter can be a value of [TIM_Input_Capture_Prescaler](#)
- **`uint32_t TIM_IC_InitTypeDef::ICFilter`**
Specifies the input capture filter. This parameter can be a number between Min_Data = 0x0 and Max_Data = 0xF

68.1.5

TIM_Encoder_InitTypeDef

TIM_Encoder_InitTypeDef is defined in the `stm32f4xx_hal_tim.h`

Data Fields

- **`uint32_t EncoderMode`**
- **`uint32_t IC1Polarity`**
- **`uint32_t IC1Selection`**
- **`uint32_t IC1Prescaler`**
- **`uint32_t IC1Filter`**
- **`uint32_t IC2Polarity`**
- **`uint32_t IC2Selection`**
- **`uint32_t IC2Prescaler`**
- **`uint32_t IC2Filter`**

Field Documentation

- **`uint32_t TIM_Encoder_InitTypeDef::EncoderMode`**
Specifies the active edge of the input signal. This parameter can be a value of [TIM_Encoder_Mode](#)
- **`uint32_t TIM_Encoder_InitTypeDef::IC1Polarity`**
Specifies the active edge of the input signal. This parameter can be a value of [TIM_Encoder_Input_Polarity](#)
- **`uint32_t TIM_Encoder_InitTypeDef::IC1Selection`**
Specifies the input. This parameter can be a value of [TIM_Input_Capture_Selection](#)

- **`uint32_t TIM_Encoder_InitTypeDef::IC1Prescaler`**
Specifies the Input Capture Prescaler. This parameter can be a value of [TIM_Input_Capture_Prescaler](#)
- **`uint32_t TIM_Encoder_InitTypeDef::IC1Filter`**
Specifies the input capture filter. This parameter can be a number between Min_Data = 0x0 and Max_Data = 0xF
- **`uint32_t TIM_Encoder_InitTypeDef::IC2Polarity`**
Specifies the active edge of the input signal. This parameter can be a value of [TIM_Encoder_Input_Polarity](#)
- **`uint32_t TIM_Encoder_InitTypeDef::IC2Selection`**
Specifies the input. This parameter can be a value of [TIM_Input_Capture_Selection](#)
- **`uint32_t TIM_Encoder_InitTypeDef::IC2Prescaler`**
Specifies the Input Capture Prescaler. This parameter can be a value of [TIM_Input_Capture_Prescaler](#)
- **`uint32_t TIM_Encoder_InitTypeDef::IC2Filter`**
Specifies the input capture filter. This parameter can be a number between Min_Data = 0x0 and Max_Data = 0xF

68.1.6

TIM_ClockConfigTypeDef

TIM_ClockConfigTypeDef is defined in the `stm32f4xx_hal_tim.h`

Data Fields

- **`uint32_t ClockSource`**
- **`uint32_t ClockPolarity`**
- **`uint32_t ClockPrescaler`**
- **`uint32_t ClockFilter`**

Field Documentation

- **`uint32_t TIM_ClockConfigTypeDef::ClockSource`**
TIM clock sources This parameter can be a value of [TIM_Clock_Source](#)
- **`uint32_t TIM_ClockConfigTypeDef::ClockPolarity`**
TIM clock polarity This parameter can be a value of [TIM_Clock_Polarity](#)
- **`uint32_t TIM_ClockConfigTypeDef::ClockPrescaler`**
TIM clock prescaler This parameter can be a value of [TIM_Clock_Prescaler](#)
- **`uint32_t TIM_ClockConfigTypeDef::ClockFilter`**
TIM clock filter This parameter can be a number between Min_Data = 0x0 and Max_Data = 0xF

68.1.7

TIM_ClearInputConfigTypeDef

TIM_ClearInputConfigTypeDef is defined in the `stm32f4xx_hal_tim.h`

Data Fields

- **`uint32_t ClearInputState`**
- **`uint32_t ClearInputSource`**
- **`uint32_t ClearInputPolarity`**
- **`uint32_t ClearInputPrescaler`**
- **`uint32_t ClearInputFilter`**

Field Documentation

- **`uint32_t TIM_ClearInputConfigTypeDef::ClearInputState`**
TIM clear Input state This parameter can be ENABLE or DISABLE
- **`uint32_t TIM_ClearInputConfigTypeDef::ClearInputSource`**
TIM clear Input sources This parameter can be a value of [TIM_ClearInput_Source](#)
- **`uint32_t TIM_ClearInputConfigTypeDef::ClearInputPolarity`**
TIM Clear Input polarity This parameter can be a value of [TIM_ClearInput_Polarity](#)
- **`uint32_t TIM_ClearInputConfigTypeDef::ClearInputPrescaler`**
TIM Clear Input prescaler This parameter must be 0: When OCREf clear feature is used with ETR source, ETR prescaler must be off
- **`uint32_t TIM_ClearInputConfigTypeDef::ClearInputFilter`**
TIM Clear Input filter This parameter can be a number between Min_Data = 0x0 and Max_Data = 0xF

68.1.8

TIM_MasterConfigTypeDef

TIM_MasterConfigTypeDef is defined in the stm32f4xx_hal_tim.h

Data Fields

- **uint32_t MasterOutputTrigger**
- **uint32_t MasterSlaveMode**

Field Documentation

- **uint32_t TIM_MasterConfigTypeDef::MasterOutputTrigger**
Trigger output (TRGO) selection This parameter can be a value of **TIM_Master_Mode_Selection**
- **uint32_t TIM_MasterConfigTypeDef::MasterSlaveMode**
Master/slave mode selection This parameter can be a value of **TIM_Master_Slave_Mode**

Note:

- When the Master/slave mode is enabled, the effect of an event on the trigger input (TRGI) is delayed to allow a perfect synchronization between the current timer and its slaves (through TRGO). It is not mandatory in case of timer synchronization mode.

68.1.9

TIM_SlaveConfigTypeDef

TIM_SlaveConfigTypeDef is defined in the stm32f4xx_hal_tim.h

Data Fields

- **uint32_t SlaveMode**
- **uint32_t InputTrigger**
- **uint32_t TriggerPolarity**
- **uint32_t TriggerPrescaler**
- **uint32_t TriggerFilter**

Field Documentation

- **uint32_t TIM_SlaveConfigTypeDef::SlaveMode**
Slave mode selection This parameter can be a value of **TIM_Slave_Mode**
- **uint32_t TIM_SlaveConfigTypeDef::InputTrigger**
Input Trigger source This parameter can be a value of **TIM_Trigger_Selection**
- **uint32_t TIM_SlaveConfigTypeDef::TriggerPolarity**
Input Trigger polarity This parameter can be a value of **TIM_Trigger_Polarity**
- **uint32_t TIM_SlaveConfigTypeDef::TriggerPrescaler**
Input trigger prescaler This parameter can be a value of **TIM_Trigger_Prescaler**
- **uint32_t TIM_SlaveConfigTypeDef::TriggerFilter**
Input trigger filter This parameter can be a number between Min_Data = 0x0 and Max_Data = 0xF

68.1.10

TIM_BreakDeadTimeConfigTypeDef

TIM_BreakDeadTimeConfigTypeDef is defined in the stm32f4xx_hal_tim.h

Data Fields

- **uint32_t OffStateRunMode**
- **uint32_t OffStateIDLEMode**
- **uint32_t LockLevel**
- **uint32_t DeadTime**
- **uint32_t BreakState**
- **uint32_t BreakPolarity**
- **uint32_t BreakFilter**
- **uint32_t AutomaticOutput**

Field Documentation

- **uint32_t TIM_BreakDeadTimeConfigTypeDef::OffStateRunMode**
TIM off state in run mode, This parameter can be a value of **TIM_OSSR_Off_State_Selection_for_Run_mode_state**

- **`uint32_t TIM_BreakDeadTimeConfigTypeDef::OffStateIDLEMode`**
TIM off state in IDLE mode, This parameter can be a value of [TIM_OSSI_Off_State_Selection_for_Idle_mode_state](#)
- **`uint32_t TIM_BreakDeadTimeConfigTypeDef::LockLevel`**
TIM Lock level, This parameter can be a value of [TIM_Lock_Level](#)
- **`uint32_t TIM_BreakDeadTimeConfigTypeDef::DeadTime`**
TIM dead Time, This parameter can be a number between Min_Data = 0x00 and Max_Data = 0xFF
- **`uint32_t TIM_BreakDeadTimeConfigTypeDef::BreakState`**
TIM Break State, This parameter can be a value of [TIM_Break_Input_enable_disable](#)
- **`uint32_t TIM_BreakDeadTimeConfigTypeDef::BreakPolarity`**
TIM Break input polarity, This parameter can be a value of [TIM_Break_Polarity](#)
- **`uint32_t TIM_BreakDeadTimeConfigTypeDef::BreakFilter`**
Specifies the break input filter. This parameter can be a number between Min_Data = 0x0 and Max_Data = 0xF
- **`uint32_t TIM_BreakDeadTimeConfigTypeDef::AutomaticOutput`**
TIM Automatic Output Enable state, This parameter can be a value of [TIM_AOE_Bit_Set_Reset](#)

68.1.11 TIM_HandleTypeDef

TIM_HandleTypeDef is defined in the `stm32f4xx_hal_tim.h`

Data Fields

- **`TIM_TypeDef * Instance`**
- **`TIM_Base_InitTypeDef Init`**
- **`HAL_TIM_ActiveChannel Channel`**
- **`DMA_HandleTypeDef * hdma`**
- **`HAL_LockTypeDef Lock`**
- **`__IO HAL_TIM_StateTypeDef State`**
- **`__IO HAL_TIM_ChannelStateTypeDef ChannelState`**
- **`__IO HAL_TIM_ChannelStateTypeDef ChannelINState`**
- **`__IO HAL_TIM_DMABurstStateTypeDef DMABurstState`**

Field Documentation

- **`TIM_TypeDef* TIM_HandleTypeDef::Instance`**
Register base address
- **`TIM_Base_InitTypeDef TIM_HandleTypeDef::Init`**
TIM Time Base required parameters
- **`HAL_TIM_ActiveChannel TIM_HandleTypeDef::Channel`**
Active channel
- **`DMA_HandleTypeDef* TIM_HandleTypeDef::hdma[7]`**
DMA Handlers array This array is accessed by a `DMA_Handle_index`
- **`HAL_LockTypeDef TIM_HandleTypeDef::Lock`**
Locking object
- **`__IO HAL_TIM_StateTypeDef TIM_HandleTypeDef::State`**
TIM operation state
- **`__IO HAL_TIM_ChannelStateTypeDef TIM_HandleTypeDef::ChannelState[4]`**
TIM channel operation state
- **`__IO HAL_TIM_ChannelStateTypeDef TIM_HandleTypeDef::ChannelINState[4]`**
TIM complementary channel operation state
- **`__IO HAL_TIM_DMABurstStateTypeDef TIM_HandleTypeDef::DMABurstState`**
DMA burst operation state

68.2 TIM Firmware driver API description

The following section lists the various functions of the TIM library.

68.2.1 TIMER Generic features

The Timer features include:

1. 16-bit up, down, up/down auto-reload counter.
2. 16-bit programmable prescaler allowing dividing (also on the fly) the counter clock frequency either by any factor between 1 and 65536.
3. Up to 4 independent channels for:
 - Input Capture
 - Output Compare
 - PWM generation (Edge and Center-aligned Mode)
 - One-pulse mode output
4. Synchronization circuit to control the timer with external signals and to interconnect several timers together.
5. Supports incremental encoder for positioning purposes

68.2.2

How to use this driver

1. Initialize the TIM low level resources by implementing the following functions depending on the selected feature:
 - Time Base : `HAL_TIM_Base_MspInit()`
 - Input Capture : `HAL_TIM_IC_MspInit()`
 - Output Compare : `HAL_TIM_OC_MspInit()`
 - PWM generation : `HAL_TIM_PWM_MspInit()`
 - One-pulse mode output : `HAL_TIM_OnePulse_MspInit()`
 - Encoder mode output : `HAL_TIM_Encoder_MspInit()`
2. Initialize the TIM low level resources :
 - a. Enable the TIM interface clock using `__HAL_RCC_TIMx_CLK_ENABLE()`;
 - b. TIM pins configuration
 - Enable the clock for the TIM GPIOs using the following function:
`__HAL_RCC_GPIOx_CLK_ENABLE()`;
 - Configure these TIM pins in Alternate function mode using `HAL_GPIO_Init()`;
3. The external Clock can be configured, if needed (the default clock is the internal clock from the APBx), using the following function: `HAL_TIM_ConfigClockSource`, the clock configuration should be done before any start function.
4. Configure the TIM in the desired functioning mode using one of the Initialization function of this driver:
 - `HAL_TIM_Base_Init`: to use the Timer to generate a simple time base
 - `HAL_TIM_OC_Init` and `HAL_TIM_OC_ConfigChannel`: to use the Timer to generate an Output Compare signal.
 - `HAL_TIM_PWM_Init` and `HAL_TIM_PWM_ConfigChannel`: to use the Timer to generate a PWM signal.
 - `HAL_TIM_IC_Init` and `HAL_TIM_IC_ConfigChannel`: to use the Timer to measure an external signal.
 - `HAL_TIM_OnePulse_Init` and `HAL_TIM_OnePulse_ConfigChannel`: to use the Timer in One Pulse Mode.
 - `HAL_TIM_Encoder_Init`: to use the Timer Encoder Interface.
5. Activate the TIM peripheral using one of the start functions depending from the feature used:
 - Time Base : `HAL_TIM_Base_Start()`, `HAL_TIM_Base_Start_DMA()`, `HAL_TIM_Base_Start_IT()`
 - Input Capture : `HAL_TIM_IC_Start()`, `HAL_TIM_IC_Start_DMA()`, `HAL_TIM_IC_Start_IT()`
 - Output Compare : `HAL_TIM_OC_Start()`, `HAL_TIM_OC_Start_DMA()`, `HAL_TIM_OC_Start_IT()`
 - PWM generation : `HAL_TIM_PWM_Start()`, `HAL_TIM_PWM_Start_DMA()`, `HAL_TIM_PWM_Start_IT()`
 - One-pulse mode output : `HAL_TIM_OnePulse_Start()`, `HAL_TIM_OnePulse_Start_IT()`
 - Encoder mode output : `HAL_TIM_Encoder_Start()`, `HAL_TIM_Encoder_Start_DMA()`, `HAL_TIM_Encoder_Start_IT()`.
6. The DMA Burst is managed with the two following functions: `HAL_TIM_DMABurst_WriteStart()`
`HAL_TIM_DMABurst_ReadStart()`

Callback registration

The compilation define `USE_HAL_TIM_REGISTER_CALLBACKS` when set to 1 allows the user to configure dynamically the driver callbacks.

Use Function `HAL_TIM_RegisterCallback()` to register a callback. `HAL_TIM_RegisterCallback()` takes as parameters the HAL peripheral handle, the Callback ID and a pointer to the user callback function.

Use function `HAL_TIM_UnRegisterCallback()` to reset a callback to the default weak function.

`HAL_TIM_UnRegisterCallback` takes as parameters the HAL peripheral handle, and the Callback ID.

These functions allow to register/unregister following callbacks:

- `Base_MspInitCallback` : TIM Base Msp Init Callback.
- `Base_MspDeInitCallback` : TIM Base Msp DeInit Callback.
- `IC_MspInitCallback` : TIM IC Msp Init Callback.
- `IC_MspDeInitCallback` : TIM IC Msp DeInit Callback.
- `OC_MspInitCallback` : TIM OC Msp Init Callback.
- `OC_MspDeInitCallback` : TIM OC Msp DeInit Callback.
- `PWM_MspInitCallback` : TIM PWM Msp Init Callback.
- `PWM_MspDeInitCallback` : TIM PWM Msp DeInit Callback.
- `OnePulse_MspInitCallback` : TIM One Pulse Msp Init Callback.
- `OnePulse_MspDeInitCallback` : TIM One Pulse Msp DeInit Callback.
- `Encoder_MspInitCallback` : TIM Encoder Msp Init Callback.
- `Encoder_MspDeInitCallback` : TIM Encoder Msp DeInit Callback.
- `HallSensor_MspInitCallback` : TIM Hall Sensor Msp Init Callback.
- `HallSensor_MspDeInitCallback` : TIM Hall Sensor Msp DeInit Callback.
- `PeriodElapsedCallback` : TIM Period Elapsed Callback.
- `PeriodElapsedHalfCpltCallback` : TIM Period Elapsed half complete Callback.
- `TriggerCallback` : TIM Trigger Callback.
- `TriggerHalfCpltCallback` : TIM Trigger half complete Callback.
- `IC_CaptureCallback` : TIM Input Capture Callback.
- `IC_CaptureHalfCpltCallback` : TIM Input Capture half complete Callback.
- `OC_DelayElapsedCallback` : TIM Output Compare Delay Elapsed Callback.
- `PWM_PulseFinishedCallback` : TIM PWM Pulse Finished Callback.
- `PWM_PulseFinishedHalfCpltCallback` : TIM PWM Pulse Finished half complete Callback.
- `ErrorCallback` : TIM Error Callback.
- `CommutationCallback` : TIM Commutation Callback.
- `CommutationHalfCpltCallback` : TIM Commutation half complete Callback.
- `BreakCallback` : TIM Break Callback.

By default, after the Init and when the state is `HAL_TIM_STATE_RESET` all interrupt callbacks are set to the corresponding weak functions: examples `HAL_TIM_TriggerCallback()`, `HAL_TIM_ErrorCallback()`.

Exception done for `MspInit` and `MspDeInit` functions that are reset to the legacy weak functionalities in the Init / DeInit only when these callbacks are null (not registered beforehand). If not, `MspInit` or `MspDeInit` are not null, the Init / DeInit keep and use the user `MspInit` / `MspDeInit` callbacks(registered beforehand)

Callbacks can be registered / unregistered in `HAL_TIM_STATE_READY` state only. Exception done `MspInit` / `MspDeInit` that can be registered / unregistered in `HAL_TIM_STATE_READY` or `HAL_TIM_STATE_RESET` state, thus registered(user) `MspInit` / `DeInit` callbacks can be used during the Init / DeInit. In that case first register the `MspInit`/`MspDeInit` user callbacks using `HAL_TIM_RegisterCallback()` before calling `DeInit` or `Init` function.

When The compilation define `USE_HAL_TIM_REGISTER_CALLBACKS` is set to 0 or not defined, the callback registration feature is not available and all callbacks are set to the corresponding weak functions.

68.2.3

Time Base functions

This section provides functions allowing to:

- Initialize and configure the TIM base.
- De-initialize the TIM base.

- Start the Time Base.
- Stop the Time Base.
- Start the Time Base and enable interrupt.
- Stop the Time Base and disable interrupt.
- Start the Time Base and enable DMA transfer.
- Stop the Time Base and disable DMA transfer.

This section contains the following APIs:

- *HAL_TIM_Base_Init()*
- *HAL_TIM_Base_DeInit()*
- *HAL_TIM_Base_MspInit()*
- *HAL_TIM_Base_MspDeInit()*
- *HAL_TIM_Base_Start()*
- *HAL_TIM_Base_Stop()*
- *HAL_TIM_Base_Start_IT()*
- *HAL_TIM_Base_Stop_IT()*
- *HAL_TIM_Base_Start_DMA()*
- *HAL_TIM_Base_Stop_DMA()*

68.2.4

TIM Output Compare functions

This section provides functions allowing to:

- Initialize and configure the TIM Output Compare.
- De-initialize the TIM Output Compare.
- Start the TIM Output Compare.
- Stop the TIM Output Compare.
- Start the TIM Output Compare and enable interrupt.
- Stop the TIM Output Compare and disable interrupt.
- Start the TIM Output Compare and enable DMA transfer.
- Stop the TIM Output Compare and disable DMA transfer.

This section contains the following APIs:

- *HAL_TIM_OC_Init()*
- *HAL_TIM_OC_DeInit()*
- *HAL_TIM_OC_MspInit()*
- *HAL_TIM_OC_MspDeInit()*
- *HAL_TIM_OC_Start()*
- *HAL_TIM_OC_Stop()*
- *HAL_TIM_OC_Start_IT()*
- *HAL_TIM_OC_Stop_IT()*
- *HAL_TIM_OC_Start_DMA()*
- *HAL_TIM_OC_Stop_DMA()*

68.2.5

TIM PWM functions

This section provides functions allowing to:

- Initialize and configure the TIM PWM.
- De-initialize the TIM PWM.
- Start the TIM PWM.
- Stop the TIM PWM.
- Start the TIM PWM and enable interrupt.
- Stop the TIM PWM and disable interrupt.
- Start the TIM PWM and enable DMA transfer.

- Stop the TIM PWM and disable DMA transfer.

This section contains the following APIs:

- *HAL_TIM_PWM_Init()*
- *HAL_TIM_PWM_DeInit()*
- *HAL_TIM_PWM_MspInit()*
- *HAL_TIM_PWM_MspDeInit()*
- *HAL_TIM_PWM_Start()*
- *HAL_TIM_PWM_Stop()*
- *HAL_TIM_PWM_Start_IT()*
- *HAL_TIM_PWM_Stop_IT()*
- *HAL_TIM_PWM_Start_DMA()*
- *HAL_TIM_PWM_Stop_DMA()*

68.2.6 TIM Input Capture functions

This section provides functions allowing to:

- Initialize and configure the TIM Input Capture.
- De-initialize the TIM Input Capture.
- Start the TIM Input Capture.
- Stop the TIM Input Capture.
- Start the TIM Input Capture and enable interrupt.
- Stop the TIM Input Capture and disable interrupt.
- Start the TIM Input Capture and enable DMA transfer.
- Stop the TIM Input Capture and disable DMA transfer.

This section contains the following APIs:

- *HAL_TIM_IC_Init()*
- *HAL_TIM_IC_DeInit()*
- *HAL_TIM_IC_MspInit()*
- *HAL_TIM_IC_MspDeInit()*
- *HAL_TIM_IC_Start()*
- *HAL_TIM_IC_Stop()*
- *HAL_TIM_IC_Start_IT()*
- *HAL_TIM_IC_Stop_IT()*
- *HAL_TIM_IC_Start_DMA()*
- *HAL_TIM_IC_Stop_DMA()*

68.2.7 TIM One Pulse functions

This section provides functions allowing to:

- Initialize and configure the TIM One Pulse.
- De-initialize the TIM One Pulse.
- Start the TIM One Pulse.
- Stop the TIM One Pulse.
- Start the TIM One Pulse and enable interrupt.
- Stop the TIM One Pulse and disable interrupt.
- Start the TIM One Pulse and enable DMA transfer.
- Stop the TIM One Pulse and disable DMA transfer.

This section contains the following APIs:

- *HAL_TIM_OnePulse_Init()*
- *HAL_TIM_OnePulse_DeInit()*
- *HAL_TIM_OnePulse_MspInit()*

- *HAL_TIM_OnePulse_MspDeInit()*
- *HAL_TIM_OnePulse_Start()*
- *HAL_TIM_OnePulse_Stop()*
- *HAL_TIM_OnePulse_Start_IT()*
- *HAL_TIM_OnePulse_Stop_IT()*

68.2.8 TIM Encoder functions

This section provides functions allowing to:

- Initialize and configure the TIM Encoder.
- De-initialize the TIM Encoder.
- Start the TIM Encoder.
- Stop the TIM Encoder.
- Start the TIM Encoder and enable interrupt.
- Stop the TIM Encoder and disable interrupt.
- Start the TIM Encoder and enable DMA transfer.
- Stop the TIM Encoder and disable DMA transfer.

This section contains the following APIs:

- *HAL_TIM_Encoder_Init()*
- *HAL_TIM_Encoder_DeInit()*
- *HAL_TIM_Encoder_MspInit()*
- *HAL_TIM_Encoder_MspDeInit()*
- *HAL_TIM_Encoder_Start()*
- *HAL_TIM_Encoder_Stop()*
- *HAL_TIM_Encoder_Start_IT()*
- *HAL_TIM_Encoder_Stop_IT()*
- *HAL_TIM_Encoder_Start_DMA()*
- *HAL_TIM_Encoder_Stop_DMA()*

68.2.9 TIM Callbacks functions

This section provides TIM callback functions:

- TIM Period elapsed callback
- TIM Output Compare callback
- TIM Input capture callback
- TIM Trigger callback
- TIM Error callback

This section contains the following APIs:

- *HAL_TIM_PeriodElapsedCallback()*
- *HAL_TIM_PeriodElapsedHalfCpltCallback()*
- *HAL_TIM_OC_DelayElapsedCallback()*
- *HAL_TIM_IC_CaptureCallback()*
- *HAL_TIM_IC_CaptureHalfCpltCallback()*
- *HAL_TIM_PWM_PulseFinishedCallback()*
- *HAL_TIM_PWM_PulseFinishedHalfCpltCallback()*
- *HAL_TIM_TriggerCallback()*
- *HAL_TIM_TriggerHalfCpltCallback()*
- *HAL_TIM_ErrorCallback()*
- *HAL_TIM_RegisterCallback()*
- *HAL_TIM_UnRegisterCallback()*

68.2.10 Detailed description of functions

HAL_TIM_Base_Init

Function name

HAL_StatusTypeDef HAL_TIM_Base_Init (TIM_HandleTypeDef * htim)

Function description

Initializes the TIM Time base Unit according to the specified parameters in the TIM_HandleTypeDef and initialize the associated handle.

Parameters

- **htim:** TIM Base handle

Return values

- **HAL:** status

Notes

- Switching from Center Aligned counter mode to Edge counter mode (or reverse) requires a timer reset to avoid unexpected direction due to DIR bit readonly in center aligned mode. Ex: call HAL_TIM_Base_DeInit() before HAL_TIM_Base_Init()

HAL_TIM_Base_DeInit

Function name

HAL_StatusTypeDef HAL_TIM_Base_DeInit (TIM_HandleTypeDef * htim)

Function description

Deinitializes the TIM Base peripheral.

Parameters

- **htim:** TIM Base handle

Return values

- **HAL:** status

HAL_TIM_Base_MspInit

Function name

void HAL_TIM_Base_MspInit (TIM_HandleTypeDef * htim)

Function description

Initializes the TIM Base MSP.

Parameters

- **htim:** TIM Base handle

Return values

- **None:**

HAL_TIM_Base_MspDeInit

Function name

void HAL_TIM_Base_MspDeInit (TIM_HandleTypeDef * htim)

Function description

Deinitializes TIM Base MSP.

Parameters

- **htim**: TIM Base handle

Return values

- **None**:

HAL_TIM_Base_Start

Function name

HAL_StatusTypeDef HAL_TIM_Base_Start (TIM_HandleTypeDef * htim)

Function description

Starts the TIM Base generation.

Parameters

- **htim**: TIM Base handle

Return values

- **HAL**: status

HAL_TIM_Base_Stop

Function name

HAL_StatusTypeDef HAL_TIM_Base_Stop (TIM_HandleTypeDef * htim)

Function description

Stops the TIM Base generation.

Parameters

- **htim**: TIM Base handle

Return values

- **HAL**: status

HAL_TIM_Base_Start_IT

Function name

HAL_StatusTypeDef HAL_TIM_Base_Start_IT (TIM_HandleTypeDef * htim)

Function description

Starts the TIM Base generation in interrupt mode.

Parameters

- **htim**: TIM Base handle

Return values

- **HAL**: status

HAL_TIM_Base_Stop_IT

Function name

HAL_StatusTypeDef HAL_TIM_Base_Stop_IT (TIM_HandleTypeDef * htim)

Function description

Stops the TIM Base generation in interrupt mode.

Parameters

- **htim**: TIM Base handle

Return values

- **HAL:** status

HAL_TIM_Base_Start_DMA

Function name

```
HAL_StatusTypeDef HAL_TIM_Base_Start_DMA (TIM_HandleTypeDef * htim, const uint32_t * pData,
uint16_t Length)
```

Function description

Starts the TIM Base generation in DMA mode.

Parameters

- **htim:** TIM Base handle
- **pData:** The source Buffer address.
- **Length:** The length of data to be transferred from memory to peripheral.

Return values

- **HAL:** status

HAL_TIM_Base_Stop_DMA

Function name

```
HAL_StatusTypeDef HAL_TIM_Base_Stop_DMA (TIM_HandleTypeDef * htim)
```

Function description

Stops the TIM Base generation in DMA mode.

Parameters

- **htim:** TIM Base handle

Return values

- **HAL:** status

HAL_TIM_OC_Init

Function name

```
HAL_StatusTypeDef HAL_TIM_OC_Init (TIM_HandleTypeDef * htim)
```

Function description

Initializes the TIM Output Compare according to the specified parameters in the TIM_HandleTypeDef and initializes the associated handle.

Parameters

- **htim:** TIM Output Compare handle

Return values

- **HAL:** status

Notes

- Switching from Center Aligned counter mode to Edge counter mode (or reverse) requires a timer reset to avoid unexpected direction due to DIR bit readonly in center aligned mode. Ex: call HAL_TIM_OC_DeInit() before HAL_TIM_OC_Init()

HAL_TIM_OC_DeInit

Function name

```
HAL_StatusTypeDef HAL_TIM_OC_DeInit (TIM_HandleTypeDef * htim)
```

Function description

DeInitializes the TIM peripheral.

Parameters

- **htim:** TIM Output Compare handle

Return values

- **HAL:** status

HAL_TIM_OC_MspInit

Function name

void HAL_TIM_OC_MspInit (TIM_HandleTypeDef * htim)

Function description

Initializes the TIM Output Compare MSP.

Parameters

- **htim:** TIM Output Compare handle

Return values

- **None:**

HAL_TIM_OC_MspDeInit

Function name

void HAL_TIM_OC_MspDeInit (TIM_HandleTypeDef * htim)

Function description

DeInitializes TIM Output Compare MSP.

Parameters

- **htim:** TIM Output Compare handle

Return values

- **None:**

HAL_TIM_OC_Start

Function name

HAL_StatusTypeDef HAL_TIM_OC_Start (TIM_HandleTypeDef * htim, uint32_t Channel)

Function description

Starts the TIM Output Compare signal generation.

Parameters

- **htim:** TIM Output Compare handle
- **Channel:** TIM Channel to be enabled This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1 selected
 - TIM_CHANNEL_2: TIM Channel 2 selected
 - TIM_CHANNEL_3: TIM Channel 3 selected
 - TIM_CHANNEL_4: TIM Channel 4 selected

Return values

- **HAL:** status

HAL_TIM_OC_Stop

Function name

HAL_StatusTypeDef HAL_TIM_OC_Stop (TIM_HandleTypeDef * htim, uint32_t Channel)

Function description

Stops the TIM Output Compare signal generation.

Parameters

- **htim:** TIM Output Compare handle
- **Channel:** TIM Channel to be disabled This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1 selected
 - TIM_CHANNEL_2: TIM Channel 2 selected
 - TIM_CHANNEL_3: TIM Channel 3 selected
 - TIM_CHANNEL_4: TIM Channel 4 selected

Return values

- **HAL:** status

HAL_TIM_OC_Start_IT

Function name

HAL_StatusTypeDef HAL_TIM_OC_Start_IT (TIM_HandleTypeDef * htim, uint32_t Channel)

Function description

Starts the TIM Output Compare signal generation in interrupt mode.

Parameters

- **htim:** TIM Output Compare handle
- **Channel:** TIM Channel to be enabled This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1 selected
 - TIM_CHANNEL_2: TIM Channel 2 selected
 - TIM_CHANNEL_3: TIM Channel 3 selected
 - TIM_CHANNEL_4: TIM Channel 4 selected

Return values

- **HAL:** status

HAL_TIM_OC_Stop_IT

Function name

HAL_StatusTypeDef HAL_TIM_OC_Stop_IT (TIM_HandleTypeDef * htim, uint32_t Channel)

Function description

Stops the TIM Output Compare signal generation in interrupt mode.

Parameters

- **htim:** TIM Output Compare handle
- **Channel:** TIM Channel to be disabled This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1 selected
 - TIM_CHANNEL_2: TIM Channel 2 selected
 - TIM_CHANNEL_3: TIM Channel 3 selected
 - TIM_CHANNEL_4: TIM Channel 4 selected

Return values

- **HAL:** status

HAL_TIM_OC_Start_DMA

Function name

HAL_StatusTypeDef HAL_TIM_OC_Start_DMA (TIM_HandleTypeDef * htim, uint32_t Channel, const uint32_t * pData, uint16_t Length)

Function description

Starts the TIM Output Compare signal generation in DMA mode.

Parameters

- **htim:** TIM Output Compare handle
- **Channel:** TIM Channel to be enabled This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1 selected
 - TIM_CHANNEL_2: TIM Channel 2 selected
 - TIM_CHANNEL_3: TIM Channel 3 selected
 - TIM_CHANNEL_4: TIM Channel 4 selected
- **pData:** The source Buffer address.
- **Length:** The length of data to be transferred from memory to TIM peripheral

Return values

- **HAL:** status

HAL_TIM_OC_Stop_DMA

Function name

HAL_StatusTypeDef HAL_TIM_OC_Stop_DMA (TIM_HandleTypeDef * htim, uint32_t Channel)

Function description

Stops the TIM Output Compare signal generation in DMA mode.

Parameters

- **htim:** TIM Output Compare handle
- **Channel:** TIM Channel to be disabled This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1 selected
 - TIM_CHANNEL_2: TIM Channel 2 selected
 - TIM_CHANNEL_3: TIM Channel 3 selected
 - TIM_CHANNEL_4: TIM Channel 4 selected

Return values

- **HAL:** status

HAL_TIM_PWM_Init

Function name

HAL_StatusTypeDef HAL_TIM_PWM_Init (TIM_HandleTypeDef * htim)

Function description

Initializes the TIM PWM Time Base according to the specified parameters in the TIM_HandleTypeDef and initializes the associated handle.

Parameters

- **htim:** TIM PWM handle

Return values

- **HAL:** status

Notes

- Switching from Center Aligned counter mode to Edge counter mode (or reverse) requires a timer reset to avoid unexpected direction due to DIR bit readonly in center aligned mode. Ex: call HAL_TIM_PWM_DeInit() before HAL_TIM_PWM_Init()

HAL_TIM_PWM_DeInit

Function name

HAL_StatusTypeDef HAL_TIM_PWM_DeInit (TIM_HandleTypeDef * htim)

Function description

DeInitializes the TIM peripheral.

Parameters

- **htim:** TIM PWM handle

Return values

- **HAL:** status

HAL_TIM_PWM_MspInit

Function name

void HAL_TIM_PWM_MspInit (TIM_HandleTypeDef * htim)

Function description

Initializes the TIM PWM MSP.

Parameters

- **htim:** TIM PWM handle

Return values

- **None:**

HAL_TIM_PWM_MspDeInit

Function name

void HAL_TIM_PWM_MspDeInit (TIM_HandleTypeDef * htim)

Function description

DeInitializes TIM PWM MSP.

Parameters

- **htim:** TIM PWM handle

Return values

- **None:**

HAL_TIM_PWM_Start

Function name

HAL_StatusTypeDef HAL_TIM_PWM_Start (TIM_HandleTypeDef * htim, uint32_t Channel)

Function description

Starts the PWM signal generation.

Parameters

- **htim:** TIM handle
- **Channel:** TIM Channels to be enabled This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1 selected
 - TIM_CHANNEL_2: TIM Channel 2 selected
 - TIM_CHANNEL_3: TIM Channel 3 selected
 - TIM_CHANNEL_4: TIM Channel 4 selected

Return values

- **HAL:** status

HAL_TIM_PWM_Stop

Function name

HAL_StatusTypeDef HAL_TIM_PWM_Stop (TIM_HandleTypeDef * htim, uint32_t Channel)

Function description

Stops the PWM signal generation.

Parameters

- **htim:** TIM PWM handle
- **Channel:** TIM Channels to be disabled This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1 selected
 - TIM_CHANNEL_2: TIM Channel 2 selected
 - TIM_CHANNEL_3: TIM Channel 3 selected
 - TIM_CHANNEL_4: TIM Channel 4 selected

Return values

- **HAL:** status

HAL_TIM_PWM_Start_IT

Function name

HAL_StatusTypeDef HAL_TIM_PWM_Start_IT (TIM_HandleTypeDef * htim, uint32_t Channel)

Function description

Starts the PWM signal generation in interrupt mode.

Parameters

- **htim:** TIM PWM handle
- **Channel:** TIM Channel to be enabled This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1 selected
 - TIM_CHANNEL_2: TIM Channel 2 selected
 - TIM_CHANNEL_3: TIM Channel 3 selected
 - TIM_CHANNEL_4: TIM Channel 4 selected

Return values

- **HAL:** status

HAL_TIM_PWM_Stop_IT

Function name

HAL_StatusTypeDef HAL_TIM_PWM_Stop_IT (TIM_HandleTypeDef * htim, uint32_t Channel)

Function description

Stops the PWM signal generation in interrupt mode.

Parameters

- **htim:** TIM PWM handle
- **Channel:** TIM Channels to be disabled This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1 selected
 - TIM_CHANNEL_2: TIM Channel 2 selected
 - TIM_CHANNEL_3: TIM Channel 3 selected
 - TIM_CHANNEL_4: TIM Channel 4 selected

Return values

- **HAL:** status

HAL_TIM_PWM_Start_DMA

Function name

HAL_StatusTypeDef HAL_TIM_PWM_Start_DMA (TIM_HandleTypeDef * htim, uint32_t Channel, const uint32_t * pData, uint16_t Length)

Function description

Starts the TIM PWM signal generation in DMA mode.

Parameters

- **htim:** TIM PWM handle
- **Channel:** TIM Channels to be enabled This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1 selected
 - TIM_CHANNEL_2: TIM Channel 2 selected
 - TIM_CHANNEL_3: TIM Channel 3 selected
 - TIM_CHANNEL_4: TIM Channel 4 selected
- **pData:** The source Buffer address.
- **Length:** The length of data to be transferred from memory to TIM peripheral

Return values

- **HAL:** status

HAL_TIM_PWM_Stop_DMA

Function name

HAL_StatusTypeDef HAL_TIM_PWM_Stop_DMA (TIM_HandleTypeDef * htim, uint32_t Channel)

Function description

Stops the TIM PWM signal generation in DMA mode.

Parameters

- **htim:** TIM PWM handle
- **Channel:** TIM Channels to be disabled This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1 selected
 - TIM_CHANNEL_2: TIM Channel 2 selected
 - TIM_CHANNEL_3: TIM Channel 3 selected
 - TIM_CHANNEL_4: TIM Channel 4 selected

Return values

- **HAL:** status

HAL_TIM_IC_Init

Function name

HAL_StatusTypeDef HAL_TIM_IC_Init (TIM_HandleTypeDef * htim)

Function description

Initializes the TIM Input Capture Time base according to the specified parameters in the TIM_HandleTypeDef and initializes the associated handle.

Parameters

- **htim:** TIM Input Capture handle

Return values

- **HAL:** status

Notes

- Switching from Center Aligned counter mode to Edge counter mode (or reverse) requires a timer reset to avoid unexpected direction due to DIR bit readonly in center aligned mode. Ex: call HAL_TIM_IC_DeInit() before HAL_TIM_IC_Init()

HAL_TIM_IC_DeInit

Function name

HAL_StatusTypeDef HAL_TIM_IC_DeInit (TIM_HandleTypeDef * htim)

Function description

Deinitializes the TIM peripheral.

Parameters

- **htim:** TIM Input Capture handle

Return values

- **HAL:** status

HAL_TIM_IC_MspInit

Function name

void HAL_TIM_IC_MspInit (TIM_HandleTypeDef * htim)

Function description

Initializes the TIM Input Capture MSP.

Parameters

- **htim:** TIM Input Capture handle

Return values

- **None:**

HAL_TIM_IC_MspDeInit

Function name

void HAL_TIM_IC_MspDeInit (TIM_HandleTypeDef * htim)

Function description

Deinitializes TIM Input Capture MSP.

Parameters

- **htim:** TIM handle

Return values

- **None:**

HAL_TIM_IC_Start**Function name**

HAL_StatusTypeDef HAL_TIM_IC_Start (TIM_HandleTypeDef * htim, uint32_t Channel)

Function description

Starts the TIM Input Capture measurement.

Parameters

- **htim:** TIM Input Capture handle
- **Channel:** TIM Channels to be enabled This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1 selected
 - TIM_CHANNEL_2: TIM Channel 2 selected
 - TIM_CHANNEL_3: TIM Channel 3 selected
 - TIM_CHANNEL_4: TIM Channel 4 selected

Return values

- **HAL:** status

HAL_TIM_IC_Stop**Function name**

HAL_StatusTypeDef HAL_TIM_IC_Stop (TIM_HandleTypeDef * htim, uint32_t Channel)

Function description

Stops the TIM Input Capture measurement.

Parameters

- **htim:** TIM Input Capture handle
- **Channel:** TIM Channels to be disabled This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1 selected
 - TIM_CHANNEL_2: TIM Channel 2 selected
 - TIM_CHANNEL_3: TIM Channel 3 selected
 - TIM_CHANNEL_4: TIM Channel 4 selected

Return values

- **HAL:** status

HAL_TIM_IC_Start_IT**Function name**

HAL_StatusTypeDef HAL_TIM_IC_Start_IT (TIM_HandleTypeDef * htim, uint32_t Channel)

Function description

Starts the TIM Input Capture measurement in interrupt mode.

Parameters

- **htim:** TIM Input Capture handle
- **Channel:** TIM Channels to be enabled This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1 selected
 - TIM_CHANNEL_2: TIM Channel 2 selected
 - TIM_CHANNEL_3: TIM Channel 3 selected
 - TIM_CHANNEL_4: TIM Channel 4 selected

Return values

- **HAL:** status

HAL_TIM_IC_Stop_IT

Function name

HAL_StatusTypeDef HAL_TIM_IC_Stop_IT (TIM_HandleTypeDef * htim, uint32_t Channel)

Function description

Stops the TIM Input Capture measurement in interrupt mode.

Parameters

- **htim:** TIM Input Capture handle
- **Channel:** TIM Channels to be disabled This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1 selected
 - TIM_CHANNEL_2: TIM Channel 2 selected
 - TIM_CHANNEL_3: TIM Channel 3 selected
 - TIM_CHANNEL_4: TIM Channel 4 selected

Return values

- **HAL:** status

HAL_TIM_IC_Start_DMA

Function name

HAL_StatusTypeDef HAL_TIM_IC_Start_DMA (TIM_HandleTypeDef * htim, uint32_t Channel, uint32_t * pData, uint16_t Length)

Function description

Starts the TIM Input Capture measurement in DMA mode.

Parameters

- **htim:** TIM Input Capture handle
- **Channel:** TIM Channels to be enabled This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1 selected
 - TIM_CHANNEL_2: TIM Channel 2 selected
 - TIM_CHANNEL_3: TIM Channel 3 selected
 - TIM_CHANNEL_4: TIM Channel 4 selected
- **pData:** The destination Buffer address.
- **Length:** The length of data to be transferred from TIM peripheral to memory.

Return values

- **HAL:** status

HAL_TIM_IC_Stop_DMA

Function name

HAL_StatusTypeDef HAL_TIM_IC_Stop_DMA (TIM_HandleTypeDef * htim, uint32_t Channel)

Function description

Stops the TIM Input Capture measurement in DMA mode.

Parameters

- **htim:** TIM Input Capture handle
- **Channel:** TIM Channels to be disabled This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1 selected
 - TIM_CHANNEL_2: TIM Channel 2 selected
 - TIM_CHANNEL_3: TIM Channel 3 selected
 - TIM_CHANNEL_4: TIM Channel 4 selected

Return values

- **HAL:** status

HAL_TIM_OnePulse_Init

Function name

HAL_StatusTypeDef HAL_TIM_OnePulse_Init (TIM_HandleTypeDef * htim, uint32_t OnePulseMode)

Function description

Initializes the TIM One Pulse Time Base according to the specified parameters in the TIM_HandleTypeDef and initializes the associated handle.

Parameters

- **htim:** TIM One Pulse handle
- **OnePulseMode:** Select the One pulse mode. This parameter can be one of the following values:
 - TIM_OPMODE_SINGLE: Only one pulse will be generated.
 - TIM_OPMODE_REPETITIVE: Repetitive pulses will be generated.

Return values

- **HAL:** status

Notes

- Switching from Center Aligned counter mode to Edge counter mode (or reverse) requires a timer reset to avoid unexpected direction due to DIR bit readonly in center aligned mode. Ex: call HAL_TIM_OnePulse_DeInit() before HAL_TIM_OnePulse_Init()
- When the timer instance is initialized in One Pulse mode, timer channels 1 and channel 2 are reserved and cannot be used for other purpose.

HAL_TIM_OnePulse_DeInit

Function name

HAL_StatusTypeDef HAL_TIM_OnePulse_DeInit (TIM_HandleTypeDef * htim)

Function description

Deinitializes the TIM One Pulse.

Parameters

- **htim:** TIM One Pulse handle

Return values

- **HAL:** status

HAL_TIM_OnePulse_MspInit

Function name

```
void HAL_TIM_OnePulse_MspInit (TIM_HandleTypeDef * htim)
```

Function description

Initializes the TIM One Pulse MSP.

Parameters

- **htim:** TIM One Pulse handle

Return values

- **None:**

HAL_TIM_OnePulse_MspDeInit

Function name

```
void HAL_TIM_OnePulse_MspDeInit (TIM_HandleTypeDef * htim)
```

Function description

DeInitializes TIM One Pulse MSP.

Parameters

- **htim:** TIM One Pulse handle

Return values

- **None:**

HAL_TIM_OnePulse_Start

Function name

```
HAL_StatusTypeDef HAL_TIM_OnePulse_Start (TIM_HandleTypeDef * htim, uint32_t OutputChannel)
```

Function description

Starts the TIM One Pulse signal generation.

Parameters

- **htim:** TIM One Pulse handle
- **OutputChannel:** See note above

Return values

- **HAL:** status

Notes

- Though OutputChannel parameter is deprecated and ignored by the function it has been kept to avoid HAL_TIM API compatibility break.
- The pulse output channel is determined when calling HAL_TIM_OnePulse_ConfigChannel().

HAL_TIM_OnePulse_Stop

Function name

```
HAL_StatusTypeDef HAL_TIM_OnePulse_Stop (TIM_HandleTypeDef * htim, uint32_t OutputChannel)
```

Function description

Stops the TIM One Pulse signal generation.

Parameters

- **htim:** TIM One Pulse handle
- **OutputChannel:** See note above

Return values

- **HAL:** status

Notes

- Though OutputChannel parameter is deprecated and ignored by the function it has been kept to avoid HAL_TIM API compatibility break.
- The pulse output channel is determined when calling HAL_TIM_OnePulse_ConfigChannel().

HAL_TIM_OnePulse_Start_IT

Function name

HAL_StatusTypeDef HAL_TIM_OnePulse_Start_IT (TIM_HandleTypeDef * htim, uint32_t OutputChannel)

Function description

Starts the TIM One Pulse signal generation in interrupt mode.

Parameters

- **htim:** TIM One Pulse handle
- **OutputChannel:** See note above

Return values

- **HAL:** status

Notes

- Though OutputChannel parameter is deprecated and ignored by the function it has been kept to avoid HAL_TIM API compatibility break.
- The pulse output channel is determined when calling HAL_TIM_OnePulse_ConfigChannel().

HAL_TIM_OnePulse_Stop_IT

Function name

HAL_StatusTypeDef HAL_TIM_OnePulse_Stop_IT (TIM_HandleTypeDef * htim, uint32_t OutputChannel)

Function description

Stops the TIM One Pulse signal generation in interrupt mode.

Parameters

- **htim:** TIM One Pulse handle
- **OutputChannel:** See note above

Return values

- **HAL:** status

Notes

- Though OutputChannel parameter is deprecated and ignored by the function it has been kept to avoid HAL_TIM API compatibility break.
- The pulse output channel is determined when calling HAL_TIM_OnePulse_ConfigChannel().

HAL_TIM_Encoder_Init

Function name

HAL_StatusTypeDef HAL_TIM_Encoder_Init (TIM_HandleTypeDef * htim, TIM_Encoder_InitTypeDef * sConfig)

Function description

Initializes the TIM Encoder Interface and initialize the associated handle.

Parameters

- **htim:** TIM Encoder Interface handle
- **sConfig:** TIM Encoder Interface configuration structure

Return values

- **HAL:** status

Notes

- Switching from Center Aligned counter mode to Edge counter mode (or reverse) requires a timer reset to avoid unexpected direction due to DIR bit readonly in center aligned mode. Ex: call HAL_TIM_Encoder_DeInit() before HAL_TIM_Encoder_Init()
- Encoder mode and External clock mode 2 are not compatible and must not be selected together Ex: A call for HAL_TIM_Encoder_Init will erase the settings of HAL_TIM_ConfigClockSource using TIM_CLOCKSOURCE_ETRMODE2 and vice versa
- When the timer instance is initialized in Encoder mode, timer channels 1 and channel 2 are reserved and cannot be used for other purpose.

HAL_TIM_Encoder_DeInit

Function name

HAL_StatusTypeDef HAL_TIM_Encoder_DeInit (TIM_HandleTypeDef * htim)

Function description

DeInitializes the TIM Encoder interface.

Parameters

- **htim:** TIM Encoder Interface handle

Return values

- **HAL:** status

HAL_TIM_Encoder_MspInit

Function name

void HAL_TIM_Encoder_MspInit (TIM_HandleTypeDef * htim)

Function description

Initializes the TIM Encoder Interface MSP.

Parameters

- **htim:** TIM Encoder Interface handle

Return values

- **None:**

HAL_TIM_Encoder_MspDeInit

Function name

void HAL_TIM_Encoder_MspDeInit (TIM_HandleTypeDef * htim)

Function description

Deinitializes TIM Encoder Interface MSP.

Parameters

- **htim:** TIM Encoder Interface handle

Return values

- **None:**

HAL_TIM_Encoder_Start

Function name

HAL_StatusTypeDef HAL_TIM_Encoder_Start (TIM_HandleTypeDef * htim, uint32_t Channel)

Function description

Starts the TIM Encoder Interface.

Parameters

- **htim:** TIM Encoder Interface handle
- **Channel:** TIM Channels to be enabled This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1 selected
 - TIM_CHANNEL_2: TIM Channel 2 selected
 - TIM_CHANNEL_ALL: TIM Channel 1 and TIM Channel 2 are selected

Return values

- **HAL:** status

HAL_TIM_Encoder_Stop

Function name

HAL_StatusTypeDef HAL_TIM_Encoder_Stop (TIM_HandleTypeDef * htim, uint32_t Channel)

Function description

Stops the TIM Encoder Interface.

Parameters

- **htim:** TIM Encoder Interface handle
- **Channel:** TIM Channels to be disabled This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1 selected
 - TIM_CHANNEL_2: TIM Channel 2 selected
 - TIM_CHANNEL_ALL: TIM Channel 1 and TIM Channel 2 are selected

Return values

- **HAL:** status

HAL_TIM_Encoder_Start_IT

Function name

HAL_StatusTypeDef HAL_TIM_Encoder_Start_IT (TIM_HandleTypeDef * htim, uint32_t Channel)

Function description

Starts the TIM Encoder Interface in interrupt mode.

Parameters

- **htim:** TIM Encoder Interface handle
- **Channel:** TIM Channels to be enabled This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1 selected
 - TIM_CHANNEL_2: TIM Channel 2 selected
 - TIM_CHANNEL_ALL: TIM Channel 1 and TIM Channel 2 are selected

Return values

- **HAL:** status

HAL_TIM_Encoder_Stop_IT

Function name

HAL_StatusTypeDef HAL_TIM_Encoder_Stop_IT (TIM_HandleTypeDef * htim, uint32_t Channel)

Function description

Stops the TIM Encoder Interface in interrupt mode.

Parameters

- **htim:** TIM Encoder Interface handle
- **Channel:** TIM Channels to be disabled This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1 selected
 - TIM_CHANNEL_2: TIM Channel 2 selected
 - TIM_CHANNEL_ALL: TIM Channel 1 and TIM Channel 2 are selected

Return values

- **HAL:** status

HAL_TIM_Encoder_Start_DMA

Function name

HAL_StatusTypeDef HAL_TIM_Encoder_Start_DMA (TIM_HandleTypeDef * htim, uint32_t Channel, uint32_t * pData1, uint32_t * pData2, uint16_t Length)

Function description

Starts the TIM Encoder Interface in DMA mode.

Parameters

- **htim:** TIM Encoder Interface handle
- **Channel:** TIM Channels to be enabled This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1 selected
 - TIM_CHANNEL_2: TIM Channel 2 selected
 - TIM_CHANNEL_ALL: TIM Channel 1 and TIM Channel 2 are selected
- **pData1:** The destination Buffer address for IC1.
- **pData2:** The destination Buffer address for IC2.
- **Length:** The length of data to be transferred from TIM peripheral to memory.

Return values

- **HAL:** status

HAL_TIM_Encoder_Stop_DMA

Function name

HAL_StatusTypeDef HAL_TIM_Encoder_Stop_DMA (TIM_HandleTypeDef * htim, uint32_t Channel)

Function description

Stops the TIM Encoder Interface in DMA mode.

Parameters

- **htim:** TIM Encoder Interface handle
- **Channel:** TIM Channels to be enabled This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1 selected
 - TIM_CHANNEL_2: TIM Channel 2 selected
 - TIM_CHANNEL_ALL: TIM Channel 1 and TIM Channel 2 are selected

Return values

- **HAL:** status

HAL_TIM_IRQHandler

Function name

void HAL_TIM_IRQHandler (TIM_HandleTypeDef * htim)

Function description

This function handles TIM interrupts requests.

Parameters

- **htim:** TIM handle

Return values

- **None:**

HAL_TIM_OC_ConfigChannel

Function name

HAL_StatusTypeDef HAL_TIM_OC_ConfigChannel (TIM_HandleTypeDef * htim, const TIM_OC_InitTypeDef * sConfig, uint32_t Channel)

Function description

Initializes the TIM Output Compare Channels according to the specified parameters in the TIM_OC_InitTypeDef.

Parameters

- **htim:** TIM Output Compare handle
- **sConfig:** TIM Output Compare configuration structure
- **Channel:** TIM Channels to configure This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1 selected
 - TIM_CHANNEL_2: TIM Channel 2 selected
 - TIM_CHANNEL_3: TIM Channel 3 selected
 - TIM_CHANNEL_4: TIM Channel 4 selected

Return values

- **HAL:** status

HAL_TIM_PWM_ConfigChannel

Function name

```
HAL_StatusTypeDef HAL_TIM_PWM_ConfigChannel (TIM_HandleTypeDef * htim, const
TIM_OC_InitTypeDef * sConfig, uint32_t Channel)
```

Function description

Initializes the TIM PWM channels according to the specified parameters in the TIM_OC_InitTypeDef.

Parameters

- **htim:** TIM PWM handle
- **sConfig:** TIM PWM configuration structure
- **Channel:** TIM Channels to be configured This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1 selected
 - TIM_CHANNEL_2: TIM Channel 2 selected
 - TIM_CHANNEL_3: TIM Channel 3 selected
 - TIM_CHANNEL_4: TIM Channel 4 selected

Return values

- **HAL:** status

HAL_TIM_IC_ConfigChannel

Function name

```
HAL_StatusTypeDef HAL_TIM_IC_ConfigChannel (TIM_HandleTypeDef * htim, const TIM_IC_InitTypeDef
* sConfig, uint32_t Channel)
```

Function description

Initializes the TIM Input Capture Channels according to the specified parameters in the TIM_IC_InitTypeDef.

Parameters

- **htim:** TIM IC handle
- **sConfig:** TIM Input Capture configuration structure
- **Channel:** TIM Channel to configure This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1 selected
 - TIM_CHANNEL_2: TIM Channel 2 selected
 - TIM_CHANNEL_3: TIM Channel 3 selected
 - TIM_CHANNEL_4: TIM Channel 4 selected

Return values

- **HAL:** status

HAL_TIM_OnePulse_ConfigChannel

Function name

```
HAL_StatusTypeDef HAL_TIM_OnePulse_ConfigChannel (TIM_HandleTypeDef * htim,
TIM_OnePulse_InitTypeDef * sConfig, uint32_t OutputChannel, uint32_t InputChannel)
```

Function description

Initializes the TIM One Pulse Channels according to the specified parameters in the TIM_OnePulse_InitTypeDef.

Parameters

- **htim**: TIM One Pulse handle
- **sConfig**: TIM One Pulse configuration structure
- **OutputChannel**: TIM output channel to configure This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1 selected
 - TIM_CHANNEL_2: TIM Channel 2 selected
- **InputChannel**: TIM input Channel to configure This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1 selected
 - TIM_CHANNEL_2: TIM Channel 2 selected

Return values

- **HAL**: status

Notes

- To output a waveform with a minimum delay user can enable the fast mode by calling the `__HAL_TIM_ENABLE_OCxFAST` macro. Then CCx output is forced in response to the edge detection on Tlx input, without taking in account the comparison.

HAL_TIM_ConfigOCrefClear

Function name

```
HAL_StatusTypeDef HAL_TIM_ConfigOCrefClear(TIM_HandleTypeDef * htim, const
TIM_ClearInputConfigTypeDef * sClearInputConfig, uint32_t Channel)
```

Function description

Configures the OCREf clear feature.

Parameters

- **htim**: TIM handle
- **sClearInputConfig**: pointer to a TIM_ClearInputConfigTypeDef structure that contains the OCREf clear feature and parameters for the TIM peripheral.
- **Channel**: specifies the TIM Channel This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1
 - TIM_CHANNEL_2: TIM Channel 2
 - TIM_CHANNEL_3: TIM Channel 3
 - TIM_CHANNEL_4: TIM Channel 4

Return values

- **HAL**: status

HAL_TIM_ConfigClockSource

Function name

```
HAL_StatusTypeDef HAL_TIM_ConfigClockSource(TIM_HandleTypeDef * htim, const
TIM_ClockConfigTypeDef * sClockSourceConfig)
```

Function description

Configures the clock source to be used.

Parameters

- **htim**: TIM handle
- **sClockSourceConfig**: pointer to a TIM_ClockConfigTypeDef structure that contains the clock source information for the TIM peripheral.

Return values

- **HAL**: status

HAL_TIM_ConfigTI1Input

Function name

HAL_StatusTypeDef HAL_TIM_ConfigTI1Input (TIM_HandleTypeDef * htim, uint32_t TI1_Selection)

Function description

Selects the signal connected to the TI1 input: direct from CH1_input or a XOR combination between CH1_input, CH2_input & CH3_input.

Parameters

- **htim:** TIM handle.
- **TI1_Selection:** Indicate whether or not channel 1 is connected to the output of a XOR gate. This parameter can be one of the following values:
 - **TIM_TI1SELECTION_CH1:** The TIMx_CH1 pin is connected to TI1 input
 - **TIM_TI1SELECTION_XORCOMBINATION:** The TIMx_CH1, CH2 and CH3 pins are connected to the TI1 input (XOR combination)

Return values

- **HAL:** status

HAL_TIM_SlaveConfigSynchro

Function name

HAL_StatusTypeDef HAL_TIM_SlaveConfigSynchro (TIM_HandleTypeDef * htim, const TIM_SlaveConfigTypeDef * sSlaveConfig)

Function description

Configures the TIM in Slave mode.

Parameters

- **htim:** TIM handle.
- **sSlaveConfig:** pointer to a TIM_SlaveConfigTypeDef structure that contains the selected trigger (internal trigger input, filtered timer input or external trigger input) and the Slave mode (Disable, Reset, Gated, Trigger, External clock mode 1).

Return values

- **HAL:** status

HAL_TIM_SlaveConfigSynchro_IT

Function name

HAL_StatusTypeDef HAL_TIM_SlaveConfigSynchro_IT (TIM_HandleTypeDef * htim, const TIM_SlaveConfigTypeDef * sSlaveConfig)

Function description

Configures the TIM in Slave mode in interrupt mode.

Parameters

- **htim:** TIM handle.
- **sSlaveConfig:** pointer to a TIM_SlaveConfigTypeDef structure that contains the selected trigger (internal trigger input, filtered timer input or external trigger input) and the Slave mode (Disable, Reset, Gated, Trigger, External clock mode 1).

Return values

- **HAL:** status

HAL_TIM_DMABurst_WriteStart

Function name

HAL_StatusTypeDef HAL_TIM_DMABurst_WriteStart (TIM_HandleTypeDef * htim, uint32_t BurstBaseAddress, uint32_t BurstRequestSrc, const uint32_t * BurstBuffer, uint32_t BurstLength)

Function description

Configure the DMA Burst to transfer Data from the memory to the TIM peripheral.

Parameters

- **htim:** TIM handle
- **BurstBaseAddress:** TIM Base address from where the DMA will start the Data write This parameter can be one of the following values:
 - TIM_DMABASE_CR1
 - TIM_DMABASE_CR2
 - TIM_DMABASE_SMCR
 - TIM_DMABASE_DIER
 - TIM_DMABASE_SR
 - TIM_DMABASE_EGR
 - TIM_DMABASE_CCMR1
 - TIM_DMABASE_CCMR2
 - TIM_DMABASE_CCER
 - TIM_DMABASE_CNT
 - TIM_DMABASE_PSC
 - TIM_DMABASE_ARR
 - TIM_DMABASE_RCR
 - TIM_DMABASE_CCR1
 - TIM_DMABASE_CCR2
 - TIM_DMABASE_CCR3
 - TIM_DMABASE_CCR4
 - TIM_DMABASE_BDTR
- **BurstRequestSrc:** TIM DMA Request sources This parameter can be one of the following values:
 - TIM_DMA_UPDATE: TIM update Interrupt source
 - TIM_DMA_CC1: TIM Capture Compare 1 DMA source
 - TIM_DMA_CC2: TIM Capture Compare 2 DMA source
 - TIM_DMA_CC3: TIM Capture Compare 3 DMA source
 - TIM_DMA_CC4: TIM Capture Compare 4 DMA source
 - TIM_DMA_COM: TIM Commutation DMA source
 - TIM_DMA_TRIGGER: TIM Trigger DMA source
- **BurstBuffer:** The Buffer address.
- **BurstLength:** DMA Burst length. This parameter can be one value between: TIM_DMABURSTLENGTH_1TRANSFER and TIM_DMABURSTLENGTH_18TRANSFERS.

Return values

- **HAL:** status

Notes

- This function should be used only when BurstLength is equal to DMA data transfer length.

HAL_TIM_DMABurst_MultiWriteStart

Function name

```
HAL_StatusTypeDef HAL_TIM_DMABurst_MultiWriteStart (TIM_HandleTypeDef * htim, uint32_t
BurstBaseAddress, uint32_t BurstRequestSrc, const uint32_t * BurstBuffer, uint32_t BurstLength,
uint32_t DataLength)
```

Function description

Configure the DMA Burst to transfer multiple Data from the memory to the TIM peripheral.

Parameters

- **htim:** TIM handle
- **BurstBaseAddress:** TIM Base address from where the DMA will start the Data write This parameter can be one of the following values:
 - TIM_DMABASE_CR1
 - TIM_DMABASE_CR2
 - TIM_DMABASE_SMCR
 - TIM_DMABASE_DIER
 - TIM_DMABASE_SR
 - TIM_DMABASE_EGR
 - TIM_DMABASE_CCMR1
 - TIM_DMABASE_CCMR2
 - TIM_DMABASE_CCER
 - TIM_DMABASE_CNT
 - TIM_DMABASE_PSC
 - TIM_DMABASE_ARR
 - TIM_DMABASE_RCR
 - TIM_DMABASE_CCR1
 - TIM_DMABASE_CCR2
 - TIM_DMABASE_CCR3
 - TIM_DMABASE_CCR4
 - TIM_DMABASE_BDTR
- **BurstRequestSrc:** TIM DMA Request sources This parameter can be one of the following values:
 - TIM_DMA_UPDATE: TIM update Interrupt source
 - TIM_DMA_CC1: TIM Capture Compare 1 DMA source
 - TIM_DMA_CC2: TIM Capture Compare 2 DMA source
 - TIM_DMA_CC3: TIM Capture Compare 3 DMA source
 - TIM_DMA_CC4: TIM Capture Compare 4 DMA source
 - TIM_DMA_COM: TIM Commutation DMA source
 - TIM_DMA_TRIGGER: TIM Trigger DMA source
- **BurstBuffer:** The Buffer address.
- **BurstLength:** DMA Burst length. This parameter can be one value between: TIM_DMABURSTLENGTH_1TRANSFER and TIM_DMABURSTLENGTH_18TRANSFERS.
- **DataLength:** Data length. This parameter can be one value between 1 and 0xFFFF.

Return values

- **HAL:** status

HAL_TIM_DMABurst_WriteStop

Function name

HAL_StatusTypeDef HAL_TIM_DMABurst_WriteStop (TIM_HandleTypeDef * htim, uint32_t BurstRequestSrc)

Function description

Stops the TIM DMA Burst mode.

Parameters

- **htim**: TIM handle
- **BurstRequestSrc**: TIM DMA Request sources to disable

Return values

- **HAL**: status

HAL_TIM_DMABurst_ReadStart

Function name

HAL_StatusTypeDef HAL_TIM_DMABurst_ReadStart (TIM_HandleTypeDef * htim, uint32_t BurstBaseAddress, uint32_t BurstRequestSrc, uint32_t * BurstBuffer, uint32_t BurstLength)

Function description

Configure the DMA Burst to transfer Data from the TIM peripheral to the memory.

Parameters

- **htim:** TIM handle
- **BurstBaseAddress:** TIM Base address from where the DMA will start the Data read This parameter can be one of the following values:
 - TIM_DMABASE_CR1
 - TIM_DMABASE_CR2
 - TIM_DMABASE_SMCR
 - TIM_DMABASE_DIER
 - TIM_DMABASE_SR
 - TIM_DMABASE_EGR
 - TIM_DMABASE_CCMR1
 - TIM_DMABASE_CCMR2
 - TIM_DMABASE_CCER
 - TIM_DMABASE_CNT
 - TIM_DMABASE_PSC
 - TIM_DMABASE_ARR
 - TIM_DMABASE_RCR
 - TIM_DMABASE_CCR1
 - TIM_DMABASE_CCR2
 - TIM_DMABASE_CCR3
 - TIM_DMABASE_CCR4
 - TIM_DMABASE_BDTR
- **BurstRequestSrc:** TIM DMA Request sources This parameter can be one of the following values:
 - TIM_DMA_UPDATE: TIM update Interrupt source
 - TIM_DMA_CC1: TIM Capture Compare 1 DMA source
 - TIM_DMA_CC2: TIM Capture Compare 2 DMA source
 - TIM_DMA_CC3: TIM Capture Compare 3 DMA source
 - TIM_DMA_CC4: TIM Capture Compare 4 DMA source
 - TIM_DMA_COM: TIM Commutation DMA source
 - TIM_DMA_TRIGGER: TIM Trigger DMA source
- **BurstBuffer:** The Buffer address.
- **BurstLength:** DMA Burst length. This parameter can be one value between: TIM_DMABURSTLENGTH_1TRANSFER and TIM_DMABURSTLENGTH_18TRANSFERS.

Return values

- **HAL:** status

Notes

- This function should be used only when BurstLength is equal to DMA data transfer length.

HAL_TIM_DMABurst_MultiReadStart

Function name

```
HAL_StatusTypeDef HAL_TIM_DMABurst_MultiReadStart (TIM_HandleTypeDef * htim, uint32_t
BurstBaseAddress, uint32_t BurstRequestSrc, uint32_t * BurstBuffer, uint32_t BurstLength, uint32_t
DataLength)
```

Function description

Configure the DMA Burst to transfer Data from the TIM peripheral to the memory.

Parameters

- **htim**: TIM handle
- **BurstBaseAddress**: TIM Base address from where the DMA will start the Data read This parameter can be one of the following values:
 - TIM_DMABASE_CR1
 - TIM_DMABASE_CR2
 - TIM_DMABASE_SMCR
 - TIM_DMABASE_DIER
 - TIM_DMABASE_SR
 - TIM_DMABASE_EGR
 - TIM_DMABASE_CCMR1
 - TIM_DMABASE_CCMR2
 - TIM_DMABASE_CCER
 - TIM_DMABASE_CNT
 - TIM_DMABASE_PSC
 - TIM_DMABASE_ARR
 - TIM_DMABASE_RCR
 - TIM_DMABASE_CCR1
 - TIM_DMABASE_CCR2
 - TIM_DMABASE_CCR3
 - TIM_DMABASE_CCR4
 - TIM_DMABASE_BDTR
- **BurstRequestSrc**: TIM DMA Request sources This parameter can be one of the following values:
 - TIM_DMA_UPDATE: TIM update Interrupt source
 - TIM_DMA_CC1: TIM Capture Compare 1 DMA source
 - TIM_DMA_CC2: TIM Capture Compare 2 DMA source
 - TIM_DMA_CC3: TIM Capture Compare 3 DMA source
 - TIM_DMA_CC4: TIM Capture Compare 4 DMA source
 - TIM_DMA_COM: TIM Commutation DMA source
 - TIM_DMA_TRIGGER: TIM Trigger DMA source
- **BurstBuffer**: The Buffer address.
- **BurstLength**: DMA Burst length. This parameter can be one value between: TIM_DMABURSTLENGTH_1TRANSFER and TIM_DMABURSTLENGTH_18TRANSFERS.
- **DataLength**: Data length. This parameter can be one value between 1 and 0xFFFF.

Return values

- **HAL**: status

HAL_TIM_DMABurst_ReadStop

Function name

HAL_StatusTypeDef HAL_TIM_DMABurst_ReadStop (TIM_HandleTypeDef * htim, uint32_t BurstRequestSrc)

Function description

Stop the DMA burst reading.

Parameters

- **htim**: TIM handle
- **BurstRequestSrc**: TIM DMA Request sources to disable.

Return values

- **HAL**: status

HAL_TIM_GenerateEvent

Function name

HAL_StatusTypeDef HAL_TIM_GenerateEvent (TIM_HandleTypeDef * htim, uint32_t EventSource)

Function description

Generate a software event.

Parameters

- **htim:** TIM handle
- **EventSource:** specifies the event source. This parameter can be one of the following values:
 - TIM_EVENTSOURCE_UPDATE: Timer update Event source
 - TIM_EVENTSOURCE_CC1: Timer Capture Compare 1 Event source
 - TIM_EVENTSOURCE_CC2: Timer Capture Compare 2 Event source
 - TIM_EVENTSOURCE_CC3: Timer Capture Compare 3 Event source
 - TIM_EVENTSOURCE_CC4: Timer Capture Compare 4 Event source
 - TIM_EVENTSOURCE_COM: Timer COM event source
 - TIM_EVENTSOURCE_TRIGGER: Timer Trigger Event source
 - TIM_EVENTSOURCE_BREAK: Timer Break event source

Return values

- **HAL:** status

Notes

- Basic timers can only generate an update event.
- TIM_EVENTSOURCE_COM is relevant only with advanced timer instances.
- TIM_EVENTSOURCE_BREAK are relevant only for timer instances supporting a break input.

HAL_TIM_ReadCapturedValue

Function name

uint32_t HAL_TIM_ReadCapturedValue (const TIM_HandleTypeDef * htim, uint32_t Channel)

Function description

Read the captured value from Capture Compare unit.

Parameters

- **htim:** TIM handle.
- **Channel:** TIM Channels to be enabled This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1 selected
 - TIM_CHANNEL_2: TIM Channel 2 selected
 - TIM_CHANNEL_3: TIM Channel 3 selected
 - TIM_CHANNEL_4: TIM Channel 4 selected

Return values

- **Captured:** value

HAL_TIM_PeriodElapsedCallback

Function name

void HAL_TIM_PeriodElapsedCallback (TIM_HandleTypeDef * htim)

Function description

Period elapsed callback in non-blocking mode.

Parameters

- **htim**: TIM handle

Return values

- **None**:

HAL_TIM_PeriodElapsedHalfCpltCallback

Function name

void HAL_TIM_PeriodElapsedHalfCpltCallback (TIM_HandleTypeDef * htim)

Function description

Period elapsed half complete callback in non-blocking mode.

Parameters

- **htim**: TIM handle

Return values

- **None**:

HAL_TIM_OC_DelayElapsedCallback

Function name

void HAL_TIM_OC_DelayElapsedCallback (TIM_HandleTypeDef * htim)

Function description

Output Compare callback in non-blocking mode.

Parameters

- **htim**: TIM OC handle

Return values

- **None**:

HAL_TIM_IC_CaptureCallback

Function name

void HAL_TIM_IC_CaptureCallback (TIM_HandleTypeDef * htim)

Function description

Input Capture callback in non-blocking mode.

Parameters

- **htim**: TIM IC handle

Return values

- **None**:

HAL_TIM_IC_CaptureHalfCpltCallback

Function name

void HAL_TIM_IC_CaptureHalfCpltCallback (TIM_HandleTypeDef * htim)

Function description

Input Capture half complete callback in non-blocking mode.

Parameters

- **htim**: TIM IC handle

Return values

- **None:**

HAL_TIM_PWM_PulseFinishedCallback

Function name

void HAL_TIM_PWM_PulseFinishedCallback (TIM_HandleTypeDef * htim)

Function description

PWM Pulse finished callback in non-blocking mode.

Parameters

- **htim:** TIM handle

Return values

- **None:**

HAL_TIM_PWM_PulseFinishedHalfCpltCallback

Function name

void HAL_TIM_PWM_PulseFinishedHalfCpltCallback (TIM_HandleTypeDef * htim)

Function description

PWM Pulse finished half complete callback in non-blocking mode.

Parameters

- **htim:** TIM handle

Return values

- **None:**

HAL_TIM_TriggerCallback

Function name

void HAL_TIM_TriggerCallback (TIM_HandleTypeDef * htim)

Function description

Hall Trigger detection callback in non-blocking mode.

Parameters

- **htim:** TIM handle

Return values

- **None:**

HAL_TIM_TriggerHalfCpltCallback

Function name

void HAL_TIM_TriggerHalfCpltCallback (TIM_HandleTypeDef * htim)

Function description

Hall Trigger detection half complete callback in non-blocking mode.

Parameters

- **htim:** TIM handle

Return values

- **None:**

HAL_TIM_ErrorCallback

Function name

void HAL_TIM_ErrorCallback (TIM_HandleTypeDef * htim)

Function description

Timer error callback in non-blocking mode.

Parameters

- **htim:** TIM handle

Return values

- **None:**

HAL_TIM_RegisterCallback

Function name

**HAL_StatusTypeDef HAL_TIM_RegisterCallback (TIM_HandleTypeDef * htim,
HAL_TIM_CallbackIDTypeDef CallbackID, pTIM_CallbackTypeDef pCallback)**

Function description

Register a User TIM callback to be used instead of the weak predefined callback.

Parameters

- **htim:** tim handle
- **CallbackID:** ID of the callback to be registered This parameter can be one of the following values:
 - HAL_TIM_BASE_MSPINIT_CB_ID Base MspInit Callback ID
 - HAL_TIM_BASE_MSPDEINIT_CB_ID Base MspDeInit Callback ID
 - HAL_TIM_IC_MSPINIT_CB_ID IC MspInit Callback ID
 - HAL_TIM_IC_MSPDEINIT_CB_ID IC MspDeInit Callback ID
 - HAL_TIM_OC_MSPINIT_CB_ID OC MspInit Callback ID
 - HAL_TIM_OC_MSPDEINIT_CB_ID OC MspDeInit Callback ID
 - HAL_TIM_PWM_MSPINIT_CB_ID PWM MspInit Callback ID
 - HAL_TIM_PWM_MSPDEINIT_CB_ID PWM MspDeInit Callback ID
 - HAL_TIM_ONE_PULSE_MSPINIT_CB_ID One Pulse MspInit Callback ID
 - HAL_TIM_ONE_PULSE_MSPDEINIT_CB_ID One Pulse MspDeInit Callback ID
 - HAL_TIM_ENCODER_MSPINIT_CB_ID Encoder MspInit Callback ID
 - HAL_TIM_ENCODER_MSPDEINIT_CB_ID Encoder MspDeInit Callback ID
 - HAL_TIM_HALL_SENSOR_MSPINIT_CB_ID Hall Sensor MspInit Callback ID
 - HAL_TIM_HALL_SENSOR_MSPDEINIT_CB_ID Hall Sensor MspDeInit Callback ID
 - HAL_TIM_PERIOD_ELAPSED_CB_ID Period Elapsed Callback ID
 - HAL_TIM_PERIOD_ELAPSED_HALF_CB_ID Period Elapsed half complete Callback ID
 - HAL_TIM_TRIGGER_CB_ID Trigger Callback ID
 - HAL_TIM_TRIGGER_HALF_CB_ID Trigger half complete Callback ID
 - HAL_TIM_IC_CAPTURE_CB_ID Input Capture Callback ID
 - HAL_TIM_IC_CAPTURE_HALF_CB_ID Input Capture half complete Callback ID
 - HAL_TIM_OC_DELAY_ELAPSED_CB_ID Output Compare Delay Elapsed Callback ID
 - HAL_TIM_PWM_PULSE_FINISHED_CB_ID PWM Pulse Finished Callback ID
 - HAL_TIM_PWM_PULSE_FINISHED_HALF_CB_ID PWM Pulse Finished half complete Callback ID
 - HAL_TIM_ERROR_CB_ID Error Callback ID
 - HAL_TIM_COMMUTATION_CB_ID Commutation Callback ID
 - HAL_TIM_COMMUTATION_HALF_CB_ID Commutation half complete Callback ID
 - HAL_TIM_BREAK_CB_ID Break Callback ID
- **pCallback:** pointer to the callback function

Return values

- **status:**

HAL_TIM_UnRegisterCallback

Function name

**HAL_StatusTypeDef HAL_TIM_UnRegisterCallback (TIM_HandleTypeDef * htim,
HAL_TIM_CallbackIDTypeDef CallbackID)**

Function description

Unregister a TIM callback TIM callback is redirected to the weak predefined callback.

Parameters

- **htim:** tim handle
- **CallbackID:** ID of the callback to be unregistered This parameter can be one of the following values:
 - HAL_TIM_BASE_MSPINIT_CB_ID Base MspInit Callback ID
 - HAL_TIM_BASE_MSPDEINIT_CB_ID Base MspDeInit Callback ID
 - HAL_TIM_IC_MSPINIT_CB_ID IC MspInit Callback ID
 - HAL_TIM_IC_MSPDEINIT_CB_ID IC MspDeInit Callback ID
 - HAL_TIM_OC_MSPINIT_CB_ID OC MspInit Callback ID
 - HAL_TIM_OC_MSPDEINIT_CB_ID OC MspDeInit Callback ID
 - HAL_TIM_PWM_MSPINIT_CB_ID PWM MspInit Callback ID
 - HAL_TIM_PWM_MSPDEINIT_CB_ID PWM MspDeInit Callback ID
 - HAL_TIM_ONE_PULSE_MSPINIT_CB_ID One Pulse MspInit Callback ID
 - HAL_TIM_ONE_PULSE_MSPDEINIT_CB_ID One Pulse MspDeInit Callback ID
 - HAL_TIM_ENCODER_MSPINIT_CB_ID Encoder MspInit Callback ID
 - HAL_TIM_ENCODER_MSPDEINIT_CB_ID Encoder MspDeInit Callback ID
 - HAL_TIM_HALL_SENSOR_MSPINIT_CB_ID Hall Sensor MspInit Callback ID
 - HAL_TIM_HALL_SENSOR_MSPDEINIT_CB_ID Hall Sensor MspDeInit Callback ID
 - HAL_TIM_PERIOD_ELAPSED_CB_ID Period Elapsed Callback ID
 - HAL_TIM_PERIOD_ELAPSED_HALF_CB_ID Period Elapsed half complete Callback ID
 - HAL_TIM_TRIGGER_CB_ID Trigger Callback ID
 - HAL_TIM_TRIGGER_HALF_CB_ID Trigger half complete Callback ID
 - HAL_TIM_IC_CAPTURE_CB_ID Input Capture Callback ID
 - HAL_TIM_IC_CAPTURE_HALF_CB_ID Input Capture half complete Callback ID
 - HAL_TIM_OC_DELAY_ELAPSED_CB_ID Output Compare Delay Elapsed Callback ID
 - HAL_TIM_PWM_PULSE_FINISHED_CB_ID PWM Pulse Finished Callback ID
 - HAL_TIM_PWM_PULSE_FINISHED_HALF_CB_ID PWM Pulse Finished half complete Callback ID
 - HAL_TIM_ERROR_CB_ID Error Callback ID
 - HAL_TIM_COMMUTATION_CB_ID Commutation Callback ID
 - HAL_TIM_COMMUTATION_HALF_CB_ID Commutation half complete Callback ID
 - HAL_TIM_BREAK_CB_ID Break Callback ID

Return values

- **status:**

HAL_TIM_Base_GetState

Function name

HAL_TIM_StateTypeDef HAL_TIM_Base_GetState (const TIM_HandleTypeDef * htim)

Function description

Return the TIM Base handle state.

Parameters

- **htim:** TIM Base handle

Return values

- **HAL:** state

HAL_TIM_OC_GetState

Function name

HAL_TIM_StateTypeDef HAL_TIM_OC_GetState (const TIM_HandleTypeDef * htim)

Function description

Return the TIM OC handle state.

Parameters

- **htim**: TIM Output Compare handle

Return values

- **HAL**: state

HAL_TIM_PWM_GetState

Function name

HAL_TIM_StateTypeDef HAL_TIM_PWM_GetState (const TIM_HandleTypeDef * htim)

Function description

Return the TIM PWM handle state.

Parameters

- **htim**: TIM handle

Return values

- **HAL**: state

HAL_TIM_IC_GetState

Function name

HAL_TIM_StateTypeDef HAL_TIM_IC_GetState (const TIM_HandleTypeDef * htim)

Function description

Return the TIM Input Capture handle state.

Parameters

- **htim**: TIM IC handle

Return values

- **HAL**: state

HAL_TIM_OnePulse_GetState

Function name

HAL_TIM_StateTypeDef HAL_TIM_OnePulse_GetState (const TIM_HandleTypeDef * htim)

Function description

Return the TIM One Pulse Mode handle state.

Parameters

- **htim**: TIM OPM handle

Return values

- **HAL**: state

HAL_TIM_Encoder_GetState

Function name

HAL_TIM_StateTypeDef HAL_TIM_Encoder_GetState (const TIM_HandleTypeDef * htim)

Function description

Return the TIM Encoder Mode handle state.

Parameters

- **htim**: TIM Encoder Interface handle

Return values

- **HAL**: state

HAL_TIM_GetActiveChannel

Function name

HAL_TIM_ActiveChannel HAL_TIM_GetActiveChannel (const TIM_HandleTypeDef * htim)

Function description

Return the TIM Encoder Mode handle state.

Parameters

- **htim**: TIM handle

Return values

- **Active**: channel

HAL_TIM_GetChannelState

Function name

HAL_TIM_ChannelStateTypeDef HAL_TIM_GetChannelState (const TIM_HandleTypeDef * htim, uint32_t Channel)

Function description

Return actual state of the TIM channel.

Parameters

- **htim**: TIM handle
- **Channel**: TIM Channel This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1
 - TIM_CHANNEL_2: TIM Channel 2
 - TIM_CHANNEL_3: TIM Channel 3
 - TIM_CHANNEL_4: TIM Channel 4
 - TIM_CHANNEL_5: TIM Channel 5
 - TIM_CHANNEL_6: TIM Channel 6

Return values

- **TIM**: Channel state

HAL_TIM_DMABurstState

Function name

HAL_TIM_DMABurstStateTypeDef HAL_TIM_DMABurstState (const TIM_HandleTypeDef * htim)

Function description

Return actual state of a DMA burst operation.

Parameters

- **htim**: TIM handle

Return values

- **DMA**: burst state

TIM_Base_SetConfig

Function name

```
void TIM_Base_SetConfig (TIM_TypeDef * TIMx, const TIM_Base_InitTypeDef * Structure)
```

Function description

Time Base configuration.

Parameters

- **TIMx:** TIM peripheral
- **Structure:** TIM Base configuration structure

Return values

- **None:**

TIM_TI1_SetConfig

Function name

```
void TIM_TI1_SetConfig (TIM_TypeDef * TIMx, uint32_t TIM_ICPolarity, uint32_t TIM_ICSelection, uint32_t TIM_ICFilter)
```

Function description

Configure the TI1 as Input.

Parameters

- **TIMx:** to select the TIM peripheral.
- **TIM_ICPolarity:** The Input Polarity. This parameter can be one of the following values:
 - TIM_ICPOLARITY_RISING
 - TIM_ICPOLARITY_FALLING
 - TIM_ICPOLARITY_BOTHEDGE
- **TIM_ICSelection:** specifies the input to be used. This parameter can be one of the following values:
 - TIM_ICSELECTION_DIRECTTI: TIM Input 1 is selected to be connected to IC1.
 - TIM_ICSELECTION_INDIRECTTI: TIM Input 1 is selected to be connected to IC2.
 - TIM_ICSELECTION_TRC: TIM Input 1 is selected to be connected to TRC.
- **TIM_ICFilter:** Specifies the Input Capture Filter. This parameter must be a value between 0x00 and 0x0F.

Return values

- **None:**

Notes

- TIM_ICFilter and TIM_ICPolarity are not used in INDIRECT mode as TI2FP1 (on channel2 path) is used as the input signal. Therefore CCMR1 must be protected against un-initialized filter and polarity values.

TIM_OC2_SetConfig

Function name

```
void TIM_OC2_SetConfig (TIM_TypeDef * TIMx, const TIM_OC_InitTypeDef * OC_Config)
```

Function description

Timer Output Compare 2 configuration.

Parameters

- **TIMx:** to select the TIM peripheral
- **OC_Config:** The output configuration structure

Return values

- **None:**

TIM_ETR_SetConfig

Function name

void TIM_ETR_SetConfig (TIM_TypeDef * TIMx, uint32_t TIM_ExtTRGPrescaler, uint32_t TIM_ExtTRGPolarity, uint32_t ExtTRGFilter)

Function description

Configures the TIMx External Trigger (ETR).

Parameters

- **TIMx:** to select the TIM peripheral
- **TIM_ExtTRGPrescaler:** The external Trigger Prescaler. This parameter can be one of the following values:
 - TIM_ETRPRESCALER_DIV1: ETRP Prescaler OFF.
 - TIM_ETRPRESCALER_DIV2: ETRP frequency divided by 2.
 - TIM_ETRPRESCALER_DIV4: ETRP frequency divided by 4.
 - TIM_ETRPRESCALER_DIV8: ETRP frequency divided by 8.
- **TIM_ExtTRGPolarity:** The external Trigger Polarity. This parameter can be one of the following values:
 - TIM_ETRPOLARITY_INVERTED: active low or falling edge active.
 - TIM_ETRPOLARITY_NONINVERTED: active high or rising edge active.
- **ExtTRGFilter:** External Trigger Filter. This parameter must be a value between 0x00 and 0x0F

Return values

- **None:**

TIM_DMADelayPulseHalfCplt

Function name

void TIM_DMADelayPulseHalfCplt (DMA_HandleTypeDef * hdma)

Function description

TIM DMA Delay Pulse half complete callback.

Parameters

- **hdma:** pointer to DMA handle.

Return values

- **None:**

TIM_DMAError

Function name

void TIM_DMAError (DMA_HandleTypeDef * hdma)

Function description

TIM DMA error callback.

Parameters

- **hdma:** pointer to DMA handle.

Return values

- **None:**

TIM_DMACaptureCplt

Function name

void TIM_DMACaptureCplt (DMA_HandleTypeDef * hdma)

Function description

TIM DMA Capture complete callback.

Parameters

- **hdma:** pointer to DMA handle.

Return values

- **None:**

TIM_DMACaptureHalfCplt

Function name

void TIM_DMACaptureHalfCplt (DMA_HandleTypeDef * hdma)

Function description

TIM DMA Capture half complete callback.

Parameters

- **hdma:** pointer to DMA handle.

Return values

- **None:**

TIM_CCxChannelCmd

Function name

void TIM_CCxChannelCmd (TIM_TypeDef * TIMx, uint32_t Channel, uint32_t ChannelState)

Function description

Enables or disables the TIM Capture Compare Channel x.

Parameters

- **TIMx:** to select the TIM peripheral
- **Channel:** specifies the TIM Channel This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1
 - TIM_CHANNEL_2: TIM Channel 2
 - TIM_CHANNEL_3: TIM Channel 3
 - TIM_CHANNEL_4: TIM Channel 4
- **ChannelState:** specifies the TIM Channel CCxE bit new state. This parameter can be: TIM_CCx_ENABLE or TIM_CCx_DISABLE.

Return values

- **None:**

TIM_ResetCallback

Function name

void TIM_ResetCallback (TIM_HandleTypeDef * htim)

Function description

Reset interrupt callbacks to the legacy weak callbacks.

Parameters

- **htim**: pointer to a TIM_HandleTypeDef structure that contains the configuration information for TIM module.

Return values

- **None**:

68.3 TIM Firmware driver defines

The following section lists the various define and macros of the module.

68.3.1 TIM

TIM

TIM Automatic Output Enable

TIM_AUTOMATICOUTPUT_DISABLE

MOE can be set only by software

TIM_AUTOMATICOUTPUT_ENABLE

MOE can be set by software or automatically at the next update event (if none of the break inputs BRK and BRK2 is active)

TIM Auto-Reload Preload

TIM_AUTORELOAD_PRELOAD_DISABLE

TIMx_ARR register is not buffered

TIM_AUTORELOAD_PRELOAD_ENABLE

TIMx_ARR register is buffered

TIM Break Input Enable

TIM_BREAK_ENABLE

Break input BRK is enabled

TIM_BREAK_DISABLE

Break input BRK is disabled

TIM Break Input Polarity

TIM_BREAKPOLARITY_LOW

Break input BRK is active low

TIM_BREAKPOLARITY_HIGH

Break input BRK is active high

CCx DMA request selection

TIM_CCDMAREQUEST_CC

CCx DMA request sent when capture or compare match event occurs

TIM_CCDMAREQUEST_UPDATE

CCx DMA requests sent when update event occurs

TIM Channel

TIM_CHANNEL_1

Capture/compare channel 1 identifier

TIM_CHANNEL_2

Capture/compare channel 2 identifier

TIM_CHANNEL_3

Capture/compare channel 3 identifier

TIM_CHANNEL_4

Capture/compare channel 4 identifier

TIM_CHANNEL_ALL

Global Capture/compare channel identifier

TIM Clear Input Polarity**TIM_CLEARINPUTPOLARITY_INVERTED**

Polarity for ETRx pin

TIM_CLEARINPUTPOLARITY_NONINVERTED

Polarity for ETRx pin

TIM Clear Input Prescaler**TIM_CLEARINPUTPRESCALER_DIV1**

No prescaler is used

TIM_CLEARINPUTPRESCALER_DIV2

Prescaler for External ETR pin: Capture performed once every 2 events.

TIM_CLEARINPUTPRESCALER_DIV4

Prescaler for External ETR pin: Capture performed once every 4 events.

TIM_CLEARINPUTPRESCALER_DIV8

Prescaler for External ETR pin: Capture performed once every 8 events.

TIM Clear Input Source**TIM_CLEARINPUTSOURCE_NONE**

OCREF_CLR is disabled

TIM_CLEARINPUTSOURCE_ETR

OCREF_CLR is connected to ETRF input

TIM Clock Division**TIM_CLOCKDIVISION_DIV1**Clock division: $tDTS=tCK_INT$ **TIM_CLOCKDIVISION_DIV2**Clock division: $tDTS=2*tCK_INT$ **TIM_CLOCKDIVISION_DIV4**Clock division: $tDTS=4*tCK_INT$ ***TIM Clock Polarity*****TIM_CLOCKPOLARITY_INVERTED**

Polarity for ETRx clock sources

TIM_CLOCKPOLARITY_NONINVERTED

Polarity for ETRx clock sources

TIM_CLOCKPOLARITY_RISING

Polarity for Tlx clock sources

TIM_CLOCKPOLARITY_FALLING

Polarity for Tlx clock sources

TIM_CLOCKPOLARITY_BOTHEDGE

Polarity for Tlx clock sources

TIM Clock Prescaler**TIM_CLOCKPRESCALER_DIV1**

No prescaler is used

TIM_CLOCKPRESCALER_DIV2

Prescaler for External ETR Clock: Capture performed once every 2 events.

TIM_CLOCKPRESCALER_DIV4

Prescaler for External ETR Clock: Capture performed once every 4 events.

TIM_CLOCKPRESCALER_DIV8

Prescaler for External ETR Clock: Capture performed once every 8 events.

TIM Clock Source**TIM_CLOCKSOURCE_INTERNAL**

Internal clock source

TIM_CLOCKSOURCE_ETRMODE1

External clock source mode 1 (ETRF)

TIM_CLOCKSOURCE_ETRMODE2

External clock source mode 2

TIM_CLOCKSOURCE_TI1ED

External clock source mode 1 (TTI1FP1 + edge detect.)

TIM_CLOCKSOURCE_TI1

External clock source mode 1 (TTI1FP1)

TIM_CLOCKSOURCE_TI2

External clock source mode 1 (TTI2FP2)

TIM_CLOCKSOURCE_ITR0

External clock source mode 1 (ITR0)

TIM_CLOCKSOURCE_ITR1

External clock source mode 1 (ITR1)

TIM_CLOCKSOURCE_ITR2

External clock source mode 1 (ITR2)

TIM_CLOCKSOURCE_ITR3

External clock source mode 1 (ITR3)

TIM Commutation Source**TIM_COMMUTATION_TRGI**

When Capture/compare control bits are preloaded, they are updated by setting the COMG bit or when an rising edge occurs on trigger input

TIM_COMMUTATION_SOFTWARE

When Capture/compare control bits are preloaded, they are updated by setting the COMG bit

TIM Counter Mode**TIM_COUNTERMODE_UP**

Counter used as up-counter

TIM_COUNTERMODE_DOWN

Counter used as down-counter

TIM_COUNTERMODE_CENTERALIGNED1

Center-aligned mode 1

TIM_COUNTERMODE_CENTERALIGNED2

Center-aligned mode 2

TIM_COUNTERMODE_CENTERALIGNED3

Center-aligned mode 3

TIM DMA Base Address**TIM_DMABASE_CR1****TIM_DMABASE_CR2****TIM_DMABASE_SMCR****TIM_DMABASE_DIER****TIM_DMABASE_SR****TIM_DMABASE_EGR****TIM_DMABASE_CCMR1****TIM_DMABASE_CCMR2****TIM_DMABASE_CCER****TIM_DMABASE_CNT****TIM_DMABASE_PSC****TIM_DMABASE_ARR****TIM_DMABASE_RCR****TIM_DMABASE_CCR1****TIM_DMABASE_CCR2**

TIM_DMABASE_CCR3

TIM_DMABASE_CCR4

TIM_DMABASE_BDTR

TIM_DMABASE_DCR

TIM_DMABASE_DMAR

TIM DMA Burst Length

TIM_DMABURSTLENGTH_1TRANSFER

The transfer is done to 1 register starting from TIMx_CR1 + TIMx_DCR.DBA

TIM_DMABURSTLENGTH_2TRANSFERS

The transfer is done to 2 registers starting from TIMx_CR1 + TIMx_DCR.DBA

TIM_DMABURSTLENGTH_3TRANSFERS

The transfer is done to 3 registers starting from TIMx_CR1 + TIMx_DCR.DBA

TIM_DMABURSTLENGTH_4TRANSFERS

The transfer is done to 4 registers starting from TIMx_CR1 + TIMx_DCR.DBA

TIM_DMABURSTLENGTH_5TRANSFERS

The transfer is done to 5 registers starting from TIMx_CR1 + TIMx_DCR.DBA

TIM_DMABURSTLENGTH_6TRANSFERS

The transfer is done to 6 registers starting from TIMx_CR1 + TIMx_DCR.DBA

TIM_DMABURSTLENGTH_7TRANSFERS

The transfer is done to 7 registers starting from TIMx_CR1 + TIMx_DCR.DBA

TIM_DMABURSTLENGTH_8TRANSFERS

The transfer is done to 8 registers starting from TIMx_CR1 + TIMx_DCR.DBA

TIM_DMABURSTLENGTH_9TRANSFERS

The transfer is done to 9 registers starting from TIMx_CR1 + TIMx_DCR.DBA

TIM_DMABURSTLENGTH_10TRANSFERS

The transfer is done to 10 registers starting from TIMx_CR1 + TIMx_DCR.DBA

TIM_DMABURSTLENGTH_11TRANSFERS

The transfer is done to 11 registers starting from TIMx_CR1 + TIMx_DCR.DBA

TIM_DMABURSTLENGTH_12TRANSFERS

The transfer is done to 12 registers starting from TIMx_CR1 + TIMx_DCR.DBA

TIM_DMABURSTLENGTH_13TRANSFERS

The transfer is done to 13 registers starting from TIMx_CR1 + TIMx_DCR.DBA

TIM_DMABURSTLENGTH_14TRANSFERS

The transfer is done to 14 registers starting from TIMx_CR1 + TIMx_DCR.DBA

TIM_DMABURSTLENGTH_15TRANSFERS

The transfer is done to 15 registers starting from TIMx_CR1 + TIMx_DCR.DBA

TIM_DMABURSTLENGTH_16TRANSFERS

The transfer is done to 16 registers starting from TIMx_CR1 + TIMx_DCR.DBA

TIM_DMABURSTLENGTH_17TRANSFERS

The transfer is done to 17 registers starting from TIMx_CR1 + TIMx_DCR.DBA

TIM_DMABURSTLENGTH_18TRANSFERS

The transfer is done to 18 registers starting from TIMx_CR1 + TIMx_DCR.DBA

TIM DMA Sources**TIM_DMA_UPDATE**

DMA request is triggered by the update event

TIM_DMA_CC1

DMA request is triggered by the capture/compare match 1 event

TIM_DMA_CC2

DMA request is triggered by the capture/compare match 2 event event

TIM_DMA_CC3

DMA request is triggered by the capture/compare match 3 event event

TIM_DMA_CC4

DMA request is triggered by the capture/compare match 4 event event

TIM_DMA_COM

DMA request is triggered by the commutation event

TIM_DMA_TRIGGER

DMA request is triggered by the trigger event

TIM Encoder Input Polarity**TIM_ENCODERINPUTPOLARITY_RISING**

Encoder input with rising edge polarity

TIM_ENCODERINPUTPOLARITY_FALLING

Encoder input with falling edge polarity

TIM Encoder Mode**TIM_ENCODERMODE_TI1**

Quadrature encoder mode 1, x2 mode, counts up/down on TI1FP1 edge depending on TI2FP2 level

TIM_ENCODERMODE_TI2

Quadrature encoder mode 2, x2 mode, counts up/down on TI2FP2 edge depending on TI1FP1 level.

TIM_ENCODERMODE_TI12

Quadrature encoder mode 3, x4 mode, counts up/down on both TI1FP1 and TI2FP2 edges depending on the level of the other input.

TIM ETR Polarity**TIM_ETRPOLARITY_INVERTED**

Polarity for ETR source

TIM_ETRPOLARITY_NONINVERTED

Polarity for ETR source

TIM ETR Prescaler

TIM_ETRPRESCALER_DIV1

No prescaler is used

TIM_ETRPRESCALER_DIV2

ETR input source is divided by 2

TIM_ETRPRESCALER_DIV4

ETR input source is divided by 4

TIM_ETRPRESCALER_DIV8

ETR input source is divided by 8

TIM Event Source

TIM_EVENTSOURCE_UPDATE

Reinitialize the counter and generates an update of the registers

TIM_EVENTSOURCE_CC1

A capture/compare event is generated on channel 1

TIM_EVENTSOURCE_CC2

A capture/compare event is generated on channel 2

TIM_EVENTSOURCE_CC3

A capture/compare event is generated on channel 3

TIM_EVENTSOURCE_CC4

A capture/compare event is generated on channel 4

TIM_EVENTSOURCE_COM

A commutation event is generated

TIM_EVENTSOURCE_TRIGGER

A trigger event is generated

TIM_EVENTSOURCE_BREAK

A break event is generated

TIM Exported Macros

__HAL_TIM_RESET_HANDLE_STATE

Description:

- Reset TIM handle state.

Parameters:

- `__HANDLE__`: TIM handle.

Return value:

- None

__HAL_TIM_ENABLE

Description:

- Enable the TIM peripheral.

Parameters:

- `__HANDLE__`: TIM handle

Return value:

- None

__HAL_TIM_MOE_ENABLE

Description:

- Enable the TIM main Output.

Parameters:

- `__HANDLE__`: TIM handle

Return value:

- None

__HAL_TIM_DISABLE

Description:

- Disable the TIM peripheral.

Parameters:

- `__HANDLE__`: TIM handle

Return value:

- None

__HAL_TIM_MOE_DISABLE

Description:

- Disable the TIM main Output.

Parameters:

- `__HANDLE__`: TIM handle

Return value:

- None

Notes:

- The Main Output Enable of a timer instance is disabled only if all the CCx and CCxN channels have been disabled

__HAL_TIM_MOE_DISABLE_UNCONDITIONALLY

Description:

- Disable the TIM main Output.

Parameters:

- `__HANDLE__`: TIM handle

Return value:

- None

Notes:

- The Main Output Enable of a timer instance is disabled unconditionally

__HAL_TIM_ENABLE_IT

Description:

- Enable the specified TIM interrupt.

Parameters:

- `__HANDLE__`: specifies the TIM Handle.
- `__INTERRUPT__`: specifies the TIM interrupt source to enable. This parameter can be one of the following values:
 - `TIM_IT_UPDATE`: Update interrupt
 - `TIM_IT_CC1`: Capture/Compare 1 interrupt
 - `TIM_IT_CC2`: Capture/Compare 2 interrupt
 - `TIM_IT_CC3`: Capture/Compare 3 interrupt
 - `TIM_IT_CC4`: Capture/Compare 4 interrupt
 - `TIM_IT_COM`: Commutation interrupt
 - `TIM_IT_TRIGGER`: Trigger interrupt
 - `TIM_IT_BREAK`: Break interrupt

Return value:

- None

__HAL_TIM_DISABLE_IT

Description:

- Disable the specified TIM interrupt.

Parameters:

- `__HANDLE__`: specifies the TIM Handle.
- `__INTERRUPT__`: specifies the TIM interrupt source to disable. This parameter can be one of the following values:
 - `TIM_IT_UPDATE`: Update interrupt
 - `TIM_IT_CC1`: Capture/Compare 1 interrupt
 - `TIM_IT_CC2`: Capture/Compare 2 interrupt
 - `TIM_IT_CC3`: Capture/Compare 3 interrupt
 - `TIM_IT_CC4`: Capture/Compare 4 interrupt
 - `TIM_IT_COM`: Commutation interrupt
 - `TIM_IT_TRIGGER`: Trigger interrupt
 - `TIM_IT_BREAK`: Break interrupt

Return value:

- None

__HAL_TIM_ENABLE_DMA

Description:

- Enable the specified DMA request.

Parameters:

- __HANDLE__: specifies the TIM Handle.
- __DMA__: specifies the TIM DMA request to enable. This parameter can be one of the following values:
 - TIM_DMA_UPDATE: Update DMA request
 - TIM_DMA_CC1: Capture/Compare 1 DMA request
 - TIM_DMA_CC2: Capture/Compare 2 DMA request
 - TIM_DMA_CC3: Capture/Compare 3 DMA request
 - TIM_DMA_CC4: Capture/Compare 4 DMA request
 - TIM_DMA_COM: Commutation DMA request
 - TIM_DMA_TRIGGER: Trigger DMA request

Return value:

- None

__HAL_TIM_DISABLE_DMA

Description:

- Disable the specified DMA request.

Parameters:

- __HANDLE__: specifies the TIM Handle.
- __DMA__: specifies the TIM DMA request to disable. This parameter can be one of the following values:
 - TIM_DMA_UPDATE: Update DMA request
 - TIM_DMA_CC1: Capture/Compare 1 DMA request
 - TIM_DMA_CC2: Capture/Compare 2 DMA request
 - TIM_DMA_CC3: Capture/Compare 3 DMA request
 - TIM_DMA_CC4: Capture/Compare 4 DMA request
 - TIM_DMA_COM: Commutation DMA request
 - TIM_DMA_TRIGGER: Trigger DMA request

Return value:

- None

__HAL_TIM_GET_FLAG

Description:

- Check whether the specified TIM interrupt flag is set or not.

Parameters:

- `__HANDLE__`: specifies the TIM Handle.
- `__FLAG__`: specifies the TIM interrupt flag to check. This parameter can be one of the following values:
 - `TIM_FLAG_UPDATE`: Update interrupt flag
 - `TIM_FLAG_CC1`: Capture/Compare 1 interrupt flag
 - `TIM_FLAG_CC2`: Capture/Compare 2 interrupt flag
 - `TIM_FLAG_CC3`: Capture/Compare 3 interrupt flag
 - `TIM_FLAG_CC4`: Capture/Compare 4 interrupt flag
 - `TIM_FLAG_COM`: Commutation interrupt flag
 - `TIM_FLAG_TRIGGER`: Trigger interrupt flag
 - `TIM_FLAG_BREAK`: Break interrupt flag
 - `TIM_FLAG_CC1OF`: Capture/Compare 1 overcapture flag
 - `TIM_FLAG_CC2OF`: Capture/Compare 2 overcapture flag
 - `TIM_FLAG_CC3OF`: Capture/Compare 3 overcapture flag
 - `TIM_FLAG_CC4OF`: Capture/Compare 4 overcapture flag

Return value:

- The: new state of `__FLAG__` (TRUE or FALSE).

__HAL_TIM_CLEAR_FLAG

Description:

- Clear the specified TIM interrupt flag.

Parameters:

- `__HANDLE__`: specifies the TIM Handle.
- `__FLAG__`: specifies the TIM interrupt flag to clear. This parameter can be one of the following values:
 - `TIM_FLAG_UPDATE`: Update interrupt flag
 - `TIM_FLAG_CC1`: Capture/Compare 1 interrupt flag
 - `TIM_FLAG_CC2`: Capture/Compare 2 interrupt flag
 - `TIM_FLAG_CC3`: Capture/Compare 3 interrupt flag
 - `TIM_FLAG_CC4`: Capture/Compare 4 interrupt flag
 - `TIM_FLAG_COM`: Commutation interrupt flag
 - `TIM_FLAG_TRIGGER`: Trigger interrupt flag
 - `TIM_FLAG_BREAK`: Break interrupt flag
 - `TIM_FLAG_CC1OF`: Capture/Compare 1 overcapture flag
 - `TIM_FLAG_CC2OF`: Capture/Compare 2 overcapture flag
 - `TIM_FLAG_CC3OF`: Capture/Compare 3 overcapture flag
 - `TIM_FLAG_CC4OF`: Capture/Compare 4 overcapture flag

Return value:

- The: new state of `__FLAG__` (TRUE or FALSE).

__HAL_TIM_GET_IT_SOURCE

Description:

- Check whether the specified TIM interrupt source is enabled or not.

Parameters:

- `__HANDLE__`: TIM handle
- `__INTERRUPT__`: specifies the TIM interrupt source to check. This parameter can be one of the following values:
 - `TIM_IT_UPDATE`: Update interrupt
 - `TIM_IT_CC1`: Capture/Compare 1 interrupt
 - `TIM_IT_CC2`: Capture/Compare 2 interrupt
 - `TIM_IT_CC3`: Capture/Compare 3 interrupt
 - `TIM_IT_CC4`: Capture/Compare 4 interrupt
 - `TIM_IT_COM`: Commutation interrupt
 - `TIM_IT_TRIGGER`: Trigger interrupt
 - `TIM_IT_BREAK`: Break interrupt

Return value:

- The: state of `TIM_IT` (SET or RESET).

__HAL_TIM_CLEAR_IT

Description:

- Clear the TIM interrupt pending bits.

Parameters:

- `__HANDLE__`: TIM handle
- `__INTERRUPT__`: specifies the interrupt pending bit to clear. This parameter can be one of the following values:
 - `TIM_IT_UPDATE`: Update interrupt
 - `TIM_IT_CC1`: Capture/Compare 1 interrupt
 - `TIM_IT_CC2`: Capture/Compare 2 interrupt
 - `TIM_IT_CC3`: Capture/Compare 3 interrupt
 - `TIM_IT_CC4`: Capture/Compare 4 interrupt
 - `TIM_IT_COM`: Commutation interrupt
 - `TIM_IT_TRIGGER`: Trigger interrupt
 - `TIM_IT_BREAK`: Break interrupt

Return value:

- None

__HAL_TIM_IS_TIM_COUNTING_DOWN

Description:

- Indicates whether or not the TIM Counter is used as downcounter.

Parameters:

- `__HANDLE__`: TIM handle.

Return value:

- False: (Counter used as upcounter) or True (Counter used as downcounter)

Notes:

- This macro is particularly useful to get the counting mode when the timer operates in Center-aligned mode or Encoder mode.

__HAL_TIM_SET_PRESCALER

Description:

- Set the TIM Prescaler on runtime.

Parameters:

- `__HANDLE__`: TIM handle.
- `__PRESC__`: specifies the Prescaler new value.

Return value:

- None

__HAL_TIM_SET_COUNTER

Description:

- Set the TIM Counter Register value on runtime.

Parameters:

- `__HANDLE__`: TIM handle.
- `__COUNTER__`: specifies the Counter register new value.

Return value:

- None

__HAL_TIM_GET_COUNTER

Description:

- Get the TIM Counter Register value on runtime.

Parameters:

- `__HANDLE__`: TIM handle.

Return value:

- 16-bit: or 32-bit value of the timer counter register (TIMx_CNT)

__HAL_TIM_SET_AUTORELOAD

Description:

- Set the TIM Autoreload Register value on runtime without calling another time any Init function.

Parameters:

- `__HANDLE__`: TIM handle.
- `__AUTORELOAD__`: specifies the Counter register new value.

Return value:

- None

__HAL_TIM_GET_AUTORELOAD

Description:

- Get the TIM Autoreload Register value on runtime.

Parameters:

- `__HANDLE__`: TIM handle.

Return value:

- 16-bit: or 32-bit value of the timer auto-reload register(TIMx_ARR)

__HAL_TIM_SET_CLOCKDIVISION

Description:

- Set the TIM Clock Division value on runtime without calling another time any Init function.

Parameters:

- __HANDLE__: TIM handle.
- __CKD__: specifies the clock division value. This parameter can be one of the following value:
 - TIM_CLOCKDIVISION_DIV1: $tDTS=tCK_INT$
 - TIM_CLOCKDIVISION_DIV2: $tDTS=2*tCK_INT$
 - TIM_CLOCKDIVISION_DIV4: $tDTS=4*tCK_INT$

Return value:

- None

__HAL_TIM_GET_CLOCKDIVISION

Description:

- Get the TIM Clock Division value on runtime.

Parameters:

- __HANDLE__: TIM handle.

Return value:

- The: clock division can be one of the following values:
 - TIM_CLOCKDIVISION_DIV1: $tDTS=tCK_INT$
 - TIM_CLOCKDIVISION_DIV2: $tDTS=2*tCK_INT$
 - TIM_CLOCKDIVISION_DIV4: $tDTS=4*tCK_INT$

__HAL_TIM_SET_ICPSCALER

Description:

- Set the TIM Input Capture prescaler on runtime without calling another time

Parameters:

- __HANDLE__: TIM handle.
- __CHANNEL__: TIM Channels to be configured. This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1 selected
 - TIM_CHANNEL_2: TIM Channel 2 selected
 - TIM_CHANNEL_3: TIM Channel 3 selected
 - TIM_CHANNEL_4: TIM Channel 4 selected
- __ICPSC__: specifies the Input Capture4 prescaler new value. This parameter can be one of the following values:
 - TIM_ICPSC_DIV1: no prescaler
 - TIM_ICPSC_DIV2: capture is done once every 2 events
 - TIM_ICPSC_DIV4: capture is done once every 4 events
 - TIM_ICPSC_DIV8: capture is done once every 8 events

Return value:

- None

__HAL_TIM_GET_ICPRESCALER

Description:

- Get the TIM Input Capture prescaler on runtime.

Parameters:

- `__HANDLE__`: TIM handle.
- `__CHANNEL__`: TIM Channels to be configured. This parameter can be one of the following values:
 - `TIM_CHANNEL_1`: get input capture 1 prescaler value
 - `TIM_CHANNEL_2`: get input capture 2 prescaler value
 - `TIM_CHANNEL_3`: get input capture 3 prescaler value
 - `TIM_CHANNEL_4`: get input capture 4 prescaler value

Return value:

- The: input capture prescaler can be one of the following values:
 - `TIM_ICPSC_DIV1`: no prescaler
 - `TIM_ICPSC_DIV2`: capture is done once every 2 events
 - `TIM_ICPSC_DIV4`: capture is done once every 4 events
 - `TIM_ICPSC_DIV8`: capture is done once every 8 events

__HAL_TIM_SET_COMPARE

Description:

- Set the TIM Capture Compare Register value on runtime without calling another time ConfigChannel function.

Parameters:

- `__HANDLE__`: TIM handle.
- `__CHANNEL__`: TIM Channels to be configured. This parameter can be one of the following values:
 - `TIM_CHANNEL_1`: TIM Channel 1 selected
 - `TIM_CHANNEL_2`: TIM Channel 2 selected
 - `TIM_CHANNEL_3`: TIM Channel 3 selected
 - `TIM_CHANNEL_4`: TIM Channel 4 selected
- `__COMPARE__`: specifies the Capture Compare register new value.

Return value:

- None

__HAL_TIM_GET_COMPARE

Description:

- Get the TIM Capture Compare Register value on runtime.

Parameters:

- `__HANDLE__`: TIM handle.
- `__CHANNEL__`: TIM Channel associated with the capture compare register This parameter can be one of the following values:
 - `TIM_CHANNEL_1`: get capture/compare 1 register value
 - `TIM_CHANNEL_2`: get capture/compare 2 register value
 - `TIM_CHANNEL_3`: get capture/compare 3 register value
 - `TIM_CHANNEL_4`: get capture/compare 4 register value

Return value:

- 16-bit: or 32-bit value of the capture/compare register (`TIMx_CCRy`)

__HAL_TIM_ENABLE_OCxPRELOAD

Description:

- Set the TIM Output compare preload.

Parameters:

- __HANDLE__: TIM handle.
- __CHANNEL__: TIM Channels to be configured. This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1 selected
 - TIM_CHANNEL_2: TIM Channel 2 selected
 - TIM_CHANNEL_3: TIM Channel 3 selected
 - TIM_CHANNEL_4: TIM Channel 4 selected

Return value:

- None

__HAL_TIM_DISABLE_OCxPRELOAD

Description:

- Reset the TIM Output compare preload.

Parameters:

- __HANDLE__: TIM handle.
- __CHANNEL__: TIM Channels to be configured. This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1 selected
 - TIM_CHANNEL_2: TIM Channel 2 selected
 - TIM_CHANNEL_3: TIM Channel 3 selected
 - TIM_CHANNEL_4: TIM Channel 4 selected

Return value:

- None

__HAL_TIM_ENABLE_OCxFAST

Description:

- Enable fast mode for a given channel.

Parameters:

- __HANDLE__: TIM handle.
- __CHANNEL__: TIM Channels to be configured. This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1 selected
 - TIM_CHANNEL_2: TIM Channel 2 selected
 - TIM_CHANNEL_3: TIM Channel 3 selected
 - TIM_CHANNEL_4: TIM Channel 4 selected

Return value:

- None

Notes:

- When fast mode is enabled an active edge on the trigger input acts like a compare match on CCx output. Delay to sample the trigger input and to activate CCx output is reduced to 3 clock cycles. Fast mode acts only if the channel is configured in PWM1 or PWM2 mode.

__HAL_TIM_DISABLE_OCxFAST

Description:

- Disable fast mode for a given channel.

Parameters:

- __HANDLE__: TIM handle.
- __CHANNEL__: TIM Channels to be configured. This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1 selected
 - TIM_CHANNEL_2: TIM Channel 2 selected
 - TIM_CHANNEL_3: TIM Channel 3 selected
 - TIM_CHANNEL_4: TIM Channel 4 selected

Return value:

- None

Notes:

- When fast mode is disabled CCx output behaves normally depending on counter and CCRx values even when the trigger is ON. The minimum delay to activate CCx output when an active edge occurs on the trigger input is 5 clock cycles.

__HAL_TIM_URS_ENABLE

Description:

- Set the Update Request Source (URS) bit of the TIMx_CR1 register.

Parameters:

- __HANDLE__: TIM handle.

Return value:

- None

Notes:

- When the URS bit of the TIMx_CR1 register is set, only counter overflow/underflow generates an update interrupt or DMA request (if enabled)

__HAL_TIM_URS_DISABLE

Description:

- Reset the Update Request Source (URS) bit of the TIMx_CR1 register.

Parameters:

- __HANDLE__: TIM handle.

Return value:

- None

Notes:

- When the URS bit of the TIMx_CR1 register is reset, any of the following events generate an update interrupt or DMA request (if enabled): _ Counter overflow underflow _ Setting the UG bit _ Update generation through the slave mode controller

__HAL_TIM_SET_CAPTUREPOLARITY

Description:

- Set the TIM Capture x input polarity on runtime.

Parameters:

- `__HANDLE__`: TIM handle.
- `__CHANNEL__`: TIM Channels to be configured. This parameter can be one of the following values:
 - `TIM_CHANNEL_1`: TIM Channel 1 selected
 - `TIM_CHANNEL_2`: TIM Channel 2 selected
 - `TIM_CHANNEL_3`: TIM Channel 3 selected
 - `TIM_CHANNEL_4`: TIM Channel 4 selected
- `__POLARITY__`: Polarity for TIx source
 - `TIM_INPUTCHANNELPOLARITY_RISING`: Rising Edge
 - `TIM_INPUTCHANNELPOLARITY_FALLING`: Falling Edge
 - `TIM_INPUTCHANNELPOLARITY_BOTHEDGE`: Rising and Falling Edge

Return value:

- None

__HAL_TIM_SELECT_CCDMAREQUEST

Description:

- Select the Capture/compare DMA request source.

Parameters:

- `__HANDLE__`: specifies the TIM Handle.
- `__CCDMA__`: specifies Capture/compare DMA request source This parameter can be one of the following values:
 - `TIM_CCDMAREQUEST_CC`: CCx DMA request generated on Capture/Compare event
 - `TIM_CCDMAREQUEST_UPDATE`: CCx DMA request generated on Update event

Return value:

- None

TIM Flag Definition

TIM_FLAG_UPDATE

Update interrupt flag

TIM_FLAG_CC1

Capture/Compare 1 interrupt flag

TIM_FLAG_CC2

Capture/Compare 2 interrupt flag

TIM_FLAG_CC3

Capture/Compare 3 interrupt flag

TIM_FLAG_CC4

Capture/Compare 4 interrupt flag

TIM_FLAG_COM

Commutation interrupt flag

TIM_FLAG_TRIGGER

Trigger interrupt flag

TIM_FLAG_BREAK

Break interrupt flag

TIM_FLAG_CC1OF

Capture 1 overcapture flag

TIM_FLAG_CC2OF

Capture 2 overcapture flag

TIM_FLAG_CC3OF

Capture 3 overcapture flag

TIM_FLAG_CC4OF

Capture 4 overcapture flag

TIM Input Capture Polarity**TIM_ICPOLARITY_RISING**

Capture triggered by rising edge on timer input

TIM_ICPOLARITY_FALLING

Capture triggered by falling edge on timer input

TIM_ICPOLARITY_BOTHEDGE

Capture triggered by both rising and falling edges on timer input

TIM Input Capture Prescaler**TIM_ICPSC_DIV1**

Capture performed each time an edge is detected on the capture input

TIM_ICPSC_DIV2

Capture performed once every 2 events

TIM_ICPSC_DIV4

Capture performed once every 4 events

TIM_ICPSC_DIV8

Capture performed once every 8 events

TIM Input Capture Selection**TIM_ICSELECTION_DIRECTTI**

TIM Input 1, 2, 3 or 4 is selected to be connected to IC1, IC2, IC3 or IC4, respectively

TIM_ICSELECTION_INDIRECTTI

TIM Input 1, 2, 3 or 4 is selected to be connected to IC2, IC1, IC4 or IC3, respectively

TIM_ICSELECTION_TRC

TIM Input 1, 2, 3 or 4 is selected to be connected to TRC

TIM Input Channel polarity**TIM_INPUTCHANNELPOLARITY_RISING**

Polarity for TIx source

TIM_INPUTCHANNELPOLARITY_FALLING

Polarity for TIx source

TIM_INPUTCHANNELPOLARITY_BOTHEDGE

Polarity for Tlx source

TIM interrupt Definition**TIM_IT_UPDATE**

Update interrupt

TIM_IT_CC1

Capture/Compare 1 interrupt

TIM_IT_CC2

Capture/Compare 2 interrupt

TIM_IT_CC3

Capture/Compare 3 interrupt

TIM_IT_CC4

Capture/Compare 4 interrupt

TIM_IT_COM

Commutation interrupt

TIM_IT_TRIGGER

Trigger interrupt

TIM_IT_BREAK

Break interrupt

TIM Lock level**TIM_LOCKLEVEL_OFF**

LOCK OFF

TIM_LOCKLEVEL_1

LOCK Level 1

TIM_LOCKLEVEL_2

LOCK Level 2

TIM_LOCKLEVEL_3

LOCK Level 3

TIM Master Mode Selection**TIM_TRGO_RESET**

TIMx_EGR.UG bit is used as trigger output (TRGO)

TIM_TRGO_ENABLE

TIMx_CR1.CEN bit is used as trigger output (TRGO)

TIM_TRGO_UPDATE

Update event is used as trigger output (TRGO)

TIM_TRGO_OC1

Capture or a compare match 1 is used as trigger output (TRGO)

TIM_TRGO_OC1REF

OC1REF signal is used as trigger output (TRGO)

TIM_TRGO_OC2REF

OC2REF signal is used as trigger output (TRGO)

TIM_TRGO_OC3REF

OC3REF signal is used as trigger output (TRGO)

TIM_TRGO_OC4REF

OC4REF signal is used as trigger output (TRGO)

TIM Master/Slave Mode**TIM_MASTERSLAVEMODE_ENABLE**

No action

TIM_MASTERSLAVEMODE_DISABLE

Master/slave mode is selected

TIM One Pulse Mode**TIM_OPMODE_SINGLE**

Counter stops counting at the next update event

TIM_OPMODE_REPETITIVE

Counter is not stopped at update event

TIM OSSI OffState Selection for Idle mode state**TIM_OSSI_ENABLE**

When inactive, OC/OCN outputs are enabled (still controlled by the timer)

TIM_OSSI_DISABLE

When inactive, OC/OCN outputs are disabled (not controlled any longer by the timer)

TIM OSSR OffState Selection for Run mode state**TIM_OSSR_ENABLE**

When inactive, OC/OCN outputs are enabled (still controlled by the timer)

TIM_OSSR_DISABLE

When inactive, OC/OCN outputs are disabled (not controlled any longer by the timer)

TIM Output Compare and PWM Modes**TIM_OCMODE_TIMING**

Frozen

TIM_OCMODE_ACTIVE

Set channel to active level on match

TIM_OCMODE_INACTIVE

Set channel to inactive level on match

TIM_OCMODE_TOGGLE

Toggle

TIM_OCMODE_PWM1

PWM mode 1

TIM_OCMODE_PWM2

PWM mode 2

TIM_OCMODE_FORCED_ACTIVE

Force active level

TIM_OCMODE_FORCED_INACTIVE

Force inactive level

TIM Output Compare Idle State**TIM_OCIDLESTATE_SET**

Output Idle state: OCx=1 when MOE=0

TIM_OCIDLESTATE_RESET

Output Idle state: OCx=0 when MOE=0

TIM Complementary Output Compare Idle State**TIM_OCNIDLESTATE_SET**

Complementary output Idle state: OCxN=1 when MOE=0

TIM_OCNIDLESTATE_RESET

Complementary output Idle state: OCxN=0 when MOE=0

TIM Complementary Output Compare Polarity**TIM_OCNPOLARITY_HIGH**

Capture/Compare complementary output polarity

TIM_OCNPOLARITY_LOW

Capture/Compare complementary output polarity

TIM Complementary Output Compare State**TIM_OUTPUTNSTATE_DISABLE**

OCxN is disabled

TIM_OUTPUTNSTATE_ENABLE

OCxN is enabled

TIM Output Compare Polarity**TIM_OCPOLARITY_HIGH**

Capture/Compare output polarity

TIM_OCPOLARITY_LOW

Capture/Compare output polarity

TIM Output Compare State**TIM_OUTPUTSTATE_DISABLE**

Capture/Compare 1 output disabled

TIM_OUTPUTSTATE_ENABLE

Capture/Compare 1 output enabled

TIM Output Fast State**TIM_OCFAST_DISABLE**

Output Compare fast disable

TIM_OCFAST_ENABLE

Output Compare fast enable

TIM Slave mode**TIM_SLAVEMODE_DISABLE**

Slave mode disabled

TIM_SLAVEMODE_RESET

Reset Mode

TIM_SLAVEMODE_GATED

Gated Mode

TIM_SLAVEMODE_TRIGGER

Trigger Mode

TIM_SLAVEMODE_EXTERNAL1

External Clock Mode 1

TIM TI1 Input Selection**TIM_TI1SELECTION_CH1**

The TIMx_CH1 pin is connected to TI1 input

TIM_TI1SELECTION_XORCOMBINATION

The TIMx_CH1, CH2 and CH3 pins are connected to the TI1 input (XOR combination)

TIM Trigger Polarity**TIM_TRIGGERPOLARITY_INVERTED**

Polarity for ETRx trigger sources

TIM_TRIGGERPOLARITY_NONINVERTED

Polarity for ETRx trigger sources

TIM_TRIGGERPOLARITY_RISING

Polarity for TIxFPx or TI1_ED trigger sources

TIM_TRIGGERPOLARITY_FALLING

Polarity for TIxFPx or TI1_ED trigger sources

TIM_TRIGGERPOLARITY_BOTHEDGE

Polarity for TIxFPx or TI1_ED trigger sources

TIM Trigger Prescaler**TIM_TRIGGERPRESCALER_DIV1**

No prescaler is used

TIM_TRIGGERPRESCALER_DIV2

Prescaler for External ETR Trigger: Capture performed once every 2 events.

TIM_TRIGGERPRESCALER_DIV4

Prescaler for External ETR Trigger: Capture performed once every 4 events.

TIM_TRIGGERPRESCALER_DIV8

Prescaler for External ETR Trigger: Capture performed once every 8 events.

TIM Trigger Selection**TIM_TS_ITR0**

Internal Trigger 0 (ITR0)

TIM_TS_ITR1

Internal Trigger 1 (ITR1)

TIM_TS_ITR2

Internal Trigger 2 (ITR2)

TIM_TS_ITR3

Internal Trigger 3 (ITR3)

TIM_TS_TI1F_ED

TI1 Edge Detector (TI1F_ED)

TIM_TS_TI1FP1

Filtered Timer Input 1 (TI1FP1)

TIM_TS_TI2FP2

Filtered Timer Input 2 (TI2FP2)

TIM_TS_ETRF

Filtered External Trigger input (ETRF)

TIM_TS_NONE

No trigger selected

69 HAL TIM Extension Driver

69.1 TIMEx Firmware driver registers structures

69.1.1 TIM_HallSensor_InitTypeDef

TIM_HallSensor_InitTypeDef is defined in the stm32f4xx_hal_tim_ex.h

Data Fields

- *uint32_t IC1Polarity*
- *uint32_t IC1Prescaler*
- *uint32_t IC1Filter*
- *uint32_t Commutation_Delay*

Field Documentation

- *uint32_t TIM_HallSensor_InitTypeDef::IC1Polarity*
Specifies the active edge of the input signal. This parameter can be a value of [TIM_Input_Capture_Polarity](#)
- *uint32_t TIM_HallSensor_InitTypeDef::IC1Prescaler*
Specifies the Input Capture Prescaler. This parameter can be a value of [TIM_Input_Capture_Prescaler](#)
- *uint32_t TIM_HallSensor_InitTypeDef::IC1Filter*
Specifies the input capture filter. This parameter can be a number between Min_Data = 0x0 and Max_Data = 0xF
- *uint32_t TIM_HallSensor_InitTypeDef::Commutation_Delay*
Specifies the pulse value to be loaded into the Capture Compare Register. This parameter can be a number between Min_Data = 0x0000 and Max_Data = 0xFFFF

69.2 TIMEx Firmware driver API description

The following section lists the various functions of the TIMEx library.

69.2.1 TIMER Extended features

The Timer Extended features include:

1. Complementary outputs with programmable dead-time for :
 - Output Compare
 - PWM generation (Edge and Center-aligned Mode)
 - One-pulse mode output
2. Synchronization circuit to control the timer with external signals and to interconnect several timers together.
3. Break input to put the timer output signals in reset state or in a known state.
4. Supports incremental (quadrature) encoder and hall-sensor circuitry for positioning purposes

69.2.2 How to use this driver

1. Initialize the TIM low level resources by implementing the following functions depending on the selected feature:
 - Hall Sensor output : `HAL_TIMEx_HallSensor_MspInit()`
2. Initialize the TIM low level resources :
 - a. Enable the TIM interface clock using `__HAL_RCC_TIMx_CLK_ENABLE()`;
 - b. TIM pins configuration
 - Enable the clock for the TIM GPIOs using the following function:
`__HAL_RCC_GPIOx_CLK_ENABLE()`;
 - Configure these TIM pins in Alternate function mode using `HAL_GPIO_Init()`;

3. The external Clock can be configured, if needed (the default clock is the internal clock from the APBx), using the following function: `HAL_TIM_ConfigClockSource`, the clock configuration should be done before any start function.
4. Configure the TIM in the desired functioning mode using one of the initialization function of this driver:
 - `HAL_TIMEx_HallSensor_Init()` and `HAL_TIMEx_ConfigCommutEvent()`: to use the Timer Hall Sensor Interface and the commutation event with the corresponding Interrupt and DMA request if needed (Note that One Timer is used to interface with the Hall sensor Interface and another Timer should be used to use the commutation event).
5. Activate the TIM peripheral using one of the start functions:
 - Complementary Output Compare : `HAL_TIMEx_OC_N_Start()`, `HAL_TIMEx_OC_N_Start_DMA()`, `HAL_TIMEx_OC_N_Start_IT()`
 - Complementary PWM generation : `HAL_TIMEx_PWMN_Start()`, `HAL_TIMEx_PWMN_Start_DMA()`, `HAL_TIMEx_PWMN_Start_IT()`
 - Complementary One-pulse mode output : `HAL_TIMEx_OnePulseN_Start()`, `HAL_TIMEx_OnePulseN_Start_IT()`
 - Hall Sensor output : `HAL_TIMEx_HallSensor_Start()`, `HAL_TIMEx_HallSensor_Start_DMA()`, `HAL_TIMEx_HallSensor_Start_IT()`.

69.2.3

Timer Hall Sensor functions

This section provides functions allowing to:

- Initialize and configure TIM HAL Sensor.
- De-initialize TIM HAL Sensor.
- Start the Hall Sensor Interface.
- Stop the Hall Sensor Interface.
- Start the Hall Sensor Interface and enable interrupts.
- Stop the Hall Sensor Interface and disable interrupts.
- Start the Hall Sensor Interface and enable DMA transfers.
- Stop the Hall Sensor Interface and disable DMA transfers.

This section contains the following APIs:

- `HAL_TIMEx_HallSensor_Init()`
- `HAL_TIMEx_HallSensor_DeInit()`
- `HAL_TIMEx_HallSensor_MspInit()`
- `HAL_TIMEx_HallSensor_MspDeInit()`
- `HAL_TIMEx_HallSensor_Start()`
- `HAL_TIMEx_HallSensor_Stop()`
- `HAL_TIMEx_HallSensor_Start_IT()`
- `HAL_TIMEx_HallSensor_Stop_IT()`
- `HAL_TIMEx_HallSensor_Start_DMA()`
- `HAL_TIMEx_HallSensor_Stop_DMA()`

69.2.4

Timer Complementary Output Compare functions

This section provides functions allowing to:

- Start the Complementary Output Compare/PWM.
- Stop the Complementary Output Compare/PWM.
- Start the Complementary Output Compare/PWM and enable interrupts.
- Stop the Complementary Output Compare/PWM and disable interrupts.
- Start the Complementary Output Compare/PWM and enable DMA transfers.
- Stop the Complementary Output Compare/PWM and disable DMA transfers.

This section contains the following APIs:

- `HAL_TIMEx_OC_N_Start()`
- `HAL_TIMEx_OC_N_Stop()`

- `HAL_TIMEx_OCN_Start_IT()`
- `HAL_TIMEx_OCN_Stop_IT()`
- `HAL_TIMEx_OCN_Start_DMA()`
- `HAL_TIMEx_OCN_Stop_DMA()`

69.2.5 Timer Complementary PWM functions

This section provides functions allowing to:

- Start the Complementary PWM.
- Stop the Complementary PWM.
- Start the Complementary PWM and enable interrupts.
- Stop the Complementary PWM and disable interrupts.
- Start the Complementary PWM and enable DMA transfers.
- Stop the Complementary PWM and disable DMA transfers.
- Start the Complementary Input Capture measurement.
- Stop the Complementary Input Capture.
- Start the Complementary Input Capture and enable interrupts.
- Stop the Complementary Input Capture and disable interrupts.
- Start the Complementary Input Capture and enable DMA transfers.
- Stop the Complementary Input Capture and disable DMA transfers.
- Start the Complementary One Pulse generation.
- Stop the Complementary One Pulse.
- Start the Complementary One Pulse and enable interrupts.
- Stop the Complementary One Pulse and disable interrupts.

This section contains the following APIs:

- `HAL_TIMEx_PWMN_Start()`
- `HAL_TIMEx_PWMN_Stop()`
- `HAL_TIMEx_PWMN_Start_IT()`
- `HAL_TIMEx_PWMN_Stop_IT()`
- `HAL_TIMEx_PWMN_Start_DMA()`
- `HAL_TIMEx_PWMN_Stop_DMA()`

69.2.6 Timer Complementary One Pulse functions

This section provides functions allowing to:

- Start the Complementary One Pulse generation.
- Stop the Complementary One Pulse.
- Start the Complementary One Pulse and enable interrupts.
- Stop the Complementary One Pulse and disable interrupts.

This section contains the following APIs:

- `HAL_TIMEx_OnePulseN_Start()`
- `HAL_TIMEx_OnePulseN_Stop()`
- `HAL_TIMEx_OnePulseN_Start_IT()`
- `HAL_TIMEx_OnePulseN_Stop_IT()`

69.2.7 Peripheral Control functions

This section provides functions allowing to:

- Configure the commutation event in case of use of the Hall sensor interface.
- Configure Output channels for OC and PWM mode.
- Configure Complementary channels, break features and dead time.
- Configure Master synchronization.

- Configure timer remapping capabilities.

This section contains the following APIs:

- `HAL_TIMEx_ConfigCommutEvent()`
- `HAL_TIMEx_ConfigCommutEvent_IT()`
- `HAL_TIMEx_ConfigCommutEvent_DMA()`
- `HAL_TIMEx_MasterConfigSynchronization()`
- `HAL_TIMEx_ConfigBreakDeadTime()`
- `HAL_TIMEx_RemapConfig()`

69.2.8 Extended Callbacks functions

This section provides Extended TIM callback functions:

- Timer Commutation callback
- Timer Break callback

This section contains the following APIs:

- `HAL_TIMEx_CommutCallback()`
- `HAL_TIMEx_CommutHalfCpltCallback()`
- `HAL_TIMEx_BreakCallback()`

69.2.9 Extended Peripheral State functions

This subsection permits to get in run-time the status of the peripheral and the data flow.

This section contains the following APIs:

- `HAL_TIMEx_HallSensor_GetState()`
- `HAL_TIMEx_GetChannelNState()`

69.2.10 Detailed description of functions

HAL_TIMEx_HallSensor_Init

Function name

```
HAL_StatusTypeDef HAL_TIMEx_HallSensor_Init(TIM_HandleTypeDef * htim, const
TIM_HallSensor_InitTypeDef * sConfig)
```

Function description

Initializes the TIM Hall Sensor Interface and initialize the associated handle.

Parameters

- **htim**: TIM Hall Sensor Interface handle
- **sConfig**: TIM Hall Sensor configuration structure

Return values

- **HAL**: status

Notes

- When the timer instance is initialized in Hall Sensor Interface mode, timer channels 1 and channel 2 are reserved and cannot be used for other purpose.

HAL_TIMEx_HallSensor_DeInit

Function name

```
HAL_StatusTypeDef HAL_TIMEx_HallSensor_DeInit(TIM_HandleTypeDef * htim)
```

Function description

DeInitializes the TIM Hall Sensor interface.

Parameters

- **htim**: TIM Hall Sensor Interface handle

Return values

- **HAL**: status

HAL_TIMEx_HallSensor_MspInit

Function name

void HAL_TIMEx_HallSensor_MspInit (TIM_HandleTypeDef * htim)

Function description

Initializes the TIM Hall Sensor MSP.

Parameters

- **htim**: TIM Hall Sensor Interface handle

Return values

- **None**:

HAL_TIMEx_HallSensor_MspDeInit

Function name

void HAL_TIMEx_HallSensor_MspDeInit (TIM_HandleTypeDef * htim)

Function description

DeInitializes TIM Hall Sensor MSP.

Parameters

- **htim**: TIM Hall Sensor Interface handle

Return values

- **None**:

HAL_TIMEx_HallSensor_Start

Function name

HAL_StatusTypeDef HAL_TIMEx_HallSensor_Start (TIM_HandleTypeDef * htim)

Function description

Starts the TIM Hall Sensor Interface.

Parameters

- **htim**: TIM Hall Sensor Interface handle

Return values

- **HAL**: status

HAL_TIMEx_HallSensor_Stop

Function name

HAL_StatusTypeDef HAL_TIMEx_HallSensor_Stop (TIM_HandleTypeDef * htim)

Function description

Stops the TIM Hall sensor Interface.

Parameters

- **htim**: TIM Hall Sensor Interface handle

Return values

- **HAL:** status

HAL_TIMEx_HallSensor_Start_IT

Function name

HAL_StatusTypeDef HAL_TIMEx_HallSensor_Start_IT (TIM_HandleTypeDef * htim)

Function description

Starts the TIM Hall Sensor Interface in interrupt mode.

Parameters

- **htim:** TIM Hall Sensor Interface handle

Return values

- **HAL:** status

HAL_TIMEx_HallSensor_Stop_IT

Function name

HAL_StatusTypeDef HAL_TIMEx_HallSensor_Stop_IT (TIM_HandleTypeDef * htim)

Function description

Stops the TIM Hall Sensor Interface in interrupt mode.

Parameters

- **htim:** TIM Hall Sensor Interface handle

Return values

- **HAL:** status

HAL_TIMEx_HallSensor_Start_DMA

Function name

HAL_StatusTypeDef HAL_TIMEx_HallSensor_Start_DMA (TIM_HandleTypeDef * htim, uint32_t * pData, uint16_t Length)

Function description

Starts the TIM Hall Sensor Interface in DMA mode.

Parameters

- **htim:** TIM Hall Sensor Interface handle
- **pData:** The destination Buffer address.
- **Length:** The length of data to be transferred from TIM peripheral to memory.

Return values

- **HAL:** status

HAL_TIMEx_HallSensor_Stop_DMA

Function name

HAL_StatusTypeDef HAL_TIMEx_HallSensor_Stop_DMA (TIM_HandleTypeDef * htim)

Function description

Stops the TIM Hall Sensor Interface in DMA mode.

Parameters

- **htim**: TIM Hall Sensor Interface handle

Return values

- **HAL**: status

HAL_TIMEx_OCN_Start

Function name

HAL_StatusTypeDef HAL_TIMEx_OCN_Start (TIM_HandleTypeDef * htim, uint32_t Channel)

Function description

Starts the TIM Output Compare signal generation on the complementary output.

Parameters

- **htim**: TIM Output Compare handle
- **Channel**: TIM Channel to be enabled This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1 selected
 - TIM_CHANNEL_2: TIM Channel 2 selected
 - TIM_CHANNEL_3: TIM Channel 3 selected

Return values

- **HAL**: status

HAL_TIMEx_OCN_Stop

Function name

HAL_StatusTypeDef HAL_TIMEx_OCN_Stop (TIM_HandleTypeDef * htim, uint32_t Channel)

Function description

Stops the TIM Output Compare signal generation on the complementary output.

Parameters

- **htim**: TIM handle
- **Channel**: TIM Channel to be disabled This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1 selected
 - TIM_CHANNEL_2: TIM Channel 2 selected
 - TIM_CHANNEL_3: TIM Channel 3 selected

Return values

- **HAL**: status

HAL_TIMEx_OCN_Start_IT

Function name

HAL_StatusTypeDef HAL_TIMEx_OCN_Start_IT (TIM_HandleTypeDef * htim, uint32_t Channel)

Function description

Starts the TIM Output Compare signal generation in interrupt mode on the complementary output.

Parameters

- **htim**: TIM OC handle
- **Channel**: TIM Channel to be enabled This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1 selected
 - TIM_CHANNEL_2: TIM Channel 2 selected
 - TIM_CHANNEL_3: TIM Channel 3 selected

Return values

- **HAL:** status

HAL_TIMEx_OCN_Stop_IT

Function name

HAL_StatusTypeDef HAL_TIMEx_OCN_Stop_IT (TIM_HandleTypeDef * htim, uint32_t Channel)

Function description

Stops the TIM Output Compare signal generation in interrupt mode on the complementary output.

Parameters

- **htim:** TIM Output Compare handle
- **Channel:** TIM Channel to be disabled This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1 selected
 - TIM_CHANNEL_2: TIM Channel 2 selected
 - TIM_CHANNEL_3: TIM Channel 3 selected

Return values

- **HAL:** status

HAL_TIMEx_OCN_Start_DMA

Function name

HAL_StatusTypeDef HAL_TIMEx_OCN_Start_DMA (TIM_HandleTypeDef * htim, uint32_t Channel, const uint32_t * pData, uint16_t Length)

Function description

Starts the TIM Output Compare signal generation in DMA mode on the complementary output.

Parameters

- **htim:** TIM Output Compare handle
- **Channel:** TIM Channel to be enabled This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1 selected
 - TIM_CHANNEL_2: TIM Channel 2 selected
 - TIM_CHANNEL_3: TIM Channel 3 selected
- **pData:** The source Buffer address.
- **Length:** The length of data to be transferred from memory to TIM peripheral

Return values

- **HAL:** status

HAL_TIMEx_OCN_Stop_DMA

Function name

HAL_StatusTypeDef HAL_TIMEx_OCN_Stop_DMA (TIM_HandleTypeDef * htim, uint32_t Channel)

Function description

Stops the TIM Output Compare signal generation in DMA mode on the complementary output.

Parameters

- **htim:** TIM Output Compare handle
- **Channel:** TIM Channel to be disabled This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1 selected
 - TIM_CHANNEL_2: TIM Channel 2 selected
 - TIM_CHANNEL_3: TIM Channel 3 selected

Return values

- **HAL:** status

HAL_TIMEx_PWMN_Start

Function name

HAL_StatusTypeDef HAL_TIMEx_PWMN_Start (TIM_HandleTypeDef * htim, uint32_t Channel)

Function description

Starts the PWM signal generation on the complementary output.

Parameters

- **htim:** TIM handle
- **Channel:** TIM Channel to be enabled This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1 selected
 - TIM_CHANNEL_2: TIM Channel 2 selected
 - TIM_CHANNEL_3: TIM Channel 3 selected

Return values

- **HAL:** status

HAL_TIMEx_PWMN_Stop

Function name

HAL_StatusTypeDef HAL_TIMEx_PWMN_Stop (TIM_HandleTypeDef * htim, uint32_t Channel)

Function description

Stops the PWM signal generation on the complementary output.

Parameters

- **htim:** TIM handle
- **Channel:** TIM Channel to be disabled This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1 selected
 - TIM_CHANNEL_2: TIM Channel 2 selected
 - TIM_CHANNEL_3: TIM Channel 3 selected

Return values

- **HAL:** status

HAL_TIMEx_PWMN_Start_IT

Function name

HAL_StatusTypeDef HAL_TIMEx_PWMN_Start_IT (TIM_HandleTypeDef * htim, uint32_t Channel)

Function description

Starts the PWM signal generation in interrupt mode on the complementary output.

Parameters

- **htim:** TIM handle
- **Channel:** TIM Channel to be disabled This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1 selected
 - TIM_CHANNEL_2: TIM Channel 2 selected
 - TIM_CHANNEL_3: TIM Channel 3 selected

Return values

- **HAL:** status

HAL_TIMEx_PWMN_Stop_IT

Function name

HAL_StatusTypeDef HAL_TIMEx_PWMN_Stop_IT (TIM_HandleTypeDef * htim, uint32_t Channel)

Function description

Stops the PWM signal generation in interrupt mode on the complementary output.

Parameters

- **htim:** TIM handle
- **Channel:** TIM Channel to be disabled This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1 selected
 - TIM_CHANNEL_2: TIM Channel 2 selected
 - TIM_CHANNEL_3: TIM Channel 3 selected

Return values

- **HAL:** status

HAL_TIMEx_PWMN_Start_DMA

Function name

HAL_StatusTypeDef HAL_TIMEx_PWMN_Start_DMA (TIM_HandleTypeDef * htim, uint32_t Channel, const uint32_t * pData, uint16_t Length)

Function description

Starts the TIM PWM signal generation in DMA mode on the complementary output.

Parameters

- **htim:** TIM handle
- **Channel:** TIM Channel to be enabled This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1 selected
 - TIM_CHANNEL_2: TIM Channel 2 selected
 - TIM_CHANNEL_3: TIM Channel 3 selected
- **pData:** The source Buffer address.
- **Length:** The length of data to be transferred from memory to TIM peripheral

Return values

- **HAL:** status

HAL_TIMEx_PWMN_Stop_DMA

Function name

HAL_StatusTypeDef HAL_TIMEx_PWMN_Stop_DMA (TIM_HandleTypeDef * htim, uint32_t Channel)

Function description

Stops the TIM PWM signal generation in DMA mode on the complementary output.

Parameters

- **htim:** TIM handle
- **Channel:** TIM Channel to be disabled This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1 selected
 - TIM_CHANNEL_2: TIM Channel 2 selected
 - TIM_CHANNEL_3: TIM Channel 3 selected

Return values

- **HAL:** status

HAL_TIMEx_OnePulseN_Start

Function name

HAL_StatusTypeDef HAL_TIMEx_OnePulseN_Start (TIM_HandleTypeDef * htim, uint32_t OutputChannel)

Function description

Starts the TIM One Pulse signal generation on the complementary output.

Parameters

- **htim:** TIM One Pulse handle
- **OutputChannel:** pulse output channel to enable This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1 selected
 - TIM_CHANNEL_2: TIM Channel 2 selected

Return values

- **HAL:** status

Notes

- OutputChannel must match the pulse output channel chosen when calling HAL_TIM_OnePulse_ConfigChannel().

HAL_TIMEx_OnePulseN_Stop

Function name

HAL_StatusTypeDef HAL_TIMEx_OnePulseN_Stop (TIM_HandleTypeDef * htim, uint32_t OutputChannel)

Function description

Stops the TIM One Pulse signal generation on the complementary output.

Parameters

- **htim:** TIM One Pulse handle
- **OutputChannel:** pulse output channel to disable This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1 selected
 - TIM_CHANNEL_2: TIM Channel 2 selected

Return values

- **HAL:** status

Notes

- OutputChannel must match the pulse output channel chosen when calling HAL_TIM_OnePulse_ConfigChannel().

HAL_TIMEx_OnePulseN_Start_IT

Function name

HAL_StatusTypeDef HAL_TIMEx_OnePulseN_Start_IT (TIM_HandleTypeDef * htim, uint32_t OutputChannel)

Function description

Starts the TIM One Pulse signal generation in interrupt mode on the complementary channel.

Parameters

- **htim:** TIM One Pulse handle
- **OutputChannel:** pulse output channel to enable This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1 selected
 - TIM_CHANNEL_2: TIM Channel 2 selected

Return values

- **HAL:** status

Notes

- OutputChannel must match the pulse output channel chosen when calling HAL_TIM_OnePulse_ConfigChannel().

HAL_TIMEx_OnePulseN_Stop_IT

Function name

HAL_StatusTypeDef HAL_TIMEx_OnePulseN_Stop_IT (TIM_HandleTypeDef * htim, uint32_t OutputChannel)

Function description

Stops the TIM One Pulse signal generation in interrupt mode on the complementary channel.

Parameters

- **htim:** TIM One Pulse handle
- **OutputChannel:** pulse output channel to disable This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1 selected
 - TIM_CHANNEL_2: TIM Channel 2 selected

Return values

- **HAL:** status

Notes

- OutputChannel must match the pulse output channel chosen when calling HAL_TIM_OnePulse_ConfigChannel().

HAL_TIMEx_ConfigCommutEvent

Function name

HAL_StatusTypeDef HAL_TIMEx_ConfigCommutEvent (TIM_HandleTypeDef * htim, uint32_t InputTrigger, uint32_t CommutationSource)

Function description

Configure the TIM commutation event sequence.

Parameters

- **htim:** TIM handle
- **InputTrigger:** the Internal trigger corresponding to the Timer Interfacing with the Hall sensor This parameter can be one of the following values:
 - TIM_TS_ITR0: Internal trigger 0 selected
 - TIM_TS_ITR1: Internal trigger 1 selected
 - TIM_TS_ITR2: Internal trigger 2 selected
 - TIM_TS_ITR3: Internal trigger 3 selected
 - TIM_TS_NONE: No trigger is needed
- **CommutationSource:** the Commutation Event source This parameter can be one of the following values:
 - TIM_COMMUTATION_TRGI: Commutation source is the TRGI of the Interface Timer
 - TIM_COMMUTATION_SOFTWARE: Commutation source is set by software using the COMG bit

Return values

- **HAL:** status

Notes

- This function is mandatory to use the commutation event in order to update the configuration at each commutation detection on the TRGI input of the Timer, the typical use of this feature is with the use of another Timer(interface Timer) configured in Hall sensor interface, this interface Timer will generate the commutation at its TRGO output (connected to Timer used in this function) each time the TI1 of the Interface Timer detect a commutation at its input TI1.

HAL_TIMEx_ConfigCommutEvent_IT

Function name

HAL_StatusTypeDef HAL_TIMEx_ConfigCommutEvent_IT (TIM_HandleTypeDef * htim, uint32_t InputTrigger, uint32_t CommutationSource)

Function description

Configure the TIM commutation event sequence with interrupt.

Parameters

- **htim:** TIM handle
- **InputTrigger:** the Internal trigger corresponding to the Timer Interfacing with the Hall sensor This parameter can be one of the following values:
 - TIM_TS_ITR0: Internal trigger 0 selected
 - TIM_TS_ITR1: Internal trigger 1 selected
 - TIM_TS_ITR2: Internal trigger 2 selected
 - TIM_TS_ITR3: Internal trigger 3 selected
 - TIM_TS_NONE: No trigger is needed
- **CommutationSource:** the Commutation Event source This parameter can be one of the following values:
 - TIM_COMMUTATION_TRGI: Commutation source is the TRGI of the Interface Timer
 - TIM_COMMUTATION_SOFTWARE: Commutation source is set by software using the COMG bit

Return values

- **HAL:** status

Notes

- This function is mandatory to use the commutation event in order to update the configuration at each commutation detection on the TRGI input of the Timer, the typical use of this feature is with the use of another Timer(interface Timer) configured in Hall sensor interface, this interface Timer will generate the commutation at its TRGO output (connected to Timer used in this function) each time the TI1 of the Interface Timer detect a commutation at its input TI1.

HAL_TIMEx_ConfigCommutEvent_DMA

Function name

HAL_StatusTypeDef HAL_TIMEx_ConfigCommutEvent_DMA (TIM_HandleTypeDef * htim, uint32_t InputTrigger, uint32_t CommutationSource)

Function description

Configure the TIM commutation event sequence with DMA.

Parameters

- **htim:** TIM handle
- **InputTrigger:** the Internal trigger corresponding to the Timer Interfacing with the Hall sensor This parameter can be one of the following values:
 - TIM_TS_ITR0: Internal trigger 0 selected
 - TIM_TS_ITR1: Internal trigger 1 selected
 - TIM_TS_ITR2: Internal trigger 2 selected
 - TIM_TS_ITR3: Internal trigger 3 selected
 - TIM_TS_NONE: No trigger is needed
- **CommutationSource:** the Commutation Event source This parameter can be one of the following values:
 - TIM_COMMUTATION_TRGI: Commutation source is the TRGI of the Interface Timer
 - TIM_COMMUTATION_SOFTWARE: Commutation source is set by software using the COMG bit

Return values

- **HAL:** status

Notes

- This function is mandatory to use the commutation event in order to update the configuration at each commutation detection on the TRGI input of the Timer, the typical use of this feature is with the use of another Timer(interface Timer) configured in Hall sensor interface, this interface Timer will generate the commutation at its TRGO output (connected to Timer used in this function) each time the TI1 of the Interface Timer detect a commutation at its input TI1.
- The user should configure the DMA in his own software, in This function only the COMDE bit is set

HAL_TIMEx_MasterConfigSynchronization

Function name

HAL_StatusTypeDef HAL_TIMEx_MasterConfigSynchronization (TIM_HandleTypeDef * htim, const TIM_MasterConfigTypeDef * sMasterConfig)

Function description

Configures the TIM in master mode.

Parameters

- **htim:** TIM handle.
- **sMasterConfig:** pointer to a TIM_MasterConfigTypeDef structure that contains the selected trigger output (TRGO) and the Master/Slave mode.

Return values

- **HAL:** status

HAL_TIMEx_ConfigBreakDeadTime

Function name

HAL_StatusTypeDef HAL_TIMEx_ConfigBreakDeadTime (TIM_HandleTypeDef * htim, const TIM_BreakDeadTimeConfigTypeDef * sBreakDeadTimeConfig)

Function description

Configures the Break feature, dead time, Lock level, OSSR/OSSR State and the AOE(automatic output enable).

Parameters

- **htim:** TIM handle
- **sBreakDeadTimeConfig:** pointer to a TIM_ConfigBreakDeadConfigTypeDef structure that contains the BDTR Register configuration information for the TIM peripheral.

Return values

- **HAL:** status

Notes

- Interrupts can be generated when an active level is detected on the break input, the break 2 input or the system break input. Break interrupt can be enabled by calling the __HAL_TIM_ENABLE_IT macro.

HAL_TIMEx_RemapConfig

Function name

HAL_StatusTypeDef HAL_TIMEx_RemapConfig (TIM_HandleTypeDef * htim, uint32_t Remap)

Function description

Configures the TIMx Remapping input capabilities.

Parameters

- **htim:** TIM handle.
- **Remap:** specifies the TIM remapping source. For TIM1, the parameter can have the following values: (**)
 - TIM_TIM1_TIM3_TRGO: TIM1 ITR2 is connected to TIM3 TRGO
 - TIM_TIM1_LPTIM: TIM1 ITR2 is connected to LPTIM1 outputFor TIM2, the parameter can have the following values: (**)
 - TIM_TIM2_TIM8_TRGO: TIM2 ITR1 is connected to TIM8 TRGO (*)
 - TIM_TIM2_ETH_PTP: TIM2 ITR1 is connected to PTP trigger output (*)
 - TIM_TIM2_USBFS_SOF: TIM2 ITR1 is connected to OTG FS SOF
 - TIM_TIM2_USBHS_SOF: TIM2 ITR1 is connected to OTG FS SOFFor TIM5, the parameter can have the following values:
 - TIM_TIM5_GPIO: TIM5 TI4 is connected to GPIO
 - TIM_TIM5_LSI: TIM5 TI4 is connected to LSI
 - TIM_TIM5_LSE: TIM5 TI4 is connected to LSE
 - TIM_TIM5_RTC: TIM5 TI4 is connected to the RTC wakeup interrupt
 - TIM_TIM5_TIM3_TRGO: TIM5 ITR1 is connected to TIM3 TRGO (*)
 - TIM_TIM5_LPTIM: TIM5 ITR1 is connected to LPTIM1 output (*)For TIM9, the parameter can have the following values: (**)
 - TIM_TIM9_TIM3_TRGO: TIM9 ITR1 is connected to TIM3 TRGO
 - TIM_TIM9_LPTIM: TIM9 ITR1 is connected to LPTIM1 outputFor TIM11, the parameter can have the following values:
 - TIM_TIM11_GPIO: TIM11 TI1 is connected to GPIO
 - TIM_TIM11_HSE: TIM11 TI1 is connected to HSE_RTC clock
 - TIM_TIM11_SPDIFRX: TIM11 TI1 is connected to SPDIFRX_FRAME_SYNC (*)(*) Value not defined in all devices.
- (**) Register not available in all devices.

Return values

- **HAL:** status

HAL_TIMEx_CommutCallback

Function name

void HAL_TIMEx_CommutCallback (TIM_HandleTypeDef * htim)

Function description

Hall commutation changed callback in non-blocking mode.

Parameters

- **htim:** TIM handle

Return values

- **None:**

HAL_TIMEx_CommutHalfCpltCallback

Function name

void HAL_TIMEx_CommutHalfCpltCallback (TIM_HandleTypeDef * htim)

Function description

Hall commutation changed half complete callback in non-blocking mode.

Parameters

- **htim:** TIM handle

Return values

- **None:**

HAL_TIMEx_BreakCallback

Function name

void HAL_TIMEx_BreakCallback (TIM_HandleTypeDef * htim)

Function description

Hall Break detection callback in non-blocking mode.

Parameters

- **htim:** TIM handle

Return values

- **None:**

HAL_TIMEx_HallSensor_GetState

Function name

HAL_TIM_StateTypeDef HAL_TIMEx_HallSensor_GetState (const TIM_HandleTypeDef * htim)

Function description

Return the TIM Hall Sensor interface handle state.

Parameters

- **htim:** TIM Hall Sensor handle

Return values

- **HAL:** state

HAL_TIMEx_GetChannelINState

Function name

HAL_TIM_ChannelStateTypeDef HAL_TIMEx_GetChannelINState (const TIM_HandleTypeDef * htim, uint32_t ChannelIN)

Function description

Return actual state of the TIM complementary channel.

Parameters

- **htim:** TIM handle
- **ChannelIN:** TIM Complementary channel This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1
 - TIM_CHANNEL_2: TIM Channel 2
 - TIM_CHANNEL_3: TIM Channel 3

Return values

- **TIM:** Complementary channel state

TIMEx_DMAMutationCplt

Function name

void TIMEx_DMAMutationCplt (DMA_HandleTypeDef * hdma)

Function description

TIM DMA Mutation callback.

Parameters

- **hdma:** pointer to DMA handle.

Return values

- **None:**

TIMEx_DMAMutationHalfCplt

Function name

void TIMEx_DMAMutationHalfCplt (DMA_HandleTypeDef * hdma)

Function description

TIM DMA Mutation half complete callback.

Parameters

- **hdma:** pointer to DMA handle.

Return values

- **None:**

69.3 TIMEx Firmware driver defines

The following section lists the various define and macros of the module.

69.3.1

TIMEx

TIMEx

TIM Extended Remapping

TIM_TIM2_TIM8_TRGO

TIM2 ITR1 is connected to TIM8 TRGO

TIM_TIM2_USBFS_SOF

TIM2 ITR1 is connected to OTG FS SOF

TIM_TIM2_USBHS_SOF

TIM2 ITR1 is connected to OTG HS SOF

TIM_TIM5_GPIO

TIM5 TI4 is connected to GPIO

TIM_TIM5_LSI

TIM5 TI4 is connected to LSI

TIM_TIM5_LSE

TIM5 TI4 is connected to LSE

TIM_TIM5_RTC

TIM5 TI4 is connected to the RTC wakeup interrupt

TIM_TIM11_GPIO

TIM11 TI1 is connected to GPIO

TIM_TIM11_HSE

TIM11 TI1 is connected to HSE_RTC clock

70 HAL UART Generic Driver

70.1 UART Firmware driver registers structures

70.1.1 UART_InitTypeDef

UART_InitTypeDef is defined in the stm32f4xx_hal_uart.h

Data Fields

- **uint32_t BaudRate**
- **uint32_t WordLength**
- **uint32_t StopBits**
- **uint32_t Parity**
- **uint32_t Mode**
- **uint32_t HwFlowCtl**
- **uint32_t OverSampling**

Field Documentation

- **uint32_t UART_InitTypeDef::BaudRate**
 This member configures the UART communication baud rate. The baud rate is computed using the following formula:
 - $\text{IntegerDivider} = ((\text{PCLKx}) / (8 * (\text{OVR8} + 1) * (\text{huart->Init.BaudRate})))$
 - $\text{FractionalDivider} = ((\text{IntegerDivider} - ((\text{uint32_t}) \text{IntegerDivider})) * 8 * (\text{OVR8} + 1)) + 0.5$ Where OVR8 is the "oversampling by 8 mode" configuration bit in the CR1 register.
 - **uint32_t UART_InitTypeDef::WordLength**
 Specifies the number of data bits transmitted or received in a frame. This parameter can be a value of [UART_Word_Length](#)
 - **uint32_t UART_InitTypeDef::StopBits**
 Specifies the number of stop bits transmitted. This parameter can be a value of [UART_Stop_Bits](#)
 - **uint32_t UART_InitTypeDef::Parity**
 Specifies the parity mode. This parameter can be a value of [UART_Parity](#)
- Note:**
- When parity is enabled, the computed parity is inserted at the MSB position of the transmitted data (9th bit when the word length is set to 9 data bits; 8th bit when the word length is set to 8 data bits).
- **uint32_t UART_InitTypeDef::Mode**
 Specifies whether the Receive or Transmit mode is enabled or disabled. This parameter can be a value of [UART_Mode](#)
 - **uint32_t UART_InitTypeDef::HwFlowCtl**
 Specifies whether the hardware flow control mode is enabled or disabled. This parameter can be a value of [UART_Hardware_Flow_Control](#)
 - **uint32_t UART_InitTypeDef::OverSampling**
 Specifies whether the Over sampling 8 is enabled or disabled, to achieve higher speed (up to fPCLK/8). This parameter can be a value of [UART_Over_Sampling](#)

70.1.2 __UART_HandleTypeDef

__UART_HandleTypeDef is defined in the stm32f4xx_hal_uart.h

Data Fields

- **USART_TypeDef * Instance**
- **UART_InitTypeDef Init**
- **const uint8_t * pTxBuffPtr**
- **uint16_t TxXferSize**
- **__IO uint16_t TxXferCount**
- **uint8_t * pRxBuffPtr**
- **uint16_t RxXferSize**

- `__IO uint16_t RxXferCount`
- `__IO HAL_UART_RxTypeTypeDef ReceptionType`
- `__IO HAL_UART_RxEventTypeTypeDef RxEventType`
- `DMA_HandleTypeDef * hdmatrix`
- `DMA_HandleTypeDef * hdmatrix`
- `HAL_LockTypeDef Lock`
- `__IO HAL_UART_StateTypeDef gState`
- `__IO HAL_UART_StateTypeDef RxState`
- `__IO uint32_t ErrorCode`
- `void(* TxHalfCpltCallback`
- `void(* TxCpltCallback`
- `void(* RxHalfCpltCallback`
- `void(* RxCpltCallback`
- `void(* ErrorCallback`
- `void(* AbortCpltCallback`
- `void(* AbortTransmitCpltCallback`
- `void(* AbortReceiveCpltCallback`
- `void(* WakeupCallback`
- `void(* RxEventCallback`
- `void(* MspInitCallback`
- `void(* MspDeInitCallback`

Field Documentation

- **`USART_TypeDef* __UART_HandleTypeDef::Instance`**
UART registers base address
- **`UART_InitTypeDef __UART_HandleTypeDef::Init`**
UART communication parameters
- **`const uint8_t* __UART_HandleTypeDef::pTxBuffPtr`**
Pointer to UART Tx transfer Buffer
- **`uint16_t __UART_HandleTypeDef::TxXferSize`**
UART Tx Transfer size
- **`__IO uint16_t __UART_HandleTypeDef::TxXferCount`**
UART Tx Transfer Counter
- **`uint8_t* __UART_HandleTypeDef::pRxBuffPtr`**
Pointer to UART Rx transfer Buffer
- **`uint16_t __UART_HandleTypeDef::RxXferSize`**
UART Rx Transfer size
- **`__IO uint16_t __UART_HandleTypeDef::RxXferCount`**
UART Rx Transfer Counter
- **`__IO HAL_UART_RxTypeTypeDef __UART_HandleTypeDef::ReceptionType`**
Type of ongoing reception
- **`__IO HAL_UART_RxEventTypeTypeDef __UART_HandleTypeDef::RxEventType`**
Type of Rx Event
- **`DMA_HandleTypeDef* __UART_HandleTypeDef::hdmatrix`**
UART Tx DMA Handle parameters
- **`DMA_HandleTypeDef* __UART_HandleTypeDef::hdmatrix`**
UART Rx DMA Handle parameters
- **`HAL_LockTypeDef __UART_HandleTypeDef::Lock`**
Locking object
- **`__IO HAL_UART_StateTypeDef __UART_HandleTypeDef::gState`**
UART state information related to global Handle management and also related to Tx operations. This parameter can be a value of `HAL_UART_StateTypeDef`

- **__IO HAL_UART_StateTypeDef __UART_HandleTypeDef::RxState**
UART state information related to Rx operations. This parameter can be a value of **HAL_UART_StateTypeDef**
- **__IO uint32_t __UART_HandleTypeDef::ErrorCode**
UART Error code
- **void(* __UART_HandleTypeDef::TxHalfCpltCallback)(struct __UART_HandleTypeDef *huart)**
UART Tx Half Complete Callback
- **void(* __UART_HandleTypeDef::TxCpltCallback)(struct __UART_HandleTypeDef *huart)**
UART Tx Complete Callback
- **void(* __UART_HandleTypeDef::RxHalfCpltCallback)(struct __UART_HandleTypeDef *huart)**
UART Rx Half Complete Callback
- **void(* __UART_HandleTypeDef::RxCpltCallback)(struct __UART_HandleTypeDef *huart)**
UART Rx Complete Callback
- **void(* __UART_HandleTypeDef::ErrorCallback)(struct __UART_HandleTypeDef *huart)**
UART Error Callback
- **void(* __UART_HandleTypeDef::AbortCpltCallback)(struct __UART_HandleTypeDef *huart)**
UART Abort Complete Callback
- **void(* __UART_HandleTypeDef::AbortTransmitCpltCallback)(struct __UART_HandleTypeDef *huart)**
UART Abort Transmit Complete Callback
- **void(* __UART_HandleTypeDef::AbortReceiveCpltCallback)(struct __UART_HandleTypeDef *huart)**
UART Abort Receive Complete Callback
- **void(* __UART_HandleTypeDef::WakeupCallback)(struct __UART_HandleTypeDef *huart)**
UART Wakeup Callback
- **void(* __UART_HandleTypeDef::RxEventCallback)(struct __UART_HandleTypeDef *huart, uint16_t Pos)**
UART Reception Event Callback
- **void(* __UART_HandleTypeDef::MspInitCallback)(struct __UART_HandleTypeDef *huart)**
UART Msp Init callback
- **void(* __UART_HandleTypeDef::MspDeInitCallback)(struct __UART_HandleTypeDef *huart)**
UART Msp DeInit callback

70.2 UART Firmware driver API description

The following section lists the various functions of the UART library.

70.2.1 How to use this driver

The UART HAL driver can be used as follows:

1. Declare a **UART_HandleTypeDef** handle structure (eg. **UART_HandleTypeDef huart**).

2. Initialize the UART low level resources by implementing the HAL_UART_MspInit() API:
 - a. Enable the USARTx interface clock.
 - b. UART pins configuration:
 - Enable the clock for the UART GPIOs.
 - Configure the UART TX/RX pins as alternate function pull-up.
 - c. NVIC configuration if you need to use interrupt process (HAL_UART_Transmit_IT() and HAL_UART_Receive_IT() APIs):
 - Configure the USARTx interrupt priority.
 - Enable the NVIC USART IRQ handle.
 - d. DMA Configuration if you need to use DMA process (HAL_UART_Transmit_DMA() and HAL_UART_Receive_DMA() APIs):
 - Declare a DMA handle structure for the Tx/Rx stream.
 - Enable the DMAx interface clock.
 - Configure the declared DMA handle structure with the required Tx/Rx parameters.
 - Configure the DMA Tx/Rx stream.
 - Associate the initialized DMA handle to the UART DMA Tx/Rx handle.
 - Configure the priority and enable the NVIC for the transfer complete interrupt on the DMA Tx/Rx stream.
 - Configure the USARTx interrupt priority and enable the NVIC USART IRQ handle (used for last byte sending completion detection in DMA non circular mode)
3. Program the Baud Rate, Word Length, Stop Bit, Parity, Hardware flow control and Mode(Receiver/Transmitter) in the huart Init structure.
4. For the UART asynchronous mode, initialize the UART registers by calling the HAL_UART_Init() API.
5. For the UART Half duplex mode, initialize the UART registers by calling the HAL_HalfDuplex_Init() API.
6. For the LIN mode, initialize the UART registers by calling the HAL_LIN_Init() API.
7. For the Multi-Processor mode, initialize the UART registers by calling the HAL_MultiProcessor_Init() API.

Note: *The specific UART interrupts (Transmission complete interrupt, RXNE interrupt and Error Interrupts) will be managed using the macros __HAL_UART_ENABLE_IT() and __HAL_UART_DISABLE_IT() inside the transmit and receive process.*

Note: *These APIs (HAL_UART_Init() and HAL_HalfDuplex_Init()) configure also the low level Hardware GPIO, CLOCK, CORTEX...etc) by calling the customized HAL_UART_MspInit() API.*

70.2.2 Callback registration

The compilation define USE_HAL_UART_REGISTER_CALLBACKS when set to 1 allows the user to configure dynamically the driver callbacks.

Use Function HAL_UART_RegisterCallback() to register a user callback. Function HAL_UART_RegisterCallback() allows to register following callbacks:

- TxHalfCpltCallback : Tx Half Complete Callback.
- TxCpltCallback : Tx Complete Callback.
- RxHalfCpltCallback : Rx Half Complete Callback.
- RxCpltCallback : Rx Complete Callback.
- ErrorCallback : Error Callback.
- AbortCpltCallback : Abort Complete Callback.
- AbortTransmitCpltCallback : Abort Transmit Complete Callback.
- AbortReceiveCpltCallback : Abort Receive Complete Callback.
- MspInitCallback : UART MspInit.
- MspDeInitCallback : UART MspDeInit. This function takes as parameters the HAL peripheral handle, the Callback ID and a pointer to the user callback function.

Use function HAL_UART_UnRegisterCallback() to reset a callback to the default weak (surcharged) function. HAL_UART_UnRegisterCallback() takes as parameters the HAL peripheral handle, and the Callback ID. This function allows to reset following callbacks:

- TxHalfCpltCallback : Tx Half Complete Callback.

- TxCpltCallback : Tx Complete Callback.
- RxHalfCpltCallback : Rx Half Complete Callback.
- RxCpltCallback : Rx Complete Callback.
- ErrorCallback : Error Callback.
- AbortCpltCallback : Abort Complete Callback.
- AbortTransmitCpltCallback : Abort Transmit Complete Callback.
- AbortReceiveCpltCallback : Abort Receive Complete Callback.
- MspInitCallback : UART MspInit.
- MspDeInitCallback : UART MspDeInit.

For specific callback RxEventCallback, use dedicated registration/reset functions: respectively HAL_UART_RegisterRxEventCallback() , HAL_UART_UnRegisterRxEventCallback().

By default, after the HAL_UART_Init() and when the state is HAL_UART_STATE_RESET all callbacks are set to the corresponding weak (surcharged) functions: examples HAL_UART_TxCpltCallback(), HAL_UART_RxHalfCpltCallback(). Exception done for MspInit and MspDeInit functions that are respectively reset to the legacy weak (surcharged) functions in the HAL_UART_Init() and HAL_UART_DeInit() only when these callbacks are null (not registered beforehand). If not, MspInit or MspDeInit are not null, the HAL_UART_Init() and HAL_UART_DeInit() keep and use the user MspInit/MspDeInit callbacks (registered beforehand).

Callbacks can be registered/unregistered in HAL_UART_STATE_READY state only. Exception done MspInit/ MspDeInit that can be registered/unregistered in HAL_UART_STATE_READY or HAL_UART_STATE_RESET state, thus registered (user) MspInit/DeInit callbacks can be used during the Init/DeInit. In that case first register the MspInit/MspDeInit user callbacks using HAL_UART_RegisterCallback() before calling HAL_UART_DeInit() or HAL_UART_Init() function.

When The compilation define USE_HAL_UART_REGISTER_CALLBACKS is set to 0 or not defined, the callback registration feature is not available and weak (surcharged) callbacks are used.

Three operation modes are available within this driver :

Polling mode IO operation

- Send an amount of data in blocking mode using HAL_UART_Transmit()
- Receive an amount of data in blocking mode using HAL_UART_Receive()

Interrupt mode IO operation

- Send an amount of data in non blocking mode using HAL_UART_Transmit_IT()
- At transmission end of transfer HAL_UART_TxCpltCallback is executed and user can add his own code by customization of function pointer HAL_UART_TxCpltCallback
- Receive an amount of data in non blocking mode using HAL_UART_Receive_IT()
- At reception end of transfer HAL_UART_RxCpltCallback is executed and user can add his own code by customization of function pointer HAL_UART_RxCpltCallback
- In case of transfer Error, HAL_UART_ErrorCallback() function is executed and user can add his own code by customization of function pointer HAL_UART_ErrorCallback

DMA mode IO operation

- Send an amount of data in non blocking mode (DMA) using HAL_UART_Transmit_DMA()
- At transmission end of half transfer HAL_UART_TxHalfCpltCallback is executed and user can add his own code by customization of function pointer HAL_UART_TxHalfCpltCallback
- At transmission end of transfer HAL_UART_TxCpltCallback is executed and user can add his own code by customization of function pointer HAL_UART_TxCpltCallback
- Receive an amount of data in non blocking mode (DMA) using HAL_UART_Receive_DMA()
- At reception end of half transfer HAL_UART_RxHalfCpltCallback is executed and user can add his own code by customization of function pointer HAL_UART_RxHalfCpltCallback
- At reception end of transfer HAL_UART_RxCpltCallback is executed and user can add his own code by customization of function pointer HAL_UART_RxCpltCallback
- In case of transfer Error, HAL_UART_ErrorCallback() function is executed and user can add his own code by customization of function pointer HAL_UART_ErrorCallback
- Pause the DMA Transfer using HAL_UART_DMAPause()

- Resume the DMA Transfer using HAL_UART_DMAResume()
- Stop the DMA Transfer using HAL_UART_DMAStop()

This subsection also provides a set of additional functions providing enhanced reception services to user. (For example, these functions allow application to handle use cases where number of data to be received is unknown).

1. Compared to standard reception services which only consider number of received data elements as reception completion criteria, these functions also consider additional events as triggers for updating reception status to caller : (+) Detection of inactivity period (RX line has not been active for a given period).
 - RX inactivity detected by IDLE event, i.e. RX line has been in idle state (normally high state) for 1 frame time, after last received byte.
2. There are two mode of transfer: (+) Blocking mode: The reception is performed in polling mode, until either expected number of data is received, or till IDLE event occurs. Reception is handled only during function execution. When function exits, no data reception could occur. HAL status and number of actually received data elements, are returned by function after finishing transfer. (+) Non-Blocking mode: The reception is performed using Interrupts or DMA. These API's return the HAL status. The end of the data processing will be indicated through the dedicated UART IRQ when using Interrupt mode or the DMA IRQ when using DMA mode. The HAL_UARTEx_RxEventCallback() user callback will be executed during Receive process The HAL_UART_ErrorCallback() user callback will be executed when a reception error is detected.
3. Blocking mode API: (+) HAL_UARTEx_ReceiveToldle()
4. Non-Blocking mode API with Interrupt: (+) HAL_UARTEx_ReceiveToldle_IT()
5. Non-Blocking mode API with DMA: (+) HAL_UARTEx_ReceiveToldle_DMA()

This subsection also provides a set of additional functions providing enhanced reception services to user. (For example, these functions allow application to handle use cases where number of data to be received is unknown). (#) Compared to standard reception services which only consider number of received data elements as reception completion criteria, these functions also consider additional events as triggers for updating reception status to caller :

- Detection of inactivity period (RX line has not been active for a given period).
 - RX inactivity detected by IDLE event, i.e. RX line has been in idle state (normally high state) for 1 frame time, after last received byte. (#) There are two mode of transfer:
- Blocking mode: The reception is performed in polling mode, until either expected number of data is received, or till IDLE event occurs. Reception is handled only during function execution. When function exits, no data reception could occur. HAL status and number of actually received data elements, are returned by function after finishing transfer.
- Non-Blocking mode: The reception is performed using Interrupts or DMA. These API's return the HAL status. The end of the data processing will be indicated through the dedicated UART IRQ when using Interrupt mode or the DMA IRQ when using DMA mode. The HAL_UARTEx_RxEventCallback() user callback will be executed during Receive process The HAL_UART_ErrorCallback() user callback will be executed when a reception error is detected. (#) Blocking mode API:
- HAL_UARTEx_ReceiveToldle() (#) Non-Blocking mode API with Interrupt:
- HAL_UARTEx_ReceiveToldle_IT() (#) Non-Blocking mode API with DMA:
- HAL_UARTEx_ReceiveToldle_DMA()

UART HAL driver macros list

Below the list of most used macros in UART HAL driver.

- __HAL_UART_ENABLE: Enable the UART peripheral
- __HAL_UART_DISABLE: Disable the UART peripheral
- __HAL_UART_GET_FLAG : Check whether the specified UART flag is set or not
- __HAL_UART_CLEAR_FLAG : Clear the specified UART pending flag
- __HAL_UART_ENABLE_IT: Enable the specified UART interrupt
- __HAL_UART_DISABLE_IT: Disable the specified UART interrupt
- __HAL_UART_GET_IT_SOURCE: Check whether the specified UART interrupt has occurred or not

Note: You can refer to the UART HAL driver header file for more useful macros

70.2.3 Initialization and Configuration functions

This subsection provides a set of functions allowing to initialize the USARTx or the UARTy in asynchronous mode.

- For the asynchronous mode only these parameters can be configured:
 - Baud Rate
 - Word Length
 - Stop Bit
 - Parity: If the parity is enabled, then the MSB bit of the data written in the data register is transmitted but is changed by the parity bit. Depending on the frame length defined by the M bit (8-bits or 9-bits), please refer to Reference manual for possible UART frame formats.
 - Hardware flow control
 - Receiver/transmitter modes
 - Over Sampling Method

The HAL_UART_Init(), HAL_HalfDuplex_Init(), HAL_LIN_Init() and HAL_MultiProcessor_Init() APIs follow respectively the UART asynchronous, UART Half duplex, LIN and Multi-Processor configuration procedures (details for the procedures are available in reference manual (RM0430 for STM32F4X3xx MCUs and RM0402 for STM32F412xx MCUs RM0383 for STM32F411xC/E MCUs and RM0401 for STM32F410xx MCUs RM0090 for STM32F4X5xx/STM32F4X7xx/STM32F429xx/STM32F439xx MCUs RM0390 for STM32F446xx MCUs and RM0386 for STM32F469xx/STM32F479xx MCUs)).

This section contains the following APIs:

- [*HAL_UART_Init\(\)*](#)
- [*HAL_HalfDuplex_Init\(\)*](#)
- [*HAL_LIN_Init\(\)*](#)
- [*HAL_MultiProcessor_Init\(\)*](#)
- [*HAL_UART_DeInit\(\)*](#)
- [*HAL_UART_MspInit\(\)*](#)
- [*HAL_UART_MspDeInit\(\)*](#)
- [*HAL_UART_RegisterCallback\(\)*](#)
- [*HAL_UART_UnRegisterCallback\(\)*](#)
- [*HAL_UART_RegisterRxEventCallback\(\)*](#)
- [*HAL_UART_UnRegisterRxEventCallback\(\)*](#)

70.2.4 IO operation functions

This section contains the following APIs:

- [*HAL_UART_Transmit\(\)*](#)
- [*HAL_UART_Receive\(\)*](#)
- [*HAL_UART_Transmit_IT\(\)*](#)
- [*HAL_UART_Receive_IT\(\)*](#)
- [*HAL_UART_Transmit_DMA\(\)*](#)
- [*HAL_UART_Receive_DMA\(\)*](#)
- [*HAL_UART_DMABase\(\)*](#)
- [*HAL_UART_DMAResume\(\)*](#)
- [*HAL_UART_DMABaseStop\(\)*](#)
- [*HAL_UARTEx_ReceiveTxdle\(\)*](#)
- [*HAL_UARTEx_ReceiveTxdle_IT\(\)*](#)
- [*HAL_UARTEx_ReceiveTxdle_DMA\(\)*](#)
- [*HAL_UARTEx_GetRxEventType\(\)*](#)
- [*HAL_UART_Abort\(\)*](#)
- [*HAL_UART_AbortTransmit\(\)*](#)
- [*HAL_UART_AbortReceive\(\)*](#)
- [*HAL_UART_Abort_IT\(\)*](#)
- [*HAL_UART_AbortTransmit_IT\(\)*](#)

- [*HAL_UART_AbortReceive_IT\(\)*](#)
- [*HAL_UART_IRQHandler\(\)*](#)
- [*HAL_UART_TxCpltCallback\(\)*](#)
- [*HAL_UART_TxHalfCpltCallback\(\)*](#)
- [*HAL_UART_RxCpltCallback\(\)*](#)
- [*HAL_UART_RxHalfCpltCallback\(\)*](#)
- [*HAL_UART_ErrorCallback\(\)*](#)
- [*HAL_UART_AbortCpltCallback\(\)*](#)
- [*HAL_UART_AbortTransmitCpltCallback\(\)*](#)
- [*HAL_UART_AbortReceiveCpltCallback\(\)*](#)
- [*HAL_UARTEx_RxEventCallback\(\)*](#)

70.2.5 Peripheral Control functions

This subsection provides a set of functions allowing to control the UART:

- [*HAL_LIN_SendBreak\(\)*](#) API can be helpful to transmit the break character.
- [*HAL_MultiProcessor_EnterMuteMode\(\)*](#) API can be helpful to enter the UART in mute mode.
- [*HAL_MultiProcessor_ExitMuteMode\(\)*](#) API can be helpful to exit the UART mute mode by software.
- [*HAL_HalfDuplex_EnableTransmitter\(\)*](#) API to enable the UART transmitter and disables the UART receiver in Half Duplex mode
- [*HAL_HalfDuplex_EnableReceiver\(\)*](#) API to enable the UART receiver and disables the UART transmitter in Half Duplex mode

This section contains the following APIs:

- [*HAL_LIN_SendBreak\(\)*](#)
- [*HAL_MultiProcessor_EnterMuteMode\(\)*](#)
- [*HAL_MultiProcessor_ExitMuteMode\(\)*](#)
- [*HAL_HalfDuplex_EnableTransmitter\(\)*](#)
- [*HAL_HalfDuplex_EnableReceiver\(\)*](#)

70.2.6 Peripheral State and Errors functions

This subsection provides a set of functions allowing to return the State of UART communication process, return Peripheral Errors occurred during communication process

- [*HAL_UART_GetState\(\)*](#) API can be helpful to check in run-time the state of the UART peripheral.
- [*HAL_UART_GetError\(\)*](#) check in run-time errors that could be occurred during communication.

This section contains the following APIs:

- [*HAL_UART_GetState\(\)*](#)
- [*HAL_UART_GetError\(\)*](#)

70.2.7 Detailed description of functions

HAL_UART_Init

Function name

HAL_StatusTypeDef HAL_UART_Init (UART_HandleTypeDef * huart)

Function description

Initializes the UART mode according to the specified parameters in the UART_InitTypeDef and create the associated handle.

Parameters

- **huart:** Pointer to a UART_HandleTypeDef structure that contains the configuration information for the specified UART module.

Return values

- **HAL:** status

HAL_HalfDuplex_Init

Function name

HAL_StatusTypeDef HAL_HalfDuplex_Init (UART_HandleTypeDef * huart)

Function description

Initializes the half-duplex mode according to the specified parameters in the UART_InitTypeDef and create the associated handle.

Parameters

- **huart:** Pointer to a UART_HandleTypeDef structure that contains the configuration information for the specified UART module.

Return values

- **HAL:** status

HAL_LIN_Init

Function name

HAL_StatusTypeDef HAL_LIN_Init (UART_HandleTypeDef * huart, uint32_t BreakDetectLength)

Function description

Initializes the LIN mode according to the specified parameters in the UART_InitTypeDef and create the associated handle.

Parameters

- **huart:** Pointer to a UART_HandleTypeDef structure that contains the configuration information for the specified UART module.
- **BreakDetectLength:** Specifies the LIN break detection length. This parameter can be one of the following values:
 - UART_LINBREAKDETECTLENGTH_10B: 10-bit break detection
 - UART_LINBREAKDETECTLENGTH_11B: 11-bit break detection

Return values

- **HAL:** status

HAL_MultiProcessor_Init

Function name

HAL_StatusTypeDef HAL_MultiProcessor_Init (UART_HandleTypeDef * huart, uint8_t Address, uint32_t WakeUpMethod)

Function description

Initializes the Multi-Processor mode according to the specified parameters in the UART_InitTypeDef and create the associated handle.

Parameters

- **huart:** Pointer to a UART_HandleTypeDef structure that contains the configuration information for the specified UART module.
- **Address:** USART address
- **WakeUpMethod:** specifies the USART wake-up method. This parameter can be one of the following values:
 - UART_WAKEUPMETHOD_IDLELINE: Wake-up by an idle line detection
 - UART_WAKEUPMETHOD_ADDRESSMARK: Wake-up by an address mark

Return values

- **HAL:** status

HAL_UART_DeInit

Function name

HAL_StatusTypeDef HAL_UART_DeInit (UART_HandleTypeDef * huart)

Function description

DeInitializes the UART peripheral.

Parameters

- **huart:** Pointer to a UART_HandleTypeDef structure that contains the configuration information for the specified UART module.

Return values

- **HAL:** status

HAL_UART_MspInit

Function name

void HAL_UART_MspInit (UART_HandleTypeDef * huart)

Function description

UART MSP Init.

Parameters

- **huart:** Pointer to a UART_HandleTypeDef structure that contains the configuration information for the specified UART module.

Return values

- **None:**

HAL_UART_MspDeInit

Function name

void HAL_UART_MspDeInit (UART_HandleTypeDef * huart)

Function description

UART MSP DeInit.

Parameters

- **huart:** Pointer to a UART_HandleTypeDef structure that contains the configuration information for the specified UART module.

Return values

- **None:**

HAL_UART_RegisterCallback

Function name

HAL_StatusTypeDef HAL_UART_RegisterCallback (UART_HandleTypeDef * huart, HAL_UART_CallbackIDTypeDef CallbackID, pUART_CallbackTypeDef pCallback)

Function description

Register a User UART Callback To be used instead of the weak predefined callback.

Parameters

- **huart:** uart handle
- **CallbackID:** ID of the callback to be registered This parameter can be one of the following values:
 - HAL_UART_TX_HALFCOMPLETE_CB_ID Tx Half Complete Callback ID
 - HAL_UART_TX_COMPLETE_CB_ID Tx Complete Callback ID
 - HAL_UART_RX_HALFCOMPLETE_CB_ID Rx Half Complete Callback ID
 - HAL_UART_RX_COMPLETE_CB_ID Rx Complete Callback ID
 - HAL_UART_ERROR_CB_ID Error Callback ID
 - HAL_UART_ABORT_COMPLETE_CB_ID Abort Complete Callback ID
 - HAL_UART_ABORT_TRANSMIT_COMPLETE_CB_ID Abort Transmit Complete Callback ID
 - HAL_UART_ABORT_RECEIVE_COMPLETE_CB_ID Abort Receive Complete Callback ID
 - HAL_UART_MSPINIT_CB_ID MspInit Callback ID
 - HAL_UART_MSPDEINIT_CB_ID MspDeInit Callback ID
- **pCallback:** pointer to the Callback function

Return values

- **HAL:** status

Notes

- The HAL_UART_RegisterCallback() may be called before HAL_UART_Init(), HAL_HalfDuplex_Init(), HAL_LIN_Init(), HAL_MultiProcessor_Init() to register callbacks for HAL_UART_MSPINIT_CB_ID and HAL_UART_MSPDEINIT_CB_ID

HAL_UART_UnRegisterCallback

Function name

**HAL_StatusTypeDef HAL_UART_UnRegisterCallback (UART_HandleTypeDef * huart,
HAL_UART_CallbackIDTypeDef CallbackID)**

Function description

Unregister an UART Callback UART callback is redirected to the weak predefined callback.

Parameters

- **huart:** uart handle
- **CallbackID:** ID of the callback to be unregistered This parameter can be one of the following values:
 - HAL_UART_TX_HALFCOMPLETE_CB_ID Tx Half Complete Callback ID
 - HAL_UART_TX_COMPLETE_CB_ID Tx Complete Callback ID
 - HAL_UART_RX_HALFCOMPLETE_CB_ID Rx Half Complete Callback ID
 - HAL_UART_RX_COMPLETE_CB_ID Rx Complete Callback ID
 - HAL_UART_ERROR_CB_ID Error Callback ID
 - HAL_UART_ABORT_COMPLETE_CB_ID Abort Complete Callback ID
 - HAL_UART_ABORT_TRANSMIT_COMPLETE_CB_ID Abort Transmit Complete Callback ID
 - HAL_UART_ABORT_RECEIVE_COMPLETE_CB_ID Abort Receive Complete Callback ID
 - HAL_UART_MSPINIT_CB_ID MspInit Callback ID
 - HAL_UART_MSPDEINIT_CB_ID MspDeInit Callback ID

Return values

- **HAL:** status

Notes

- The HAL_UART_UnRegisterCallback() may be called before HAL_UART_Init(), HAL_HalfDuplex_Init(), HAL_LIN_Init(), HAL_MultiProcessor_Init() to un-register callbacks for HAL_UART_MSPINIT_CB_ID and HAL_UART_MSPDEINIT_CB_ID

HAL_UART_RegisterRxEventCallback

Function name

HAL_StatusTypeDef HAL_UART_RegisterRxEventCallback (UART_HandleTypeDef * huart, pUART_RxEventCallbackTypeDef pCallback)

Function description

Register a User UART Rx Event Callback To be used instead of the weak predefined callback.

Parameters

- **huart:** Uart handle
- **pCallback:** Pointer to the Rx Event Callback function

Return values

- **HAL:** status

HAL_UART_UnRegisterRxEventCallback

Function name

HAL_StatusTypeDef HAL_UART_UnRegisterRxEventCallback (UART_HandleTypeDef * huart)

Function description

UnRegister the UART Rx Event Callback UART Rx Event Callback is redirected to the weak HAL_UARTEx_RxEventCallback() predefined callback.

Parameters

- **huart:** Uart handle

Return values

- **HAL:** status

HAL_UART_Transmit

Function name

HAL_StatusTypeDef HAL_UART_Transmit (UART_HandleTypeDef * huart, const uint8_t * pData, uint16_t Size, uint32_t Timeout)

Function description

Sends an amount of data in blocking mode.

Parameters

- **huart:** Pointer to a UART_HandleTypeDef structure that contains the configuration information for the specified UART module.
- **pData:** Pointer to data buffer (u8 or u16 data elements).
- **Size:** Amount of data elements (u8 or u16) to be sent
- **Timeout:** Timeout duration

Return values

- **HAL:** status

Notes

- When UART parity is not enabled (PCE = 0), and Word Length is configured to 9 bits (M1-M0 = 01), the sent data is handled as a set of u16. In this case, Size must indicate the number of u16 provided through pData.

HAL_UART_Receive

Function name

HAL_StatusTypeDef HAL_UART_Receive (UART_HandleTypeDef * huart, uint8_t * pData, uint16_t Size, uint32_t Timeout)

Function description

Receives an amount of data in blocking mode.

Parameters

- **huart**: Pointer to a UART_HandleTypeDef structure that contains the configuration information for the specified UART module.
- **pData**: Pointer to data buffer (u8 or u16 data elements).
- **Size**: Amount of data elements (u8 or u16) to be received.
- **Timeout**: Timeout duration

Return values

- **HAL**: status

Notes

- When UART parity is not enabled (PCE = 0), and Word Length is configured to 9 bits (M1-M0 = 01), the received data is handled as a set of u16. In this case, Size must indicate the number of u16 available through pData.

HAL_UART_Transmit_IT

Function name

HAL_StatusTypeDef HAL_UART_Transmit_IT (UART_HandleTypeDef * huart, const uint8_t * pData, uint16_t Size)

Function description

Sends an amount of data in non blocking mode.

Parameters

- **huart**: Pointer to a UART_HandleTypeDef structure that contains the configuration information for the specified UART module.
- **pData**: Pointer to data buffer (u8 or u16 data elements).
- **Size**: Amount of data elements (u8 or u16) to be sent

Return values

- **HAL**: status

Notes

- When UART parity is not enabled (PCE = 0), and Word Length is configured to 9 bits (M1-M0 = 01), the sent data is handled as a set of u16. In this case, Size must indicate the number of u16 provided through pData.

HAL_UART_Receive_IT

Function name

HAL_StatusTypeDef HAL_UART_Receive_IT (UART_HandleTypeDef * huart, uint8_t * pData, uint16_t Size)

Function description

Receives an amount of data in non blocking mode.

Parameters

- **huart:** Pointer to a UART_HandleTypeDef structure that contains the configuration information for the specified UART module.
- **pData:** Pointer to data buffer (u8 or u16 data elements).
- **Size:** Amount of data elements (u8 or u16) to be received.

Return values

- **HAL:** status

Notes

- When UART parity is not enabled (PCE = 0), and Word Length is configured to 9 bits (M1-M0 = 01), the received data is handled as a set of u16. In this case, Size must indicate the number of u16 available through pData.

HAL_UART_Transmit_DMA

Function name

HAL_StatusTypeDef HAL_UART_Transmit_DMA (UART_HandleTypeDef * huart, const uint8_t * pData, uint16_t Size)

Function description

Sends an amount of data in DMA mode.

Parameters

- **huart:** Pointer to a UART_HandleTypeDef structure that contains the configuration information for the specified UART module.
- **pData:** Pointer to data buffer (u8 or u16 data elements).
- **Size:** Amount of data elements (u8 or u16) to be sent

Return values

- **HAL:** status

Notes

- When UART parity is not enabled (PCE = 0), and Word Length is configured to 9 bits (M1-M0 = 01), the sent data is handled as a set of u16. In this case, Size must indicate the number of u16 provided through pData.

HAL_UART_Receive_DMA

Function name

HAL_StatusTypeDef HAL_UART_Receive_DMA (UART_HandleTypeDef * huart, uint8_t * pData, uint16_t Size)

Function description

Receives an amount of data in DMA mode.

Parameters

- **huart:** Pointer to a UART_HandleTypeDef structure that contains the configuration information for the specified UART module.
- **pData:** Pointer to data buffer (u8 or u16 data elements).
- **Size:** Amount of data elements (u8 or u16) to be received.

Return values

- **HAL:** status

Notes

- When UART parity is not enabled (PCE = 0), and Word Length is configured to 9 bits (M1-M0 = 01), the received data is handled as a set of u16. In this case, Size must indicate the number of u16 available through pData.
- When the UART parity is enabled (PCE = 1) the received data contains the parity bit.

HAL_UART_DMAPause

Function name

HAL_StatusTypeDef HAL_UART_DMAPause (UART_HandleTypeDef * huart)

Function description

Pauses the DMA Transfer.

Parameters

- **huart:** Pointer to a UART_HandleTypeDef structure that contains the configuration information for the specified UART module.

Return values

- **HAL:** status

HAL_UART_DMAResume

Function name

HAL_StatusTypeDef HAL_UART_DMAResume (UART_HandleTypeDef * huart)

Function description

Resumes the DMA Transfer.

Parameters

- **huart:** Pointer to a UART_HandleTypeDef structure that contains the configuration information for the specified UART module.

Return values

- **HAL:** status

HAL_UART_DMAStop

Function name

HAL_StatusTypeDef HAL_UART_DMAStop (UART_HandleTypeDef * huart)

Function description

Stops the DMA Transfer.

Parameters

- **huart:** Pointer to a UART_HandleTypeDef structure that contains the configuration information for the specified UART module.

Return values

- **HAL:** status

HAL_UARTEx_ReceiveTxdle

Function name

HAL_StatusTypeDef HAL_UARTEx_ReceiveTxdle (UART_HandleTypeDef * huart, uint8_t * pData, uint16_t Size, uint16_t * RxLen, uint32_t Timeout)

Function description

Receive an amount of data in blocking mode till either the expected number of data is received or an IDLE event occurs.

Parameters

- **huart:** UART handle.
- **pData:** Pointer to data buffer (uint8_t or uint16_t data elements).
- **Size:** Amount of data elements (uint8_t or uint16_t) to be received.
- **RxLen:** Number of data elements finally received (could be lower than Size, in case reception ends on IDLE event)
- **Timeout:** Timeout duration expressed in ms (covers the whole reception sequence).

Return values

- **HAL:** status

Notes

- HAL_OK is returned if reception is completed (expected number of data has been received) or if reception is stopped after IDLE event (less than the expected number of data has been received) In this case, RxLen output parameter indicates number of data available in reception buffer.
- When UART parity is not enabled (PCE = 0), and Word Length is configured to 9 bits (M = 01), the received data is handled as a set of uint16_t. In this case, Size must indicate the number of uint16_t available through pData.

HAL_UARTEx_ReceiveToldle_IT

Function name

HAL_StatusTypeDef HAL_UARTEx_ReceiveToldle_IT (UART_HandleTypeDef * huart, uint8_t * pData, uint16_t Size)

Function description

Receive an amount of data in interrupt mode till either the expected number of data is received or an IDLE event occurs.

Parameters

- **huart:** UART handle.
- **pData:** Pointer to data buffer (uint8_t or uint16_t data elements).
- **Size:** Amount of data elements (uint8_t or uint16_t) to be received.

Return values

- **HAL:** status

Notes

- Reception is initiated by this function call. Further progress of reception is achieved thanks to UART interrupts raised by RXNE and IDLE events. Callback is called at end of reception indicating number of received data elements.
- When UART parity is not enabled (PCE = 0), and Word Length is configured to 9 bits (M = 01), the received data is handled as a set of uint16_t. In this case, Size must indicate the number of uint16_t available through pData.

HAL_UARTEx_ReceiveToldle_DMA

Function name

HAL_StatusTypeDef HAL_UARTEx_ReceiveToldle_DMA (UART_HandleTypeDef * huart, uint8_t * pData, uint16_t Size)

Function description

Receive an amount of data in DMA mode till either the expected number of data is received or an IDLE event occurs.

Parameters

- **huart:** UART handle.
- **pData:** Pointer to data buffer (uint8_t or uint16_t data elements).
- **Size:** Amount of data elements (uint8_t or uint16_t) to be received.

Return values

- **HAL:** status

Notes

- Reception is initiated by this function call. Further progress of reception is achieved thanks to DMA services, transferring automatically received data elements in user reception buffer and calling registered callbacks at half/end of reception. UART IDLE events are also used to consider reception phase as ended. In all cases, callback execution will indicate number of received data elements.
- When the UART parity is enabled (PCE = 1), the received data contain the parity bit (MSB position).
- When UART parity is not enabled (PCE = 0), and Word Length is configured to 9 bits (M = 01), the received data is handled as a set of uint16_t. In this case, Size must indicate the number of uint16_t available through pData.

HAL_UARTEx_GetRxEventType

Function name

HAL_UART_RxEventTypeTypeDef HAL_UARTEx_GetRxEventType (UART_HandleTypeDef * huart)

Function description

Provide Rx Event type that has lead to RxEvent callback execution.

Parameters

- **huart:** UART handle.

Return values

- **Rx:** Event Type (returned value will be a value of UART RxEvent type values)

Notes

- When HAL_UARTEx_ReceiveToidle_IT() or HAL_UARTEx_ReceiveToidle_DMA() API are called, progress of reception process is provided to application through calls of Rx Event callback (either default one HAL_UARTEx_RxEventCallback() or user registered one). As several types of events could occur (IDLE event, Half Transfer, or Transfer Complete), this function allows to retrieve the Rx Event type that has lead to Rx Event callback execution.
- This function is expected to be called within the user implementation of Rx Event Callback, in order to provide the accurate value : In Interrupt Mode : HAL_UART_RXEVENT_TC : when Reception has been completed (expected nb of data has been received) HAL_UART_RXEVENT_IDLE : when Idle event occurred prior reception has been completed (nb of received data is lower than expected one) In DMA Mode : HAL_UART_RXEVENT_TC : when Reception has been completed (expected nb of data has been received) HAL_UART_RXEVENT_HT : when half of expected nb of data has been received HAL_UART_RXEVENT_IDLE : when Idle event occurred prior reception has been completed (nb of received data is lower than expected one). In DMA mode, RxEvent callback could be called several times; When DMA is configured in Normal Mode, HT event does not stop Reception process; When DMA is configured in Circular Mode, HT, TC or IDLE events don't stop Reception process;

HAL_UART_Abort

Function name

HAL_StatusTypeDef HAL_UART_Abort (UART_HandleTypeDef * huart)

Function description

Abort ongoing transfers (blocking mode).

Parameters

- **huart:** UART handle.

Return values

- **HAL:** status

Notes

- This procedure could be used for aborting any ongoing transfer started in Interrupt or DMA mode. This procedure performs following operations : Disable UART Interrupts (Tx and Rx)Disable the DMA transfer in the peripheral register (if enabled)Abort DMA transfer by calling HAL_DMA_Abort (in case of transfer in DMA mode)Set handle State to READY
- This procedure is executed in blocking mode : when exiting function, Abort is considered as completed.

HAL_UART_AbortTransmit

Function name

HAL_StatusTypeDef HAL_UART_AbortTransmit (UART_HandleTypeDef * huart)

Function description

Abort ongoing Transmit transfer (blocking mode).

Parameters

- **huart:** UART handle.

Return values

- **HAL:** status

Notes

- This procedure could be used for aborting any ongoing Tx transfer started in Interrupt or DMA mode. This procedure performs following operations : Disable UART Interrupts (Tx)Disable the DMA transfer in the peripheral register (if enabled)Abort DMA transfer by calling HAL_DMA_Abort (in case of transfer in DMA mode)Set handle State to READY
- This procedure is executed in blocking mode : when exiting function, Abort is considered as completed.

HAL_UART_AbortReceive

Function name

HAL_StatusTypeDef HAL_UART_AbortReceive (UART_HandleTypeDef * huart)

Function description

Abort ongoing Receive transfer (blocking mode).

Parameters

- **huart:** UART handle.

Return values

- **HAL:** status

Notes

- This procedure could be used for aborting any ongoing Rx transfer started in Interrupt or DMA mode. This procedure performs following operations : Disable UART Interrupts (Rx)Disable the DMA transfer in the peripheral register (if enabled)Abort DMA transfer by calling HAL_DMA_Abort (in case of transfer in DMA mode)Set handle State to READY
- This procedure is executed in blocking mode : when exiting function, Abort is considered as completed.

HAL_UART_Abort_IT

Function name

HAL_StatusTypeDef HAL_UART_Abort_IT (UART_HandleTypeDef * huart)

Function description

Abort ongoing transfers (Interrupt mode).

Parameters

- **huart:** UART handle.

Return values

- **HAL:** status

Notes

- This procedure could be used for aborting any ongoing transfer started in Interrupt or DMA mode. This procedure performs following operations : Disable UART Interrupts (Tx and Rx)Disable the DMA transfer in the peripheral register (if enabled)Abort DMA transfer by calling HAL_DMA_Abort_IT (in case of transfer in DMA mode)Set handle State to READYAt abort completion, call user abort complete callback
- This procedure is executed in Interrupt mode, meaning that abort procedure could be considered as completed only when user abort complete callback is executed (not when exiting function).

HAL_UART_AbortTransmit_IT

Function name

HAL_StatusTypeDef HAL_UART_AbortTransmit_IT (UART_HandleTypeDef * huart)

Function description

Abort ongoing Transmit transfer (Interrupt mode).

Parameters

- **huart:** UART handle.

Return values

- **HAL:** status

Notes

- This procedure could be used for aborting any ongoing Tx transfer started in Interrupt or DMA mode. This procedure performs following operations : Disable UART Interrupts (Tx)Disable the DMA transfer in the peripheral register (if enabled)Abort DMA transfer by calling HAL_DMA_Abort_IT (in case of transfer in DMA mode)Set handle State to READYAt abort completion, call user abort complete callback
- This procedure is executed in Interrupt mode, meaning that abort procedure could be considered as completed only when user abort complete callback is executed (not when exiting function).

HAL_UART_AbortReceive_IT

Function name

HAL_StatusTypeDef HAL_UART_AbortReceive_IT (UART_HandleTypeDef * huart)

Function description

Abort ongoing Receive transfer (Interrupt mode).

Parameters

- **huart:** UART handle.

Return values

- **HAL:** status

Notes

- This procedure could be used for aborting any ongoing Rx transfer started in Interrupt or DMA mode. This procedure performs following operations : Disable UART Interrupts (Rx)Disable the DMA transfer in the peripheral register (if enabled)Abort DMA transfer by calling HAL_DMA_Abort_IT (in case of transfer in DMA mode)Set handle State to READYAt abort completion, call user abort complete callback
- This procedure is executed in Interrupt mode, meaning that abort procedure could be considered as completed only when user abort complete callback is executed (not when exiting function).

HAL_UART_IRQHandler

Function name

```
void HAL_UART_IRQHandler (UART_HandleTypeDef * huart)
```

Function description

This function handles UART interrupt request.

Parameters

- **huart:** Pointer to a UART_HandleTypeDef structure that contains the configuration information for the specified UART module.

Return values

- **None:**

HAL_UART_TxCpltCallback

Function name

```
void HAL_UART_TxCpltCallback (UART_HandleTypeDef * huart)
```

Function description

Tx Transfer completed callbacks.

Parameters

- **huart:** Pointer to a UART_HandleTypeDef structure that contains the configuration information for the specified UART module.

Return values

- **None:**

HAL_UART_TxHalfCpltCallback

Function name

```
void HAL_UART_TxHalfCpltCallback (UART_HandleTypeDef * huart)
```

Function description

Tx Half Transfer completed callbacks.

Parameters

- **huart:** Pointer to a UART_HandleTypeDef structure that contains the configuration information for the specified UART module.

Return values

- **None:**

HAL_UART_RxCpltCallback

Function name

```
void HAL_UART_RxCpltCallback (UART_HandleTypeDef * huart)
```

Function description

Rx Transfer completed callbacks.

Parameters

- **huart:** Pointer to a UART_HandleTypeDef structure that contains the configuration information for the specified UART module.

Return values

- **None:**

HAL_UART_RxHalfCpltCallback

Function name

void HAL_UART_RxHalfCpltCallback (UART_HandleTypeDef * huart)

Function description

Rx Half Transfer completed callbacks.

Parameters

- **huart:** Pointer to a UART_HandleTypeDef structure that contains the configuration information for the specified UART module.

Return values

- **None:**

HAL_UART_ErrorCallback

Function name

void HAL_UART_ErrorCallback (UART_HandleTypeDef * huart)

Function description

UART error callbacks.

Parameters

- **huart:** Pointer to a UART_HandleTypeDef structure that contains the configuration information for the specified UART module.

Return values

- **None:**

HAL_UART_AbortCpltCallback

Function name

void HAL_UART_AbortCpltCallback (UART_HandleTypeDef * huart)

Function description

UART Abort Complete callback.

Parameters

- **huart:** UART handle.

Return values

- **None:**

HAL_UART_AbortTransmitCpltCallback

Function name

void HAL_UART_AbortTransmitCpltCallback (UART_HandleTypeDef * huart)

Function description

UART Abort Complete callback.

Parameters

- **huart:** UART handle.

Return values

- **None:**

HAL_UART_AbortReceiveCpltCallback

Function name

void HAL_UART_AbortReceiveCpltCallback (UART_HandleTypeDef * huart)

Function description

UART Abort Receive Complete callback.

Parameters

- **huart:** UART handle.

Return values

- **None:**

HAL_UARTEx_RxEventCallback

Function name

void HAL_UARTEx_RxEventCallback (UART_HandleTypeDef * huart, uint16_t Size)

Function description

Reception Event Callback (Rx event notification called after use of advanced reception service).

Parameters

- **huart:** UART handle
- **Size:** Number of data available in application reception buffer (indicates a position in reception buffer until which, data are available)

Return values

- **None:**

HAL_LIN_SendBreak

Function name

HAL_StatusTypeDef HAL_LIN_SendBreak (UART_HandleTypeDef * huart)

Function description

Transmits break characters.

Parameters

- **huart:** Pointer to a UART_HandleTypeDef structure that contains the configuration information for the specified UART module.

Return values

- **HAL:** status

HAL_MultiProcessor_EnterMuteMode

Function name

HAL_StatusTypeDef HAL_MultiProcessor_EnterMuteMode (UART_HandleTypeDef * huart)

Function description

Enters the UART in mute mode.

Parameters

- **huart:** Pointer to a UART_HandleTypeDef structure that contains the configuration information for the specified UART module.

Return values

- **HAL:** status

HAL_MultiProcessor_ExitMuteMode

Function name

HAL_StatusTypeDef HAL_MultiProcessor_ExitMuteMode (UART_HandleTypeDef * huart)

Function description

Exits the UART mute mode: wake up software.

Parameters

- **huart:** Pointer to a UART_HandleTypeDef structure that contains the configuration information for the specified UART module.

Return values

- **HAL:** status

HAL_HalfDuplex_EnableTransmitter

Function name

HAL_StatusTypeDef HAL_HalfDuplex_EnableTransmitter (UART_HandleTypeDef * huart)

Function description

Enables the UART transmitter and disables the UART receiver.

Parameters

- **huart:** Pointer to a UART_HandleTypeDef structure that contains the configuration information for the specified UART module.

Return values

- **HAL:** status

HAL_HalfDuplex_EnableReceiver

Function name

HAL_StatusTypeDef HAL_HalfDuplex_EnableReceiver (UART_HandleTypeDef * huart)

Function description

Enables the UART receiver and disables the UART transmitter.

Parameters

- **huart:** Pointer to a UART_HandleTypeDef structure that contains the configuration information for the specified UART module.

Return values

- **HAL:** status

HAL_UART_GetState

Function name

HAL_UART_StateTypeDef HAL_UART_GetState (const UART_HandleTypeDef * huart)

Function description

Returns the UART state.

Parameters

- **huart:** Pointer to a UART_HandleTypeDef structure that contains the configuration information for the specified UART module.

Return values

- **HAL:** state

HAL_UART_GetError

Function name

uint32_t HAL_UART_GetError (const UART_HandleTypeDef * huart)

Function description

Return the UART error code.

Parameters

- **huart:** Pointer to a UART_HandleTypeDef structure that contains the configuration information for the specified UART.

Return values

- **UART:** Error Code

UART_Start_Receive_IT

Function name

HAL_StatusTypeDef UART_Start_Receive_IT (UART_HandleTypeDef * huart, uint8_t * pData, uint16_t Size)

Function description

Start Receive operation in interrupt mode.

Parameters

- **huart:** UART handle.
- **pData:** Pointer to data buffer (u8 or u16 data elements).
- **Size:** Amount of data elements (u8 or u16) to be received.

Return values

- **HAL:** status

Notes

- This function could be called by all HAL UART API providing reception in Interrupt mode.
- When calling this function, parameters validity is considered as already checked, i.e. Rx State, buffer address, ... UART Handle is assumed as Locked.

UART_Start_Receive_DMA

Function name

HAL_StatusTypeDef UART_Start_Receive_DMA (UART_HandleTypeDef * huart, uint8_t * pData, uint16_t Size)

Function description

Start Receive operation in DMA mode.

Parameters

- **huart:** UART handle.
- **pData:** Pointer to data buffer (u8 or u16 data elements).
- **Size:** Amount of data elements (u8 or u16) to be received.

Return values

- **HAL:** status

Notes

- This function could be called by all HAL UART API providing reception in DMA mode.
- When calling this function, parameters validity is considered as already checked, i.e. Rx State, buffer address, ... UART Handle is assumed as Locked.

70.3 UART Firmware driver defines

The following section lists the various define and macros of the module.

70.3.1 UART

UART

UART Error Code

HAL_UART_ERROR_NONE

No error

HAL_UART_ERROR_PE

Parity error

HAL_UART_ERROR_NE

Noise error

HAL_UART_ERROR_FE

Frame error

HAL_UART_ERROR_ORE

Overrun error

HAL_UART_ERROR_DMA

DMA transfer error

HAL_UART_ERROR_INVALID_CALLBACK

Invalid Callback error

UART Exported Macros

__HAL_UART_RESET_HANDLE_STATE

Description:

- Reset UART handle gstate & RxState.

Parameters:

- **__HANDLE__:** specifies the UART Handle. UART Handle selects the USARTx or UARTy peripheral (USART, UART availability and x,y values depending on device).

Return value:

- None

__HAL_UART_FLUSH_DRREGISTER

Description:

- Flushes the UART DR register.

Parameters:

- `__HANDLE__`: specifies the UART Handle. UART Handle selects the USARTx or UARTy peripheral (USART, UART availability and x,y values depending on device).

__HAL_UART_GET_FLAG

Description:

- Checks whether the specified UART flag is set or not.

Parameters:

- `__HANDLE__`: specifies the UART Handle. UART Handle selects the USARTx or UARTy peripheral (USART, UART availability and x,y values depending on device).
- `__FLAG__`: specifies the flag to check. This parameter can be one of the following values:
 - `UART_FLAG_CTS`: CTS Change flag (not available for UART4 and UART5)
 - `UART_FLAG_LBD`: LIN Break detection flag
 - `UART_FLAG_TXE`: Transmit data register empty flag
 - `UART_FLAG_TC`: Transmission Complete flag
 - `UART_FLAG_RXNE`: Receive data register not empty flag
 - `UART_FLAG_IDLE`: Idle Line detection flag
 - `UART_FLAG_ORE`: Overrun Error flag
 - `UART_FLAG_NE`: Noise Error flag
 - `UART_FLAG_FE`: Framing Error flag
 - `UART_FLAG_PE`: Parity Error flag

Return value:

- The: new state of `__FLAG__` (TRUE or FALSE).

__HAL_UART_CLEAR_FLAG

Description:

- Clears the specified UART pending flag.

Parameters:

- `__HANDLE__`: specifies the UART Handle. UART Handle selects the USARTx or UARTy peripheral (USART, UART availability and x,y values depending on device).
- `__FLAG__`: specifies the flag to check. This parameter can be any combination of the following values:
 - `UART_FLAG_CTS`: CTS Change flag (not available for UART4 and UART5).
 - `UART_FLAG_LBD`: LIN Break detection flag.
 - `UART_FLAG_TC`: Transmission Complete flag.
 - `UART_FLAG_RXNE`: Receive data register not empty flag.

Return value:

- None

Notes:

- PE (Parity error), FE (Framing error), NE (Noise error), ORE (Overrun error) and IDLE (Idle line detected) flags are cleared by software sequence: a read operation to USART_SR register followed by a read operation to USART_DR register. RXNE flag can be also cleared by a read to the USART_DR register. TC flag can be also cleared by software sequence: a read operation to USART_SR register followed by a write operation to USART_DR register. TXE flag is cleared only by a write to the USART_DR register.

__HAL_UART_CLEAR_PFLAG

Description:

- Clears the UART PE pending flag.

Parameters:

- `__HANDLE__`: specifies the UART Handle. UART Handle selects the USARTx or UARTy peripheral (USART, UART availability and x,y values depending on device).

Return value:

- None

__HAL_UART_CLEAR_FFLAG

Description:

- Clears the UART FE pending flag.

Parameters:

- `__HANDLE__`: specifies the UART Handle. UART Handle selects the USARTx or UARTy peripheral (USART, UART availability and x,y values depending on device).

Return value:

- None

__HAL_UART_CLEAR_NFLAG

Description:

- Clears the UART NE pending flag.

Parameters:

- `__HANDLE__`: specifies the UART Handle. UART Handle selects the USARTx or UARTy peripheral (USART, UART availability and x,y values depending on device).

Return value:

- None

__HAL_UART_CLEAR_OREFLAG

Description:

- Clears the UART ORE pending flag.

Parameters:

- `__HANDLE__`: specifies the UART Handle. UART Handle selects the USARTx or UARTy peripheral (USART, UART availability and x,y values depending on device).

Return value:

- None

__HAL_UART_CLEAR_IDLEFLAG

Description:

- Clears the UART IDLE pending flag.

Parameters:

- `__HANDLE__`: specifies the UART Handle. UART Handle selects the USARTx or UARTy peripheral (USART, UART availability and x,y values depending on device).

Return value:

- None

__HAL_UART_ENABLE_IT

Description:

- Enable the specified UART interrupt.

Parameters:

- `__HANDLE__`: specifies the UART Handle. UART Handle selects the USARTx or UARTy peripheral (USART, UART availability and x,y values depending on device).
- `__INTERRUPT__`: specifies the UART interrupt source to enable. This parameter can be one of the following values:
 - `UART_IT_CTS`: CTS change interrupt
 - `UART_IT_LBD`: LIN Break detection interrupt
 - `UART_IT_TXE`: Transmit Data Register empty interrupt
 - `UART_IT_TC`: Transmission complete interrupt
 - `UART_IT_RXNE`: Receive Data register not empty interrupt
 - `UART_IT_IDLE`: Idle line detection interrupt
 - `UART_IT_PE`: Parity Error interrupt
 - `UART_IT_ERR`: Error interrupt(Frame error, noise error, overrun error)

Return value:

- None

__HAL_UART_DISABLE_IT

Description:

- Disable the specified UART interrupt.

Parameters:

- `__HANDLE__`: specifies the UART Handle. UART Handle selects the USARTx or UARTy peripheral (USART, UART availability and x,y values depending on device).
- `__INTERRUPT__`: specifies the UART interrupt source to disable. This parameter can be one of the following values:
 - `UART_IT_CTS`: CTS change interrupt
 - `UART_IT_LBD`: LIN Break detection interrupt
 - `UART_IT_TXE`: Transmit Data Register empty interrupt
 - `UART_IT_TC`: Transmission complete interrupt
 - `UART_IT_RXNE`: Receive Data register not empty interrupt
 - `UART_IT_IDLE`: Idle line detection interrupt
 - `UART_IT_PE`: Parity Error interrupt
 - `UART_IT_ERR`: Error interrupt(Frame error, noise error, overrun error)

Return value:

- None

__HAL_UART_GET_IT_SOURCE

Description:

- Checks whether the specified UART interrupt source is enabled or not.

Parameters:

- `__HANDLE__`: specifies the UART Handle. UART Handle selects the USARTx or UARTy peripheral (USART, UART availability and x,y values depending on device).
- `__IT__`: specifies the UART interrupt source to check. This parameter can be one of the following values:
 - `UART_IT_CTS`: CTS change interrupt (not available for UART4 and UART5)
 - `UART_IT_LBD`: LIN Break detection interrupt
 - `UART_IT_TXE`: Transmit Data Register empty interrupt
 - `UART_IT_TC`: Transmission complete interrupt
 - `UART_IT_RXNE`: Receive Data register not empty interrupt
 - `UART_IT_IDLE`: Idle line detection interrupt
 - `UART_IT_ERR`: Error interrupt

Return value:

- The: new state of `__IT__` (TRUE or FALSE).

__HAL_UART_HWCONTROL_CTS_ENABLE

Description:

- Enable CTS flow control.

Parameters:

- `__HANDLE__`: specifies the UART Handle. The Handle Instance can be any USARTx (supporting the HW Flow control feature). It is used to select the USART peripheral (USART availability and x value depending on device).

Return value:

- None

Notes:

- This macro allows to enable CTS hardware flow control for a given UART instance, without need to call `HAL_UART_Init()` function. As involving direct access to UART registers, usage of this macro should be fully endorsed by user. As macro is expected to be used for modifying CTS Hw flow control feature activation, without need for USART instance Deinit/Init, following conditions for macro call should be fulfilled : UART instance should have already been initialised (through call of `HAL_UART_Init()`) macro could only be called when corresponding UART instance is disabled (i.e `__HAL_UART_DISABLE(__HANDLE__)`) and should be followed by an Enable macro (i.e `__HAL_UART_ENABLE(__HANDLE__)`).

__HAL_UART_HWCONTROL_CTS_DISABLE

Description:

- Disable CTS flow control.

Parameters:

- `__HANDLE__`: specifies the UART Handle. The Handle Instance can be any USARTx (supporting the HW Flow control feature). It is used to select the USART peripheral (USART availability and x value depending on device).

Return value:

- None

Notes:

- This macro allows to disable CTS hardware flow control for a given UART instance, without need to call `HAL_UART_Init()` function. As involving direct access to UART registers, usage of this macro should be fully endorsed by user. As macro is expected to be used for modifying CTS Hw flow control feature activation, without need for USART instance Deinit/Init, following conditions for macro call should be fulfilled : UART instance should have already been initialised (through call of `HAL_UART_Init()`) macro could only be called when corresponding UART instance is disabled (i.e `__HAL_UART_DISABLE(__HANDLE__)`) and should be followed by an Enable macro (i.e `__HAL_UART_ENABLE(__HANDLE__)`).

__HAL_UART_HWCONTROL_RTS_ENABLE

Description:

- Enable RTS flow control This macro allows to enable RTS hardware flow control for a given UART instance, without need to call

Parameters:

- `__HANDLE__`: specifies the UART Handle. The Handle Instance can be any USARTx (supporting the HW Flow control feature). It is used to select the USART peripheral (USART availability and x value depending on device).

Return value:

- None

Notes:

- As macro is expected to be used for modifying RTS Hw flow control feature activation, without need for USART instance Deinit/Init, following conditions for macro call should be fulfilled : UART instance should have already been initialised (through call of `HAL_UART_Init()`) macro could only be called when corresponding UART instance is disabled (i.e `__HAL_UART_DISABLE(__HANDLE__)`) and should be followed by an Enable macro (i.e `__HAL_UART_ENABLE(__HANDLE__)`).

__HAL_UART_HWCONTROL_RTS_DISABLE

Description:

- Disable RTS flow control This macro allows to disable RTS hardware flow control for a given UART instance, without need to call

Parameters:

- `__HANDLE__`: specifies the UART Handle. The Handle Instance can be any USARTx (supporting the HW Flow control feature). It is used to select the USART peripheral (USART availability and x value depending on device).

Return value:

- None

Notes:

- As macro is expected to be used for modifying RTS Hw flow control feature activation, without need for USART instance Deinit/Init, following conditions for macro call should be fulfilled : UART instance should have already been initialised (through call of `HAL_UART_Init()`) macro could only be called when corresponding UART instance is disabled (i.e `__HAL_UART_DISABLE(__HANDLE__)`) and should be followed by an Enable macro (i.e `__HAL_UART_ENABLE(__HANDLE__)`).

__HAL_UART_ONE_BIT_SAMPLE_ENABLE**Description:**

- Macro to enable the UART's one bit sample method.

Parameters:

- `__HANDLE__`: specifies the UART Handle.

Return value:

- None

__HAL_UART_ONE_BIT_SAMPLE_DISABLE**Description:**

- Macro to disable the UART's one bit sample method.

Parameters:

- `__HANDLE__`: specifies the UART Handle.

Return value:

- None

__HAL_UART_ENABLE**Description:**

- Enable UART.

Parameters:

- `__HANDLE__`: specifies the UART Handle.

Return value:

- None

__HAL_UART_DISABLE**Description:**

- Disable UART.

Parameters:

- `__HANDLE__`: specifies the UART Handle.

Return value:

- None

UART Flags**UART_FLAG_CTS****UART_FLAG_LBD****UART_FLAG_TXE****UART_FLAG_TC****UART_FLAG_RXNE****UART_FLAG_IDLE****UART_FLAG_ORE****UART_FLAG_NE****UART_FLAG_FE****UART_FLAG_PE**

UART Hardware Flow Control

UART_HWCONTROL_NONE

UART_HWCONTROL_RTS

UART_HWCONTROL_CTS

UART_HWCONTROL_RTS_CTS

UART Interrupt Definitions

UART_IT_PE

UART_IT_TXE

UART_IT_TC

UART_IT_RXNE

UART_IT_IDLE

UART_IT_LBD

UART_IT_CTS

UART_IT_ERR

UART LIN Break Detection Length

UART_LINBREAKDETECTLENGTH_10B

UART_LINBREAKDETECTLENGTH_11B

UART Transfer Mode

UART_MODE_RX

UART_MODE_TX

UART_MODE_TX_RX

UART Over Sampling

UART_OVERSAMPLING_16

UART_OVERSAMPLING_8

UART Parity

UART_PARITY_NONE

UART_PARITY_EVEN

UART_PARITY_ODD

UART Reception type values

HAL_UART_RECEPTION_STANDARD

Standard reception

HAL_UART_RECEPTION_TOIDLE

Reception till completion or IDLE event

UART RxEvent type values**HAL_UART_RXEVENT_TC**

RxEvent linked to Transfer Complete event

HAL_UART_RXEVENT_HT

RxEvent linked to Half Transfer event

HAL_UART_RXEVENT_IDLE***UART State*****UART_STATE_DISABLE****UART_STATE_ENABLE*****UART Number of Stop Bits*****UART_STOPBITS_1****UART_STOPBITS_2*****UART Wakeup Functions*****UART_WAKEUPMETHOD_IDLELINE****UART_WAKEUPMETHOD_ADDRESSMARK*****UART Word Length*****UART_WORDLENGTH_8B****UART_WORDLENGTH_9B**

71 HAL USART Generic Driver

71.1 USART Firmware driver registers structures

71.1.1 USART_InitTypeDef

USART_InitTypeDef is defined in the stm32f4xx_hal_usart.h

Data Fields

- **uint32_t BaudRate**
- **uint32_t WordLength**
- **uint32_t StopBits**
- **uint32_t Parity**
- **uint32_t Mode**
- **uint32_t CLKPolarity**
- **uint32_t CLKPhase**
- **uint32_t CLKLastBit**

Field Documentation

- **uint32_t USART_InitTypeDef::BaudRate**
This member configures the Usart communication baud rate. The baud rate is computed using the following formula:
 - IntegerDivider = ((PCLKx) / (8 * (husart->Init.BaudRate)))
 - FractionalDivider = ((IntegerDivider - ((uint32_t) IntegerDivider)) * 8) + 0.5
- **uint32_t USART_InitTypeDef::WordLength**
Specifies the number of data bits transmitted or received in a frame. This parameter can be a value of [USART_Word_Length](#)
- **uint32_t USART_InitTypeDef::StopBits**
Specifies the number of stop bits transmitted. This parameter can be a value of [USART_Stop_Bits](#)
- **uint32_t USART_InitTypeDef::Parity**
Specifies the parity mode. This parameter can be a value of [USART_Parity](#)
Note:
 - When parity is enabled, the computed parity is inserted at the MSB position of the transmitted data (9th bit when the word length is set to 9 data bits; 8th bit when the word length is set to 8 data bits).
- **uint32_t USART_InitTypeDef::Mode**
Specifies whether the Receive or Transmit mode is enabled or disabled. This parameter can be a value of [USART_Mode](#)
- **uint32_t USART_InitTypeDef::CLKPolarity**
Specifies the steady state of the serial clock. This parameter can be a value of [USART_Clock_Polarity](#)
- **uint32_t USART_InitTypeDef::CLKPhase**
Specifies the clock transition on which the bit capture is made. This parameter can be a value of [USART_Clock_Phase](#)
- **uint32_t USART_InitTypeDef::CLKLastBit**
Specifies whether the clock pulse corresponding to the last transmitted data bit (MSB) has to be output on the SCLK pin in synchronous mode. This parameter can be a value of [USART_Last_Bit](#)

71.1.2 __USART_HandleTypeDef

__USART_HandleTypeDef is defined in the stm32f4xx_hal_usart.h

Data Fields

- **USART_TypeDef * Instance**
- **USART_InitTypeDef Init**
- **const uint8_t * pTxBuffPtr**
- **uint16_t TxXferSize**
- **__IO uint16_t TxXferCount**

- **uint8_t * pRxBuffPtr**
- **uint16_t RxXferSize**
- **__IO uint16_t RxXferCount**
- **DMA_HandleTypeDef * hdmatx**
- **DMA_HandleTypeDef * hdmrx**
- **HAL_LockTypeDef Lock**
- **__IO HAL_USART_StateTypeDef State**
- **__IO uint32_t ErrorCode**
- **void(* TxHalfCpltCallback**
- **void(* TxCpltCallback**
- **void(* RxHalfCpltCallback**
- **void(* RxCpltCallback**
- **void(* TxRxCpltCallback**
- **void(* ErrorCallback**
- **void(* AbortCpltCallback**
- **void(* MspInitCallback**
- **void(* MspDeInitCallback**

Field Documentation

- **USART_TypeDef* __USART_HandleTypeDef::Instance**
USART registers base address
- **USART_InitTypeDef __USART_HandleTypeDef::Init**
Usart communication parameters
- **const uint8_t* __USART_HandleTypeDef::pTxBuffPtr**
Pointer to Usart Tx transfer Buffer
- **uint16_t __USART_HandleTypeDef::TxXferSize**
Usart Tx Transfer size
- **__IO uint16_t __USART_HandleTypeDef::TxXferCount**
Usart Tx Transfer Counter
- **uint8_t* __USART_HandleTypeDef::pRxBuffPtr**
Pointer to Usart Rx transfer Buffer
- **uint16_t __USART_HandleTypeDef::RxXferSize**
Usart Rx Transfer size
- **__IO uint16_t __USART_HandleTypeDef::RxXferCount**
Usart Rx Transfer Counter
- **DMA_HandleTypeDef* __USART_HandleTypeDef::hdmatx**
Usart Tx DMA Handle parameters
- **DMA_HandleTypeDef* __USART_HandleTypeDef::hdmrx**
Usart Rx DMA Handle parameters
- **HAL_LockTypeDef __USART_HandleTypeDef::Lock**
Locking object
- **__IO HAL_USART_StateTypeDef __USART_HandleTypeDef::State**
Usart communication state
- **__IO uint32_t __USART_HandleTypeDef::ErrorCode**
USART Error code
- **void(* __USART_HandleTypeDef::TxHalfCpltCallback)(struct __USART_HandleTypeDef *husart)**
USART Tx Half Complete Callback
- **void(* __USART_HandleTypeDef::TxCpltCallback)(struct __USART_HandleTypeDef *husart)**
USART Tx Complete Callback
- **void(* __USART_HandleTypeDef::RxHalfCpltCallback)(struct __USART_HandleTypeDef *husart)**
USART Rx Half Complete Callback
- **void(* __USART_HandleTypeDef::RxCpltCallback)(struct __USART_HandleTypeDef *husart)**
USART Rx Complete Callback
- **void(* __USART_HandleTypeDef::TxRxCpltCallback)(struct __USART_HandleTypeDef *husart)**
USART Tx Rx Complete Callback

- `void(* __USART_HandleTypeDef::ErrorCallback)(struct __USART_HandleTypeDef *husart)`
USART Error Callback
- `void(* __USART_HandleTypeDef::AbortCpltCallback)(struct __USART_HandleTypeDef *husart)`
USART Abort Complete Callback
- `void(* __USART_HandleTypeDef::MspInitCallback)(struct __USART_HandleTypeDef *husart)`
USART Msp Init callback
- `void(* __USART_HandleTypeDef::MspDeInitCallback)(struct __USART_HandleTypeDef *husart)`
USART Msp DeInit callback

71.2 USART Firmware driver API description

The following section lists the various functions of the USART library.

71.2.1 How to use this driver

The USART HAL driver can be used as follows:

1. Declare a USART_HandleTypeDef handle structure (eg. USART_HandleTypeDef husart).
 2. Initialize the USART low level resources by implementing the HAL_USART_MspInit() API:
 - a. Enable the USARTx interface clock.
 - b. USART pins configuration:
 - Enable the clock for the USART GPIOs.
 - Configure the USART pins as alternate function pull-up.
 - c. NVIC configuration if you need to use interrupt process (HAL_USART_Transmit_IT(), HAL_USART_Receive_IT() and HAL_USART_TransmitReceive_IT() APIs):
 - Configure the USARTx interrupt priority.
 - Enable the NVIC USART IRQ handle.
 - d. DMA Configuration if you need to use DMA process (HAL_USART_Transmit_DMA(), HAL_USART_Receive_DMA() and HAL_USART_TransmitReceive_DMA() APIs):
 - Declare a DMA handle structure for the Tx/Rx stream.
 - Enable the DMAx interface clock.
 - Configure the declared DMA handle structure with the required Tx/Rx parameters.
 - Configure the DMA Tx/Rx stream.
 - Associate the initialized DMA handle to the USART DMA Tx/Rx handle.
 - Configure the priority and enable the NVIC for the transfer complete interrupt on the DMA Tx/Rx stream.
 - Configure the USARTx interrupt priority and enable the NVIC USART IRQ handle (used for last byte sending completion detection in DMA non circular mode)
 3. Program the Baud Rate, Word Length, Stop Bit, Parity, Hardware flow control and Mode(Receiver/Transmitter) in the husart Init structure.
 4. Initialize the USART registers by calling the HAL_USART_Init() API:
 - These APIs configures also the low level Hardware GPIO, CLOCK, CORTEX...etc) by calling the customized HAL_USART_MspInit(&husart) API.
- Note:** *The specific USART interrupts (Transmission complete interrupt, RXNE interrupt and Error Interrupts) will be managed using the macros __HAL_USART_ENABLE_IT() and __HAL_USART_DISABLE_IT() inside the transmit and receive process.*
5. Three operation modes are available within this driver :

Polling mode IO operation

- Send an amount of data in blocking mode using HAL_USART_Transmit()
- Receive an amount of data in blocking mode using HAL_USART_Receive()

Interrupt mode IO operation

- Send an amount of data in non blocking mode using HAL_USART_Transmit_IT()

- At transmission end of transfer HAL_USART_TxHalfCpltCallback is executed and user can add his own code by customization of function pointer HAL_USART_TxCpltCallback
- Receive an amount of data in non blocking mode using HAL_USART_Receive_IT()
- At reception end of transfer HAL_USART_RxCpltCallback is executed and user can add his own code by customization of function pointer HAL_USART_RxCpltCallback
- In case of transfer Error, HAL_USART_ErrorCallback() function is executed and user can add his own code by customization of function pointer HAL_USART_ErrorCallback

DMA mode IO operation

- Send an amount of data in non blocking mode (DMA) using HAL_USART_Transmit_DMA()
- At transmission end of half transfer HAL_USART_TxHalfCpltCallback is executed and user can add his own code by customization of function pointer HAL_USART_TxHalfCpltCallback
- At transmission end of transfer HAL_USART_TxCpltCallback is executed and user can add his own code by customization of function pointer HAL_USART_TxCpltCallback
- Receive an amount of data in non blocking mode (DMA) using HAL_USART_Receive_DMA()
- At reception end of half transfer HAL_USART_RxHalfCpltCallback is executed and user can add his own code by customization of function pointer HAL_USART_RxHalfCpltCallback
- At reception end of transfer HAL_USART_RxCpltCallback is executed and user can add his own code by customization of function pointer HAL_USART_RxCpltCallback
- In case of transfer Error, HAL_USART_ErrorCallback() function is executed and user can add his own code by customization of function pointer HAL_USART_ErrorCallback
- Pause the DMA Transfer using HAL_USART_DMABase()
- Resume the DMA Transfer using HAL_USART_DMAResume()
- Stop the DMA Transfer using HAL_USART_DMAStop()

USART HAL driver macros list

Below the list of most used macros in USART HAL driver.

- `__HAL_USART_ENABLE`: Enable the USART peripheral
- `__HAL_USART_DISABLE`: Disable the USART peripheral
- `__HAL_USART_GET_FLAG`: Check whether the specified USART flag is set or not
- `__HAL_USART_CLEAR_FLAG`: Clear the specified USART pending flag
- `__HAL_USART_ENABLE_IT`: Enable the specified USART interrupt
- `__HAL_USART_DISABLE_IT`: Disable the specified USART interrupt

Note: You can refer to the USART HAL driver header file for more useful macros

71.2.2 Callback registration

The compilation define `USE_HAL_USART_REGISTER_CALLBACKS` when set to 1 allows the user to configure dynamically the driver callbacks.

Use Function `@ref HAL_USART_RegisterCallback()` to register a user callback. Function `@ref HAL_USART_RegisterCallback()` allows to register following callbacks:

- `TxHalfCpltCallback`: Tx Half Complete Callback.
- `TxCpltCallback`: Tx Complete Callback.
- `RxHalfCpltCallback`: Rx Half Complete Callback.
- `RxCpltCallback`: Rx Complete Callback.
- `TxRxCpltCallback`: Tx Rx Complete Callback.
- `ErrorCallback`: Error Callback.
- `AbortCpltCallback`: Abort Complete Callback.
- `MspInitCallback`: USART MspInit.
- `MspDeInitCallback`: USART MspDeInit. This function takes as parameters the HAL peripheral handle, the Callback ID and a pointer to the user callback function.

Use function @ref HAL_USART_UnRegisterCallback() to reset a callback to the default weak (surcharged) function. @ref HAL_USART_UnRegisterCallback() takes as parameters the HAL peripheral handle, and the Callback ID. This function allows to reset following callbacks:

- TxHalfCpltCallback : Tx Half Complete Callback.
- TxCpltCallback : Tx Complete Callback.
- RxHalfCpltCallback : Rx Half Complete Callback.
- RxCpltCallback : Rx Complete Callback.
- TxBxCpltCallback : Tx Rx Complete Callback.
- ErrorCallback : Error Callback.
- AbortCpltCallback : Abort Complete Callback.
- MspInitCallback : USART MspInit.
- MspDeInitCallback : USART MspDeInit.

By default, after the @ref HAL_USART_Init() and when the state is HAL_USART_STATE_RESET all callbacks are set to the corresponding weak (surcharged) functions: examples @ref HAL_USART_TxCpltCallback(), @ref HAL_USART_RxHalfCpltCallback(). Exception done for MspInit and MspDeInit functions that are respectively reset to the legacy weak (surcharged) functions in the @ref HAL_USART_Init() and @ref HAL_USART_DeInit() only when these callbacks are null (not registered beforehand). If not, MspInit or MspDeInit are not null, the @ref HAL_USART_Init() and @ref HAL_USART_DeInit() keep and use the user MspInit/MspDeInit callbacks (registered beforehand).

Callbacks can be registered/unregistered in HAL_USART_STATE_READY state only. Exception done MspInit/ MspDeInit that can be registered/unregistered in HAL_USART_STATE_READY or HAL_USART_STATE_RESET state, thus registered (user) MspInit/DeInit callbacks can be used during the Init/DeInit. In that case first register the MspInit/MspDeInit user callbacks using @ref HAL_USART_RegisterCallback() before calling @ref HAL_USART_DeInit() or @ref HAL_USART_Init() function.

When The compilation define USE_HAL_USART_REGISTER_CALLBACKS is set to 0 or not defined, the callback registration feature is not available and weak (surcharged) callbacks are used.

71.2.3 Initialization and Configuration functions

This subsection provides a set of functions allowing to initialize the USART in asynchronous and in synchronous modes.

- For the asynchronous mode only these parameters can be configured:
 - Baud Rate
 - Word Length
 - Stop Bit
 - Parity: If the parity is enabled, then the MSB bit of the data written in the data register is transmitted but is changed by the parity bit. Depending on the frame length defined by the M bit (8-bits or 9-bits), please refer to Reference manual for possible USART frame formats.
 - USART polarity
 - USART phase
 - USART LastBit
 - Receiver/transmitter modes

The HAL_USART_Init() function follows the USART synchronous configuration procedures (details for the procedures are available in reference manual (RM0430 for STM32F4X3xx MCUs and RM0402 for STM32F412xx MCUs RM0383 for STM32F411xC/E MCUs and RM0401 for STM32F410xx MCUs RM0090 for STM32F4X5xx/ STM32F4X7xx/STM32F429xx/STM32F439xx MCUs RM0390 for STM32F446xx MCUs and RM0386 for STM32F469xx/STM32F479xx MCUs)).

This section contains the following APIs:

- [HAL_USART_Init\(\)](#)
- [HAL_USART_DeInit\(\)](#)
- [HAL_USART_MspInit\(\)](#)
- [HAL_USART_MspDeInit\(\)](#)
- [HAL_USART_RegisterCallback\(\)](#)
- [HAL_USART_UnRegisterCallback\(\)](#)

71.2.4 IO operation functions

This subsection provides a set of functions allowing to manage the USART synchronous data transfers.

The USART supports master mode only: it cannot receive or send data related to an input clock (SCLK is always an output).

1. There are two modes of transfer:
 - Blocking mode: The communication is performed in polling mode. The HAL status of all data processing is returned by the same function after finishing transfer.
 - No-Blocking mode: The communication is performed using Interrupts or DMA. These APIs return the HAL status. The end of the data processing will be indicated through the dedicated USART IRQ when using Interrupt mode or the DMA IRQ when using DMA mode. The HAL_USART_TxCpltCallback(), HAL_USART_RxCpltCallback() and HAL_USART_TxRxCpltCallback() user callbacks will be executed respectively at the end of the transmit or Receive process. The HAL_USART_ErrorCallback() user callback will be executed when a communication error is detected.
2. Blocking mode APIs are :
 - HAL_USART_Transmit() in simplex mode
 - HAL_USART_Receive() in full duplex receive only
 - HAL_USART_TransmitReceive() in full duplex mode
3. Non Blocking mode APIs with Interrupt are :
 - HAL_USART_Transmit_IT() in simplex mode
 - HAL_USART_Receive_IT() in full duplex receive only
 - HAL_USART_TransmitReceive_IT() in full duplex mode
 - HAL_USART_IRQHandler()
4. Non Blocking mode functions with DMA are :
 - HAL_USART_Transmit_DMA() in simplex mode
 - HAL_USART_Receive_DMA() in full duplex receive only
 - HAL_USART_TransmitReceive_DMA() in full duplex mode
 - HAL_USART_DMABuffer() in full duplex mode
 - HAL_USART_DMAStop()
5. A set of Transfer Complete Callbacks are provided in non Blocking mode:
 - HAL_USART_TxHalfCpltCallback()
 - HAL_USART_TxCpltCallback()
 - HAL_USART_RxHalfCpltCallback()
 - HAL_USART_RxCpltCallback()
 - HAL_USART_ErrorCallback()
 - HAL_USART_TxRxCpltCallback()
6. Non-Blocking mode transfers could be aborted using Abort APIs :
 - HAL_USART_Abort()
 - HAL_USART_Abort_IT()
7. For Abort services based on interrupts (HAL_USART_Abort_IT), a Abort Complete Callbacks is provided:
 - HAL_USART_AbortCpltCallback()
8. In Non-Blocking mode transfers, possible errors are split into 2 categories. Errors are handled as follows :
 - Error is considered as Recoverable and non blocking : Transfer could go till end, but error severity is to be evaluated by user : this concerns Frame Error, Parity Error or Noise Error in Interrupt mode reception . Received character is then retrieved and stored in Rx buffer, Error code is set to allow user to identify error type, and HAL_USART_ErrorCallback() user callback is executed. Transfer is kept ongoing on USART side. If user wants to abort it, Abort services should be called by user.
 - Error is considered as Blocking : Transfer could not be completed properly and is aborted. This concerns Overrun Error In Interrupt mode reception and all errors in DMA mode. Error code is set to allow user to identify error type, and HAL_USART_ErrorCallback() user callback is executed.

This section contains the following APIs:

- `HAL_USART_Transmit()`
- `HAL_USART_Receive()`
- `HAL_USART_TransmitReceive()`
- `HAL_USART_Transmit_IT()`
- `HAL_USART_Receive_IT()`
- `HAL_USART_TransmitReceive_IT()`
- `HAL_USART_Transmit_DMA()`
- `HAL_USART_Receive_DMA()`
- `HAL_USART_TransmitReceive_DMA()`
- `HAL_USART_DMAPause()`
- `HAL_USART_DMAResume()`
- `HAL_USART_DMAStop()`
- `HAL_USART_Abort()`
- `HAL_USART_Abort_IT()`
- `HAL_USART_IRQHandler()`
- `HAL_USART_TxCpltCallback()`
- `HAL_USART_TxHalfCpltCallback()`
- `HAL_USART_RxCpltCallback()`
- `HAL_USART_RxHalfCpltCallback()`
- `HAL_USART_TxRxCpltCallback()`
- `HAL_USART_ErrorCallback()`
- `HAL_USART_AbortCpltCallback()`

71.2.5 Peripheral State and Errors functions

This subsection provides a set of functions allowing to return the State of USART communication process, return Peripheral Errors occurred during communication process

- `HAL_USART_GetState()` API can be helpful to check in run-time the state of the USART peripheral.
- `HAL_USART_GetError()` check in run-time errors that could be occurred during communication.

This section contains the following APIs:

- `HAL_USART_GetState()`
- `HAL_USART_GetError()`

71.2.6 Detailed description of functions

HAL_USART_Init

Function name

HAL_StatusTypeDef HAL_USART_Init (USART_HandleTypeDef * husart)

Function description

Initialize the USART mode according to the specified parameters in the USART_InitTypeDef and initialize the associated handle.

Parameters

- **husart**: Pointer to a USART_HandleTypeDef structure that contains the configuration information for the specified USART module.

Return values

- **HAL**: status

HAL_USART_DeInit

Function name

HAL_StatusTypeDef HAL_USART_DeInit (USART_HandleTypeDef * husart)

Function description

DeInitializes the USART peripheral.

Parameters

- **husart:** Pointer to a USART_HandleTypeDef structure that contains the configuration information for the specified USART module.

Return values

- **HAL:** status

HAL_USART_MspInit

Function name

void HAL_USART_MspInit (USART_HandleTypeDef * husart)

Function description

USART MSP Init.

Parameters

- **husart:** Pointer to a USART_HandleTypeDef structure that contains the configuration information for the specified USART module.

Return values

- **None:**

HAL_USART_MspDeInit

Function name

void HAL_USART_MspDeInit (USART_HandleTypeDef * husart)

Function description

USART MSP DeInit.

Parameters

- **husart:** Pointer to a USART_HandleTypeDef structure that contains the configuration information for the specified USART module.

Return values

- **None:**

HAL_USART_RegisterCallback

Function name

HAL_StatusTypeDef HAL_USART_RegisterCallback (USART_HandleTypeDef * husart, HAL_USART_CallbackIDTypeDef CallbackID, pUSART_CallbackTypeDef pCallback)

Function description

Register a User USART Callback To be used instead of the weak predefined callback.

Parameters

- **husart:** usart handle
- **CallbackID:** ID of the callback to be registered This parameter can be one of the following values:
 - HAL_USART_TX_HALFCOMPLETE_CB_ID Tx Half Complete Callback ID
 - HAL_USART_TX_COMPLETE_CB_ID Tx Complete Callback ID
 - HAL_USART_RX_HALFCOMPLETE_CB_ID Rx Half Complete Callback ID
 - HAL_USART_RX_COMPLETE_CB_ID Rx Complete Callback ID
 - HAL_USART_TX_RX_COMPLETE_CB_ID Rx Complete Callback ID
 - HAL_USART_ERROR_CB_ID Error Callback ID
 - HAL_USART_ABORT_COMPLETE_CB_ID Abort Complete Callback ID
 - HAL_USART_MSPINIT_CB_ID MspInit Callback ID
 - HAL_USART_MSPDEINIT_CB_ID MspDeInit Callback ID
- **pCallback:** pointer to the Callback function

Return values

- **HAL:** status +

Notes

- The HAL_USART_RegisterCallback() may be called before HAL_USART_Init() in HAL_USART_STATE_RESET to register callbacks for HAL_USART_MSPINIT_CB_ID and HAL_USART_MSPDEINIT_CB_ID

HAL_USART_UnRegisterCallback

Function name

HAL_StatusTypeDef HAL_USART_UnRegisterCallback (USART_HandleTypeDef * husart, HAL_USART_CallbackIDTypeDef CallbackID)

Function description

Unregister an USART Callback USART callback is redirected to the weak predefined callback.

Parameters

- **husart:** usart handle
- **CallbackID:** ID of the callback to be unregistered This parameter can be one of the following values:
 - HAL_USART_TX_HALFCOMPLETE_CB_ID Tx Half Complete Callback ID
 - HAL_USART_TX_COMPLETE_CB_ID Tx Complete Callback ID
 - HAL_USART_RX_HALFCOMPLETE_CB_ID Rx Half Complete Callback ID
 - HAL_USART_RX_COMPLETE_CB_ID Rx Complete Callback ID
 - HAL_USART_TX_RX_COMPLETE_CB_ID Rx Complete Callback ID
 - HAL_USART_ERROR_CB_ID Error Callback ID
 - HAL_USART_ABORT_COMPLETE_CB_ID Abort Complete Callback ID
 - HAL_USART_MSPINIT_CB_ID MspInit Callback ID
 - HAL_USART_MSPDEINIT_CB_ID MspDeInit Callback ID

Return values

- **HAL:** status

Notes

- The HAL_USART_UnRegisterCallback() may be called before HAL_USART_Init() in HAL_USART_STATE_RESET to un-register callbacks for HAL_USART_MSPINIT_CB_ID and HAL_USART_MSPDEINIT_CB_ID

HAL_USART_Transmit

Function name

HAL_StatusTypeDef HAL_USART_Transmit (USART_HandleTypeDef * husart, const uint8_t * pTxData, uint16_t Size, uint32_t Timeout)

Function description

Simplex Send an amount of data in blocking mode.

Parameters

- **husart**: Pointer to a USART_HandleTypeDef structure that contains the configuration information for the specified USART module.
- **pTxData**: Pointer to data buffer (u8 or u16 data elements).
- **Size**: Amount of data elements (u8 or u16) to be sent.
- **Timeout**: Timeout duration.

Return values

- **HAL**: status

Notes

- When USART parity is not enabled (PCE = 0), and Word Length is configured to 9 bits (M1-M0 = 01), the sent data is handled as a set of u16. In this case, Size must indicate the number of u16 provided through pTxData.

HAL_USART_Receive

Function name

HAL_StatusTypeDef HAL_USART_Receive (USART_HandleTypeDef * husart, uint8_t * pRxData, uint16_t Size, uint32_t Timeout)

Function description

Full-Duplex Receive an amount of data in blocking mode.

Parameters

- **husart**: Pointer to a USART_HandleTypeDef structure that contains the configuration information for the specified USART module.
- **pRxData**: Pointer to data buffer (u8 or u16 data elements).
- **Size**: Amount of data elements (u8 or u16) to be received.
- **Timeout**: Timeout duration.

Return values

- **HAL**: status

Notes

- To receive synchronous data, dummy data are simultaneously transmitted.
- When USART parity is not enabled (PCE = 0), and Word Length is configured to 9 bits (M1-M0 = 01), the received data is handled as a set of u16. In this case, Size must indicate the number of u16 available through pRxData.

HAL_USART_TransmitReceive

Function name

HAL_StatusTypeDef HAL_USART_TransmitReceive (USART_HandleTypeDef * husart, const uint8_t * pTxData, uint8_t * pRxData, uint16_t Size, uint32_t Timeout)

Function description

Full-Duplex Send and Receive an amount of data in full-duplex mode (blocking mode).

Parameters

- **husart**: Pointer to a USART_HandleTypeDef structure that contains the configuration information for the specified USART module.
- **pTxData**: Pointer to TX data buffer (u8 or u16 data elements).
- **pRxData**: Pointer to RX data buffer (u8 or u16 data elements).
- **Size**: Amount of data elements (u8 or u16) to be sent (same amount to be received).
- **Timeout**: Timeout duration

Return values

- **HAL**: status

Notes

- When USART parity is not enabled (PCE = 0), and Word Length is configured to 9 bits (M1-M0 = 01), the sent data and the received data are handled as sets of u16. In this case, Size must indicate the number of u16 available through pTxData and through pRxData.

HAL_USART_Transmit_IT

Function name

HAL_StatusTypeDef HAL_USART_Transmit_IT (USART_HandleTypeDef * husart, const uint8_t * pTxData, uint16_t Size)

Function description

Simplex Send an amount of data in non-blocking mode.

Parameters

- **husart**: Pointer to a USART_HandleTypeDef structure that contains the configuration information for the specified USART module.
- **pTxData**: Pointer to data buffer (u8 or u16 data elements).
- **Size**: Amount of data elements (u8 or u16) to be sent.

Return values

- **HAL**: status

Notes

- When USART parity is not enabled (PCE = 0), and Word Length is configured to 9 bits (M1-M0 = 01), the sent data is handled as a set of u16. In this case, Size must indicate the number of u16 provided through pTxData.
- The USART errors are not managed to avoid the overrun error.

HAL_USART_Receive_IT

Function name

HAL_StatusTypeDef HAL_USART_Receive_IT (USART_HandleTypeDef * husart, uint8_t * pRxData, uint16_t Size)

Function description

Simplex Receive an amount of data in non-blocking mode.

Parameters

- **husart**: Pointer to a USART_HandleTypeDef structure that contains the configuration information for the specified USART module.
- **pRxData**: Pointer to data buffer (u8 or u16 data elements).
- **Size**: Amount of data elements (u8 or u16) to be received.

Return values

- **HAL**: status

Notes

- To receive synchronous data, dummy data are simultaneously transmitted.
- When USART parity is not enabled (PCE = 0), and Word Length is configured to 9 bits (M1-M0 = 01), the received data is handled as a set of u16. In this case, Size must indicate the number of u16 available through pRxData.

HAL_USART_TransmitReceive_IT

Function name

HAL_StatusTypeDef HAL_USART_TransmitReceive_IT (USART_HandleTypeDef * husart, const uint8_t * pTxData, uint8_t * pRxData, uint16_t Size)

Function description

Full-Duplex Send and Receive an amount of data in full-duplex mode (non-blocking).

Parameters

- **husart**: Pointer to a USART_HandleTypeDef structure that contains the configuration information for the specified USART module.
- **pTxData**: Pointer to TX data buffer (u8 or u16 data elements).
- **pRxData**: Pointer to RX data buffer (u8 or u16 data elements).
- **Size**: Amount of data elements (u8 or u16) to be sent (same amount to be received).

Return values

- **HAL**: status

Notes

- When USART parity is not enabled (PCE = 0), and Word Length is configured to 9 bits (M1-M0 = 01), the sent data and the received data are handled as sets of u16. In this case, Size must indicate the number of u16 available through pTxData and through pRxData.

HAL_USART_Transmit_DMA

Function name

HAL_StatusTypeDef HAL_USART_Transmit_DMA (USART_HandleTypeDef * husart, const uint8_t * pTxData, uint16_t Size)

Function description

Simplex Send an amount of data in DMA mode.

Parameters

- **husart**: Pointer to a USART_HandleTypeDef structure that contains the configuration information for the specified USART module.
- **pTxData**: Pointer to data buffer (u8 or u16 data elements).
- **Size**: Amount of data elements (u8 or u16) to be sent.

Return values

- **HAL**: status

Notes

- When USART parity is not enabled (PCE = 0), and Word Length is configured to 9 bits (M1-M0 = 01), the sent data is handled as a set of u16. In this case, Size must indicate the number of u16 provided through pTxData.

HAL_USART_Receive_DMA

Function name

HAL_StatusTypeDef HAL_USART_Receive_DMA (USART_HandleTypeDef * husart, uint8_t * pRxData, uint16_t Size)

Function description

Full-Duplex Receive an amount of data in DMA mode.

Parameters

- **husart:** Pointer to a USART_HandleTypeDef structure that contains the configuration information for the specified USART module.
- **pRxData:** Pointer to data buffer (u8 or u16 data elements).
- **Size:** Amount of data elements (u8 or u16) to be received.

Return values

- **HAL:** status

Notes

- When USART parity is not enabled (PCE = 0), and Word Length is configured to 9 bits (M1-M0 = 01), the received data is handled as a set of u16. In this case, Size must indicate the number of u16 available through pRxData.
- The USART DMA transmit stream must be configured in order to generate the clock for the slave.
- When the USART parity is enabled (PCE = 1) the data received contain the parity bit.

HAL_USART_TransmitReceive_DMA

Function name

HAL_StatusTypeDef HAL_USART_TransmitReceive_DMA (USART_HandleTypeDef * husart, const uint8_t * pTxData, uint8_t * pRxData, uint16_t Size)

Function description

Full-Duplex Transmit Receive an amount of data in DMA mode.

Parameters

- **husart:** Pointer to a USART_HandleTypeDef structure that contains the configuration information for the specified USART module.
- **pTxData:** Pointer to TX data buffer (u8 or u16 data elements).
- **pRxData:** Pointer to RX data buffer (u8 or u16 data elements).
- **Size:** Amount of data elements (u8 or u16) to be received/sent.

Return values

- **HAL:** status

Notes

- When USART parity is not enabled (PCE = 0), and Word Length is configured to 9 bits (M1-M0 = 01), the sent data and the received data are handled as sets of u16. In this case, Size must indicate the number of u16 available through pTxData and through pRxData.
- When the USART parity is enabled (PCE = 1) the data received contain the parity bit.

HAL_USART_DMAPause

Function name

HAL_StatusTypeDef HAL_USART_DMAPause (USART_HandleTypeDef * husart)

Function description

Pauses the DMA Transfer.

Parameters

- **husart:** Pointer to a USART_HandleTypeDef structure that contains the configuration information for the specified USART module.

Return values

- **HAL:** status

HAL_USART_DMAResume

Function name

HAL_StatusTypeDef HAL_USART_DMAResume (USART_HandleTypeDef * husart)

Function description

Resumes the DMA Transfer.

Parameters

- **husart:** Pointer to a USART_HandleTypeDef structure that contains the configuration information for the specified USART module.

Return values

- **HAL:** status

HAL_USART_DMAStop

Function name

HAL_StatusTypeDef HAL_USART_DMAStop (USART_HandleTypeDef * husart)

Function description

Stops the DMA Transfer.

Parameters

- **husart:** Pointer to a USART_HandleTypeDef structure that contains the configuration information for the specified USART module.

Return values

- **HAL:** status

HAL_USART_Abort

Function name

HAL_StatusTypeDef HAL_USART_Abort (USART_HandleTypeDef * husart)

Function description

Abort ongoing transfer (blocking mode).

Parameters

- **husart:** USART handle.

Return values

- **HAL:** status

Notes

- This procedure could be used for aborting any ongoing transfer (either Tx or Rx, as described by TransferType parameter) started in Interrupt or DMA mode. This procedure performs following operations : Disable PPP Interrupts (depending of transfer direction)Disable the DMA transfer in the peripheral register (if enabled)Abort DMA transfer by calling HAL_DMA_Abort (in case of transfer in DMA mode)Set handle State to READY
- This procedure is executed in blocking mode : when exiting function, Abort is considered as completed.

HAL_USART_Abort_IT

Function name

HAL_StatusTypeDef HAL_USART_Abort_IT (USART_HandleTypeDef * husart)

Function description

Abort ongoing transfer (Interrupt mode).

Parameters

- **husart:** USART handle.

Return values

- **HAL:** status

Notes

- This procedure could be used for aborting any ongoing transfer (either Tx or Rx, as described by TransferType parameter) started in Interrupt or DMA mode. This procedure performs following operations : Disable PPP Interrupts (depending of transfer direction)Disable the DMA transfer in the peripheral register (if enabled)Abort DMA transfer by calling HAL_DMA_Abort_IT (in case of transfer in DMA mode)Set handle State to READYAt abort completion, call user abort complete callback
- This procedure is executed in Interrupt mode, meaning that abort procedure could be considered as completed only when user abort complete callback is executed (not when exiting function).

HAL_USART_IRQHandler

Function name

void HAL_USART_IRQHandler (USART_HandleTypeDef * husart)

Function description

This function handles USART interrupt request.

Parameters

- **husart:** Pointer to a USART_HandleTypeDef structure that contains the configuration information for the specified USART module.

Return values

- **None:**

HAL_USART_TxCpltCallback

Function name

void HAL_USART_TxCpltCallback (USART_HandleTypeDef * husart)

Function description

Tx Transfer completed callbacks.

Parameters

- **husart:** Pointer to a USART_HandleTypeDef structure that contains the configuration information for the specified USART module.

Return values

- **None:**

HAL_USART_TxHalfCpltCallback

Function name

void HAL_USART_TxHalfCpltCallback (USART_HandleTypeDef * husart)

Function description

Tx Half Transfer completed callbacks.

Parameters

- **husart:** Pointer to a USART_HandleTypeDef structure that contains the configuration information for the specified USART module.

Return values

- **None:**

HAL_USART_RxCpltCallback

Function name

void HAL_USART_RxCpltCallback (USART_HandleTypeDef * husart)

Function description

Rx Transfer completed callbacks.

Parameters

- **husart:** Pointer to a USART_HandleTypeDef structure that contains the configuration information for the specified USART module.

Return values

- **None:**

HAL_USART_RxHalfCpltCallback

Function name

void HAL_USART_RxHalfCpltCallback (USART_HandleTypeDef * husart)

Function description

Rx Half Transfer completed callbacks.

Parameters

- **husart:** Pointer to a USART_HandleTypeDef structure that contains the configuration information for the specified USART module.

Return values

- **None:**

HAL_USART_TxRxCpltCallback

Function name

void HAL_USART_TxRxCpltCallback (USART_HandleTypeDef * husart)

Function description

Tx/Rx Transfers completed callback for the non-blocking process.

Parameters

- **husart:** Pointer to a USART_HandleTypeDef structure that contains the configuration information for the specified USART module.

Return values

- **None:**

HAL_USART_ErrorCallback

Function name

```
void HAL_USART_ErrorCallback (USART_HandleTypeDef * husart)
```

Function description

USART error callbacks.

Parameters

- **husart:** Pointer to a USART_HandleTypeDef structure that contains the configuration information for the specified USART module.

Return values

- **None:**

HAL_USART_AbortCpltCallback

Function name

```
void HAL_USART_AbortCpltCallback (USART_HandleTypeDef * husart)
```

Function description

USART Abort Complete callback.

Parameters

- **husart:** USART handle.

Return values

- **None:**

HAL_USART_GetState

Function name

```
HAL_USART_StateTypeDef HAL_USART_GetState (const USART_HandleTypeDef * husart)
```

Function description

Returns the USART state.

Parameters

- **husart:** Pointer to a USART_HandleTypeDef structure that contains the configuration information for the specified USART module.

Return values

- **HAL:** state

HAL_USART_GetError

Function name

```
uint32_t HAL_USART_GetError (const USART_HandleTypeDef * husart)
```

Function description

Return the USART error code.

Parameters

- **husart:** Pointer to a USART_HandleTypeDef structure that contains the configuration information for the specified USART.

Return values

- **USART:** Error Code

71.3 USART Firmware driver defines

The following section lists the various define and macros of the module.

71.3.1 USART

USART

USART Clock

USART_CLOCK_DISABLE

USART_CLOCK_ENABLE

USART Clock Phase

USART_PHASE_1EDGE

USART_PHASE_2EDGE

USART Clock Polarity

USART_POLARITY_LOW

USART_POLARITY_HIGH

USART Error Code

HAL_USART_ERROR_NONE

No error

HAL_USART_ERROR_PE

Parity error

HAL_USART_ERROR_NE

Noise error

HAL_USART_ERROR_FE

Frame error

HAL_USART_ERROR_ORE

Overrun error

HAL_USART_ERROR_DMA

DMA transfer error

HAL_USART_ERROR_INVALID_CALLBACK

Invalid Callback error

USART Exported Macros

__HAL_USART_RESET_HANDLE_STATE

Description:

- Reset USART handle state.

Parameters:

- __HANDLE__**: specifies the USART Handle. USART Handle selects the USARTx peripheral (USART availability and x value depending on device).

Return value:

- None

__HAL_USART_GET_FLAG

Description:

- Check whether the specified USART flag is set or not.

Parameters:

- __HANDLE__**: specifies the USART Handle. USART Handle selects the USARTx peripheral (USART availability and x value depending on device).
- __FLAG__**: specifies the flag to check. This parameter can be one of the following values:
 - USART_FLAG_TXE: Transmit data register empty flag
 - USART_FLAG_TC: Transmission Complete flag
 - USART_FLAG_RXNE: Receive data register not empty flag
 - USART_FLAG_IDLE: Idle Line detection flag
 - USART_FLAG_ORE: Overrun Error flag
 - USART_FLAG_NE: Noise Error flag
 - USART_FLAG_FE: Framing Error flag
 - USART_FLAG_PE: Parity Error flag

Return value:

- The: new state of **__FLAG__** (TRUE or FALSE).

__HAL_USART_CLEAR_FLAG

Description:

- Clear the specified USART pending flags.

Parameters:

- __HANDLE__**: specifies the USART Handle. USART Handle selects the USARTx peripheral (USART availability and x value depending on device).
- __FLAG__**: specifies the flag to check. This parameter can be any combination of the following values:
 - USART_FLAG_TC: Transmission Complete flag.
 - USART_FLAG_RXNE: Receive data register not empty flag.

Return value:

- None

Notes:

- PE (Parity error), FE (Framing error), NE (Noise error), ORE (Overrun error) and IDLE (Idle line detected) flags are cleared by software sequence: a read operation to USART_SR register followed by a read operation to USART_DR register. RXNE flag can be also cleared by a read to the USART_DR register. TC flag can be also cleared by software sequence: a read operation to USART_SR register followed by a write operation to USART_DR register. TXE flag is cleared only by a write to the USART_DR register.

__HAL_USART_CLEAR_PFLAG

Description:

- Clear the USART PE pending flag.

Parameters:

- `__HANDLE__`: specifies the USART Handle. USART Handle selects the USARTx peripheral (USART availability and x value depending on device).

Return value:

- None

__HAL_USART_CLEAR_FFLAG

Description:

- Clear the USART FE pending flag.

Parameters:

- `__HANDLE__`: specifies the USART Handle. USART Handle selects the USARTx peripheral (USART availability and x value depending on device).

Return value:

- None

__HAL_USART_CLEAR_NFLAG

Description:

- Clear the USART NE pending flag.

Parameters:

- `__HANDLE__`: specifies the USART Handle. USART Handle selects the USARTx peripheral (USART availability and x value depending on device).

Return value:

- None

__HAL_USART_CLEAR_OREFLAG

Description:

- Clear the USART ORE pending flag.

Parameters:

- `__HANDLE__`: specifies the USART Handle. USART Handle selects the USARTx peripheral (USART availability and x value depending on device).

Return value:

- None

__HAL_USART_CLEAR_IDLEFLAG

Description:

- Clear the USART IDLE pending flag.

Parameters:

- `__HANDLE__`: specifies the USART Handle. USART Handle selects the USARTx peripheral (USART availability and x value depending on device).

Return value:

- None

__HAL_USART_ENABLE_IT

Description:

- Enables or disables the specified USART interrupts.

Parameters:

- `__HANDLE__`: specifies the USART Handle. USART Handle selects the USARTx peripheral (USART availability and x value depending on device).
- `__INTERRUPT__`: specifies the USART interrupt source to check. This parameter can be one of the following values:
 - USART_IT_TXE: Transmit Data Register empty interrupt
 - USART_IT_TC: Transmission complete interrupt
 - USART_IT_RXNE: Receive Data register not empty interrupt
 - USART_IT_IDLE: Idle line detection interrupt
 - USART_IT_PE: Parity Error interrupt
 - USART_IT_ERR: Error interrupt(Frame error, noise error, overrun error)

Return value:

- None

__HAL_USART_DISABLE_IT

__HAL_USART_GET_IT_SOURCE

Description:

- Checks whether the specified USART interrupt has occurred or not.

Parameters:

- `__HANDLE__`: specifies the USART Handle. USART Handle selects the USARTx peripheral (USART availability and x value depending on device).
- `__IT__`: specifies the USART interrupt source to check. This parameter can be one of the following values:
 - USART_IT_TXE: Transmit Data Register empty interrupt
 - USART_IT_TC: Transmission complete interrupt
 - USART_IT_RXNE: Receive Data register not empty interrupt
 - USART_IT_IDLE: Idle line detection interrupt
 - USART_IT_ERR: Error interrupt
 - USART_IT_PE: Parity Error interrupt

Return value:

- The: new state of `__IT__` (TRUE or FALSE).

__HAL_USART_ONE_BIT_SAMPLE_ENABLE

Description:

- Macro to enable the USART's one bit sample method.

Parameters:

- `__HANDLE__`: specifies the USART Handle.

Return value:

- None

__HAL_USART_ONE_BIT_SAMPLE_DISABLE

Description:

- Macro to disable the USART's one bit sample method.

Parameters:

- `__HANDLE__`: specifies the USART Handle.

Return value:

- None

__HAL_USART_ENABLE**Description:**

- Enable USART.

Parameters:

- **__HANDLE__**: specifies the USART Handle. USART Handle selects the USARTx peripheral (USART availability and x value depending on device).

Return value:

- None

__HAL_USART_DISABLE**Description:**

- Disable USART.

Parameters:

- **__HANDLE__**: specifies the USART Handle. USART Handle selects the USARTx peripheral (USART availability and x value depending on device).

Return value:

- None

USART Flags**USART_FLAG_TXE****USART_FLAG_TC****USART_FLAG_RXNE****USART_FLAG_IDLE****USART_FLAG_ORE****USART_FLAG_NE****USART_FLAG_FE****USART_FLAG_PE****USART Interrupts Definition****USART_IT_PE****USART_IT_TXE****USART_IT_TC****USART_IT_RXNE****USART_IT_IDLE****USART_IT_ERR****USART Last Bit****USART_LASTBIT_DISABLE****USART_LASTBIT_ENABLE**

USART Mode

USART_MODE_RX

USART_MODE_TX

USART_MODE_TX_RX

USART NACK State

USART_NACK_ENABLE

USART_NACK_DISABLE

USART Parity

USART_PARITY_NONE

USART_PARITY_EVEN

USART_PARITY_ODD

USART Number of Stop Bits

USART_STOPBITS_1

USART_STOPBITS_0_5

USART_STOPBITS_2

USART_STOPBITS_1_5

USART Word Length

USART_WORDLENGTH_8B

USART_WORDLENGTH_9B

72 HAL WWDG Generic Driver

72.1 WWDG Firmware driver registers structures

72.1.1 WWDG_InitTypeDef

WWDG_InitTypeDef is defined in the stm32f4xx_hal_wwdg.h

Data Fields

- **uint32_t Prescaler**
- **uint32_t Window**
- **uint32_t Counter**
- **uint32_t EWIMode**

Field Documentation

- **uint32_t WWDG_InitTypeDef::Prescaler**
Specifies the prescaler value of the WWDG. This parameter can be a value of **WWDG_Prescaler**
- **uint32_t WWDG_InitTypeDef::Window**
Specifies the WWDG window value to be compared to the downcounter. This parameter must be a number Min_Data = 0x40 and Max_Data = 0x7F
- **uint32_t WWDG_InitTypeDef::Counter**
Specifies the WWDG free-running downcounter value. This parameter must be a number between Min_Data = 0x40 and Max_Data = 0x7F
- **uint32_t WWDG_InitTypeDef::EWIMode**
Specifies if WWDG Early Wakeup Interrupt is enable or not. This parameter can be a value of **WWDG_EWI_Mode**

72.1.2 WWDG_HandleTypeDef

WWDG_HandleTypeDef is defined in the stm32f4xx_hal_wwdg.h

Data Fields

- **WWDG_TypeDef * Instance**
- **WWDG_InitTypeDef Init**

Field Documentation

- **WWDG_TypeDef* WWDG_HandleTypeDef::Instance**
Register base address
- **WWDG_InitTypeDef WWDG_HandleTypeDef::Init**
WWDG required parameters

72.2 WWDG Firmware driver API description

The following section lists the various functions of the WWDG library.

72.2.1 WWDG Specific features

Once enabled the WWDG generates a system reset on expiry of a programmed time period, unless the program refreshes the counter (T[6;0] downcounter) before reaching 0x3F value (i.e. a reset is generated when the counter value rolls down from 0x40 to 0x3F).

- An MCU reset is also generated if the counter value is refreshed before the counter has reached the refresh window value. This implies that the counter must be refreshed in a limited window.
- Once enabled the WWDG cannot be disabled except by a system reset.
- If required by application, an Early Wakeup Interrupt can be triggered in order to be warned before WWDG expiration. The Early Wakeup Interrupt (EWI) can be used if specific safety operations or data logging must be performed before the actual reset is generated. When the downcounter reaches 0x40, interrupt occurs. This mechanism requires WWDG interrupt line to be enabled in NVIC. Once enabled, EWI interrupt cannot be disabled except by a system reset.
- WWDGRST flag in RCC CSR register can be used to inform when a WWDG reset occurs.

- The WWDG counter input clock is derived from the APB clock divided by a programmable prescaler.
- $\text{WWDG clock (Hz)} = \text{PCLK1} / (4096 * \text{Prescaler})$
- $\text{WWDG timeout (mS)} = 1000 * (\text{T}[5:0] + 1) / \text{WWDG clock (Hz)}$ where T[5:0] are the lowest 6 bits of Counter.
- WWDG Counter refresh is allowed between the following limits :
 - $\text{min time (mS)} = 1000 * (\text{Counter} - \text{Window}) / \text{WWDG clock}$
 - $\text{max time (mS)} = 1000 * (\text{Counter} - 0x40) / \text{WWDG clock}$
- Typical values:
 - Counter min (T[5:0] = 0x00) at 42MHz (PCLK1) with zero prescaler: max timeout before reset: approximately 97.52us
 - Counter max (T[5:0] = 0x3F) at 42MHz (PCLK1) with prescaler dividing by 8: max timeout before reset: approximately 49.93ms

72.2.2

How to use this driver

Common driver usage

- Enable WWDG APB1 clock using `__HAL_RCC_WWDG_CLK_ENABLE()`.
- Configure the WWDG prescaler, refresh window value, counter value and early interrupt status using `HAL_WWDG_Init()` function. This will automatically enable WWDG and start its downcounter. Time reference can be taken from function exit. Care must be taken to provide a counter value greater than 0x40 to prevent generation of immediate reset.
- If the Early Wakeup Interrupt (EWI) feature is enabled, an interrupt is generated when the counter reaches 0x40. When `HAL_WWDG_IRQHandler` is triggered by the interrupt service routine, flag will be automatically cleared and `HAL_WWDG_WakeupCallback` user callback will be executed. User can add his own code by customization of callback `HAL_WWDG_WakeupCallback`.
- Then the application program must refresh the WWDG counter at regular intervals during normal operation to prevent an MCU reset, using `HAL_WWDG_Refresh()` function. This operation must occur only when the counter is lower than the refresh window value already programmed.

Callback registration

The compilation define `USE_HAL_WWDG_REGISTER_CALLBACKS` when set to 1 allows the user to configure dynamically the driver callbacks. Use Functions `HAL_WWDG_RegisterCallback()` to register a user callback.

- Function `HAL_WWDG_RegisterCallback()` allows to register following callbacks:
 - `EwiCallback` : callback for Early WakeUp Interrupt.
 - `MspInitCallback` : WWDG MspInit. This function takes as parameters the HAL peripheral handle, the Callback ID and a pointer to the user callback function.
- Use function `HAL_WWDG_UnRegisterCallback()` to reset a callback to the default weak (surcharged) function. `HAL_WWDG_UnRegisterCallback()` takes as parameters the HAL peripheral handle and the Callback ID. This function allows to reset following callbacks:
 - `EwiCallback` : callback for Early WakeUp Interrupt.
 - `MspInitCallback` : WWDG MspInit.

When calling `HAL_WWDG_Init` function, callbacks are reset to the corresponding legacy weak (surcharged) functions: `HAL_WWDG_EarlyWakeupCallback()` and `HAL_WWDG_MspInit()` only if they have not been registered before.

When compilation define `USE_HAL_WWDG_REGISTER_CALLBACKS` is set to 0 or not defined, the callback registering feature is not available and weak (surcharged) callbacks are used.

WWDG HAL driver macros list

Below the list of available macros in WWDG HAL driver.

- `__HAL_WWDG_ENABLE`: Enable the WWDG peripheral
- `__HAL_WWDG_GET_FLAG`: Get the selected WWDG's flag status
- `__HAL_WWDG_CLEAR_FLAG`: Clear the WWDG's pending flags
- `__HAL_WWDG_ENABLE_IT`: Enable the WWDG early wakeup interrupt

72.2.3 Initialization and Configuration functions

This section provides functions allowing to:

- Initialize and start the WWDG according to the specified parameters in the WWDG_InitTypeDef of associated handle.
- Initialize the WWDG MSP.

This section contains the following APIs:

- [HAL_WWDG_Init\(\)](#)
- [HAL_WWDG_MspInit\(\)](#)
- [HAL_WWDG_RegisterCallback\(\)](#)
- [HAL_WWDG_UnRegisterCallback\(\)](#)

72.2.4 IO operation functions

This section provides functions allowing to:

- Refresh the WWDG.
- Handle WWDG interrupt request and associated function callback.

This section contains the following APIs:

- [HAL_WWDG_Refresh\(\)](#)
- [HAL_WWDG_IRQHandler\(\)](#)
- [HAL_WWDG_EarlyWakeupCallback\(\)](#)

72.2.5 Detailed description of functions

HAL_WWDG_Init

Function name

HAL_StatusTypeDef HAL_WWDG_Init (WWDG_HandleTypeDef * hwwdg)

Function description

Initialize the WWDG according to the specified.

Parameters

- **hwwdg**: pointer to a WWDG_HandleTypeDef structure that contains the configuration information for the specified WWDG module.

Return values

- **HAL**: status

HAL_WWDG_MspInit

Function name

void HAL_WWDG_MspInit (WWDG_HandleTypeDef * hwwdg)

Function description

Initialize the WWDG MSP.

Parameters

- **hwwdg**: pointer to a WWDG_HandleTypeDef structure that contains the configuration information for the specified WWDG module.

Return values

- **None**:

Notes

- When rewriting this function in user file, mechanism may be added to avoid multiple initialize when HAL_WWDG_Init function is called again to change parameters.

HAL_WWDG_RegisterCallback

Function name

```
HAL_StatusTypeDef HAL_WWDG_RegisterCallback (WWDG_HandleTypeDef * hwwdg,
HAL_WWDG_CallbackIDTypeDef CallbackID, pWWDG_CallbackTypeDef pCallback)
```

Function description

Register a User WWDG Callback To be used instead of the weak (surcharged) predefined callback.

Parameters

- **hwwdg**: WWDG handle
- **CallbackID**: ID of the callback to be registered This parameter can be one of the following values:
 - HAL_WWDG_EWI_CB_ID Early WakeUp Interrupt Callback ID
 - HAL_WWDG_MSPINIT_CB_ID MspInit callback ID
- **pCallback**: pointer to the Callback function

Return values

- **status**:

HAL_WWDG_UnRegisterCallback

Function name

```
HAL_StatusTypeDef HAL_WWDG_UnRegisterCallback (WWDG_HandleTypeDef * hwwdg,
HAL_WWDG_CallbackIDTypeDef CallbackID)
```

Function description

Unregister a WWDG Callback WWDG Callback is redirected to the weak (surcharged) predefined callback.

Parameters

- **hwwdg**: WWDG handle
- **CallbackID**: ID of the callback to be registered This parameter can be one of the following values:
 - HAL_WWDG_EWI_CB_ID Early WakeUp Interrupt Callback ID
 - HAL_WWDG_MSPINIT_CB_ID MspInit callback ID

Return values

- **status**:

HAL_WWDG_Refresh

Function name

```
HAL_StatusTypeDef HAL_WWDG_Refresh (WWDG_HandleTypeDef * hwwdg)
```

Function description

Refresh the WWDG.

Parameters

- **hwwdg**: pointer to a WWDG_HandleTypeDef structure that contains the configuration information for the specified WWDG module.

Return values

- **HAL**: status

HAL_WWDG_IRQHandler

Function name

```
void HAL_WWDG_IRQHandler (WWDG_HandleTypeDef * hwwdg)
```

Function description

Handle WWDG interrupt request.

Parameters

- **hwwdg**: pointer to a WWDG_HandleTypeDef structure that contains the configuration information for the specified WWDG module.

Return values

- **None**:

Notes

- The Early Wakeup Interrupt (EWI) can be used if specific safety operations or data logging must be performed before the actual reset is generated. The EWI interrupt is enabled by calling HAL_WWDG_Init function with EWIMode set to WWDG_EWI_ENABLE. When the downcounter reaches the value 0x40, and EWI interrupt is generated and the corresponding Interrupt Service Routine (ISR) can be used to trigger specific actions (such as communications or data logging), before resetting the device.

HAL_WWDG_EarlyWakeupCallback

Function name

```
void HAL_WWDG_EarlyWakeupCallback (WWDG_HandleTypeDef * hwwdg)
```

Function description

WWDG Early Wakeup callback.

Parameters

- **hwwdg**: pointer to a WWDG_HandleTypeDef structure that contains the configuration information for the specified WWDG module.

Return values

- **None**:

72.3 WWDG Firmware driver defines

The following section lists the various define and macros of the module.

72.3.1

WWDG

WWDG

WWDG Early Wakeup Interrupt Mode

WWDG_EWI_DISABLE

EWI Disable

WWDG_EWI_ENABLE

EWI Enable

WWDG Exported Macros

__HAL_WWDG_ENABLE

Description:

- Enable the WWDG peripheral.

Parameters:

- **__HANDLE__**: WWDG handle

Return value:

- None

__HAL_WWDG_ENABLE_IT

Description:

- Enable the WWDG early wakeup interrupt.

Parameters:

- __HANDLE__: WWDG handle
- __INTERRUPT__: specifies the interrupt to enable. This parameter can be one of the following values:
 - WWDG_IT_EWI: Early wakeup interrupt

Return value:

- None

Notes:

- Once enabled this interrupt cannot be disabled except by a system reset.

__HAL_WWDG_GET_IT

Description:

- Check whether the selected WWDG interrupt has occurred or not.

Parameters:

- __HANDLE__: WWDG handle
- __INTERRUPT__: specifies the it to check. This parameter can be one of the following values:
 - WWDG_FLAG_EWIF: Early wakeup interrupt IT

Return value:

- The: new state of WWDG_FLAG (SET or RESET).

__HAL_WWDG_CLEAR_IT

Description:

- Clear the WWDG interrupt pending bits.

Parameters:

- __HANDLE__: WWDG handle
- __INTERRUPT__: specifies the interrupt pending bit to clear. This parameter can be one of the following values:
 - WWDG_FLAG_EWIF: Early wakeup interrupt flag

__HAL_WWDG_GET_FLAG

Description:

- Check whether the specified WWDG flag is set or not.

Parameters:

- __HANDLE__: WWDG handle
- __FLAG__: specifies the flag to check. This parameter can be one of the following values:
 - WWDG_FLAG_EWIF: Early wakeup interrupt flag

Return value:

- The: new state of WWDG_FLAG (SET or RESET).

__HAL_WWDG_CLEAR_FLAG

Description:

- Clear the WWDG's pending flags.

Parameters:

- __HANDLE__: WWDG handle
- __FLAG__: specifies the flag to clear. This parameter can be one of the following values:
 - WWDG_FLAG_EWIF: Early wakeup interrupt flag

Return value:

- None

__HAL_WWDG_GET_IT_SOURCE

Description:

- Check whether the specified WWDG interrupt source is enabled or not.

Parameters:

- __HANDLE__: WWDG Handle.
- __INTERRUPT__: specifies the WWDG interrupt source to check. This parameter can be one of the following values:
 - WWDG_IT_EWI: Early Wakeup Interrupt

Return value:

- state: of __INTERRUPT__ (TRUE or FALSE).

WWDG Flag definition

WWDG_FLAG_EWIF

Early wakeup interrupt flag

WWDG Interrupt definition

WWDG_IT_EWI

Early wakeup interrupt

WWDG Prescaler

WWDG_PRESCALER_1

WWDG counter clock = (PCLK1/4096)/1

WWDG_PRESCALER_2

WWDG counter clock = (PCLK1/4096)/2

WWDG_PRESCALER_4

WWDG counter clock = (PCLK1/4096)/4

WWDG_PRESCALER_8

WWDG counter clock = (PCLK1/4096)/8